新一代人工智能系列教材

人工智能

导论：模型与算法

吴飞 编著

高等教育出版社

新一代人工智能系列教材

吴飞 编著

1910595

人工智能导论：模型与算法

台州学院图书馆 藏书

浙江工业大学 藏書 图書馆

高等教育出版社·北京

人工智能导论：模型与算法

Rengong Zhineng Daolun:
Moxing yu Suanfa

图书在版编目（CIP）数据

人工智能导论：模型与算法 / 吴飞编著. -- 北京：
高等教育出版社，2020.5
ISBN 978-7-04-053466-5

Ⅰ. ①人… Ⅱ. ①吴… Ⅲ. ①人工智能－教材 Ⅳ.
①TP18

中国版本图书馆CIP数据核字(2020)第017215号

郑重声明
高等教育出版社依法对本书
享有专有出版权。任何未经许可的
复制、销售行为均违反
《中华人民共和国著作权法》，
其行为人将承担相应的民事责任
和行政责任；构成犯罪的，将被
依法追究刑事责任。
为了维护市场秩序，保护读者的
合法权益，避免读者误用盗版书
造成不良后果，我社将配合
行政执法部门和司法机关对
违法犯罪的单位和个人进行
严厉打击。社会各界人士如发现
上述侵权行为，希望及时举报，
本社将奖励举报有功人员。
反盗版举报电话
（010）58581999　58582371
58582488
反盗版举报传真
（010）82086060
反盗版举报邮箱
dd@hep.com.cn
通信地址
北京市西城区德外大街4号
高等教育出版社法律事务
与版权管理部
邮政编码　100120

策划编辑　韩　飞
责任编辑　韩　飞
书籍设计　张申申
插图绘制　于　博
责任校对　窦丽娜
责任印制　赵义民

出版发行　高等教育出版社
社址　北京市西城区德外大街4号
邮政编码　100120
购书热线　010-58581118
咨询电话　400-810-0598
网址
http://www.hep.edu.cn
http://www.hep.com.cn
网上订购
http://www.hepmall.com.cn
http://www.hepmall.com
http://www.hepmall.cn
印刷　北京盛通印刷股份有限公司
开本　787mm×1092mm　1/16
印张　22.5
字数　350千字
版次　2020年5月第1版
印次　2020年5月第1次印刷
定价　37.00元

本书如有缺页、倒页、脱页等
质量问题，请到所购图书销
售部门联系调换

版权所有　侵权必究
物料号　53466-00

内容提要

本书是一本体系完整、突出算法和教学资源丰富的人工智能教材，可帮助读者掌握人工智能知识的脉络体系，从算法和模型方面来了解人工智能具能、使能和赋能的原理。

全书共9章，第1章绪论，第2章逻辑与推理，第3章搜索求解，第4章监督学习，第5章无监督学习，第6章深度学习，第7章强化学习，第8章人工智能博弈，第9章人工智能未来发展和趋势。书中附有习题和编程题目。

本书可作为人工智能专业和计算机类相关专业的本科生或研究生学习人工智能的教材。由于书中各章内容相对独立，教师可根据课程计划和专业需要选择讲授内容。

新一代人工智能系列教材编委会

主任　潘云鹤（中国工程院）

副主任　郑南宁（西安交通大学）　高文（北京大学）

吴澄（清华大学）　陈纯（浙江大学）

林金安（高等教育出版社）

成员　王飞跃（中科院自动化所）　李波（北京航空航天大学）

周志华（南京大学）　吴枫（中国科学技术大学）

黄铁军（北京大学）　周杰（清华大学）

刘挺（哈尔滨工业大学）　薛建儒（西安交通大学）

杨小康（上海交通大学）　高新波（西安电子科技大学）

刘成林（中科院大学）　薛向阳（复旦大学）

古天龙（桂林电子科技大学）　黄河燕（北京理工大学）

于剑（北京交通大学）　何钦铭（浙江大学）

吴飞（浙江大学）

序

人工智能是引领这一轮科技革命、产业变革和社会发展的战略性技术，具有溢出带动性很强的头雁效应。当前，新一代人工智能正在全球范围内蓬勃发展，促进人类社会生活、生产和消费模式巨大变革，为经济社会发展提供新动能，推动经济社会高质量发展，加速新一轮科技革命和产业变革。

2017年7月，国务院发布了《新一代人工智能发展规划》，指出了人工智能正走向新一代。新一代人工智能（AI 2.0）的概念除了继续用电脑模拟人的智能行为外，还纳入了更综合的信息系统，如互联网、大数据、云计算等去探索由人、物、信息交织的更大更复杂的系统行为，如制造系统、城市系统、生态系统等的智能化运行和发展。这就为人工智能打开了一扇新的大门和一个新的发展空间。人工智能将从各个角度与层次，宏观、中观和微观地，去发挥“头雁效应”，去渗透我们的学习、工作与生活，去改变我们的发展方式。

要发挥人工智能赋能产业、赋能社会，真正成为推动国家和社会高质量发展的强大引擎，需要大批掌握这一技术的优秀人才。因此，中国人工智能的发展十分需要重视人工智能技术及产业的人才培养。

高校是科技第一生产力、人才第一资源、创新第一动力的结合点。因此，高校有责任把人工智能人才的培养置于核心的基础地位，把人工智能协同创新摆在重要位置。国务院《新一代人工智能发展规划》和教育部《人工智能科技创新行动计划》发布后，为切实应对经济社会对人工智能人才的需求，我国一流高校陆续成立协同创新中心、人工智能学院、人工智能研究院等机构，为人工智能高层次人才、专业人才、交叉人才及产业应用人才培养搭建平台。我们正处于一个百年未遇、大有可为的历史机遇期，要紧紧抓住新一代人工智能发展的机遇，勇立潮头、砥砺前行，通过凝练教学成果及把握科学研究前沿方向的高质量教材来“传道、授业、解惑”，提高教学质量，投身人工智能人才培养主战场，为我国构筑人工智能发展先发优势和贯彻教育强国、科技强国、创新驱动战略贡献力量。

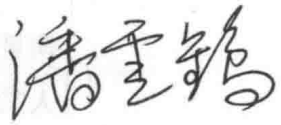
为促进人工智能人才培养，推动人工智能重要方向教材和在线开放课程建设，国家新一代人工智能战略咨询委员会和高等教育出版社于2018年3月成立了“新一代人工智能系列教材”编委会，聘请我担任编委会主任，吴澄院士、郑南宁院士、高文院士、陈纯院士和高等教育出

版社林金安副总编辑担任编委会副主任。

根据新一代人工智能发展特点和教学要求，编委会陆续组织编写和出版有关人工智能基础理论、算法模型、技术系统、硬件芯片和伦理安全以及“智能+”学科交叉等方面内容的系列教材，发布在线开放共享课程，以形成各具优势、衔接前沿、涵盖完整、交叉融合具有中国特色的人工智能一流教材体系。

“AI赋能、教育先行、创新引领、产学协同”，人工智能于1956年从达特茅斯学院出发，踏上了人类发展历史舞台，今天正发挥“头雁效应”，推动人类变革大潮，“其作始也简，其将毕也必巨”。我希望“新一代人工智能系列教材”的出版能够为人工智能各类型人才培养做出应有贡献。

衷心感谢编委会委员、教材作者、高等教育出版社编辑等为“新一代人工智能系列教材”出版所付出的时间和精力。



前言

人工智能（artificial intelligence，AI）是以机器为载体所展示出来的人类智能，因此人工智能也被称为机器智能（machine intelligence）。

浙江大学一贯重视人工智能教学工作，在1978年设立计算机系之初，何志均先生即开始招收人工智能方向研究生，并且编写教材亲自讲授“人工智能”课程。从那时开始，“人工智能”课程就一直是浙江大学计算机学院开设的核心课程之一。

笔者长期从事人工智能、多媒体分析与检索、跨媒体计算等方面的研究，2016年开始在浙江大学给计算机学院本科生和研究生讲授人工智能有关课程。在授课过程中，迫切感觉到需要向学生介绍以符号主义为核心的逻辑推理、以问题求解为核心的探寻搜索、以数据驱动为核心的机器学习、以行为主义为核心的强化学习、以博弈对抗为核心的群体智能等方法，让学生了解人工智能发展至今的全貌。

对人工智能的深入了解，要从算法层面对人工智能技术“知其意，悟其理，守其则，践其行”。为此，本书以模型和算法为核心按照如下内容进行组织：第1章介绍人工智能起源和发展，第2章介绍命题逻辑、谓词逻辑、知识图谱和因果推理的内容，第3章介绍启发式搜索、对抗搜索（含最小最大搜索、Alpha-Beta剪枝搜索）和蒙特卡洛树搜索的内容，第4章介绍机器学习基本概念、线性回归、决策树、线性判别分析、Ada Boosting、支持向量机和产生式学习等内容，第5章介绍*K*-means、主成分分析、特征人脸分析、潜在语义分析和EM算法等内容，第6章介绍前向神经网络、卷积神经网络、循环神经网络及其在自然语言和计算机视觉等方面的应用等内容，第7章介绍马尔科夫决策过程、策略优化与策略评估、Q学习和深度强化学习等内容，第8章介绍博弈论中纳什均衡、博弈策略求解和博弈规则设计以及非完全信息博弈等内容，第9章对人工智能未来发展和趋势进行了简略总结。

本书MOOC

为了加强对书中所涉及知识点的理解，书中每章都包含了选择题、思考题、计算题和证明题等习题。同时，附录部分设置了黑白棋搜索、逻辑推理、图像恢复、人脸识别、深度学习、强化学习、对抗学习7个编程实践题目，以锻炼大家的实践能力。

本书是爱课程网中国大学MOOC平台上开设的在线开放课程“人工智能：模型与算法”的主讲教材，同时，本书所对应视频在“学习强国”上也可访问。

本书可用于“人工智能导论”及同类课程的教学。由于书中各章内容相对独立，教师可根据课程计划和专业需要选择讲授内容。本书对应课程的教学时数一般为32 ~ 56学时。

感谢上海交通大学施鹏飞教授、南京航空航天大学陈松灿教授和复旦大学薛向阳教授对本书进行的仔细审阅，感谢韩亚洪教授、况琨博士和博士生张圣宇、周乐夔、廖冰兵、林宇箫、段新宇、蒋胤傑、田琪以及方宏波等参与编撰工作。衷心感谢“新一代人工智能系列教材”编委会潘云鹤院士、吴澄院士、郑南宁院士、高文院士、陈纯院士等专家学者和高等教育出版社在本书出版过程中予以的热情指导和帮助。

“岁月不居，时节如流”，难忘在前行路上浙江大学人工智能研究所历任所长何志均先生、潘云鹤院士和庄越挺教授对笔者的培养和帮助，难忘学术前辈、研究同行和家人对笔者的热情支持和无私厚爱。

笔者曾参与过科技部和教育部组织的有关人工智能规划的编撰工作，深感“对不同行业人才大力进行人工智能能力培养与人工智能专业人才支撑不同行业发展”同等重要，希望本书能够为人工智能人才培养贡献微薄之力。

由于水平有限，书中内容仍然会存在不足，欢迎教师和读者提出宝贵意见。E-mail: wufei@zju.edu.cn

吴飞

2019年10月于求是园

目录

第 1 章 绪论 ……001
1.1 人工智能的起源 ……001
1.2 可计算载体：形式化与机械化 ……002
1.3 智能计算方法 ……005
1.3.1 符号主义为核心的逻辑推理 ……005
1.3.2 问题求解为核心的探寻搜索 ……007
1.3.3 数据驱动为核心的机器学习 ……008
1.3.4 行为主义为核心的强化学习 ……010
1.3.5 博弈对抗为核心的决策智能 ……011
1.4 本书内容介绍 ……012
1.5 小结 ……013
习题 1 ……015
参考文献 ……016

第 2 章 逻辑与推理 ……019
2.1 命题逻辑 ……019
2.2 谓词逻辑 ……025
2.3 知识图谱推理 ……032
2.3.1 FOIL 归纳推理 ……033
2.3.2 路径排序推理 ……038
2.3.3 其他知识推理算法 ……040
2.4 因果推理 ……041
2.4.1 辛普森悖论 ……041
2.4.2 因果图的基本概念 ……046
2.4.3 干预的因果效应 ……051
2.4.4 反事实模型 ……054
2.4.5 因果分析的层次化 ……057
2.5 小结 ……058
习题 2 ……058
参考文献 ……060

第 3 章 搜索求解 ……063
3.1 搜索算法基础 ……063
3.1.1 搜索基本问题和求解 ……063
3.1.2 搜索算法的评价指标 ……066
3.1.3 搜索算法框架 ……066
3.1.4 树搜索和图搜索 ……069
3.2 启发式搜索 ……071
3.2.1 启发函数与评价函数 ……071
3.2.2 贪婪最佳优先搜索 ……072
3.2.3 A* 搜索 ……073
3.2.4 A 算法性能分析 ……075
3.3 对抗搜索 ……081
3.3.1 最小最大搜索 ……083
3.3.2 Alpha-Beta 剪枝算法 ……086
*3.3.3 Alpha-Beta 剪枝算法性能分析 ……091
3.4 蒙特卡洛树搜索 ……094
3.4.1 探索与利用机制的平衡 ……094
3.4.2 上限置信区间算法 ……097
3.4.3 蒙特卡洛树搜索 ……099
3.5 小结 ……105
习题 3 ……106
参考文献 ……109

第 4 章 机器学习：监督学习 ……111
4.1 机器学习基本概念 ……111
4.1.1 机器学习的种类 ……112
4.1.2 监督学习的基本概念 ……113
4.2 回归分析 ……115
4.2.1 一元线性回归 ……116
4.2.2 多元线性回归 ……118
4.2.3 逻辑斯蒂回归/对数几率回归 ……120
4.3 决策树 ……125
4.3.1 决策树分类案例 ……125

4.3.2 构建决策树……127
4.4 线性判别分析……129
4.5 Ada Boosting……134
4.5.1 可计算学习理论……134
4.5.2 Ada Boosting算法……136
4.5.3 从霍夫丁不等式解释Ada Boosting算法……139
4.5.4 Ada Boosting分类例子……141
4.6 支持向量机……144
4.6.1 VC维与结构风险最小化……144
4.6.2 线性可分支持向量机……146
4.6.3 松弛变量，软间隔与hinge损失函数……147
4.6.4 核函数解决线性不可分的情况……150
4.7 生成学习模型……151
4.8 小结……153
习题4……154
参考文献……157

■第5章 统计机器学习：无监督学习……159
5.1 K均值聚类……159
5.2 主成分分析……163
5.2.1 方差、协方差和相关系数……163
5.2.2 主成分分析……165
5.2.3 其他常用降维方法……167
5.3 特征人脸方法……170
5.3.1 奇异值分解……171
5.3.2 特征人脸方法……172
5.4 潜在语义分析……174
5.4.1 潜在语义分析思想……174
5.4.2 潜在语义分析例子……179
5.5 期望最大化算法……181
5.5.1 二硬币投掷例子……182
5.5.2 三硬币投掷例子……185
5.5.3 EM算法一般形式……188
5.6 小结……191
习题5……192
参考文献……195

■第6章 深度学习……197
6.1 深度学习的历史发展……197
6.2 前馈神经网络……199
6.2.1 若干概念……200
6.2.2 感知机模型……206
6.2.3 参数优化与学习……208
6.3 卷积神经网络……214
6.3.1 卷积计算……215
6.3.2 池化……219
6.3.3 神经网络正则化……221
6.4 循环神经网络……223
6.4.1 循环神经网络模型……223
6.4.2 长短时记忆网络……227
6.4.3 门控循环单元……230
6.5 深度生成学习……231
6.5.1 生成对抗网络……232
6.5.2 生成对抗网络算法……233
6.5.3 条件生成对抗网络……234
6.5.4 用生成对抗网络抵御对抗样本攻击……235
6.6 深度学习在自然语言和计算机视觉上的应用……238
6.6.1 词向量模型……238
6.6.2 图像分类与目标定位……241
6.7 小结……242
习题6……243
参考文献……245

■第7章 强化学习……249
7.1 强化学习问题定义……249
7.1.1 强化学习基本概念……249
7.1.2 马尔可夫决策过程……251
7.1.3 强化学习问题定义……255
7.1.4 贝尔曼方程（Bellman equation）……256

7.2 基于价值的强化学习 ……259
7.2.1 策略迭代的基本模式 ……259
7.2.2 策略优化定理 ……260
7.2.3 策略评估方法 ……261
7.2.4 基于价值的强化学习算法 ……266
7.2.5 探索与利用 ……271
7.2.6 参数化与深度强化学习 ……274
7.3 基于策略的强化学习 ……278
7.3.1 策略梯度定理 ……278
7.3.2 基于蒙特卡洛采样的策略梯度法 ……279
7.3.3 Actor-Critic算法 ……282
7.4 深度强化学习的应用 ……284
7.4.1 深度强化学习在围棋游戏中的应用 ……284
7.4.2 深度强化学习在实际应用中的问题 ……287
7.5 小结 ……291
习题 7 ……292
参考文献 ……293

第 8 章 人工智能博弈 ……297
8.1 博弈论的相关概念 ……297
8.1.1 博弈论的诞生 ……297
8.1.2 博弈论术语与囚徒困境 ……298
8.1.3 博弈的分类 ……300
8.1.4 纳什均衡 ……301
8.1.5 人工智能与博弈论 ……304
8.2 博弈策略求解 ……304
8.2.1 研究问题 ……304
8.2.2 虚拟遗憾最小化算法 ……304
8.2.3 安全子博弈 ……314
8.3 博弈规则设计 ……317
8.3.1 研究问题 ……317
8.3.2 双边匹配算法 ……318
8.3.3 单边匹配算法 ……320
8.4 非完全信息博弈的实际应用 ……321
8.5 小结 ……325
习题 8 ……325
参考文献 ……327

第9章 人工智能未来发展和趋势 ……329
9.1 类脑计算 ……329
9.2 自动化机器学习 ……330
9.3 神经网络模型压缩 ……332
9.4 人工智能芯片 ……333
9.5 量子机器学习 ……334
9.6 人工智能伦理与治理 ……335
9.7 人工智能算法开发框架 ……336
9.8 小结 ……337
参考文献 ……338

附录 设计实验 ……339
实验一 黑白棋 AI 算法 ……339
实验二 逻辑推理实验 ……340
实验三 线性回归模型 ……341
实验四 人脸识别 ……342
实验五 深度学习 ……343
实验六 强化学习 ……344
实验七 对抗样本 ……344

第1章 绪论

1

“我们必须知道，我们必将知道”
——大卫·希尔伯特（David Hilbert）

人工智能（artificial intelligence，AI）是以机器为载体所展示出来的人类智能，因此人工智能也被称为机器智能（machine intelligence）。人类一直在不懈努力，让机器模拟人类在视觉、听觉、语言和行为等方面的某些功能以提升生产能力、帮助人类完成更为复杂或有危险的工作，更多造福人类社会。对人类智能的模拟可通过以符号主义为核心的逻辑推理、以问题求解为核心的探寻搜索、以数据驱动为核心的机器学习、以行为主义为核心的强化学习、以博弈对抗为核心的群体智能（两人及以上）等方法来实现。符号主义人工智能将概念（如命题等）符号化，从若干判断（前提）出发得到新判断（结论）；问题求解的探寻搜索依据已有信息来寻找满足约束条件的待求解问题的答案；数据驱动的机器学习方法则是从数据出发，从数据中发现数据所承载语义（如概念）的内在模式，利用学习得到的内在模式完成识别和分类等任务；以行为主义为核心的强化学习根据环境所提供的奖罚反馈来学习所处状态可施加的最佳行动，在“探索（未知空间）- 利用（已有经验）（exploration vs. exploitation）”之间寻找平衡，完成某个序列化任务，具备自我学习能力；博弈对抗则推动机器学习从“数据拟合”优化解的求取向“均衡解”的求取迈进。

“工欲善其事，必先利其器”，逻辑推理、搜索求解、机器学习和强化学习均是一种“计算”，要通过具体的计算载体实现。20世纪初涌现的可计算思想推动了原始递归函数、λ-演算和图灵机等“计算载体”的出现，由于图灵机以机械方式进行“计算”，因此成为现代计算机的理论模型，宣告自动计算时代的到来，也成为人工智能的“机器载体”。

当前以数据建模和学习为核心的人工智能通过整合数据、模型和算力在计算机视觉、自然语言、语音识别等特定领域取得了显著进展，研制的若干人工智能系统（如谷歌的Alpha Go、卡耐基·梅隆大学的Libratus以及IBM的Watson）等在挑战人类智力方面取得了巨大进步，这些人工智能算法或系统属于“领域人工智能（domain-specific AI）或弱人工智能（narrow AI）”的研究范畴。现有算法或系统还无法通过自身思考达到更高层次的智能，它们与具有从数据和经验中高度自我学习、直觉推理、自适应等特点的通用人工智能（artificial general intelligence）依然存在着差距，任何智能程度的机器都无法完全取代人类。

但是，人类一直会在探索智能的道路上不忘初心、矢志前行，“我们必须知道，我们必将知道”，大卫·希尔伯特（David Hilbert）1930年所喊出的与“不可知论主义”相对的话语会激励人类探索未知世界的决心和信心。

1.1 人工智能的起源

MOOC微视频：人工智能的历史发展

1955年8月，John McCarthy（时任达特茅斯学院（Dartmouth）数学系助理教授，1971年度图灵奖获得者）、Marvin Lee Minsky（时任哈佛大学数学系和神经学系Junior Fellow，1969年度图灵奖获得者）、Claude Shannon（贝尔实验室，信息理论之父）和Nathaniel Rochester（IBM，第一代通用计算机701主设计师）四位学者在一份题为《人工智能达特茅斯夏季研究项目提案》（“A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence”）的项目建议书中，首次使用

了“Artificial Intelligence（人工智能）”这个术语，从此，人工智能开始登上了人类历史舞台[9]。

在这个项目建议书中，四位学者希望美国洛克菲勒基金会（Rockefeller Foundation）能够提供一笔研究经费，资助一批学者于1956年夏天在达特茅斯学院研究“让机器能像人那样认知、思考和学习，即用计算机模拟人的智能”这样的前沿研究。

这个报告列举了人工智能需要研究的七个方面问题，分别是自动计算模拟人脑高级功能、使用通用语言进行计算机编程以模仿人脑推理、神经元相互连接形成概念、对计算复杂性的度量、算法自我提升、算法的抽象能力、随机性与创造力。

从那个时代开始，人工智能便沿着模拟人类某一方面功能（如感知、推理和运动等）而不断发展，如计算机视觉算法识别特定对象、自然语言理解算法来进行人机问答、无人驾驶汽车在特定场景行进等。

从人工智能的计算性和智能性角度而言，可给出如下人工智能的一个定义：人工智能是以机器为载体所实现的人类智能或生物智能。

这里就产生了两个问题：承载计算之能的器械如何产生、如何利用计算之器来模拟人类智能。对前一个问题的研究使得人类从手工计算时代迈入了自动计算时代，对后一个问题的研究形成了符号逻辑（以推理为核心）、联结主义（以统计机器学习为手段）和行为学派（从环境交互过程中进行策略学习）等人工智能不同研究方法。

1.2 可计算载体：形式化与机械化

MOOC微视频：可计算思想的思源与发展

早期数学概念（如公理或定理）一般通过自然语言来描述，如《庄子·天下篇》中“一尺之棰,日取其半,万世不竭”对极限思想的描述以及亚里士多德在《逻辑学》中所阐述的“由于凡人都会死以及苏格拉底是人，因此苏格拉底是要死的”的三段式推理。

但是，自然语言在很大程度上难以精确表达数学概念，为了解决这一问题，研究人员开始用符号来表达数学概念，即通过形式化语言来刻画数学概念，如“任意一个直角三角形的两条直角边的平方和等于斜边的平方”可形式化如下表示：

$\forall \Delta \in$ 直角三角形，x，y和z分别是三角形的两条直角边和斜边，则$x^2+y^2=z^2$

如果所有的数学概念都可以被形式化描述，在给定一组推理规则后，就可以从已有知识（如公理、定理和命题等）出发，根据推理规则，得到众多先前未定义的知识。这样的系统被称为形式化系统。

如果已有知识和推理规则都是正确的，则形式化系统可源源不断得到所有可以被推导出来的知识。为了保证形式化系统的有效性，需要其具有如下性质。

- 完备性：所有能够从某形式化系统推导出来的知识，都可以从这个形式化系统推导出来。
- 一致性：所有可推导出来的知识不会同时推导出来其否定，如推导得到$A \geqslant 0$，则不会同时推导得到$A<0$。也就是说形式化系统是自洽的（self-consisitent）、非矛盾的。
- 可判定性：对于形式化系统推导得到的任何知识，存在算法可在有限步内判定其为真或为假。

上述问题就是1900年德国数学家大卫·希尔伯特（David Hilbert，1862—1943）在巴黎举行的第二届国际数学会议上以“数学问题”为题的演讲中提出了23个重要数学难题的第二个问题，即证明算术公理的相容性（the compatibility of the arithmetical axioms）。希尔伯特希望构建起有效的形式化大厦，使得推理这一过程能够“自动计算”。

1931年，库尔特·弗雷德里希·哥德尔（Kurt Friedrich Gödel）发表了一篇影响深远的论文《论数学原理及有关系统中不可判定命题》，提出了著名的“哥德尔不完备定理”。“哥德尔不完备定理”指出：任何表达力足够强的（递归可枚举）形式系统都不可能同时具有一致性和完备性，而且这个系统本身的一致性不能在系统内被证明。

“哥德尔不完备定理”触及了不可计算这一难题，即有些数学问题是不可求解的（是不可判定问题），如费马大定理（当整数n大于2时，方程$x^n+y^n=z^n$没有整数解）是不可计算的。

哥德尔通过原始递归函数、阿隆佐·邱奇（Alonzo Church，1903—1995）通过λ-验算（λ-calculus）分别从数学角度和数理逻辑角度研究了可计算问题。很显然，原始递归函数和λ-演算无法成为人工智能的“机器载体”。

1937年，图灵（Alan Turing，1912—1954）发表了《论数字计算在决断难题中的应用》论文，提出了现代计算机的理论模型“图灵机模型”。

图灵机模型是一个抽象的机械式计算装置，它有一条两端无限长的纸带，纸带上为一个一个可擦写小方格，小方格中可放入某个计算任务所需的数据、程序指令或者方格为空。在图灵机模型中，程序控制器事先存储了若干指令集，读写控制指针从左到右，依次从纸带方格中读入相应信息，触发程序控制器中指令集的对应指令去执行某一操作，并将计算结果写入纸带方格。上述步骤周而复始，直至计算结束。一旦计算结束（即图灵停机），纸带上所记载的信息就是计算结果。

如图1.1所示，给出了gcd(21, 16)×10+17这一计算任务。gcd( )为求取两个数之间的最大公约数函数。这样，这一计算任务意味着24和16之间最大公约数与10相乘结果再与17相加。

在计算这一任务时，图灵机首先读入gcd(24, 16)这一信息，触发最大公约数这一指令得到24和16的最大公约数8，然后分别触发乘法指令和加法指令操作，最终在纸带方格上留下97这一计算结果后图灵机停止工作。可以看出，当“gcd(24, 16)×10+17”这一任务被计算完毕后，在原来数字17的方格上出现了97这一结果，其他数字或程序信息均被擦写掉了。

图灵机在执行上述任务过程中仅用了有限步骤后停机，因此可以说这一任务是可计算的。

邱奇–图灵论题（Church–Turing thesis, computability thesis）指出：凡是可计算的函数都可以用图灵机计算。现在已经知道，原始递归函数、λ–演算和图灵机在功能上是等效的，即任何可计算函数均可通过这三种方式完成计算、任何不可计算函数均无法通过这三种方式完成计算。

“工欲善其事，必先利其器”，如前所述，原始递归函数和λ–演算

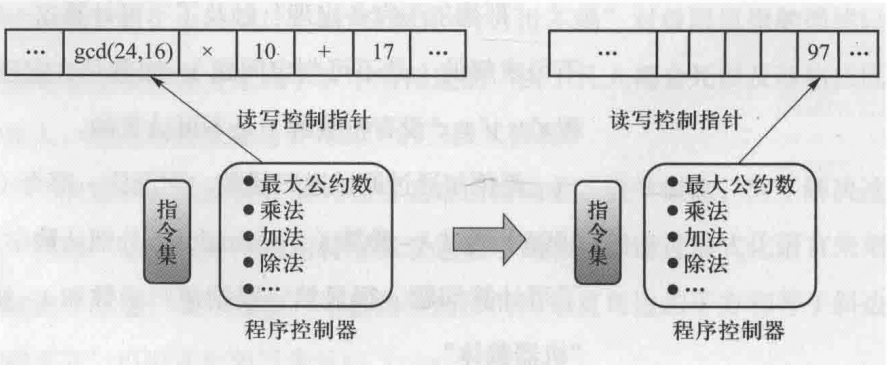


图1.1 图灵机模型进行计算的示意图

通过形式化方法进行“计算”，图灵机模型则通过机械化机制进行“计算”，因此图灵机模型成为现代计算机的理论模型，推动了自动计算时代的到来，也成为人工智能的“机器载体”。由于这一伟大贡献，图灵被誉为理论计算机之父，计算机界最高奖图灵奖就是用其名字命名。

1.3 智能计算方法

以机器为载体来模拟人类智能的某一方面，可通过以符号主义为核心的逻辑推理、以问题求解为核心的探寻搜索、以数据驱动为核心的机器学习、以行为主义为核心的强化学习、以博弈对抗为核心的决策智能等方法来实现。下面对这些方法进行简要介绍。

1.3.1 符号主义为核心的逻辑推理

推理是进行思维模拟的基本形式之一，是从一个或几个已知的判断（前提）推出新判断（结论）的过程。推理、搜索和约束满足一起并称为人工智能问题求解中的三大方法。在人工智能发展初期，脱胎于数理逻辑的符号主义人工智能（symbolic AI）是人工智能研究的一种主流学派。

推理只有建立在一套高度概括、抽象、严格化和精确化的符号系统中，才能得到飞跃发展。因此，在符号主义人工智能中，概念（命题等）不再是用自然语言来描述，而是通过所定义的“符号”及符号之间的关系来表示，所解决的问题就是构造一个证明来阐释其成立或者不成立（证伪）。

例如，使用$human(x)$和$immortal(x)$来分别表示x是人以及x是不朽的，同时使用$\neg immortal(x)$来表示x不是不朽的。这样可如下来表示“凡人均不是不朽的”这一命题：$human(x) \rightarrow \neg immortal(x)$。由于苏格拉底是人这一命题是正确的，即存在$human(Socrates)$，于是下述推导成立：$\forall x(human(x) \rightarrow \neg immortal(x)) \quad \wedge human(Socrates) \rightarrow \neg immortal(Socrates)$，即从“凡人均不是不朽”和“苏格拉底是人”这两个命题推导出“苏格拉底不是不朽的”。上述逻辑推导过程就是亚里士多德（Aristotle，BC 384—BC 322）所提出的三段论（syllogism）推导方法。当然，亚里士多德在他那个时代是通过命题逻辑来解释三段论的，而不是通过上

述谓词逻辑来进行阐述的。

推理一般包括归纳（inductive）推理、演绎（deductive）推理和因果（causality）推理等主流方法。

归纳推理是从个别事实出发，推演出一般性知识作为结论的推理过程，即从特殊到一般、由具体到抽象、由现象到本质。如从“汉语、英语和法语是联合国工作语言”“俄语、阿拉伯语与西班牙语是联合国工作语言”和“没有其他语言是联合国工作语言”可以推导出“联合国工作语言为汉语、英语、法语、俄语、阿拉伯语与西班牙语六种语言”。

与归纳推理相反，演绎推理是从一般性前提出发，通过推导即“演绎”，得出具体陈述或个别结论的过程。如上述亚里士多德三段论，以及由“所有的金属都会导电”和“铜是一种金属”推导演绎出“铜会导电”。

人类并不总是从概率角度来思维，而是善于因果效应角度进行思考。哲学上把现象和现象之间的“引起和被引起”的关系，叫做因果关系，其中引起某种现象产生的现象叫做原因，被某种现象引起的现象叫做结果。如“力，形之所以奋也”中，“力”是因，“形之奋”是果。因果推理就是用来判断事物间存在的原因和结果这样的关系，即因果关系，这是一种重要的推理方法。一般采用结构因果模型（structural causal model，SCM）和因果图（causal diagram）来进行因果推理[13, 16, 19]。

图灵奖获得者Judea Pearl将推理按照由易到难程度分成三个层次[17, 18]：① 关联（association），可直接从数据中计算得到的统计相关；② 干预（intervention），无法直接从观测数据就能得到关系，如“某个商品涨价会产生什么结果”这个问题不仅与新价格有关，而且会与客户购买行为、用户收入等因素相关；③ 反事实（counterfactual），某个事情已经发生了，则在相同环境中，这个事情不发生会带来怎样的新结果。

这里可使用“反事实”这个概念来定义事物间的因果关系。反事实框架（counterfactual framework，也叫potential outcomes）是大卫·刘易斯（David Lewis，1941—2001）等人提出的推断因果关系的标准[7,8]。事实是指在某个特定变量（A）的影响下可观测到的某种状态或结果（B）。“反事实”是指在该特定变量（A）取负向值时可观测到的状态或结果（B'）。条件变量A对于结果变量B的因果性就是A成立时B的状态与A取负向值时“反事实”状态（B'）之间的差异。如果这种差异存在且在统

计上是显著的，则说明条件变量与结果变量之间存在因果关系。

表1.1给出了归纳推理、演绎推理和因果推理之间的差异描述，其中A、B和A_i是不同的命题。

表1.1　不同推理方法的差异描述

推理方法	推理方式	说明
归纳推理	如果A_i（i为若干取值），那么B	从若干事实出发推理出一般性规律
演绎推理	如果A，那么B	A是B的前提，但不是唯一前提，因此A是B的充分条件。当然，在特殊情况下A也可为B的充分必要条件
因果推理	因为A，所以B	A是B的唯一前提，因此“如果没有A，那么没有B”也成立。

在表1.1中，归纳推理从若干个具体观测结果A_i得出一般性结论B，由于A_i可能无法涵盖全量数据，因此只能说A_i与B之间存在概率性，而不是必然性。

演绎推理完成了“如果A，那么B”的推导，这里A是B的充分条件。A是B的充分条件意味着A蕴含B，即B是A的子集。“如果A，那么B”成立也等同于“如果没有B，那么没有A”（即原命题成立，则逆否命题成立）。在演绎推理中，只要前提是正确的、推理规则也是正确的，则推理得到的结论一定是有效的。

因果推理则完成了“因为A，所以B”的推导，其对应“如果没有A，那么没有B”。要注意的是，因果性与相关性不是同一个概念。比如“公鸡鸣叫（A）”与“太阳升起（B）”具有相关性，但是不具有因果性，因为“公鸡不鸣叫，太阳依然升起（即如果没有A，那么没有B是不成立的）”。

本节讨论了若干充分条件等知识，可用如下描述来刻画这些知识内涵：有之必然，无之未必不然，是谓充分条件；有之未必然，无之必不然，是谓必要条件；有之必然，无之必不然，是谓充要条件。

逻辑推理在人工智能领域的应用推动了专家系统（expert system）的产生。专家系统中存储了问题求解所需领域知识，诸如事实、规则和其他信息等，可根据用户提交的问题，应用相应推理规则，得到问题的解决方案。

1.3.2 问题求解为核心的探寻搜索

现实世界中许多问题都可以通过搜索的方法来求解，例如，最佳

出行线路的计算推荐或是寻找合理安排的课程表。搜索是人工智能求解中的一种主要技术，其依据已有信息来寻找满足约束条件的待求解问题答案。搜索算法一般可以包括无信息（uninformed）搜索、有信息（informed）搜索和对抗搜索（adversarial search）。

无信息搜索是一种盲目搜索方法，按照其利用的搜索策略来扩展被搜索空间中的结点次序可分类为广度优先搜索和深度优先搜索等搜索方法。

有信息搜索也称为启发式搜索，其在搜索的过程中可利用与所求解问题相关的辅助信息，代表算法为贪婪最佳优先搜索（greedy best-first search）和A*搜索。

对抗搜索也称为博弈搜索（game search），指在一个竞争的环境中，智能体（agents）之间通过竞争实现相反的利益，一方最大化这个利益，另外一方最小化这个利益。代表性算法包括最小最大搜索（minimax search）、Alpha-Beta剪枝搜索（pruning search）和蒙特卡洛树搜索（Monte-Carlo tree search）。最小最大搜索是在对抗搜索中最为基本的一种让玩家来计算最优策略的方法；Alpha-Beta剪枝搜索改进了最小最大搜索需要遍历搜索空间所有状态这一不足，在搜索过程中可剪除无需搜索的分支结点，且不影响搜索结果；蒙特卡洛树搜索通过采样机制来选择合理结果而非穷举所有结果来实现搜索。

1.3.3 数据驱动为核心的机器学习

与逻辑推理直接定义公理、命题和规则不同，数据驱动（data-driven）方法是从数据出发，从承载表达某一概念的数据中直接学习该概念所涉及的模式，然后基于学习得到的模板对未知数据进行分类或识别。如果说逻辑推理可视为“从知识到知识”，那么机器学习完成了“从数据到知识”的过程。

按照学习算法对数据利用方式的不同，机器学习算法可分为监督学习、无监督学习和半监督学习等。在监督学习中，收集n个标注数据作为训练数据集，数据集中的标注数据及其标注信息记为$(X, Y) = \{(x_i, y_i), i = 1, \cdots, n\}$，其中第$i$个样本数据或者其特征表达记为$x_i$，$y_i$为$x_i$所对应的类别等标注信息。这里的$x_i$可以是图像或文档等数据，$y_i$为所对应的人脸、汽车或情感等语义类别信息。

为了得到x_i中所蕴含语义信息的模式，监督学习基于训练数据，从假设空间（hypothesis space）这一学习范围中学习得到一个最优映射函数f（也被称为决策函数），映射函数f将数据映射到语义标注空间，实现数据的分类和识别。记函数f对x_i的映射结果为$f(x_i)$，很显然一个良好的监督学习算法就是使得$f(x_i)$与y_i之间的差值越小越好，即模型学会了如何将x_i映射为其所表达的高层语义。一旦训练得到了映射函数f，就可以利用映射函数f来对未知数据进行识别和分类。

除了从训练数据中直接来学习映射函数外，也可以通过概率模型来进行识别和分类。如在判别式学习方法中，可从训练数据中学习条件概率分布$P(y_i|x_i)$，进而根据这一取值来判断数据x_i属于y_i的概率，以实现对x_i的分类和识别。或者如在生成式学习方法中，学习数据和类别标签的联合概率分布$P(X, Y)$，再通过贝叶斯理论来求取后验概率，完成识别和分类。

赫布理论（Hebbian theory）指出“神经元之间持续重复经验刺激可导致突触传递效能增加（Neurons that fire together, wire together）”，这一“认为神经元之间突触的强弱变化是学习与记忆的生理学基础”理论为联结主义人工智能研究提供了认知神经心理学基础[3]。作为联结主义方法的代表，深度学习是近期在许多领域取得了显著效果的一种监督学习方法，与其他监督学习算法利用手工构造特征（hand-crafted features，如直方图、纹理或形状等）去学习构造分类映射函数或条件概率分布函数不同，深度学习算法一般通过构造逐层抽象的“端到端（end-to-end）”机制来学习隐含在数据内部的隐含模式，得到更强表达力和泛化能力的特征表达，然后将深度学习得到的特征用于识别和分类等任务[4, 6]。例如，卷积神经网络（convolutional neural network，CNN）通过卷积层、池化层（pooling）、激活函数（activation）和全连接层（fully connected）来构造深度模型，辅以误差后向传播（error back-propogation）等机制学习模型参数，得到输入数据的更具区别力的特征表达[20, 26, 27]。

与监督学习不同，顾名思义，无监督学习指数据本身不包含标注信息、半监督学习指一部分数据有标注信息而一部分数据没有标注信息。

监督学习算法包括回归分析、提升算法（boosting）、支持向量机和决策树等判别学习方法，也包括隐狄利克雷分布（latent dirichlet

allocation，LDA）和隐马尔可夫链等生成式学习方法。无监督学习包括聚类、降维（如主成分分析）和期望极大（expectation maximization，EM）算法等。

1.3.4 行为主义为核心的强化学习

人工智能领域的一个主要研究目标是实现完全自主的智能体，该智能体能与其所处环境进行交互，根据环境所提供的奖励反馈或惩罚反馈来学习所处状态可施加的最佳行动（如碰墙后折返），通过运用“尝试-试错（trial and error）”与平衡“探索（未知空间）与利用（已有经验）（exploration vs. exploitation）”等机制不断进步，改进行动策略。强化学习（reinforcement learning，RL）就是这样一种赋予智能体自监督学习能力，使其能够自主与环境交互，做出序列决策，完成序列化形式的任务，向“学会学习（learning to learn）”这一能力塑造目标而努力[23]。

强化学习起源于行为主义理论，即智能体如何在环境刺激和反馈下做出适当回应。当该回应满足智能体预期时，则说明该智能体已经学习到了某种知识。强化学习就是通过不断与环境交互，利用环境给出的奖惩来不断地改进策略。与前面所介绍的学习算法不同，强化学习解决的是序贯决策优化问题，即智能体与环境不断交互，在某个状态采取某一行为后进入一个新的状态，根据环境给出的奖励或惩罚反馈（reward）来改进策略（即在某个状态下应该采取怎样动作），以求获得最大的累积奖惩（accumulated reward）。

表1.2给出了监督学习、无监督学习和强化学习在学习依据、数据来源、决策过程和学习目标四个方面的差异。

表1.2 监督学习、无监督学习和强化学习的差异描述

	监督学习	无监督学习	强化学习
学习依据	基于监督信息	基于对数据结构的假设	基于评价（evaluative）
数据来源	一次性给定（含标注信息）	一次性给定（无标注信息）	在序列交互（interactive）中产生，且只有在一个序列结束后才会反馈明确奖惩值
决策过程	根据标注信息做出单步静态决策（one-shot）	无	根据环境给出的滞后回报做出序列决策（sequential）
学习目标	样本空间到高级语义空间的映射	同一类数据的分布模式	选择能够获取最大收益的状态到动作的映射

由于强化学习是一种针对序列优化的学习方法，因此其与马尔可夫决策过程（markov decision process，MDP）息息相关，MDP刻画了当前状态采取某一行动后进入后续状态，且因为采取了这一行动会从环境获得一定的奖励反馈或惩罚反馈的机制。

在强化学习中，有一个重要的概念叫Q学习[25]。Q学习被用来学习智能体的q函数，q函数记录了某个状态下采取某一动作所能够收到的奖励值或惩罚值。Q学习可以为智能体构造一个状态-行为效用（state-action utility）矩阵（表格），矩阵中行和列分别代表状态和行为，矩阵的行列值为某一状态下采取某个行为所能够获得的回报。这样，完成训练后可得到一张Q表格，智能体可根据这一表格来查找当前状态下效用值较大的动作，从而完成任务。有趣的是，最早提出Q学习算法的Watkins在回忆时指出之所以选择Q作为函数名，是借用了“质量（quality）”首字母之意。

但是在实际环境中，智能体所位于状态和可采取行为可能异常庞大，如果一个状态在训练中从未出现过，Q学习是无法处理的，使得Q学习预测能力和泛化能力较弱。一个可行的思路是将q函数参数化（parametrize），用神经网络来拟合q函数，这就是深度学习与强化学习结合而形成的深度强化学习（deep reinforcement learning，DRL）[10]。

1.3.5 博弈对抗为核心的决策智能

博弈论（Game Theory）是经济学的一个分支，但是随着计算机科学的发展，人工智能技术越来越多地应用在博弈论的研究当中，传统博弈论与人工智能的深度融合闪烁着决策智能的光芒。

现代博弈论的思想起源于1944年冯·诺依曼（John von Neumann）与奥斯卡·摩根斯特恩（Oskar Morgenstern）合著的《博弈论与经济行为》[14]。这本书用数学形式来阐述了博弈论及其应用。将博弈论发扬光大的是约翰·纳什（John Forbes Nash Jr.）于1950年在其博士论文中提出的“非合作博弈（non-cooperative Games）”及其均衡解一定存在这一纳什均衡思想[11, 12]，纳什本人因为这一贡献于1994年获得了诺贝尔经济学奖。

博弈行为是多个带有相互竞争性质的主体，为了达到各自目标和利益，采取的带有对抗性质的行为，即“两害相权取其轻，两利相权取其

重”。现代博弈论主要研究博弈行为中最优的对抗策略及其稳定局势，协助对弈者在一定规则范围内寻求最合理的行为方式，推动机器学习从“数据拟合”过程中以“求取最优解”为核心向博弈对抗过程中“求取均衡解”为核心的转变。

上述人工智能方法各有千秋、各有优劣，如逻辑推理方法解释性强，但难以拓展；搜索方法从已有答案中进行查找，但受困于机械式匹配的桎梏；数据驱动模型擅于预测识别，但是其过程难以理解；强化学习手段能对未知空间进行探索，但其依赖于策略学习；博弈对抗在完全信息条件下表现优良，但难以找到非完全信息条件下的通用框架。因此，需要有机协调知识指导下演绎、数据驱动中归纳、行为强化内规划等不同的人工智能方法和手段，建立知识、数据和反馈于一体的人工智能理论和模型，这是目前人工智能研究的重点。

应该指出，在解决现实生活中的实际问题时，一般是多种机器学习算法融汇使用，如Deepmind研制的AlphaGo通过深度学习来构造黑白相间棋盘的特征表达、通过强化学习来进行自我博弈以提高智能体学习能力、通过蒙特卡洛树搜索来寻找较佳落子[21]。

近年来，由于脑科学、神经科学和认知科学与信息科学等学科的进步，科学家们已能在微观尺度上观测基因和蛋白质结构，在介观尺度上研究细胞，神经环路和网络结构，在宏观尺度上研究脑区结构以及认知行为。因此，模拟和仿真大脑结构和功能的类脑计算（也称为脑启发计算）也是当前一个研究热点[2]。

类脑计算是受脑功能和脑神经网络连接机制启发的一种计算架构，它以神经形态计算的模式来部分模拟大脑功能与其结构的对应关系和反馈连接，即通过仿真、模拟和借鉴大脑生理结构和信息处理过程的装置、模型和方法，在结构层次模拟人脑、在器件层次逼近大脑等。

1.4 本书内容介绍

MOOC微视频：人工智能研究的基本内容

人工智能具有“至小有内”内涵和“至大无外”外延的双重特点，人工智能作为一种使能技术，天然具有与其他学科研究进行交叉的内禀特性，与计算机科学、机器人、认知科学进行广泛交叉，并且在科学、

工业和社会学等方面应用活跃。

本书主要从数据智能这一角度来介绍人工智能，主要内容如下。

第2章“逻辑与推理”主要介绍命题逻辑、谓词逻辑、知识图谱和因果推理的内容。

第3章“搜索求解”主要介绍启发式搜索、对抗搜索（含最小最大搜索、Alpha-Beta剪枝搜索）和蒙特卡洛树搜索的内容。

第4章“机器学习：监督学习”主要介绍机器学习基本概念、线性回归、决策树、线性判别分析、Ada Boosting、支持向量机和生成学习等内容。

第5章“机器学习：非监督学习”主要介绍*K*-means、主成分分析、特征人脸分析、隐性文法分析和EM算法等内容。

第6章“深度学习”主要介绍前向神经网络、卷积神经网络、循环神经网络及其在自然语言和计算机视觉等方面的应用。

第7章“强化学习”主要介绍马尔可夫决策过程、策略优化与策略评估、Q学习和深度强化学习的内容。

第8章“人工智能博弈”主要介绍了博弈论中纳什均衡、博弈策略求解和博弈规则设计以及非完全信息博弈等内容。

第9章对人工智能未来发展和趋势进行了简略总结。

1.5 小结

1955年提出人工智能这一概念时，人工智能的“初心”是“仿真”人类行为。经过长期发展，人们已经慢慢意识到，在脑科学和神经科学等尚未弄清人脑功能机理的前提下，难以用机器直接实现人脑功能。物理学家费曼（Richard Feynman）曾经说过，“What I cannot create, I do not understand（不可造者，未能知也）”。如果把这个说法用于机器“复制”人脑功能这一语境，则意味着需要全面了解神经进化（neuroevolution）[22]机理，这一方面的研究仍然需要付出巨大的努力。

经典人工智能理论框架建立在以递归可枚举为核心的演绎逻辑和语义描述基础方法之上，由于先决条件问题（qualification problem，

即枚举描述促发某一行为发生的所有前提条件）和隐性分支问题（ramification problem，即枚举刻画某个行为可能导致的所有后续潜在结果）的存在，使得人们难以事先拟好智能算法能够处理的所有情况，因此智能算法在处理不确定性、开放性和动态性等问题时难以发挥作用。

当前以数据建模和学习为核心的人工智能通过整合数据、模型和算力在计算机视觉、自然语言、语音识别等特定领域取得了显著进展，研制的若干人工智能系统（如IBM的问答竞智系统Watson、谷歌的围棋竞智系统AlphaGo以及卡耐基·梅隆大学的扑克竞智系统Libratus等）在挑战人类智力方面取得了巨大进步，这些人工智能算法或系统属于“领域人工智能（domain-specific AI）或弱人工智能（narrow AI）”研究范畴。现有算法或系统还无法通过自身思考达到更高层次的智能，它们与具有自我学习、直觉推理、自适应和能力迁移等特点的通用人工智能（artificial general intelligence）依然存在着差距，也与人类在摄食、情绪和感情等方面所展示的行为具有明显区别，但是这些在特定领域取得成功的研究为人们探索与发展新一代人工智能提供了重要的借鉴和新的方法[1, 5, 15, 24]。

理解人类认知并建立可计算认知模型需要厘清从行为到神经系统和回路再到细胞核分子不同层面的因果关联，因此理解人类认知的神经基础是促进人工智能取得长足发展一个很有吸引力但遥远的目标。

当前人工智能的计算载体图灵机模型所能够完成的可计算任务都是可递归的（可递归意味着可枚举和有序），人类面临的许多现实问题具有不确定性、脆弱性和开放性等特点，人类也是智能机器的服务对象和最终“价值判断”的仲裁者，因此，需要将人类智能与机器智能进行有效协同。建立在演绎逻辑、语义描述和计算观测数据与假设空间之间置信度的贝叶斯理论的基础方法之上的机器智能都无法完全取代人类智能，因此迫切需要将人的作用或认知模型引入到人工智能系统中，形成混合-增强智能形态（hybrid-augmented intelligence），这种形态是人工智能或机器智能可行的、重要的成长模式[28]，也是发展人工智能的可行之道。

哲学家丹尼尔·丹尼特（Daniel Dennett）曾经说过，“提出更好的问题，打破旧的设问习惯与传统是人类认识自身、认识世界这一宏伟事业中非常困难的一部分。”

“其作始也简，其将毕也必巨”，人工智能必将影响和推动人类社

会的深刻变化。“我们必须知道，我们必将知道”，相信大卫·希尔伯特（David Hilbert）在1930年所持有的“可认知”乐观主义态度将鼓舞人工智能研究者和实践者进行不断探索。

习题1

1. 为保证形式化系统的有效性，下面哪个选项不是形式化系统需要具有的性质？（　　）

   A. 完备性　　B. 可靠性

   C. 一致性　　D. 可判定性

2. Deepmind研制的AlphaGo算法没有使用哪个人工智能方法？（　　）

   A. 强化学习　　B. 深度学习

   C. 蒙特卡洛树搜索　　D. 逻辑推理

3. 图灵奖获得者Judea Pearl将推理按照由易到难程度分成三个层次：（　　）。

   A. 介入、关联、反事实　　B. 关联、介入、反事实

   C. 反事实、介入、关联　　D. 关系、介入、反事实

4. 以下哪种搜索方法属于启发式搜索？（　　）

   A. 广度优先搜索　　B. 蒙特卡洛树搜索

   C. 深度优先搜索　　D. A*搜索

5. 以下哪一种算法不属于监督学习算法？（　　）

   A. 隐马尔可夫链　　B. 支持向量机

   C. 聚类　　D. 决策树

6. 以下哪种学习方法的学习目标是学习同一类数据的分布模式？（　　）

   A. 监督学习　　B. 无监督学习

   C. 强化学习　　D. 博弈对抗

7. 以机器为载体来模拟人类智能的某一方面，可通过以符号主义为核心的逻辑推理、以问题求解为核心的探寻搜索、以数据驱动为核心的机器学习、以行为主义为核心的强化学习、以博弈对抗为核心的决策智能等方法来实现。请比较上述人工智能不同方法的优劣。

8. 请参考图1.1，画出使用图灵机模型计算（5+2）×3的过程示意图。

9. 推理方法在生活中无处不在，请你针对归纳推理、演绎推理和因果推理分别举出实际例子。

10. 监督学习和强化学习在做出预测后，都需要某种“反馈信息”来提升学习性能。请描述两种学习方法中利用“反馈信息”的不同。

11. 目前人工智能在大多数任务上所体现的性能与人类智能存在

不小差距。机器智能和人类智能之间进行相互协同所形成的人机混合增强是推动人工智能发展的务实之路。“假舆马者，非利足也，而致千里；假舟楫者，非能水也，而绝江河”，人类可以通过外界帮助来提升能力。请列举若干人机混合增强的例子。

参考文献

[1] GHAHRAMANI Z. Probabilistic machine learning and artificial intelligence [J]. Nature, 521: 452–459, 2015.

[2] HASSABIS D, KUMARAN D, SUMMERFIELD C, et al. Neuroscience-Inspired Artificial Intelligence [J]. Neuron, 95(2): 245–258, 2017.

[3] HEBB D O.The Organization of Behavior [M] .New York: Wiley & Sons, 1949.

[4] HINTON G, SALAKHUTDINOV R. Reducing the dimensionality of data with neural networks [J] .Science, 313(5786): 504–507, 2006 .

[5] JORDAN M, MITCHELL T. Machine learning: trends, perspectives, and prospects [J]. Science, 349(6245): 255–260, 2015.

[6] LECUN Y, BENGIO Y, HINTON G. Deep learning [J]. Nature, 521(7553): 436–444, 2015.

[7] LEWIS D. Counterpart Theory and Quantified Modal Logic [J]. Journal of Philosophy, 65: 113–126, 1968.

[8] LEWIS D. Counterfactuals [M]. Oxford: Blackwell Publishers and Cambridge, MA: Harvard University Press, 1973, Reprinted with revisions, 1986.

[9] MCCARTHY J, MINSKY M, ROCHESTER N, et al. A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence, 1955 .

[10] MNIH V, KAVUKCUOGLU K, SILVER D, et.al. Human-level control through deep reinforcement learning [J]. Nature, 518: 529–533, 2015.

[11] NASH J F. Equilibrium points in n-person games [J]. Proceedings of the National Academy of Sciences of the United States of America, 36(1): 48–49 , 1950 .

[12] NASH J F. Non-Cooperative Games [D]. Princeton: Princeton University, 1950.

[13] NEYMAN J S. On the Application of Probability Theory to Agricultural Experiments [J]. Essay on Principles. Section 9, Statistical Science, 5: 465–472, 1923.

[14] NEUMANN J, MORGENSTERN O. Theory of Games and Economic Behavior [M]. Princeton: Princeton University Press, 1944.

[15] PAN Y H.Heading toward artificial intelligence 2.0 [J]. Engineering 2(4): 409– 413, 2016.

[16] PEARL J. Causal diagrams for empirical research [J]. Biometrika, 82(4): 669–710, 1995.

[17] PEARL J. Causality: models, reasoning and inference[M]2nd edition. Cambridge: Cambridge University Press, 2013.

[18] PEARL J. Theoretical impediments to machine learning with seven sparks from the causal revolution [J]. arXiv: 1801.04016, 2018.

[19] RUBIN D. Estimating causal effects of treatments in randomized and non-randomized studies [J]. Journal of Educational Psychology, 66(5): 688–701, 1974.

[20] RUMELHART D E, HINTON G E, WILLIAMS R J. Learning representations by back-propagating errors [J]. Nature, 323 (6088): 533–536, 1986.

[21] SILVER D, HUANG A, MADDISON C J, et.al. Mastering the game of Go with deep neural networks and tree search[J]. Nature, 529 (7587): 484–489, 2016.

[22] STANLEY K O, CLUNE J, LEHMAN J, et al. Designing neural networks through neuroevolution [J]. Nature machine intelligence, 1: 24–35, 2019.

[23] SUTTON R S, BARTO A G. Reinforcement Learning: An Introduction [M] .2nd Edition. Cambridge, MA: MIT Press, 2018.

[24] TENENBAUM J B, KEMP C, GRIFFITHS T L, GOODMAN N D. How to grow a mind: Statistics, structure, and abstraction [J]. Science, 331, 1279–1285, 2011.

[25] WATKINS C J, DAYAN P. Q-learning [J]. Machine Learning, 8: 279–292, 1992.

[26] WERBOS P J. Backpropagation Through Time: What It Does and How to Do It [J]. Proceedings of the IEEE, 78(10): 1550–1560, 1990.

[27] WERBOS P J. Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences [D] .Harvard University, 1974.

[28] ZHENG N N, LIU Z Y, REN P J, et al. Hybrid-augmented intelligence: collaboration and cognition [J]. Frontiers of Information Technology & Electronic Engineering, 18(2): 153–179, 2017.

第2章 逻辑与推理

2

逻辑与推理是人工智能的核心问题。人类思维活动的一个重要功能就是设定一些逻辑规则，然后进行分析，如通过归纳和演绎等手段对现有观测现象由果溯因（归纳）或由因溯果（推理），从观测现象中得到结论。

关于逻辑与推理的研究由来已久。古希腊学者亚里士多德被誉为“逻辑学之父”，他的逻辑理论完全是以演绎推理的方式架构而成的，其中就包含了著名的“三段论”方法。亚里士多德的逻辑理论自提出以来一直主导着逻辑学界，直到19世纪，布尔提出了数理逻辑理论，才取代了其在逻辑学界的支配地位。

墨翟（尊称为墨子）被认为是东方逻辑学的奠基人。他提出了名、辞、说三种基本思维形式和由故、理、类三物构成的逻辑推理。《墨辨·经下》里有个著名的描述归纳推理的句子：“擢虑不疑，说在有无”。这里的“擢”即抽引，表示从一个典型个别事例中抽引出一个一般命题（虑），这种归纳推理的方式不用怀疑，因为其理由或根据是“与典型事例同类的事物联系是存在，还是不存在”。

擢虑不疑，说在有无
——墨子《墨辨·经下》

在人工智能发展初期，人工智能研究的主流学派——符号主义人工智能（symbolic AI）也是脱胎于逻辑推理。在符号主义人工智能中，所有概念均可通过人类可理解的符号及符号之间的关系来表示，对符号进行逻辑推理，从而实现对符号所描述内容的辨析。

本章将从命题逻辑、谓词逻辑、知识图谱和因果推理四个部分介绍逻辑与推理。命题逻辑和谓词逻辑都是数理逻辑的一部分。命题逻辑（propositional logic）是应用一套形式化规则对以符号表示的描述性陈述（称为命题）进行推理的系统。命题逻辑以原子命题作为最基本的单位，无法对原子命题进行分解，分离其主语和谓语，因此需要引入更加强大的逻辑表示方法——谓词逻辑（predicate logic）。

有了命题逻辑和谓词逻辑这两种工具以后，可以将人类的知识转换成计算机能读懂的图的形式——知识图谱，并在知识图谱上进行推理得到新知识。因果推理是一种重要的推理方法，被用来判断事物间存在的原因和结果这样的关系，即因果关系。

2.1 命题逻辑

MOOC微视频：
命题逻辑

在数理逻辑中，为了表达概念，陈述理论和规则，常常需要用语言进行描述。但是日常使用的自然语言，往往叙述时不够准确，容易产生二义性，因此需要引入一种目标语言。这种目标语言由表达判断的一些语言汇集而成，它和一些公式符号相结合，就形成了数理逻辑的形式符号体系。命题逻辑是数理逻辑的基础。

定义2.1 命题：命题是一个能确定为真或为假的陈述句。

命题通常用小写符号来表示，如p或者q。命题总是具有一个“值”，称为真值。真值有为真或为假两种，分别用符号T（True）和F（False）表示。只有具有确定真值的陈述句才是命题，无法判断正确或错误的描述性句子都不能作为命题。

例2.1　判断下列句子是否为命题。

（1）北京是中国的首都。

（2）请出去。

（3）您去开会吗？

（4）13能被6整除。

（5）这座山真高啊！

（6）我正在说谎。

本例中（1）和（4）是命题，且（1）是真命题、（4）是假命题。（2）是祈使句、（3）是疑问句、（5）是感叹句，这3个句子都无法从现有条件中判断其真假，因此不是命题。（6）既不能为真，也不能为假，是悖论，所以（6）不是命题。悖论不是命题，在本书中不做过多讨论。

定义2.2　原子命题：原子命题指不包含其他命题作为其组成部分的命题，又称简单命题。

可通过命题联结词对已有命题进行组合，从而得到新命题。这些通过命题联结词得到的命题被称为复合命题。

定义2.3　复合命题：复合命题指包含其他命题作为其组成部分的命题。

表2.1介绍了五种主要的命题联结词。

表2.1　五种主要的命题联结词

命题联结词	表示形式	意义
与（and）	$p \wedge q$	命题合取（conjunction），即“p且q”
或（or）	$p \vee q$	命题析取（disjunction），即“p或q”
非（not）	$\neg p$	命题否定（negation），即“非p”
条件（conditional）	$p \rightarrow q$	命题蕴含（implication），p称为前件，q称为后件，即“如果p，则q”
双向条件（bi-conditional）	$p \leftrightarrow q$	命题双向蕴含（bi-implication），即“p当且仅当q”

例2.2　以下是几个复合命题的例子。

（1）如果今天天气晴朗，三年级二班同学就去春游。

（2）如果计算机机房停电，那么今天的实验将无法进行。

（3）艾伦·图灵不仅是一名数学家，而且是一名逻辑学家。

复合命题的真假可通过真值表来计算，表2.2介绍了五种主要联结词构成的复合命题的真值表。

表2.2 五种主要联结词构成的复合命题的真值表

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$
F	F	T	F	F	T	T
F	T	T	F	T	T	F
T	F	F	F	T	F	F
T	T	F	T	T	T	T

值得注意的是，条件命题联结词中前件为假时，无论后件取值如何，复合命题均为真。“如果p，则q（$p \rightarrow q$）”定义的是一种蕴含关系，也就是命题q包含命题p（p是q的子集）。p不成立相当于p是一个空集，而空集是任何集合的子集。因此，当p不成立时，“如果p，则q（$p \rightarrow q$）”恒为真。

定义2.4 逻辑等价：给定命题p和命题q，如果p和q在所有情况下都具有相同的真假结果，那么p和q在逻辑上等价，一般用≡来表示，即$p \equiv q$。

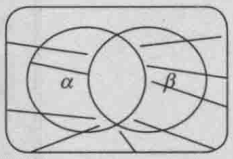
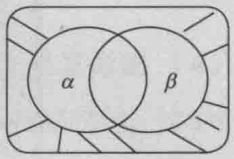
逻辑等价为命题进行形式转换带来了可能，基于这些转换不再需要逐一列出p和q的真值表来判断两者是否在逻辑上等价，而是可直接根据已有逻辑等价公式来判断p和q在逻辑上是否等价。

表2.3和表2.4分别给出了一些逻辑等价的例子及其解释。

表2.3 一些逻辑等价的例子

$\alpha \wedge \beta \equiv \beta \wedge \alpha$（$\wedge$的交换律）	$(\alpha \rightarrow \beta) \equiv \neg \alpha \vee \beta$（蕴涵消除）
$\alpha \vee \beta \equiv \beta \vee \alpha$（$\vee$的交换律）	$(\alpha \leftrightarrow \beta) \equiv (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$（双向消除）
$(\alpha \wedge \beta) \wedge \gamma \equiv \alpha \wedge (\beta \wedge \gamma)$（$\wedge$的结合律）	$\neg(\alpha \wedge \beta) \equiv (\neg \alpha \vee \neg \beta)$（De Morgan定律）
$(\alpha \vee \beta) \vee \gamma \equiv \alpha \vee (\beta \wedge \gamma)$（$\vee$的结合律）	$\neg(\alpha \vee \beta) \equiv (\neg \alpha \wedge \neg \beta)$（De Morgan定律）
$\neg(\neg \alpha) \equiv \alpha$（双重否定）	$(\alpha \wedge (\beta \vee \gamma)) \equiv (\alpha \wedge \beta) \vee (\alpha \wedge \gamma)$（$\wedge$对$\vee$的分配律）
$(\alpha \rightarrow \beta) \equiv \neg \beta \rightarrow \neg \alpha$（逆否命题）	$(\alpha \vee (\beta \wedge \gamma)) \equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$（$\vee$对$\wedge$的分配律）

表2.4 若干逻辑等价的解释

$(\alpha \to \beta) \equiv \neg\beta \to \neg\alpha$（逆否命题）	秋天天气变凉 → 大雁南飞越冬≡大雁没有南飞越冬 → 秋天天气没有变凉
$(\alpha \to \beta) \equiv \neg\alpha \vee \beta$（蕴涵消除）	α 为假，则命题恒为真；α 为真，命题与 β 真值相同
$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$（De Morgan定律）	
$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$（De Morgan定律）	

定义了逻辑关系之后，可以基于逻辑关系来进行推理。推理是按照某种策略从前提出发推出结论的过程。在本书中用符号 $\Rightarrow$ 来表示推理过程，$\Rightarrow$ 的左侧表示推理的前提，$\Rightarrow$ 的右侧表示推理的结论。以下是一些命题逻辑中常见的推理规则。

- 假言推理（modus ponens）：$\alpha \to \beta$，$\alpha \Rightarrow \beta$；
- 与消解（and-elimination）：$\alpha_1 \wedge \alpha_2 \wedge \cdots \wedge \alpha_n \Rightarrow \alpha_i$（$1 \leqslant i \leqslant n$）；
- 与导入（and-introduction）：$\alpha_1, \alpha_2, \cdots, \alpha_n \Rightarrow \alpha_1 \wedge \alpha_2 \wedge \cdots \wedge \alpha_n$；
- 双重否定（double-negation elimination）：$\neg\neg\alpha \Rightarrow \alpha$；
- 单项消解或单项归结（unit resolution）：$\alpha \vee \beta, \neg\beta \Rightarrow \alpha$；
- 消解或归结（resolution）：$\alpha \vee \beta, \neg\beta \vee \gamma \Rightarrow \alpha \vee \gamma$。

例2.3 已知以下命题成立，请应用归结法证明命题 γ 是成立的。

（a）$\alpha \vee \beta$；

（b）$\alpha \to \gamma$；

（c）$\beta \to \gamma$。

解：应用归结法如下。

（1）$\alpha \vee \beta$（已知）；

（2）$\neg\alpha \vee \gamma$（（b）进行蕴涵消除）；

（3）$\neg\beta \vee \gamma$（（c）进行蕴涵消除）；

（4）$(\neg\alpha \vee \gamma) \wedge (\neg\beta \vee \gamma)$（由（2）和（3））；

（5）$(\neg\alpha \wedge \neg\beta) \vee \gamma$（（4）使用分配律）；

（6）$\neg(\alpha \vee \beta) \vee \gamma$（（5）使用De Morgan定律）；

（7）γ（（1）和（6）进行归结）。

上面这个结论也可将归结法和反证法结合起来，通过如下步骤进行证明。

解：应用归结法和反证法如下。

（1）$\alpha \vee \beta$（已知）；

（2）$\neg\alpha \vee \gamma$（（b）进行蕴涵消除）；

（3）$\neg\beta \vee \gamma$（（c）进行蕴涵消除）；

（4）$\neg\gamma$（假设命题γ不成立）；

（5）$\beta \vee \gamma$（（1）和（2）进行归结）；

（6）$\neg\alpha$（（2）和（4）进行单项归结）；

（7）$\neg\beta$（（3）和（4）进行单项归结）；

（8）γ（（5）和（7）进行单项归结）；

（9）（4）和（8）矛盾，假设不成立，因此命题γ成立。

例2.4　应用归结法证明以下命题集是不可满足的。

（a）$\alpha \vee \beta$；

（b）$\neg\alpha \vee \beta$；

（c）$\alpha \vee \neg\beta$；

（d）$\neg\alpha \vee \neg\beta$。

解：应用归结法如下。

（1）$\alpha \vee \beta$（已知）；

（2）$\neg\alpha \vee \beta$（已知）；

（3）$\alpha \vee \neg\beta$（已知）；

（4）$\neg\alpha \vee \neg\beta$（已知）；

（5）β（（1）和（2）进行归结）；

（6）$\neg\beta$（（3）和（4）进行归结）；

（7）（5）和（6）矛盾，因此原命题集是不可满足的。

例2.5　应用归结法证明以下命题集是不可满足的。

（a）$\alpha \vee \gamma$；

（b）$\neg\beta\vee\gamma$；

（c）$\neg\gamma\vee\alpha$；

（d）$\neg\alpha\vee\beta$；

（e）$\neg\alpha\vee\neg\gamma$。

解：应用归结法如下：

（1）$\alpha\vee\gamma$（已知）；

（2）$\neg\beta\vee\gamma$（已知）；

（3）$\neg\gamma\vee\alpha$（已知）；

（4）$\neg\alpha\vee\beta$（已知）；

（5）$\neg\alpha\vee\neg\gamma$（已知）；

（6）α（（1）和（3）进行归结）；

（7）β（（4）和（6）进行单项归结）；

（8）γ（（2）和（7）进行单项归结）；

（9）$\neg\alpha$（（5）和（8）进行单项归结）；

（10）（6）和（9）矛盾，因此原命题集是不可满足的。

范式（normal form）是命题逻辑中的重要概念。范式是把命题公式化归为一种标准的形式，范式最大的作用是可以进行两个命题的等价判定。

定义2.5　析取范式（disjunctive normal form，DNF）：有限个简单合取式构成的析取式称为析取范式。

定义2.6　合取范式（conjunctive normal form，CNF）：有限个简单析取式构成的合取式称为合取范式。

析取范式和合取范式统称为范式。假设$\alpha_i(i=1,2,\cdots,k)$为简单的合取式，则$\alpha=\alpha_1\vee\alpha_2\vee\cdots\vee\alpha_k$为析取范式，例如：$(\neg\alpha_1\wedge\alpha_2)\vee\alpha_3$，$\neg\alpha_1\vee\alpha_3\vee\alpha_2$等。假设$\alpha_i(i=1,2,\cdots,k)$为简单的析取式，则$\alpha=\alpha_1\wedge\alpha_2\wedge\cdots\wedge\alpha_k$为合取范式，例如：$(\alpha_1\vee\alpha_2)\wedge\neg a_3$，$\neg\alpha_1\wedge\alpha_3\wedge(\neg\alpha_2\vee\alpha_4)$等。

一个析取范式是不成立的，当且仅当它的每个简单合取式都不成立。一个合取范式是成立的，当且仅当它的每个简单析取式都是成立的。任意命题公式都存在着与之等值的析取范式与合取范式。命题公式的析取范式与合取范式都不是唯一的。

例2.6　求 $\neg(\alpha \rightarrow \beta) \vee \neg\gamma$ 的析取范式与合取范式。

解：$\neg(\alpha \rightarrow \beta) \vee \neg\gamma$

$\Leftrightarrow \neg(\neg\alpha \vee \beta) \vee \neg\gamma$

$\Leftrightarrow (\alpha \wedge \neg\beta) \vee \neg\gamma$（析取范式）

$\Leftrightarrow (\alpha \vee \neg\gamma) \wedge (\neg\beta \vee \neg\gamma)$（合取范式）

2.2 谓词逻辑

命题逻辑是数理逻辑的一部分。在命题逻辑中，原子命题是最基本的单位。由原子命题出发，通过使用命题联结词，构成复合命题。命题逻辑只能把复合命题分解为简单命题（即原子命题），无法对原子命题所包含的丰富语义（如个体、部分或全体等）进行刻画。因此，命题逻辑无法表达局部与整体、一般与个别的关系。

例2.7　以下是著名的苏格拉底三段论。

α：所有的人都是要死的。

β：苏格拉底是人。

γ：所以苏格拉底是要死的。

可以看到，虽然苏格拉底论断是正确的，但无法通过命题逻辑来进行推理判断（$\alpha \wedge \beta \rightarrow \gamma$）。其中的原因是上述的原子命题蕴涵了个体（苏格拉底）、群体（所有的人）和关系（都是要死的）等内在丰富语义，命题逻辑无法表现这种内在丰富语义。因此，需要引入更加强大的逻辑表示方法，分离其主语（个体或群体）和谓语（关系），这就是谓词逻辑。

在谓词逻辑中，将原子命题进一步细化，分解出个体、谓词和量词，来表达个体与总体的内在联系和数量关系，这就是谓词逻辑的研究内容。

定义2.7　个体：个体是指所研究领域中可以独立存在的具体或抽象的概念。

其中，具体或特定的个体称为个体常量，用于表示个体常量的符号称为常量符号，如“张三”“李四”“1”“2”“3”；抽象或泛指的个体称

为个体变量，用于表示个体变量的符号称为变量符号，通常用小写字母表示，如x、y、z。所有个体对应的集合称为个体域，个体域可以是有穷集合，如{“硅晶体”，“计算机”，“类脑芯片”，“无人车”，“程序员”，“深度学习”}，个体域也可以是无穷的集合，如实数集合。

定义2.8　谓词：谓词是用来刻画个体属性或者描述个体之间关系存在性的元素，其值为真或为假。

其中，包含一个参数的谓词称为一元谓词，表示一元关系，通常用于刻画个体是否包含特定的属性，如$p(x)$：x是质数，表示某个数是否是质数；包含多个参数的谓词称为多元谓词，用于表示个体之间的多元关系，通常用于描述个体之间是否存在特定的关联，如$Father(x, y)$表示x是y的父亲。

需要注意的是函数与谓词的区别，函数中个体变量用个体常量（来自定义域）代入后结果仍是个体（值域），如定义函数$f(x) = x + 10$，则$f(2) = 12$；谓词中个体变量用个体常量带入后就变成了命题，如定义谓词$car(x)$：x是车，这个谓词中x用吉普车代替，则car（吉普车）是命题。函数是从定义域到值域的映射；谓词是从定义域到$\{True, False\}$的映射。

例2.8　将下列客观事实符号化。

（1）Richard是国王。

（2）Lucy和Lily是姐妹。

（3）北京是中国的首都。

解：（1）*King*(Richard)。其中，Richard是一个个体常量，*King*是一个描述“国王”这个一元关系的谓词。

（2）*Sister*(Lucy, Lily)。其中，Lucy和Lily是两个个体常量，*Sister*是一个描述“姐妹”这个二元关系的谓词。

（3）*Capital*(北京, 中国)。其中，北京和中国是两个个体常量，*Capital*是一个描述“首都”这个二元关系的谓词。

虽然，使用个体和谓词的组合可以描述简单的客观事实，但在某些复杂的情况下，需要对某一对象集合的共同属性进行描述。但是，对每个对象的属性特征进行一一列举太过冗余，因此，量词的引入可以有效地解决这一问题。

定义2.9　全称量词：全称量词表示一切的、所有的、凡是、每一个等，用符号∀表示。

定义2.10　存在量词：存在量词表示存在、有一个、某些等，用符号∃表示。

$\forall x$表示定义域中的所有个体，$(\forall x)P(x)$表示定义域中的所有个体具有性质P；$\exists x$表示定义域中存在一个或若干个个体，$(\exists x)P(x)$表示定义域中存在一个个体或若干个体具有性质P。如谓词$P(x)$：x表示能够制造工具，则$(\forall x)P(x)$表示定义域中的所有个体都能够制造工具，$(\exists x)P(x)$表示定义域中的某个/某些个体能够制造工具，P(小王)表示小王能够制造工具（该命题或者为真、或者为假）。

全称量词和存在量词统称为量词。只包含个体谓词和个体量词的谓词逻辑称为一阶谓词逻辑。一阶谓词逻辑无法对谓词和量词进行量化，需要高阶谓词和高阶量词来对其进行量化，包含高阶谓词和高阶量词的谓词逻辑称为高阶谓词逻辑。在本书中，只讨论一阶谓词逻辑。

定理2.1　全称量词的描述形式可以用相应的存在量词的描述形式替换，即任意用全称量词描述的事实都存在对应的用存在量词描述的形式。

设$P(x)$是谓词，则全称量词和存在量词存在如下关系：

（1）$(\forall x)P(x)\equiv\neg(\exists x)\neg P(x)$

（2）$(\forall x)\neg P(x)\equiv\neg(\exists x)P(x)$

（3）$\neg(\forall x)P(x)\equiv(\exists x)\neg P(x)$

（4）$\neg(\forall x)\neg P(x)\equiv(\exists x)P(x)$

定义2.11　约束变元：在全称量词或存在量词的约束条件下的变量符号称为约束变元。

定义2.12　自由变元：不受全称量词或存在量词约束的变量符号称为自由变元。

定理2.2　自由变元既可以存在于量词的约束范围之内，也可以存在于量词的约束范围之外。

设$A(x)$是包含变元x的谓词公式，B是不包含变元x的谓词公式，则如下逻辑等价关系成立：

（1）$(\forall x)(A(x)\vee B)\equiv(\forall x)A(x)\vee B$

（2）$(\forall x)(A(x)\wedge B)\equiv(\forall x)A(x)\wedge B$

（3）$(\exists x)(A(x)\vee B)\equiv(\exists x)A(x)\vee B$

（4）$(\exists x)(A(x)\wedge B)\equiv(\exists x)A(x)\wedge B$

定理2.3 在约束变元相同的情况下，量词的运算满足分配律。

设$A(x)$和$B(x)$是包含变元x的谓词公式，则如下逻辑等价关系成立：

（1）$(\forall x)(A(x)\vee B(x))\equiv(\forall x)A(x)\vee(\forall x)B(x)$

（2）$(\forall x)(A(x)\wedge B(x))\equiv(\forall x)A(x)\wedge(\forall x)B(x)$

（3）$(\exists x)(A(x)\vee B(x))\equiv(\exists x)A(x)\vee(\exists x)B(x)$

（4）$(\exists x)(A(x)\wedge B(x))\equiv(\exists x)A(x)\wedge(\exists x)B(x)$

例2.9 使用全称量词或存在量词描述下列事实。

（1）所有的国王都是人。

（2）所有的国王都头戴皇冠。

解：（1）“所有的国王都是人”表示的含义为“对于所有的x，如果x是国王，那么x是人”，其符号化表示为$(\forall x)(King(x)\rightarrow Person(x))$。其中$x$是变量符号，由于$x$受到全称量词的约束，因此$x$是约束变元；$King(x)$是一个一元谓词，表示$x$是国王，$Person(x)$是一个一元谓词，表示$x$是人。

（2）“所有的国王都头戴皇冠”表示的含义为“对于所有的x，如果x是国王，那么x头戴皇冠”，符号化表示为$(\forall x)(King(x)\rightarrow Head_On(\text{Crown}, x))$。其中$x$是变量符号，由于$x$受到全称量词的约束，因此$x$是约束变元；Crown是一个常量符号，表示皇冠；$King(x)$是一个一元谓词，表示$x$是国王，$Head_On(\text{Crown}, x)$是一个二元谓词，表示$x$头戴皇冠。

定理2.4 当公式中存在多个量词时，若多个量词都是全称量词或者都是存在量词，则量词的位置可以互换；若多个量词中既有全称量词又有存在量词，则量词的位置不可以随意互换。

设$A(x, y)$是包含变元x, y的谓词公式，则如下关系成立：

（1）$(\forall x)(\forall y)A(x, y)\Leftrightarrow(\forall y)(\forall x)A(x, y)$

（2）$(\exists x)(\exists y)A(x, y)\Leftrightarrow(\exists y)(\exists x)A(x, y)$

（3）$(\forall x)(\forall y)A(x, y)\Rightarrow(\exists y)(\forall x)A(x, y)$

（4）$(\forall x)(\forall y)A(x, y)\Rightarrow(\exists x)(\forall y)A(x, y)$

（5）$(\exists y)(\forall x)A(x, y)\Leftrightarrow(\forall x)(\exists y)A(x, y)$

（6）$(\exists x)(\forall y)A(x, y)\Leftrightarrow(\forall y)(\exists x)A(x, y)$

（7）$(\forall x)(\exists y)A(x, y)\Rightarrow(\exists y)(\exists x)A(x, y)$

（8）$(\forall y)(\exists x)A(x, y)\Rightarrow(\exists x)(\exists y)A(x, y)$

定义2.13　项：项是描述对象的逻辑表达式，被递归地定义为：

（1）常量符号和变量符号是项；

（2）若$f(x_1, x_2,\cdots, x_n)$是n元函数符号，$t_1, t_2,\cdots, t_n$是项，则$f(t_1, t_2,\cdots, t_n)$是项；

（3）有限次数地使用上述规则产生的符号串是项。

定义2.14　原子谓词公式：若$P(x_1, x_2, \cdots, x_n)$是n元谓词，$t_1, t_2, \cdots, t_n$是项，则称$P(t_1, t_2, \cdots, t_n)$为原子谓词公式，简称原子公式。

定义2.15　合式公式：合式公式是由逻辑联结词和原子公式构成的用于陈述事实的复杂语句，又称谓词公式，由以下规则定义：

（1）命题常项、命题变项、原子谓词公式是合式公式；

（2）如果A是合式公式，则$\neg A$也是合式公式；

（3）如果A和B是合式公式，则$A\wedge B$、$A\vee B$、$A\rightarrow B$、$B\rightarrow A$、$A\leftrightarrow B$都是合式公式；

（4）如果A是合式公式，x是个体变项，则$(\exists x)\mathrm{A}(x)$和$(\forall x)\mathrm{A}(x)$也是合式公式；

（5）有限次数地使用上述规则构成的表达式是合式公式。

例2.10　使用合式公式描述下列事实。

（1）Tom不仅喜欢踢足球，还喜欢打篮球；

（2）Tom的所有同学都喜欢他；

（3）不是所有的男生都喜欢打篮球；

（4）一个人是华侨，当且仅当他在国外定居并且具有中国国籍；

（5）男生都爱看世界杯。

解：（1）$Like(\mathrm{Tom}, \mathrm{Football})\wedge Like(\mathrm{Tom}, \mathrm{Basketball})$。其中，$Like(\mathrm{Tom}, x)$表示Tom喜欢$x$，Football和Basketball分别表示踢足球和打篮球。

（2）$(\forall x)(Classmate(\mathrm{Tom}, x)\rightarrow Like(x, \mathrm{Tom}))$。其中，$Classmate(\mathrm{Tom}, x)$表示$x$是Tom的同学，$Like(x, \mathrm{Tom})$表示$x$喜欢Tom。

（3）$\neg(\forall x)(Boy(x)\rightarrow Like(x, \mathrm{Basketball}))$。其中，$Boy(x)$表示$x$

是男生，$Like(x,$ Basketball$)$表示x喜欢打篮球。

（4）$(\forall x)(Overseas_Chinese(x)\leftrightarrow(Overseas(x)\wedge Chinese(x)))$。其中，$Overseas_Chinese(x)$表示$x$是华侨，$Overseas(x)$表示$x$在国外定居，$Chinese(x)$表示$x$具有中国国籍。

（5）$(\forall x)(Boy(x)\rightarrow Like(x,$ WorldCup$))$。其中，$Boy(x)$表示x是男生，$Like(x,$ WorldCup$)$表示x喜欢看世界杯。

可以利用谓词逻辑来进行推理。设$A(x)$是谓词公式，x, y是变元，a是常量符号，以下是一些谓词逻辑中的推理规则。

（1）全称量词消去（universal instantiation，UI）：$(\forall x)A(x)\Rightarrow A(y)$。

（2）全称量词引入（universal generalization，UG）：$A(y)\Rightarrow(\forall x)A(x)$。

（3）存在量词消去（existential instantiation，EI）：$(\exists x)A(x)\Rightarrow A(a)$。

（4）存在量词引入（existential generalization，EG）：$A(a)\Rightarrow(\exists x)A(x)$。

例2.11　已知$(\forall x)(P(x)\rightarrow Q(x))$，$(\forall x)(Q(x)\rightarrow R(x))$，试证明$(\forall x)(P(x)\rightarrow R(x))$。

解：（1）$(\forall x)(P(x)\rightarrow Q(x))$；

（2）$P(x)\rightarrow Q(x)$（全称量词消去）；

（3）$(\forall x)(Q(x)\rightarrow R(x))$；

（4）$Q(x)\rightarrow R(x)$（全称量词消去）；

（5）$P(x)\rightarrow R(x)$（由2和4知）；

（6）$(\forall x)(P(x)\rightarrow R(x))$（全称量词引入）。

例2.12　已知$(\forall x)(F(x)\rightarrow(G(x)\wedge H(x)))$，$(\exists x)(F(x)\wedge P(x))$，试证明$(\exists x)(P(x)\wedge H(x))$。

解：（1）$(\forall x)(F(x)\rightarrow(G(x)\wedge H(x)))$；

（2）$(\exists x)(F(x)\wedge P(x))$；

（3）$F(a)\wedge P(a)$（2的存在量词消去）；

（4）$F(a)\rightarrow(G(a)\wedge H(a))$（1的全称量词消去）；

（5）$F(a)$（由3知）；

（6）$G(a)\wedge H(a)$（由4和5的假言推理）；

（7）$P(a)$（由3知）；

（8）$H(a)$（由6知）；

（9）$P(a)\wedge H(a)$（7和8的合取）；

（10）$(\exists x)(P(x)\wedge H(x))$（9的存在量词引入）。

为了对自然语言所描述的内容进行精确表达，可通过逻辑语言对自然语言进行形式化。

例2.13　证明苏格拉底三段论“所有人都是要死的，苏格拉底是人，所以苏格拉底是要死的”。

解：设$F(x)$：x是人；$G(x)$：x是要死的人；a是苏格拉底。

前提：$(\forall x)(F(x)\rightarrow G(x))$；$F(a)$。

结论：$G(a)$。

证明：（1）$(\forall x)(F(x)\rightarrow G(x))$；

（2）$F(a)\rightarrow G(a)$（1的全称量词消去）；

（3）$F(a)$；

（4）$G(a)$（2和3的假言推理）。

例2.14　已知每架飞机或者停在地面或者飞在天空，且并非每架飞机都飞在天空，请证明：有些飞机停在地面。

解：设$plane(x)$：x是飞机；$on_ground(x)$：x停在地面；$in_sky(x)$：x飞在天空。

前提：$(\forall x)(plane(x)\rightarrow on_ground(x)\vee in_sky(x))$，$\neg(\forall x)(plane(x)\rightarrow in_sky(x))$。

结论：$(\exists x)(plane(x)\wedge on_ground(x))$。

证明：（1）$\neg(\forall x)(plane(x)\rightarrow in_sky(x))$；

（2）$(\exists x)\neg(plane(x)\rightarrow in_sky(x))$（定理2.1）；

（3）$(\exists x)\neg(\neg plane(x)\vee in_sky(x))$（蕴含消除）；

（4）$(\exists x)(plane(x)\wedge\neg in_sky(x))$（De Morgan定律）；

（5）$plane(a)\wedge\neg in_sky(a)$（存在量词消去）；

（6）$plane(a)$（由5知）；

（7）$\neg in_sky(a)$（由5知）；

（8）$(\forall x)(plane(x) \rightarrow on_ground(x) \vee in_sky(x))$；

（9）$plane(a) \rightarrow on_ground(a) \vee in_sky(a)$（全称量词消去）；

（10）$on_ground(a) \vee in_sky(a)$（6和9的假言推理）；

（11）$on_ground(a)$（由7和10知）；

（12）$plane(a) \wedge on_ground(a)$（由6和11知）；

（13）$(\exists x)\ (plane(x) \wedge on_ground(x))$（存在量词引入）。

有了逻辑推理这个工具，可以对知识进行表示和推理，从而构建这样一个系统，该系统可以模拟领域专家才能解决的复杂问题，这就是早期人工智能的一个重要分支——专家系统（expert system）。一般来说，专家系统由知识库和推理机组成。先对某个领域的自然语言、文本语句等信息进行逻辑化表示形成知识库，在给定新的问题时，对该问题也进行逻辑化表示，并将该问题表示输入到推理机中，推理机根据已有的知识库和问题表示进行推理，从而输出答案。比较著名的专家系统有第一个商用专家系统EXSYS、智能语音专家系统Siri、智能诊断系统MYCIN等。

2.3 知识图谱推理

知识图谱（knowledge graph）由有向图（directed graph）构成，被用来描述现实世界中实体及实体之间的关系，是人工智能中进行知识表达的重要方式。在知识图谱中，每个结点表示客观世界中的一个实体，两个结点之间的连线表示结点具有某一关系。知识图谱中存在连线的两个实体可表达为形如<left_node，relation，right_node>的三元组形式，这种三元组也可以表示为一阶逻辑（first order logic，FOL）的形式，从而为基于知识图谱的推理创造了条件。

关系推理是统计关系学习研究的基本问题，也是当前知识图谱领域研究的热点问题。对实体之间存在的关系进行推理，能够从现有知识中发现新的知识，在实体间建立新关联，从而扩充和丰富现有知识库[7]。例如，从<奥巴马，出生地，夏威夷>和<夏威夷，属于，美国>两个

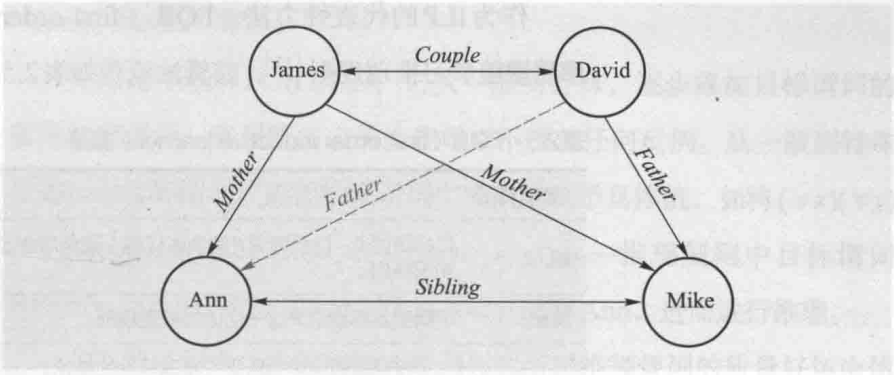


图2.1 一个简单的家庭关系知识图谱

三元组，可推理得到＜奥巴马，国籍，美国＞。

图2.1所示为一个简单的家庭关系知识图谱。其中，结点代表实体，即家庭成员，边代表家庭成员之间的关系。可利用一阶谓词来表达刻画知识图谱中结点之间存在的关系，如图2.1中形如 <James, *Couple*, David> 的关系可用一阶逻辑的形式来描述，即*Couple*(James, David)。其中，*Couple*(*x*, *y*)是一阶谓词，*Couple*是图2.1中实体之间具有的关系，*x*和*y*是谓词变量，可由知识图谱中的实体将其实例化。很显然，从图2.1已有关系可推知David和Ann具有父女关系，但这一关系在图2.1的初始图（无虚线）中并不存在，是需要推理的目标。

如果定义如下推理规则：$(\forall x)(\forall y)(\forall z)(Mother(z, y) \wedge Couple(x, z) \rightarrow Father(x, y))$，就可在现有知识*Mother*（James, Ann）和*Couple*（David, James）基础上进行推理，得到新的知识*Father*（David, Ann），即补充了David和Ann的关系，从而对原来的知识图谱进行了完善，扩展了实体之间的关系。

本节将介绍知识图谱推理中具有代表性的两个方法：归纳逻辑程序设计（inductive logic programming，ILP）和路径排序算法（path ranking algorithm，PRA）[5]。

2.3.1 FOIL归纳推理

归纳逻辑程序设计（ILP）是机器学习和逻辑程序设计交叉领域的研究内容。Gordon Plotkin和Ehud Shapiro推动了在形式逻辑框架下进行归纳机器学习，为ILP研究打下了理论基础[8, 16]。ILP使用一阶谓词逻辑进行知识表示，通过修改和扩充逻辑表达式对现有知识归纳，完成推理任务。

作为ILP的代表性方法，FOIL（first order inductive learner）通过**序贯覆盖**学习推理规则[13]，其算法流程如表2.5所示。

表2.5 FOIL（first order inductive learner）算法

FOIL算法	
输入：	目标谓词P、目标谓词P的训练样例（正例集合E^+和反例集合E^-）以及其他背景知识样例。
输出：	可得到目标谓词P这一结论的推理规则。
	（1）将目标谓词作为所学习推理规则的结论。 （2）将其他谓词逐一作为前提约束谓词加入推理规则，计算所得到推理规则的FOIL信息增益值，选取可带来最大信息增益值的前提约束谓词加入原来的推理规则，得到新的推理规则，并将训练样例集合中与该推理规则不符的样例去掉。 （3）重复2过程，直到所得到的推理规则不覆盖任何反例。

FOIL算法需要给定目标谓词P，在图2.1中，假设目标谓词是*Father*(*x*, *y*)，*Father*在知识图谱中表达了两个家庭成员之间的关系，x和y在分别取值David和Mike时，*Father*(*x*, *y*)为真。

在FOIL中，目标谓词是需要推断规则的结论，也称为规则头。在给定推理结论后，FOIL算法学习得到使得结论满足的前提条件，即目标谓词作为结论的推理规则。

为了学习推理规则，需要构造目标谓词P的训练样例，训练样例包含正例集合E^+和反例集合E^-。当目标谓词P为*Father*(*x*, *y*)时，从图2.1中知道目标谓词P只有一个正例*Father*（David, Mike）。反例在知识图谱中一般不会显式给出，但可从知识图谱中构造出来。例如，从知识图谱中已经知道*Couple*（David, James）成立，则*Father*（David, James）可作为目标谓词P的一个反例，记为¬*Father*（David, James）。需要注意的是，只有在已知两个实体之间存在的关系且确定这一关系与目标谓词相悖时，才能将这两个实体用于构建目标谓词的反例（从而保证所构造的反例与该两个实体之间已知关系是相悖的），而不能在不知两个实体是否满足目标谓词前提下将它们用于构造目标谓词的反例。例如，不能用David和Ann来构建目标谓词反例，因为从现有知识中不能确定这两个实体之间是否存在*Father*这一关系，即*Father*（David, Ann）有可能是正例，而在本例中两者恰恰存在目标谓词所指定的关系，虽然David和Ann所具有的*Father*这一关系在知识图谱中暂时未能显式刻画。

此外在推理中还需要其他背景知识，这些背景知识来自知识图谱中目标谓词以外的其他谓词实例化结果，如*Sibling*（Ann, Mike）和*Couple*

（David, James）等。

可以这样理解FOIL算法：它从一般到特殊，逐步添加目标谓词的前提约束谓词，直到所构成的推理规则不覆盖任何反例。从一般到特殊指的是对目标谓词或前提约束谓词中的变量赋予具体值，如将$(\forall x)(\forall y)(\forall z)(Mother(z, y) \wedge Couple(x, z) \rightarrow Father(x, y))$这一推理规则中目标谓词$Father(x, y)$所包含的$x$和$y$分别赋值为David和Ann，进而进行推理。

在推理过程中，添加前提约束谓词后所得推理规则的质量好坏由信息增益值（information gain）这一评估准则来判断。在FOIL中，信息增益值计算方法如下：

$$FOIL_Gain = \widehat{m_+} \cdot \left(\log_2 \frac{\widehat{m_+}}{\widehat{m_+} + \widehat{m_-}} - \log_2 \frac{m_+}{m_+ + m_-} \right)$$

其中，$\widehat{m_+}$和$\widehat{m_-}$是增加前提约束谓词后所得新推理规则能够覆盖的正例和反例数目，m_+和m_-是原推理规则所覆盖正例和反例的数目。

例2.15　从图2.1给出的知识图谱通过FOIL算法学习得到如下规则：$(\forall x)(\forall y)(\forall z)(Mother(z, y) \wedge Couple(x, z) \rightarrow Father(x, y))$

在给定目标谓词$Father(x, y)$之后，可如表2.6所示来构造背景知识样例和训练样例。

表2.6　目标谓词$Father(x, y)$对应的样例

背景知识样例集合	*Sibling*(Ann, Mike) *Couple*(David, James) *Mother*(James, Ann) *Mother*(James, Mike)	目标谓词训练样例集合	*Father*(David, Mike) ¬*Father*(David, James) ¬*Father*(James, Ann) ¬*Father*(James, Mike) ¬*Father*(Ann, Mike)

按照表2.5中FOIL算法所列步骤依次将每个候选前提约束谓词加入到推理规则中，并计算所得新的推理规则对应的FOIL增益值。基于计算所得FOIL增益值来确定最佳前提约束谓词。表2.7给出了添加前提约束谓词后信息增益的计算结果。

给定目标谓词$Father(x, y)$，此时推理规则只有目标谓词，因此推理规则所覆盖正例和反例样本数分别是训练样本中正例和反例的数目，即1和4，因此$m_+ = 1$和$m_- = 4$。

表2.7 添加前提约束谓词所得信息增益值计算过程

推理规则		推理规则涵盖的正例和反例数		FOIL信息增益值
目标谓词	前提约束谓词	正例	反例	信息增益值
Father(*x*, *y*)←	空集	$m_+ = 1$	$m_- = 4$	/
	Mother(*x*, *y*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 2$	NA
	Mother(*x*, *z*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 2$	NA
	Mother(*y*, *x*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	Mother(*y*, *z*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 1$	NA
	Mother(*z*, *x*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 1$	NA
	Mother(*z*, *y*)	$\widehat{m}_+ = 1$	$\widehat{m}_- = 3$	0.32
	Sibling(*x*, *y*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 1$	NA
	Sibling(*x*, *z*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 1$	NA
	Sibling(*y*, *x*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
Father(*x*, *y*)←	*Sibling*(*y*, *z*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 1$	NA
	Sibling(*z*, *x*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	Sibling(*z*, *y*)	$\widehat{m}_+ = 1$	$\widehat{m}_- = 2$	0.74
	Couple(*x*, *y*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 1$	NA
	Couple(***x***, ***z***)	$\widehat{\boldsymbol{m}}_+ = \mathbf{1}$	$\widehat{\boldsymbol{m}}_- = \mathbf{1}$	**1.32**
	Couple(*y*, *x*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	Couple(*y*, *z*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	Couple(*z*, *x*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 2$	NA
	Couple(*z*, *y*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 1$	NA

如果将*Mother*(*x*, *y*)作为前提约束谓词，加入推理规则，可得到*Mother*(*x*, *y*)→*Father*(*x*, *y*)。在表2.6所提供背景知识样例集合中，*Mother*(*x*, *y*)有两个实例，即*Mother*（James, Ann）和*Mother*（James, Mike）。对于*Mother*（James, Ann）这一实例，*x* = James和*y* = Ann，将*x*和*y*代入*Father*(*x*, *y*)得到*Father*（James, Ann），从表2.6所提供的训练样本集合知道*Father*（James, Ann）是一个反例；对于*Mother*（James, Mike）这一实例，*x* = James，*y* = Mike，将*x*和*y*代入*Father*(*x*, *y*)得到*Father*（James, Mike），从表2.6所提供的训练样本集合可知*Father*（James, Mike）是一个反例。因此推理规则*Mother*(*x*, *y*)→*Father*(*x*, *y*)所覆盖的正例和反例数量分别为0和2，即$\widehat{m}_+ = 0$，$\widehat{m}_- = 2$。由于$\widehat{m}_+ = 0$，代入*FOIL_Gain*公式时会出现负无穷的情况，此时*FOIL_Gain*记为NA（Not Available）。

类似地，可将背景知识样例中出现的所有谓词（*Mother*(*,*)、

Sibling(*, *)和*Couple*(*, *)）依次作为前提约束加入推理规则，*代表所有可能的变元，如上计算得到推理规则的*FOIL_Gain*取值。

如果将*Couple*(*x*, *z*)作为前提约束谓词，加入推理规则，可得到*Couple*(*x*, *z*)→*Father*(*x*, *y*)推理形式。在背景知识样例中，*Couple*(*x*, *z*)只有一个实例*Couple*(David, James)，即x = David和z = James，将其代入*Father*(*x*, *y*)得到*Father*(David, *y*)。在训练样本中存在正例*Father*(David, Mike）以及反例¬*Father*（David, James），因此*Couple*(*x*, *z*)→*Father*(*x*, *y*)所覆盖的正例和反例数目均为1，$FOIL_Gain = \widehat{m}_+ \cdot \left(\log_2 \frac{\widehat{m}_+}{\widehat{m}_+ + \widehat{m}_-} - \log_2 \frac{m_+}{m_+ + m_-} \right) = 1 \cdot \left(\log_2 \frac{1}{1+1} - \log_2 \frac{1}{1+4} \right) = 1.32$。

一旦将*Couple*(*x*, *z*)作为前提约束谓词加入，所得新推理规则对应的*FOIL_Gain*取值最大，因此选择*Couple*(*x*, *z*)加入推理规则，得到*Couple*(*x*, *z*)→*Father*(*x*, *y*)这一新的推理规则，并将训练样例集合中与该推理规则不符样例去掉。此时，训练样本集合中就只有正例*Father*(David, Mike)和反例¬*Father*(David, James)两个实例了。注意，这里不符的含义是当*Couple*(*x*, *z*)中x取值为David时，与*Father*(David, *)或¬*Father*(David, *)无法匹配的实例。

接着，再用相同方法继续将其他谓词逐一作为前提约束谓词加入推理规则进行考察，用FOIL增益值来判断选取最优推理规则，如表2.8所示。

表2.8　继续添加前提约束谓词所得信息增益值计算过程

推理规则		推理规则涵盖的正例和反例数		FOIL信息增益值
现有规则	拟加入前提约束谓词	正例	反例	信息增益值
Father(*x*, *y*)←*Couple*(*x*, *z*)		$m_+ = 1$	$m_- = 1$	/
Father(*x*, *y*)←*Couple*(*x*, *z*)	∧*Mother*(*x*, *y*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	∧*Mother*(*x*, *z*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	∧*Mother*(*y*, *x*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	∧*Mother*(*y*, *z*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	∧*Mother*(*z*, *x*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	∧*Mother*(*z*, *y*)	$\widehat{m}_+ = \mathbf{1}$	$\widehat{m}_- = \mathbf{0}$	**1**
	∧*Sibling*(*x*, *y*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	∧*Sibling*(*x*, *z*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	∧*Sibling*(*y*, *x*)	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA

续表

推理规则		推理规则涵盖的正例和反例数		FOIL信息增益值
现有规则	拟加入前提约束谓词	正例	反例	信息增益值
$Father(x, y) \leftarrow Couple(x, z)$	$\wedge Sibling(y, z)$	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	$\wedge Sibling(z, x)$	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	$\wedge Sibling(z, y)$	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	$\wedge Couple(x, y)$	$\widehat{m}_+ = 0$	$\widehat{m}_- = 1$	NA
	$\wedge Couple(x, z)$	$\widehat{m}_+ = 1$	$\widehat{m}_- = 1$	0
	$\wedge Couple(y, x)$	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	$\wedge Couple(y, z)$	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	$\wedge Couple(z, x)$	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA
	$\wedge Couple(z, y)$	$\widehat{m}_+ = 0$	$\widehat{m}_- = 0$	NA

从表2.8可知，当*Mother*(*z*, *y*)作为前提约束谓词加入到推理规则后*FOIL_Gain*值最大，因此将*Mother*(*z*, *y*)加入，得到新的推理规则*Mother*(*z*, *y*)∧*Couple*(*x*, *z*)→*Father*(*x*, *y*)。当*x* = David、*y* = Mike和*z* = James时，该推理规则覆盖表2.6中训练样本集合的正例*Father*(David, Mike)，且不覆盖任意反例，因此算法学习结束。

当学习得到(∀*x*)(∀*y*)(∀*z*)(*Mother*(*z*, *y*)∧*Couple*(*x*, *z*)→*Father*(*x*, *y*))这一推理规则后，由于从图2.1所给出的知识图谱中已知道*Mother*(James, Ann)和*Couple*(David, James)，因此可推理得到新的知识*Father*(David, Ann)。

从上面这个学习过程可知，给定目标谓词，FOIL算法从实例（正例、反例、背景知识样例）出发，不断测试所得推理规则是否还包含反例，一旦不包含，则学习结束，由此充分展示了“归纳学习”的能力。在学得推理规则后，再给推理规则中的变量赋予具体例子，经过“演绎”得到新的知识，如图2.2所示。

2.3.2 路径排序推理

与FOIL算法不同，路径排序推理算法（PRA）[5]的基本思想是将实体之间的关联路径作为特征，来学习目标关系的分类器。路径排序算法的工作流程主要分为以下三步。

（1）特征抽取：生成并选择路径特征集合。生成路径的方式有随机

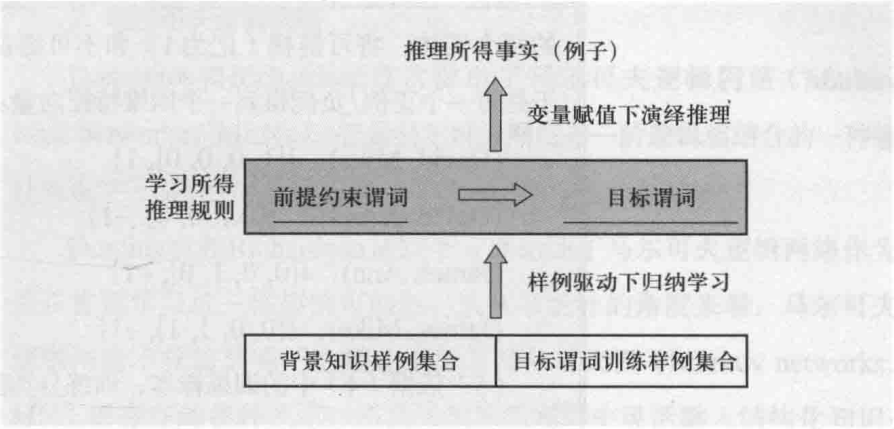


图2.2 结合归纳学习和演绎推理的FOIL模型

游走（random walk）、广度优先搜索、深度优先搜索等。

（2）特征计算：计算每个训练样例的特征值$P(s \to t; \pi_j)$。该特征值可以表示从实体结点s出发，通过关系路径π_j到达实体结点t的概率；也可以表示为布尔值，表示实体s到实体t之间是否存在路径π_j；还可以是实体s和实体t之间路径出现的频次、频率等。

（3）分类器训练：根据训练样例的特征值，为目标关系训练分类器。当训练好分类器后，即可将该分类器用于推理两个实体之间是否存在目标关系。

例2.16　对图2.1中的知识图谱采用路径排序算法。

（1）目标关系：*Father*。

（2）对于目标关系*Father*，生成四组训练样例，一个为正例、三个为负例：

正例：(David, Mike)

负例：(David, James), (James, Ann), (James, Mike)

（3）从知识图谱采样得到路径，每一路径链接上述每个训练样例中两个实体：

(David, Mike)对应路径：$Couple \to Mother$

(David, James)对应路径：$Father \to Mother^{-1}$（$Mother^{-1}$与*Mother*为相反关系）

(James, Ann)对应路径：$Mother \to Sibling$

(James, Mike)对应路径：$Couple \to Father$

（4）对于每一个正例/负例，判断上述四条路径可否链接其包含

的两个实体，将可链接（记为1）和不可链接（记为0）作为特征，于是每一个正例/负例得到一个四维特征向量：

(David, Mike)：{[1, 0, 0, 0], 1}

(David, James)：{[0, 1, 0, 0], –1}

(James, Ann)：{[0, 0, 1, 0], –1}

(James, Mike)：{[0, 0, 1, 1], –1}

（5）依据（4）中的训练样本，训练分类器M。

（6）预测。对于图2.1中形如(David, Ann)的样例，得到其特征值为[1, 0, 0, 0]，将特征向量输入到分类器M中，如果分类器M给出分类结果为1，则*Father*（David, Ann）成立。

2.3.3 其他知识推理算法

下面简要介绍下基于分布式表示（distributed representation）的知识推理和基于演绎推理的马尔可夫逻辑网络。

1. 基于分布式表示的知识推理

随着深度学习应用的进展，在自然语言处理（natural language processing，NLP）中，通过无监督学习将一个单词表示成一个隐向量（latent vector）的分布式表示方法取得了良好效果。研究者将知识图谱中的结点和结点之间的关系均映射到连续数值空间中（即分布式表示），同时对映射过程进行约束，使得知识图谱中结点之间的拓扑结构得以保留，从而有效地对知识图谱进行表示和推理[2]。

如给定图2.1中*Father*（David, Mike）这一个样例知识，可首先学得David和Mike的分布式表达（向量表达）**David**和**Mike**以及谓词*Father*的分布式表达***Father***，然后再学习一个度量函数ψ使得**David** + ***Father***的结果与**Mike**很接近，即ψ(**david+*Father***, **Mike**)取值最小（也即**david+*Father*** ≈ **Mike**）。于是，给定所有<left_node, relation, right_node>形式的样例知识，就可去学习度量函数ψ，使得ψ(**left_node**, **relation**, **right_node**)取值最小。

一旦学习得到度量函数ψ，就可用其去判断任意两个结点之间是否具有某一关系。

2. 马尔可夫逻辑网络

Domingos和Richardson首次提出了马尔可夫逻辑网络（Markov logic networks，MLNs），它是马尔可夫网络与一阶逻辑相结合的一种统计关系学习模型[3, 14]。

Domingos和Richardson从以下方面论证了马尔可夫逻辑网络作为关系推理学习统一框架的可能性：从概率统计的角度来看，马尔可夫逻辑网络不仅简洁明了地描述了庞大马尔可夫网（Markov networks，MN）所存在的各种关系，还在马尔可夫网络中灵活融入结构化知识；从一阶谓词逻辑的角度来看，马尔可夫逻辑网可在一阶谓词逻辑中添加不确定性[14]。

马尔可夫逻辑网络在自然语言处理、复杂网络、信息抽取、计算机视觉等领域都有重要的应用前景。

2.4 因果推理

亚里士多德认为自然界有一种“原因”关系的存在。这种“原因”观念不同于近代以来的“因果”观念，“原因”与“为什么”相对应，并不与“结果”相对应。

哲学上把现象和现象之间那种“引起和被引起”的关系，叫做因果关系，其中引起某种现象产生的现象叫做原因，被某种现象引起的现象叫做结果。因果推理是一种重要的推理手段，是人类智能的重要组成。回答诸如“吸烟是否导致癌症”和“某个广告发布是否导致了某个商品销量上涨”等问题时，往往需要因果推理的能力。

2.4.1 辛普森悖论

为了说明数据中存在因果关系，先来看一个有趣的例子[11]。表2.9列出了某组病人在是否尝试新药以后的恢复情况：不用药病人的恢复率高于用药病人的恢复率。

然而，当对所有病人按照性别分组后，可得到表2.10。当分别比较按照性别分组的两类病人的恢复率时，却发现用药病人的恢复率均高于不用药病人的恢复率，这与表2.9的统计结果正好相反。这就是著名的辛

普森悖论（Simpson's paradox），其指出在总体样本上成立的某种关系却在分组样本里恰好相反。

表2.9 某组病人在是否尝试新药以后的恢复情况

	不用药	用药
恢复人数	289	273
总人数	350	350
恢复率(%)	83	78

表2.10 以性别分组后的某组病人在是否尝试新药以后的恢复情况

	不用药		用药	
	男性	女性	男性	女性
恢复人数	234	55	81	192
总人数	270	80	87	263
恢复率（%）	87	69	93	73

从数学角度而言，上述悖论可写成初等数学不等式$\frac{b}{a}<\frac{d}{c}$，$\frac{b'}{a'}<\frac{d'}{c'}$，$\frac{b+b'}{a+a'}>\frac{d+d'}{c+c'}$。这样的例子在现实生活中经常出现，如加州大学伯克利分校的著名统计学家Peter Bickel教授于1975年在*Science*杂志上发表了一篇论文，报告了加州大学伯克利分校研究生院男女录取率的差异。他发现，总体上男性录取率高于女性，然而按照不同专业分组后，却发现女性录取率高于男性[1]。

辛普森悖论表明，在某些情况下，忽略潜在的“第三个变量”（本例中性别就是用药与否和恢复率之外的第三个变量），可能会改变已有的结论，而人们常常却一无所知。从观测结果中寻找引发结果的原因、考虑数据生成的过程，由果溯因，就是本节要介绍的因果推理[10, 11, 12]。

因果推理主要使用两种模型，一个是结构因果模型（structural causal model，SCM），也被称为因果模型（causal model）或Neyman–Rubin因果模型。这一模型最早可追溯于Jerzy Neyman在1923年用波兰语所撰写论文中提出的“潜在结果”（potential outcome）的概念[6]。之后，Donald Rubin发展了“潜在结果”这一概念，并将其和缺失数据的理论联系在一起[15]。

另外一个模型是Judea Pearl于1995年提出的因果图（causal diagram）[9]。一般而言，因果图是有向无环图（directed acyclic

graphs，DAG）。有向无环图指的是一个无回路的有向图，即从图中任意一个结点出发经过任意条边，均无法回到该结点。有向无环图刻画了图中所有结点之间的依赖关系。DAG可用于描述数据的生成机制。这种描述变量联合分布或者数据生成机制的模型，被称为“贝叶斯网络”（Bayesian network）。

下面分别介绍两种因果推理模型，并定义变量之间存在“原因”这一关系的概念。

定义2.16 结构因果模型：结构因果模型由两组变量集合U和V以及一组函数f组成。其中，f是根据模型中其他变量取值而给V中每一个变量赋值的函数。

定义2.17 结构因果模型中的原因：如果变量X出现在给变量Y赋值的函数中，则X是Y的直接原因（direct cause）。如果X是Y的直接原因或者其他原因，均称X是Y的原因。

U中的变量被称为外生变量（exogenous variables），即这些变量处于模型之外，不对其阐述和解释；V中的变量称为内生变量（endogenous variables）。以图中的结点来说明内生变量和外生变量的关系：每一个内生变量都至少是一个外生变量的后代；而每一个外生变量都不是其他外生或内生变量的后代，它们没有祖先，也就是说，外生变量都是图中的根结点。如果知道了每一个外生变量的值，就可以使用函数f来计算出每一个内生变量的值。

例2.17 在结构因果模型框架下讨论某种治疗方案X对肝脏功能Y的因果关系。在讨论X对Y的因果关系时，可能会假设肝脏功能Y会受到水污染Z的影响，由于水污染Z不会受到治疗方案X和肝脏功能Y的影响，因此，可将X和Y作为内生变量，Z作为外生变量来进行研究。

每个结构因果模型M都与一个因果图G相对应[11]。因果图中的结点是结构因果模型中U和V所包括的变量，结点之间的边表示函数f。在M中，若变量X的函数f_x包含了变量Y（X的取值依赖于Y），则在G中有一条从Y到X的有向边。本书主要讨论因果图为有向无环图的结构因果模型。

下面给出因果图中变量之间存在“原因”关系的概念。

定义2.18　因果图中的原因：在因果图中，若变量Y是另一个变量X的孩子，则X是Y的直接原因；若Y是X的后代，则X是Y的潜在原因。

例2.18　假设某一商品销量Z、该商品与其他商品的价格差X以及为该商品促销所支出的广告费Y三个变量所形成的结构因果模型如下表示：

$$U=\{X,\ Y\},\ V=\{Z\},\ F=\{f_Z\}$$

$$f_Z: Z=2X+5Y$$

由于X和Y都出现在函数f_Z中，因此，X和Y都是Z的直接原因。若X和Y有其他祖先，则这些祖先是Z的潜在原因。图2.3是该结构因果模型所对应的因果图。

图2.3 商品销量Z、价格差X和广告费Y的因果图

结构因果模型和因果图各有其优势和不足，需要组合使用才能更加直观和准确地描述因果关系。在上面的例子中，可以发现因果图更加直观地反映变量之间的关系，但无法对变量之间的关系进行量化，如价格差X和广告费Y是如何影响商品销量Z的。然而在很多时候，变量之间的量化关系无法直接给出或含有某些未知因素，此时，可用部分指定的结构因果模型来表示变量之间的因果关系，如下例。

例2.19　当例2.18中的商品销量、价格差和广告费的量化关系无法给出，并含有某些未知因素时，其部分指定的结构因果模型如下：

$$U=\{U_1,U_2,U_3\},\ V=\{X,Y,Z\},\ F=\{f_X,f_Y,f_Z\}$$

$$X=f_X(U_1)$$

$$Y=f_Y(U_2)$$

$$Z=f_Z(X,Y,U_3)$$

其中U_1，U_2，U_3代表某些未知的外生变量，有时又被称为“误差项”（error term）或“忽略因素”（omitted factor）。

因果图的另一个好处是可以有效地表示联合概率分布。虽然在结构因果模型中可以用一个n元函数来表示n个变量之间的关系，但有些时候却无法给出这样的函数。即使给出了这样的n元函数，有时候也会由于变量太多但样本量不足从而导致联合概率分布的估计不准确。因此，可以考虑对联合概率分布进行分解，下面介绍**乘积分解规则**（rule of product decomposition）。

对于任意的有向无环图模型，模型中d个变量的联合概率分布由每个结点与其父结点之间条件概率P（$child|parents$）的乘积给出：

$$P(x_1, x_2, \cdots, x_d) = \prod_{j=1}^{d} P\left(x_j \middle| x_{pa(j)}\right) \qquad (式2.1)$$

其中，$x_{pa(j)}$表示结点x_j的父结点集合（所有指向x_j的结点）。（式2.1）包含了变量之间某种普遍成立的独立性假设，将在后续章节做详细介绍。

例如，对于一个简单的链式图$X \to Y \to Z$，其联合概率分布可直接写成：

$$P(X=x, Y=y, Z=z) = P(X=x)P(Y=y|X=x)P(Z=z|Y=y)$$

在对该模型生成的数据集上进行联合概率分布的估计时，只需要对x，$(y|x)$，$(z|y)$这三个元组进行计数即可，而无需统计所有可能出现的元组，从而节省了大量的时间和空间。也就是说，图模型上的独立性假设将一个“高维”的估计问题转换成了一个“低维”的概率分布问题。试想，若结构因果模型中含有大量的变量但数据量不够多，而且变量之间的图结构也无从得知，则对其联合概率分布进行估计就会变得十分困难，这就是所谓的“维度灾难”（curse of dimensionality）。

例2.20　考虑链式图$X \to Y \to Z$，其中X表示气候好，Y表示水果产量高，Z表示水果价格低，给出P(气候好，水果产量高，水果价格低)的联合概率。

解：使用乘积分解规则，将P（气候好，水果产量高，水果价格低）转换为：

P（气候好）P（水果产量高 | 气候好）P（水果价格低 | 水果产量高）

根据常识，假设P（气候好）$=0.5$，当然也可设置为更低或更高

的取值。类似地，在气候好的时候水果应该生长得比较好，产量会比较高，因此假设P（水果产量高 | 气候好）$=0.8$。同样，由于市场机制，水果产量高的时候，水果价格会比较低，设P（水果价格低 | 水果产量高）$=0.9$。因此计算出P（气候好，水果产量高，水果价格低）$=P$（气候好）P（水果产量高 | 气候好）P（水果价格低 | 水果产量高）$=0.5\times0.8\times0.9=0.36$。

例2.21　给出如图2.4中8个变量的联合概率。

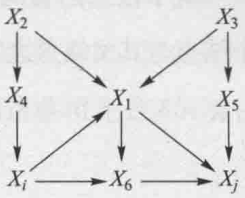


图2.4　因果图$\mathcal{H}$

解：使用乘积分解规则：

$$P(X_1, X_2, X_3, X_4, X_5, X_6, X_i, X_j)$$
$$= P(X_2)\times P(X_3)\times P(X_1|X_2, X_3, X_i)\times P(X_4|X_2)\times P(X_5|X_3)\times$$
$$P(X_6|X_1, X_i)\times P(X_i|X_4)\times P(X_j|X_1, X_5, X_6)$$

2.4.2　因果图的基本概念

2.4.1节提到每个结构因果模型都有一个因果图与之对应，因果图可以更加直观地反映变量之间的关系。本节将介绍几种因果图的基本结构：链结构、分连结构和汇连结构。

1. 链

链（chain）是因果图的一种基本结构。它包含三个结点两条边，其中一条边由第一个结点指向第二个结点，另一条边由第二个结点指向第三个结点。

例2.22　考虑如下的结构因果模型：

$$U=\{U_1, U_2, U_3\},\ V=\{X, Y, Z\},\ F=\{f_X, f_Y, f_Z\}$$
$$f_X: X=U_1$$
$$f_Z: Z=3X+10+U_2$$

$$f_Y: Y = 5Z + U_3$$

其因果图如图2.5所示。

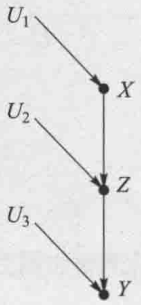


图2.5 链式因果图

首先回顾一下概率论中所学的相关性、独立性和条件独立性的概念。若$P(X|Y)=P(X)$或$P(Y|X)=P(Y)$，则称事件X，Y是相互独立的；反之则称X，Y是相关的。若$P(X, Y|Z)=P(X|Z)P(Y|Z)$，则称事件X，Y在给定事件Z时条件独立。对于上例中的链式因果图，可发现给定Z时，X和Y的联合概率为：

$$P(X,Y|Z)=\frac{P(X,Y,Z)}{P(Z)}=\frac{P(X)P(Z|X)P(Y|Z)}{P(Z)}=P(X|Z)P(Y|Z)$$

即在链式图$X\to Z\to Y$中，X和Y在给定Z时条件独立。其中，上式的第一步使用了条件概率的定义，第二步使用了乘积分解规则，最后一步使用了贝叶斯公式$P(X)P(Z|X)=P(Z)P(X|Z)=P(X, Z)$。

链式图$X\to Z\to Y$中，X和Y在给定Z时条件独立也可以这样理解：在给定Z时，若X的取值发生变化，则为了维持Z的取值不变，U_2的取值会发生变化；而由于Y的取值只依赖于Z和U_3，但Z是给定的，因此Y的取值不会发生变化。也就是说，给定Z时，X的取值不会影响Y的取值，因此给定Z时，Y和X是条件独立的。

由上面的例子可知，在任意的因果图中有以下定理。

定理2.5（链中的条件独立性） 对于变量X和Y，若X和Y之间只有一条单向的路径，变量Z是截断（intercept）该路径的集合中的任一变量，则在给定Z时，X和Y条件独立。

2. 分连

分连（fork）也是因果图的一种基本结构。它包含三个结点两条边，

两条边分别由第一个结点指向第二个结点和第三个结点。

例2.23　考虑如下的结构因果模型：

$$U=\{U_1, U_2, U_3\},\ V=\{X, Y, Z\},\ F=\{f_X, f_Y, f_Z\}$$

$$f_Z: Z=U_1$$

$$f_X: X=3Z+7+U_2$$

$$f_Y: Y=6Z+U_3$$

其因果图如图2.6所示。

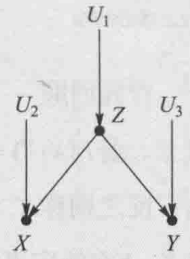


图2.6 分连因果图

类似上述提到的链结构，可发现在分连结构中，给定Z时，X和Y的联合概率为：

$$P(X,Y|Z)=\frac{P(X,Y,Z)}{P(Z)}=\frac{P(Z)P(X|Z)P(Y|Z)}{P(Z)}=P(X|Z)P(Y|Z)$$

即在分连图$X\leftarrow Z\rightarrow Y$中，$X$和$Y$在给定$Z$时条件独立。上式的第一步使用了条件概率的定义，第二步使用了乘积分解规则。

类似链结构，在分连图$X\leftarrow Z\rightarrow Y$中，$X$和$Y$在给定$Z$时条件独立可以这样理解：若给定$Z$，$Y$和$X$的取值不会变化，它们的值只会分别随$U_3$和$U_2$变化，因此给定$Z$时，$Y$和$X$是条件独立的。

定理2.6（分连中的条件独立性） 若变量Z是变量X和Y的共同原因，且X到Y只有一条路径，则在给定Z时，X和Y条件独立。

3. 汇连

汇连（collider）又叫碰撞也是因果图的一种基本结构。它包含三个结点两条边，两条边分别由第一个结点和第二个结点指向第三个结点。

例2.24　考虑如下的结构因果模型：

$$U=\{U_1, U_2, U_3\},\ V=\{X, Y, Z\},\ F=\{f_X, f_Y, f_Z\}$$

$$f_X: X = U_1$$

$$f_Y: Y = U_2$$

$$f_Z: Z = X + Y + U_3$$

其因果图如图2.7所示。

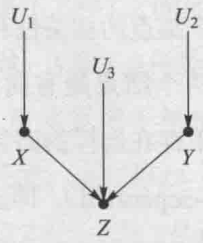


图2.7 汇连因果图

类似上述提到的链结构和分连结构，可发现在汇连结构中，给定Z时，X和Y的联合概率为：

$$P(X,Y|Z)=\frac{P(X,Y,Z)}{P(Z)}=\frac{P(X)P(Y)P(Z|X,Y)}{P(Z)}\neq P(X|Z)P(Y|Z)$$

即在汇连图$X\rightarrow Z\leftarrow Y$中，$X$和$Y$在给定$Z$时条件相关。上式的第一步使用了条件概率的定义，第二步使用了乘积分解规则。

类似地，在汇连图$X\rightarrow Z\leftarrow Y$中，$X$和$Y$在给定$Z$时条件相关可以这样理解：给定$Z$，当$X$的取值发生变化时，为了保证$Z$的取值不变，$Y$的取值也一定会发生变化，因而给定$Z$时，$Y$和$X$是相关的。不妨设例2.24中的$Z=10$，$U_3=1$，当$X$取值为3时，$Y$的取值为6；当$X$的取值为4时，$Y$的取值为5；当$X$取值为8时，$Y$的取值为1。也就是说，给定$Z$时，$Y$的取值会随$X$的取值变化而变化，因此，给定$Z$时，$Y$和$X$是相关的。

类似上述提到的链结构和分连结构，在任意的因果图中有如下定理。

定理2.7（汇连中的条件独立性） 若变量Z是变量X和Y的汇连结点，且X到Y只有一条路径，则X和Y相互独立，但在给定Z或Z的后代时，X和Y是相关的。

对于定理2.7中，给定Z的后代，X和Y是相关的，可以这样理解：不妨设Z的某个后代为W，给定W时，则W的祖先也给定，W的祖先给定时，则W的祖先的祖先也给定……因为Z是W的祖先，因此Z也会给定，此时，X和Y是相关的。

上述提到的链结构、分连结构和汇连结构只是因果图中最基本最简单的模型，真实的因果图往往含有更多的结点和更多的路径，表现出更复杂的结构。那是否存在一种方法，使得其可以用于判断任意复杂的因果图上变量间的相关性和独立性呢？这种方法就是D-分离。

D-分离（directional separation，D-separation），可用于判断任意两个结点的相关性和独立性。若存在一条路径将这两个结点连通，则称这两个结点是有向连接（D-connected）的，即这两个结点是相关的；若不存在这样的路径将这两个结点连通，则称这两个结点是有向分离的（D-separated），即这两个结点相互独立。

下面介绍D-分离的定义。

定义2.19　D-分离：路径p被限定集Z阻塞（block）当且仅当：

（1）路径p含有链结构$A\to B\to C$或分连结构$A\leftarrow B\to C$且中间结点B在Z中，或

（2）路径p含有汇连结构$A\to B\leftarrow C$且汇连结点B及其后代都不在Z中。

若Z阻塞了结点X和结点Y之间的每一条路径，则称给定Z时，X和Y是D-分离，即给定Z时，X和Y条件独立。

例2.25　分析图2.8中结点的关系。

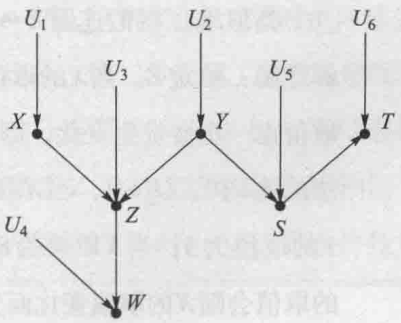


图2.8 包含链式、分连和汇连结构的因果图

不妨如下考虑X和T的关系。

（1）若限定集为$\varnothing$时，X和T相互独立。因为X和T之间含有一个汇连结构$X\to Z\leftarrow Y$，且Z及其后代都不在限定集$\varnothing$中，此时X和T是有向分离，即X和T相互独立。

（2）若限定集为$\{W\}$时，X和T是相关的。因为X和T之间含有一个链式结构$Y\to S\to T$（S不在限定集中），一个分连结构$Z\leftarrow Y\to S$（Y不在限定集$\{W\}$中），一个汇连结构$X\to Z\leftarrow Y$（Z的后代W在限

定集{W}中），根据D-分离的定义，这些结构都无法阻塞X和T之间的路径，因此X和T是有向连接的，即X和T是相关的。

（3）若限定集为{W, Y}时，X和T条件独立。因为X和T之间只有一条路径，且这条路径含有一个分连结构$Z \leftarrow Y \rightarrow S$，且$Y$在限定集{$W$, Y}中，因此Y阻塞了X和T之间的唯一路径，X和T是有向分离，即限定集为{W, Y}时，X和T条件独立。

有了D-分离方法，可将其用于判断任意复杂的因果图上变量间的相关性和独立性。反之，也可将D-分离用于因果图的测试。例如，从数据中得出在给定Z时，X和Y是相关的，但在某个假设的因果图中发现：给定Z时，利用D-分离方法得到X和Y是条件独立的，因此该假设的因果图是错误的，应该对其进行修正。不仅如此，D-分离还会指出因果图上应该修正的地方：X和Y之间必定存在一条路径使得其不被Z所阻塞。

2.4.3 干预的因果效应

2.4.2节提到的图模型和D-分离可用于判断因果图中结点间的相关性和独立性。但正如统计学中的名言——“相关性不是因果性”（correlation is not causation），两个变量之间的相关性并不足以说明其中一个变量是另一个变量的原因。最著名的例子是，冰淇淋的销量上升与犯罪率的上升相关，并不是由于冰淇淋导致了犯罪，而是由于它们都是天气炎热导致的。因此，随机对照试验（randomized controlled trial）成了统计学中进行因果推理的黄金标准。在随机对照试验中，影响结果变量的输入变量只有一个是随机变化的，其他的均保持不变，因此，若结果变量发生变化，则一定是由于该输入变量引起的。

然而，现实世界中很多问题都无法采用随机对照试验。例如，人们无法随意控制天气，研究天气对犯罪率的影响；人们无法随意控制人群吸烟，研究同一个人群是否吸烟的影响；人们无法随意控制股市，研究某些因素对股票价格的影响。本节介绍的干预将可以有效地解决这一类问题。

干预（intervention）指的是固定（fix）系统中的变量，然后改变系统，观察其他变量的变化。为了与X自然取值x时进行区分，在

对X进行干预时，引入“do算子”（do-calculus），记作$do(X=x)$。因此，$P(Y=y|X=x)$表示的是当发现$X=x$时，$Y=y$的概率；而$P(Y=y|do(X=x))$表示的是对X进行干预，固定其值为x时，$Y=y$的概率。用统计学的术语来说，$P(Y=y|X=x)$反映的是在取值为x的个体X上，Y的总体分布；而$P(Y=y|do(X=x))$反映的是如果将每一个X取值都固定为x时，Y的总体分布。

以本节最开始提到的辛普森悖论为例，其因果图可以用图2.9表示，其中，X表示用药情况（1表示用药，0表示不用药），Y表示病人的恢复情况（1表示恢复，0表示未恢复），Z表示性别（1表示男性，0表示女性）。为了比较病人用药与否的恢复情况，可以对用药情况分别进行干预，即将病人都分别固定成用药病人和不用药病人。不妨设$do(X=1)$和$do(X=0)$分别表示这两种干预，并估计其中的差别：

$$P\left(Y=1|do\left(X=1\right)\right)-P\left(Y=1|do\left(X=0\right)\right) \qquad （式2.2）$$

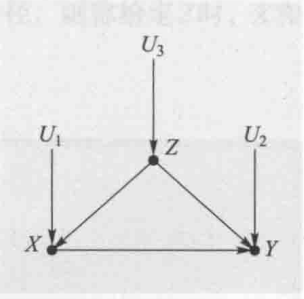


图2.9 辛普森悖论的因果图

（式2.2）被称为“因果效应差”（causal effect difference）或“平均因果效应”（average causal effect，ACE），$P(Y=y|do(X=x))$被称为因果效应。对用药情况X进行干预并固定其值为x时，可将所有指向X的边均移除，则因果效应$P(Y=y|do(X=x))$等价于图2.10所示的引入干预的操纵图模型（manipulated model）中的条件概率$P_m(Y=y|X=x)$。

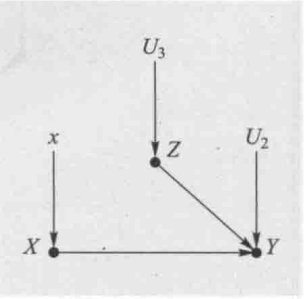


图2.10 引入干预的操纵图模型

计算因果效应的关键在于计算操纵概率（manipulated probability）P_m。P_m与正常（无干预，如图2.9）条件下的概率P有两个相同的重要性质：① 边缘概率$P(Z=z)$不随干预而变化，因为Z的取值不会因为去掉从Z到X的箭头而变化；② 条件概率$P(Y=y|X=x, Z=z)$不变，因为Y关于X和Z的函数$f_Y=(X, Z, U_2)$并未改变。因此，有如下等式：

$$P_m\left(Z=z\right)=P\left(Z=z\right),\ P_m\left(Y=y|X=x,Z=z\right)=P\left(Y=y|X=x,Z=z\right)$$

在操纵图模型中，X和Z是D-分离，即$P_m(Z=z|X=x)=P_m(Z=z)=P(Z=z)$。将三个等式组合在一起，则因果效应$P(Y=y|do(X=x))$为：

$$\begin{aligned} P\left(Y=y\middle|do\left(X=x\right)\right)&=P_m\left(Y=y\middle|X=x\right)\\ &=\sum_z P_m\left(Y=y\middle|X=x,Z=z\right)P_m\left(Z=z\middle|X=x\right)\\ &=\sum_z P_m\left(Y=y\middle|X=x,Z=z\right)P_m\left(Z=z\right) \end{aligned} \quad (式2.3)$$

其中（式2.3）的第一行来源于定义，第二行来源于贝叶斯公式，第三行来源于X和Z在操纵图模型中的独立性。最后，得到如下正常（无干预）条件下的概率表示的因果效应：

$$P\left(Y=y\middle|do\left(X=x\right)\right)=\sum_z P\left(Y=y\middle|X=x,Z=z\right)P\left(Z=z\right) \quad (式2.4)$$

（式2.4）被称为调整公式（adjustment formula），对于Z的每一个取值z，（式2.4）计算X和Y的条件概率并取均值。这个过程称之为“Z调整”（adjusting for Z）或“Z控制”（controlling for Z）。（式2.4）的右端只包含正常（无干预）条件下的概率P，即可用正常（无干预）条件下的条件概率来计算干预后的条件概率。

下面将调整公式用于本节最开始提到的辛普森悖论，其中$X=1$表示用药，$Y=1$表示病人恢复，$Z=1$表示男性，$Z=0$表示女性，则有：

$$\begin{aligned} &P\left(Y=1\middle|do\left(X=1\right)\right)\\ &=P\left(Y=1\middle|X=1,Z=1\right)P\left(Z=1\right)+P\left(Y=1\middle|X=1,Z=0\right)P\left(Z=0\right) \end{aligned}$$

代入表2.10的数据，则将病人干预为用药病人的因果效应为：

$$\begin{aligned} &P\left(Y=1\middle|do\left(X=1\right)\right)\\ &=0.93\times\frac{\left(87+270\right)}{\left(350+350\right)}+0.73\times\frac{\left(263+80\right)}{\left(350+350\right)}=0.832 \end{aligned}$$

类似地，将病人干预为不用药病人的因果效应为：

$$\begin{aligned} &P\left(Y=1\middle|do\left(X=0\right)\right)\\ &=0.87\times\frac{\left(87+270\right)}{\left(350+350\right)}+0.69\times\frac{\left(263+80\right)}{\left(350+350\right)}=0.7818 \end{aligned}$$

在分别计算完用药病人和不用药病人的干预因果效应后，再计算其因果效应差：

$$ACE=P\left(Y=1\middle|do\left(X=1\right)\right)-P\left(Y=1\middle|do\left(X=0\right)\right)=0.832-0.7818=0.0502$$

说明用药病人的恢复率高于不用药病人的恢复率，即该新药能帮助治愈病人。

在上面的例子中，通过将Z放入调整公式中，能够利用正常（无干

预）条件下的条件概率计算出干预后的因果效应。那么，哪些变量（或变量集合）可以放入调整公式中呢？在进行干预时，由于要将X固定，并将所有指向X的箭头都去掉，即使得X的父结点失效，因此，应该将X的父结点放入调整公式中。令X的父结点集合为$PA(X)$，则有如下定理。

定理2.8（因果效应） 给定因果图G，PA表示X的父结点集合，则X对Y的因果效应为：

$$P\left(Y=y\middle|do\left(X=x\right)\right)=\sum_{z}P\left(Y=y\middle|X=x,PA=z\right)P\left(PA=z\right)$$

其中，z是PA的非空子集。

2.4.4 反事实模型

人的一生就是在不断地随环境变化而做出选择。唐朝大书法家、诗人颜真卿在《劝学》中写道："三更灯火五更鸡，正是男儿读书时。黑发不知勤学早，白首方悔读书迟"；南宋词人陆游也曾说过"书到用时方恨少"。正如诗仙李白在《秋风词》中写到"早知如此，何必当初"，人们经常会听到周围的人或自己抱怨——"早知道是这样，我当初就（不）该……"，"如果当时……，现在就……"。

这种类似"如果当时……，现在就……"的语句就是本节将要讨论的反事实（counterfactual），也称为"虚拟事实"或"平行世界"，其中，"如果"后面的语句被称为假设条件（hypothetical condition）或前件（antecedent）。

反事实模型（counterfactual model，也叫作potential outcomes）是大卫·刘易斯（David Lewis）等人提出的推断因果关系的标准[4]。

反事实描述的是：假设存在一个虚拟的平行世界，里面的所有因素与现实世界一模一样，两个相同的个体他和"他"，分别在现实世界和平行世界中同时同地做了不同的选择，现在他知道了现实世界中的结果，他想知道平行世界中的那个"他"的选择所带来的结果。然而，平行世界并不存在。幸运的是，反事实将告诉他另一个"他"的选择所带来的结果。

若用符号$U=u$表示某个叫"张三"的个体的特征，X表示某个称为"身高"的变量，则$X(u)$表示张三的身高。反事实语句"在环境$U=u$下，若$X=x$，则$Y=y$"可表示成$Y_x(u)=y$，其中Y和X是V中的两个变量。考虑如下的结构因果模型M：

$$X = aU + 1$$

$$Y = bX + U + 2$$

首先计算反事实$Y_x(u)$，即在环境$U=u$时，若$X=x$，则Y应如何取值。将$X=x$代入第一个等式中，则有“修正”（modified）模型M_x：

$$X = x$$

$$Y = bX + U + 2$$

将$U=u$和$X=x$代入第二个等式中，则有：

$$Y_x(u) = bx + u + 2$$

该结果与预期一致，即“Y的值本来就应该为X的b倍加上u，再加上常数2”。另一个不明显的结果是反事实$X_y(u)$，即“在环境$U=u$下，若$Y=y$，则$X=x$”。将$Y=y$代入模型M中，则有修正模型M_y：

$$X = aU + 1$$

$$Y = y$$

再将$U=u$代入M_y的第一个等式，此时有$X_y(u) = au + 1$，即“若$Y=y$，X的取值不变”。X在反事实条件下的不变性表明，“对未来结果的假设并不会改变过去的选择”，即“早知如此，依旧当初”。

上述的模型M中包含多种反事实，不妨设U的取值范围为$\{1, 2, 3\}$，$a=1$，$b=2$，则在不同x和y的取值情况下，$X(u)$，$Y(u)$，以及反事实$Y_x(u)$和$X_y(u)$的取值情况如表2.11所示。

表2.11 因果模型M中$X(u)$，$Y(u)$，$Y_x(u)$和$X_y(u)$的取值情况

u	$X(u)$	$Y(u)$	$Y_1(u)$	$Y_2(u)$	$Y_3(u)$	$X_1(u)$	$X_2(u)$	$X_3(u)$
1	2	7	5	7	9	2	2	2
2	3	10	6	8	10	3	3	3
3	4	13	7	9	11	4	4	4

从上面的例子中可以看出反事实与前述的干预的区别：干预计算的是概率分布，它的计算结果是一个概率；反事实计算的是在假设$X=x$下，Y的取值，它的计算结果是一个值。从实验者的角度来看，干预描述的是总体的行为；反事实描述的是在环境$U=u$下，某个个体的行为。

下面介绍反事实在个体行为推理时的作用。图2.11表示的是兴趣爱好与知识渊博程度的因果图。其中，X表示求知欲望的程度，Z表示看书的数量，Y表示知识的渊博程度。为了表示方便，将三个变量均进行归

一化，即它们的均值均为0，方差均为1。

不妨设图2.11的结构因果模型为：

$$X = U_1$$

$$Z = aX + U_2$$

$$Y = bZ + cX + U_3$$

其中，$U_i(i = 1, 2, 3)$为外生变量，它们相互独立。a, b, c的取值可从统计数据中进行估计，不妨设$a = 0.8$，$b = 0.6$，$c = 0.4$。设有一个叫张三的同学，发现其$X = 0.3$，$Z = 0.4$，$Y = 0.5$，那么，如果张三将其看书的数量提高一倍，那么张三的知识渊博程度为多少呢?

在该因果模型中，变量的值由系数和外生变量决定，且其中的外生变量将作用于所有个体。因此，可将张三的观测值$X = 0.3$，$Z = 0.4$，$Y = 0.5$作为证据E，反推外生变量的值。即有如下等式：

$$U_1 = 0.3$$

$$U_2 = 0.4 - 0.8 \times 0.3 = 0.16$$

$$U_3 = 0.5 - 0.6 \times 0.4 - 0.4 \times 0.3 = 0.14$$

接下来，用$Z = 0.8$表示张三的看书数量提升一倍，修正后的模型如图2.12所示。则反事实$Y_{Z=0.8}(U_1 = 0.3, U_2 = 0.16, U_3 = 0.14)$：

$$Y_{Z=0.8}(U_1 = 0.3, U_2 = 0.16, U_3 = 0.14)$$

$$= 0.6 \times 0.8 + 0.4 \times 0.3 + 0.14 = 0.74$$

即，若将张三的看书数量提升一倍，则他的知识渊博程度将由0.5变为0.74。

由以上的例子，可以得出反事实计算的三个步骤：

图2.11 求知欲望与知识渊博程度因果图

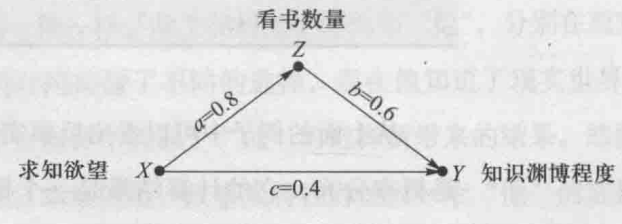
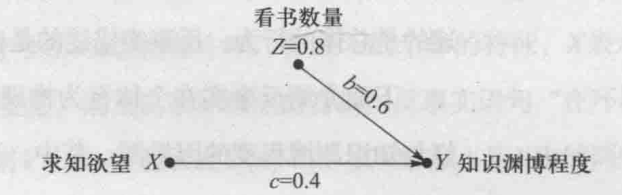


图2.12 将看书数量提升一倍后的反事实模型图



（1）溯因（abduction）：利用现有的证据E确定环境U；

（2）动作（action）：对模型M进行修改，移除等式X中的变量并将其替换为$X=x$，得到修正模型M_x；

（3）预测（prediction）：利用修正模型M_x和环境U计算反事实$Y_x(U)$的值。

2.4.5 因果分析的层次化

Judea Pearl把因果分析分类为如表2.12所示的层次化[12]，包括从可观测结果中得到的数据关联、对观测结果进行干预介入后的决策问题、对已经发生的结果进行否定的反事实推理。

在表2.12中，A可视为一种观测变量，y和y'可视为一种结果。

表2.12 因果模型的层次化示意图

可观测问题	What if we see A (what is?)	$P(y\|A)$	关联（association）：直接可从数据中计算得到的统计相关
决策行动问题	What if we do A (what if?)	$P(y\|do(A))$（如果采取A行为，则y将如何）	干预（intervention）：无法直接从观测数据就能得到关系，如“某个商品涨价会产生什么结果”
反事实问题	What if we did things differently (why)	$P(y'\|A)$（如果A为真，则y'将如何）	反事实（counterfactual）：某个事情已经发生了，则在相同环境中，这个事情不发生会带来怎样的新结果

图2.13给出了对反事实推理的一种图示解释。一旦收集了表示生活习惯（如吸烟和熬夜等）的数据，就可对这些数据建模，来对身体健康状态进行推理。

在反事实推理中，往往会改变某个变量，使得训练好的推理模型发生变化。如图2.13中，从收集的观测数据中已经训练优化得到一个数据联合发布模型$\boldsymbol{P}$，但在反事实推理中，因为模型变量出现了变化，因此需要在一个新的模型$\boldsymbol{P'}$下进行分析。从这个角度来看，反事实推理的确

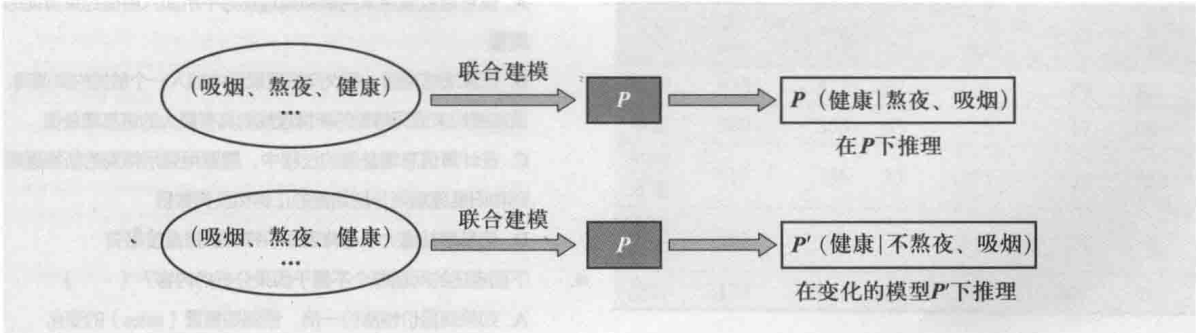


图2.13 统计学习与因果推理区别示意图

变得更困难。

应该指出，因果在很大程度上是客观世界中现象折射到人类自身认知体系的一种文化现象，是人类对于客观世界的现象关联做出的一种自认为合理的解释，这种解释需要建立在可靠的论证基础上。

2.5 小结

逻辑与推理具有一定的区别。逻辑本身不承载推理的使命，而是研究什么是推理以及提出一些一般性的原则来保证推理结果的正确性，如归结法等。

推理的方法有很多种，如演绎推理、归纳推理和因果推理等，在实际任务中，一般是多种推理方法叠加协同，完成特定任务。

习题2

1. p和q均是原子命题，“如果p那么q”是由p和q组合得到的复合命题。下面对“如果p那么q”这一复合命题描述不正确的是（　　）。

   A.“如果p那么q”定义的是一种蕴涵关系（即充分条件）

   B.“如果p那么q”意味着命题q包含着命题p，即p是q的子集

   C. 无法用真值表来判断“如果p那么q”的真假

   D. 当p不成立时，“如果p那么q”恒为真

2. 下面哪个逻辑等价关系是不成立的？（　　）

   A. $\forall x \neg P(x) \equiv \neg \exists x P(x)$　　B. $\neg \forall x P(x) \equiv \exists x \neg P(x)$

   C. $\forall x P(x) \equiv \neg \exists x \neg P(x)$　　D. $\exists x P(x) \equiv \neg \forall x P(x)$

3. 下面对一阶归纳推理（FOIL）中信息增益值（information gain）阐释不正确的是（　　）。

   A. 信息增益值用来判断向推理规则中所加入前提约束谓词的质量

   B. 在算法结束前，每次向推理规则中加入一个前提约束谓词，该前提约束谓词得到的新推理规则具有最大的信息增益值

   C. 在计算信息增益值的过程中，需要用到所得到的新推理规则和旧推理规则分别涵盖的正例和反例数目

   D. 信息增益值大小与背景知识样例数目直接相关

4. 下面描述的问题哪个不属于因果分析的内容？（　　）

   A. 如果商品价格涨价一倍，预测销售量（sales）的变化

B. 如果广告投入增长一倍，预测销售量（sales）的变化

C. 如果放弃吸烟，预测癌症（cancer）的概率

D. 购买了一种商品的顾客是否会购买另外一种商品

5. 判断下列句子是否为命题。

a）$x>2$。

b）今天的天气真好啊！

c）浙江大学始创于1897年。

6. 应用归结法证明以下命题集是不可满足的。

a）$\alpha \vee \beta$。

b）$\beta \rightarrow \gamma$。

c）$\neg \alpha \wedge \neg \gamma$。

7. 使用全称量词或存在量词描述下列事实。

a）不是所有的篮球运动员的身高都超过1.8m。

b）所有实数的平方都大于或等于零。

8. 已知$(\forall x)(F(x) \rightarrow (G(x) \vee H(x)))$，$\neg(\forall x)(F(x) \rightarrow G(x))$，试证明$(\exists x)(F(x) \wedge H(x))$。

9. 如果将图2.1中James和Ann之间的*Mother*关系去掉，同时添加David和Ann之间的*Father*关系，则可得到图2.14。试运用FOIL算法推出James和Ann的*Mother*关系。

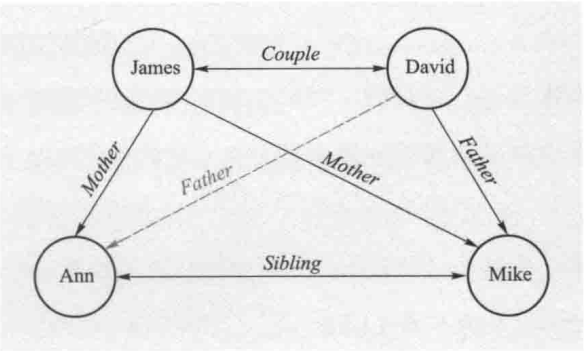


图2.14 家庭关系图

10. 在图2.14中运用路径排序算法推出James和Ann的*Mother*关系。

11. 表2.13给出了某所大学不同系别在录取学生过程中的申请人数、申请人性别以及录取人数的统计数据。请对这些数据进行分析，解释其在录取过程中是否存在所谓的“歧视女生”问题？

表2.13 申请人数与录取人数数据

系别	男性申请者	录取	比例（%）	女性申请者	录取	比例（%）
英语	825	512	62	108	89	82
俄语	560	353	63	25	17	68
西班牙语	417	138	33	375	131	35
意大利语	373	22	6	341	24	7
总计	2175	1024	47	849	261	31

12. 图2.15给出了不同变量之间的依赖关系，请写出因果图$\mathcal{H}$中8个变量之间的联合概率形式，并区分哪些变量是内生变量，哪些变量是外生变量。

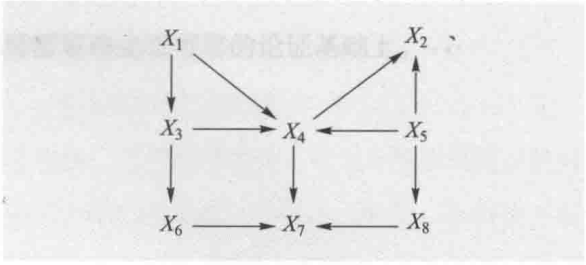


图2.15 因果图$\mathcal{H}$

*13. 给出五个可能的限定集Z，使其阻塞图2.15因果图$\mathcal{H}$中的结点X_6和结点X_8。

参考文献

[1] BICKEL P J, HAMMEL E A, O'CONNELL J W.Sex bias in graduate admissions: Data from Berkeley [J]. Science, 187(4175): 398–404, 1975.

[2] BORDES A, USUNIER N, GARCIA D A, et al.Translating embeddings for modeling multi-relational data [J]. Proceedings of Advances in neural information processing systems(NIPS), 2787–2795, 2013.

[3] DOMINGOS P, RICHARDSON M. Markov logic: A unifying framework for statistical relational learning [J]. Proceedings of the ICML 2004 Workshop on Statistical Relational Learning and its Connections to Other Fields, 49–54, 2004.

[4] LEWIS D. Counterfactuals [M]. Oxford: Blackwell Publishers and Cambridge, MA: Harvard University Press, 1973, Reprinted with revisions, 1986.

[5] LAO N, COHEN W W. Relational retrieval using a combination of path-constrained random walks [J]. Machine learning, 81(1): 53–67, 2010.

[6] NEYMAN J S. On the Application of Probability Theory to Agricultural Experiments [J]. Essay on Principles. Section 9, Statistical Science, 5: 465–472, 1923.

[7] NICKEL M, MURPHY K, TRESP V, et al. A Review of Relational Machine Learning for Knowledge Graphs [J]. Proceedings of the IEEE, 104(1): 11–33, 2016.

[8] PLOTKIN G. Automatic Methods of Inductive Inference[D]. University of Edinburgh, 1970.

[9] PEARL J. Causal diagrams for empirical research [J]. Biometrika, 82(4): 669–710, 1995.

[10] PEARL J. Causality: models, reasoning, and inference [M]. 2nd edition. Cambridge: Cambridge University Press, 2009.

[11] PEARL J, GLYMOUR M, JEWELL N P. Causal inference in statistics: a primer [M]. New Jersey : John Wiley & Sons, 2016.

[12] PEARL J.Theoretical impediments to machine learning with seven sparks from the causal revolution [J] .arXiv: 1801.04016, 2018.

[13] QUINLAN J R. Learning logical definitions from relations[J]. Machine learning, 5(3): 239–266, 1990.

[14] RICHARDSON M, DOMINGOS P.Markov logic networks

[J]. Machine learning, 62(1–2): 107–136, 2006.

[15] RUBIN D. Estimating causal effects of treatments in randomized and non–randomized studies [J]. Journal of Educational Psychology, 66(5): 688–701, 1974.

[16] SHAPIRO E. The model inference system [C] //Proceedings of the 7th international joint conference on Artificial intelligence–Volume 2. San Francisco: Morgan Kaufmann Publishers Inc., 1981.

[17] SOCHER R, CHEN D, MANNING C D, et al. Reasoning with neural tensor networks for knowledge base completion [J]. Proceedings of Advances in neural information processing systems(NIPS): 926–934, 2013.

第3章 搜索求解

3

故记诵者，学问之舟车也
——清 · 章学诚《文史通义》

早期人工智能算法进行问题求解的过程就是在给定具体问题后，算法在问题所对应解空间中进行搜索，得到与问题相对应、满足一定的约束条件的最佳答案，即寻找一条从初始状态（问题所处状态）到目标状态（答案所处状态）的最佳路径。

如果将人工智能算法看作一个智能体，在基于搜索方式的问题求解中，智能体从初始状态出发，在每一个当前状态，会根据一定的约束条件，从类似记忆体的已知信息空间中搜寻解决之道。基于搜索的问题求解这一手段在现实中是非常常见的，如最短路径搜索、对抗博弈中搜寻最优行动等。

在搜索过程中，如果搜索空间有限或可控，智能算法会采取穷尽（exhaustive）搜索或剪枝（pruning）搜索。反之，如果搜索空间极大（如围棋等），一般会采取采样（sampling）搜索策略。

本章3.1节介绍无信息搜索问题和求解的基本框架，3.2节在此基础上介绍利用辅助信息来辅助搜索的内容，3.3节介绍简单博弈问题中搜索算法，3.4节介绍使用蒙特卡洛法来求解近似最优解方法。如果要更加深入了解本章探讨的内容，请参考相关文献[2, 7]。

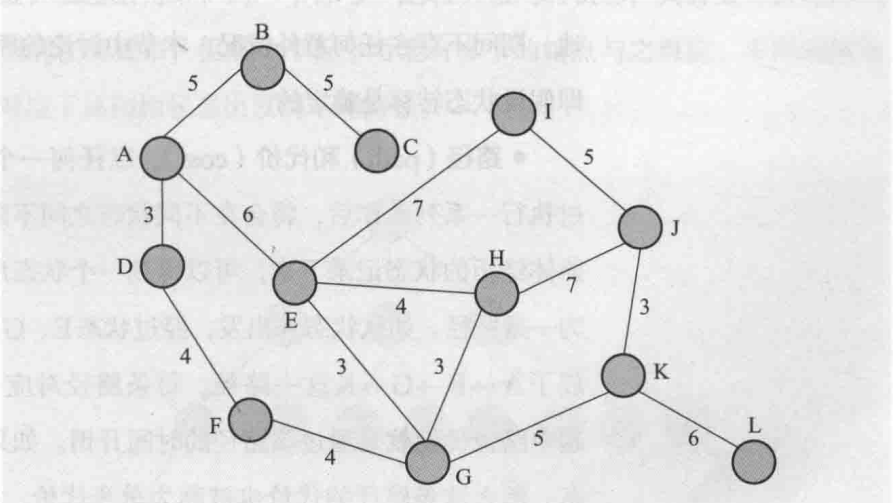
3.1 搜索算法基础

3.1.1 搜索基本问题和求解

搜索算法广义来说是探索特定问题所对应解的一种算法。根据搜索问题的不同性质，所求解方法不尽相同。本节将介绍一种最简单的搜索问题，以此展开对搜索算法的讨论。

假设图3.1为某地区的交通网络图。图中每个结点对应一个城市，每条边表示所连接两个城市之间存在直达列车，每一条边上数字表示其所连接两个城市之间的列车行驶时间。现在要寻找从城市A到城市K之间一条行驶时间最短路线，求取最短路线正是搜索算法要解决的一个典型问题。

图3.1 某地区的交通网络图



人类为了解决以上问题，一般会在脑中进行规划，然后按照计划执行。人们在脑中假想了一个智能体，它代替人们辗转于各个城市之间，计算城市之间的车程时间，寻找一种最佳的方案。为了介绍最短路径搜索，下面定义若干术语。

- **状态**（state）。广义来说状态是对智能体和环境当前情形的描述。为了简化，常常会剔除一些与问题求解无关的细节描述（如每个城市的人口情况等），只保留一些对求解模型产生作用的信息，这些信息在搜索问题中被称为状态。如图3.1所示问题中，智能体要从城市A出发找到一条到城市K最短路线，影响其下一步行动的信息只有它当前所在城市，因此这个问题中每个城市即为一个状态。智能体刚开始所在状态（即城市A）被称为初始状态，达成目标时所在状态（即城市K）被称为终止状态或目标状态。

- **动作**（action）。为了探索问题的答案，智能体需要不断从一个状态转移到下一个状态。智能体为了从一个状态转移到另外一个状态所采取的行为被称为动作。在图3.1的问题中，动作即为乘坐一趟列车从当前城市到下一城市。显然在这个问题中，任何状态下可选择的动作都是离散的，这个性质也适用于本章讨论的所有问题。

- **状态转移**（state transition）。智能体选择了一个动作之后，其所处状态也会发生相应变化，这个过程被称为状态转移。例如，在图3.1的问题中，智能体在城市A乘上开往城市D的列车，那么智能体将发生从状态A到状态D的状态转移。这个例子中，假设智能体可事先知道从当前状态乘坐某辆列车将要抵达的目的地，例如智能体已知从城市A可抵达城市D。同时假设每辆列车必然会严格遵照其路线和时间安排到达目的地，期间不存在任何意外情况。本节中讨论的所有算法都基于这个假设，即假设状态转移是确定的。

- **路径**（path）**和代价**（cost）。以任何一个状态为起点，智能体通过执行一系列动作后，将会在不同状态之间不断转移。将这个过程中智能体经历的状态记录下来，可以得到一个状态序列，这个状态序列被称为一条路径。如从状态A出发，经过状态E、G，最终到达状态K，就形成了A→E→G→K这一路径。每条路径对应一个代价，在图3.1的问题中路径代价就是通过该路径的时间开销。如果一条路径中仅有两个状态，那么这条路径的代价也被称为单步代价，图3.1中每条边上的数值

即为单步代价取值。在下文中，如果不特别说明，一般假定单步代价取值为非负。

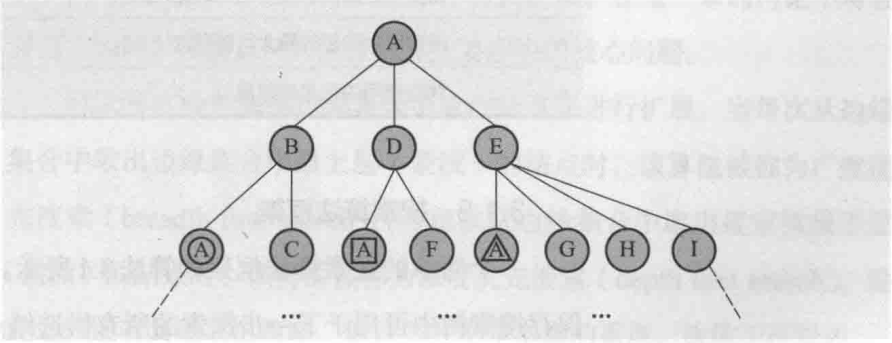
- **目标测试**（goal test）。goal_test(s)函数用于判断状态s是不是目标状态。在图3.1的问题中，如果当前状态为K，那么目标测试通过，否则不通过。目标测试通过意味着搜索算法完成。搜索算法完成并不意味着算法已经找到了最短路径。

本章3.1和3.2节将讨论通过以上五个要素定义的搜索问题，3.3和3.4节将会改变或放宽其中部分条件，来讨论不同类型的搜索问题和算法。

在解决问题的过程中，搜索算法会时刻记录所有从初始结点出发已经探索过的路径，每次从中选出一条，从该路径末尾状态出发进行一次状态转移，探索一条尚未被探索过新路径，如此循环，直到找到一条从初始状态到终止状态的路径为止。当然，寻找得到的从初始状态到终止状态的路径是否满足约束条件，例如该路径是否为最短路径，还需要另行判断。

可以用一棵树来记录算法探索过的路径，图3.2对应了图3.1中部分搜索结果。这样一棵树被称为搜索树，树中每个结点用一个状态（城市）标记（即每个状态对应一个城市）。在图3.2所示搜索树第三层中有三个标号均为A的结点，它们在图中分别被圆圈、正方形和三角形框住。虽然这三个标号均为A的结点对应同一个城市，即三个标号为A的结点所对应状态相同，但是这三个结点在搜索树中却是不同结点，因为它们分别代表了从初始状态出发到达城市A的三条不同路径。这三个结点表示的路径分别为：A→B→A、A→D→A和A→E→A。因此需要注意的是，在搜索树中，同一个标号一定表示相同的状态，其含义为智能体当前所在的城市，但是一个标号可能有多个的结点与之对应，不同的结点对应了从初始状态出发的不同路径。

图3.2 搜索树示例



可见，搜索树记录了从根结点出发目前所有探索过的路径。搜索算法可以被看成是一个构建搜索树的过程，从根结点（初始状态）开始，不断展开每个结点的后继结点，直到某个结点通过了目标测试。

3.1.2 搜索算法的评价指标

在深入讨论搜索算法之前，有必要先明确搜索算法的评价标准，以便比较不同搜索策略的性能差异。常见的评判标准有如下四个。

- 完备性：当问题存在解时，算法是否能保证找到一个解。当然，这个解可能不是最优解。
- 最优性：搜索算法是否能保证找到的第一个解是最优解。
- 时间复杂度：找到一个解所需时间。
- 空间复杂度：在算法的运行过程中需要消耗的内存量。

完备性和最优性刻画了算法找到解的能力以及所求的解的质量，时间复杂度和空间复杂度衡量了算法的资源消耗，它们通常用$\mathcal{O}$符号（big O notation）来描述。

在本章中，时间复杂度将通过扩展的结点数量，空间复杂度将通过算法同时记录的结点数量来衡量。尽管有一些被忽略的因素也会影响这两个复杂度，例如从集合中找出最小值的代价。

具体而言，搜索算法的复杂度可能和以下这些变量有关：分支因子b，即搜索树中每个结点最大的分支数目；最浅的目标结点深度d，即搜索树中最早出现的目标结点所在层数；搜索树中路径的最大可能长度m；状态空间的大小n。表3.1给出了这些变量的含义。

表3.1 搜索树中用于估计复杂度的变量含义

符号	含义
b	分支因子，即搜索树中每个结点最大的分支数目
d	根结点到最浅的目标结点的路径长度
m	搜索树中路径的最大可能长度
n	状态空间中状态的数量

3.1.3 搜索算法框架

一个简单的搜索算法框架如算法3.1所示。在算法中，集合$\mathcal{F}$用于保存搜索树中可用于下一步探索的所有候选结点，这个集合被称为**边缘**

（fringe）**集合**，有时也被叫做**开表**（open list）。算法中有两个函数需要进一步说明，其中函数pick_from用于从边缘集合中选出一个结点元素，函数successor_nodes用于计算当前结点需要探索的后继结点（孩子结点）。

在算法3.1中，第1行对边缘集合进行初始化，使其只包含根结点。第3行从当前边缘集合中选出结点n，第4行将结点n从边缘集合中去除，第5 ~ 7行判断取出的结点n是否通过目标测试，如果通过目标测试，则返回该结点所对应的从根结点抵达其的路径，注意如果此时不停止算法而是继续搜索下去，有可能通过花费更多时间找到代价更小的解；第8行将结点n的后继结点加入边缘集合。第3 ~ 8行的一系列操作通常被称为对结点n进行**扩展**（expand）。某个非目标结点被扩展意味着该结点从边缘集合$\mathcal{F}$中被取出（然后从该集合中删除），接着将该结点的后继结点放入边缘集合$\mathcal{F}$。

算法 3.1　简单的搜索算法框架

函数：TreeSearch（）
输入：根结点root，结点选择函数pick_from， 后继结点计算函数successor_nodes 输出：从初始状态到终止状态的路径
1　$\mathcal{F}\leftarrow${根结点} 2　**while** $\mathcal{F}\neq\varnothing$ **do** 3　　$n\leftarrow$pick_from($\mathcal{F}$) 4　　$\mathcal{F}\leftarrow\mathcal{F}-\{n\}$ 5　　**if** goal_test(n) **then** 6　　　**return** *n.path* 7　　**end** 8　　$\mathcal{F}\leftarrow\mathcal{F}\cup$successor_nodes($n$) 9　**end**

函数pick_from决定了扩展结点的顺序，函数successor_nodes决定了哪些结点可被放入边缘集合以在今后被扩展。在这一章的讨论中将会看到，如何扩展结点往往是搜索算法要解决的核心问题。

可以有多种方法从边缘集合中取出结点来进行扩展。当每次从边缘集合中取出边缘集合中最上层（最浅）的结点时，该算法被称为广度优先搜索（breadth first search）；当每次从边缘集合中取出搜索树最下层（最深）的结点时，该算法被称为深度优先搜索（depth first search）。除此之外还有其他取出策略，但这些并不是本章的重点，这里不再赘述。

一个结点的后继结点可如下定义：首先找出该结点，然后找出与该结点通过一次状态转移可到达的相邻结点，所找到的每个相邻结点均是该结点所对应后继结点。如果对于每个结点都扩展其所有后继结点，便会得到如图3.2中所示的搜索树。

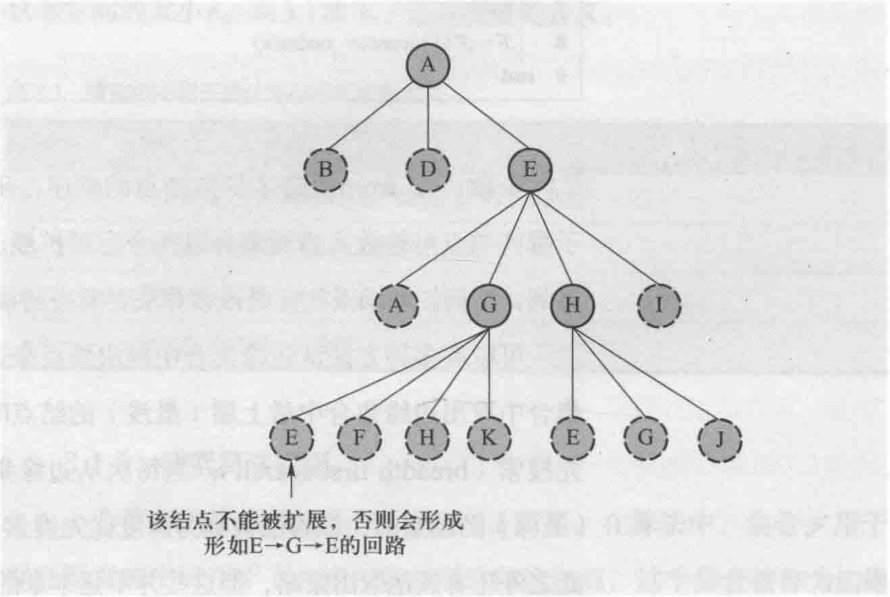
实际上，给定当前结点，并不是其所有的后继结点都值得被探索。在某些情况下，主动放弃一些后继结点能够提高搜索效率而不会影响最终搜索结果，甚至能解决无限循环（即算法不停机）问题。放弃扩展部分结点的做法被称为**剪枝**（pruning），其对应的搜索算法被称为剪枝搜索。

注意到图3.2中最左侧的路径为A→B→A→B→…，这意味着在某些搜索策略下（如深度优先搜索），算法可能会沿着搜索树的左侧路径在状态A和状态B之间陷入无限循环，即出现环路或回路，搜索算法无法终止，因此算法不具有完备性。

为了解决算法不完备的问题，一个直观办法是对可能产生环路的结点进行剪枝。举例来说，假设某时刻从根结点出发所构建得到的搜索树如图3.3所示。图中实线结点表示已经被扩展过，也可以说已经在搜索树中，虚线结点为边缘集合中的结点。

显然，对于图3.3中路径A→E→G而言，当结点G被扩展时，此时结点G被从边缘集合中取出并删除，结点G的后续结点{E, F, H, K}被放入边缘集合。对于所得新的边缘集合，第四层最左端的结点E不应该被拓展，否则会出现从E出发最后回到E的环路（E→G→E）。

图3.3 正在构建中的搜索树



由于不考虑代价为负值的环路，因此给定任意一条包含环路的路径，必然能找到另一条具有同样起点和终点、但是不存在环路的不同路径来代替该路径，且后者具有更小或相同的代价（这里相同代价意味着存在零值回路），因此排除环路对算法找到最优解不产生任何负面影响。另一方面，如果状态数量有限，那么对于不包含环路的路径而言，其长度也必然是有限的，即搜索树中路径的最大可能长度m取值是有限的，可进一步推导出搜索树中结点个数有限，因此算法必然能够在有限时间内终止。

总的来说，排除环路能够有效地解决算法不停机问题，且保留了算法找到最优解的可能性，能够得到最优解，因此解决因环路存在所产生的不完备问题，并不影响算法取得最优结果。

3.1.4 树搜索和图搜索

一般而言，可以采用**图搜索**（graph search）和**树搜索**（tree search）两种方法来完成搜索。对于最短路径搜索这个问题，在以搜索图结构为基础的图搜索中，搜索算法认为一个状态（城市）只能对应搜索图中的一个结点。在以搜索树结构为基础的树搜索中，同一个状态（即同一个城市）可以多次出现在搜索树中，因为相同状态（城市）可以同时出现在搜索树中代表不同结点。

在实际操作中，也可以依据搜索树来构建图搜索算法，即在树搜索的基础上，从边缘集合中剪枝去除那些会导致搜索树中出现重复状态（同一城市）结点。可见图搜索算法可以从搜索图（动机）和搜索树（实现）两个角度来理解，为了保证和树搜索算法思路的一致性，在没有特别说明的情况下，本章默认从搜索树的角度展开对图搜索的讨论。

仍然以图3.3为例。在图搜索中，由于同一个状态（城市）不能出现两次，因此当结点G被扩展而产生A→E→G路径、且结点G的后续结点{E, F, H, K}被放入边缘集合后，图3.3中第四层最左端的结点E不应该被扩展，否则会形成A→E→G→E路径，而导致状态E（城市E）出现了两次。

从这个例子不难归纳，在图搜索策略下，在边缘集合中所有产生环路的结点都要被剪枝。显然，图搜索算法虽然不允许环路的存在，但也不会排除所有潜在的可行解。因此在状态数量有限情况下，采用图搜索策略的算法是完备的。

值得注意的是，结点G的后续结点{E, F, H, K}中H也不能被扩展，否则会得到A→E→G→H路径，而搜索树中已经有路径A→E→H，从而使得H重复出现。为了便于区分，把类似A→E→G→E路径中的E和A→E→G→H中的H这样的边缘结点称为不可扩展边缘结点，而其余边缘结点称为可扩展边缘结点。

算法3.2　图搜索算法

函数：GraphSearch( )

输入：根结点root，结点选择函数pick_from，
后继结点计算函数successor_nodes
输出：从初始状态到终止状态的路径

```
1  F←{根结点}
2  C←Ø
3  while F ≠ Ø do
4  |  n←pick_from(F)
5  |  F←F-{n}
6  |  if goal_test(n) then
7  |  |  return n.path
8  |  end
9  |  if n.state ∉ C then
10 |  |  C ← C ∪ {n.state}
11 |  |  F←F ∪ successor_nodes(n)
12 |  end
13 end
```

算法3.1通常用于描述树搜索算法，而算法3.2给出了图搜索算法的流程。图搜索算法和树搜索算法不同之处在于，图搜索算法维护了一个集合$\mathcal{C}$，用于存储所有被扩展过结点的状态，这个集合被称为**闭表**（closed list）。图搜索算法在第9 ~ 12行来判断从当前边缘结点集合中取出结点的状态是否已经在闭表中，如果是则剪枝该结点（即不做任何操作），如果不在闭表中，则将其加入闭表，同时将其后继结点加入边缘集合。

显然，图搜索和树搜索相比扩展的结点更少，因此其时间复杂度和空间复杂度均比树搜索更优。例如，在树搜索下，广度优先搜索的时间和空间复杂度均为$O(b^{d+1})$，而在图搜索下，由于每个状态至多一次被加入搜索树，因此其时间复杂度和空间复杂度为$O(bn)$，其中b、d、n的含义见表3.1。由于状态空间的大小n常常远小于b^d，因此图搜索能够带来更好的效率提升。

然而图搜索也存在问题，有些情况它会破坏搜索算法原本的最优性。

假设图3.1中，E–H和H–G之间的单步代价变为1，那么此时从A到K的最短路径为A→E→H→G→K。若按照图搜索算法得到了形如图3.3中所示搜索树时，位于搜索树第三层之结点H后续结点{E, G, J}中G结点将被剪枝，则导致最短路径无法被探索到。

由于存在这个问题，有些原本能够保证得到最优解的树搜索算法，在改用图搜索时可能得到的解不再是最优的。因此在采用图搜索时，可能要针对具体算法添加更严格的条件限制或调整算法的流程来保证其最优性。

3.2 启发式搜索

3.1节提到的搜索算法，如深度优先搜索和广度优先搜索，是根据问题定义中提供的信息展开探索的，但是在有些情况下，还知道一些能够辅助算法做出决策的额外信息。利用这些辅助信息的搜索算法被称为**有信息**（informed）**搜索**，或**启发式**（heuristic）**搜索**，这些辅助信息被称为启发信息。

相对地，像深度优先搜索和广度优先搜索这样不利用额外信息的算法被称为**无信息**（uninformed）**搜索**。本节将介绍两种最有代表性的启发式搜索算法，分别是贪婪最佳优先搜索算法（greedy best-first search）[3]和A*算法[4]。

3.2.1 启发函数与评价函数

启发信息通常被形式化为一个关于结点的函数$h(n)$，其用于估计结点n距离达成目标还需付出多少代价，这个函数往往被称为**启发函数**（heuristic function）。由于启发函数是用来估计结点之间的距离，因此它通常是非负的。但正如在后文中将看到的，若启发函数存在一些负值时，有时并不会影响搜索算法性质。在图3.1的问题中，启发函数可以定义为以一定速度通过任意两个城市之间直线距离所需时间。在利用启发信息来辅助搜索时，常见的做法是使用这些信息来规定从边缘集合挑选结点次序，即更改算法3.1中pick_from函数。

对于边缘集合中的任意结点n，函数$f(n)$决定了搜索算法扩展结点n

的优先度，这个函数被称为**评价函数**（evaluation function）。我们规定：边缘集合中某个结点的评价函数值越小，其被挑选的优先级越高。在深度优先搜索中，结点n的$f(n)$值可定义为该结点深度的倒数；在广度优先搜索中，结点n的$f(n)$值可以定义为该结点的深度。

3.2.2 贪婪最佳优先搜索

不难想到，启发信息可以用于计算$f(n)$函数值，以此来影响算法的流程。一种直观的想法是，优先扩展那些距离目标近（启发函数取值小）的结点，即令$f(n)=h(n)$，其中$h(n)$为结点n的启发函数值，其用来计算结点n到达目标结点所付出代价。令评价函数取值和启发函数取值相等的启发式搜索算法被称为贪婪最佳优先搜索算法（greedy best-first search）。

在图3.1的寻找最短路径问题中，假设状态相同的结点拥有相同的启发函数值，该值表示从当前状态到终止状态（即城市K）开销的时间的估计值，假设每个状态（城市）对应的启发函数取值如表3.2所示。

表3.2 每个状态（城市）所对应启发函数取值

状态	A	B	C	D	E	F	G	H	I	J	K	L
$h(n)$	13	10	6	12	7	8	5	3	6	3	0	6

根据表3.2，贪婪最佳优先搜索的过程如图3.4所示。图中仍然用实线结点表示已经被扩展过的结点，虚线结点表示边缘结点。边缘结点下的数字表示该结点所对应评价函数取值。

算法首先扩展根结点（起始结点）A，得到与A相邻、评价函数取值分别为10、12和7的三个边缘结点B、D和E。算法从中选择离目标结点代价最小的结点E进行扩展，此时评价函数取值为3、5、6的H、G和I三个结点被加入边缘集合，算法继续选择图中所示边缘集合中评价函数取值最小的H结点进行扩展。以此类推，最后算法找到了一条从起始结点到终点结点的路径A→E→H→J→K，但这条路径并不是最短路径，实际上最短路径为A→E→G→K。

图3.4中的算法采用了排除环路的剪枝方法，因此它是完备的，即总是可以找到一个解。但是该算法并不是最优的，按照图3.4的求解过程无法找到最优解。

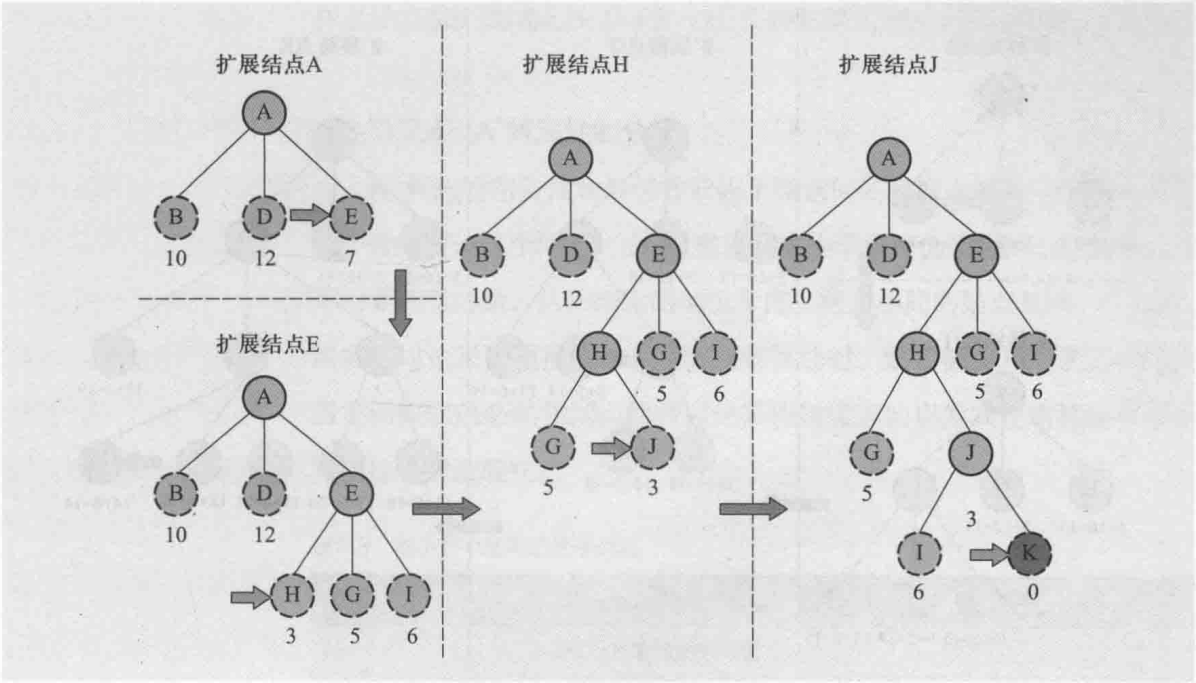


图3.4 贪婪最佳优先搜索的过程

在最坏的情况下，贪婪最佳优先搜索算法时间和空间复杂度均为$O(b^m)$，其中b和m分别表示搜索树的分支因子和最大深度（见表3.1）。当然，如果不排除环路，贪婪最佳优先算法是不完备的，读者不妨自行尝试使用有环路的贪婪最佳优先搜索算法来求解从B到K的路径。

3.2.3 A*搜索

贪婪最佳优先算法的问题在于算法总是贪婪地扩展与目标结点之间代价最小的结点，而不考虑从初始结点到达该结点所需的代价。

在图3.4中，虽然第四步扩展的J结点距离目标结点的估计代价很小（取值为3），但从起始结点到达该结点的路径A→E→H→J代价为17，已经超过了实际上最短路径的代价（值为14），因此路径A→E→H→J并不是一个好的探索方向。

为了解决这个问题，一种直观方案可把从起始结点到当前结点的路径代价在评价函数中予以考虑。这样，定义一个结点n的函数$g(n)$，$g(n)$取值为从起始结点到结点n的路径代价，于是评价函数可定义为$f(n)=g(n)+h(n)$。按照这个定义所得到的搜索算法被称为A*算法，其评价函数可认为是从起始结点到结点n的实际最小代价与从结点n到目标结点的后续估计最小代价之和，如下式所示。

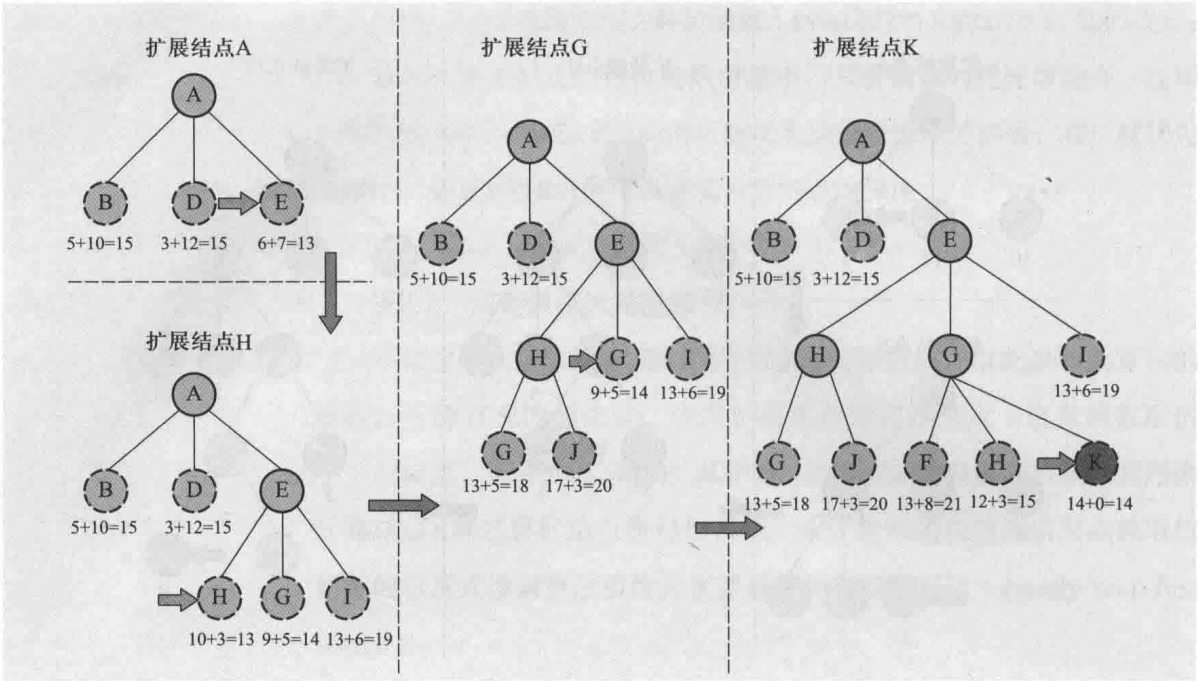


图3.5 A*算法的搜索过程

$$\underbrace{f(n)}_{\text{评价函数}} = \underbrace{g(n)}_{\substack{\text{起始结点到结点}n\text{代价}\\(\text{当前最小代价})}} + \underbrace{h(n)}_{\substack{\text{结点}n\text{到目标结点代价}\\(\text{后续估计最小代价})}}$$

根据表3.2给出的启发函数值，A*算法的搜索过程如图3.5所示。图中仍然用实线结点表示已经被扩展过的结点，虚线结点表示边缘结点。每个边缘结点下的等式即为$g(n)+h(n)=f(n)$的取值。

算法首先扩展根结点（起始结点）A，得到三个边缘结点B、D和E。其中路径A→B的实际代价为5，结点B的启发函数取值为10，因此结点B的评价函数取值为5 + 10 = 15。同理可算出D和E的评价函数值分别为15和13。算法选择评价函数值最小的E结点进行扩展，得到评价值分别为13、14、19的三个新的边缘结点H、G、I。算法从当前边缘集合中选择评价函数值最小的H扩展，得到评价函数值为18和20的两个结点。到目前为止，算法的流程和贪婪最佳优先搜索保持一致，两者区别体现在下一步操作上。

不同于贪婪最佳优先搜索选择扩展结点J，A*算法选择扩展评价函数值最小的第三层中结点G（A→E→G），接着又扩展了评价函数值最小的K，顺利到达终止状态。此处扩展次序的差异使得A*算法能够顺利找到最短路径A→E→G→K，而贪婪最佳优先搜索不能。

值得注意的是，每次从边缘结点集合中所扩展的结点并非总是当前结点的相邻后继结点，如在图3.5中，所扩展的第三层结点G（A→E→G）

并不是先前扩展结点H(A→E→H)在搜索树上的相邻后继结点。

*3.2.4 A*算法性能分析

A*算法的完备性和最优性取决于搜索问题和启发函数的性质。为了便于详细讨论这个问题，表3.3整理了本小节中将会使用的一些符号。另外需要注意的是，从图搜索的动机考虑，状态相同的结点是同一个结点，因此在讨论采用图搜索的启发式搜索算法时，需要假设相同状态的结点启发函数取值必然相同，而在讨论采用树搜索的启发式搜索算法时并不要求这个假设成立。

表3.3 本小节中使用的若干符号

符号	含义
$h(n)$	结点n的启发函数取值
$g(n)$	从起始结点到结点n所对应路径的代价
$f(n)$	结点n的评价函数取值
$c(n, a, n')$	从结点n执行动作a到达结点n'的单步代价
$h^*(n)$	从结点n出发到达终止结点的最小代价

设计一个良好的启发函数需要满足如下两种性质。

- **可容性**（admissible）[①]：对于任意结点n，有$h(n) \leqslant h^*(n)$，如果n是目标结点，则有$h(n)=0$。如表3.3所示，$h^*(n)$是从结点n出发到达终止结点所付出的最小代价。可以这样理解满足可容性的启发函数，启发函数不会过高估计（over-estimate）从结点n到终止结点所应该付出的代价（即估计代价小于等于实际代价）。在图3.1的问题中，对每个结点而言，其状态（城市）到目标状态（城市）K之间的行驶距离不会小于两城之间的距离，因此列车以最大行驶速度通过直线距离花费的时间必然不会超过实际代价，将该时间作为启发函数值，是满足可容性条件的。

- **一致性**（consistency）：启发函数的一致性指满足条件$h(n) \leqslant c(n, a, n')+h(n')$，这里$c(n, a, n')$表示结点$n$通过动作$a$到达其相应的后继结点$n'$的代价。在图3.1中，假设结点$n$和$n'$对应的状态（城市）为$s$和$s'$，如果将单步代价$c(n, a, n')$重新定义为$s$和$s'$在二维平面上的欧氏距离，则根据欧氏距离三角不等式原则，城市s到目标城市K之间直线距

① 在许多文献中可容性要求启发函数值非负，但这个条件对保证A*算法性质而言不是必需的。

离一定小于等于从城市s到其相邻城市s'的直线距离与城市s'到目标城市K之间直线距离之和，此时，将当前状态（城市）与目标状态（城市）K之间直线距离作为启发函数值，则启发函数满足一致性条件。

一致性是一个比可容性更强的性质，即一个启发函数如果满足一致性条件，且终止结点的启发函数值均为0，那么该启发函数必然满足可容性条件。这并不难证明，考虑以任意结点n_1作为根结点的搜索树子树，如果这棵子树中不存在终止结点，那么可以认为$h^*(n_1)$取值无穷大，因此启发函数在n_1上满足可容性。如果这棵子树中存在终止结点，假设能够找出一条从n_1到达终止结点代价最小的有限长（状态转移次数有限）路径。注意有些情况下可能不存在这样的路径，例如，存在代价为负值的回路的情形，或最短路径代价有限而长度无限的情形等，这些特殊情况此处不予考虑。

如图3.6中以n_1为根结点的子树，假设从该结点到达终止结点代价最小的路径为$n_1 \to n_2 \to \cdots \to \mathrm{K}$。由于$h(\mathrm{K})=0$且启发函数满足一致性条件，因此存在：

$$\begin{aligned} h(n_1) &\leqslant h(n_2)+c(n_1, a_1, n_2) \\ &\leqslant h(n_3)+c(n_2, a_2, n_3)+c(n_1, a_1, n_2) \\ &\leqslant \cdots \\ &\leqslant c(n_1, a_1, n_2)+c(n_2, a_2, n_3)+\cdots+c(n_t, a_t, \mathrm{K}) \\ &= h^*(n_1) \end{aligned}$$

即满足一致性条件的启发函数一定满足可容性条件。

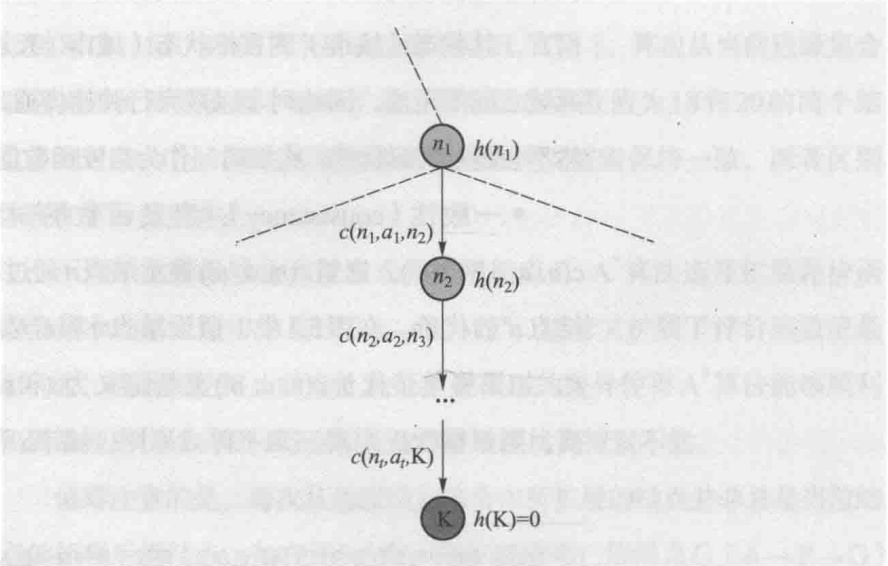


图3.6 从n_1结点到终止结点的最短路径

在这些准备知识的基础上，接下来对A*算法的性能进行详细探讨。在一些常见的搜索问题中，状态数量是有限的，此时图搜索A*算法和排除环路的树搜索A*均是完备的，即一定能够找到一个解。在更普遍的情况下，如果所求解问题和启发函数满足以下条件，则A*算法是完备的。

- 搜索树中分支数量是有限的，即每个结点的后继结点数量是有限的。
- 单步代价的下界是一个正数。
- 启发函数有下界。

文献[4]中给出了启发函数下界为0时A*算法完备性的证明，但这个证明不难推广到下界取值为任意M的情况。接下来的讨论将在A*算法是完备的前提下进行，此时最关心的问题是：什么情况下A*算法满足最优性呢？

实际上，如果启发函数是可容的，那么树搜索的A*算法满足最优性。启发函数满足可容性时，假设树搜索的A*算法找到的第一个终止结点为n。对于此时边缘集合中任意结点n'，根据算法每次扩展时的策略，即选择评价函数取值最小的边缘结点，有$f(n) \leqslant f(n')$。由于A*算法对评价函数定义为$f(n)=g(n)+h(n)$，且$h(n)=0$，有$f(n)=g(n)+h(n)=g(n)$，$f(n')=g(n')+h(n')$，综合可得：

$$g(n) \leqslant g(n')+h(n') \leqslant g(n')+h^*(n')$$

其中$g(n)$为从起始结点到结点n所对应路径的代价，$g(n')+h^*(n')$表示从初始结点出发经过结点n'到达终止结点的最小代价。也就是说此时扩展其他任何一个边缘结点都不可能找到比结点n所对应路径代价更小的路径，因此结点n对应的是一条最短路径，即算法满足最优性。

如果A*算法采用的是图搜索策略，那么即使启发函数满足可容性条件，也不能保证算法最优性。这是因为，如果有多个结点对应同一个状态（即存在多条到达同一个城市的路径），图搜索算法只会扩展其中的一个结点，而剪枝其余的结点，然而最短路径有可能经过其中某些被剪枝结点，导致算法无法找到这些最短路径。为了构造一个图搜索A*算法失败的例子，可把表3.2中D、F和G状态的启发函数值改为0。此时启发函数仍然满足可容性，但图搜索的A*算法会优先扩展结点A→D→F→G，而放弃最短路径经过的结点A→E→G。

为了使A*算法保持最优性，一种方法是对图搜索算法进行修改。为了便于理解，这里从搜索图的角度来解释这个修改方案。当算法从不

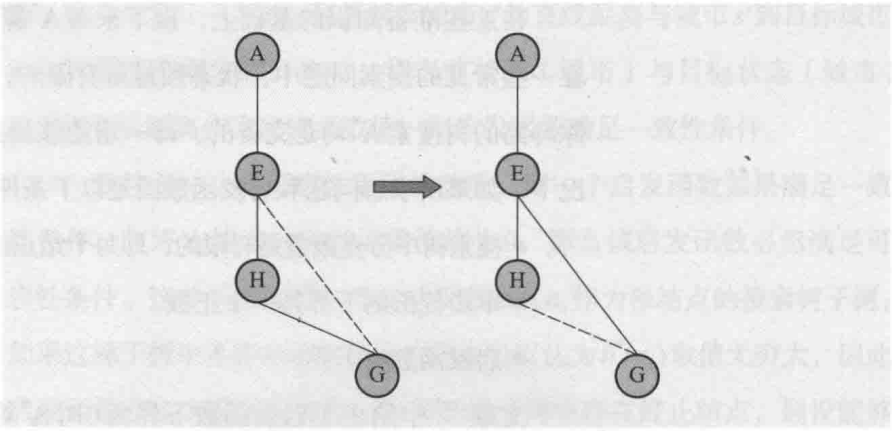


图3.7 修改后的图搜索A*算法扩展A→E→G结点

同路径扩展同一结点时，不保留最先探索的那条路径，而是保留代价最小的那条路径。如图3.7所示，有A→E→G和A→E→H→G两条到达G结点的不同路径，其中A→E→H→G已经被探索过了。当算法考虑A→E→G时，原本的图搜索算法应该放弃扩展该结点，但因为A→E→G的代价更小，修改后的算法选择将到达G结点的路径从A→E→H→G替换为A→E→G。在实际操作中，算法通过为每个状态（城市）记录一个前驱状态（城市），来记录当前到达每个状态（城市）的最短路径。在处理通向相同状态的不同路径时，算法会更新当前的前驱状态。

另一种让图搜索A*算法满足最优性的方法是要求启发函数满足一致性。根据前文的证明，满足一致性的启发函数必然满足可容性，因此满足一致性的启发函数同样能保证树搜索A*算法是最优的。

文献[4]给出了在启发函数满足一致性条件时，图搜索A*算法保持最优性的证明，这里对其思路稍加调整，以便配合从搜索树的角度来理解图搜索的A*算法。

引理3.1 启发函数满足一致性条件时，给定一个从搜索树中得到的结点序列，每个结点是前一个结点的后继，那么评价函数对这个序列中的结点取值按照结点出现的先后次序而依次单调非减。

现在考虑序列中结点n，对结点n采取动作a而得到的后继结点n'。由于启发函数满足一致性条件，因此有$h(n) \leqslant c(n, a, n')+h(n')$。于是存在：

$$f(n')=g(n')+h(n')=g(n)+c(n,a,n')+h(n')\geqslant g(n)+h(n)=f(n)$$

即A*算法构建的搜索树中父结点的评价函数值一定小于等于子结点的评价函数值，因此如果结点序列中每个结点均是前一个结点的后继，评价函数对这个序列中的结点取值按照结点出现的先后次序而依次单调

非减。由于这个结论的成立与A*算法扩展结点的策略无关，因此即使这个序列中有一部分结点被图搜索算法剪枝而不在搜索树中，该结论同样成立。

引理3.2 假设状态t加入搜索树时对应的结点为n，在结点n被加入搜索树后，若对应状态t的任何结点n'出现在边缘集合中，那么必然有$g(n) \leqslant g(n')$。

假设在当前搜索树中，结点n将是下一个被加入搜索树的结点，即结点n目前是一个可扩展的边缘结点，可证明此时从根结点到结点n'的路径上必然存在一个可扩展的边缘结点。如果n'在边缘集合中，如图3.8（a）所示，由于n可扩展，因此状态t不在搜索树中，那么n'也是一个可扩展的边缘结点；如果n'不在边缘集合中，如图3.8（b）所示，那么从根结点到n'的路径中必然有一个是可扩展的边缘结点m，否则n'不可能在未来出现在边缘集合中。总的来说，从根结点到n'的路径中必然存在一个可扩展边缘结点m。在特殊情况下，结点m可能是结点n'，也可能是结点n。

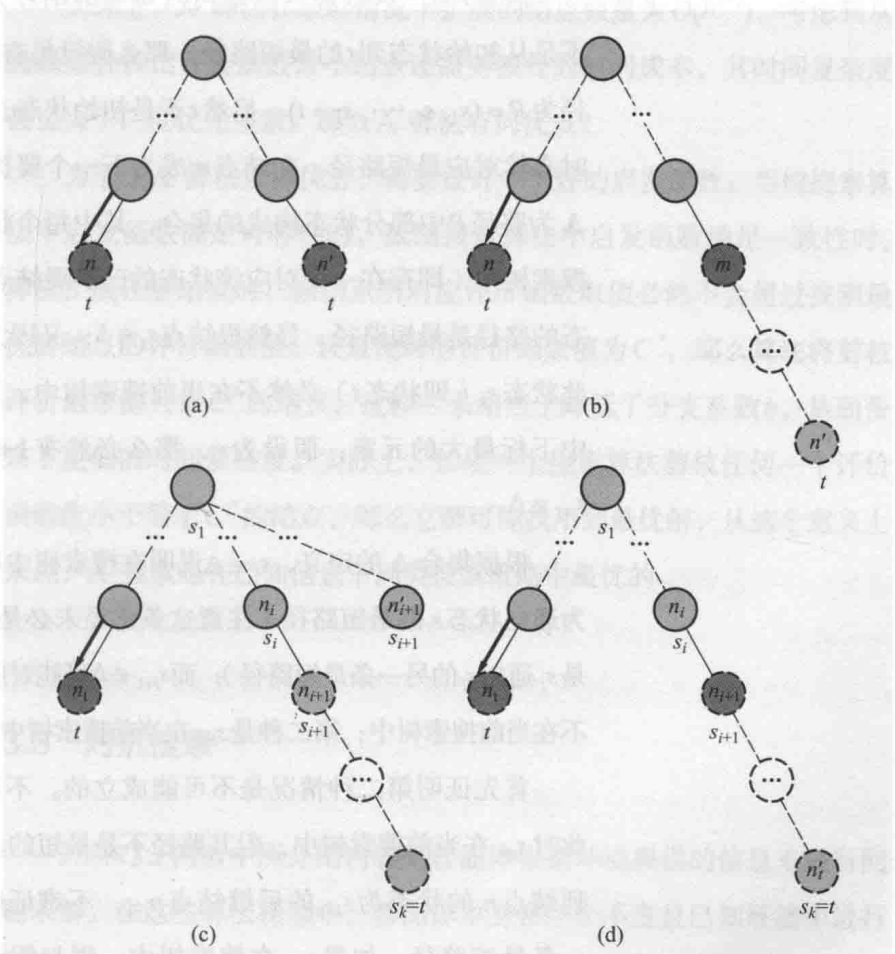


图3.8 图搜索A*算法最优性证明过程示意图，其中已在搜索树中的结点用实线表示，不在搜索树中的结点用虚线表示，可扩展的边缘结点用深灰色标出，结点的代号标注在结点内，结点对应的状态标注在相应结点的下方

根据A*算法选择扩展结点的顺序，有$f(n) \leqslant f(m)$，又根据引理一，由于m和n'在同一条路径中，有$f(m) \leqslant f(n')$，因此$f(n) \leqslant f(n')$。因为n和n'对应同一个状态t，有$h(n) = h(n')$，因此得出结论$g(n) \leqslant g(n')$。这个性质说明，每个状态被加入搜索树时，其路径代价必然小于等于任何将来出现在边缘集合中通向该状态的路径的代价。注意，这个结论并不等价于找到的第一条路径就是最优的（即并不能证明算法最优性），因为不是所有的路径对应的搜索树结点都会出现在边缘集合中的，可能存在一些路径因为图搜索的剪枝而自始至终不会出现在边缘集合中。例如，在图3.5展示的A*算法搜索过程中，假设算法采用的是图搜索，则因为路径A→E→G被扩展，路径A→D→F→G将失去被扩展的机会，因此从起始结点到终止结点的路径A→D→F→G→K永远不会出现在边缘集合中。

算法的最优性：对于任意一个状态t，它第一次被加入搜索树时的路径必然是最短路径。

假设t第一次被加入搜索树时对应的结点为n_t，如果n_t对应的路径不是从初始状态到t的最短路径，那么假设另有从初始状态到t的最短路径为$P = (s_1, s_2, \cdots, s_k = t)$。显然$t$不是初始状态，因为根结点加入搜索树时必然对应最短路径。在结点n_t成为下一个要扩展的结点时，定义集合Δ为路径P中部分状态构成的集合，其中每个状态满足：该状态在当前搜索树中（即存在一个对应该状态的已扩展结点）且搜索树中通向该状态的路径是最短路径，显然根结点$s_1 \in \Delta$；又因为结点n_t可以被扩展，因此状态s_k（即状态t）必然不在当前搜索树中，所以有$s_k \notin \Delta$。取Δ集合中下标最大的元素，假设为s_i，那么必然有$1 \leqslant i < k$，因此必然有状态$s_{i+1} \notin \Delta$。

根据集合Δ的定义，$s_i \in \Delta$说明在搜索树中存在一个结点n_i，其路径为通向状态s_i的最短路径（注意这条路径未必是$s_1 \to s_2 \to \cdots \to s_i$，也可能是$s_1$通向$s_i$的另一条最短路径）；而$s_{i+1} \notin \Delta$可能对应两种情况：第一种是$s_{i+1}$不在当前搜索树中；第二种是$s_{i+1}$在当前搜索树中，但其路径不是最短的。

首先证明第二种情况是不可能成立的。不妨假设第二种情况成立，此时s_{i+1}在当前搜索树中，但其路径不是最短的。如图3.8（c）所示，找到结点n_i的状态为s_{i+1}的后继结点n_{i+1}，不难证明该结点对应通向s_{i+1}的一条最短路径。如果n_{i+1}在搜索树中，则与假设矛盾，因此n_{i+1}必然不

在搜索树中，即n_{i+1}是一个边缘结点。根据假设，状态s_{i+1}存在于搜索树中，因此必然能找到搜索树中的结点n'_{i+1}对应状态s_{i+1}，根据引理二有$g(n'_{i+1}) \leqslant g(n_{i+1})$。由于$n_{i+1}$对应通往$s_{i+1}$的最短路径，即$g(n'_{i+1}) \geqslant g(n_{i+1})$，综上可知$n'_{i+1}$也必然对应一条最短路径，这与假设矛盾。至此已证明第二种情况是不可能成立的，根据排除法可知s_{i+1}必然不在当前搜索树中，因此n_{i+1}是一个可扩展的边缘结点。

如图3.8（d）所示，当n_t为下一步扩展的结点时，由于n_{i+1}为一个可扩展的边缘结点，根据A*算法的扩展策略，有$f(n_t) \leqslant f(n_{i+1})$。从$n_{i+1}$开始沿着路径$(s_{i+2}, \cdots, s_k = t)$找到结点$n'_t$，因为$n_{i+1}$为$n'_t$的祖先结点，根据引理一，有$f(n_{i+1}) \leqslant f(n'_t)$，因此有$f(n_t) \leqslant f(n'_t)$。又因为$n_t$和$n'_t$两个结点都对应状态$t$，有相同的启发函数值，因此$g(n_t) \leqslant g(n'_t)$。另外，由于$n_{i+1}$对应到$s_{i+1}$的最短路径，显然从$n_{i+1}$出发沿着最短路径找到的$n'_t$对应通往状态$t$的一条最短路径，即$g(n'_t) \leqslant g(n_t)$。结合两个不等式有$g(n'_t) = g(n_t)$，因此$n_t$对应的路径必然是最短的，即算法满足最优性。

实际上，A*算法在最坏情况下扩展的结点数量为$O(b^{d+1})$，考虑到从边缘集合找出评价函数最小结点还需要额外的时间成本，其时间复杂度甚至大于广度优先搜索。那么A*算法有何优点？

为了让A*算法发挥优势，需要设计一个好的启发函数。当树搜索算法中启发函数满足可容性时，或图搜索算法中启发函数满足一致性时，算法扩展任意结点时，该结点所对应评价函数取值必然不会超过找到最优解结点的评价函数值。设最优解的评价函数值为C^*，那么算法将剪枝评价函数值大于C^*的结点，这样一来相当于降低了分支系数b，从而带来了更低的时间复杂度。实际上，如果一个搜索算法剪枝任何一个评价函数值小于等于C^*的结点，那么它都可能找不到最优解，从这个意义上来说，A*搜索是在已知信息下同类搜索策略中最优的。

3.3 对抗搜索

3.1和3.2两节中所介绍内容是智能体根据环境提供的信息来进行问题求解。在这些算法模型中，智能体本身在一个不变且已知环境中进行搜索。然而在现实中，智能体往往不是孤立存在的，它进行问题求解活

动时不能避免与其他智能体打交道。一个简单的例子是在进行多人游戏时，每个角色在求解最佳行动方案时不可能不将其他角色行动考虑在内。

本节将介绍一种特殊情况下的多智能体搜索问题，即在信息确定、全局可观察、竞争对手轮流行动、输赢收益零和（zero-sum）假设下的两人博弈问题。其中零和是博弈论的一个概念，属非合作博弈范畴，其指参与博弈各方在严格竞争下，一方获得收益必然意味着另一方遭受损失，博弈各方收益和损失相加的总和永远为“零”，使得双方不存在合作的可能。

求解该问题的算法称为对抗搜索（adversarial search），也称为博弈搜索（game search）。在这个搜索框架中，两个智能体同处于一个竞争的环境，智能体之间通过竞争来实现各自相反利益，即一方想最大化自身的利益，而另一方则想最小化对手的利益。通俗地说，对抗搜索的过程就是两个智能体各自选择对自己有利的策略。

与3.1节中对问题进行形式化描述一致，对二人博弈问题求解也需要先形式化描述如下元素。

- **状态**：状态s包括当前的游戏局面和当前行动的智能体。初始状态s_0包括初始游戏局面和初始行动的玩家。由于本节讨论的问题假设两个竞争对手轮流行动，因此第i步行动的玩家是确定的，函数player(s)给出状态s下行动的智能体。
- **动作**：给定状态s，动作指的是player(s)在当前局面下可以采取的操作a，记动作集合为actions(s)。
- **状态转移**：给定状态s和动作$a \in$ actions(s)，状态转移函数result(s, a)决定了在s状态采取a动作后所得后继状态。
- **终局状态检测**：终止状态检测函数terminal_test(s)用于测试游戏是否在状态s结束。
- **终局得分**：终局得分utility(s, p)表示在终局状态s时玩家p的得分。在二人零和博弈中，两名玩家的终局得分之和应该是固定的，因此算法只需记录其中一人的终局得分为utility(s)，则另一人的得分可按照零和原则相应算出。

可见，对抗搜索与3.1节描述的搜索问题基本相符，两者最大的不同在于3.1节描述的搜索问题以代价来衡量一条路径的好坏，而对抗搜索则以终局得分来衡量游戏参与者所采取行动的优劣。在3.1节的搜索问题

中，一条路径往往只有一个代价，在对抗搜索中，有多少玩家就有多少终局得分，因为不同玩家对同一条路径的评价标准是不同的。

3.3.1 最小最大搜索

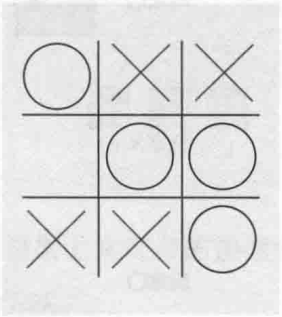


图3.9“井”字棋的盘面，此时执“○”棋子的玩家获胜

最小最大搜索（minimax search）是求解对抗搜索问题的基本算法，该算法交替探索两名玩家的决策，并假设两名玩家在进行决策时总是理性地倾向于最大化自己的得分，这在零和博弈中也等价于最小化对手的得分。

这里以一个简单的两人博弈游戏——“井”字棋（tic-tac-toe）为例来介绍对抗搜索。“井”字棋是一个在3×3个方格所构成棋面上展开的游戏。在游戏中，游戏双方各执一种棋子，分别记为“○”和“×”。双方轮流将一枚棋子落入一个空白方格中，直到有一方的三枚棋子连成一条直线（水平、垂直、对角线）则判为胜利，棋盘方格被填满而无人获胜则判为平局。如图3.9所示，给出了执“○”玩家获胜的一种盘面情况。为了方便表示，以下将执“○”玩家简称玩家0，执“×”玩家简称为玩家1。

假设初始行动的玩家为玩家1。可想而知，如果要交替搜索两名玩家的行动，那么搜索树的第1层为初始状态，玩家1在第1层采取行动，第2层对应玩家1采取了不同行动而得到的分支；玩家0在第2层的采取行动，第3层对应玩家0采取了不同行动而得到的分支；玩家1在第3层采取行动，第4层再次对应玩家1采取不同行动而得到的分支，依此类推。

图3.10展示了按照玩家交替对决模式所画搜索树的一部分，其中在每一层左侧标出了当前行动的玩家。游戏树中每个结点用当前盘面状况来表示，玩家和当前盘面状况组合在一起即为对抗搜索问题中的完整状态。图3.10中给出了三种终局状态，从左到右分别对应玩家1获胜，平局以及玩家0获胜。从玩家1的角度看，将这三种终局状态对应的得分分别为 +1，0和-1。

在这个定义下，由于每个玩家都想要最大化自己的得分，因此轮到玩家1行动时，玩家1会总是选择能够最大化终局得分的搜索树分支，而玩家0总是选择能够最小化终局得分的搜索树分支，最大最小算法的名字即由此而来。

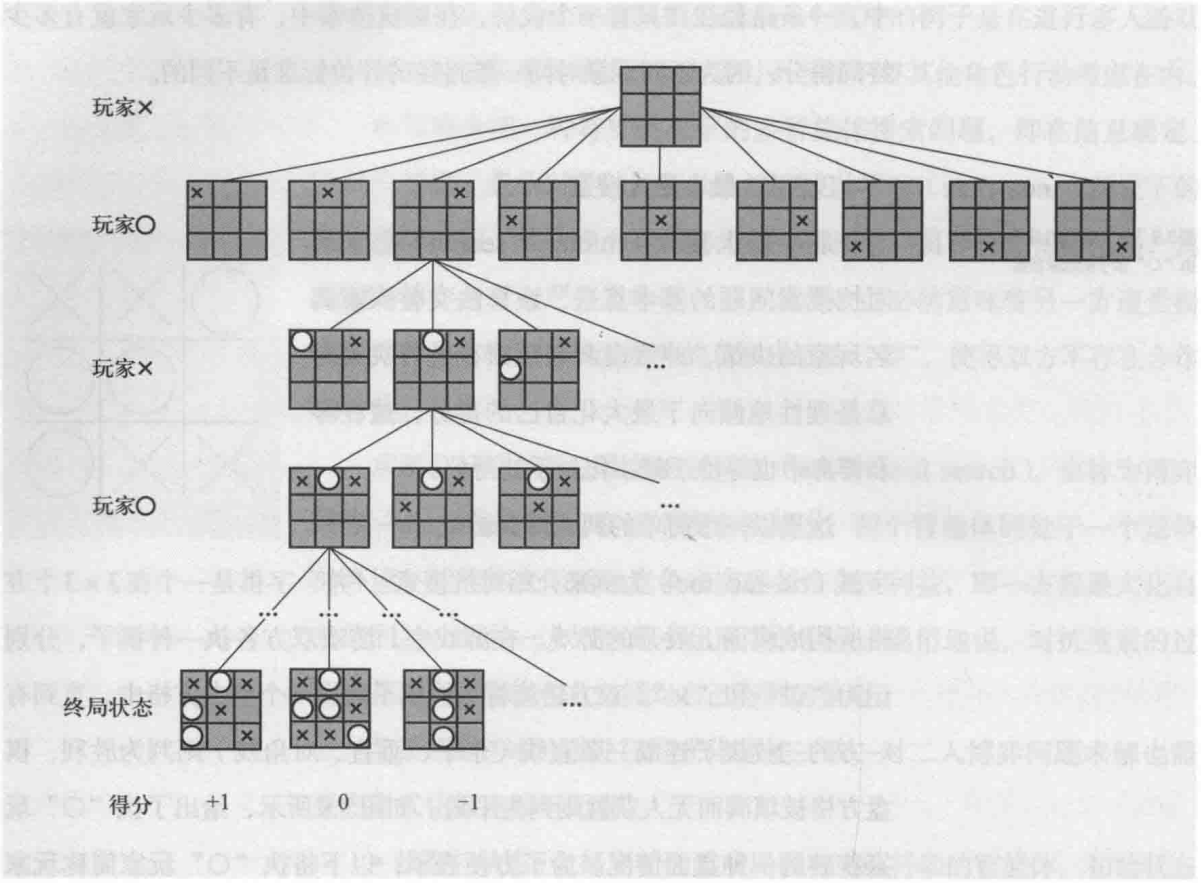


图3.10 “井”字棋所对应的对抗搜索树

为了方便推广到其他游戏，不妨把以最大化得分为目标的玩家称为玩家MAX，把以最小化得分为目标的玩家称为玩家MIN。然而一个搜索树分支可能对应多个终局状态，怎样才能取得最大化或是最小化的终局得分呢？

假设搜索树如图3.11所示，叶子结点表示终局状态，其下方的数字表示终局得分。假设在第二层由玩家MIN行动，那么他必然选择一个动作，该动作使得终局状态得分最低。例如在B结点时，玩家MIN会选择b_1动作，这样的终局得分为3，相比于选择动作b_2和b_3，会给玩家MAX带来最低的得分。同理在结点C和D时，玩家MIN会分别选择动作c_1和d_3。这样一来，第二层的每个结点都有了一个分数，这个分数表示了玩家MIN在第二层结点按照最优策略选择动作后所得终局状态得分。因此，在第一层由玩家MAX行动时，他将会选择能够到达第二层中分数最大结点的动作，即玩家MAX采取动作a_1得到结点B，于是A结点得分和B结点得分同样为3，它表示当两名玩家均按照最优策略行动，玩家MAX能够得到的终局得分。玩家MAX和玩家MIN交替采取的策略在图3.11中由粗箭头方向所示。

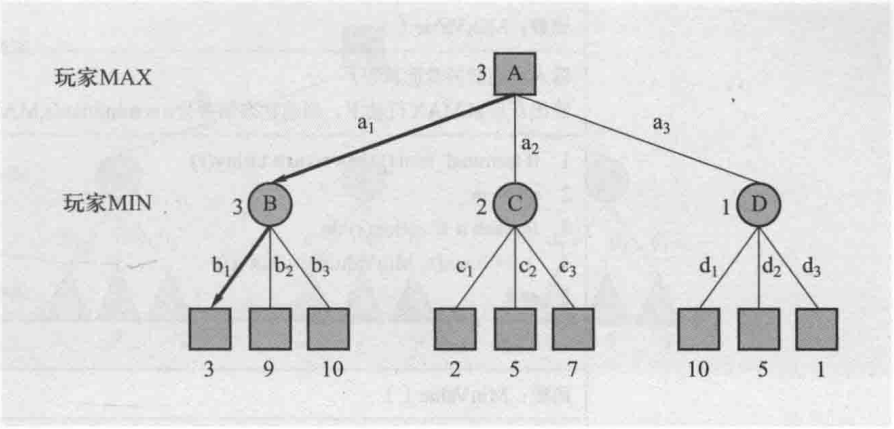


图3.11 玩家MAX和玩家MIN在最优策略下采取动作的示意图

对图3.11中博弈双方策略进行归纳，可以根据上文定义的搜索要素，如下定义计算搜索树中每个结点获得分数的方法。

$$\text{minimax}(s)=\begin{cases}\text{utility}(s), & \text{if terminal_test(s)}\\ \max_{a\in\text{actions}(s)}\text{minimax}(\text{result}(s,a)), & \text{if player}(s)=\text{MAX}\\ \min_{a\in\text{actions}(s)}\text{minimax}(\text{result}(s,a)), & \text{if player}(s)=\text{MIN}\end{cases}\quad(式3.1)$$

最小最大搜索就是在计算搜索树中每个结点分数之后，每一步由玩家根据自己的角色来选择使得分数最大或分数最小的行动。

算法3.3给出了最小最大算法的流程，该算法假设玩家MAX为初始行动玩家。这个假设并不会使问题失去一般性，如果实际上初始行动的是玩家MIN，则可以通过将终局得分取相反数的方法来交换玩家MIN和玩家MAX。函数MinValue和MaxValue递归求解了每个搜索树结点的分数，函数MinimaxDecision根据每个结点的分数求解玩家MAX在当前盘面下所采取的最优动作。

最小最大算法对游戏搜索树执行了一个完整的深度优先搜索。因此，如果搜索树是有限的，那么最小最大算法必然能在有限时间内终止；如果对手总是理性地按照最优策略来交替采取行动，那么最小最大算法得到的解是最优的。根据表3.1中的定义，如果m是游戏树的最大深度，在每个结点最多存在b种有效走法，那么最小最大算法的时间复杂度为$O(b^m)$，空间复杂度为$O(bm)$。

算法 3.3　通过最小最大搜索求解最优动作的算法

函数：MinimaxDecision（）
输入：当前的盘面状态s 输出：玩家MAX行动下，当前最优动作a^*
$a^*\leftarrow\text{argmax}_{a\in\text{action}(s)}\text{MinValue}(\text{result}(s,a))$

函数：MaxValue（）
输入：当前的盘面状态s 输出：玩家MAX行动下，当前状态的得分v = minimax(s,MAX)
1 **if** terminal_test(s) **then return** utility(s) 2 $v \leftarrow -\infty$ 3 **foreach** $a \in$ action(s) **do** 4 $\quad v \leftarrow$ max(v, MinValue(result(s, a)) 5 **end**
函数：MinValue（）
输入：当前的盘面状态s 输出：玩家MIN行动下，当前状态的得分v = minimax(s,MIN)
1 **if** terminal_test(s) **then return** utility(s) 2 $v \leftarrow +\infty$ 3 **foreach** $a \in$ action(s) **do** 4 $\quad v \leftarrow$ min(v, MaxValue(result(s, a))) 5 **end**

3.3.2 Alpha-Beta剪枝算法

和深度优先搜索一样，如果搜索树极大，则最小最大算法无法在合理时间内返回结果。对于比较复杂的游戏如国际象棋等，执行最小最大算法的时间开销将是巨大的。为了减少时间开销，可在最小最大算法中采用一种常见剪枝方法——Alpha-Beta剪枝[5, 8]，以减少搜索空间。

正如3.1小节中对剪枝的定义，Alpha-Beta剪枝搜索算法在最小最大算法中可减少被搜索的结点数，即在保证得到与原最小最大算法同样的搜索结果时，剪去了不影响最终结果的搜索分枝。

将图3.11中搜索树稍微修改为图3.12所示结果。假设叶子结点不代表终局状态，而是各自代表一棵子树，其中搜索算法尚不知道动作c_2和c_3所对应子树在游戏结束时能够给玩家MAX多少收益分数，因此假设收益分数分别为x和y。那么此时玩家MAX在根结点所能得到分数可如下计算：

$$\begin{aligned}\text{minimax}(A) &= \max\big(\min(3, 9, 10), \min(2, x, y), \min(10, 5, 1)\big)\\ &= \max\big(3, \min(2, x, y), 1\big)\end{aligned}$$

由于$\min(2, x, y)$的值小于等于2，所以即使不知道x和y的值，根结点的分数也可以算出来等于3。也就是说，算法没有必要计算动作c_2和c_3对应的两个子树在游戏结束时所得分数，就能决定在根结点采取动作a_1从

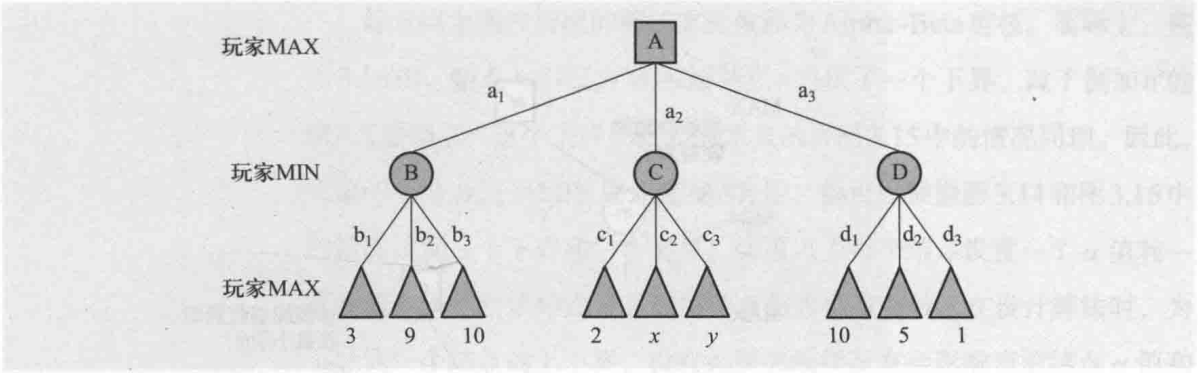


图3.12 一棵对抗搜索树在搜索过程中某时刻的状态示意图

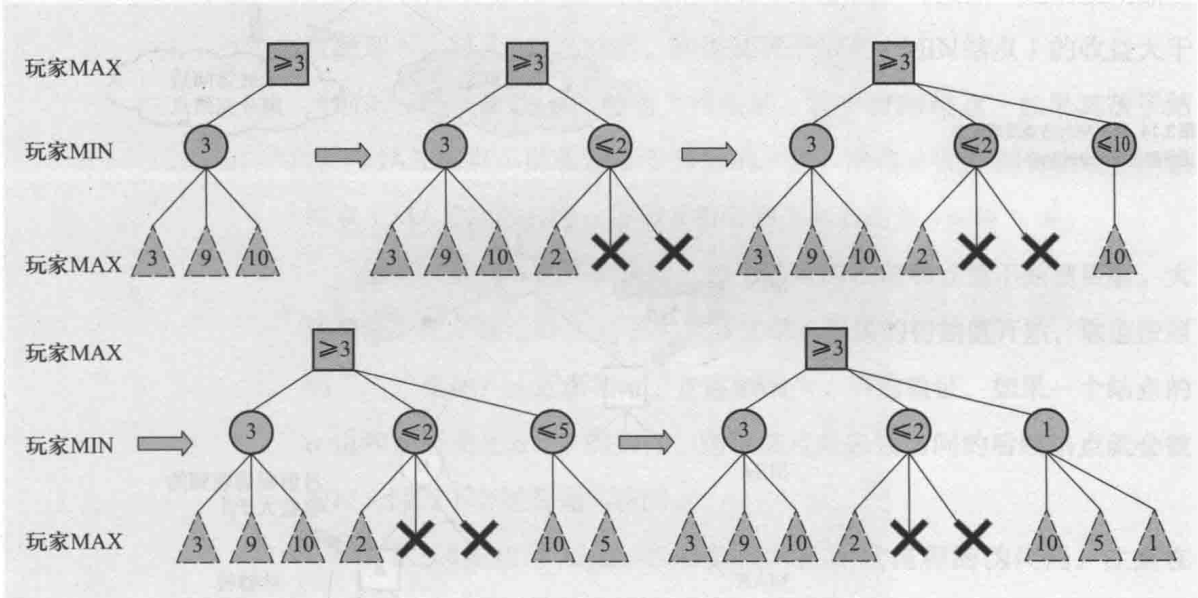


图3.13 存在剪枝的搜索树部分扩展过程

而在游戏结束时可获得收益3，因此动作c_2和c_3所对应子树可以被剪枝。

为了说明剪枝不影响对抗搜索结果，图3.13给出了剪枝过程示意。剪枝的关键在于算法对每个结点有一个预期的取值范围。例如，在计算出B结点的分数为3后，那么算法可以确定根结点A的分数不会小于3。在计算采取了动作c_1后所对应子树的分数为2后，算法能够判断结点C所得分数不会大于2，于是可将尚未访问的结点C的孩子结点均剪枝掉而不再访问，因为结点C最终取值不可能影响根结点A的取值分数，即玩家MAX的最终收益。

可把上述剪枝思路推广到一般情况。假设有一个位于MIN层的结点m，已知该结点能够向其上MAX结点反馈的收益为α（Alpha）。n是与结点m位于同一层的某个兄弟（sibling）结点的后代结点。如果在结点n的后代结点被访问一部分后，直到结点n能够向其上一层MAX结点反馈收益小于α，则结点n的未被访问孩子结点将被剪枝。

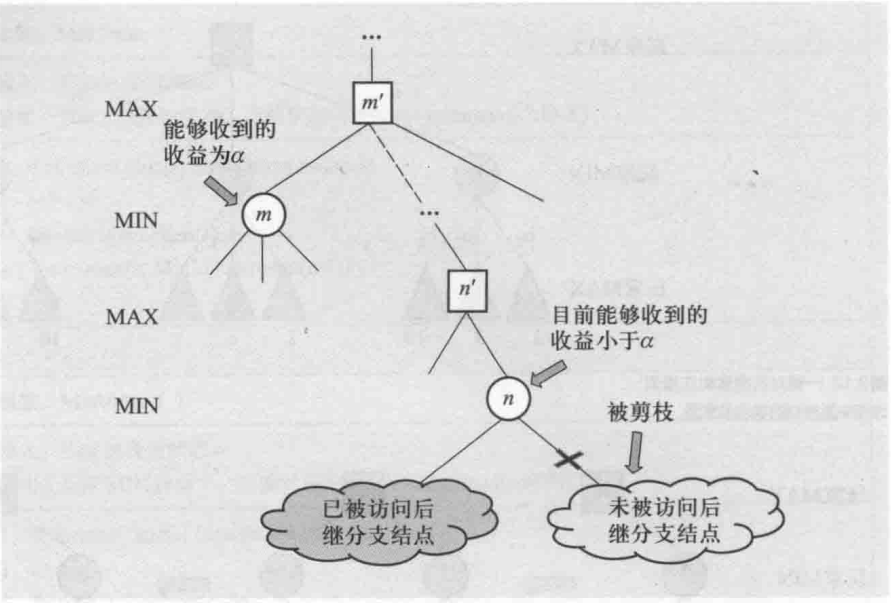


图3.14 基于MIN结点反馈收益进行剪枝（Alpha剪枝）

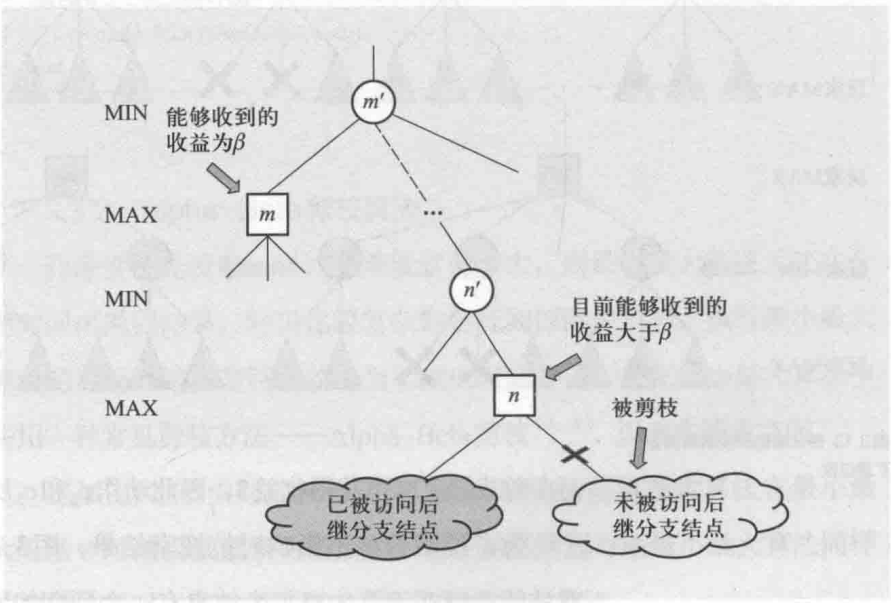


图3.15 基于MAX结点反馈收益进行剪枝（Beta剪枝）

如图3.14所示，玩家MAX在结点m'能够从位于其下MIN层的结点m得到取值至少为α的收益，因此结点n那些未被访问的后继分支结点对结点m'取值没有任何影响了。

上面介绍了对MIN结点的后续结点进行剪枝方法。下面通过图3.15来介绍对MAX结点的后续结点进行剪枝方法。

如图3.15所示，考虑位于MAX层的结点m，已知结点m能够从其下MIN层结点收到的收益为β（Beta）。结点n是结点m上层结点m'的位于MAX层的后代结点，如果目前已知结点n能够收到的收益大于β，则不再扩展结点n的未被访问后继结点，因为位于MIN层的结点m'只会选择收益小于或等于β的结点来采取行动。

综合以上两种情况的剪枝方法被称为Alpha-Beta剪枝。实际上，在图3.14中，结点m不仅为MIN层结点n提供了一个下界，对于例如n'的MAX层结点，这个下界仍然是有意义的，图3.15中的情况同理。因此，无论一个结点位于MIN层还是MAX层，都可以根据图3.14和图3.15中的情况找到一个下界和一个上界，即可以为每个结点设置一个α值和一个β值，来判断该结点及其后继结点是否可被剪枝。在设计算法时，为了计算一个结点的上下界，没有必要去枚举所有会影响当前结点α值和β值的结点，只需要继承父结点的α值和β值，再按照一定的规则加以更新即可。对于MAX结点，如果其孩子结点（MIN结点）的收益大于当前的α值，则将α值更新为该收益；对于MIN结点，如果其孩子结点（MAX结点）的收益小于当前的β值，则将β值更新为该收益。根结点（MAX结点）的α值和β值分别被初始化为$-\infty$和$+\infty$。

随着搜索算法不断被执行，每个结点的α值和β值不断被更新。大体来说，每个结点的$[\alpha, \beta]$从其父结点提供的初始值开始，取值按照如下形式变化：α逐渐增加、β逐渐减少。不难验证，如果一个结点的α值和β值满足$\alpha > \beta$的条件，则该结点尚未被访问的后续结点就会被剪枝，因而不会被智能体访问。

算法3.4给出了Alpha-Beta剪枝对抗搜索流程的伪代码，注意在MinValue和MaxValue函数中第5行，当前结点初始α和β值是由父结点提供的。

假设一棵完整的最小最大搜索树如图3.16所示，图3.17给出了每个结点α值和β值的详细变化过程，这个过程可对照算法3.4进行验证。

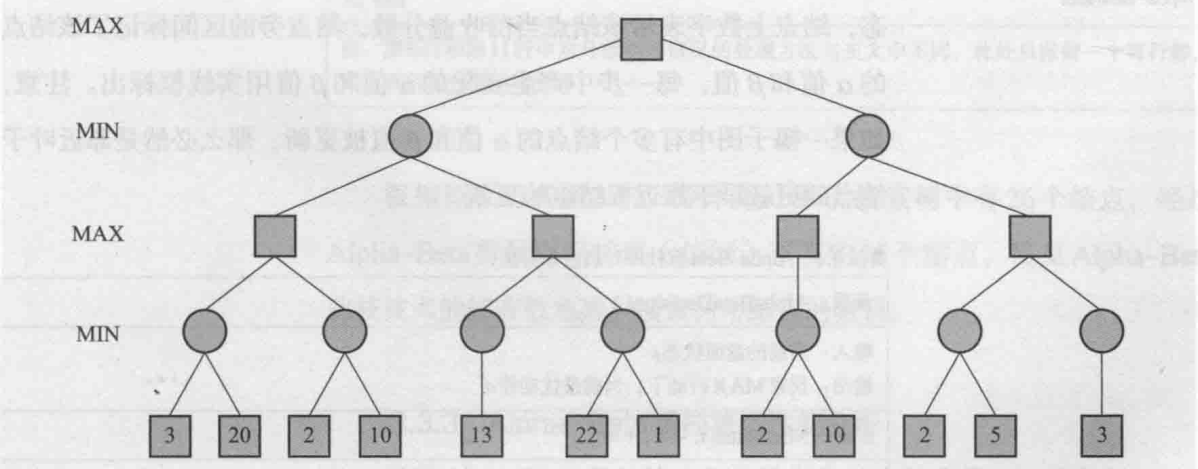


图3.16 一棵完整的最小最大搜索树

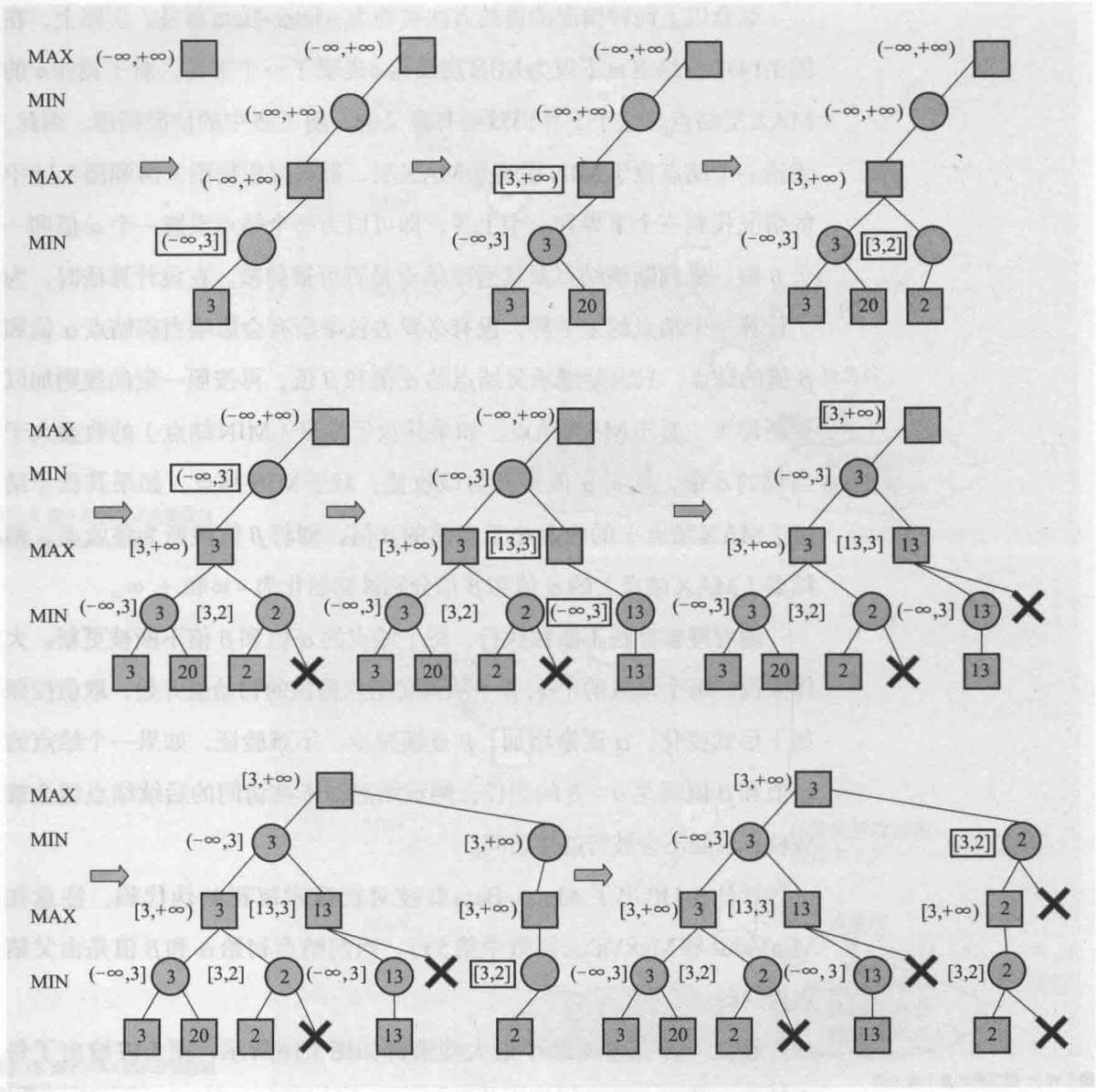


图3.17 对图3.16中搜索树的Alpha-Beta剪枝

图3.17中每幅子图对应了扩展一个终局状态或进行剪枝后的搜索树状态，结点上数字表示该结点当前收益分数，结点旁的区间标记了该结点的α值和β值，每一步中产生变化的α值和β值用实线框标出。注意，如果一幅子图中有多个结点的α值和β值被更新，那么必然是靠近叶子结点的更新早于靠近根结点的更新。

算法3.4 Alpha-Beta剪枝的对抗搜索算法

函数：AlphaBetaDecision( )
输入：当前的盘面状态s 输出：玩家MAX行动下，当前最优动作a^*
$v, a^* \leftarrow \text{MaxValue}(s, -\infty, +\infty)$

函数：MaxValue( )

输入：当前的盘面状态s，当前结点的下界α和上界β
输出：玩家MAX行动下，当前状态的得分v = minimax(s, MAX)和最优动作a^*
前置条件：$\alpha \leqslant \beta$

```
1  if terminal_test(s) then return utility(s), null
2  v ← −∞
3  a* ← null
4  foreach a ∈ action(s) do
5  |  v′, a′ ← MinValue(result(s, a), α, β)
6  |  if v′>v then
7  |  |  v ← v′
8  |  |  a* ← a
9  |  end
10 |  α ← max(α, v)
11 |  if α ≥ β then return v, a*
12 end
```

注：第6行和第11行中对分数相同情况的处理方法与正文中不同，此处只保留一个可行解，而放弃所有分数相同的解

函数：MinValue( )

输入：当前的盘面状态s，当前结点的下界α和上界β
输出：玩家MIN行动下，当前状态的得分v = minimax(s, MIN)和最优动作a^*
前置条件：$\alpha \leqslant \beta$

```
1  if terminal_test(s) then return utility(s), null
2  v ← +∞
3  a* ← null
4  foreach a ∈ action(s) do
5  |  v′, a′ ← MaxValue(result(s, a), α, β)
6  |  if v′<v then
7  |  |  v ← v′
8  |  |  a* ← a
9  |  end
10 |  β ← min(β, v)
11 |  if α ≥ β then return v, a*
12 end
```

注：第6行和第11行中对分数相同情况的处理方法与正文中不同，此处只保留一个可行解，而放弃所有分数相同的解

根据图3.16和图3.17，原本最小最大搜索树中有26个结点，经过Alpha-Beta剪枝后只扩展（访问）了其中15个结点，可见Alpha-Beta剪枝技术能够有效地减少搜索树中结点的数目。

*3.3.3 Alpha-Beta剪枝算法性能分析

考虑Alpha-Beta剪枝算法何时扩展的结点数量最少。首先每一层中

最左端每个结点的孩子结点必然不会被剪枝。对于其他结点，直观看来最好的情况为只扩展其最左端的孩子结点，而剪枝（即不再访问）该结点的其余孩子结点。图3.18给出了最好情况下的剪枝结果示意图，每一层中只有最左端结点的孩子结点均被访问，同一层中其他结点仅被访问最左端孩子结点，其他孩子结点被剪枝。为了出现如图3.18的剪枝效率最高情况，对于MIN层的结点而言，它的第一个孩子结点的收益必须小于该结点的初始下界α；对于MAX层的结点而言，它的第一个孩子结点的收益必须大于该结点的初始上界β。要说明的是，搜索树中结点出现的先后顺序会影响剪枝效率，针对这个问题，本章习题中给出了一个具体的例子。

如果仅根据以上分析，似乎除了每层最左端的结点，其余结点都可以只扩展一个孩子结点，这样的算法在最优情况下是否能够达到线性（关于m）的复杂度？当然这是不可能的。图3.19（b）给出了更为具体的最优剪枝情况示意图。在图中可见，虚线框出的结点必然会被扩展，这是上文的分析没有考虑到之处。实际上，如果一个结点导致了其兄弟结点被剪枝，可知其孩子结点必然被扩展。如在图3.19（b）中κ_{11}和κ_{21}两个结点分别导致了其右端的兄弟被剪枝，于是可以肯定κ_{11}和κ_{21}全部孩子结点必然被扩展。

下面以图3.19（a）为示意来进行证明。由于A结点为MAX或MIN结点时证明过程基本相同，不妨假设A结点为MAX结点。假设结点A的初始Alpha和Beta值为$[\alpha, \beta]$，如果A的孩子结点B导致了B右端的兄弟结点被剪枝。由于剪枝直到B的收益分数计算完后才发生，可知结点B必然是A已扩展孩子结点中收益分数最大的结点；同时根据剪枝的发生条件，可知B的收益分数α'必然满足$\alpha' > \beta$。假设扩展结点B时的上下界为$[\alpha'', \beta]$，其中α''来自结点B被扩展时结点A的下界

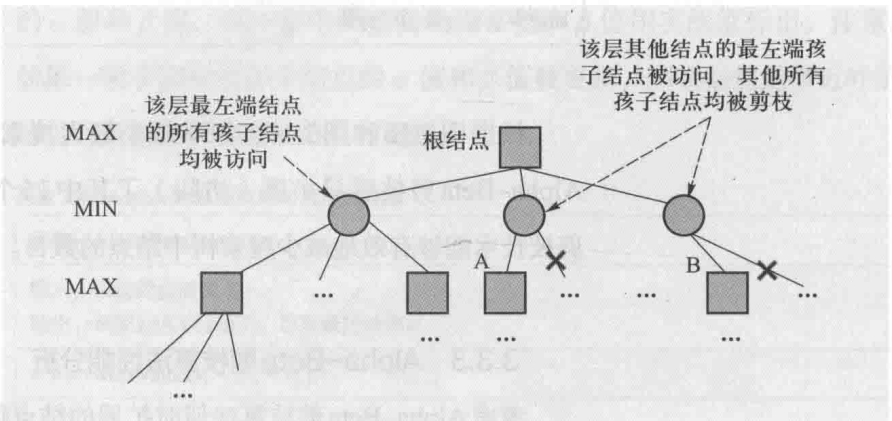


图3.18 每一层最左端结点的所有孩子结点均被访问，其他结点仅有最左端孩子结点被访问、其他孩子结点被剪枝

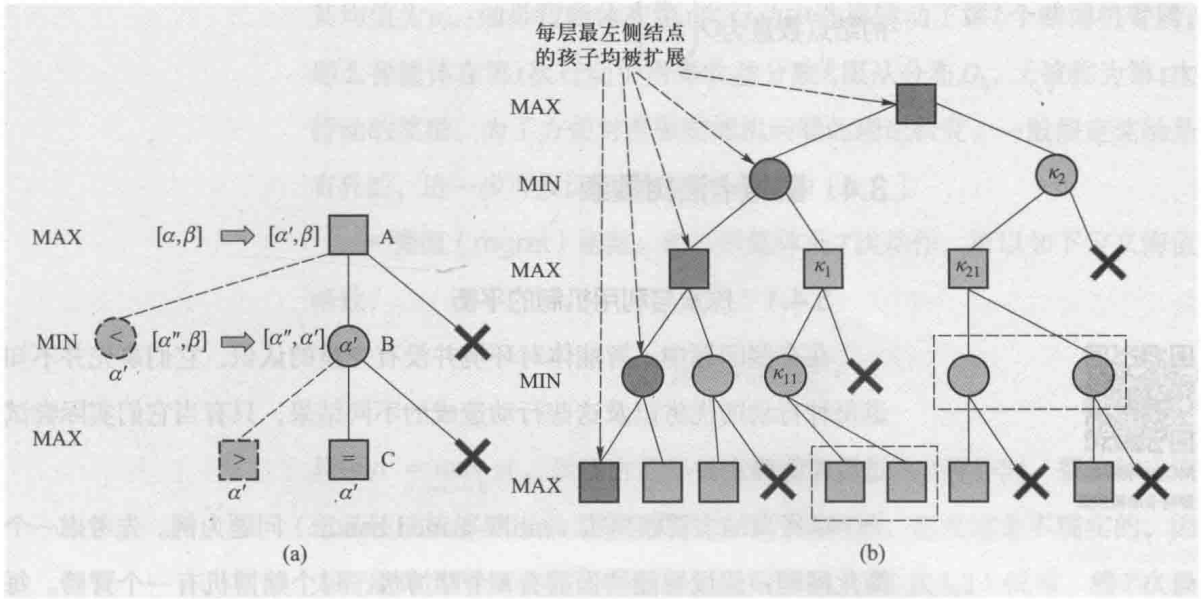


图3.19 一种较为复杂的剪枝情况分析

Alpha。根据假设，B结点至少有一个孩子结点（C结点）被扩展，因此$\alpha'' \leqslant \beta$，同时又根据$\alpha' > \beta$，因此推出$\alpha'' < \alpha'$。如果结点C导致B的其余子结点被剪枝，仿照对结点B的分析，可知结点C是结点B已扩展子结点中收益分数最小的一个。于是，结点B的收益分数必然是结点C的收益分数，即$\text{minimax}(C) = \text{minimax}(B) = \alpha'$，同时C的收益分数必然满足$\alpha' < \alpha''$，该式与前面的不等式矛盾。由此不难归纳出结论：如果某个结点导致了其右侧兄弟结点被剪枝，那么该结点的所有孩子结点必然已被全部扩展。

图3.19（b）展示了Alpha–Beta剪枝算法效率最优的情况，为了方便说明，假设图中每个结点恰好有b个子结点（此处b取值为2）。在图3.19（b）中，搜索树每层最左端结点的孩子结点必然全部被扩展。其他结点的孩子结点被扩展情况分为如下两类：① 只扩展其最左端孩子结点；② 它是其父亲结点唯一被扩展的孩子结点，且它的所有孩子结点（如果存在）一定被扩展。如在图3.19（b）中，κ_1和κ_2两个结点属于上述第一类结点，即仅扩展它们的最左端孩子结点。κ_{11}和κ_{21}两个结点属于上述第二类结点，即它们是其父亲结点唯一被扩展的孩子结点，且它的所有孩子结点一定被扩展。

基于以上分析，不难计算出Alpha–Beta剪枝在最优效率下扩展的结点数量为$O\left(b^{\left\lfloor \frac{m+1}{2} \right\rfloor}\right)$，比起原本最小最大搜索的$O(b^m)$有显著提升。这一计算过程留为本章习题。要注意的是，当m为偶数时，最优效率下扩展

的结点数量为$O\left(b^{\frac{m}{2}}\right)$。

3.4 蒙特卡洛树搜索

3.4.1 探索与利用机制的平衡

MOOC微视频：
蒙特卡洛树搜索

在有些问题中，智能体对环境并没有完整的认识，它们事先并不知道每种行动的优劣以及这些行动造成的不同结果，只有当它们实际尝试过后才能确定。

以经典的多臂赌博机（multi-armed bandit）问题为例。先考虑一个简化问题：假设智能体面前有K个赌博机，每个赌博机有一个臂膀。每次转动一个赌博机臂膀，赌博机则会随机吐出一些硬币或不吐出硬币，将所吐出的硬币的币值用收益分数来表示。现在假设给智能体$\tau(\tau>K)$次转动臂膀的机会，那么智能体如何选择赌博机、转动τ次赌博机臂膀，能够获得更多的收益分数呢？

或许可以让智能体先把K个赌博机的臂膀依次转动一遍，观察在摇动每个赌博机臂膀时的收益分数，然后去转动那些收益分数高的赌博机臂膀。但是，由于从每个赌博机获得的收益分数是随机的，显然一个刚刚给用户带来可观收益分数的赌博机在下一次摇动其臂膀时就可能不会获得可观收益分数了，因此这一方法不可取。那么是否有方法能够指导智能体的高效地获取收益分数呢？在介绍解决这个问题的方法前，先定义如下要素。

- **状态**。每个被摇动的臂膀即为一个状态，记K个状态分别为$\{s_1, s_2, \cdots, s_K\}$，没有摇动任何臂膀的初始状态记为$s_0$。
- **动作**。动作对应着摇动一个赌博机的臂膀，在多臂赌博机问题中，任意状态下的动作集合都为$\{a_1, a_2, \cdots, a_K\}$，分别对应摇动某个赌博机的臂膀。
- **状态转移**。选择动作a_i（$1\leqslant i\leqslant K$）后，将相应地改变为s_i。

然而这个问题和前两节讨论的搜索问题的不同之处在于，由于存在随机性，智能体摇动赌博机臂膀若干个τ次，任意两个τ次的结果可能都不一样，为此引入如下要素：

- **奖励**（reward）。假设从第i个赌博机获得收益分数的分布为D_i，

其均值为μ_i。如果智能体在第t次行动中选择转动了第l_t个赌博机臂膀，那么智能体在第t次行动中所得收益分数$\hat{r}_t$服从分布D_{l_t}，$\hat{r}_t$被称为第t次行动的奖励。为了方便对多臂赌博机问题的理论研究，一般假定奖励是有界的，进一步可假设奖励的取值范围为[0, 1]。

• **悔值（regret）函数。**根据智能体前T次动作，可以如下定义悔值函数：

$$\rho_T = T\mu^* - \sum_{t=1}^{T} \hat{r}_t \tag{式3.2}$$

其中$\mu^* = \max_{i=1,\cdots,K} \mu_i$。显然为了尽量减少悔恨，在每次操作时，智能体应该总是转动能够给出最大期望奖励的赌博机臂膀，但是这是不现实的，因为智能体并不知道哪个臂膀的奖励期望最大。（式3.2）说明，将T次操作中最优策略的期望得分减去智能体的实际得分，就是悔值函数的结果。显然，问题求解的目标为最小化悔值函数的期望，该悔值函数的取值取决于智能体所采取的策略。

为了解决这个问题，智能体采取的方法和前面几节中的方法略有不同，其最大差别在于：它不能预先计算好最优策略，然后实行，而是需要一边探索一边调整自己的策略，争取获得更好的收益。

一种直观的做法是，智能体记录下每次摇动的赌博机臂膀和获得的相应收益分数。给定第i（$1 \leqslant i \leqslant K$）个赌博机，记在过去$t-1$次摇动赌博机臂膀的行动中，一共摇动第$i$个赌博机臂膀的次数为第$T_{(i,\,t-1)}$。于是，可以计算得到第$i$个赌博机在过去$T_{(i,\,t-1)}$次被摇动过程中的收益分数平均值$\bar{x}_{i,\,T_{(i,t-1)}}$。这样，智能体在第$t$步，只要选择$\bar{x}_{i,\,T_{(i,t-1)}}$值最大的赌博机臂膀进行摇动，这是贪心算法的思路。

如图3.20所示，假设一共有五台赌博机（即$K=5$），第i个赌博机的得分满足均值为μ_i、方差为$\frac{1}{75}$（即支撑区间长度为0.4）的均匀分布，其中均值$(\mu_1, \cdots, \mu_5) = (0.3, 0.4, 0.5, 0.6, 0.7)$。假设某个赌博机的臂膀没有被摇动时，因为无法计算其平均收益分数，不妨将这个赌博机得分均值设定正无穷大。在计算机上模拟执行$T=10\,000$次贪心算法，来摇动五台赌博机。模拟的结果如图3.21所示。可见在10 000次模拟中，智能体趋向于摇动4号赌博机，其他编号赌博机被摇动次数很少。由于其他编号赌博机被摇动次数较少，可见对这些赌博机平均收益分数的估计结果与实际均值结果相差很大。

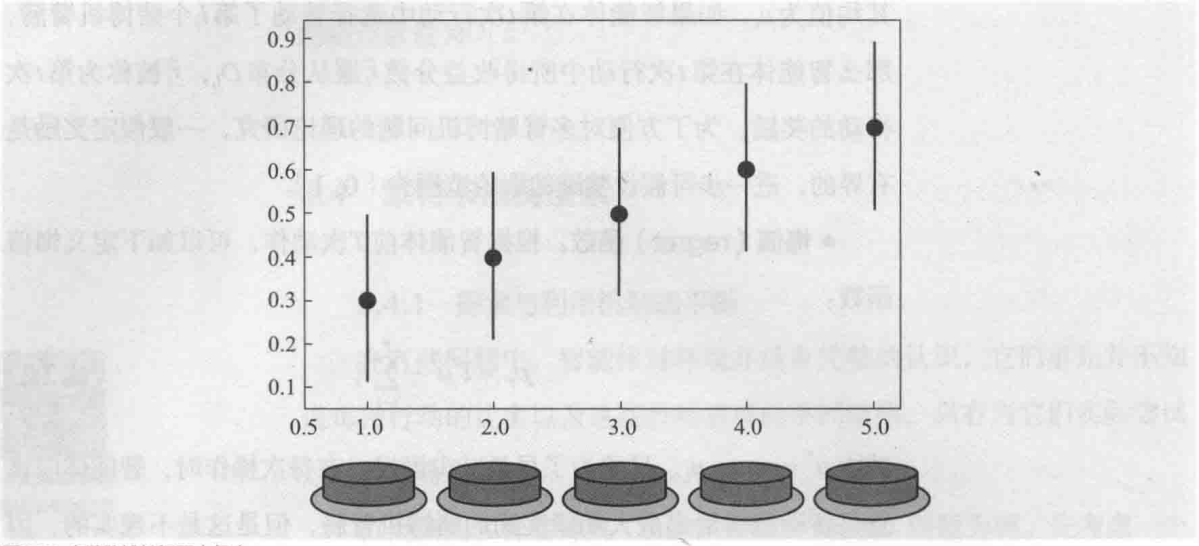


图3.20 多臂赌博机问题中每个赌博机收益分数的分布情况

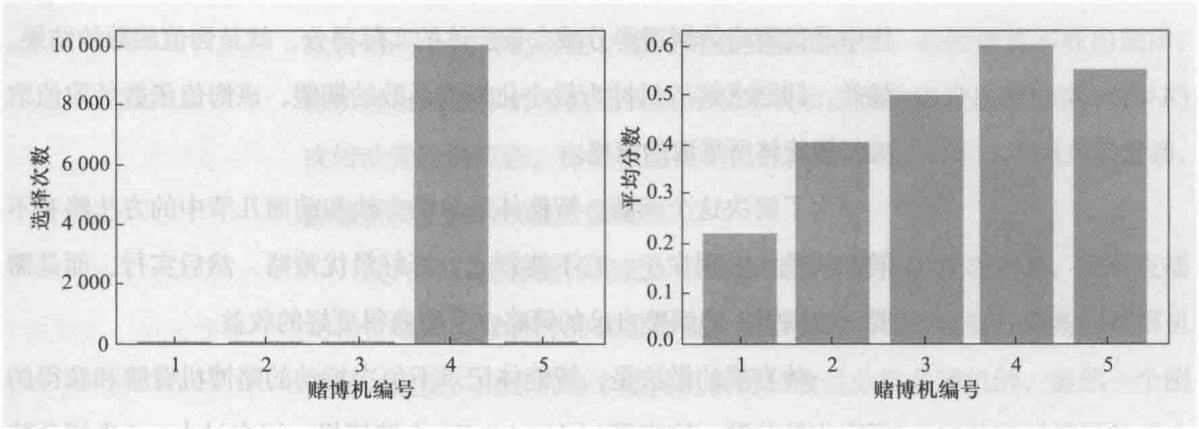


图3.21 贪心算法选取10 000个动作的一次模拟实验结果

需要说明的是，如果重新进行一个10 000次贪心算法的模拟，由于随机性的存在，其结果和图3.21会存在不同。比如在另外一个10 000次模拟中，贪心算法首先选择编号为2的赌博机臂膀摇动多次，则会导致后续行动中会对编号为2的赌博机摇动次数最多，忽略了其他编号的赌博机。

在上面的实验中，可以发现：贪心算法对编号4的赌博机“情有独钟”，很少去摇动其他编号的赌博机。因此，虽然对编号4赌博机的平均收益分数估计越来越准确，但是却难以尝试去摇动其他编号赌博机的臂膀，从而无法对其他赌博机的平均收益分数进行准确估计。而这些不准确的分数估计又反过来让智能体错误地认为5号赌博机不如4号赌博机，因而无法做出更好的选择。

上述困境体现了**探索**（exploration）和**利用**（exploitation）之间存在对立关系。贪心算法基本上是**利用**从已有尝试结果中所得估计来指导

后续动作（如图3.21中倾向于选择4号赌博机），但问题是所得估计往往不能准确反映未被（大量）探索过的动作，如图3.21中由于很少摇动其他编号的赌博机而无法准确估计其他编号赌博机可能带来的收益分数。因此，需要在贪心算法中增加一个能够改变其“惯性”的内在动力，以使得贪心算法能够访问那些尚未被（充分）访问过的空间。

ϵ–贪心算法就是这样一种在探索与利用之间进行平衡的搜索算法。在第t步，ϵ–贪心算法按照如下机制来选择摇动赌博机：

$$l_t = \begin{cases} \text{argmax}_i \overline{x}_{i,T_{(i,t-1)}}, & \text{以} 1-\epsilon \text{的概率} \\ \text{随机的} i \in \{1, 2, \cdots, K\} & \text{以} \epsilon \text{的概率} \end{cases} \quad \text{（式3.3）}$$

即以$1-\epsilon$的概率选择在过去$t-1$次摇动赌博机臂膀行动中所得平均收益分数最高的赌博机进行摇动；以ϵ的概率随机选择一个赌博机进行摇动。

以图3.21中的样本为基础，用ϵ–贪心算法再选择10 000个动作，其中$\epsilon = 0.1$，用计算机模拟的结果如图3.22所示。显然与图3.21相比，ϵ–贪心算法对估计摇动五个赌博机臂膀所带来期望分数更加准确了。并且因为编号为5的赌博机的平均收益分数最高，算法倾向于选择动作a_5，与预期相符。这说明在ϵ–贪心算法中加入探索机制发挥了作用。但是，如果在图3.21的基础上继续使用贪心算法而不引入探索机制，那么算法极有可能继续选择10 000个a_4动作、而继续忽略其他编号的赌博机。

3.4.2 上限置信区间算法

ϵ–贪心算法虽然能有效地促使算法进行探索，但通过这种随机机制来选择动作进行探索的做法很可能不是最优的。直观来看，执行一个动作后所得奖励的样本少，那么算法对这个动作的奖励期望估计会不准确。

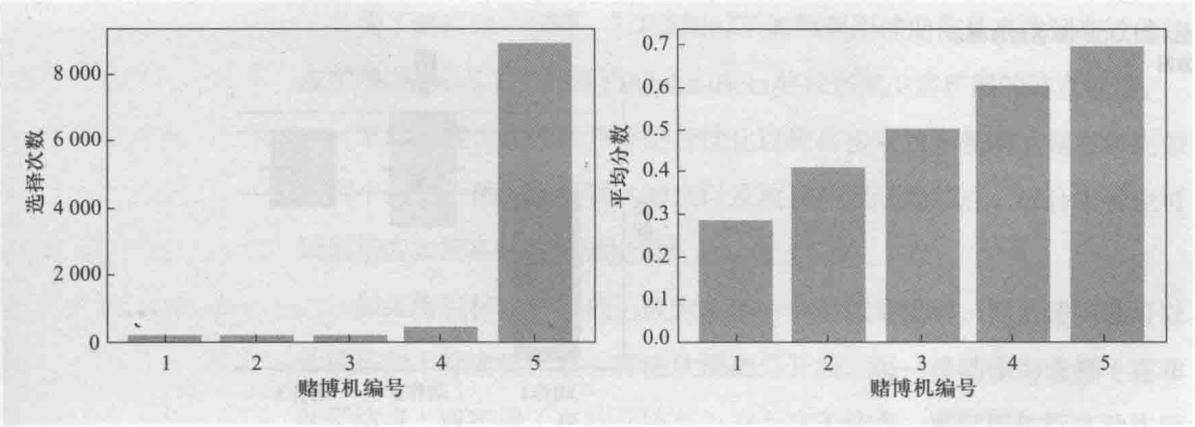


图3.22 在图3.21的基础上再进行10 000次ϵ–贪心算法的实验结果

例如，可能存在一个给出更好奖励期望的动作，但因为智能体对其探索次数少而认为其期望奖励小。因此，需要对那些探索次数少或几乎没有被探索过的动作赋予更高的优先级。

但是ϵ-贪心算法没有将每个动作被探索的次数纳入考虑；这似乎是不合理的。另一方面，简单地优先去尝试探索次数少的动作也未必合理，因为只需要少量的探索次数，就能够较好估计方差较小的动作的奖励期望。

总结以上想法，在探索过程中，应该优先探索估计值不确定度高的动作。另外还应注意到，如果从已有探索样本中知道一个动作的奖励估计值极端偏小，那么即使该动作的估计值不确定度很大，算法也没有必要将其作为优先探索对象。上限置信区间（upper confidence bounds，UCB1）[1]算法正是采用这一思路来进行探索。

UCB1算法的策略是：为每个动作的奖励期望计算一个估计范围，优先采用估计范围上限较高的动作。如图3.23所示，动作1的奖励期望取值的不确定度（估计范围）虽然最大，但是因为其均值太小，因此UCB1算法不优先考虑探索动作1。动作2和3的奖励期望的均值相同，但是动作2的奖励期望取值的不确定度（估计范围）更大，于是因为置信上限更大，动作2会被UCB1算法优先考虑。

接下来的问题在于算法应该如何计算奖励期望的估计范围，尤其是奖励期望的置信上界呢？假设算法在过去第t次已经对动作a_i探索了$T_{(i,\,t-1)}$次，在当前问题中对应摇动了第i个赌博机的臂膀$T_{(i,\,t-1)}$次，执行动作a_i所收到收益分数的均值为$\overline{x}_{i,\,T_{(i,\,t-1)}}$。这$T_{(i,\,t-1)}$次动作可以看作$T_{(i,\,t-1)}$个取值范围在[0, 1]的独立同分布随机变量的样本，根据霍夫丁不等式（Hoeffding's inequality），有

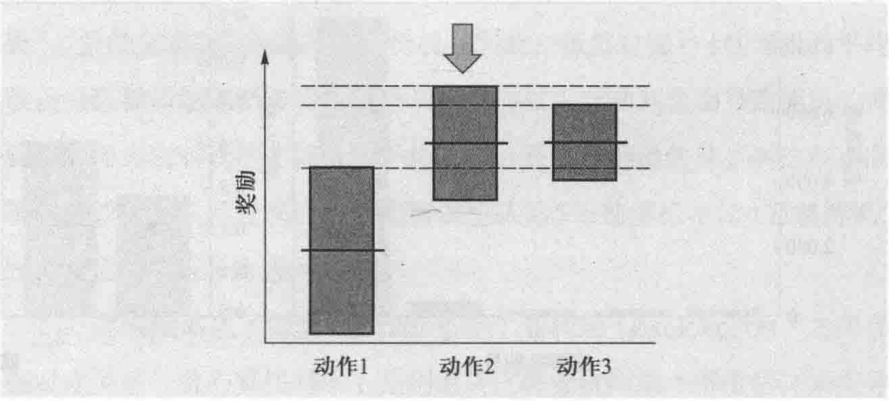


图3.23 UCB1算法的策略示意图

$$P\left(\mu_i - \overline{x}_{i,T_{(i,t-1)}} > \delta\right) \leqslant e^{-2T_{(i,t-1)}\delta^2} \qquad \text{（不等式3.4）}$$

这个不等式的含义是第i个动作的奖励期望μ_i大于$\overline{x}_{i,T_{(i,t-1)}}+\delta$之和的概率不超过$e^{-2T_{(i,t-1)}\delta^2}$，其中$\delta>0$。因此只需找到一个$\delta$，使得（不等式3.4）右侧足够小，即可认为$\overline{x}_{i,T_{(i,t-1)}}+\delta$是$\mu_i$的一个上界。

这里可找到一个随时间增长快速趋近于0的函数t^{-4}，令$e^{-2T_{(i,t-1)}\delta^2}=t^{-4}$，则$\delta=\sqrt{\dfrac{2\ln t}{T_{(i,t-1)}}}$，因此$\mu_i$的上界为$\overline{x}_{i,T_{(i,t-1)}}+\sqrt{\dfrac{2\ln t}{T_{(i,t-1)}}}$。当然，要选择一个随着$t$取值增大而收敛到0的函数，未必一定要选$t^{-4}$，如果换成关于$t$的其他幂函数，结果将会和当前$\delta$取值相差一个常数系数，因此奖励期望的上界也可以写成$\overline{x}_{i,T_{(i,t-1)}}+C\sqrt{\dfrac{2\ln t}{T_{(i,t-1)}}}$，其中$C$是一个预先指定的超参数，可以理解为用来调节探索和利用两者权重的因子。

UCB1算法的策略可以描述为，在第t次时选择动作a_{l_t}使得（式3.5）取值最大，其中l_t由如下式子计算得到：

$$l_t = \operatorname{argmax}_i \overline{x}_{i,T_{(i,t-1)}} + C\sqrt{\frac{2\ln t}{T_{(i,t-1)}}} \qquad \text{（式3.5）}$$

文献[1]中证明了：UCB1算法在经过T次测试后，悔值函数的期望上界为$O\left(\dfrac{K\ln T}{\Delta}\right)$，其中$\Delta=\min\limits_{\substack{i=1,\cdots,K;\\ \mu_i<\mu^*}}\mu^*-\mu_i$。

3.4.3 蒙特卡洛树搜索

无论是3.1节中介绍的一般搜索问题，还是3.3节中介绍的对抗搜索问题，在问题特别复杂时，搜索树可能会变得十分巨大，以至于搜索算法很难在短时间内探索整棵搜索树。

为了解决这个问题，3.2节探讨了如何利用辅助信息来找到高效的结点扩展顺序，3.3节介绍了Alpha-Beta剪枝来减少需扩展的结点数量。

不难发现，对搜索算法进行优化以提高搜索效率基本上是在解决如下两个问题：优先扩展哪些结点以及放弃扩展哪些结点，综合来看也可以概括为如何高效地扩展搜索树。

如果将目标稍微降低，改为求解一个近似最优解，则上述问题可以看成是如下探索性问题：算法从根结点开始，每一步动作为选择（在非叶子结点）或扩展（在叶子结点）一个孩子结点。可以用执行该动作后

所收获的奖励来判断该动作优劣。奖励可以根据从当前结点出发到达目标路径的代价或游戏终局分数来定义。算法会倾向于扩展获得奖励较高的结点。

算法事先不知道每个结点将会得到怎样的代价（或终局分数）分布，只能通过采样式探索来得到计算奖励的样本。由于这个算法利用蒙特卡洛法通过采样来估计每个动作优劣，因此它被称为**蒙特卡洛树搜索**（Monte-Carlo tree search）算法[6]。

蒙特卡洛树搜索算法分为以下四个步骤。

- **选择**（selection）：选择指算法从搜索树的根结点开始，向下递归选择子结点，直至到达叶子结点或者到达具有还未被扩展过的子结点的结点L，如图3.24（a）所示。这个向下递归选择过程可由UCB1算法来实现，在递归选择过程中记录下每个结点被选择次数和每个结点得到的奖励均值。

- **扩展**（expansion）：如果结点L不是一个终止结点（或对抗搜索的终局结点），则随机扩展它的一个未被扩展过的后继边缘结点M，如图3.24（b）所示。

- **模拟**（simulation）：从结点M出发，模拟扩展搜索树，直到找到一个终止结点，如图3.24（c）所示。模拟过程使用的策略和采用UCB1算法实现的选择过程并不相同，前者通常会使用比较简单的策略，例如使用随机策略。

- **反向传播**（Back Propagation）：用模拟所得结果（终止结点的代价或游戏终局分数）回溯更新模拟路径中M以上（含M）结点的奖励均值和被访问次数，如图3.24（d）所示。

以图3.16中的对抗搜索问题为例，以最小最大算法为基础的蒙特卡洛树搜索算法也被称为上限置信区间树搜索（upper confidence bounds for trees，UCT）算法。图3.25展示了蒙特卡洛树搜索的一次迭代中4个完整步骤，其中结点中数字记录了经过该结点所得收益分数之和（左侧数字）和访问该结点的次数（右侧数字）。

在图3.25（a）中，搜索树中已经有三个结点，注意此时根结点的分数之和为负数，这是用蒙特卡洛树搜索求解对抗搜索问题时的特殊措施，原因将在讨论反向传播步骤时提到。此时分别计算第二层结点的UCB值：左侧结点为$\frac{10}{1}+\sqrt{\frac{2\ln 2}{1}}=11.18$，右侧结点为$\frac{5}{1}+\sqrt{\frac{2\ln 2}{1}}=6.18$，因

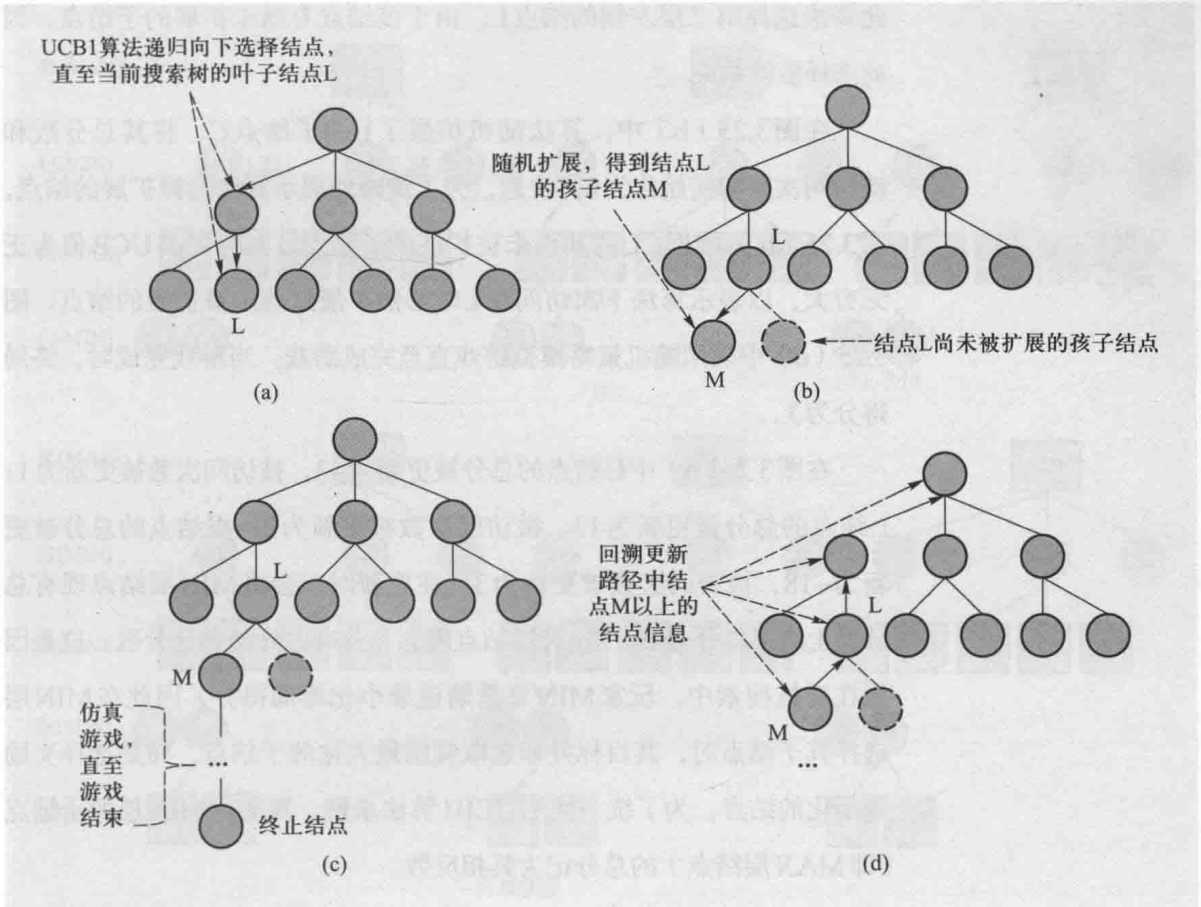


图3.24 蒙特卡洛树搜索的四个步骤

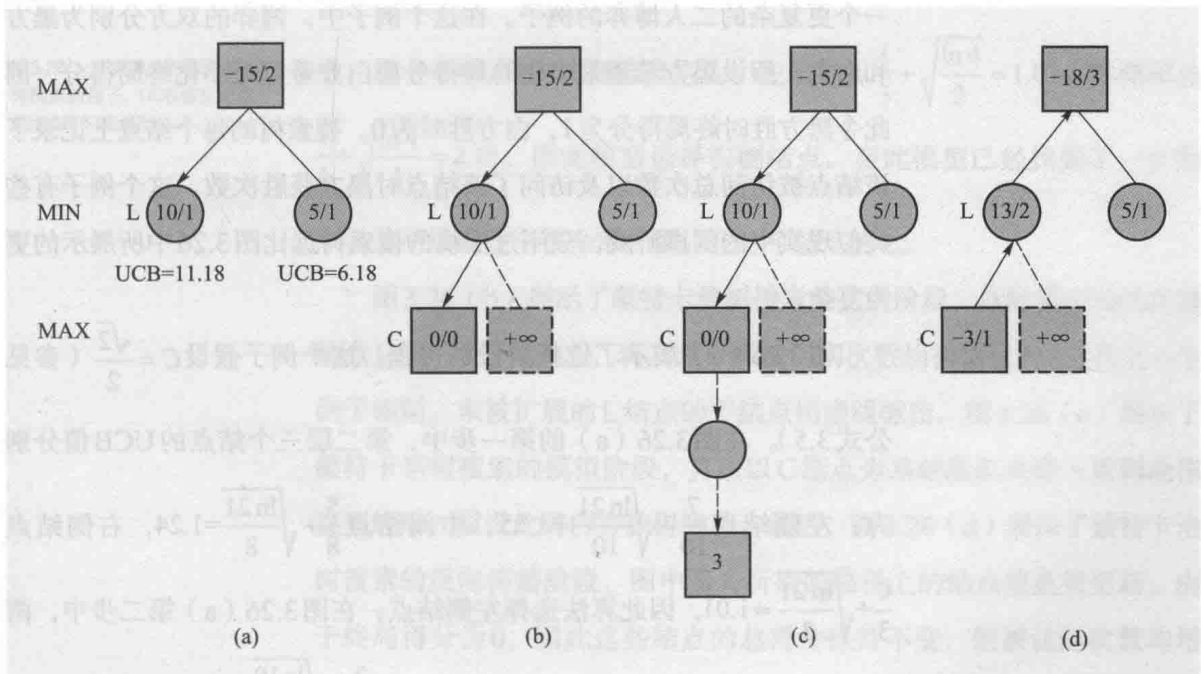


图3.25 蒙特卡洛树搜索算法，结点中数字代表总收益分数/被访问次数

此算法选择第二层左侧的结点L，由于该结点有尚未扩展的子结点，因此选择阶段结束。

在图3.25（b）中，算法随机扩展了L的子结点C，将其总分数和被访问次数均初始化为0。注意，为了清晰地展示算法选择扩展的结点，图3.25（b）画出了L的其他未被扩展的子结点，并标记其UCB值为正无穷大，以表示算法下次访问到L时必然扩展这些未被扩展的结点。图3.25（c）中采用随机策略模拟游戏直至完成游戏。当游戏完成时，终局得分为3。

在图3.25（d）中C结点的总分被更新为-3，被访问次数被更新为1；L结点的总分被更新为13，被访问次数被更新为2；根结点的总分被更新为-18，被访问次数被更新为3。在更新时，会将MIN层结点现有总分加上终局得分分数，MAX层结点现有总分减去终局得分分数。这是因为在对抗搜索中，玩家MIN总是期望最小化终局得分，因此在MIN层选择其子结点时，其目标并非选取奖励最大化的子结点，而是选择奖励最小化的结点，为了统一使用UCB1算法求解，算法将MIN层的子结点（即MAX层结点）的总分记为其相反数。

为了对蒙特卡洛树搜索算法有一个更加清晰的理解，图3.26给出了一个更复杂的二人博弈的例子。在这个例子中，博弈的双方分别为黑方和白方，假设黑方希望最大化终局得分而白方希望最小化终局得分，因此令黑方胜时终局得分为1，白方胜时为0。搜索树的每个结点上记录了该结点被访问总次数以及访问了该结点时黑方获胜次数。这个例子有些类似现实中的围棋游戏，只不过围棋的搜索树远比图3.26中所展示的更为复杂。

图3.26（a）展示了选择阶段的过程，这个例子假设$C=\frac{\sqrt{2}}{2}$（参见公式3.5）。在图3.26（a）的第一步中，第二层三个结点的UCB值分别为：左侧结点$\frac{7}{10}+\sqrt{\frac{\ln 21}{10}}=1.25$，中间结点$\frac{5}{8}+\sqrt{\frac{\ln 21}{8}}=1.24$，右侧结点$\frac{0}{3}+\sqrt{\frac{\ln 21}{3}}=1.01$，因此算法选择左侧结点；在图3.26（a）第二步中，两个子结点的UCB值分别为：左侧结点$\frac{-2}{4}+\sqrt{\frac{\ln 10}{4}}=0.259$，右侧结点$\frac{-5}{6}+\sqrt{\frac{\ln 10}{6}}=-0.214$，因此算法选择左侧结点；在图3.26（a）第三步

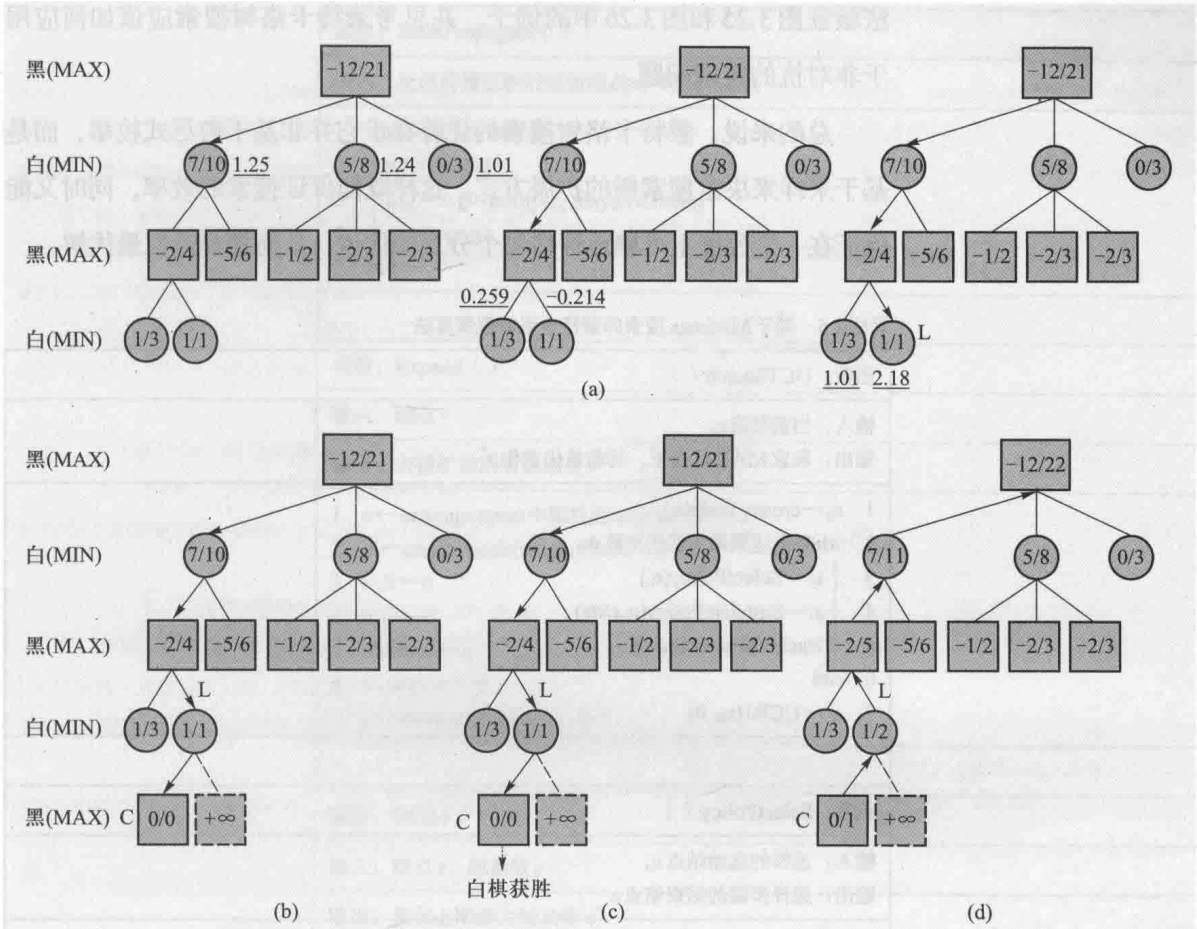


图3.26 一个更复杂的蒙特卡洛树搜索的例子，UCB值在对应结点边用下画线标出

中，两个子结点的UCB值分别为：左侧结点$\frac{1}{3}+\sqrt{\frac{\ln 4}{3}}=1.01$，右侧结点$\frac{1}{1}+\sqrt{\frac{\ln 4}{1}}=2.18$，因此模型选择右侧结点。至此模型已经找到了一个结点，其子结点并未被全部扩展，记这个结点为L。

图3.26（b）展示了蒙特卡洛树搜索的扩展阶段，此时算法随机扩展结点L的一个子结点C，将其总分和被访问次数均初始化为0。和上一个例子相同，未被扩展的L结点的子结点用虚线标出。图3.26（c）展示了蒙特卡洛树搜索的模拟阶段，算法以C结点为基础随机采样一系列动作直到终局，假设此局白方获得胜利。最后，图3.26（d）展示了蒙特卡洛树搜索的反向传播阶段，图中箭头所指的路径上的结点信息被更新。由于终局得分为0，因此这些结点的总得分保持不变，但被访问次数均增加1。

算法3.5给出了蒙特卡洛树搜索在解决对抗问题时的流程，其中N表示某个结点被访问的总次数、Q是每个结点的累积分数。读者可对照算

法验证图3.25和图3.26中的例子，并思考蒙特卡洛树搜索应该如何应用于非对抗的搜索问题。

总的来说，蒙特卡洛树搜索的优势在于它并非基于穷尽式枚举，而是基于采样来决定搜索树的扩展方式。这样既能保证搜索的效率，同时又能保证在一定程度上准确地评估每个分支的优劣，从而找到近似最优解。

算法3.5　基于Minimax搜索的蒙特卡洛树搜索算法

函数：UCTSearch（）

输入：当前状态s_0
输出：玩家MAX行动下，当前最优动作a^*

```
1  v0←create_node(s0)
2  while 未达到最大迭代次数 do
3  |  vl←SelectPolicy(v0)
4  |  sr←SimulatePolicy(vl.state)
5  |  BackPropagate(vl, sr)
6  end
7  a*←UCB1(v0, 0)
```

函数：SelectPolicy（）

输入：选择的起始结点v_0
输出：选择步骤的结束结点v

```
1  v←v0
2  while not terminal_test(v.state) do
3  |  if v 存在未被扩展的子结点 then
4  |  |  return Expand(v)
5  |  else
6  |  |  v←UCB1(v, Cp)
7  |  end
8  end
```

函数：SimulatePolicy（）

输入：状态s_0
输出：模拟的终止状态s

```
1  s←s0
2  while not terminal_test(s) do
3  |  a←从action(s)中随机采样
4  |  s←result(s, a)
5  end
```

函数：BackPropagate（）

输入：反向传播更新的起始结点v，终局状态s_t

```
while v is not null do
    v.N←v.N+1
    v.Q←v.Q−utility(s_t, player(v.state))
    v←v.parent
end
```

函数：Expand（）

输入：结点v

输出：未被扩展的后继结点v'

```
a←action(v.state)中随机的未探索动作
v'←create_node(result(v.state, a))
v'.N←0
v'.Q←0
v'.parent←v
v'.children←∅
v.children←v.children∪{v'}
```

函数：UCB1（）

输入：结点v，超参数c

输出：置信上限最大的动作a^*

$$a^* \leftarrow \operatorname{argmax}_{v' \in v.children} \frac{v'.Q}{v'.N} + c\sqrt{\frac{2\ln v.N}{v'.N}}$$

3.5 小结

根据所设定问题，直接从搜索空间中寻找符合条件的答案。如果搜索空间较大，则需要采取剪枝和采样等手段，前者与搜索树结点出现的位置次序有关，后者则需要对可能结点进行大量仿真模拟。

搜索是人工智能智能能力体现的一种手段，匹配式搜索与数据采样式学习相互结合，使得搜索超越了传统匹配计算模式。

习题3

1. 以下关于用搜索算法求解最短路径问题的说法中，不正确的是（　　）。

A. 给定两个状态，可能不存在两个状态之间的路径；也可能存在两个状态之间的路径，但不存在最短路径（如考虑存在负值的回路情况）

B. 假设状态数量有限，当所有单步代价都相同且大于0时，深度优先的图搜索是最优的

C. 假设状态数量有限，当所有单步代价都相同且大于0时，广度优先的图搜索是最优的

D. 图搜索算法通常比树搜索算法的时间效率更高

2. 以下关于启发函数和评价函数的说法中正确的是（　　）。

A. 启发函数不会过高估计从当前结点到目标结点之间的实际代价

B. 取值恒为0的启发函数必然是可容的

C. 评价函数通常是对当前结点到目标结点距离的估计

D. 如果启发函数满足可容性，那么在树搜索A^*算法中结点的评价函数值按照扩展顺序单调非减；启发函数满足一致性时图搜索A^*算法也满足该性质

3. 假如可以对围棋的规则作出如下修改，其中哪个修改方案不影响使用本章介绍的最小最大算法求解该问题？（　　）

A. 由双方轮流落子，改为黑方连落两子后白方落一子

B. 双方互相不知道对方落子的位置

C. 由两人对弈改为三人对弈

D. 终局时黑方所占的每目（即每个交叉点）计1分，且事先给定了白方在棋盘上每个位置取得一目所获取的分数，假设这些分数各不相同。双方都以取得最高得分为目标

4. 下列关于探索与利用的说法中，不正确的是（　　）。

A. 在多臂赌博机问题中，过度探索会导致算法很少主动去选择比较好的摇臂

B. 在多臂赌博机问题中，过度利用可能导致算法对部分臂膀额奖励期望估计不准确

C. 在ϵ贪心算法中，ϵ的值越大，表示算法越倾向于探索

D. 在多臂赌博机问题中，某时刻UCB1算法选择的臂膀置信上界为R，则此时任意摇动一个臂膀，得到的硬币数量不会超过R

5. 下列关于蒙特卡洛树搜索算法的说法中，不正确的是（　　）。

A. 选择过程体现了探索与利用的平衡

B. 算法进入扩展步骤时，当前结点的所有子结点必然都未被扩展

C. 模拟步骤采取的策略与选择步骤不一定要相同

D. 反向传播只需要更新当前路径上已被扩展的结点

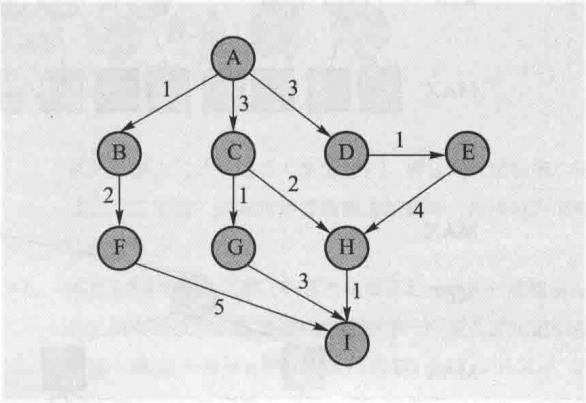
6. 如图3.27所示，假设每个结点代表一个状态，结点之间的箭

头表示状态转移关系，箭头旁的数字表示状态转移的代价。若使用以下搜索算法寻找从状态A到状态I的路径，请画出算法终止（找到第一条路径）时的搜索树，并在搜索树中标出结点的扩展顺序，以及找到的路径。若有多个结点拥有相同的扩展优先度，则优先扩展对应路径字典序较小的结点。

（1）基于树搜索的广度优先搜索。

（2）基于图搜索的深度优先搜索。

图3.27 状态转移图



7. 考虑图3.27中的问题，给定每个状态的启发函数如表3.4所示。若仍以状态A为初始状态、状态I为终止状态，请分别使用以下算法求解从A到I的路径，并按照第6题中的方法画出搜索树。若有多个结点拥有相同的扩展优先度，则优先扩展对应路径字典序较小的结点。

表3.4 启发函数的取值

状态	A	B	C	D	E	F	G	H	I
启发函数	5	4	3	2	5	5	2	1	0

（1）基于树搜索的贪婪最佳优先搜索。

（2）基于图搜索的A*算法。

8. 根据第7题中的搜索过程，回答下列问题。

（1）贪婪最佳优先搜索和A*算法均能找到最短路径，且贪婪最佳优先搜索扩展的结点数更少，这是否与正文中的结论“A*搜索是在已知信息下同类搜索策略中最优的”相矛盾，为什么？

（2）不难验证，第7题中给出的启发函数既是可容的也是一致的，从这个角度来说这是一个“好”的启发函数。显然这个启发函数并不足够“好”，以至于A*算法在该问题中效率低于贪婪最佳优先搜索。那么怎样的启发函数才能提高搜索效率？搜索效率最高时启发函数的值应该是多少？

9. 图3.28展示了一棵最小最大搜索树，可采用Alpha-Beta剪枝算法进行对抗搜索。假设对于每个结点的后继结点，算法按照从左向右的方向扩展。同时假设当Alpha值等于Beta值时，算法不进行剪枝。请回答下列问题。

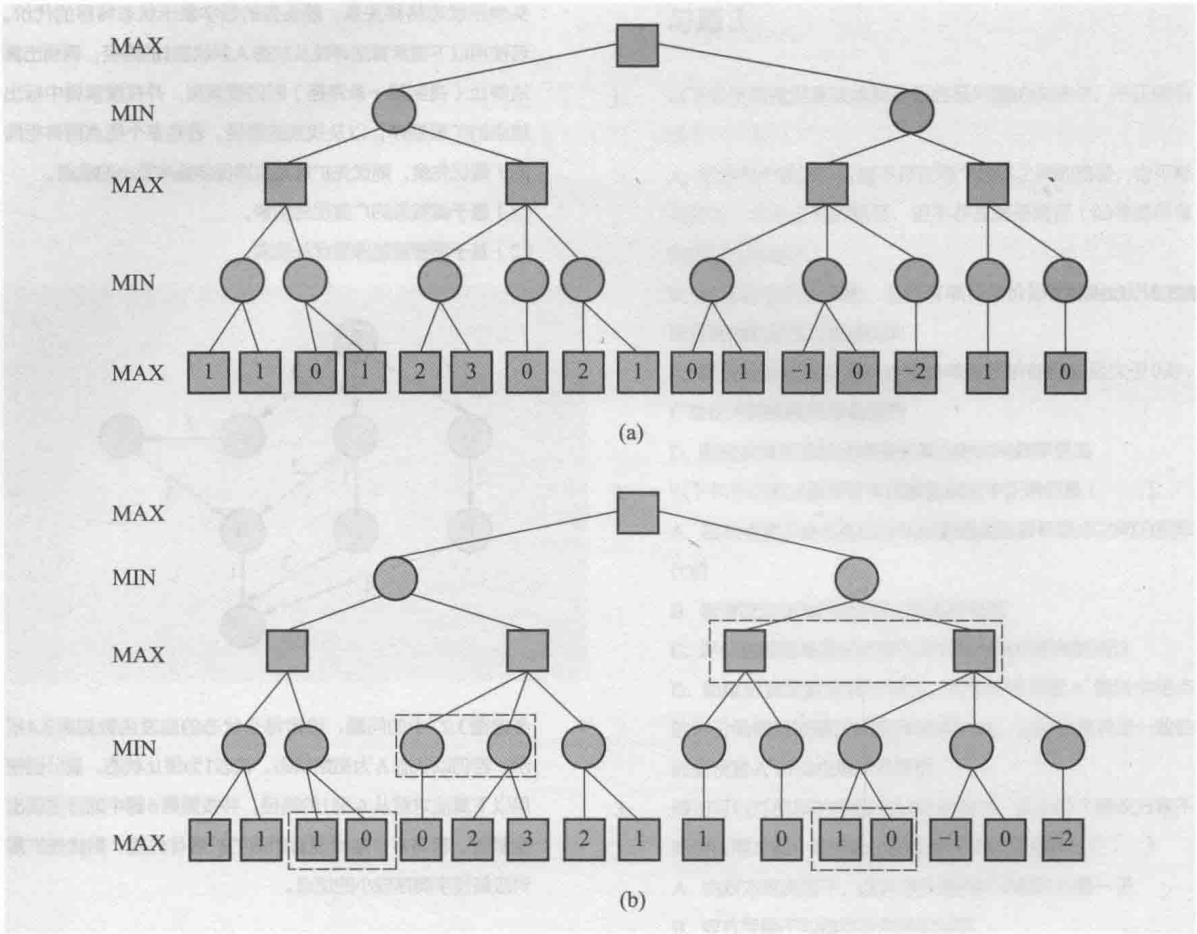


图3.28 一棵最小最大搜索树

（1）对图3.28（a）中所示搜索树进行搜索，请画出在算法结束时搜索树的状态，用“×”符号标出被剪枝的子树，并计算该算法扩展的结点数量。

（2）图3.28（b）展示了和图3.28（a）中完全相同的搜索树，只不过对搜索算法的顺序做了一些调整（扩展顺序交换的结点用虚线框标出）。请画出在算法结束时搜索树的状态，用“×”符号标出被剪枝的子树，并计算该算法扩展的结点数量。

10. 图3.29展示了一个蒙特卡洛树搜索的例子。其中每个叶子结点（终止结点）下标出了该结点对应的奖励。为了最大化取得的奖励，可利用蒙特卡洛树搜索求解奖励最大的路径。假设执行了若干步骤后，算法的状态如图3.29所示，结点内的数字分别表示“总奖励/访问次数”，虚线结点表示尚未扩展的结点。算法此时正要开始下一轮选择-扩展-模拟-反向传播的迭代。

（1）假设UCB1算法中的超参数$C=1$，请计算并画出算法选择过程经过的路径。

（2）请继续执行扩展、模拟、反向传播步骤，并画出完成后的搜索树状态。（为了避免随机性，假设扩展总是扩展最左侧的未扩展结点，模拟总是选择最左侧的路径。）

（3）尝试进行若干次迭代，请问此时算法是否能有效地找

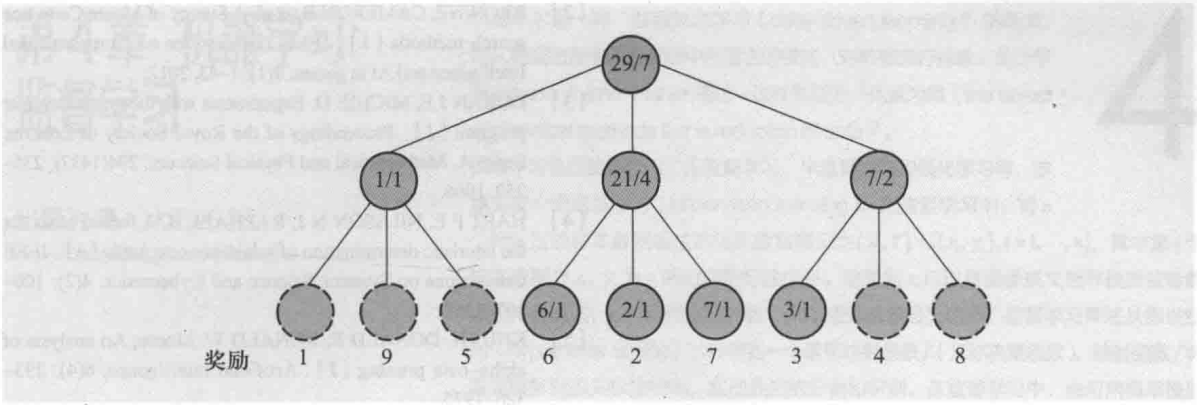


图3.29 一棵蒙特卡洛树搜索的搜索树

到奖励最大的叶子结点（奖励为9），那么进行足够多次迭代以后又如何？如果希望提高算法的效率，应该做出怎样的调整？

11. 本章3.2.4小节中证明了在启发函数满足一致性时图搜索A*算法具有最优性，证明过程中用到了一个没有详加说明的结论：给定一条从s_1到s_k的最短路径$s_1 \to s_2 \to \cdots \to s_k$，对于任意$1<i<k$和任意一条从$s_1$到$s_i$的最短路径$P$，则路径$P \to s_{i+1} \to \cdots \to s_k$仍然是$s_1$到$s_k$的一条最短路径。试证明该结论。

12. 针对本章正文中算法3.5中给出的蒙特卡洛树搜索算法的伪代码，回答以下问题。

（1）函数BackPropagate中的第3行为何要减去而不是加上终局得分？如果要修改为加上终局得分，那么该得分应该是以哪位玩家为视角得到的分数？

（2）算法3.5给出了利用蒙特卡洛树搜索算法求解对抗搜索问题的流程，如果要将蒙特卡洛树搜索应用于非对抗搜索，应该如何修改该算法？

*13. 假设一棵最小最大搜索树的最大路径长度为m，每个非叶子结点恰好有b（$b \geqslant 2$）个孩子结点。若采用Alpha-Beta剪枝算法求解该问题，试证明在本章正文图3.19中所示最优情况下，算法扩展结点数量的上界为$O\left(b^{\left\lfloor \frac{m+1}{2} \right\rfloor}\right)$。

*14. Dijkstra算法是一个著名的求解最短路径的图搜索算法，该算法令每个结点评价函数的值为该结点对应路径的代价（即正文中定义的$g(n)$函数）。

（1）当图中不存在负值代价时，Dijkstra算法满足最优性。试根据本章中对图搜索A*算法最优性的讨论，证明该结论。

（2）假设已经通过其他方法证明了（1）中的结论，试根据该结论证明，启发函数满足一致性时，A*算法满足最优性。

参考文献

[1] AUER P, FISCHER N. Finite-time Analysis of the Multiarmed Bandit Problem [J]. Machine Learning, 47(2-3): 235-256, 2002.

[2] BROWNE, CAMERON B, et al. A Survey of Monte Carlo tree search methods [J]. IEEE Transactions on Computational Intelligence and AI in games, 4(1): 1–43, 2012.

[3] DORAN J E, MICHIE D. Experiments with the graph traverser program [J]. Proceedings of the Royal Society of London. Series A, Mathematical and Physical Sciences, 294(1437): 235–259, 1966.

[4] HART P E, NILSSON N J, RAPHAEL B. A formal basis for the heuristic determination of minimum cost paths [J]. IEEE transactions on Systems Science and Cybernetics, 4(2): 100–107, 1968.

[5] KNUTH, DONALD E, RONALD W. Moore, An analysis of alpha–beta pruning [J]. Artificial Intelligence, 6(4): 293–326, 1975.

[6] KOCSIS L, SZEPESVARI C. Bandit based monte–carlo planning [C] // Proceedings of the 17th European conference on Machine Learning(ECML). Machine Learning, 282–293, 2006.

[7] RUSSELL, STUART J, NORVIG P. Artificial intelligence: a modern approach [M]. Malaysia: Pearson Education Limited, 2016.

[8] SAMUEL A L. Some studies in machine learning using the game of checkers [J]. IBM Journal of research and development, 3(3): 210–229, 1959.

第4章 机器学习：监督学习

化繁为简、大巧不工
——Ronald Aylmer Fisher

4

机器学习是一种“数据驱动学习（data-driven learning）”的范式，即从数据出发来学习数据中所蕴含的模式，对数据进行抽象。统计学家 Ronald Aylmer Fisher 将这一过程概括为“化繁为简（the object of statistical methods is the reduction of data）”。

机器学习包括监督学习、无监督学习、半监督学习和强化学习等，本章主要介绍监督学习（supervised learning）。在监督学习中，将 n 个标注过的样本数据组成的训练数据集记为 $(X,Y)=\{(x_i,y_i),i=1,\cdots,n\}$，其中第 i 个样本数据为 x_i，y_i 为 x_i 所对应的标注信息。这里的 x_i 可以是图像或文档等数据或者数据的特征表达，y_i 为所对应的人脸、汽车或情感等语义信息。监督学习算法从假设空间（hypothesis space）学习得到一个最优映射函数 f（又称决策函数），映射函数 f 将数据映射到语义标注空间，实现数据的分类和识别。在监督学习中，也可用概率模型 $P(y_i|x_i)$ 计算结果来判断数据 x_i 属于 y_i 的概率，进而实现 x_i 的分类和识别。

监督学习中涉及了机器学习基本概念，如评价学习算法性能的泛化能力（generalization）、平衡模型复杂化和简单化的正则化（regularization）方法、探究某一任务是否可以被学习的概率近似正确学习（probably approximately correct learning, PAC Learning）理论等。

本章先引入机器学习和监督学习的基本概念，在此基础上介绍包括回归分析、决策树、线性判别分析和提升（boosting）算法和支持向量机在内的监督学习的模型方法。

4.1 机器学习基本概念

MOOC微视频：机器学习基本概念

机器学习是智能体（或模型）从数据中自动学习知识的一种人工智能方法，该词最早由 Arthur Samuel[25] 提出。如果智能体通过观测到新的数据，进行一系列处理和更新后，在处理未来的问题上的能力得到提升，就能说该智能体在数据中学习到了新的知识。

机器学习涵盖了很多领域的不同任务，例如，对图像数据（在计算机中通常存储为RGB三通道的整数矩阵）的分析处理中推理出该图像所属的类别信息（如人和山峦等），进而在类别信息基础上进行深层次推理；对文本数据（通常将词表示为one-hot encoding或者通过Word2Vec[22]词嵌入技术表示为连续的多维向量）进行分析处理，推理出该文本单词中所蕴含情感（如喜悦、愤怒等）。如图4.1所示。

从数学的角度来看，机器学习的目标即从原始数据中提取特征，学习一个映射函数 f 将上述特征（或原始数据）映射到语义空间，寻找数据和任务目标之间的关系。

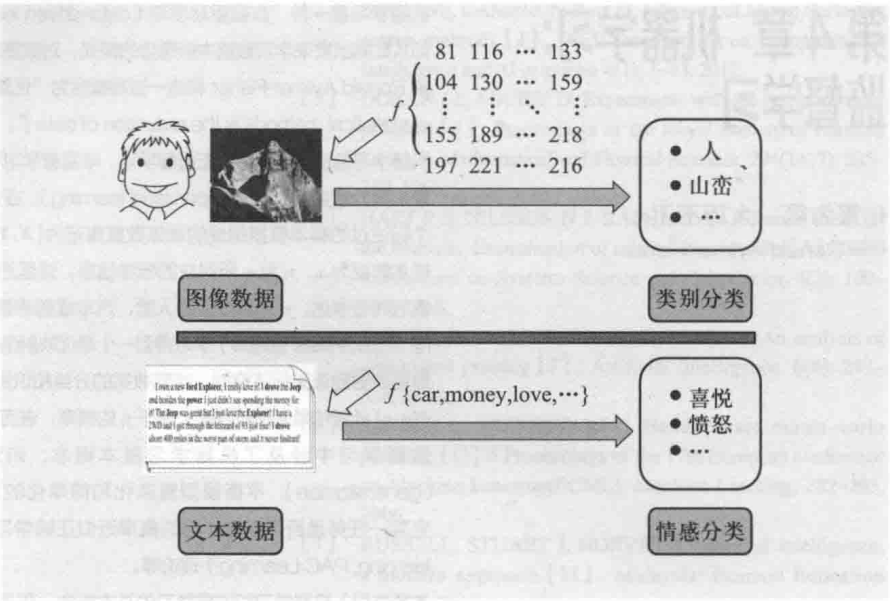


图4.1 机器学习示意图

4.1.1 机器学习的种类

机器学习通常被分为三种类型（或称学习场景），即监督学习、无监督学习和强化学习。

监督学习是一种在实践中运用最为广泛的一种机器学习方法，其目标是给定带有标签信息的训练集合$\mathcal{D}=\{(x_i,y_i)\}_{i=1}^{N}$，学习一个从输入$x$到输出$y$的映射，其中$\mathcal{D}$被称为训练集，$N$是训练样例的数量。监督学习一般被应用在回归或分类任务中。

在很多简单的场景下，监督学习输入为d维数值向量，如该向量的某些维度代表了某天的天气、湿度、风力等。输入向量的每一维又被称为特征或属性。在前面举例的场景中，输入就是一些更为复杂的结构体，如图片，自然语言等。分类任务中，监督学习的输出通常是名义变量，其取值是有限的，如根据历史的天气数据判断明天是否会下雨，输出为“下雨”“不下雨”两个名义取值；在回归任务中，监督学习的输出通常为真实数值，如根据历史人口数据预测下一年的人口数量，输出为真实数值。

无监督学习最大的特点是数据无标签，通常给定输入$\mathcal{D}=\{x_i\}_{i=1}^{N}$，其目标是挖掘数据中的有用信息，这种方法一般被应用在聚类或若干降维任务中。介于监督学习和无监督学习之间还存在**半监督学习**，这种方法依赖于部分被标注的数据。

强化学习是一种序列数据决策学习方法，一般为从与环境交互中学

习。当有良好定义或给出的回报值（reward）或惩罚值时，强化学习可以让智能体（agent）很好地学习到在不同的状态（state）下如何选择行为方式（action）。

4.1.2 监督学习的基本概念

正如前面内容所述，监督学习的目标是从带标签的数据中学习一个映射函数f，如表4.1中给出的身份分类例子。

在表4.1中，数学好坏、是否会编程等就是特征或属性，程序员、作家和医生等就是名义标签信息，学习目标是给出一个在训练集中未出现过的例子，映射函数能做出良好预测f（数学好 = Yes，会编程 = Yes，身体好 = ？，嗓门大 = ？）→程序员。

表4.1 是否为程序员的分类数据

人员	数学好	身体好	会编程	嗓门大
程序员A	Yes	No	Yes	Yes
作家A	No	No	Yes	No
程序员B	Yes	No	Yes	No
…	…	…	…	…
医生A	Yes	Yes	No	Yes
程序员C	Yes	Yes	Yes	Yes
程序员D	Yes	Yes	Yes	No

除了标识了类别信息的标注数据，选择好适合具体问题的学习模型，还需要选择对学习结果进行度量的损失函数。

将映射函数记为f，第i个训练对记为(x_i, y_i)，f对x_i的预测结果记为$f(x_i)$，那么损失函数就是用来计算预测值$f(x_i)$和真实值y_i之间差异的函数。很显然，在训练过程中希望映射函数在训练数据集上得到所有样本“损失和”最小，即$\min \sum_{i=1}^{n} Loss\left(f\left(x_i\right), y_i\right)$。

常用的损失函数包括0–1损失函数、平方损失函数、绝对损失函数和对数损失函数（又被称为对数似然函数）。表4.2给出了若干损失函数的定义。

监督学习一般包含三个部分的内容：从训练数据集中学习得到映射函数f，在测试数据集上测试映射函数f，在未知数据集上测试映射函数f（一般为真正投入使用）。

表4.2 常见损失函数

损失函数名称	损失函数定义
0 ~ 1损失函数	$Loss(y_i, f(x_i)) = \begin{cases} 1, f(x_i) \neq y_i \\ 0, f(x_i) = y_i \end{cases}$
平方损失函数	$Loss(y_i, f(x_i)) = (y_i - f(x_i))^2$
绝对损失函数	$Loss(y_i, f(x_i)) = \lvert y_i - f(x_i) \rvert$
对数损失函数/ 对数似然损失函数	$Loss(y_i, P(y_i \lvert x_i)) = -\log P(y_i \lvert x_i)$

训练集中数据产生的损失一般被称为**经验风险**（empirical risk）。经验风险越小说明学习模型对训练数据拟合程度越好。经验风险被定义为：

$$\frac{1}{n}\sum_{i=1}^{n} Loss(y_i, f(x_i))$$

当向测试集中不断加入从真实数据分布中采样的样本时，在测试集上产生的损失会不断逼近期望风险（expected risk）。期望风险越小，学习所得模型越好。期望风险被定义为：

$$\int_{x\times y} Loss(y, f(x)) P(x, y) \mathrm{d}x\mathrm{d}y$$

使用优化算法优化训练集损失函数是在追求经验风险最小化，而机器学习的目标是追求期望风险最小化[24]。

期望风险与经验风险的关系总结如下：期望风险是模型关于联合分布的期望损失，经验风险是模型关于训练样本集的平均损失。根据大数定律，当样本容量趋于无穷时，经验风险趋于期望风险，所以在实践中很自然用经验风险来估计期望风险。由于现实中训练样本数目有限，用经验风险估计期望风险并不理想，要对经验风险进行一定的约束。

模型泛化能力与经验风险、期望风险之间的关系总结如表4.3所示。

表4.3 模型泛化能力与经验风险、期望风险的关系

经验风险小 （训练集上表现好）	期望风险小 （测试集上表现好）	泛化能力强
经验风险小 （训练集上表现好）	期望风险大 （测试集上表现不好）	过学习（模型过于复杂）
经验风险大 （训练集上表现不好）	期望风险大 （测试集上表现不好）	欠学习
经验风险大 （训练集上表现不好）	期望风险小 （测试集上表现好）	“神仙算法”或“黄粱美梦”

经验风险最小化的问题在于仅反映了局部数据，期望风险最小化的问题在于无法得到全量数据，为了更有效地训练泛化能力强的模型，提出结构风险最小化（structural risk minimization）：为了防止过学习，基于过学习时参数值通常都较大这一发现，在经验风险上加上表示模型复杂度的正则化项（regularizer）或惩罚项（penalty term），在最小化经验风险与降低模型复杂度之间寻找平衡：

$$\frac{1}{n}\sum_{i=1}^{n} Loss\left(y_i, f\left(x_i\right)\right)+\lambda J\left(f\right)$$

其中$J(f)$是惩罚项，λ是惩罚项系数，即用来调整惩罚的强度。

主要的监督学习方法包括生成方法（generative approach）和判别方法（discriminative approach），所学到的模型分别称为生成模型（generative model）和判别模型（discriminative model）。

判别方法直接学习判别函数$f(X)$或者条件概率分布$P(Y|X)$作为预测的模型，即判别模型。判别模型关心在给定输入数据下，预测该数据的输出是什么。典型判别模型包括回归模型、神经网络、支持向量机和Ada boosting等。

生成模型从数据中学习联合概率分布$P(X, Y)$，这可以通过似然概率$P(X|Y)$和类概率$P(Y)$的乘积来求取。$P\left(Y|X\right)=\frac{P\left(X,Y\right)}{P\left(X\right)}$或者$P\left(Y|X\right)=\frac{P\left(X|Y\right)P\left(Y\right)}{P\left(X\right)}$（$P(X|Y)$为似然概率：如何从输入数据产生输出，从输入来建模输出）。相比于判别模型，生成模型更蕴含着一种授之以鱼、不如授之以“渔”的意味。生成模型典型方法为贝叶斯方法、隐马尔可夫链。生成模型最大的难点在于联合分布概率$P(X, Y)$或似然概率$P(X|Y)$求取很困难。

4.2 回归分析

在现实生活中，往往需要分析若干变量之间的关系，如碳排放量与气候变暖之间的关系、某一商品广告投入量与该商品销售量之间的关系等，这种分析不同变量之间存在关系的研究叫回归分析，刻画不同变量之间关系的模型被称为回归模型。这个模型如果是线性的，则被称为线

性回归模型，如果是非线性的，则被称为非线性回归模型。一旦确定了回归模型，就可以进行预测等分析工作，如从碳排放量预测气候变化程度、从广告投入量预测商品销售量等。

4.2.1 一元线性回归

勒让德[31]和高斯[32]最早提出通过最小二乘法来实现回归分析。回归（regression）这一种表达最先由英国著名生物学家兼统计学家高尔顿（Galton）提出[14]。高尔顿通过分析统计数据发现，父母平均身高每增加一个单位，其成年子女平均身高只增加0.516个单位，它反映了这种“衰退（regression）”效应（“回归”到正常人平均身高），即子女平均身高y和父母平均身高x之间存在关系：

$$y = 33.73 + 0.516x$$

虽然x和y之间并不总是具有“衰退”（回归）关系，但是“线性回归”这一名称就保留了下来了。

给出任意一对父母平均身高，则可根据上述方程，计算得到其子女平均身高。但是如何求取上述线性方程（预测方程）的参数？

本节从表4.4所给出的例子中来讲解回归模型中参数求取过程。表4.4给出了芒提兹尼欧（Montesinho）地区发生森林火灾的部分历史数据[7]。表中列举了每次发生森林火灾时的气温温度取值x和受到火灾影响的森林面积y。

表4.4 芒提兹尼欧（Montesinho）地区发生森林火灾的部分数据

气温温度x	5.1	8.2	11.5	13.9	15.1	16.2	19.6	23.3
火灾影响面积y	2.14	4.62	8.24	11.24	13.99	16.33	19.23	28.74

从表4.4所给数据，可否对气温温度与火灾所影响的森林面积之间关系进行建模呢？初步观察之后，可以使用简单的线性模型构建两者之间关系，即气温温度x与火灾所影响的森林面积y之间存在$y = ax+b$形式的关系。

有了模型假设和数据（表中的8个数据点，每一个数据点是一个(x, y)的组合），利用数据来确定模型参数（模型假设中的a和b）。

为了求取这个线性关系中的参数a和b，需要给定若干组(x, y)数据，然后从这些数据出发，来计算参数a和b。表4.4给出了8组(x, y)数据，

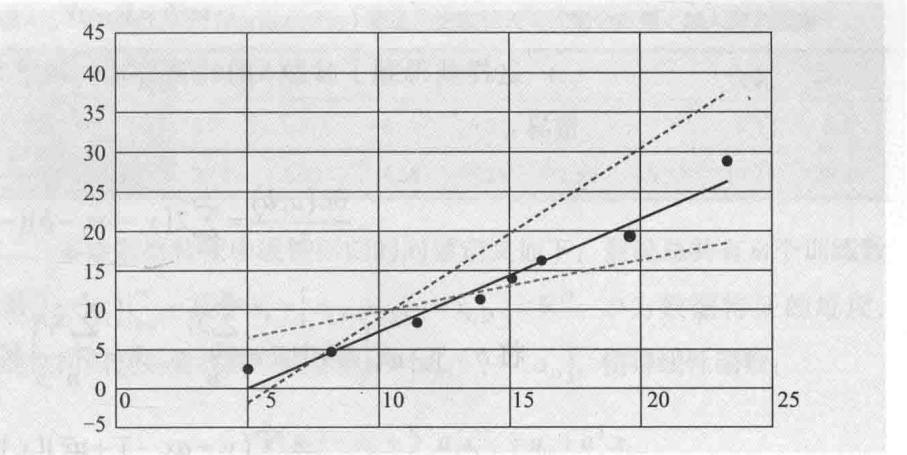


图4.2 火灾所影响的森林面积与天气温度之间的关系图

一元线性回归实际上就是寻找一条用$y=ax+b$表达的直线，使得这条直线尽可能靠近或穿过这8组(x, y)数据，即能够以最小误差来拟合这8组(x, y)数据，如图4.2所示。

最佳回归模型将使得残差平方和的平均值$\frac{1}{N}\sum(y-\tilde{y})^2$最小，残差即预测值和真实值之间的绝对值。残差平方和的平均值最小只与参数a和b有关，最优解即是使得残差最小所对应的a和b的值。

一般而言，回归模型$y_i=ax_i+b(1\leqslant i\leqslant n)$的参数求解过程为：记在当前参数下第$i$个训练样本$x_i$的预测值为$\hat{y}_i$，计算$x_i$的标注值（实际值）$y_i$与预测值$\hat{y}_i$之差$(y_i-\hat{y}_i)^2$，计算训练集中$n$个样本所产生误差总和$L(a,b)=\sum_{i=1}^{n}(y_i-ax_i-b)^2$，使用最小二乘法找到误差总和最小，要使函数具有最小值，可对$L(a, b)$参数a和b分别求导，令其导数值为零，再求取参数a和b的取值。

优化目标：

$$\min_{a,b} L(a,b)=\sum_{i=1}^{n}(y_i-ax_i-b)^2$$

损失函数对b求偏导：

$$\frac{\partial L(a,b)}{\partial b}=\sum_{i=1}^{n}2(y_i-ax_i-b)(-1)=0$$

$$\Rightarrow\sum_{i=1}^{n}(y_i-ax_i-b)=0$$

$$\Rightarrow\sum_{i=1}^{n}(y_i)-a\sum_{i=1}^{n}x_i-\sum_{i=1}^{n}b=0$$

$$\Rightarrow n\bar{y}-na\bar{x}-nb=0$$

$$\Rightarrow b=\bar{y}-a\bar{x}$$

这样就得到了参数b的计算公式，接下来求a，损失函数对a求偏导：

$$\frac{\partial L(a,b)}{\partial a}=\sum_{i=1}^{n}2(y_i-ax_i-b)(-x_i)=0$$

将$b=\bar{y}-a\bar{x}$（其中$\bar{y}=\frac{\sum_{i=1}^{n}y_i}{n}$，$\bar{x}=\frac{\sum_{i=1}^{n}x_i}{n}$）代入上式：

$$\Rightarrow\sum_{i=1}^{n}(y_i-ax_i-\bar{y}+a\bar{x})(x_i)=0$$

$$\Rightarrow\sum_{i=1}^{n}(y_ix_i-ax_ix_i-\bar{y}x_i+a\bar{x}x_i)=0$$

$$\Rightarrow\sum_{i=1}^{n}(y_ix_i-\bar{y}x_i)-a\sum_{i=1}^{n}(x_ix_i-\bar{x}x_i)=0$$

$$\Rightarrow\left(\sum_{i=1}^{n}x_iy_i-n\bar{y}\,\bar{x}\right)-a\left[\sum_{i=1}^{n}\left(x_ix_i-n\bar{x}^2\right)\right]=0$$

$$\Rightarrow a=\frac{\sum_{i=1}^{n}x_iy_i-n\bar{x}\,\bar{y}}{\sum_{i=1}^{n}x_ix_i-n\bar{x}^2}$$

可以看出只要给出了训练样本$(x_i, y_i)(i=1, \cdots, n)$，就可以从训练样本出发，建立一个线性回归方程，使得对训练样本数据而言，该线性回归方程预测的结果与样本标注结果之间的差值和最小。

这样，对于上面的案例，可以求得参数a和b分别为：

$$a=\frac{x_1y_1+x_2y_2+\cdots+x_8y_8-8\bar{x}\,\bar{y}}{x_1^2+x_2^2+\cdots+x_8^2-8\bar{x}^2}=1.428$$

$$b=\bar{y}-a\bar{x}=-7.09$$

即预测芒提兹尼欧地区火灾所影响森林面积与气温温度之间的一元线性回归模型为“火灾所影响的森林面积 = 1.428 × 天气温度 -7.09”，即$y=1.428x-7.09$。

4.2.2 多元线性回归

接下来扩展到数据特征的维度是多维的情况，在上述数据中增加一个影响火灾影响面积的潜在因素——风力，如表4.5所示。

表4.5 芒提兹尼欧（Montesinho）地区历史森林火灾的部分数据（加入风力因素）

气温x	5.1	8.2	11.5	13.9	15.1	16.2	19.6	23.3
风力z	4.5	5.8	4	6.3	4	7.2	6.3	8.5
火灾影响面积y	2.14	4.62	8.24	11.24	13.99	16.33	19.23	28.74

多维数据特征中线性回归的问题定义如下：假设总共有m个训练数据$\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{m}$，其中$\boldsymbol{x}_i=[x_{i,1}, x_{i,2}, \cdots, x_{i,D}]\in\mathbb{R}^D$，$D$为数据特征的维度，线性回归就是要找到一组参数$\boldsymbol{a}=[a_0, a_1, \cdots, a_D]$，使得线性函数：

$$f(\boldsymbol{x}_i)=a_0+\sum_{j=1}^{D}a_j x_{i,j}=a_0+\boldsymbol{a}^{\mathrm{T}}\boldsymbol{x}_i$$

最小化均方误差函数：

$$J_m=\frac{1}{m}\sum_{i=1}^{m}\left(y_i-f(\boldsymbol{x}_i)\right)^2$$

为了方便，使用矩阵来表示所有的训练数据和数据标签。

$$\boldsymbol{X}=[x_1, \cdots, x_m], \boldsymbol{y}=[y_1, \cdots, y_m]$$

其中每一个数据x_i会扩展一个维度，其值为1，对应参数a_0。均方误差函数可以表示为：

$$J_m(\boldsymbol{a})=\left(\boldsymbol{y}-\boldsymbol{X}^{\mathrm{T}}\boldsymbol{a}\right)^{\mathrm{T}}\left(\boldsymbol{y}-\boldsymbol{X}^{\mathrm{T}}\boldsymbol{a}\right)$$

在特征维度只有一维的时候线性回归有闭式解（closed form solution），多维情况下同样存在。

均方误差函数$J_n(\boldsymbol{a})$对所有参数$\boldsymbol{a}$求导可得：

$$\nabla J(\boldsymbol{a})=-2\boldsymbol{X}\left(\boldsymbol{y}-\boldsymbol{X}^{\mathrm{T}}\boldsymbol{a}\right)$$

因为均方误差函数$J_n(\boldsymbol{a})$是一个二次的凸函数，所以函数只存在一个极小值点，也同样是最小值点，所以令$\nabla J(\boldsymbol{a})=0$可得

$$\boldsymbol{X}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{a}=\boldsymbol{X}\boldsymbol{y}$$

$$\boldsymbol{a}=\left(\boldsymbol{X}\boldsymbol{X}^{\mathrm{T}}\right)^{-1}\boldsymbol{X}\boldsymbol{y}$$

对于上面的例子，转化为矩阵的表示形式为：

$$\boldsymbol{X}=\begin{bmatrix}5.1 & 8.2 & 11.5 & 13.9 & 15.1 & 16.2 & 19.6 & 23.3\\ 4.5 & 5.8 & 4 & 6.3 & 4 & 7.2 & 6.3 & 8.5\\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1\end{bmatrix}$$

$$\boldsymbol{y}=[2.14 \quad 4.62 \quad 8.24 \quad 11.24 \quad 13.99 \quad 16.33 \quad 19.23 \quad 28.74]^{\mathrm{T}}$$

其中矩阵$\boldsymbol{X}$多出一行全1，是因为常数项a_0，可以看作是数值为全1的特征的对应系数。计算可得

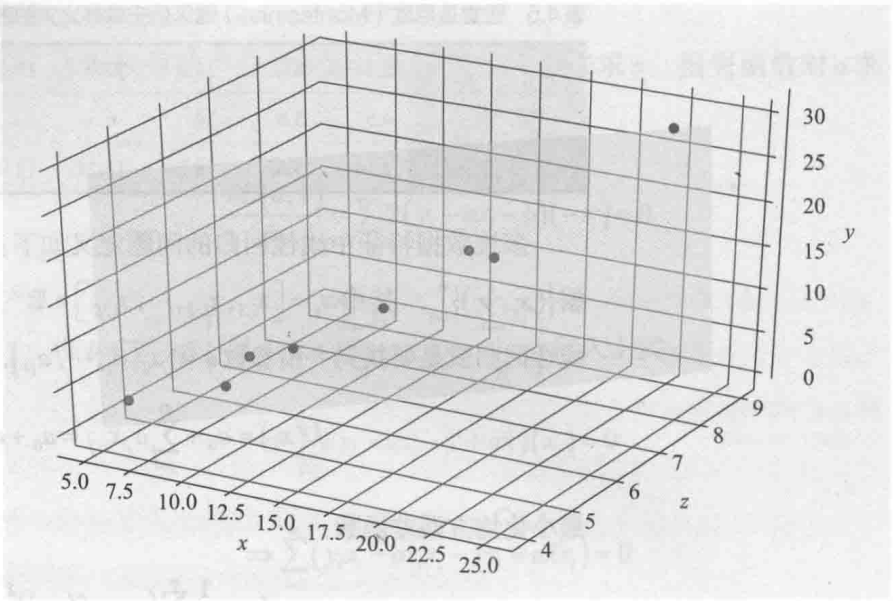


图4.3 二维特征数据线性拟合结果

$$\boldsymbol{a}=\begin{bmatrix}1.312 & 0.626 & -9.103\end{bmatrix}$$

所以对应的线性函数为：

$$y=-9.103+1.312x+0.626z$$

在3D空间可视化结果如图4.3所示。

4.2.3 逻辑斯蒂回归/对数几率回归

线性回归一个明显的问题是对离群点（和大多数数据点距离较远的数据点，outlier）非常敏感，导致模型建模不稳定，使结果有偏，为了缓解这个问题（特别是在二分类场景中）带来的影响，可考虑逻辑斯蒂回归（logistic regression）[9]。

考虑下面的一个案例：某家商铺调查统计了用户对某件商品的拒绝意愿（选择不购买商品的人数/受调查的总人数）随商品价格的变化情况如图4.4所示。

在图4.4中，给出了通过残差平均值最小化得到线性回归模型。在这个线性回归模型中，把0.5作为阈值来判断决定用户是否愿意购买商品，即当拒绝意愿小于0.5时，用户会选择购买商品，反之用户会拒绝购买商品。可以看到图4.4所估计的线性回归模型对用户拒绝购买商品意愿的预测和观测所得实际情况一致。

但是，如果在图4.4中加入一个离群点，该点表示当商品价格为15时，用户拒绝意愿为1（即用户不愿意购买该商品）。加入了该离群点后，

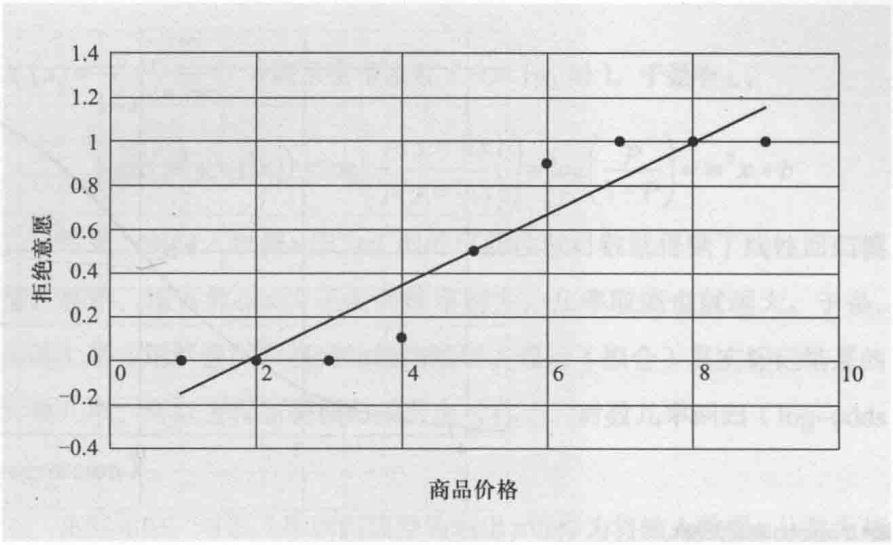


图4.4 用户拒绝购买商品意愿与商品价格之间的关系

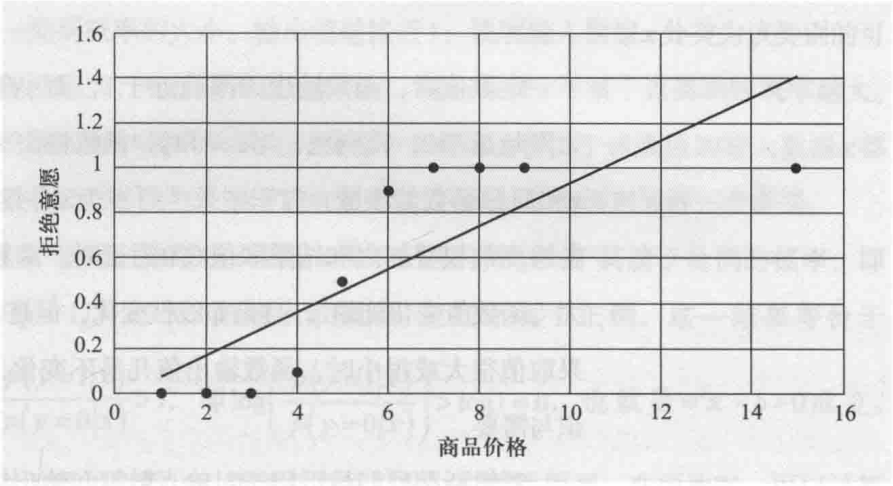


图4.5 加入一个离群点后用户拒绝购买商品意愿与商品价格之间的关系

所估计得到线性回归模型如图4.5所示。

在图4.5中，该回归模型会预测用户将购买价格略大于5的商品（因拒绝购买该商品的拒绝意愿值小于0.5），这将与真实调查数据存在偏差。

逻辑斯蒂回归（logistic regression）就是在回归模型中引入sigmoid函数的一种非线性回归模型。Logistic回归模型可如下表示：

$$y=\frac{1}{1+\mathrm{e}^{-z}}=\frac{1}{1+\mathrm{e}^{-\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}+b\right)}},\ \text{其中}y\in(0,1),z=\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}+\mathrm{b}$$

这里$\frac{1}{1+\mathrm{e}^{-z}}$是sigmoid函数、$\boldsymbol{x}\in\mathbb{R}^{d}$是输入数据、$\boldsymbol{w}\in\mathbb{R}^{d}$和$b\in\mathbb{R}$是回归函数的参数。

图4.6给出了sigmoid函数的可视化结果。从图4.6可知，sigmoid函数具有如下性质：① sigmoid函数是单调递增的，其值域为(0, 1)，因此使sigmoid函数输出可作为概率值，在前面介绍的线性回归中，回归函数的值域一般为$(-\infty,+\infty)$；② 对输入z取值范围没有限制，但当z大

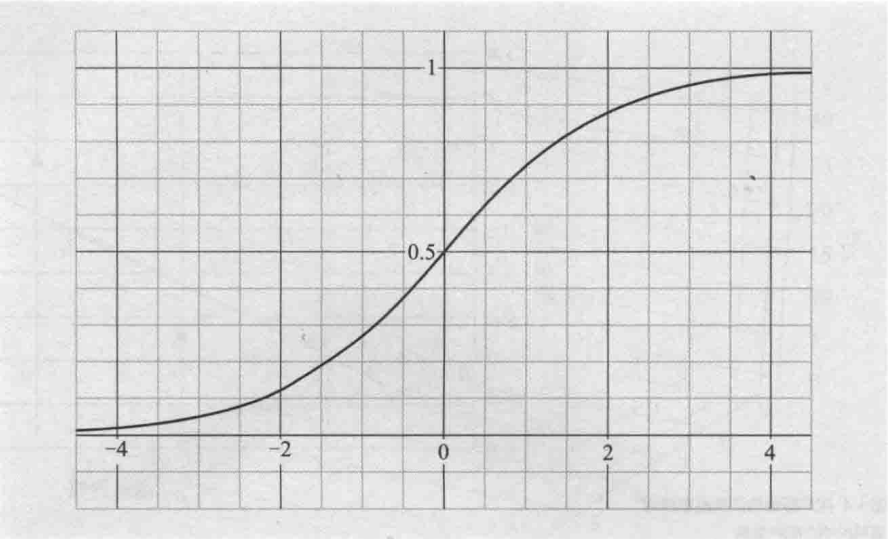


图4.6 sigmoid函数的可视化

于一定数值后，函数输出无限趋近于1，而小于一定数值后，函数输出无限趋近于0，特别地，当$z=0$时，函数输出为0.5，这里z是输入数据$\boldsymbol{x}$和回归函数的参数$\boldsymbol{w}$内积结果（可视为$\boldsymbol{x}$各维度进行加权叠加）；③$\boldsymbol{x}$各维度加权叠加之和结果取值在0附近时，函数输出值的变化幅度比较大（函数值变化陡峭），且是非线性变化，但是，各维度加权叠加之和结果取值很大或很小时，函数输出值几乎不变化，这是基于概率的一种认识与需要。

逻辑斯蒂回归虽可用于对输入数据和输出结果之间复杂关系进行建模，但由于逻辑斯蒂回归函数的输出具有概率意义，使得逻辑斯蒂回归函数更多用于二分类问题（$y=1$表示输入数据$\boldsymbol{x}$属于正例，$y=0$表示输入数据$\boldsymbol{x}$属于负例）。

$y=\dfrac{1}{1+\mathrm{e}^{-\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}+b\right)}}$可用来计算输入数据$\boldsymbol{x}$属于正例概率，这里$y$理解为输入数据$\boldsymbol{x}$为正例的概率、$1-y$理解为输入数据$\boldsymbol{x}$为负例的概率，即$P(y=1|\boldsymbol{x})$。$\dfrac{P}{1-P}$被称为几率（odds），反映了输入数据$\boldsymbol{x}$作为正例的相对可能性。$\dfrac{P}{1-P}$的对数几率（log odds）或logit函数可表示为$\log\left(\dfrac{P}{1-P}\right)$。

显然，可以得到$P\left(y=1|\boldsymbol{x}\right)=h_{\theta}\left(x\right)=\dfrac{1}{1+\mathrm{e}^{-\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}+b\right)}}$和$P\left(y=0|\boldsymbol{x}\right)=1-$

$h_\theta(x)=\dfrac{e^{-(w^{\mathrm{T}}x+b)}}{1+e^{-(w^{\mathrm{T}}x+b)}}$。$\theta$表示模型参数（$\theta=\{w, b\}$）。于是有：

$$\mathrm{logit}\left(P(y=1|x)\right)=\log\left(\frac{P(y=1|x)}{P(y=0|x)}\right)=\log\left(\frac{P}{1-P}\right)=w^{\mathrm{T}}x+b$$

可见，对输入数据x作为正例的可能性取对数就得到了线性回归模型。显然，输入数据x为正例的概率越大，几率取值也就越大。于是，实际上是在用线性回归模型的输出结果去逼近（拟合）真实标记结果的对数几率，所以逻辑斯蒂回归函数也被称为“对数几率回归（log-odds regression）”。

在实际中，对数几率回归模型的输出y可作为将输入数据x分类为某一类别概率的大小。输出值越接近1，说明输入数据x分类为该类别的可能性越大，相反输出值越接近0，输入数据x不属于该类别的概率越大。于是，可以根据具体应用设置一个阈值，将大于该阈值的输入数据x都归属到某个类别，小于该阈值的输入数据x都归属到另外一个类别。

显然，如果输入数据x属于正例的概率大于其属于负例的概率，即$P(y=1|x)>0.5$，则输入数据x可被判断属于正例。这一结果等价于$\dfrac{P(y=1|x)}{P(y=0|x)}>1$，即$\log\left(\dfrac{P(y=1|x)}{P(y=0|x)}\right)>\log 1=0$，也就是$w^{\mathrm{T}}x+b>0$成立。从这里可以看出，逻辑斯蒂回归是一个线性模型。在预测时，可以计算线性函数$w^{\mathrm{T}}x+b$取值是否大于0来判断输入数据x的类别归属。

为了求取逻辑斯蒂回归模型中的参数，还需定义损失函数或目标函数来求解能对观测所得数据进行最佳拟合的参数，典型的做法是最大化对数似然（log likelihood）函数。

模型参数的似然函数被定义为$\mathcal{L}(\theta|\mathcal{D})=P(\mathcal{D}|\theta)$，其中$\mathcal{D}=\{(x_i, y_i)\}$（$1\leqslant i\leqslant n$）表示所有观测数据（或训练数据），$\theta$表示模型参数（$\theta=\{w, b\}$）。在最大化对数似然函数过程中，一般假设观测所得每一个样本数据是独立同分布（independent and identically distributed，i.i.d），于是可得：

$$\mathcal{L}(\theta|\mathcal{D})=P(\mathcal{D}|\theta)=\prod_{i=1}^{n}P(y_i|x,\theta)=\prod_{i=1}^{n}\left(h_\theta(x_i)\right)^{y_i}\left(1-h_\theta(x_i)\right)^{1-y_i}$$

对上述公式取对数：

$$\mathrm{l}(\theta)=\log\left(\mathcal{L}\left(\theta|\mathcal{D}\right)\right)=\sum_{i=1}^{n}y_i\log\left(h_\theta\left(x_i\right)\right)+\left(1-y_i\right)\log\left(1-h_\theta\left(x_i\right)\right)$$

如此就推导出了参数θ的最大似然估计。最大似然估计目的是计算似然函数的最大值，而分类过程是需要损失函数最小化。因此，在上式前加一个负号得到损失函数，这一损失函数就是对数似然的相反数（negative log-likelihood log，又叫交叉熵）：

$$\begin{aligned}\mathcal{T}\left(\theta\right)&=-\mathrm{l}(\theta)=-\log\left(\mathcal{L}\left(\theta|\mathcal{D}\right)\right)\\&=-\left(\sum_{i=1}^{n}y_i\log\left(h_\theta\left(x_i\right)\right)+\left(1-y_i\right)\log\left(1-h_\theta\left(x_i\right)\right)\right)\end{aligned}$$

$\mathcal{T}(\theta)$等价于：$\mathcal{T}\left(\theta\right)=\begin{cases}-\log\left(h_\theta\left(x_i\right)\right) & \text{当}y=1\\ -\log\left(1-h_\theta\left(x_i\right)\right) & \text{当}y=0\end{cases}$

一旦推导得到了逻辑回归的损失函数$\mathcal{T}(\theta)$，需要通过最小化损失函数来求解模型的参数θ，即输入数据$\boldsymbol{x}$各维度的加权系数。对于线性回归模型而言，可以使用最小二乘法，但对于逻辑回归而言使用传统最小二乘法求解是不合适的，需要考虑使用迭代算法进行优化求解，常见的就是“梯度下降法（gradient descent）”。

梯度下降法的一般迭代公式如下：

$$\theta_j=\theta_j+\Delta\theta_j=\theta_j-\eta\frac{\partial J(\theta)}{\partial\theta_j}$$

因此每次只需要求得损失函数对参数θ_j的梯度，就可以不断逼近一个极小值点，其中θ_j表示第j次迭代后更新的参数，η表示学习率（learning rate）。在逻辑斯蒂回归中，这里的$J(\theta)$就是上述的对数损失函数$\mathrm{l}(\theta)$。对数损失函数对参数θ的偏导如下（其中，$h'_\theta(x)=h_\theta(x)\left(1-h_\theta(x)\right),\log' x=\frac{1}{x}$）

$$\begin{aligned}\frac{\partial J(\theta)}{\partial\theta_j}&=-\sum_{i=1}^{n}\left(y_i\frac{1}{h_\theta\left(x_i\right)}\frac{\partial h_\theta\left(x_i\right)}{\partial\theta_j}+\left(1-y_i\right)\frac{1}{1-h_\theta\left(x_i\right)}\frac{\partial\left(1-h_\theta\left(x_i\right)\right)}{\partial\theta_j}\right)\\&=-\sum_{i=1}^{n}\frac{1}{h_\theta\left(x_i\right)}\left(\frac{y_i}{h_\theta\left(x_i\right)}-\frac{1-y_i}{1-h_\theta\left(x_i\right)}\right)\\&=-\sum_{i=1}^{n}x_ih_\theta\left(x_i\right)\left(1-h_\theta\left(x_i\right)\right)\left(\frac{y_i}{h_\theta\left(x_i\right)}-\frac{1-y_i}{1-h_\theta\left(x_i\right)}\right)\\&=-\sum_{i=1}^{n}x_i\left(y_i\left(1-h_\theta\left(x_i\right)\right)-\left(1-y_i\right)h_\theta\left(x_i\right)\right)\end{aligned}$$

$$=\sum_{i=1}^{n}\left(y_i-h_\theta\left(x_i\right)\right)x_i$$

将求导结果代入梯度下降迭代公式得：

$$\theta_j=\theta_j-\eta\sum_{i=1}^{n}\left(y_i-h_\theta\left(x_i\right)\right)x_i$$

这样就可以利用迭代公式不断地更新模型参数，直至收敛。常用的梯度下降法包括批量梯度下降法、随机梯度下降法和小批量梯度下降法，这里不进行详细讲解。

逻辑斯蒂回归只能用于解决二分类问题，将它推广为多项逻辑斯蒂回归模型（multi-nominal logistic model，也即softmax函数），用于处理多类分类问题，可以得到softmax回归。

4.3 决策树

决策树是一种通过树形结构来进行分类的方法。在决策树中，树形结构中每个非叶子结点表示对分类目标在某个属性上的一个判断，每个分支代表基于该属性做出的一个判断，每个叶子结点代表一种分类结果，所以决策树可以看作是一系列以叶子结点为输出的决策规则（decision rules）[21]。

4.3.1 决策树分类案例

决策树将分类问题分解为若干基于单个信息的推理任务，采用树状结构来逐步完成决策判断。事实上，人们在逻辑推理过程中经常使用决策树的思想。

下面通过一个例子来解释决策树的分类。银行的数据分析师希望通过历史的贷款记录，包括用户的4种特征（年龄，银行流水，婚姻状况，房产状况）以及最终是否给予贷款，来建立分类模型辅助决策者进行决策。

银行收集整理的数据如表4.6所示。

通过观察，可以画出如图4.7所示的决策树，由图可知以下几点。

表4.6 是否给予贷款与申请人自身状况的关系

序号	年龄	银行流水	是否结婚	拥有房产	是否给予贷款
1	>30	高	否	是	否
2	>30	高	否	否	否
3	20~30	高	否	是	是
4	<20	中	否	是	是
5	<20	低	否	是	是
6	<20	低	是	否	否
7	20~30	低	是	否	是
8	>30	中	否	是	否
9	>30	低	是	是	是
10	<20	中	否	是	是
11	>30	中	是	否	是
12	20~30	中	否	否	是
13	20~30	高	是	是	是
14	<20	中	否	否	否

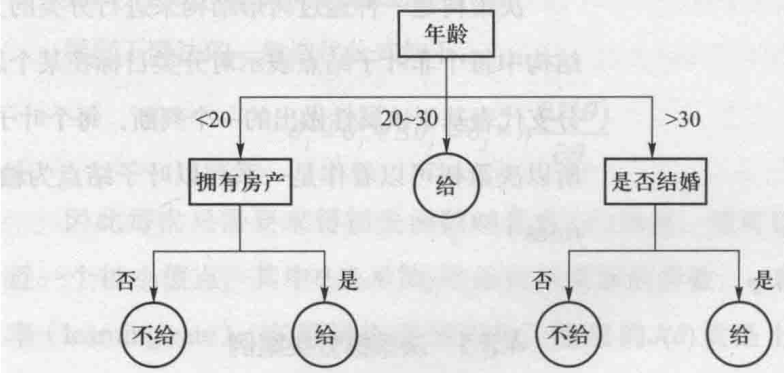


图4.7 银行贷款决策树

• 第一层是年龄状况，分为小于20岁，20岁至30岁，大于30岁三种取值。

• 如果年龄在20岁和30岁之间，样本子集为{3, 7, 12, 13}，这些样本的标签均为“给予贷款”，所以为叶子结点。

• 如果年龄大于30岁，样本子集为{1, 2, 8, 9, 11}，这些样本具有不同的标签，要进一步使用其他属性对这个样本子集进行划分。经观察，通过“是否结婚”这一属性值，可以将该样本子集进一步划分成{1, 2, 8}（未婚）和{9, 11}（已婚）两个样本子集。此时这两个样本子

集内标签一样，不需要再划分。

● 如果年龄小于20岁，样本子集为{4, 5, 6, 10, 14}，这些样本具有不同的标签，同样需要继续划分。通过观察，“拥有房产”这个属性值可将该样本子集进一步划分成{4, 5, 10}（无房产）和{6, 14}（有房产）两个样本子集。此时这两个样本子集内标签一样，不需要再划分。

“银行流水”这一特点及其属性值在这次决策树构造过程中没有使用。

建立决策树的过程，就是不断选择属性值对样本集进行划分，直至每个子样本为同一个类别。上面的案例数据较少，可以通过观察或是穷举的方法来不断选择属性值对样本集进行划分，对于较大的数据集，需要理论和方法来评价不同属性值划分的子样本集的好坏程度，并基于该方法构建决策树。

4.3.2 构建决策树

构建决策树时划分属性的顺序选择是重要的。性能好的决策树随着划分不断进行，决策树分支结点样本集的“纯度”会越来越高，即其所包含样本尽可能属于相同类别。

信息熵（entropy）就是一种衡量样本集合“纯度”的指标，如果计算选择不同属性划分后样本集的“纯度”，那么就可以比较和选择属性。信息熵越大，说明该集合的不确定性越大，“纯度”越低。选择属性划分样本集前后信息熵的减少量被称为信息增益（information gain），也就是说信息增益被用来衡量样本集合复杂度（不确定性）所减少的程度。

假设有K个信息，其组成了集合样本D，记第k个信息发生的概率为$p_k(1 \leqslant k \leqslant K)$。如下定义这$K$个信息的信息熵：

$$E(D) = -\sum_{k=1}^{K} p_k \log_2 p_k$$

$E(D)$值越小，表示D包含的信息越确定，也称D的纯度越高。需要指出，所有p_k累加起来的和为1。

现在应用熵这个度量标准来构建决策树。表4.6中14个样本分属于“给予贷款（9个样本）”和“不给予贷款（5个样本）”两个类别，即$K=2$。

$$Ent(D) = -\sum_{k=1}^{2} p_k \log_2 p_k = -\left(\frac{9}{14} \times \log_2 \frac{9}{14} + \frac{5}{14} \times \log_2 \frac{5}{14}\right) = 0.940$$

表4.6中有年龄、银行流水、是否结婚、拥有房产4个人物相关的属性特征，下面计算这4个特点所对应的信息熵。

以年龄为例，包含“>30”、“20 ~ 30”、“<20”3个属性取值。这3个属性取值对14个样本进行划分，在决策树中产生了3个分支结点，每个分支结点包含一个数据子集，3个数据子集构成了对原数据的划分。如“20 ~ 30”这一属性取值包含4个样本{3, 7, 12, 13}。

对年龄属性划分出的子样本集情况的统计如表4.7所示。这里记属性取值为a_i（即a_0=“>30”，a_1=“20 ~ 30”，a_2=“<20”）。属性取值a_i划分出的子样本集记为D_i，该子样本集包含样本数量记为$|D_i|$。

表4.7 年龄属性划分后子样本集情况统计

年龄属性取值a_i	“>30”	“20~30”	“<20”
对应样本数$\|D_i\|$	5	4	5
正负样本数量	(2+, 3-)	(4+, 0-)	(3+, 2-)

根据表4.7的统计情况，计算每个属性值划分出的子样本集（即每个分支结点）的信息熵：

$$\text{“>30”：}\ Ent(D_0)=-\left(\frac{2}{5}\times\log_2\frac{2}{5}+\frac{3}{5}\times\log_2\frac{3}{5}\right)=0.971$$

$$\text{“20 ~ 30”：}\ Ent(D_1)=-\left(\frac{4}{4}\times\log_2\frac{4}{4}+0\right)=0$$

$$\text{“<20”：}\ Ent(D_2)=-\left(\frac{3}{5}\times\log_2\frac{3}{5}+\frac{2}{5}\times\log_2\frac{2}{5}\right)=0.971$$

得到上述三个的信息熵后，可进一步计算使用年龄属性对原样本集进行划分后的信息增益，计算公式如下：

$$Gain(D, A)=Ent(D)-\sum_{i=1}^{n}\frac{|D_i|}{|D|}Ent(D_i)$$

将A= 年龄 代入。于是选择年龄这一属性划分后的信息增益为：

$$Gain(D, \text{年龄})=0.940-\left(\frac{5}{14}\times0.971+\frac{4}{14}\times0+\frac{5}{14}\times0.971\right)=0.246$$

同理，可以计算银行流水、是否结婚、是否拥有房产3个人物属性的信息增益。通过比较4种属性信息增益的高低来选择最佳属性对原样本集进行划分，得到最大的“纯度”。如果划分后的不同子样本集都只存在同

类样本，那么停止划分。在该案例中，最终可构建如图4.7的决策树。

一般而言，信息增益偏向选择分支多的属性[11]（如上述案例偏向于选择年龄属性），这在一些场合容易导致模型过拟合。为了解决这个问题，一个直接的想法是对分支过多进行惩罚，这就是另外一个“纯度”衡量指标，信息增益率的核心思想。除了计算属性划分后的信息增益为$Gain(D, A)$外，还需要计算划分行为本身带来的信息$info$，$info$和$Gain-ratio$计算公式如下：

$$info = -\sum_{i=1}^{n} \frac{|D_i|}{|D|} \log_2 \frac{|D_i|}{|D|}$$

$$Gain - ratio = Gain(D, A) / info$$

另一种计算更简易的度量指标是如下的$Gini$系数：

$$Gini(D) = 1 - \sum_{k=1}^{K} p_k^2$$

相对于信息熵的计算$E(D) = -\sum_{k=1}^{K} p_k \log_2 p_k$，不用计算对数log，计算更为简易。

4.4 线性判别分析

线性判别分析（linear discriminant analysis，LDA）是一种基于监督学习的降维方法，也称为Fisher线性判别分析（fisher's discriminant analysis，FDA）[12]。对于一组具有标签信息的高维数据样本，LDA利用其类别信息，将其线性投影到一个低维空间上，在低维空间中同一类别样本尽可能靠近，不同类别样本尽可能彼此远离。

LDA与主成分分析（PCA）紧密相关，它们都在寻找最佳解释数据的变量线性组合[18]。

图4.8给出了用矩形和圆圈来表示患有某一疾病或者不患有某一疾病的两类人群。通过调查，这两类人群分别用吸烟频率高低和运动频率高低来描述。当然，在图4.8中，假设这两类人群样本数据符合高斯分布。

为了对这两类人群进行区分，需要将其投影到一个低维空间中。从图中可见，将这些数据向x轴方向和y轴方向投影后，总会存在重叠部分（即若干人群在投影后空间中不可区分）。但是，如果将数据向直线w所

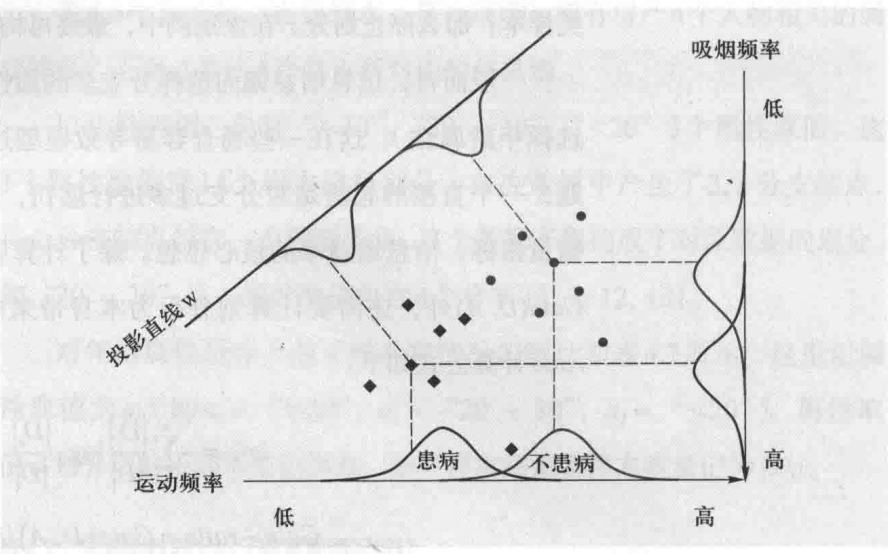


图4.8 两个类别数据所对应的不同投影方式

位于方向投影，则可见两类数据可被完全区分开来，更进一步分析，在所得投影空间中，同一类别人群数据聚集在一起、不同类别人群数据具有较大间隔，体现了“类内方差小、类间间隔大”的原则。

这好比《论语·子路》中所言“君子和而不同，小人同而不和。”虽然同一类别中每个数据各有特色，但是它们均具有类似内在模式（如在它们所嵌入低维空间存在一个由均值和方差所构成的恰当高斯分布，使得同一类别数据可被投影映射到一起）。

有了直观的认识之后，下面从定量的角度来介绍如何找到投影方向。

假设样本集为 $\boldsymbol{D}=\{(\boldsymbol{x}_1,y_1),(\boldsymbol{x}_2,y_2),(\boldsymbol{x}_3,y_3),\cdots,(\boldsymbol{x}_N,y_N)\}$，样本 $\boldsymbol{x}_i\in\mathbb{R}^d$ 的类别标签为 y_i。其中，y_i 的取值范围是 $\{\mathcal{C}_1,\mathcal{C}_2,\cdots,\mathcal{C}_K\}$，即一共有 K 类样本。

定义 $\boldsymbol{X}$ 为所有样本构成集合、N_i 为第 i 个类别所包含样本个数、$\boldsymbol{X}_i$ 为第 i 类样本的集合、$\boldsymbol{m}$ 为所有样本的均值向量、$\boldsymbol{m}_i$ 为第 i 类样本的均值向量。$\boldsymbol{\Sigma}_i$ 为第 i 类样本的协方差矩阵，其定义为：

$$\boldsymbol{\Sigma}_i=\sum_{\boldsymbol{x}\in\boldsymbol{X}_i}(\boldsymbol{x}-\boldsymbol{m}_i)(\boldsymbol{x}-\boldsymbol{m}_i)^{\mathrm{T}}$$

先来看 $K=2$ 的情况，即二分类问题。在二分类问题中，训练样本归属于 $\mathcal{C}_1$ 或 $\mathcal{C}_2$ 两个类别，并通过如下的线性函数投影到一维空间上：

$$y(\boldsymbol{x})=\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}\left(\boldsymbol{w}\in\mathbb{R}^n\right)$$

投影之后类别 $\mathcal{C}_1$ 的协方差矩阵 $\boldsymbol{s}_1$ 为：

$$s_1 = \sum_{x \in C_1} \left(\boldsymbol{w}^{\mathrm{T}} \boldsymbol{x} - \boldsymbol{w}^{\mathrm{T}} \boldsymbol{m}_1 \right)^2 = \boldsymbol{w}^{\mathrm{T}} \sum_{x \in C_1} \left[(\boldsymbol{x} - \boldsymbol{m}_1)(\boldsymbol{x} - \boldsymbol{m}_1)^{\mathrm{T}} \right] \boldsymbol{w}$$

同理可得到投影之后类别C_2的协方差矩阵s_2。

可以看到，投影后两个协方差矩阵s_1和s_2分别为$\boldsymbol{w}^{\mathrm{T}}\boldsymbol{\Sigma}_1\boldsymbol{w}$和$\boldsymbol{w}^{\mathrm{T}}\boldsymbol{\Sigma}_2\boldsymbol{w}$。由于将数据样本投影到了一维空间直线上，$s_1$和$s_2$都是实数。$s_1$和$s_2$可用来衡量同一类别数据样本之间“分散程度”。为了使得归属于同一类别的样本数据在投影后的空间中尽可能靠近，需要最小化$s_1 + s_2$取值。

在投影之后的空间中，归属于两个类别的数据样本中心可分别如下计算：

$$m_1 = \boldsymbol{w}^{\mathrm{T}} \boldsymbol{m}_1, \quad m_2 = \boldsymbol{w}^{\mathrm{T}} \boldsymbol{m}_2$$

这样，就可以通过$\|m_2 - m_1\|_2^2$来衡量不同类别之间的距离。为了使得归属于不同类别的样本数据在投影后空间中尽可能彼此远离，需要最大化过$\|m_2 - m_1\|_2^2$取值。

同时考虑上面两点，就得到了需要最大化的目标$J(\boldsymbol{w})$，定义如下：

$$J(\boldsymbol{w}) = \frac{\|m_2 - m_1\|_2^2}{s_1 + s_2}$$

可以把上述式子右边改写成与$\boldsymbol{w}$相关的式子：

$$J(\boldsymbol{w}) = \frac{\left\| \boldsymbol{w}^{\mathrm{T}} (\boldsymbol{m}_2 - \boldsymbol{m}_1) \right\|_2^2}{\boldsymbol{w}^{\mathrm{T}} \boldsymbol{\Sigma}_1 \boldsymbol{w} + \boldsymbol{w}^{\mathrm{T}} \boldsymbol{\Sigma}_2 \boldsymbol{w}} = \frac{\boldsymbol{w}^{\mathrm{T}} (\boldsymbol{m}_2 - \boldsymbol{m}_1)(\boldsymbol{m}_2 - \boldsymbol{m}_1)^{\mathrm{T}} \boldsymbol{w}}{\boldsymbol{w}^{\mathrm{T}} (\boldsymbol{\Sigma}_1 + \boldsymbol{\Sigma}_2) \boldsymbol{w}} = \frac{\boldsymbol{w}^{\mathrm{T}} \boldsymbol{S}_b \boldsymbol{w}}{\boldsymbol{w}^{\mathrm{T}} \boldsymbol{S}_w \boldsymbol{w}}$$

其中，$\boldsymbol{S}_b$称为类间散度矩阵（between-class scatter matrix），即衡量两个类别均值点之间的“分离”程度，可定义如下：

$$\boldsymbol{S}_b = (\boldsymbol{m}_2 - \boldsymbol{m}_1)(\boldsymbol{m}_2 - \boldsymbol{m}_1)^{\mathrm{T}}$$

$\boldsymbol{S}_w$则称为类内散度矩阵（within-class scatter matrix），即衡量每个类别中数据点的“分离”程度，可定义如下：

$$\boldsymbol{S}_w = \boldsymbol{\Sigma}_1 + \boldsymbol{\Sigma}_2$$

由于$J(\boldsymbol{w})$的分子和分母都是关于$\boldsymbol{w}$的二项式，因此最后的解只与$\boldsymbol{w}$的方向有关，与$\boldsymbol{w}$的长度无关，因此可令分母$\boldsymbol{w}^{\mathrm{T}}\boldsymbol{S}_w\boldsymbol{w} = 1$，然后用拉格朗日乘子法来求解这个问题。拉格朗日乘子法就是求在某个/某些约束条件下的函数极值的方法，其主要思想是将约束条件函数与原函数联立，从而求出使原函数取得极值时各个变量的解。

上述带约束条件（即$\boldsymbol{w}^{\mathrm{T}}\boldsymbol{S}_w\boldsymbol{w} - 1 = 0$）的函数极大值（即$\boldsymbol{w}^{\mathrm{T}}\boldsymbol{S}_b\boldsymbol{w}$取值最大）优化问题所对应拉格朗日函数为：

$$L(\boldsymbol{w})=\boldsymbol{w}^{\mathrm{T}}\boldsymbol{S}_b\boldsymbol{w}-\lambda\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{S}_w\boldsymbol{w}-1\right)$$

对$\boldsymbol{w}$求偏导并使其求导结果为零，可得$\boldsymbol{S}_w^{-1}\boldsymbol{S}_b\boldsymbol{w}=\lambda\boldsymbol{w}$。由此可见，$\lambda$和$\boldsymbol{w}$分别是$\boldsymbol{S}_w^{-1}\boldsymbol{S}_b$的特征根和特征向量，$\boldsymbol{S}_w^{-1}\boldsymbol{S}_b\boldsymbol{w}=\lambda\boldsymbol{w}$也被称为Fisher线性判别（Fisher linear discrimination）。

因为$\boldsymbol{S}_b=(\boldsymbol{m}_2-\boldsymbol{m}_1)(\boldsymbol{m}_2-\boldsymbol{m}_1)^{\mathrm{T}}$，令实数$\lambda_w=(\boldsymbol{m}_2-\boldsymbol{m}_1)^{\mathrm{T}}\boldsymbol{w}$，那么$\boldsymbol{S}_b\boldsymbol{w}=(\boldsymbol{m}_2-\boldsymbol{m}_1)(\boldsymbol{m}_2-\boldsymbol{m}_1)^{\mathrm{T}}\boldsymbol{w}=(\boldsymbol{m}_2-\boldsymbol{m}_1)\lambda_w$，将其代入Fisher线性判别，可得：

$$\boldsymbol{S}_w^{-1}\boldsymbol{S}_b\boldsymbol{w}=\boldsymbol{S}_w^{-1}(\boldsymbol{m}_2-\boldsymbol{m}_1)\lambda_w=\lambda\boldsymbol{w}$$

由于对$\boldsymbol{w}$的放大和缩小操作不影响结果，因此可约去上式中的未知数λ和λ_w，得到：

$$\boldsymbol{w}=\boldsymbol{S}_w^{-1}(\boldsymbol{m}_2-\boldsymbol{m}_1)$$

于是，为了获得“类内汇聚、类间间隔”的最佳投影结果，只需要分别求出待投影数据的均值和方差，就可以设计得到最佳投影方向$\boldsymbol{w}$，这就是线性判别分析的做法。将原始数据通过$\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}$进行投影，实现了从高维到低维的映射，因此也是一种降维操作，实现了数据约简，且这一降维结果保持了样本数据的“类内汇聚、类间间隔”结构分布。

LDA模型可从两类问题被拓展到多类问题[15, 23]。假设n个原始高维数据所构成的类别种类为K、每个原始数据被投影映射到低维空间中的维度为r。

令投影矩阵$\boldsymbol{W}=(\boldsymbol{w}_1, \boldsymbol{w}_2, \cdots, \boldsymbol{w}_r)$，可知$\boldsymbol{W}$是一个$n\times r$矩阵。于是，$\boldsymbol{W}^{\mathrm{T}}\boldsymbol{m}_i$为第$i$类样本数据中心在低维空间的投影结果，$\boldsymbol{W}^{\mathrm{T}}\boldsymbol{\Sigma}_i\boldsymbol{W}$为第$i$类样本数据协方差在低维空间的投影结果。

类内散度矩阵$\boldsymbol{S}_w$重新定义如下：

$$\boldsymbol{S}_w=\sum_{i=1}^{K}\boldsymbol{\Sigma}_i,\ \text{其中}\ \boldsymbol{\Sigma}_i=\sum_{x\in class\ i}(\boldsymbol{x}-\boldsymbol{m}_i)(\boldsymbol{x}-\boldsymbol{m}_i)^{\mathrm{T}}$$

在上式中，$\boldsymbol{m}_i$是第i个类别中所包含样本数据的均值。

在两类问题中，类间散度矩阵$\boldsymbol{S}_b$是度量两个类别均值点的分离情况。类似地，令$\boldsymbol{m}_i$为第i类样本的均值、$\boldsymbol{m}$为所有样本的均值。如果某个类别中落入的样本数据多，则这个类别权重就大，因此引入权重$\frac{N_i}{N}$（N是样本数据总数、N_i是第i个类别中所包含样本数据总数），于是类间散度矩阵$\boldsymbol{S}_b$可重新定义如下：

$$\boldsymbol{S}_b=\sum_{i=1}^{K}\frac{N_i}{N}(\boldsymbol{m}_i-\boldsymbol{m})(\boldsymbol{m}_i-\boldsymbol{m})^{\mathrm{T}}$$

可以看出，在$\boldsymbol{S}_b$求取过程中，不再是计算两个类别均值点的散列程度，而是计算每个类别均值点与所有样本均值点之间散列程度的加权累加。

注意到优化目标中分子分母已不再是实数，因此常见的方法是将多类LDA映射投影方向的优化目标$J(\boldsymbol{W})$改为：

$$J(\boldsymbol{W})=\frac{\prod_{diag}\boldsymbol{W}^{\mathrm{T}}\boldsymbol{S}_b\boldsymbol{W}}{\prod_{diag}\boldsymbol{W}^{\mathrm{T}}\boldsymbol{S}_w\boldsymbol{W}}$$

其中，$\prod_{diag}\mathbb{A}$为矩阵$\mathbb{A}$主对角元素的乘积。

继续对$J(\boldsymbol{W})$进行变形：

$$J(\boldsymbol{W})=\frac{\prod_{diag}\boldsymbol{W}^{\mathrm{T}}\boldsymbol{S}_b\boldsymbol{W}}{\prod_{diag}\boldsymbol{W}^{\mathrm{T}}\boldsymbol{S}_w\boldsymbol{W}}=\frac{\prod_{i=1}^{r}\boldsymbol{w}_i^{\mathrm{T}}\boldsymbol{S}_b\boldsymbol{w}_i}{\prod_{i=1}^{r}\boldsymbol{w}_i^{\mathrm{T}}\boldsymbol{S}_w\boldsymbol{w}_i}=\prod_{i=1}^{r}\frac{\boldsymbol{w}_i^{\mathrm{T}}\boldsymbol{S}_b\boldsymbol{w}_i}{\boldsymbol{w}_i^{\mathrm{T}}\boldsymbol{S}_w\boldsymbol{w}_i}$$

显然需要使乘积式子中每个$\frac{\boldsymbol{w}_i^{\mathrm{T}}\boldsymbol{S}_b\boldsymbol{w}_i}{\boldsymbol{w}_i^{\mathrm{T}}\boldsymbol{S}_w\boldsymbol{w}_i}$取值最大，这就是二分类问题的求解目标，即每一个$\boldsymbol{w}_i$都是$\boldsymbol{S}_w^{-1}\boldsymbol{S}_b\boldsymbol{W}=\lambda\boldsymbol{W}$的一个解。

由于$\frac{\boldsymbol{w}_i^{\mathrm{T}}\boldsymbol{S}_b\boldsymbol{w}_i}{\boldsymbol{w}_i^{\mathrm{T}}\boldsymbol{S}_w\boldsymbol{w}_i}$与$\lambda$正相关，因此所求映射矩阵$\boldsymbol{W}$就由矩阵$\boldsymbol{S}_w^{-1}\boldsymbol{S}_b$前$r$个最大特征根所对应的特征向量$(\boldsymbol{w}_1, \boldsymbol{w}_2, \cdots, \boldsymbol{w}_r)$构成。

于是，给定原始d维数据样本$\boldsymbol{x}_i$，通过$\boldsymbol{x}_i\boldsymbol{W}$将其从$d$维空间映射到$r$维空间，实现了原始数据的降维，也使得原始数据形成了具有区别力的紧凑表达。在后续分类等操作中，只是使用$\boldsymbol{x}_i$降维后的结果，而不是$\boldsymbol{x}_i$的原始特征。

对线性判别分析的降维步骤描述如下：

（1）计算数据样本集中每个类别样本的均值；

（2）计算类内散度矩阵$\boldsymbol{S}_w$和类间散度矩阵$\boldsymbol{S}_b$；

（3）根据$\boldsymbol{S}_w^{-1}\boldsymbol{S}_b\boldsymbol{W}=\lambda\boldsymbol{W}$来求解$\boldsymbol{S}_w^{-1}\boldsymbol{S}_b$所对应前$r$个最大特征值所对应特征向量$(\boldsymbol{w}_1, \boldsymbol{w}_2, \cdots, \boldsymbol{w}_r)$，构成矩阵$\boldsymbol{W}$；

（4）通过矩阵$\boldsymbol{W}$将每个样本映射到低维空间，实现特征降维。

需要注意的是，通过LDA对原始d维数据进行降维后，所得维度r最大取值为$\min(K-1, d)$，这是因为$\boldsymbol{S}_b$的秩为$\min(K-1, d)$。这也说明了在二分类问题中，原始高维数据只能被投影到一维空间中（无论其原始维度是多少）。

主成分分析（PCA）是一种无监督学习的降维方法（无需样本类别标签），线性判别分析（LDA）是一种监督学习的降维方法（需要样本类别标签）。PCA和LDA均是优化寻找特征向量$\boldsymbol{w}$来实现降维，其中PCA寻找投影后数据之间方差最大的投影方向、LDA寻找“类内方差小、类间距离大（最小化类内距离、最大化类间距离）”投影方向。

PCA对高维数据降维后的维数是与原始数据特征维度相关（与数据类别标签无关）。假设原始数据维度为d，那么PCA所得数据的降维维度可以为小于d的任意维度。LDA降维后所得到维度是与数据样本的类别个数K有关。假设原始d维数据一共有K个类别，那么LDA所得数据的降维维度小于或等于$\min(K-1, d)$。

4.5 Ada Boosting

Ada Boosting（自适应提升）通过集成（ensemble）手段来达到提升（boosting）算法性能目的，该算法的核心思想是[13]：对于一个复杂的分类任务，可将其分解为若干个子任务，然后将若干个子任务综合构建到一起，最终完成该复杂分类任务。由于每一个子任务只能完成整体任务的“部分”，因此是一个弱分类器（weak classifiers），将若干个弱分类器组合起来，就形成了一个强分类器（strong classifier），从而能完成复杂分类任务。在这里，所谓弱分类器指那些错误率略低于50%的算法。

这种思想古代哲人有所提及，如“能用众力，则无敌于天下矣；能用众智，则无畏于圣人矣”（《三国志·吴志·孙权传》）以及“积力之所举，则无不胜也；众智之所为，则无不成也”（《淮南子·主术训》）。

这一节先介绍可计算学习理论，然后介绍与该理论相关的Ada Boosting算法。

4.5.1 可计算学习理论

人工智能起源于可计算理论，可计算理论关心什么样的问题可以被计算（computable）。一个任务，如果是图灵可停机的，那么该任务可计算。可学习理论（learnability theory）关心什么样的任务是可以被习得，从而能被算法模型来完成。例如，是否存在一个算法，可以识别人脸，

或是跟踪人体的运动，这样的算法背后的原理又是什么。Leslie Valiant（2010年图灵奖获得者）和其学生Michael Kearns 两位学者提出了这个问题并进行了有益探索，逐渐完善了计算学习理论。

在解释计算学习理论之前，这里先简单介绍Wassily Hoeffding于1963年提出并证明的霍夫丁不等式（Hoeffding's inequality）。如果要统计某一电视节目的收视率，比较可行的方法只能抽样一部分人口，然后统计所抽样人口中观看该电视节目的比例作为该电视节目收视率。假设全国人口中观看该电视节目的人口比例为u、抽样人口中观看该电视节目的人口比例为v，则存在满足如下条件的霍夫丁不等式：$P(|u-v| \geqslant \varepsilon) \leqslant 2e^{-2N\varepsilon^2}$（其中，$N$是采样人口总数、$\varepsilon \in (0, 1)$是所设定的可容忍误差范围）。

可以看出，如果N越大，通过采样计算所得收视率v与实际收视率u之间差值超过可容忍误差ε的概率就越小。也就是说，随着N逐渐增加，v和u相等的结论大概近似正确（probably approximately correct，PAC）。

在上述收视率统计问题中，由于不可能通过寻访每个受众而统计得到真实的收视率。于是，是否存在一个学习算法，可得到大概近似正确的收视率呢？这就属于概率近似正确（probably approximately correct，PAC）[29]所研究范畴。PAC研究的问题包括：如何知道学习所得的假设（hypothesis）是正确的、为了接近真实假设所需要的训练数据是多少、假设空间的复杂度如何度量以及如何选择假设空间[10]。

这里借助表4.1的案例来介绍概率近似正确的一些基本概念。

- 学习任务描述：人具有是否数学好、是否身体好、是否会编程、是否嗓门大这四类特征，学习算法（即分类器）需要根据这四类特征判断某个人是否是程序员。

- 实例集$\boldsymbol{X}$：一组特征组合被称为一个实例$\boldsymbol{x} \in \boldsymbol{X}$，在这个任务中，有4类特征，每类特征有“是”或“否”两个取值，所以实例集大小为$|X| = 2 \times 2 \times 2 \times 2 = 16$。

- 概念集$\boldsymbol{C}$：每个实例所对应标签被称为概念$\boldsymbol{c} \in \boldsymbol{C}$，在这个任务中，标签指是否为程序员，是为1，否为0。如下给出了四个均可分类为程序员的实例。

$$\boldsymbol{C} = \{((1,0,1,1),1),((1,0,1,0),1),((1,1,1,1),1),((1,1,1,0),1)\}$$

• 假设空间 $\mathcal{H}$：一个假设 $h(h \in \mathcal{H})$ 可以被简单认为是一个分类器。在上述的案例中，一个假设或一个分类器会对某类特征做出三种评估，如对于“是否数学好”这类特征，一个假设会认为“数学好是程序员的内在秉性”，或者认为“数学不好是程序员的内在秉性”，或者认为“是否数学好对于判断一个人是否是程序员没有作用”。所以上述任务的假设空间大小为 $|\mathcal{H}| = 3 \times 3 \times 3 \times 3 + 1 = 82$。最后的1指一个极端情况，即无论四类特征如何取值，假设空间均不会将实例判断为程序员。

• 实例分布 $\mathfrak{D}$：所有实例数据的统计分布。

• 数据集 $\boldsymbol{D}$：任务所对应的数据集。

概率近似正确可学习（probably approximately correct learnable，PAC-learnable）考虑在样本大小为 n 的实例集 $\boldsymbol{X}$ 上定义的概念集 $\boldsymbol{C}$。则概念集 $\boldsymbol{C}$ 概率近似正确可学习指的是：存在一个复杂度为 $O\left(\text{poly}\left(\frac{1}{\epsilon}, \frac{1}{\delta}\right)\right)$ 的算法 $A(\epsilon, \delta)$，对于所有的 $\boldsymbol{c} \in \boldsymbol{C}$ 以及 $\boldsymbol{X}$ 上的所有分布 $\boldsymbol{\mathcal{D}} \in \mathfrak{D}$，当 $0 < \epsilon, \delta < \frac{1}{2}$ 时，算法 A 可产生一个假设 $h \in \mathcal{H}$，h 至少以概率 $1-\delta$ 得到错误率小于 ϵ 的性能，即 $Pr_D(\boldsymbol{err}_{c,D}(h) \leqslant \epsilon) \geqslant 1-\delta$。这里可见 h 能够完成任务的概率大于0.5（因为 $\sigma < \frac{1}{2}$，所以 $1-\sigma > \frac{1}{2}$），即 h 的性能大于随机分类器（因此 h 是弱分类器）。

在概率近似正确可学习中，算法模型可分为“强可学习模型”和“弱可学习模型”。强可学习（strongly learnable）指学习模型能够以较高精度对绝大多数样本完成识别分类任务；弱可学习（weakly learnable）指学习模型仅能完成若干部分样本识别与分类，其精度略高于随机猜测（即弱可学习模型完成任务的正确率要高于0.5）。概率近似正确指出，强可学习和弱可学习是等价的，也就是说，如果已经发现了“弱学习算法”，可将其提升（boosting）为“强学习算法”。Ada Boosting算法就是这样的提升方法。具体而言，Ada Boosting将一系列弱分类器组合起来，构成一个强分类器。

4.5.2 Ada Boosting算法

Ada Boosting算法需要解决两个核心问题：① 在每个弱分类器学习过程中，如何提高在上一轮中被错误分类样本的权重，即关注目前尚未被正确分类的样本；② 如何将一系列弱分类器组合成强分类器，给分类误

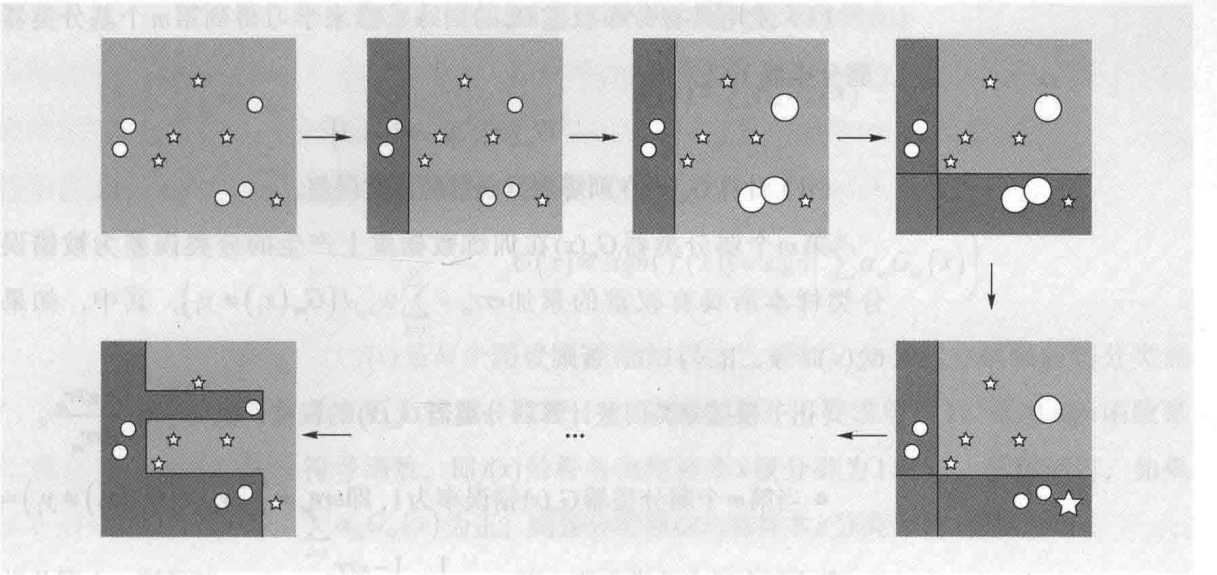


图4.9 Ada Boosting算法学习过程示意图

差小的弱分类器赋予更大权重，同时减少分类误差大的弱分类器的权重。

图4.9示意了Ada Boosting算法通过集成学习方法，将若干个弱分类器组合为一个强分类器，完成两类分类任务。

在图4.9中，把垂直于或平行于x轴的线性分类器作为基分类器（弱分类器），通过组合若干个基分类器来完成图中圆形和星形两类数据的分类。在Ada Boosting中，依次学习得到不同的基分类器，通过线性加权方式对这些基分类器进行组合，最终得到一个强分类器将圆形和星形两类数据区分开来。在这个过程中，被错误分类数据的权重在下一轮基分类器训练之前会提高（在图中该数据形状变大），以便让后续分类器更关注前面分类器无法区分的数据。

Ada Boosting算法的流程如下。

（1）数据样本权重初始化

给定包含N个标注数据的训练集合$\boldsymbol{\Gamma}$，$\boldsymbol{\Gamma}=\{(x_1, y_1), \cdots, (x_N, y_N)\}$。$x_i(1 \leqslant i \leqslant N) \in \boldsymbol{X} \subseteq \mathbb{R}^n$，$y_i \in \boldsymbol{Y}=\{-1, 1\}$

Ada Boosting算法将从这些标注数据出发，训练得到一系列弱分类器，并将这些弱分类器线性组合成一个强分类器。初始化每个训练样本的权重：

$$\boldsymbol{D}_1=\left(w_{1,1}, \cdots, w_{1,i}, \cdots, w_{1,N}\right)，\text{其中 } w_{1,i}=\frac{1}{N}(1 \leqslant i \leqslant N)$$

（2）分别训练M个基分类器

设$m=1, 2, \cdots, M$。

（i）使用具有分布权重$\boldsymbol{D}_m$的训练数据来学习得到第m个基分类器（弱分类器）G_m。

$$G_m(x): \boldsymbol{X} \to \{-1, 1\}$$

（ii）计算$G_m(x)$在训练数据集上的分类误差。

第m个弱分类器$G_m(x)$在训练数据集上产生的分类误差为被错误分类样本所具有权重的累加$err_m = \sum_{i=1}^{N} w_{m,i} I\left(G_m(x_i) \neq y_i\right)$，其中，如果$G_m(x_i) \neq y_i$，I(·)=1，否则为0。

（iii）根据分类误差计算弱分类器$G_m(x)$的权重：$\alpha_m = \frac{1}{2}\ln\frac{1-err_m}{err_m}$。

- 当第m个弱分类器$G_m(x)$错误率为1，即$err_m = \sum_{i=1}^{N} w_{m,i} I\left(G_m(x_i) \neq y_i\right) =$ 1，意味每个样本分类出错，则$\alpha_m = \frac{1}{2}\ln\frac{1-err_m}{err_m} \to -\infty$，赋予第$m$个弱分类器$G_m(x)$极低权重。
- 当第m个弱分类器$G_m(x)$错误率为$\frac{1}{2}$，$\alpha_m = \frac{1}{2}\ln\frac{1-err_m}{err_m} = 0$。前面知道，错误率为$\frac{1}{2}$，相等于该弱分类器性能相当于随机分类器所取得的性能，因此其权重为0。如果错误率$err_m < \frac{1}{2}$，权重$\alpha_m > 0$。可知权重α_m随err_m减少而增大，即错误率越小的弱分类器会赋予更大权重。

（iv）更新下一轮第$m+1$个分类器训练时第i个训练样本的权重：$\boldsymbol{D}_{m+1} = \left(w_{m+1,1}, \cdots, w_{m+1,i}, \cdots, w_{m+1,N}\right)$，$w_{m+1,i} = \frac{w_{m,i}}{Z_m} e^{-\alpha_m y_i G_m(x_i)}$，其中$Z_m$是归一化因子以使得$\boldsymbol{D}_{m+1}$为概率分布，$Z_m = \sum_{i=1}^{N} w_{m,i} e^{-\alpha_m y_i G_m(x_i)}$。

可以把对第i个训练样本更新后的分布权重写为如下分段函数形式：

$$w_{m+1,i} = \begin{cases} \frac{w_{m,i}}{Z_m} e^{-\alpha_m}, & G_m(x_i) = y_i \\ \frac{w_{m,i}}{Z_m} e^{\alpha_m}, & G_m(x_i) \neq y_i \end{cases}$$

可见，如果第i个训练样本无法被第m个弱分类器$G_m(x)$分类成功，则需要增大该样本权重，否则减少该样本权重。这样，被错误分类样本会在训练第$m+1$个弱分类器$G_{m+1}(x)$时被“重点关注”。

在每一轮学习过程中，Ada Boosting算法均在划重点（即重视当前尚未被正确分类的样本）

（3）以线性加权形式来组合弱分类器得到$f(x)$

$$f(x)=\sum_{i=1}^{M}\alpha_m G_m(x)$$

从而得到强分类器$G(x)$：

$$G(x)=sign(f(x))=sign\left(\sum_{i=1}^{M}\alpha_m G_m(x)\right)$$

$f(x)$是M个弱分类器的加权线性累加。分类能力越强的弱分类器具有更大权重。注意，α_m累加之和并不要求等于1。这里$sign$函数是符号函数，即$f(x)$的符号决定样本x被分类为1或-1。具体而言，如果$\sum_{i=1}^{M}\alpha_m G_m(x)$为正，则强分类器$G(x)$将样本$x$分类为1；否则为-1。

4.5.3 从霍夫丁不等式解释Ada Boosting算法

假设有M个弱分类器$G_m(1\leqslant m\leqslant M)$，则$M$个弱分类器线性组合所产生的误差满足如下条件：

$$P\left(\sum_{i=1}^{M}G_m(x)\neq\zeta(x)\right)\leqslant e^{-\frac{1}{2}M(1-2\varepsilon)^2}$$

其中$\zeta(x)$是真实分类函数，$\varepsilon\in(0,1)$。上式表明，如果所“组合”的弱分类器越多，则学习分类误差呈指数级下降，直至为零。

上述不等式成立有两个前提条件：① 每个弱分类器产生的误差相互独立；② 每个弱分类器的误差率小于50%。因为每个弱分类器均是在同一个训练集上产生，条件①难以满足。也就说，“准确性（对分类结果而言）”和“差异性（对每个弱分类器而言）”难以同时满足。

Ada Boosting采取了序列化学习机制，即一个分类器学习完成之后才会进行下一个分类器的学习，后一个分类器的学习受到前面分类器学习的影响。有些学习策略，如Bagging的集成学习方法，采用并行学习的策略，不同分类器的学习互不影响，两种学习策略各有优劣，目标不同，这里不进行详细展开。

Ada Boosting实际是在**最小化指数损失函数**（minimization of exponential loss）：

$$\sum_i e^{-y_i f(x_i)}=\sum_i e^{-y_i\sum_{m=1}^{M}\alpha_m G_m(x_i)}$$

其中$f(x_i)$为一系列基分类器的线性加权组合，y_i是数据标签（$y_i\in$

{-1, 1}），该损失函数表明Ada Boosting学习目标是同时优化分类器权重α_m和基分类器$G_m(x_i)$的参数。在优化过程中，Ada Boosting采用的是一种轮次按序优化方法，即在前$m-1$个基分类器$G_1, \cdots, G_{m-1}$以及它们所对应权重$\alpha_1, \cdots, \alpha_{m-1}$都确定后，再去训练基分类器$G_m$参数和其对应的权重$\alpha_m$。

Ada Boosting的分类误差上界如下所示：

$$\frac{1}{N}\sum_{i=1}^{N} I\left(G\left(x_i\right) \neq y_i\right) \leqslant \frac{1}{N}\sum_{i} \mathrm{e}^{-y_i f(x_i)} = \prod_{m} Z_m$$

前半段的不等式容易证明，因为当$G(x_i) \neq y_i$时，$y_i f(x_i) \leqslant 0$，所以$I(G(x_i) \neq y_i) = 1 \leqslant \mathrm{e}^{y_i f(x_i)}$；当$G(x_i) = y_i$时，$y_i f(x_i) \geqslant 0$，所以$I(G(x_i) = y_i) = 0 \leqslant \mathrm{e}^{y_i f(x_i)}$。后半部分等式证明如下：

$$\begin{aligned}
&\frac{1}{N}\sum_{i} \mathrm{e}^{-y_i f(x_i)} \\
&= \frac{1}{N}\sum_{i} \mathrm{e}^{-\sum_{m=1}^{M} \alpha_m y_i G_m(x_i)} \\
&= \frac{1}{N}\sum_{i} \prod_{m=1}^{M} \mathrm{e}^{-\alpha_m y_i G_m(x_i)} \\
&= \sum_{i} w_{1i} \prod_{m=1}^{M} \mathrm{e}^{-\alpha_m y_i G_m(x_i)} \\
&= Z_1 \sum_{i} w_{2i} \prod_{m=2}^{M} \mathrm{e}^{-\alpha_m y_i G_m(x_i)} \\
&= Z_1 Z_2 \sum_{i} w_{3i} \prod_{m=3}^{M} \mathrm{e}^{-\alpha_m y_i G_m(x_i)} \\
&= \cdots \\
&= Z_1 Z_2 \cdots Z_{M-1} \sum_{i} w_{Mi} \mathrm{e}^{-\alpha_M y_i G_M(x_i)} \\
&= \prod_{m=1}^{M} Z_m
\end{aligned}$$

在第m次迭代中，Ada Boosting总是趋向于将具有最小误差的学习模型选做本轮次生成的弱分类器G_m，促使累积误差快速下降。

Ada Boosting作为一种提升算法，可归属为集成学习（ensemble learning）这一类分类方法。另外一种常用的集成学习方法是袋装法（bootstrap aggregating，bagging）[6]，与提升算法不同，袋装法在学习过程中不改变样本数据的权重，通过并行和随机采样的思想来将多个弱学习模型集成为强学习模型，这里不进行详细展开。

4.5.4 Ada Boosting分类例子

本节通过一个简单的两类分类例子来介绍Ada Boosting算法过程。表4.8给出了10个数据点$x_i(i\in\{1,2,\cdots,10\})$取值及其所对应的类别标签$y_i\in\{1,-1\}(i\in\{1,2,\cdots,10\})$。

表4.8 两类分类问题数据

	1	2	3	4	5	6	7	8	9	10
x	-9	-7	-5	-3	-1	1	3	5	7	9
y	-1	-1	1	1	-1	-1	-1	-1	1	1

根据表4.8所给出的数据，要构造若干个弱分类器，然后将这些弱分类器组合为一个强分类器，完成分类任务。

为了简化说明，这里定义每个弱分类器G为一种分段函数，由一个阈值ε构成，形式如下：

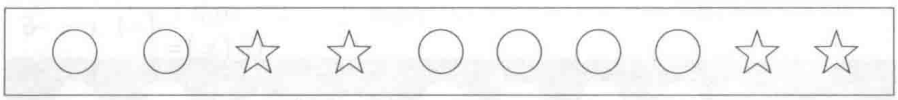
$$G(x_i)=\begin{cases}-1 & x_i<\varepsilon\\ 1 & x_i>\varepsilon\end{cases}\quad 或\quad G(x_i)=\begin{cases}1 & x_i<\varepsilon\\ -1 & x_i>\varepsilon\end{cases}$$

当然，在实际中，可根据需要使用其他的弱分类器。从4.5.2节所介绍的Ada Boosting算法出发，针对表4.8所给出例子来构造强分类器的过程如下。

（1）数据样本权重初始化

$$\boldsymbol{D}_1=(w_{1,1},\cdots,w_{1,i},\cdots,w_{1,10}),\ 其中\ w_{1,i}=\frac{1}{10}(1\leqslant i\leqslant 10)$$

下面用图来辅助说明算法流程。图中圆圈所代表的数据点标签为-1、五角星所代表的数据点标签为1，每个形状的颜色深浅代表这些数据被当前所学习得到（组合）分类器给出标签预测值大小，颜色越深表示越接近标签值-1、颜色越浅表示越接近标签值1。



（2）分别训练M个基分类器（弱分类器）

对$m=1$。

（i）使用具有分布权重$\boldsymbol{D}_1$的训练数据来学习得到第$m=1$个基分类器G_1。不难看出，当阈值$\varepsilon=6$时，基分类器G_1具有最小错误率。G_1分

类器表示如下：

$$G_1(x_i)=\begin{cases}-1 & x_i<6\\ 1 & x_i>6\end{cases}$$

（ii）计算$G_1(x)$在训练数据集上的分类误差，样例3、4被错误分类，因此G_1的分类误差为$err_1=\sum_{i=1}^{N}w_{1,i}I\left(G_1(x_i)\neq y_i\right)=0.1+0.1=0.2$。

（iii）根据分类误差计算弱分类器$G_1(x)$的权重：$\alpha_1=\frac{1}{2}\ln\frac{1-err_1}{err_1}=$ 0.693 1。

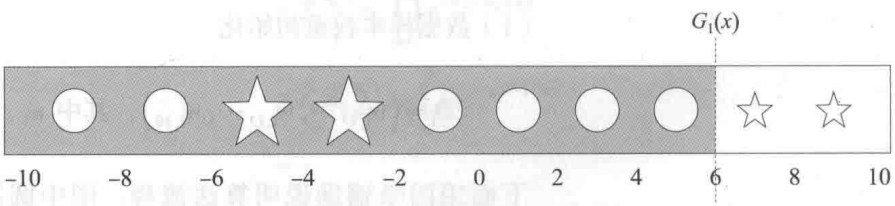
（iv）更新下一轮第$m=2$个分类器训练时第i个训练样本的权重：$\boldsymbol{D}_2=\left\{w_{2,i}\right\}_0^{10}$，$w_{2,i}=\frac{w_{1,i}}{Z_1}\mathrm{e}^{-\alpha_1 y_i G_1(x_i)}$，可得到数据样本新的权重：

$$\boldsymbol{D}_2=(0.062\,5,0.062\,5,0.25,0.25,0.062\,5,0.062\,5,$$
$$0.062\,5,0.062\,5,0.062\,5,0.062\,5)$$

（v）通过加权线性组合得到当前的分类器：

$$f_1(x)=\sum_{i=1}^{M}\alpha_m G_m(x)=0.693\ 1G_1(x)$$

在下图中，被分类错误数据样本的形状尺寸比其他数据样本形状稍大，以表示被分类错误的样本数据权重增大。



对$m=2$。

（i）对于具有分布权重为$\boldsymbol{D}_2$的训练数据，当阈值$\varepsilon=-6$时，基分类器G_2具有最小的错误率。G_2分类器表示如下：

$$G_2(x_i)=\begin{cases}-1 & x_i<-6\\ 1 & x_i>-6\end{cases}$$

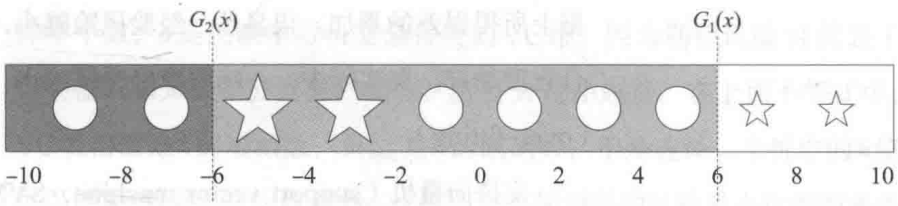
（ii）分类误差$err_2=\sum_{i=1}^{N}w_{2,i}I\left(G_2(x_i)\neq y_i\right)=0.25$。

（iii）弱分类器$G_2(x)$的权重$\alpha_2=\frac{1}{2}\ln\frac{1-err_2}{err_2}=0.543\ 9$。

（iv）当进行下一轮分类器训练时，样本权重更新如下：

$$\boldsymbol{D}_3 = (0.041\,666\,67, 0.041\,666\,67, 0.166\,666\,67, 0.166\,666\,67,$$
$$0.125, 0.125, 0.125, 0.125, 0.041\,666\,67, 0.041\,666\,67)$$

（v）通过加权线性组合得到当前的分类器 $f_2(x) = 0.693\,1G_1(x) + 0.543\,9G_2(x)$。



对 $\boldsymbol{m} = 3$。

（i）对于具有分布权重为 $\boldsymbol{D}_3$ 的训练样本数据，当阈值 $\varepsilon = -2$ 时，基分类器 G_3 具有最小的错误率。G_3 分类器表示如下：

$$G_3(x_i) = \begin{cases} -1 & x_i > -2 \\ 1 & x_i < -2 \end{cases}$$

（ii）分类误差 $err_3 = \sum_{i=1}^{N} w_{3,i} I(G_3(x_i) \neq y_i) = 0.166\,7$。

（iii）弱分类器 $G_3(x)$ 的权重 $\alpha_3 = \frac{1}{2}\ln\frac{1-err_3}{err_3} = 0.804\,7$。

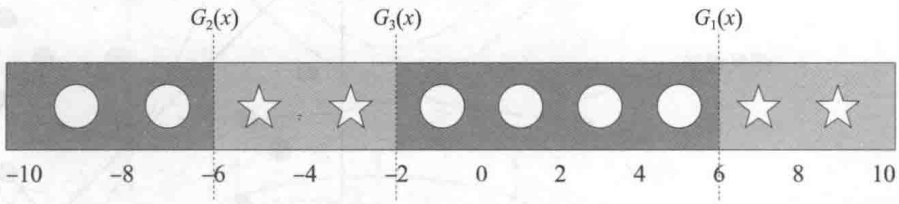
（iv）下一轮弱分类器训练时，训练数据样本的权重更新如下：

$$\boldsymbol{D}_4 = (0.125, 0.125, 0.1, 0.1, 0.075, 0.075, 0.075, 0.075, 0.125, 0.125)$$

（v）通过加权线性组合得到当前的分类器 $f_3(x) = 0.693\,1G_1(x) + 0.543\,9G_2(x) + 0.804\,7G_3(x)$。

在 $f_3(x)$ 的基础上，构造强分类器 $G(x) = sign(f_3(x)) = sign(0.693\,1G_1(x) + 0.543\,9G_2(x) + 0.804\,7G_3(x))$。

这里 *sign*( ) 是符号函数，其输入值大于0时，符号函数输出为1，反之为−1。由于 $G(x)$ 在训练样本上分类错误率为0，算法终止，得到最终的强分类器。



4.6 支持向量机

传统识别理论认为，算法模型性能可用从训练样本集所得经验风险（empirical risk）来衡量。经验风险指算法模型在训练样本集中所有数据上所得误差的累加。很显然，经验风险越小，算法模型对训练数据拟合程度越好。在实际中，一味要求经验风险小，往往会造成过学习问题（over-fitting）。

支持向量机（support vector machine，SVM）通过结构风险最小化（structural risk minimization）来解决过学习问题。如图4.10给出了一个两分类问题。图中存在多个可将样本分开的超平面（hyper-plane）。但是，在这些超平面中，支持向量机学习算法会去寻找一个最佳超平面，使得每个类别中距离超平面最近的样本点到超平面的最小距离最大[8]。

4.6.1 VC维与结构风险最小化

在理论上，支持向量机认为：分类器对未知数据（即测试数据）进行分类时所产生的期望风险（expected risk，真实风险）不是由经验风险单独决定的，而是由两部分组成：① 从训练集合数据所得经验风险（如果经验风险小、期望风险很大，则是过学习）；② 置信风险（confidence risk），它与分类器的VC维及训练样本数目有关。如果 $0 \leqslant \eta \leqslant 1$，Vapnik推导出期望风险 $\Re$ 和经验风险 $\Re_{emp}$ 以 $1-\eta$ 的概率满足如下关系：

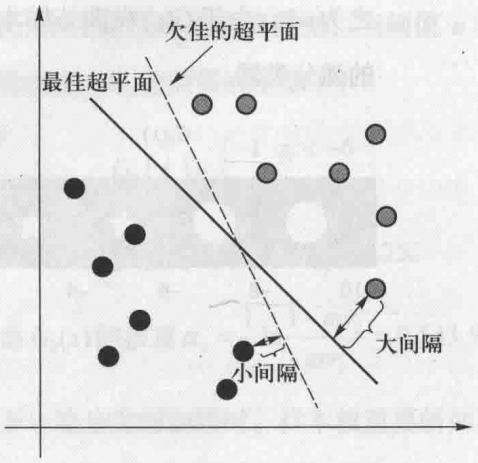


图4.10 两类分类问题的最佳分类平面

$$\Re \leqslant \Re_{\text{emp}} + \sqrt{\frac{h\left(\log\left(2n/h\right)+1\right)-\log\left(\eta/4\right)}{n}}$$

其中，$\sqrt{\frac{h\left(\log\left(2n/h\right)+1\right)-\log\left(\eta/4\right)}{n}}$叫做“VC置信值”，$n$是训练样本个数，$h$是反映学习机复杂程度的VC维。因为期望风险$\Re$代表了分类器对未知数据分类推广能力，所以$\Re$越小越好。在上面不等式中，VC置信值是$h$的增函数，$\Re_{\text{emp}}$是$h$的减函数，于是选择一个折中的$h$值可以使期望风险$\Re$达到最小。支持向量机使用结构风险最小化准则来选取VC维h，使每一类别数据之间的分类间隔（margin）最大，最终使期望风险$\Re$最小。

假设空间的VC维　将n个数据分为两类，可以有2^n种分法，即可理解成有2^n个学习问题。若存在一个假设$\mathcal{H}$，能准确无误地将2^n种问题进行分类，那么n就是$\mathcal{H}$的VC维[30]。

一般地，在r维空间中，线性决策面的VC维为$r+1$。VC维就是对假设空间$\mathcal{H}$复杂度的一种度量。当样本数n固定时，如果VC维越高，则算法模型的复杂性越高。VC维越大，通常推广能力越差，置信风险会变大。如果算法模型一味降低经验风险，则会提高模型复杂度，导致VC维很高，置信风险大，使得期望风险就大，如图4.11所示。

结构风险最小化　通过VC理论，可认识到期望风险（即真实风险）$\Re$与经验风险$\Re_{\text{emp}}$之间是有差别的，这个差别项被称为置信风险，它与训练样本个数和模型复杂度都有密切的关系。用复杂度高的模型去拟

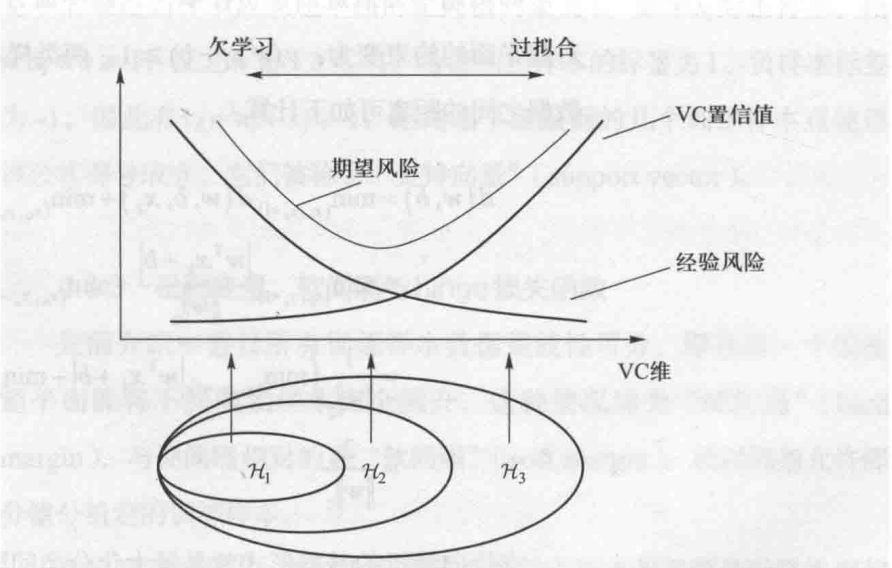


图4.11 经验风险、期望风险、VC置信度、VC维、过学习和欠学习的关系

合小样本，往往会导致过拟合，因此需要给经验风险 $\Re_{emp}$ 加上一个惩罚项或者正则化项，以同时考虑经验风险与置信风险。这一思路被称为结构风险最小化。这样，在小样本情况下可取得较好性能。在保证分类精度高（经验风险小）同时，有效降低算法模型VC维，可使算法模型在整个样本集上的期望风险得到控制。当训练样本给定时，分类间隔越大，则对应的分类超平面集合的VC维就越小。

4.6.2 线性可分支持向量机

对于 n 个训练数据 $(\boldsymbol{x}_i, y_i) \in \boldsymbol{D}(i = 1, \cdots, n)$，记其类别标签为1或-1（即 $y_i = 1$ 或 $y_i = -1$）分别代表了正样本或负样本。支持向量机从这些训练数据出发，寻找一个最优的超平面，其方程为 $\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b = 0$。这里 $\boldsymbol{w} = (w_1, w_2, \cdots, w_d)$ 为超平面的法向量，与超平面的方向有关；b 为偏置项，是一个标量，其决定了超平面与原点之间的距离。

显然，为了保证每个样本都被正确分类，对于正样本要求满足 $\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b \geqslant 0$，对于负样本要求满足 $\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b < 0$。

样本空间中任意样本 $\boldsymbol{x}$ 到该平面距离可表示为：

$$r = d(\boldsymbol{w}, b, x) = \frac{\left|\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b\right|}{\|\boldsymbol{w}\|_2} \left(\|\boldsymbol{w}\|_2 = \sqrt{\boldsymbol{w}^{\mathrm{T}}\boldsymbol{w}}\right)$$

由于法向量 $\boldsymbol{w}$ 中的值可按比例任意缩放而不改变法向量方向，使得分类平面不唯一。为此，对 $\boldsymbol{w}$ 和 b 添加如下约束：

$$r_{\min_i \left|\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_i + b\right|} = 1$$

即离超平面最近的正负样本代入超平面方程后其绝对值为1。于是对超平面的约束变为：$y_i(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_i + b) \geqslant 1$。两类样本中离分类超平面最近的数据之间的距离可如下计算：

$$\begin{aligned} d(\boldsymbol{w}, b) &= \min_{(\boldsymbol{x}_k, y_k = 1)} d(\boldsymbol{w}, b, \boldsymbol{x}_k) + \min_{(\boldsymbol{x}_m, y_m = -1)} d(\boldsymbol{w}, b, \boldsymbol{x}_m) \\ &= \min_{(\boldsymbol{x}_k, y_k = 1)} \frac{\left|\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_k + b\right|}{\|\boldsymbol{w}\|_2} + \min_{(\boldsymbol{x}_m, y_m = -1)} \frac{\left|\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_m + b\right|}{\|\boldsymbol{w}\|_2} \\ &= \frac{1}{\|\boldsymbol{w}\|_2} \left(\min_{(\boldsymbol{x}_k, y_k = 1)} \left|\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_k + b\right| + \min_{(\boldsymbol{x}_m, y_m = -1)} \left|\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_m + b\right|\right) \\ &= \frac{2}{\|\boldsymbol{w}\|_2} \end{aligned}$$

支持向量机的基本形式就是最大化分类间隔，即在满足约束的条件

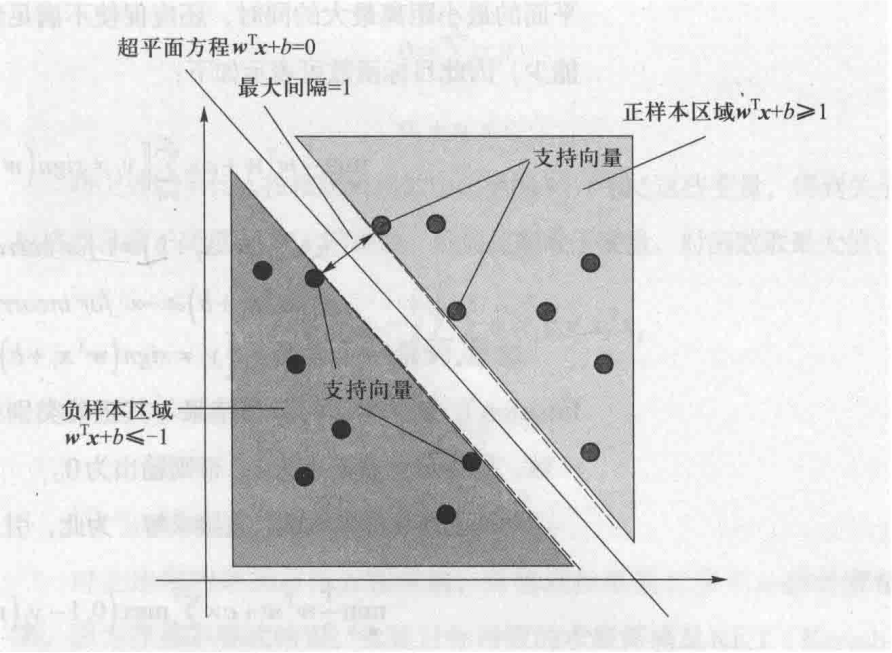


图4.12 线性分类中分类平面及其支持向量

下找到参数w和b使得γ最大，即等价于：

$$\min_{w,b}\frac{\|w\|^2}{2}=\min_{w,b}\frac{1}{2}w^Tw$$

$$\text{s.t.}y_i\left(w^Tx_i+b\right)\geqslant 1,\ i=1,2,\cdots,n$$

上面带约束条件的最值优化问题是一个凸二次规划（convex quadratic problem）问题，可通过构造拉格朗日函数以及将其转换成对偶（dual）问题求解[5]。

图4.12给出了线性分类超平面的示意图。可见，正负类别中距离超平面最近的样本点到超平面的距离被约束为1。如果x_i为正样本，则$w^Tx_i+b\geqslant 1$；反之$w^Tx_i+b\leqslant -1$。考虑到正样本的标签为1、负样本标签为-1，因此有$y_i(w^Tx_i+b)\geqslant 1$。距离超平面最近的几个训练样本点使得该公式等号成立，它们被称为“支持向量”（support vector）。

4.6.3 松弛变量，软间隔与hinge损失函数

先前介绍中假设所有训练样本数据是线性可分，即存在一个线性超平面能将不同类别样本完全隔开，这种情况称为“硬间隔”（hard margin），与硬间隔相对的是“软间隔”（soft margin）。软间隔指允许部分错分给定的训练样本。

在软间隔情况下，算法在使得每个类别中距离超平面最近样本到超

平面的最小距离最大的同时，还应促使不满足约束条件的样本数目尽可能少，因此目标函数可表示如下：

$$\min_{\boldsymbol{w},b}\frac{1}{2}\boldsymbol{w}^{\mathrm{T}}\boldsymbol{w}+c\times\sum_{i=1}^{n}\left[\!\left[y_i\neq sign\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_i+b\right)\right]\!\right]$$

$$\text{s.t. } y_i\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_i+b\right)\geqslant 1 \textit{ for correct } \boldsymbol{x}_i$$

$$y_i\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_i+b\right)\geqslant -\infty \textit{ for incorrect } \boldsymbol{x}_i$$

这里，c是一个常数。$\left[\!\left[y_i\neq sign\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_i+b\right)\right]\!\right]$是指示函数（indicator function），表示对$\boldsymbol{x}_i$的分类结果与其正确类别标签y_i是否一致，如果不一致，则指示函数输出为1、否则输出为0。

然而上述目标函数难以直接求解。为此，引入hinge损失函数，则有：

$$\min_{\boldsymbol{w},b}\frac{1}{2}\boldsymbol{w}^{\mathrm{T}}\boldsymbol{w}+c\times\sum_{i=1}^{n}\max\left(0,1-y_i\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_i+b\right)\right)$$

很显然，所有被正确分类的数据存在$y_i(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_i+b)\geqslant 1$，因此在hinge损失函数作用下，这些被正确分类数据的hinge损失$\max(0,1-y_i(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_i+b))=0$。于是，hinge函数只会记录分类错误数据的损失。

记$\xi_i=\max\left(0,1-y_i\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_i+b\right)\right)$（$\xi_i$被称为第$i$个变量的“松弛变量”，slack variables），显然$\xi_i\geqslant 0$。每一个样本对应一个松弛变量，用来表示该样本被分类错误所产生的损失。于是，可将上式重写为：

$$\min_{\boldsymbol{w},b}\frac{1}{2}\boldsymbol{w}^{\mathrm{T}}\boldsymbol{w}+c\times\sum_{i=1}^{n}\xi_i$$

$$\text{s.t. } y_i\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_i+b\right)\geqslant 1-\xi_i$$

$$\xi_i\geqslant 0, i=1,2,\cdots,n$$

这就是最常见的“软间隔支持向量机”需要优化的目标函数（也被称为原问题，prime problem）。

对于上述目标函数，通过拉格朗日乘子法可得到拉格朗日函数：

$$L(\boldsymbol{w},b,\boldsymbol{\alpha},\boldsymbol{\xi},\boldsymbol{\mu})=\frac{1}{2}\boldsymbol{w}^{\mathrm{T}}\boldsymbol{w}+c\times\sum_{i=1}^{n}\xi_i+\sum_{i=1}^{n}\alpha_i\left(1-\xi_i-y_i\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_i+b\right)\right)-\sum_{i=1}^{n}\mu_i\xi_i$$

其中$\alpha_i\geqslant 0$和$\mu_i\geqslant 0$是拉格朗日乘子。先固定拉格朗日乘子α_i和μ_i，将上述问题转换成对偶问题（dual problem）。然后对$\boldsymbol{w}$、b和ξ_i偏导，并且令导数为0，可得：

$$\boldsymbol{w}=\sum_{i=1}^{n}\alpha_i y_i\boldsymbol{x}_i$$

$$0=\sum_{i=1}^{n}\alpha_i y_i$$

$$\alpha_i+\mu_i=c$$

将上述结果代入拉格朗日函数中，消除$\boldsymbol{w}$、b和ξ_i这些变量，得到关于拉格朗日乘子变量α_i和μ_i的函数，然后控制乘子变量，对函数取最大值：

$$\max_{\alpha}\sum_{i}^{n}\alpha_i-\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j y_i y_j \boldsymbol{x}_i^{\mathrm{T}}\boldsymbol{x}_j$$

$$\text{s.t.}\sum_{i=1}^{n}\alpha_i y_i=0$$

$$0\leqslant\alpha_i\leqslant c,\quad i=1,2,\cdots,n$$

对上述带约束的目标方程求解，可得到拉格朗日乘子α_i的计算结果。因为存在不等式约束，上述目标函数的求解需满足KKT（Karush-Kuhn-Tucker）条件：

$$\begin{cases}\alpha_i\geqslant 0\\ y_i f(\boldsymbol{x}_i)-1\geqslant 0\left(\text{其中}f(\boldsymbol{x}_i)=\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_i+b\right)\\ \alpha_i\left(y_i f(\boldsymbol{x}_i)-1\right)=0\end{cases}$$

于是对任意训练样本$(\boldsymbol{x}_i, y_i)$，总有$\alpha_i=0$或者$y_i f(\boldsymbol{x}_i)=1$。当$\alpha_i=0$时，该样本不会在模型中出现，即该样本数据对$f(x)$不会产生任何影响。当$y_i f(\boldsymbol{x}_i)=1$时，$\alpha_i>0$，此时该样本数据$\boldsymbol{x}_i$位于最大间隔边界上，是一个支持向量。因此可得出一个支持向量机的重要性质：**训练完成后，大部分的训练样本都可以舍去，最终模型仅与若干支持向量有关。**

α可以采用SMO算法（sequential minimal optimization）进行求解，要深入了解的读者参考原论文[19]，本节仅简要概述其基本思路。

SMO先固定α_i之外的所有参数，在此基础上求解α_i。由于存在约束$\sum_{i=1}^{n}\alpha_i y_i=0$，若固定$\alpha_i$之外的其他变量，则$\alpha_i$可以由其他变量推算出来。于是SMO每次选择两个变量α_i和α_j，并固定其他参数，不断执行如下步骤直至收敛：

（1）选取一对需更新的变量α_i和α_j；

（2）固定除α_i和α_j之外的所有参数，简化约束得到$\alpha_i y_i+\alpha_j y_j=-\sum_{i\neq i,\,j}^{n}\alpha_k y_k=$常数；

（3）计算满足使得目标函数最大的α_i和α_j并更新。

关于α_i和α_j的选取有多种方式，比较简单常用的方式是选取两个

对应样本间隔最大的α_i和α_j。在$\boldsymbol{\alpha}$收敛后，通过$\boldsymbol{w}=\sum_{i=1}^{n}\alpha_i y_i \boldsymbol{x}_i$来求解分类超平面$\boldsymbol{w}$。理论上可以使用任意支持向量借助$y_i f(\boldsymbol{x}_i)=1$计算偏移量$b$，现实中也常将所有支持向量带入进行求解的结果去取平均值得到b，这种方式更加鲁棒。

4.6.4 核函数解决线性不可分的情况

在现实任务中，原始样本空间并不一定是线性可分的，对于这样的问题，可将样本中原始空间映射到一个更加高维的特征空间中去，使得样本在这个特征空间中高概率线性可分。如果原始空间是有限维，那么一定存在一个高维特征空间使样本可分[26]。

令$\boldsymbol{\phi}(\boldsymbol{x})$是样本$\boldsymbol{x}$在高维特征空间中的特征向量，那么在特征空间中的超平面所对应的模型可以表示为：

$$f(\boldsymbol{x})=\boldsymbol{w}^{\mathrm{T}}\boldsymbol{\phi}(\boldsymbol{x})+b$$

特征空间中的目标函数为：

$$\min_{w,b}\frac{1}{2}\|\boldsymbol{w}\|^2$$

$$\text{s.t. } y_i\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{\phi}(\boldsymbol{x})+b\right)\geqslant 1,\ i=1,2,\cdots,n$$

使用拉格朗日乘子法获得的对偶问题为：

$$\max_{\alpha}\sum_{i=1}^{n}\alpha_i-\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j y_i y_j\boldsymbol{\phi}(\boldsymbol{x}_i)^{\mathrm{T}}\boldsymbol{\phi}(\boldsymbol{x}_j)$$

$$\text{s.t.}\sum_{i=1}^{n}\alpha_i y_i=0,\quad \alpha_i\geqslant 0,\quad i=1,2,\cdots,n$$

上式的求解过程中，会发现$\boldsymbol{\phi}(\boldsymbol{x}_i)^{\mathrm{T}}\boldsymbol{\phi}(\boldsymbol{x}_j)$用常规方法难以求解，因为无法确定$\boldsymbol{\phi}(\boldsymbol{x})$的具体形式。但可设想存在一个函数$\kappa(\boldsymbol{x}_i,\boldsymbol{x}_j)$，使得$\boldsymbol{x}_i,\boldsymbol{x}_j$在原始样本空间中通过函数$\kappa$计算的结果等于它们在特征空间中的内积，即：

$$\kappa(\boldsymbol{x}_i,\boldsymbol{x}_j)=\boldsymbol{\phi}(\boldsymbol{x}_i)^{\mathrm{T}}\boldsymbol{\phi}(\boldsymbol{x}_j)$$

$\kappa(\boldsymbol{x}_i,\boldsymbol{x}_j)$被称为**核函数**。常用的核函数见表4.9。

此外核函数的某些组合也是核函数，例如：若κ_1,κ_2是核函数，γ_1,γ_2是正数，则线性组合$\gamma_1\kappa_1+\gamma_2\kappa_2$也是核函数；若$\kappa_1,\kappa_2$是核函数，则核函数的直积$\kappa_1(x,z)\kappa_2(x,z)$也是核函数；若$\kappa_1$是核函数，则对于任意函数$f(x)$，有$\kappa(x,z)=f(x)\kappa_1(x,z)f(z)$是核函数。

表4.9 常见核函数

线性	$\kappa(x_i, x_j) = x_i x_j$
多项式	$\kappa(x_i, x_j) = (\gamma x_i x_j + c)^n$
Radial basis function	$\kappa(x_i, x_j) = e^{\frac{\|x_i - x_j\|^2}{2\sigma^2}}$
Sigmoid	$\kappa(x_i, x_j) = \tanh(x_i, x_j - \gamma)$

4.7 生成学习模型

本章介绍的监督学习可以分为判别式学习（discriminative learning）和生成学习（generative learning），之前所介绍的方法都是判别式学习。这一小节主要来介绍生成学习。

给定标注训练数据$(\boldsymbol{X}, \boldsymbol{C})$，其中$\boldsymbol{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3, \cdots, \boldsymbol{x}_n\}$代表$n$个样本数据，$\boldsymbol{C} = \{c_1, c_2, c_3, \cdots, c_n\}(c_i \in \boldsymbol{C})$代表每个样本所对应的类别标签的集合。监督学习从标注数据出发，或者学习得到一个决策函数f_θ（θ为模型f的参数）来直接给出输入数据$\boldsymbol{x}$所对应的类别标签$f_\theta(\boldsymbol{x})$，或者计算条件概率$P(c_i|\boldsymbol{x})$，从而将$\boldsymbol{x}$分类为条件概率值最大的那个类别。

生成学习方法从数据中学习联合概率分布$P(\boldsymbol{X}, \boldsymbol{C})$，然后求出条件概率分布$P(\boldsymbol{C}|\boldsymbol{X})$作为预测模型，即$P(c_i|\boldsymbol{x}) = \dfrac{P(\boldsymbol{x}, c_i)}{P(\boldsymbol{x})}$。当然由于$P(\boldsymbol{x}, c_i)$一般难以求解，因此通过如下方法来求解联合概率分布$P(\boldsymbol{x}, c_i)$：

$$P(\boldsymbol{x}, c_i) = \underbrace{P(\boldsymbol{x}|c_i)}_{\text{似然概率}} \times \underbrace{P(c_i)}_{\text{先验概率}}$$

于是可得：

$$\underbrace{P(c_i|\boldsymbol{x})}_{\text{后验概率}} = \frac{\underbrace{P(\boldsymbol{x}, c_i)}_{\text{联合概率}}}{P(\boldsymbol{x})} = \frac{\underbrace{P(\boldsymbol{x}|c_i)}_{\text{似然概率}} \times \underbrace{P(c_i)}_{\text{先验概率}}}{P(\boldsymbol{x})}$$

对于每个类别而言，每个输入数据x的概率$P(x)$是一样的，因此可将$P(x)$从上式中去掉，于是可得到后验概率的如下计算方法：

$$\underbrace{P(c_i|\boldsymbol{x})}_{\text{后验概率}} = \underbrace{P(\boldsymbol{x}|c_i)}_{\text{似然概率}} \times \underbrace{P(c_i)}_{\text{先验概率}}$$

也就是说，输入数据$\boldsymbol{x}$属于类别c_i的后验概率等于“似然概率（从

类别c_i模型中能够产生$\boldsymbol{x}$的概率）”与“先验概率（类别c_i出现的概率）”之积，选取“似然概率”与“先验概率”乘积最大取值所对应的类别作为分类结果。这个式子是著名的贝叶斯公式。似然概率显然“编码”了输入数据被类别模型产生的关系。

与生成学习方法不同，判别式方法由数据直接学习决策函数f_θ或者条件概率分布$P(c_i|\boldsymbol{x})$来作为预测的模型。如果直接来学习决策函数f_θ，则不涉及与样本数据有关的概率计算，直接从底层特征出发来得到类别输出（如支持向量机、回归分析和Ada boosting等）。

假设输入数据有“0”和“1”两种取值，类别标签有“阳性”和“阴性”两种取值，表4.10给出了输入数据及其对应标签之间的分布。

表4.10　输入数据－类别标签的样本分布

输入数据 \ 类别	阳性	阴性
$x=0$	6	2
$x=1$	0	4

在表4.10中，一共有12个样本－标签数据，其中$(x=0,\ c=$ 阳性$)$样本出现了6次，$(x=0,\ c=$ 阴性$)$样本出现了2次，$(x=1,\ c=$ 阳性$)$样本没有出现，$(x=1,\ c=$ 阴性$)$样本出现了4次。

基于给出的样本分布，可以如下来解释生成学习和判别式学习之间的不同。

（1）生成学习

输入样本和类别标签的联合概率分布为：$P(0,\text{阳性})=\frac{6}{12}=\frac{1}{2}$、$P(0,\text{阴性})=\frac{2}{12}=\frac{1}{6}$、$P(1,\text{阳性})=0$、$P(1,\text{阴性})=\frac{4}{12}=\frac{1}{3}$。一旦给出输入数据，假定输入数据的概率为某个常数，就可以通过计算$\frac{P(0,\text{阳性})}{P(0)}$或者$\frac{P(0,\text{阴性})}{P(0)}$以及$\frac{P(1,\text{阳性})}{P(1)}$或者$\frac{P(1,\text{阴性})}{P(1)}$，将输入数据归属到所得结果最大对应的类别。这里要注意，样本－标签数据的联合概率分布累加之和为1。

当然，在实际中一般难以获得样本数据－类别标签的联合概率分布，因此往往需要利用贝叶斯公式来计算似然概率与先验概率之间的乘积，进而基于生成学习方法来得到输入样本属于某个类别的概率。

（2）判别式学习

输入样本和类别标签的条件概率分布为：$P(\text{阳性}|0)=\frac{6}{8}=\frac{3}{4}$、$P(\text{阳性}|1)=\frac{2}{8}=\frac{1}{4}$；$P(\text{阳性}|1)=\frac{0}{4}=0$、$P(\text{阳性}|1)=\frac{4}{4}=1$。这里要注意，输入样本为0或为1前提下，其所对应的类别概率累加之和为1。

常见的生成学习模型有隐马尔可夫模型、隐狄利克雷分布等。隐马尔可夫模型（hidden markov model，HMM）最早由Baum提出[2]，是对时序数据进行分析处理的生成学习模型。在“隐”（hidden）马尔可夫过程中，所谓的“隐”，是指得到的随机变量序列不是状态序列，要从得到的随机变量序列得到状态序列，需要进行其他的变化运算。HMM常被用于语音识别、手写识别以及手势识别[17, 28]等。隐狄利克雷分布（latent dirichlet allocation，LDA）[4, 20]可从文档数据中挖掘内隐主题（topic），实现了文档表达从“单词词袋（bag of words）”到“主题词袋（bag of topics）”的跨越。生成学习模型涉及较多复杂的知识概念，这里就不做具体介绍了。

Andrew在2001年以回归判别模型和贝叶斯产生模型为对比，通过理论和实验证明了如下结论[1]：判别模型的渐近误差（asymptotic error）更小，然而生成模型能够更迅速地收敛到自己的极小渐近误差。随着训练样本的逐渐增大，生成模型一开始能够表现得更好，但是判别模型随后很有可能赶超上来。

4.8 小结

从数据中学习概念或模式形成判断和决策能力是机器学习的一个基本目标。监督学习从标注数据出发，学习得到一个映射函数，将原始输出映射到语义任务空间，架构起了“从底层特征到高层语义”的“桥梁”。

监督学习需要依赖于标注大数据来进行学习，难以利用无标注数据。如何形成数据依赖灵活，且在学习过程中有效利用知识或先验，是监督学习今后发展的重要方向。同时，一个任务是否可以被学习（learnability）仍然是学术界的热点和难点[27]。

习题4

1. 请判断下面说法是否正确：线性区别分析是在最大化类间方差和类内方差的比值（　　）。

   A. 正确　　B. 错误

2. 在决策树建立过程中，使用一个属性对某个结点对应的数据集合进行划分后，结果具有高信息熵（high entropy），对于结果的描述，最贴切的是（　　）。

   A. 纯度高　　B. 纯度低

   C. 有用　　D. 无用

   E. 以上描述都不贴切

3. 在一个监督学习任务中，每个数据样本有4个属性和一个类别标签，每种属性分别有3、2、2和2种可能的取值，类别标签有3种不同的取值。请问可能有多少种不同的样本？（注意，并不是在某个数据集中最多有多少种不同的样本，而是考虑所有可能的样本）（　　）。

   A. 3　　B. 6

   C. 12　　D. 24

   E. 48　　F. 72

4. 关于支持向量机，哪一种选择能正确填充下面的说法？（　　）

   支持向量机是一种_____模型，其目标是_____分类间隔。

   A. 生成，最大化　　B. 生成，最小化

   C. 判别，最大化　　D. 判别，最小化

5. 以下哪种方法不属于监督学习方法？（　　）

   A. 聚类　　B. 决策树

   C. 线性回归　　D. 朴素贝叶斯

   E. 支持向量机　　F. 以上都不属于监督学习方法

6. 在Adaboosting的迭代中，从第t轮到第$t+1$轮，某个被错误分类样本的惩罚增加了，可能因为该样本（　　）。

   A. 被第t轮训练的弱分类器错误分类

   B. 被第t轮后的集成分类器（强分类器）错误分类

   C. 被到第t轮为止训练的大多数弱分类器错误分类

   D. B和C都正确

   E. A, B和C都正确

7. 假设有两批从同样的真实数据分布中采样得到去完成同一任务的数据集A和B。A包含100KB数据，B包含10KB数据。按照9∶1这一同样比例随机将A和B分别划分为训练集和测试集。图4.13给出了数据集A和数据集B随着模型复杂度增加所对应的训练误差（A, Train以及B, Train）和测试误差（A, Test以及B, Test）曲线图。请指出哪个图正确表示了随着模型复杂度增加所对应训练误差和测试误差的变化曲线图。（　　）

8. 考虑一个在两维特征空间中的二分类问题。训练集包含8个样本，其在二维空间中的分布如图4.14所示。图4.14中不同

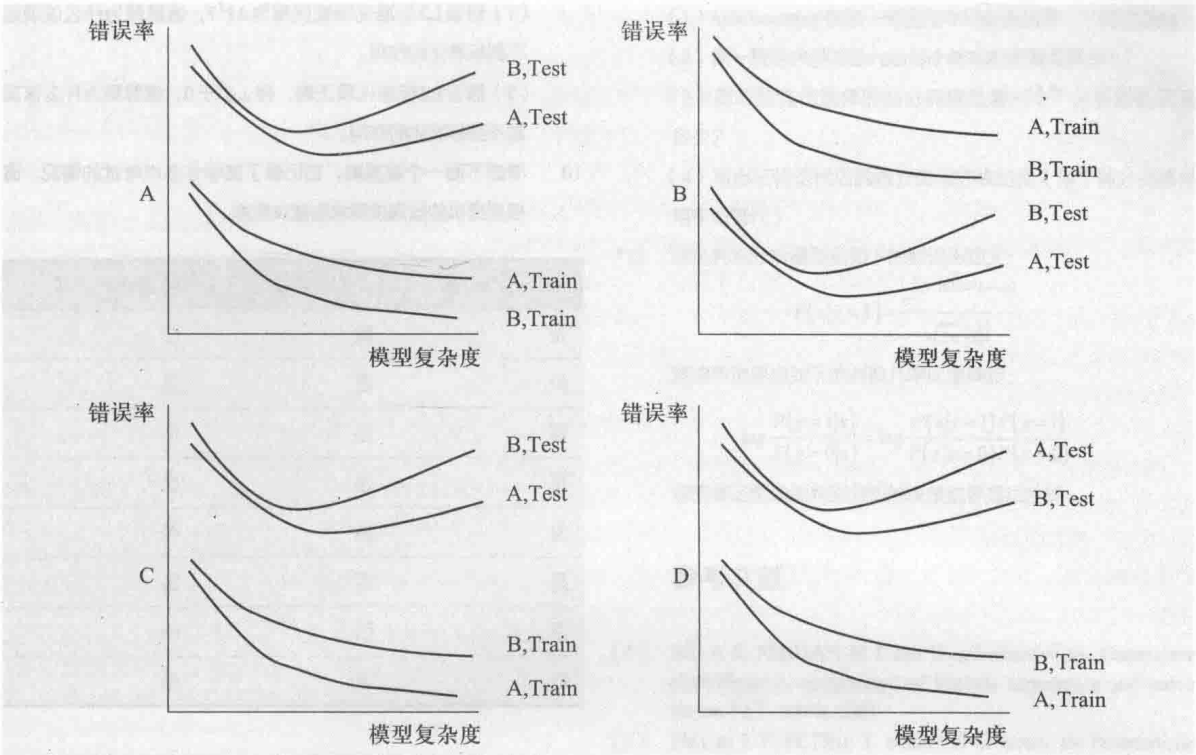


图4.13 数据集A和数据集B随模型复杂度增加所对应的训练误差和测试误差变化示意图

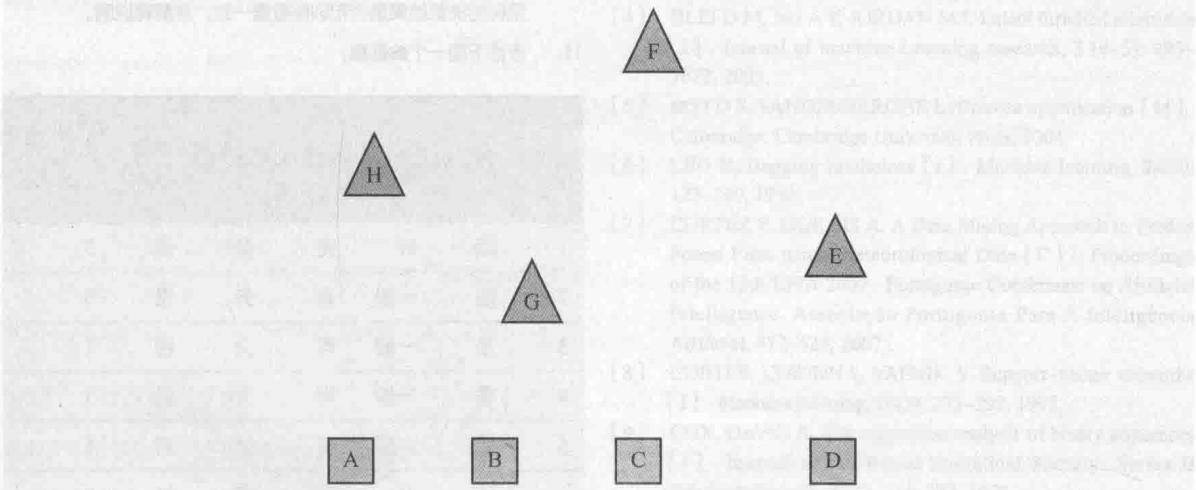


图4.14 两类分类问题中8个样本的分布图

类别的样本用不同的形状表示，根据图4.14回答如下问题。

（1）使用实线画出使用SVM分类器可得到的分类边界，并用圆圈标出支持向量。

（2）在该分类边界下所得训练错误率为多少？

（3）移除哪些样本可以让分类边界发生变化？

9. 加入L2标准化（normalization）后，线性回归的损失函数的标准形式为：

$$L=(Y-Xw)^{\mathrm{T}}(Y-Xw)+\lambda w^{\mathrm{T}}w \qquad \text{其中}\lambda>0$$

（1）假设L2标准化项被误写为$\lambda Y^T Y$，请解释为什么该项起不到标准化的作用。

（2）假设L2标准化项正确，但λ小于0，请解释为什么该项起不到标准化的作用。

10. 考虑下面一个数据集，它记录了某学生多次考试的情况，请根据提供的数据按要求构建决策树。

是否通过考试	是否认真复习	是否超常发挥
是	是	否
是	是	是
是	是	否
是	是	是
是	是	否
是	否	是
否	否	否
否	否	是

（1）根据信息增益率选择第一个属性，构建一个深度为1的决策树（根结点的深度为1）。

（2）根据信息增益率构建完整的决策树。请回答，这两个决策树的决策结果是否和训练数据一致，并解释说明。

11. 考虑下面一个数据集：

序号	出去玩	天气状况	有同伴	零花钱	特殊节日	心情指数（1差~5好）
1	是	好	无	多	是	5
2	是	一般	有	多	是	5
3	是	一般	有	少	否	1
4	是	一般	有	少	否	3
5	是	一般	有	少	否	5
6	是	好	无	多	是	5
7	是	好	无	多	是	5
8	否	一般	无	多	是	1
9	否	一般	有	少	否	1
10	否	一般	无	少	否	5

对于如上数据，考虑使用Ada boosting方法来训练“是否出去玩”强分类器。每个弱分类器可考虑对单个属性的分类，比如对于“心情指数”这一属性，可考虑心情指数>2和心情指数<4两个方面。请回答下列问题。

（1）Ada boosting在第一轮迭代中将会选择哪一个弱分类器？

（2）第一轮迭代前与迭代后每个样本的权重是多少？

（3）第二轮迭代选择的弱分类器是哪一个？，分类器权重是多少？

（4）写出三轮迭代后的强分类器的表达式（每个弱分类器可用字母替代）

*12. 线性判别分析模型由如下的假设所定义：

$$P(x|y=k)=\frac{e^{\frac{1}{2}(x-\mu_k)^T\Sigma^{-1}(x-\mu_k)}}{\sqrt{2\pi}|\Sigma|^{\frac{1}{2}}}$$

其决策边界由如下的对数几率比值确定：

$$\log\frac{P(y=1|x)}{P(y=0|x)}=\log\frac{P(x|y=1)P(y=1)}{P(x|y=0)P(y=0)}$$

请简要证明线性判别分析的决策边界是线性的。

参考文献

[1] NG A Y, JORDAN M I. On Discriminative vs. Generative classifiers: A comparison of logistic regression and naive Bayes [J]. NIPS, 2001.

[2] BAUM L E, PETRIE T. Statistical Inference for Probabilistic Functions of Finite State Markov Chains [J]. The Annals of Mathematical Statistics, 37 (6): 1554–1563, 1966.

[3] BISHOP C M. Pattern Recognition and Machine Learning [M]. New York: Springer–Verlag inc., 2006.

[4] BLEI D M, NG A Y, JORDAN M I. Latent dirichlet allocation [J]. Journal of machine Learning research, 3 (4–5): 993–1022, 2003.

[5] BOYD S, VANDENBERGHE L. Convex optimization [M]. Cambridge: Cambridge University Press, 2004.

[6] LEO B. Bagging predictors [J]. Machine learning, 24(2): 123–140, 1996.

[7] CORTEZ P, MORAIS A. A Data Mining Approach to Predict Forest Fires using Meteorological Data [C] // Proceedings of the 13th EPIA 2007– Portuguese Conference on Artificial Intelligence. Associação Portuguesa Para A Inteligência Artificial, 512–523, 2007 .

[8] CORTES, CORINNA, VAPNIK V. Support–vector networks [J]. Machine learning, 20(3): 273–297, 1995.

[9] COX, DAVID R. The regression analysis of binary sequences [J]. Journal of the Royal Statistical Society: Series B (Methodological), 20(2): 215–232, 1958.

[10] DAVID B S, et al. Learnability can be undecidable [J]. Nature Machine Intelligence, 1(1): 44–48, 2019.

[11] DENG, HOUTAO, RUNGER G, et al. Bias of importance measures for multi–valued attributes and solutions [C] // Proceedings of the International Conference on Artificial Neural Networks. Berlin: Springer , 2011.

[12] FISHER R. The Use of Multiple Measurements in Taxonomic Problems [J]. Annals of Eugenics, 7 (2): 179–188, 1936.

[13] FREUND, YOAV, ROBERT E. Schapire. Experiments with a new boosting algorithm [J]. International Conference of Machine Learning (ICML), 1996.

[14] GALTON F. Regression Towards Mediocrity in Hereditary Stature [J]. The Journal of the Anthropological Institute of Great Britain and Ireland, 15: 246–263, 1886.

[15] GARSON G D. Discriminant Function Analysis [M]. New Jersey: Statistical Associates Publishers, 2012.
[16] HARRIS Z S. Distributional structure [J]. Word, 10(2–3): 146–162, 1954.
[17] JORDAN M I, KLEINBERG J, SCHOLKOPF B. Pattern Recognition and Machine Learning [M]. Berlin: Springer, 2006.
[18] MARTINEZ A M, KAK A C. PCA versus LDA [J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 23 (2): 228–233, 2011.
[19] PLATT J. Sequential minimal optimization: A fast algorithm for training support vector machines [J]. MSR–TR–98–14, 1998.
[20] PRITCHARD J K, STEPHENS M, DONNELLY P. Inference of population structure using multilocus genotype data [J]. Genetics, 155 (2): 945–959, 2000.
[21] QUINLAN J R. Simplifying decision trees [J]. International journal of man–machine studies, 27(3): 221–234, 1987.
[22] MIKOLOV, TOMAS, et al. Efficient estimation of word representations in vector space [J]. arXiv preprint arXiv: 1301.3781, 2013.
[23] RAO R C. The utilization of multiple measurements in problems of biological classification [J]. Journal of the Royal Statistical Society, Series B, 10 (2): 159–203, 1948.
[24] ROUX L N, BENGIO Y, FITZGIBBON A. Improving First and Second–Order Methods by Modeling Uncertainty [M] // Optimization for Machine Learning. Cambridge, MA: MIT Press, 2011.
[25] SAMUEL A L. Some Studies in Machine Learning Using the Game of Checkers [J]. IBM Journal of Research and Development, 3(3): 210–229, 1959.
[26] SHAWE–TAYLOR J, CRISTIANINI N. Kernel methods for pattern analysis [M]. Cambridge: Cambridge university press, 2004.
[27] DAVID B S, HRUBES P, MORAN S, et al. Learnability can be undecidable [J]. Nature Machine Intelligence, 1: 44–48, 2019.
[28] STARNER T, PENTLAND A. Real–Time American Sign Language Visual Recognition From Video Using Hidden Markov Models [D]. Boston: MIT, 1995.
[29] VALIANT L G. A theory of the learnable [J]. Communications of the ACM, 27(11): 1134–1142, 1984.
[30] VAPNIK, VLADIMIR N A. Ya Chervonenkis, On the uniform convergence of relative frequencies of events to their probabilities [J]. Measures of complexity, 17(2): 264–280, 1971.
[31] LEGENDRE A M. Nouvelles Méthodes Pour la Détermination des Orbites des Comètes [M] .French edition. [S.l.] : [s.n.], 1805 .
[32] GAUSS C F. Theoria motus corporum coelestium in sectionibus conicis solem ambientium [J]. Nineteenth Century Collections Online (NCCO), Science, Technology, and Medicine, 7: 1780–1925, 1809.

第5章 统计机器学习：无监督学习

5

物以类聚，人以群分
——《战国策 · 齐策三》

无监督学习就是从无标注数据样本出发，学习数据样本中蕴含的模式，完成如聚类或降维等特定任务。无监督学习在人类学习中占据主导地位：通过观察客观事物去洞悉其内在秉性，而不是被逐一告知每个客观事物的概念名称。

“皮之不存，毛将焉附”。在无监督学习中，虽然数据本身没有标注信息，但是由于数据是内容（如概念和语义等）的载体，因此刻画相同内容的数据载体必然具有相似模式。从这个意义上而言，从无标注数据中挖掘其固有模式，透过数据看本质，就成为无监督学习要解决的主要问题。

物以类聚，人以群分。知类、辨群是人类认识世界的重要阶段。图 5.1 给出了一个无监督学习进行数据聚类的示意。在图 5.1 中，虽然没有对数据进行标注，但是无监督学习算法根据颜色或形状来将未标注数据归属为两个集合，如把相同颜色数据或者相同形状的数据归属到同样集合中。

在现实世界中，高质量标注数据难以获得，因此无监督学习是监督学习的必要补充。常用的无监督学习算法包括如 K 均值聚类（K-means）、层次聚类（hierarchical clustering）等聚类算法，如自编码器（autoencoder）[27]、生成对抗网络（generative adversarial nets，GAN）[28] 等基于神经网络的算法，以及如期望最大算法（expectation-maximization algorithm，EM）[7] 等隐变量学习模型。本章主要从聚类、特征降维和模型学习等角度来介绍无监督学习，包括 K 均值聚类、主成分分析、特征人脸方法、隐性语义分析和期望最大化（EM）算法。

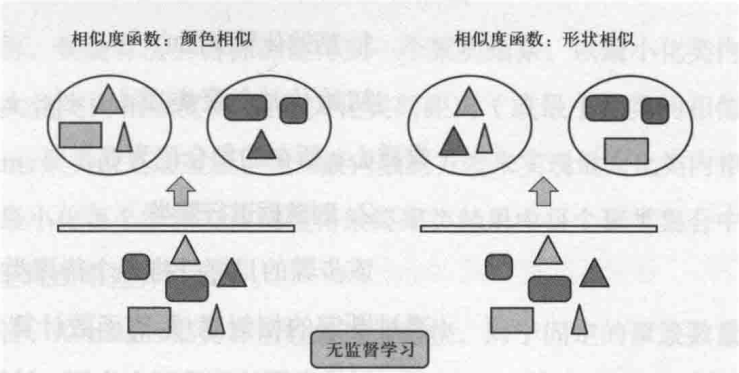


图5.1 无监督学习示意图

5.1 K均值聚类

K均值聚类（K-means）术语最早由MacQueen提出[29]，是一种聚类算法。聚类任务是将大量数据根据他们的数据特征相似性分成少量簇的任务。K-means算法要求特征变量连续，数据没有异常值。

K-means算法的目标是将n个d维的数据划分为K个聚簇，使得簇内方差最小化。由于可能的聚类结果数量巨大，要求一个特定的聚类算法总是能达到最优是不切实际的。所以，K-means算法找到的是一个“局部”最优，即没有任何其他的聚类结果，能够让簇内的方差更小，但该局部最优不保证是全局最优[2]。K-means同时也是一个易受初始值影响

的迭代算法，可以用不同的初始值重复几次，以达到上述的“局部”最优，常用的初始化方法包括forgy和random partition[8]。

K-means是一种很简单易用的聚类算法，被应用于大量任务中，如入侵检测[24]、云计算[25]、客户细分[26]等。

K-means算法的过程如下：

算法5.1 *K*-means聚类

输入：n个数据的特征向量（d维），聚簇数目K。 输出：每个数据所属聚簇标签。
算法步骤： 1. 初始化聚类质心； 2. 根据预定的相似度/距离函数（通常为欧氏距离）对数据进行聚类； 3. 更新聚类质心； 4. 重复2和3，直到收敛。

下面详细解释每一步。

1. 初始化聚类质心

初始化K个聚类质心$c=\{c_1,c_2,\cdots,c_K\}$，$c_j\in\mathbb{R}^d\ (1\leqslant j\leqslant K)$，每个聚类质心$c_j$所在的集合记为$G_j$。

2. 对数据进行聚类

该步骤的目标是将每个待聚类数据放入唯一一个聚类集合中。首先通过既定的相似度/距离函数计算每个待聚类数据和K个聚类质心的距离。以最常用的欧氏距离为例，计算x_i和c_j之间的欧氏距离：

$$dist\left(x_i,c_j\right)=\sqrt{\sum_{o=1}^{d}\left(x_{i,0}-c_{j,0}\right)^2}\ (1\leqslant i\leqslant n,1\leqslant j\leqslant K)$$

将每个x_i放入与之距离最近的聚类质心所在聚类集合中，即$\mathrm{argmin}_{c_j\in C}dist(x_i,c_j)$。

3. 更新聚类质心

根据聚类结果更新聚类质心，即根据每个聚类集合中所包含的数据，求均值得到该聚类集合新的质心：

$$c_j=\frac{1}{\left|G_j\right|}\sum_{x_i\in G_j}x_i$$

这也是*K*-means名字的来源，均值操作简单高效，这让*K*-means算法可以轻易地被应用在较大规模数据上。不过，均值操作也带了许多问题，比如易受异常值影响，在下面的分析中会进一步介绍。

4. 迭代

算法迭代，即继续根据新的聚类质心，按照欧氏距离大小，将每个待聚类数据放入唯一一个聚类集合中，再根据新的聚类结果更新聚类质心。

聚类迭代终止判断不唯一，经常使用的方法有以下两种。

- 已经达到了迭代次数上限。
- 前后两次迭代中，聚类质心保持不变。

可以从另外一个视角来理解K均值聚类算法：最小化每个类簇的方差。K-means算法在迭代过程中不断减少聚簇内的数据到聚类质心的欧氏距离，这同样也是聚簇内数据方差最小化的目标函数：

$$\underset{G}{\operatorname{argmin}} \sum_{i=1}^{K} \sum_{x \in G_i} \|x - c_i\|^2 = \underset{G}{\operatorname{argmin}} \sum_{i=1}^{K} |G_i| \operatorname{var} G_i$$

$$\text{第}i\text{个类簇的方差：} \operatorname{var}(G_i) = \frac{1}{|G_i|} \sum_{x \in G_i} \|x - c_i\|^2$$

一般而言，聚类算法的目标都是得到一个聚类结果，以最小化类内距离（或最大化类内相似度），而最大化类间距离（或最小化类间相似度）。K-means聚类就是通过最小化聚簇内数据方差来实现最大化类内相似度的，即最小化每个类簇方差将使得最终聚类结果中每个聚类集合中所包含数据呈现出来差异性最小。

通常而言，K-means聚类算法收敛速度很快，对于固定的聚簇数量K和维度d，K-means能够在$O(n^{dK+1})$时间复杂度内收敛[10]。但是在最坏情况下较慢，对于某些特定的点集，即使是在二维平面上，也需要指数级时间复杂度$2^{\Omega(n)}$收敛[9]。

K-means聚类算法简单易用的特点很大程度上来源于均值操作，但是该操作同时也带来了问题。K-means聚类算法假设数据是没有离群点的，它对离群点的处理也和其他数据一样，而离群点对均值影响较大，会让聚类质心偏离没有离群点时的质心，影响聚类结果。可以使用其他目标函数或是K-medoids算法来减小离群点的影响。K-medoids算法选取的中心点（medoids）是属于聚簇中的一个点，这是和K-means聚类算法的主要区别。

K-means聚类算法需要事先确定聚类数目K，但很多时候并不知道数据应被聚为多少类。一种方法是遍历一个范围内的候选值然后测试错误率，但是通常因为可选范围较大而不切实际。

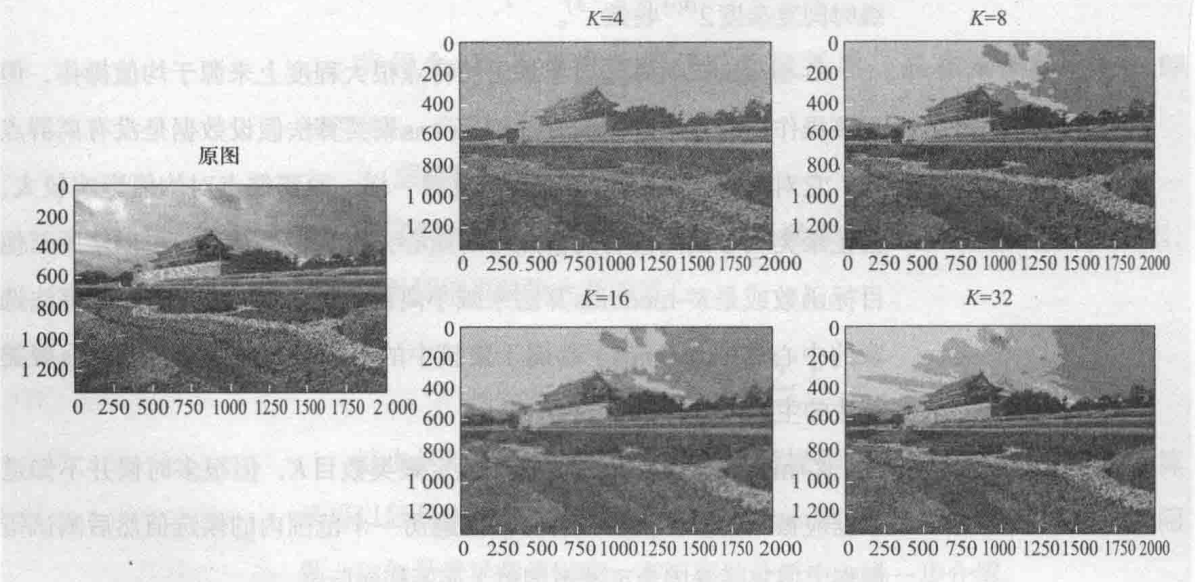
K-means聚类算法对数据的尺度敏感，或者说，对数据所在的坐标空间敏感，如某个长度特征以厘米为单位还是以米为单位对最后结果会有较大影响。这是因为欧氏距离假设数据每个维度的重要性是一样的。

同时也看到，*K*-means聚类对每个数据的归属判定是一种非1即0的，也就是说，每个数据只可能属于一个聚簇，而不可能同时属于另一个聚簇，这种聚类方法称为“硬聚类”（hard clustering）。然而，某些位于聚类边缘的数据点往往会因为数据或初始值的轻微改变而改变所属聚簇类别，这种数据点可能有更好的归属判定方式。所以相应地存在一些聚类算法如高斯混合模型，它对数据的归属判定是概率形式的，即不是非1即0，而是介于0和1的中间值，其含义是某个数据属于不同聚簇的概率。

综上所述，*K*-means具有简单易用的特点，但也有诸多不足，这些不足可以被后来的研究提出的聚类算法解决，例如，层次聚类（hierarchical clustering）不需要事先决定聚类数目*K*，基于降维的聚类算法谱聚类（spectral clustering）对数据所在坐标空间不敏感，这些算法在这里不进行展开。

K-means算法有许多应用，如图像压缩和文本聚类。图5.2给出了图像压缩的一个例子，将某张图像的所有像素点的颜色作为数据集合，选取固定数量*K*个聚簇，将所有的像素点通过*K*-means算法分配到这*K*个聚簇中，然后使用聚簇中心代替聚簇内部的像素点，这样一张图像就可以只使用这*K*种不同的颜色来代替，从而实现图像压缩。图5.2展示了

图5.2 *K*-means图像压缩



对天安门图像压缩的结果，选取更小的K值有更高的压缩率，但是图像色彩失真更严重，选取更大的K值压缩率会下降，但是色彩更加丰富。

5.2 主成分分析

主成分分析（principle components analysis）[11, 12]是一种特征降维的方法，在消除数据噪声、冗余等方面有着广泛的应用。主成分分析也被称为KL变换（Karhunen-Loève transform，KLT）、霍林特变换（Hotelling transform）或者本征正交分解（proper orthogonal decomposition，POD）等。

顾名思义，主成分分析即通过分析找到数据特征的主要成分，使用这些主要成分来代替原始数据。这样一方面可以加深对数据本身的理解（认识到数据的主要成分），另一方面，简化后的数据（主要成分）在用于下游的其他任务时，有着噪声少、易于处理计算的特点。这种使用主要成分而不是全部数据的思想也与“如无必要，勿增实体”的奥卡姆剃刀原则相符，体现了“简单有效原理”。

主成分分析要求“降维后的结果要保持原始数据的原有结构”，例如，对于图像数据，要求保持视觉对象区域构成的空间分布；对于文本数据，要求保持单词之间的（共现）相似或不相似的特性。从数学的角度而言，主成分分析要求最大程度保持原始高维数据总体方差结构。

主成分分析被广泛应用于各种领域，比如量化金融[13, 14]、神经科学[15]。

5.2.1 方差、协方差和相关系数

在进入到主成分分析算法的具体内容之前，先回顾下包括方差、协方差、相关系数在内的预备知识。

方差：方差描述了样本数据的波动程度，数值上等于各个数据与样本均值之差的平方和之平均数，假设有n个数据，记为$X=\{x_i\}$ $(i=1, \cdots, n)$，那么样本方差（sample variance）即为：

$$\mathrm{var}(X)=\frac{1}{n-1}\sum_{i=1}^{n}(x_i-u)^2$$

其中u是样本均值，$u=\frac{1}{n}\sum_{i=1}^{n}x_i$。上述样本方差公式里分母为$n-1$的目的是为了让对方差的估计是无偏估计（unbiased estimator）。其原因留为书后的习题。

协方差：协方差衡量了两个变量之间的相关度，假设有两个变量，观察到不同时刻两个变量的取值，记为$(X,Y)=\{(x_i,y_i)\}(i=1,\cdots,n)$，那么两个变量的协方差为：

$$\operatorname{cov}(X,Y)=\frac{1}{n-1}\sum_{i=1}^{n}(x_i-E(X))(y_i-E(Y))$$

其中$E(X)$和$E(Y)$分别是X和Y的样本均值，分别定义如下$E(X)=\frac{1}{n}\sum_{i=1}^{n}x_i, E(Y)=\frac{1}{n}\sum_{i=1}^{n}y_i$。

表5.1给出一个方差与协方差的例子。

表5.1　方差与协方差的计算例子（无偏估计）

编号	x_i	y_i	$x_i-E(X)$	$y_i-E(Y)$	$[x_i-E(X)][y_i-E(Y)]$
1	1	7	-8.33	-16.67	138.89
2	3	11	-6.33	-12.67	80.22
3	6	17	-3.33	-6.67	22.22
4	10	25	0.67	1.33	0.89
5	15	35	5.67	11.33	64.22
6	21	47	11.67	23.33	272.22
	$E(X)=9.33$	$E(Y)=23.67$	$\operatorname{var}(X)=57.87$	$\operatorname{var}(Y)=231.47$	$\operatorname{cov}(X,Y)=115.73$

对于一组两维变量（如广告投入-商品销售、天气状况-旅游出行等），可通过计算它们之间的协方差值来判断这组数据给出的两维变量是否存在关联关系：

当协方差$\operatorname{cov}(X,Y)>0$时，称$X$与$Y$正相关；

当协方差$\operatorname{cov}(X,Y)<0$时，称$X$与$Y$负相关；

当协方差$\operatorname{cov}(X,Y)=0$时，称$X$与$Y$不相关。

由于协方差会受到变量取值尺度的影响，可以通过皮尔逊相关系数（Pearson correlation coefficient）将两组变量之间的关联度规整到一定的取值范围内。皮尔逊相关系数定义如下：

$$\operatorname{corr}(X,Y)=\frac{\operatorname{cov}(X,Y)}{\sqrt{(\operatorname{Var}(X)\operatorname{Var}(Y))}}=\frac{\operatorname{cov}(X,Y)}{\sigma_x\sigma_y}$$

其中$\sigma_x\sigma_y$分别为X和Y的标准差。

皮尔逊相关系数所具有的性质如下：

$$|\mathrm{corr}(X,Y)|\leqslant 1$$

$\mathrm{corr}(X, Y)=1$的充要条件是存在常数a和b，使得$Y=aX+b$，在上面的案例中，可以计算得$\mathrm{corr}(X, Y)=1.0$，同时可以找到常数$a=2$和$b=5$，即$Y=2X+5$完全拟合上面的数据。

皮尔逊相关系数是对称的，即$\mathrm{corr}(X, Y)=\mathrm{corr}(Y, X)$。

由此衍生出如下性质：皮尔逊相关系数刻画了变量X和Y之间线性相关程度，如果$|\mathrm{corr}(X, Y)|$的取值越大，则两者在线性相关的意义下相关程度越大。$|\mathrm{corr}(X, Y)|=0$表示两者不存在线性相关关系（可能存在其他非线性相关的关系）。

正线性相关意味着变量X增加的情况下，变量Y也随之增加；负线性相关意味着变量X减少的情况下，变量Y随之增加。

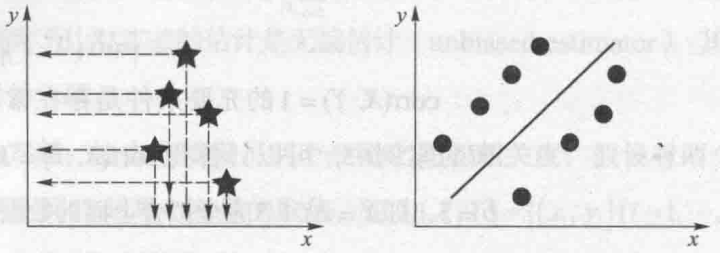
注意，线性无关与之前学习过的另外一个概念“独立”不同，线性无关是一个比独立要弱的概念，即两个变量独立一定线性无关，即$|\mathrm{corr}(X, Y)|=0$，但是两个线性无关的变量不一定独立（可能存在其他复杂的关联关系）。独立指两个变量彼此之间不相互影响。

5.2.2 主成分分析

回到特征降维，降维需要尽可能将数据向方差最大的方向进行投影，使得数据所蕴含的信息丢失得尽可能少。如图5.3（a）所示，向y方向投影（使得二维数据映射为一维）就比向x方向投影结果在降维这个意义上而言要好；图5.3（b）则是黑线方向投影要好。这样的投影结果更好地保留了未降维前数据的离散程度。

主成分分析（principle component analysis）的思想是将d维特征数据映射到l维空间（一般$d\gg l$），去除原始数据之间的冗余性（通过去除相关性手段达到这一目的）。将原始数据向这些数据方差最大的方向进行投影，一旦发现了方差最大的投影方向，则继续寻找保持方差第二的方向且进行投影，其目标是使得数据每一维的方差都尽可能大。主成分分析的算法解释如下。

假设有n个d维样本数据所构成的集合$D=\{x_1, x_2, \cdots, x_n\}$，其中$x_i\left(1\leqslant i\leqslant n\right)\in\mathbb{R}^d$。集合$D$可以表示成一个$n\times d$的矩阵$\boldsymbol{X}$。假定数据每一维度的特征均值均为零（已经标准化）。主成分分析的目的是求取一个



(a) 向y方向投影更好　　(b) 向黑线投影更好

图5.3 PCA降维示意图

$d \times l$的映射矩阵$\boldsymbol{W}$。给定一个样本x_i，可将x_i从d维空间如下映射到l维空间：$(x_i)_{1\times d}(\boldsymbol{W})_{d\times l}$。将所有降维后的数据用$\boldsymbol{Y}$表示，有$\boldsymbol{Y}=\boldsymbol{XW}$，其中$\boldsymbol{Y}\in\mathbb{R}^{n\times l}$是降维后的结果，$\boldsymbol{X}\in\mathbb{R}^{n\times d}$是原始数据，$\boldsymbol{W}\in\mathbb{R}^{d\times l}$是映射矩阵。

降维后n个l维样本数据$\boldsymbol{Y}$的方差为：

$$\begin{aligned}\operatorname{var}(\boldsymbol{Y}) &= \frac{1}{n-1}tr\left(\boldsymbol{Y}^{\mathrm{T}}\boldsymbol{Y}\right)\\ &= \frac{1}{n-1}tr\left(\boldsymbol{W}^{\mathrm{T}}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{XW}\right)\\ &= tr\left(\boldsymbol{W}^{\mathrm{T}}\frac{1}{n-1}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{XW}\right)\end{aligned}$$

其中tr表示矩阵的迹（trace），即一个方阵主对角线（从左上方到右下方的对角线）上各个元素的总和。降维前n个d维样本数据$\boldsymbol{X}$的协方差矩阵记为：

$$\boldsymbol{\Sigma}=\frac{1}{n-1}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{X}$$

主成分析的优化求解目标函数为：

$$\max_{\boldsymbol{W}} tr\left(\boldsymbol{W}^{\mathrm{T}}\boldsymbol{\Sigma W}\right)$$

该优化求解目标需要满足如下约束条件

$$\boldsymbol{w}_i^{\mathrm{T}}\boldsymbol{w}_i=1 \quad i\in\{1,2,\cdots,l\}$$

这是带约束的最优化问题求解，可以通过拉格朗日乘子法将上述转化为无约束的最优化问题，拉格朗日函数如下：

$$L(\boldsymbol{W},\lambda)=tr\left(\boldsymbol{W}^{\mathrm{T}}\boldsymbol{\Sigma W}\right)-\sum_{i=1}^{l}\lambda_i\left(\boldsymbol{w}_i^{\mathrm{T}}\boldsymbol{w}_i-1\right)$$

其中$\lambda_i(1\leqslant i\leqslant l)$为拉格朗日乘子，$\boldsymbol{w}_i$为矩阵$\boldsymbol{W}$第$i$列。

对上述拉格朗日函数中的变量$\boldsymbol{w}_i$求偏导并令导数为零，有$\boldsymbol{\Sigma w}_i=\lambda_i\boldsymbol{w}_i$

上式表明：每一个$\boldsymbol{w}_i$均是n个d维样本数据$\boldsymbol{X}$的协方差矩阵$\boldsymbol{\Sigma}$的特征

向量，λ_i是这个特征向量所对应的特征值。

$$\boldsymbol{\Sigma w}_i=\lambda_i \boldsymbol{w}_i,\ \text{且}\ tr\left(\boldsymbol{W}^{\mathrm{T}}\boldsymbol{\Sigma W}\right)=\sum_{i=1}^{l}\boldsymbol{w}_i^{\mathrm{T}}\boldsymbol{\Sigma w}_i=\sum_{i=1}^{l}\lambda_i。$$

可见，在主成分分析中，最优化的方差等于原始样本数据$\boldsymbol{X}$的协方差矩阵$\boldsymbol{\Sigma}$的特征根之和。

为了使方差最大，可以求出协方差矩阵$\boldsymbol{\Sigma}$的特征向量和特征根，然后取前l个最大特征根所对应的特征向量组成映射矩阵$\boldsymbol{W}$即可。

注意，每个特征向量$\boldsymbol{w}_i$与原始数据x_i的维数是一样的，均为d。

算法5.2　主成分分析

输入：n个d维样本数据所构成的矩阵$\boldsymbol{X}$，降维后的维数l。
输出：映射矩阵$\boldsymbol{W}=\{\boldsymbol{w}_1, \boldsymbol{w}_2, \cdots, \boldsymbol{w}_l\}$。

算法步骤：

1. 对于每个样本数据x_i进行中心化处理：$x_i=x_i-\mu, \mu=\frac{1}{n}\sum_{j=1}^{n}x_j$；
2. 计算原始样本数据的协方差矩阵：$\boldsymbol{\Sigma}=\frac{1}{n-1}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{X}$；
3. 对协方差矩阵$\boldsymbol{\Sigma}$特征值分解，对所得特征根进行排序$\lambda_1\geqslant\lambda_2\geqslant\cdots\geqslant\lambda_l$；
4. 取前l个最大特征根所对应特征向量$\boldsymbol{w}_1, \boldsymbol{w}_2, \cdots, \boldsymbol{w}_l$组成映射矩阵。

5.2.3 其他常用降维方法

除了主成分分析外还有许多降维方法，这里对它们做简单的介绍。

1. 非负矩阵分解

非负矩阵分解（non-negative matrix factorization，NMF）顾名思义，即将非负的大矩阵分解成两个非负的小矩阵。该方法于1999年由Lee和Seung发表于*Nature*上[3]。矩阵分解的一般目标为将原始矩阵$\boldsymbol{D}$表示为矩阵$\boldsymbol{W}$和矩阵$\boldsymbol{H}$相乘的形式，即$\boldsymbol{D}=\boldsymbol{WH}$。上述的主成分分析方法即可以实现这一点，不过主成分分析不要求原始矩阵和无法保证分解结果矩阵中所有元素为正数（即非负）。在很多实际情况中，如图像所组成的像素矩阵或自然语言中常用的单词-文档矩阵中并不存在为负数的元素，因此，对非负矩阵的分解是很有意义的问题。

非负矩阵分解将给定的非负矩阵$\boldsymbol{D}\in\mathbb{R}^{n\times d}$分解为两个非负矩阵$\boldsymbol{W}\in\mathbb{R}^{n\times l}$和$\boldsymbol{H}\in\mathbb{R}^{l\times d}$，并满足下面的条件：

$$\boldsymbol{D}\approx\boldsymbol{WH}\quad D_{i,j}, W_{i,\mu}, H_{\mu,j}\geqslant 0$$

其中，$0\leqslant i<n-1$，$0\leqslant j<d-1$，$0\leqslant\mu<l-1$。

对一个非负矩阵$\boldsymbol{D}$进行非负矩阵分解的结果$\boldsymbol{W}$和$\boldsymbol{H}$并不唯一。假设

某个矩阵分解结果做下面的变换，利用任意的一个可逆矩阵$\boldsymbol{A}$以及它的逆矩阵$\boldsymbol{A}^{-1}$，可得：

$$\boldsymbol{WH}=\boldsymbol{WAA}^{-1}\boldsymbol{H}$$

很容易找到矩阵$\boldsymbol{A}$使得矩阵$\hat{\boldsymbol{W}}=\boldsymbol{WA}$和$\hat{\boldsymbol{H}}=\boldsymbol{A}^{-1}\boldsymbol{H}$均非负；这样就找到了另外一种分解，即$\boldsymbol{D}\approx\hat{\boldsymbol{W}}\hat{\boldsymbol{H}}$。

为了得到原始矩阵$\boldsymbol{D}$的分解矩阵$\boldsymbol{W}$和$\boldsymbol{H}$，首先定义原始矩阵$\boldsymbol{D}$与$\boldsymbol{WH}$之间的误差，即代价函数（cost function）。代价函数被用来度量原始矩阵$\boldsymbol{D}$与其近似矩阵$\boldsymbol{WH}$之间的误差，这样求取原始矩阵的分解矩阵$\boldsymbol{W}$和$\boldsymbol{H}$的问题可以转变为一个误差最小化问题。两种常用的代价函数如下。

（1）欧式距离平方函数

$$\left\|\boldsymbol{D}-\boldsymbol{WH}\right\|_F^2=\sum_{0\leqslant i<n,\,0\leqslant j<d}\left(D_{i,j}-\left(\boldsymbol{WH}\right)_{i,j}\right)^2$$

（2）KL散度（Kullback–Leibler divergence）函数

$$D_{KL}\left(\boldsymbol{D}\middle\|\boldsymbol{WH}\right)=\sum_{0\leqslant i<n,\,0\leqslant j<d}D_{i,j}\log\frac{D_{i,j}}{\left(\boldsymbol{WH}\right)_{i,j}}-D_{i,j}+\left(\boldsymbol{WH}\right)_{i,j}$$

当代价函数是欧式距离平方函数时，矩阵$\boldsymbol{W}$和$\boldsymbol{H}$可使用如下的迭代方法求解。

$$H_{a,\mu}\leftarrow H_{a,\mu}\frac{\left(\boldsymbol{W}^{\mathrm{T}}\boldsymbol{D}\right)_{a,\mu}}{\left(\boldsymbol{W}^{\mathrm{T}}\boldsymbol{WH}\right)}\quad W_{i,a}\leftarrow W_{i,a}\frac{\left(\boldsymbol{DH}^{\mathrm{T}}\right)_{i,a}}{\left(\boldsymbol{WHH}^{\mathrm{T}}\right)_{i,a}}$$

其中，$0\leqslant a<l$，$0\leqslant\mu<d$，$0\leqslant i<n$。

当代价函数是KL散度函数时，矩阵$\boldsymbol{W}$和$\boldsymbol{H}$可使用如下的迭代方法求解。

$$H_{a,\mu}\leftarrow H_{a,\mu}\frac{\sum_i W_{i,a}D_{i,\mu}/\left(\boldsymbol{WH}\right)_{i,\mu}}{\sum_k W_{k,a}}\quad W_{i,a}\leftarrow W_{i,a}\frac{\sum_\mu H_{a,\mu}D_{i,\mu}/\left(\boldsymbol{WH}\right)_{i,\mu}}{\sum_v W_{a,v}}$$

非负矩阵分解依据了“对整体的感知是由对组成整体的部分进行感知而构成的（纯加性的）”这一观点，即“部分组成整体（part–of–the–whole）”。这也符合直观的理解：整体是由部分组成的。其要求分解后的所有分量均为非负值（即纯加性描述），并且同时实现非线性的维数降维。

2. 多维尺度法

与主成分分析保持数据方差结构的思想不同，多维尺度法（metric multidimensional scaling，MDS）[4]保持原始数据之间两两距离不变。

MDS算法会计算原始数据两两之间的距离，形成一个距离矩阵，以此来保证降维之前距离近的数据在降维之后也能很接近。不过，也正因为MDS需要知道这样的距离矩阵，所以无法对新数据集合进行降维（如测试集），这被称为“out-of-sample”问题。

令N是原始数据数量，每个数据由d维特征表示，即$x_i=\{x_{i1}, x_{i2}, \cdots, x_{id}\}$，可以将所有原始数据表示为$\boldsymbol{X}$，$\boldsymbol{X}=[x_1, \cdots, x_i, \cdots, x_n]$。多维尺度法首先计算任意两个数据$x_i$和$x_j(1\leqslant i, j\leqslant n)$之间的距离$s_{ij}$，得到距离矩阵$\boldsymbol{A}$，即$\boldsymbol{A}=(s_{ij})$。$\boldsymbol{A}$是一个$n\times n$大小的矩阵，$x_i$和$x_j$之间的距离度量函数由具体应用来定（如可选取余弦距离等）。

接着计算中心化矩阵$\boldsymbol{H}=\boldsymbol{I}-\frac{1}{n}\mathbf{1}_n\mathbf{1}_n^{\mathrm{T}}$，其中$\boldsymbol{I}$是单位矩阵，$\mathbf{1}_n$是全1的$n$维列向量，可以得到如下$n\times n$大小的矩阵$\boldsymbol{B}$：

$$\boldsymbol{B}=-\frac{1}{2}\boldsymbol{HAH}$$

假设矩阵$\boldsymbol{B}$的秩为p，对矩阵$\boldsymbol{B}$进行奇异值分解，可得：

$$\boldsymbol{B}=\boldsymbol{\Gamma\Lambda\Gamma}^{\mathrm{T}}$$

其中$\boldsymbol{\Lambda}=diag(\lambda_1, \cdots, \lambda_l)$是一个对角矩阵，对角线上的元素为矩阵$\boldsymbol{B}$的非零特征根；$\boldsymbol{\Gamma}=(\gamma_1, \cdots, \gamma_l)$是一个$n\times l$矩阵，其中$\gamma_j$为特征根$\lambda_j(1\leqslant j\leqslant l)$所对应的特征向量。

于是$\boldsymbol{X}=[x_1, \cdots, x_i, \cdots, x_n]$中$d$维原始数据可通过如下变换转变为$l$维数据，从而实现降维：

$$\overline{\boldsymbol{X}}^{\mathrm{T}}=\left[\overline{x}_1, \overline{x}_2, \cdots, \overline{x}_n\right]^{\mathrm{T}}=\boldsymbol{\Gamma\Lambda}^{1/2}$$

其中$\overline{x}_i(1\leqslant i\leqslant n)$表示原始数据$x_i$降维后的结果。

3. 局部线性嵌入

局部线性嵌入（locally linear embedding，LLE）相关工作最早发表在*Science*上[5]。上述的PCA和MDS都属于线性降维方法，LLE是一种非线性降维方法。LLE的基本假设是：一个流形的局部可以近似于一个欧式空间，每个样本均可以利用其邻居进行线性重构。换句话说，如果假设数据是局部线性的（即使数据的原始高维空间是非线性流形嵌入），于是可以用和邻居数据的线性关系进行局部重构。

局部线性嵌入保留了每个数据的局部性质，利用局部线性来逼近全局非线性，通过相互重叠的局部邻域提供全局结构信息，最终保持数据整体几何信息。

局部线性嵌入算法的基本步骤如下。

（1）假设n个原始数据$x_i(1 \leqslant i \leqslant n)$的维数为$d$，计算任意数据$x_i$ $(1 \leqslant i \leqslant n)$的$k$近邻。

（2）如下计算数据点的局部重建权值，使样本点的重建误差最小：

$$\min_{\{w_{ij}\}} \sum_{i=1}^{n} \left\| x_i - \sum_{x_j \in N_k(x_i)} w_{ij} x_j \right\|^2$$

$$\text{s.t.} \quad \sum_{x_j \in N_k(x_i)} w_{ij} = 1, i = 1, \cdots, n$$

其中，$N_k(x_i)$表示x_i的k个近邻，w_{ij}间表示使用近邻x_j来重建x_i时的权重，k个近邻的权重和为1。

（3）将每个数据点重建权重组成矩阵$\boldsymbol{W}$，实际上是要寻找原始数据$x_i(1 \leqslant i \leqslant n)$的低维嵌入$y_i(1 \leqslant i \leqslant n)$，使得重建误差最小，即$\min \Phi(\boldsymbol{Y}) = \sum_{i=1}^{n} \left\| y_i - \sum_{j=1}^{n} w_{ij} y_j \right\|^2$。

（4）在具体计算中，需要对降维结果加以一定约束，即$\sum_{i=1}^{n} y_i = 0$，$\frac{1}{n} \sum_{i=1}^{n} y_i y_i^{\mathrm{T}} = \boldsymbol{I}$，其中$\boldsymbol{I}$是$n$维单位矩阵。这样，局部线性嵌入就变为如下优化函数：

$$\min \Phi(\boldsymbol{Y}) = \sum_{i=1}^{n} \left\| \boldsymbol{Y} I_i - \boldsymbol{Y} W_i \right\|^2 = \sum_{i=1}^{n} \left\| \boldsymbol{Y} (I_i - W_i) \right\|^2 = \min tr\left(\boldsymbol{Y} \boldsymbol{M} \boldsymbol{Y}^{\mathrm{T}} \right)$$

$$\text{s.t.} \quad \boldsymbol{Y} \boldsymbol{Y}^{\mathrm{T}} = \boldsymbol{I}$$

其中$\boldsymbol{M} = (\boldsymbol{I} - \boldsymbol{W})^{\mathrm{T}} (\boldsymbol{I} - \boldsymbol{W})$。使用拉格朗日乘子，可得$\boldsymbol{M} \boldsymbol{Y}^{\mathrm{T}} = \lambda \boldsymbol{Y}^{\mathrm{T}}$，取矩阵$\boldsymbol{M}$最小的前$k$个非零特征根所对应特征向量构成的矩阵即可将原始数据$x_i(1 \leqslant i \leqslant n)$从$d$维空间转变为$k$维空间中的$y_i(1 \leqslant i \leqslant n)$。在实际计算中，矩阵$\boldsymbol{M}$最小特征值几乎为零，因此通常取第2 ~ $(k+1)$之间的特征值所对应特征向量。

5.3 特征人脸方法

特征人脸是基于外观的人脸识别方法，其目的是捕捉人脸图像集合中的特征信息，并使用该信息对各个人脸图像进行编码和比较。换句话说，特征人脸提取了人脸图像的面部信息，这些信息可能与人的直觉相符，如眼睛、鼻子和嘴唇的大小形状特征。这样，特征人脸就可以通过

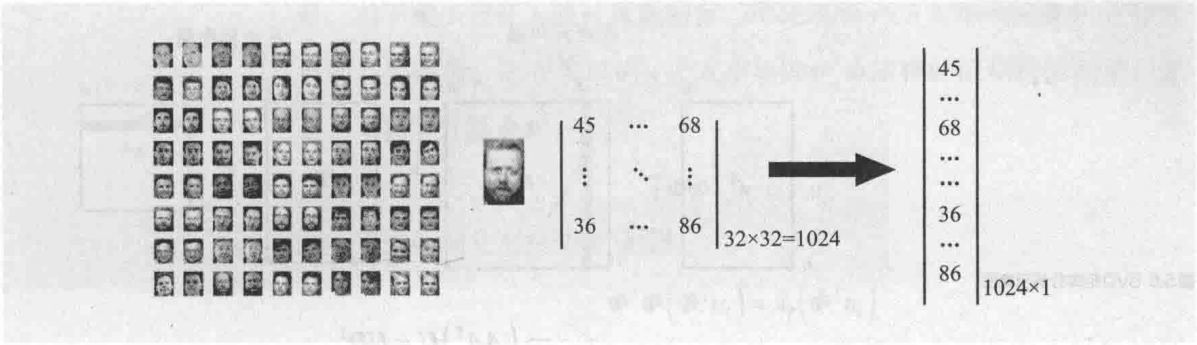


图5.4 二维灰度图像的向量化表示

少量特征有效表现原始面部图像，减少了计算和空间复杂度。

使用主成分分析的手段来表示特征人脸的思想最先由Sirovich和Kirby[6]提出，本质是使用一组特征向量线性组合来表示原始人脸，进而实现人脸识别。

假设总共有n张灰度人脸图像，每张人脸图像Γ_i的分辨率为$d\times d$。如果$d=32$，即人脸图像的分辨率为32×32，人脸图像在计算机中可以表示为一个32×32的像素矩阵，如果将该矩阵铺平，可以转换为一个1 024维的列向量，如图5.4所示。

该1 024维的列向量是原始灰度图像的表达，但是直觉表明，人脸具有一定的拓扑结构，像素点之间有较强的空间位置关系，也就是说，可以使用一个低维向量来表达原图像大部分的信息。这样就可以使用上一章讲述的主成分分析算法来实现降维。不过在特征维度较高的情况下，主成分分析算法暴力求解特征向量是一个耗时操作。这里介绍一种新的矩阵分解方法——奇异值分解（singular value decomposition，SVD）来实现主成分分析，对原始数据进行降维。

5.3.1 奇异值分解

奇异值分解会将矩阵$\boldsymbol{A}$分解为三个子矩阵，即$\boldsymbol{A}=\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^{\mathrm{T}}$，其中矩阵$\boldsymbol{U}$和矩阵$\boldsymbol{V}$都是酉矩阵，即满足$\boldsymbol{U}\boldsymbol{U}^{\mathrm{T}}=\boldsymbol{V}\boldsymbol{V}^{\mathrm{T}}=\boldsymbol{I}$。矩阵$\boldsymbol{D}$是对角矩阵，它的每一项都是正实数。与特征分解不同，奇异值分解不要求被分解的矩阵$\boldsymbol{A}$是方阵，假如矩阵$\boldsymbol{A}$的维度是$n\times d$，则矩阵$\boldsymbol{U}$的维度为$n\times n$，矩阵$\boldsymbol{D}$的维度为$n\times d$，矩阵$\boldsymbol{V}^{\mathrm{T}}$的维度为$d\times d$，如图5.5所示，注意这里的$n$一般指样本个数，$d$为每个样本的维数大小。

那么如何求解分解后的三个子矩阵呢？容易发现：

$$\boldsymbol{A}\boldsymbol{A}^{\mathrm{T}}=\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^{\mathrm{T}}\left(\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^{\mathrm{T}}\right)^{\mathrm{T}}=\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^{\mathrm{T}}\boldsymbol{V}\boldsymbol{D}^{\mathrm{T}}\boldsymbol{U}^{\mathrm{T}}=\boldsymbol{U}\boldsymbol{D}\boldsymbol{D}^{\mathrm{T}}\boldsymbol{U}^{\mathrm{T}}=\boldsymbol{U}\boldsymbol{D}^{2}\boldsymbol{U}^{\mathrm{T}}$$

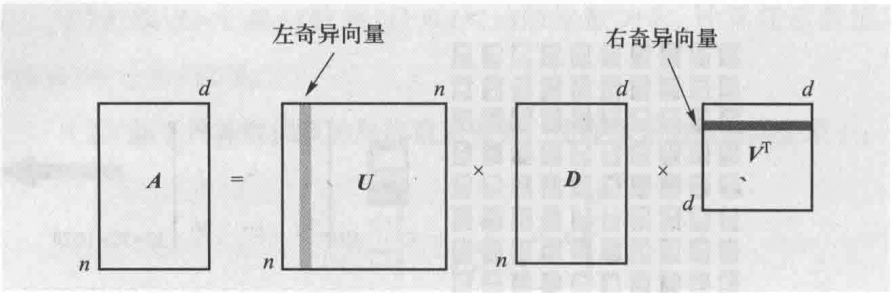


图5.5 SVD矩阵分解示意图

$$\Rightarrow \left(\boldsymbol{A}\boldsymbol{A}^{\mathrm{T}}\right)\boldsymbol{U}=\boldsymbol{U}\boldsymbol{D}^{2}$$

所以矩阵$\boldsymbol{U}$即为矩阵$\boldsymbol{A}\boldsymbol{A}^{\mathrm{T}}$所有特征向量构成的矩阵，同理矩阵$\boldsymbol{V}$即为矩阵$\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}$所有特征向量构成的矩阵。同时，矩阵$\boldsymbol{D}$就等于矩阵$\boldsymbol{A}\boldsymbol{A}^{\mathrm{T}}$或矩阵$\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}$的所有特征值开根号后组成的对角矩阵。或者也可以使用矩阵$\boldsymbol{V}$和矩阵$\boldsymbol{U}$，通过式子$\boldsymbol{U}^{\mathrm{T}}\boldsymbol{A}\boldsymbol{V}=\boldsymbol{D}$得到矩阵$\boldsymbol{D}$。

虽然上述的求解比PCA对特征协方差矩阵做特征分解更为繁琐，但是存在一些算法不用求出矩阵$\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}$，也可以得到右奇异矩阵$\boldsymbol{V}$，继而选取前$K$个特征向量作为主要成分实现主成分分析。这样可以更加高效地实现PCA，事实上，许多常用的算法包如scikit-learn中的PCA实现就是使用SVD分解。注意，从矩阵$\boldsymbol{U}$选取前l个奇异向量可以实现对原矩阵$\boldsymbol{A}$行数的压缩，而从矩阵$\boldsymbol{V}$选取前l个奇异向量可以实现对原矩阵$\boldsymbol{A}$列数的压缩，即特征维度的压缩，这也就是PCA算法所要达到的目标。

5.3.2 特征人脸方法

接下来使用奇异值分解得到特征人脸。

首先计算所有的n张灰度人脸图像向量的均值人脸向量，计算为$\boldsymbol{\Psi}=\frac{1}{n}\sum_{i=1}^{n}\boldsymbol{\Gamma}_i$。得到均值人脸后，用每张人脸图像向量减去均值人脸向量，即进行中心化处理，得到处理后的人脸向量，记为$\boldsymbol{\phi}_i=\boldsymbol{\Gamma}_i-\boldsymbol{\Psi}$ $(1\leqslant i\leqslant n)$。

现在，所有处理后的人脸向量可以被表示为一个$n\times d^2$的矩阵，记为$\boldsymbol{\Phi}$，$\boldsymbol{\Phi}$中的每一行即为中心化处理后的人脸向量。

如果使用主成分分析，需要计算维度为$d^2\times d^2$的特征的协方差矩阵$\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{\Phi}$，然后计算协方差矩阵的特征向量（右奇异向量），但是当$d^2\gg n$时，这就是一个相对耗时甚至不切实际的方法。注意到：

$$\boldsymbol{U}^{\mathrm{T}}\boldsymbol{A}=\boldsymbol{U}^{\mathrm{T}}\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^{\mathrm{T}}=\boldsymbol{D}\boldsymbol{V}^{\mathrm{T}}\Rightarrow u_i^{\mathrm{T}}\boldsymbol{A}=d_i v_i^{\mathrm{T}}$$

所以可以使用SVD求取左奇异向量和特征值，然后得到右奇异向

量。对于减少特征人脸计算量而言，可先求取$n \times n$大小矩阵$\boldsymbol{\Phi}\boldsymbol{\Phi}^{\mathrm{T}}$的特征值和特征向量，而非求取$d^2 \times d^2$大小矩阵$\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{\Phi}$的特征值和特征向量。于是，可如下计算$\boldsymbol{\Phi}\boldsymbol{\Phi}^{\mathrm{T}}$。

$$\left(\boldsymbol{\Phi}\boldsymbol{\Phi}^{\mathrm{T}}\right)u_i = \lambda_i u_i$$

上式两端分别左乘$\boldsymbol{\Phi}^{\mathrm{T}}$可得：

$$\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{\Phi}\left(\boldsymbol{\Phi}^{\mathrm{T}}u_i\right) = \lambda_i\left(\boldsymbol{\Phi}^{\mathrm{T}}u_i\right)$$

可知$\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{u}_i$是矩阵$\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{\Phi}$的特征向量，这样就通过先求取$\boldsymbol{\Phi}\boldsymbol{\Phi}^{\mathrm{T}}$的特征向量$\boldsymbol{u}_i$，得到了$\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{\Phi}$的特征向量$\boldsymbol{v}_i = \boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{u}_i (1 \leqslant i \leqslant n)$。这里每一个大小为$d^2 \times 1$的列向量$\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{u}_i$就是一个特征人脸，$n$个$d^2 \times 1$的列向量$\boldsymbol{v}_i$所组成的矩阵记为$\boldsymbol{V}_{d^2 \times n}$，该矩阵称为特征人脸空间。需要注意的是，$\boldsymbol{V}_{d^2 \times n}$中每一列就是一个特征人脸。

对于先前得到的每幅人脸图像$\boldsymbol{\phi}_i = \boldsymbol{\Gamma}_i - \boldsymbol{\Psi} (i \leqslant i \leqslant n)$，可以如下得到$\boldsymbol{\phi}_i$在特征人脸空间中的表示：

$$\boldsymbol{\Omega}_i = \left(\boldsymbol{\phi}_i\right)_{1 \times d^2} \boldsymbol{V}_{d^2 \times n}$$

这里将$\boldsymbol{\phi}_i$从$d \times d$大小转换成了n维大小的向量，n维向量中每个值反映了$\boldsymbol{\phi}_i$与每个特征人脸相关性大小。于是可以用这n个相关性系数来表征原始人脸。

这样就实现了将人脸从像素点空间转换到人脸空间，即用特征人脸来表达人脸。要比较两幅人脸图像$\boldsymbol{\phi}_i$和$\boldsymbol{\phi}_j$是否相似只需要比较其对应在特征人脸空间的表示$\boldsymbol{\Omega}_i$和$\boldsymbol{\Omega}_j$是否相似。

同理，对于训练集以外的任意$d \times d$大小的人脸图像，也可以通过上述转换得到其在特征人脸空间n维向量大小的表示，即这种特征人脸投影方法能够处理训练集以外（out-of-sample）的数据。另外也可以对特征人脸空间的大小进行选择，即选择与应用合适的特征人脸个数来构成特征人脸空间。

图5.6和图5.7给出了对Olivetti Research Laboratory的人脸库进行特征人脸处理，以及使用选取的前l个最大特征值对应的特征人脸对原始灰度人脸进行压缩和重建的结果。这一人脸数据库包括40个不同肤色、种族和性别的人，在不同的时间和不同的光照条件下拍摄的脸部灰度图像各10幅。其中若干人脸图像存在部分脸部遮挡（戴眼镜）和脸部有不同侧向。

可以看出，每幅特征人脸较好表达了原始人脸的全局信息，但是无法凸显人脸中的局部特性。这也验证了主成分分析方法生成的特征人脸

图5.6 特征人脸向量的可视化

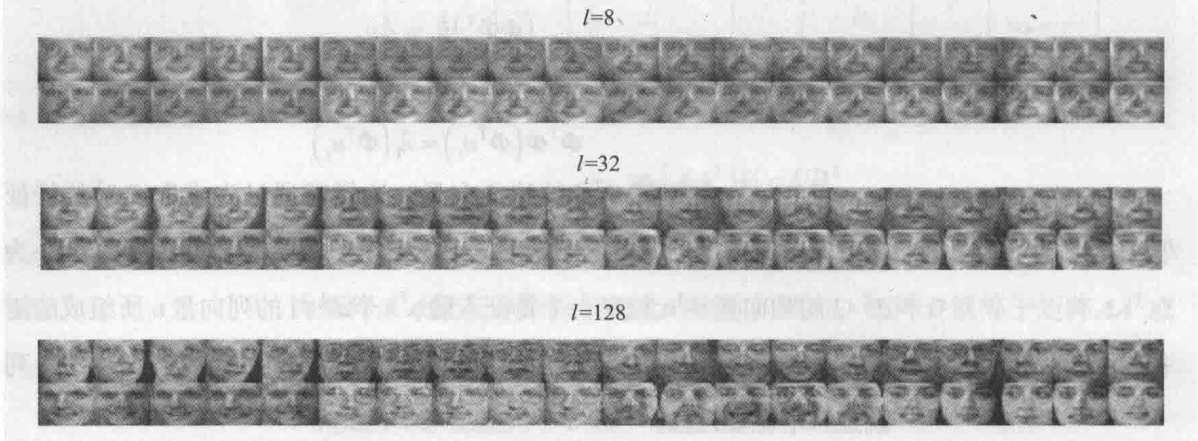


图5.7 特征人脸重建原始灰度人脸图像

只能提取原始图像中存在的全局信息，存在无法如非负矩阵分解方法那样能够提取局部信息的局限性。

使用少量的特征人脸对原始人脸压缩和重建就可以有人脸的大致模样，但是一方面人脸细节模糊，另一方面重建后的人脸相互趋同；随着特征人脸数量l的增加，人物细节变得越来越清晰，特征人脸空间能够表达的人脸的多样性也相应地增加了。如何选择特征人脸数量l是对重建质量多样性和算法空间时间复杂度的权衡。

5.4 潜在语义分析

潜在语义分析，又叫隐性语义分析（latent semantic analysis，LSA或latent semantic indexing，LSI），是一种从海量文本数据中学习单词-单词、单词-文档以及文档-文档之间隐性关系，进而得到文档和单词表达特征的方法。该方法的基本思想是综合考虑某些单词在哪些文档中同时出现，以此来决定该词语的含义与其他的词语的相似度。LSI被广泛运用在了语义理解任务中，如自动文档分类[17]、文本摘要[18]和关系发现[19]。

5.4.1 潜在语义分析思想

潜在语义分析先构建一个单词-文档（term-document）矩阵$\boldsymbol{A}$，进

而寻找该矩阵的低秩逼近（low rank approximation）[16]，来挖掘单词-单词、单词-文档以及文档-文档之间的关联关系。

下面以9篇文章来解释LSA的方法。为了使得表达简洁，仅选取每篇文章的题目。

a1: Efficient Algorithms for Non-convex Isotonic Regression through Submodular Optimization

a2: Combinatorial Optimization with Graph Convolutional Networks and Guided Tree Search

a3: An Improved Analysis of Alternating Minimization for Structured Multi-Response Regression

a4: Analysis of Krylov Subspace Solutions of Regularized Non-Convex Quadratic Problems

a5: Post: Device Placement with Cross-Entropy Minimization and Proximal Policy Optimization

b1: CRISPR/Cas9 and TALENs generate heritable mutations for genes involved in small RNA processing of Glycine max and Medicago truncatula

b2: Generation of D1-1 TALEN isogenic control cell line from Dravet syndrome patient iPSCs using TALEN-mediated editing of the SCN1A gene

b3: Genome-Scale CRISPR Screening Identifies Novel Human Pluripotent Gene Networks

b4: Champions: A phase 1/2 clinical trial with dose escalation of SB-913 ZFN-mediated in vivo human genome editing for treatment of MPS II (Hunter syndrome)

前5篇论文内容属于机器学习（machine learning）类别、后4篇论文内容属于基因编辑（gene editing）类别。从这9篇论文标题中，筛选有实际意义且至少出现在两篇文章标题中的10个单词，分别是nonconvex, regression, optimization, network, analysis, minimization, gene, syndrome, editing, human。这样，10个单词和9篇文章就可以形成一个10×9大小的单词-文档矩阵$\boldsymbol{A}$（见表5.2）

表5.2 单词-文档（term-document）矩阵

	a1	a2	a3	a4	a5	b1	b2	b3	b4
nonconvex	1	0	0	1	0	0	0	0	0
regression	1	0	1	0	0	0	0	0	0
optimization	1	1	0	0	1	0	0	0	0
network	0	1	0	0	0	0	0	1	0
analysis	0	0	1	1	0	0	0	0	0
minimization	0	0	1	0	1	0	0	0	0
gene	0	0	0	0	0	1	1	1	0
syndrome	0	0	0	0	0	0	1	0	1
editing	0	0	0	0	0	0	1	0	1
human	0	0	0	0	0	0	0	1	1

单词-文档矩阵中每一行表示某个单词在不同文档标题中所出现次数，如单词regression分别在文档a1和文档a3的标题中各出现了一次，那么这两处相应位置值为1。

假如仅仅基于单词-文档矩阵进行文档检索，如当用户输入“optimization”这一检索请求，检索与“optimization”相关的文档。由于文档a3标题中不包含这一单词，则文档a3被认为是不相关文档，但实际上文档a3所涉及“minimization”内容与优化问题相关。出现这一问题是因为单词-文档矩阵只是刻画了单词是否在文档中出现与否这一现象，而无法对单词-单词、单词-文档以及文档-文档之间的语义关系进行建模。另外，如果用户检索“eat an apple”，则文档“Apple is a great company”会被检索出来，而实际上该文档中单词“Apple”所指苹果公司，而非水果，造成这一结果的原因是一些单词具有“一词多义”。因此需要一种方法能够建模单词-单词、单词-文档以及文档-文档之间语义关系，解决包括“异词同义”和“一词多义”在内的诸多挑战。

隐性语义分析就是这样的一种技术，它通过对由大规模文本语料库构建的单词-文档矩阵进行分解，来挖掘隐藏在海量文本中的上下文关联关系。通常隐性语义分析会对单词-文档矩阵进行奇异值分解，图5.8为对表5.2所得到的单词-文档矩阵进行奇异值分解的示意（假设单词总数为M，文档数为N）。

可以看到这个分解示意图和在特征人脸中介绍的SVD分解有一些

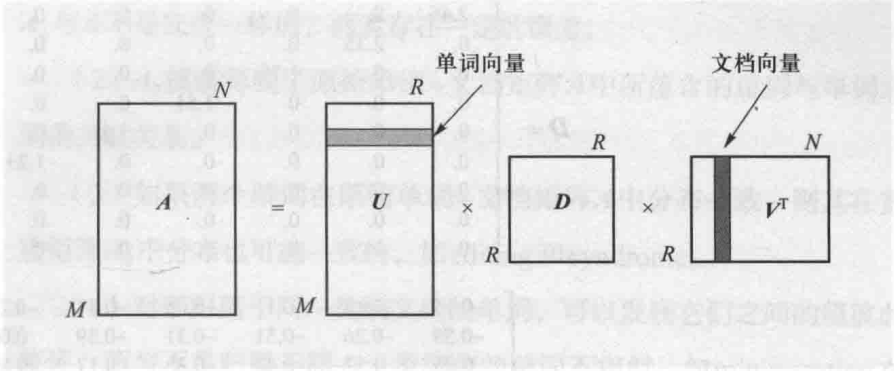


图5.8 单词－文档矩阵的奇异值分解示意

差异，这是因为这里在原SVD分解结果的基础上对每个矩阵都选取一部分，其中R是被分解矩阵$\boldsymbol{A}$的秩，每个矩阵在隐性语义分析场景下的具体含义如下：

矩阵$\boldsymbol{D}$是一个$R\times R$的对角矩阵，其对角线上值按照从大到小进行排序。

$$\boldsymbol{D}=\mathrm{diag}(\sigma_1,\cdots,\sigma_R),\sigma_1\geqslant\sigma_2\geqslant\cdots\geqslant\sigma_k\geqslant\cdots\geqslant\sigma_R$$

矩阵$\boldsymbol{U}$是一个$M\times R$的矩阵，$\boldsymbol{U}$中的每一个行向量被称为LSI单词向量（LSI term vectors）。每个M维大小的文档可以被单词向量嵌入到R维空间。

$\boldsymbol{V}$是一个$N\times R$的矩阵，$\boldsymbol{V}^{\mathrm{T}}$是一个$R\times N$的矩阵，$\boldsymbol{V}$中的每一个行向量被称为LSI文档向量（LSI document vectors）。

可以认为，原始文档和单词内嵌在了一个隐性空间中，这个隐性空间中单词向量和文档向量的表示蕴含了单词和单词之间、文档和文档之间的关联关系，正交矩阵$\boldsymbol{U}$、正交矩阵$\boldsymbol{V}$和对角矩阵$\boldsymbol{D}$联合起来将原始单词－文档矩阵表示成了线性无关的向量或因素值（factor values）。和特征人脸中类似，如果取前k个奇异值（这里的k即前述的R），以及它们对应的维度为k的单词向量和文档向量，就可以重建一个维度为$M\times N$的矩阵$\boldsymbol{A}_k$，$\boldsymbol{A}_k=\sum_{i=1}^{k}u_i\sigma_i v_i^{\mathrm{T}}$。很显然，重建矩阵$\boldsymbol{A}_k$与原始矩阵$\boldsymbol{A}$不完全相同，是对原始矩阵$\boldsymbol{A}$的逼近。

对表5.2中的单词－文档矩阵进行奇异值分解，得到的结果如下。

$$\boldsymbol{U}=\begin{bmatrix}
-0.09 & -0.38 & -0.14 & -0.04 & 0.71 & -0.02 & 0.12 & 0.33 & 0.45 & -0. \\
-0.1 & -0.47 & -0.2 & -0.06 & 0. & -0.06 & -0.76 & -0.38 & -0.01 & 0. \\
-0.18 & -0.53 & 0.44 & 0.51 & 0. & -0.14 & 0.16 & 0.09 & -0.42 & -0. \\
-0.28 & -0.08 & 0.57 & -0.23 & -0. & 0.32 & 0.17 & -0.5 & 0.38 & 0. \\
-0.07 & -0.35 & -0.4 & -0.46 & -0. & 0.22 & 0.46 & -0.19 & -0.45 & -0. \\
-0.09 & -0.38 & -0.14 & -0.04 & -0.71 & -0.02 & 0.12 & 0.33 & 0.45 & -0. \\
-0.52 & 0.13 & 0.16 & -0.47 & 0. & -0.65 & -0.04 & 0.16 & -0.11 & -0. \\
-0.45 & 0.16 & -0.32 & 0.34 & -0. & 0.01 & 0.11 & -0.19 & 0.08 & 0.71 \\
-0.45 & 0.16 & -0.32 & 0.34 & -0. & 0.01 & 0.11 & -0.19 & 0.08 & -0.71 \\
-0.42 & 0.1 & 0.09 & -0.11 & -0. & 0.63 & -0.32 & 0.49 & -0.22 & -0.
\end{bmatrix}$$

$$D=\begin{bmatrix}
2.46 & 0. & 0. & 0. & 0. & 0. & 0. & 0. & 0. \\
0. & 2.35 & 0. & 0. & 0. & 0. & 0. & 0. & 0. \\
0. & 0. & 1.79 & 0. & 0. & 0. & 0. & 0. & 0. \\
0. & 0. & 0. & 1.51 & 0. & 0. & 0. & 0. & 0. \\
0. & 0. & 0. & 0. & 1.41 & 0. & 0. & 0. & 0. \\
0. & 0. & 0. & 0. & 0. & 1.23 & 0. & 0. & 0. \\
0. & 0. & 0. & 0. & 0. & 0. & 0.93 & 0. & 0. \\
0. & 0. & 0. & 0. & 0. & 0. & 0. & 0.73 & 0. \\
0. & 0. & 0. & 0. & 0. & 0. & 0. & 0. & 0.15
\end{bmatrix}$$

$$V^{T}=\begin{bmatrix}
-0.15 & -0.18 & -0.1 & -0.06 & -0.11 & -0.21 & -0.58 & -0.49 & -0.54 \\
-0.59 & -0.26 & -0.51 & -0.31 & -0.39 & 0.06 & 0.19 & 0.07 & 0.18 \\
0.05 & 0.57 & -0.42 & -0.3 & 0.17 & 0.09 & -0.26 & 0.46 & -0.3 \\
0.27 & 0.18 & -0.37 & -0.33 & 0.32 & -0.31 & 0.13 & -0.54 & 0.38 \\
0.5 & 0. & -0.5 & 0.5 & -0.5 & 0. & 0. & 0. & -0. \\
-0.19 & 0.15 & 0.11 & 0.16 & -0.14 & -0.53 & -0.52 & 0.25 & 0.53 \\
-0.51 & 0.36 & -0.19 & 0.62 & 0.3 & -0.04 & 0.21 & -0.2 & -0.1 \\
0.05 & -0.56 & -0.33 & 0.19 & 0.58 & 0.21 & -0.3 & 0.2 & 0.16 \\
0.07 & -0.27 & -0.07 & 0. & 0.14 & -0.72 & 0.38 & 0.33 & -0.36
\end{bmatrix}$$

当$k=2$，即选取最大的前两个特征根及其对应的特征向量对矩阵A进行重建。下面给出了选取矩阵U、矩阵D和矩阵V的子部分重建所得矩阵A_2。

$$A_2=\begin{bmatrix}
 & a1 & a2 & a3 & a4 & a5 & b1 & b2 & b3 & b4 \\
nonconvex & 0.56 & 0.27 & 0.49 & 0.3 & 0.37 & -0.01 & -0.05 & 0.05 & -0.05 \\
regression & 0.68 & 0.33 & 0.59 & 0.36 & 0.45 & -0.01 & -0.06 & 0.05 & -0.06 \\
optimization & 0.79 & 0.4 & 0.68 & 0.41 & 0.53 & 0.02 & 0.02 & 0.14 & 0.02 \\
network & 0.22 & 0.18 & 0.17 & 0.1 & 0.15 & 0.13 & 0.36 & 0.32 & 0.33 \\
analysis & 0.51 & 0.25 & 0.44 & 0.27 & 0.34 & -0.01 & -0.06 & 0.03 & -0.06 \\
minimization & 0.56 & 0.27 & 0.49 & 0.3 & 0.37 & -0.01 & -0.05 & 0.05 & -0.05 \\
gene & 0.01 & 0.16 & -0.03 & -0.02 & 0.02 & 0.29 & 0.81 & 0.66 & 0.75 \\
syndrome & -0.05 & 0.11 & -0.07 & -0.05 & -0.02 & 0.26 & 0.72 & 0.58 & 0.67 \\
editing & -0.05 & 0.11 & -0.07 & -0.05 & -0.02 & 0.26 & 0.72 & 0.58 & 0.67 \\
human & 0.01 & 0.13 & -0.02 & -0.01 & 0.02 & 0.23 & 0.65 & 0.53 & 0.6
\end{bmatrix}$$

回到之前举的一个例子，用户输入“optimization”来检索与之相关的文档。尽管单词“optimization”在文档a3中没有出现，但是在重建矩阵A_2中，对应的位置被0.68取代，说明单词“optimization”对表征文档a3所蕴含内容具有重要作用，这也符合文档a3描述的minimization问题是一个optimization问题的事实。

在单词-矩阵A中，文档b3所对应network、gene和human三个单词取值为1，在重建矩阵A_2中，network、gene和human三个单词取值分别为0.32、0.66和0.53。可见，network在表征文档b3时重要性降低，因为算法认为这一单词在机器学习相关文档的表达中更具有区别性。

对比总结重建矩阵和原始单词-文档矩阵的差异，可以发现下面的现象。

（1）由于A_2是从最大两个特征根及其对应特征向量重建得到，因此

A_2与A不是完全一样的，两者存在一定的误差。

（2）A_2捕获得到了原始单词-文档矩阵A中所蕴含的单词与单词之间的关联关系。

（3）如果两个单词在原始单词-文档矩阵A中分布一致，则其在重建矩阵A_2中分布也可能一致的，如editing和syndrome。

（4）对于归属于同一类别文档的单词，可以发现它们之间的值彼此接近，而与不是归属于同一个类别中的单词不相似，如minimization在机器学习类别文档中均为正数，而在基因编辑类别文档中几乎为负数。

5.4.2 潜在语义分析例子

皮尔逊相关系数可被用来刻画两个变量之间的线性相关程度，其值域取值为[-1, 1]。皮尔逊相关系数为正表示两个变量正线性相关，取值越大，则这两个变量在线性相关的意义下相关程度越大；皮尔逊相关系数为负则表示这两个变量存在负线性相关。负线性相关意味着一个变量减少时，另外一个变量会增加。两个变量之间的皮尔逊相关系数为0表示两者不存在线性相关关系（可能存在其他非线性相关的关系）。

基于单词-文档矩阵A，可以计算任意两个文档之间的皮尔逊相关系数，从而得到如下文档-文档相关系数矩阵。

	a1	a2	a3	a4	a5	b1	b2	b3	b4
a1	1.	0.22	0.05	0.22	0.22	-0.22	-0.43	-0.43	-0.43
a2	0.22	1.	-0.33	-0.25	0.37	-0.17	-0.33	0.22	-0.33
a3	0.05	-0.33	1.	0.22	0.22	-0.22	-0.43	-0.43	-0.43
a4	0.22	-0.25	0.22	1.	-0.25	-0.17	-0.33	-0.33	-0.33
a5	0.22	0.37	0.22	-0.25	1.	-0.17	-0.33	-0.33	-0.33
b1	-0.22	-0.17	-0.22	-0.17	-0.17	1.	0.51	0.51	-0.22
b2	-0.43	-0.33	-0.43	-0.33	-0.33	0.51	1.	0.05	0.52
b3	-0.43	0.22	-0.43	-0.33	-0.33	0.51	0.05	1.	0.05
b4	-0.43	-0.33	-0.43	-0.33	-0.33	-0.22	0.52	0.05	1.

基于重建单词-文档矩阵A_2，可以计算任意两个文档之间的皮尔逊相关系数，得到如下更新后的文档-文档相关系数矩阵。

	a1	a2	a3	a4	a5	b1	b2	b3	b4
a1	1.	0.97	1.	1.	1.	-0.95	-0.95	-0.93	-0.95
a2	0.97	1.	0.97	0.97	0.98	-0.85	-0.86	-0.83	-0.86
a3	1.	0.97	1.	1.	1.	-0.95	-0.96	-0.94	-0.96
a4	1.	0.97	1.	1.	1.	-0.95	-0.96	-0.94	-0.96
a5	1.	0.98	1.	1.	1.	-0.94	-0.95	-0.93	-0.95
b1	-0.95	-0.85	-0.95	-0.95	-0.94	1.	1.	1.	1.
b2	-0.95	-0.86	-0.96	-0.96	-0.95	1.	1.	1.	1.
b3	-0.93	-0.83	-0.94	-0.94	-0.93	1.	1.	1.	1.
b4	-0.95	-0.86	-0.96	-0.96	-0.95	1.	1.	1.	1.

在基于单词-文档矩阵A所构建的文档-文档相互关系矩阵中，同一类别文档之间的相关性系数普遍很低，如a1-a3，a2-a3，a2-a4等。例如，机器学习类别所包含五篇文档的平均相关性仅为0.069，基因编辑类别所包含四篇文档的平均相关性为0.237。

在基于重建单词-文档矩阵A_2所构建的文档-文档相关系数矩阵中，由于经过隐性语义分析，可以看到相关性矩阵中的数值已经可以反映不同文档所蕴含的语义关系。机器学习类别所包含五篇文档的平均相关性上升到0.989，基因编辑类别所包含四篇文档的平均相关性上升到1.00。机器学习类别所包含文档与基因编辑类别所包含文档之间的平均相关性为-0.927。

同样的，可以分析单词-单词在重建前后的语义关系。

基于单词-文档矩阵A，可以计算任意两个单词之间的皮尔逊相关系数，从而得到如下单词-单词相关系数矩阵。

	nonconvex	regression	optimization	network	analysis	minimization	gene	syndrome	editing	human
nonconvex	1.	0.36	0.19	-0.29	0.36	-0.29	-0.38	-0.29	-0.29	-0.29
regression	0.36	1.	0.19	-0.29	0.36	0.36	-0.38	-0.29	-0.29	-0.29
optimization	0.19	0.19	1.	0.19	-0.38	0.19	-0.5	-0.38	-0.38	-0.38
network	-0.29	-0.29	0.19	1.	-0.29	-0.29	0.19	-0.29	-0.29	0.36
analysis	0.36	0.36	-0.38	-0.29	1.	0.36	-0.38	-0.29	-0.29	-0.29
minimization	-0.29	0.36	0.19	-0.29	0.36	1.	-0.38	-0.29	-0.29	-0.29
gene	-0.38	-0.38	-0.5	0.19	-0.38	-0.38	1.	0.19	0.19	0.19
syndrome	-0.29	-0.29	-0.38	-0.29	-0.29	-0.29	0.19	1.	1.	0.36
editing	-0.29	-0.29	-0.38	-0.29	-0.29	-0.29	0.19	1.	1.	0.36
human	-0.29	-0.29	-0.38	0.36	-0.29	-0.29	0.19	0.36	0.36	1.

基于重建单词-文档矩阵A_2，可以计算任意两个单词之间的皮尔森相关系数，从而得到如下更新后的单词-单词相关系数矩阵。

	nonconvex	regression	optimization	network	analysis	minimization	gene	syndrome	editing	human
nonconvex	1.	1.	1.	-0.57	1.	1.	-0.88	-0.89	-0.89	-0.87
regression	1.	1.	1.	-0.57	1.	1.	-0.88	-0.89	-0.89	-0.88
optimization	1.	1.	1.	-0.51	1.	1.	-0.84	-0.86	-0.86	-0.84
network	-0.57	-0.57	-0.51	1.	-0.58	-0.57	0.89	0.88	0.88	0.9
analysis	1.	1.	1.	-0.58	1.	1.	-0.88	-0.9	-0.9	-0.88
minimization	1.	1.	1.	-0.57	1.	1.	-0.88	-0.89	-0.89	-0.87
gene	-0.88	-0.88	-0.84	0.89	-0.88	-0.88	1.	1.	1.	1.
syndrome	-0.89	-0.89	-0.86	0.88	-0.9	-0.89	1.	1.	1.	1.
editing	-0.89	-0.89	-0.86	0.88	-0.9	-0.89	1.	1.	1.	1.
human	-0.87	-0.88	-0.84	0.9	-0.88	-0.87	1.	1.	1.	1.

通过观察两个相关系数矩阵可以发现，基于重建单词-文档矩阵A_2得到的相关系数矩阵中单词-单词之间的相关关系更加明确，在同一文档中出现过的单词之间相关系数趋近为1，没有同时出现在同一文档中

的单词之间相关系数趋近为-1。单词network相比于其他单词更加“中立”一些，这是因为单词network既出现在了机器学习这一类别文档标题a2中，也出现在基因编辑这一类别文档b3中。不过单词network与基因编辑这一类别单词相关度更高，这是因为单词network与gene和human这两个具有明显基因编辑特点的单词同时出现在文档b3中，只与optimization这一明显具有机器学习特点的单词同时出现在文档a2中。

可以看出，隐性语义分析对原始单词-文档矩阵中所蕴含关系进行了有效挖掘，能刻画单词-单词、单词-文档以及文档-文档之间的语义关系。

5.5 期望最大化算法

假设由n个数据样本构成的集合$\mathcal{D}=\{x_1, x_2, \cdots, x_n\}$从参数为$\Theta$的某个模型（如高斯模型等）以一定概率独立采样得到。于是，可以通过最大似然估计算法（maximum likelihood estimation，MLE）来求取参数Θ，使得在参数为Θ的模型下数据集$\mathcal{D}$出现的可能性最大，即$\hat{\Theta}=\arg\max_{\Theta} P(\mathcal{D}|\Theta)$。

或者也可利用最大后验估计（maximum a posteriori estimation，MAP）从数据集$\mathcal{D}$来如下估计参数Θ：$\hat{\Theta}=\arg\max_{\Theta} P(\Theta|\mathcal{D})=\arg\max_{\Theta}\frac{P(\mathcal{D}|\Theta)P(\Theta)}{P(\mathcal{D})}$。由于$P(\mathcal{D})$与$\Theta$无关，则可得$\arg\max_{\Theta}\frac{P(\mathcal{D}|\Theta)P(\Theta)}{P(\mathcal{D})}=\arg\max_{\theta} P(\mathcal{D}|\Theta)P(\Theta)$，对这个式子取对数，得到$\arg\max_{\Theta}\log P(\mathcal{D}|\Theta)+\log P(\Theta)$。可见，最大后验估计与最大似然估计相比，增加了一项与Θ相关的先验概率$P(\Theta)$。

当然，无论是最大似然估计算法或者是最大后验估计算法，都是充分利用已有数据，在参数模型确定（只是参数值未知）情况下，对所优化目标中的参数求导，令导数为0，求取模型的参数值。

但是，在解决一些具体问题时，难以事先就将模型确定下来，然后利用数据来求取模型中的参数值。在这种情况下，无法直接利用最大似然估计算法或者最大后验估计算法来求取模型参数。

本节介绍Dempster，Laird和Rubin提出的期望最大化（expectation

maximization，EM）算法[7]来解决这个问题。

EM算法是一种重要的用于解决含有隐变量（latent variable）问题的参数估计方法。EM算法分为求取期望（E步骤，expectation）和期望最大化（M步骤，maximization）两个步骤。在EM算法的E步骤时，先假设模型参数的初始值，估计隐变量取值；在EM算法的M步骤时，基于观测数据、模型参数和隐变量取值一起来最大化“拟合”数据，更新模型参数。基于所更新的模型参数，得到新的隐变量取值（EM算法的E步骤），然后继续极大化“拟合”数据，更新模型参数（EM算法的M步骤）。以此类推迭代，直到算法收敛，得到合适的模型参数。

可以证明，E步骤和M步骤的迭代可不断优化模型参数，使之更拟合数据。EM算法不能保证找到全局的最大值，有可能找到一个鞍点（saddle point）[20]。EM算法在统计中具有许多应用，如机器学习、数据挖掘以及贝叶斯统计，特别在混合模型的参数估计中应用广泛[21, 22, 23]。在介绍EM算法的具体内容之前，先看如下两个例子。

5.5.1 二硬币投掷例子

下面介绍两个硬币投掷的例子[1]。假设有A和B两个硬币，进行5轮掷币实验：在每一轮实验中，先随机选择一个硬币，然后用所选择的硬币投掷10次，将投掷结果作为本轮实验观测结果，如表5.3所示。H代表硬币正面朝上，T代表硬币反面朝上。

表5.3 两个硬币投掷5轮（每轮10次）的结果

1	H	T	T	T	H	H	T	H	T	H
2	H	H	H	H	T	H	H	H	H	H
3	H	T	H	H	H	H	H	T	H	H
4	H	T	H	T	T	T	H	H	T	T
5	T	H	H	H	T	H	H	H	T	H

在每一轮实验中，并不知道完成本轮投掷所选择的硬币是A还是B，因此也就不知道每一轮中10次投掷所观测到的硬币正面（或者硬币反面）是由A投掷出来还是由B投掷出来。但是，现在需要从这10轮观测数据出发，计算硬币A或硬币B被投掷为正面的概率。记硬币A或硬币B被投掷为正面的概率为$\Theta=\{\Theta_A, \Theta_B\}$。$\Theta$为投掷两个硬币这个模型的参

数，所求取得到的Θ值应该以最大概率（极大似然）“拟合”表5.3观测结果。

这里要注意的是，表5.3中所列观测结果是不完全数据，即从观测结果只能够知道硬币正面或反面，但是并不知道这个正面或反面来自哪一个硬币，因此每一轮中所选择投掷的硬币是一个“隐变量”。由于这个问题中存在隐变量，因此下面采用EM算法来计算硬币A或硬币B被投掷为正面的概率。

1. 求取期望（E步骤，Expectation）

初始化每一轮中硬币A和硬币B投掷为正面的概率为$\hat{\Theta}_{\mathrm{A}}^{(0)}=0.60$和$\hat{\Theta}_{\mathrm{B}}^{(0)}=0.50$。下面根据这个初始化概率，结合5轮实验结果来更新硬币A和硬币B投掷为正面的概率。

在第一轮试验中，所观测得到的投掷结果为“HTTTHHTHTH”，这10次投掷结果由硬币A投掷所得概率为：

$$
\begin{aligned}
&P\left(\text{选择硬币A投掷}\middle|\text{硬币投掷结果},\Theta\right)\\
&=\frac{P\left(\text{选择硬币A投掷},\text{硬币投掷结果}\middle|\Theta\right)}{P\left(\text{选择硬币A投掷},\text{硬币投掷结果}\middle|\Theta\right)+P\left(\text{选择硬币B投掷},\text{硬币投掷结果}\middle|\Theta\right)}\\
&=\frac{(0.6)^5\times(0.4)^5}{(0.6)^5\times(0.4)^5+(0.5)^{10}}=0.45
\end{aligned}
$$

这10次结果由硬币B投掷所得概率为：

$$
\begin{aligned}
&P\left(\text{选择硬币B投掷}\middle|\text{硬币投掷结果},\Theta\right)\\
&=1-P\left(\text{选择硬币A投掷}\middle|\text{硬币投掷结果},\Theta\right)\\
&=0.55
\end{aligned}
$$

一旦得到了在第一轮试验中选择硬币A或硬币B投掷的概率，则可以根据这个概率来计算第一轮中硬币A和硬币B投掷正面的期望次数。例如，硬币A为正面期望次数为$N_{\text{正面总次数}}\times P_{\text{选硬币A概率}}=5\times0.45=2.25$，其他项计算类似。

同理，可以计算得到其他4轮试验中硬币A和硬币B投掷正面的次数，以及5轮试验中硬币A或硬币B投掷为正面的总次数（见表5.4）。

2. 期望最大化（M步骤，Maximization）

在上面的计算中，通过初始化硬币A和硬币B投掷得到正面概率$\hat{\Theta}_{\mathrm{A}}^{(0)}$和$\hat{\Theta}_{\mathrm{B}}^{(0)}$，得到每一轮中选择硬币A和选择硬币B概率这一“隐变量”，进而可计算得到每一轮中硬币A和硬币B投掷正面次数。

表5.4 两个硬币在每一轮中被选择概率以及投掷正面/反面的次数

轮次	选硬币A概率	选硬币B概率	硬币A为正面期望次数	硬币A为反面期望次数	硬币B为正面期望次数	硬币B为反面期望次数
1	0.45	0.55	2.25	2.25	2.75	2.75
2	0.80	0.20	7.24	0.80	1.76	0.20
3	0.73	0.27	5.87	1.47	2.13	0.53
4	0.35	0.65	1.41	2.11	2.59	3.89
5	0.65	0.35	4.53	1.94	2.47	1.07
合计			21.30	8.57	11.70	8.43

在这些信息基础上，可更新得到硬币A和硬币B投掷为正面的概率，从而得到新的模型参数：

$$\hat{\Theta}_{\mathrm{A}}^{(1)}=\frac{21.30}{21.30+8.57}=0.713 \quad \hat{\Theta}_{\mathrm{B}}^{(1)}=\frac{11.70}{11.70+8.43}=0.581$$

接下来，可在新的概率值基础上继续计算每一轮投掷中选择硬币A或硬币B的概率，进而计算得到5轮中硬币A和硬币B投掷正面的总次数，从而得到硬币A和硬币B投掷为正面的更新概率值$\hat{\Theta}_{\mathrm{A}}^{(2)}$和$\hat{\Theta}_{\mathrm{B}}^{(2)}$。上述算法不断迭代，直至算法收敛，最终得到硬币A和硬币B投掷为正面的概率$\Theta=\{\Theta_{\mathrm{A}},\Theta_{\mathrm{B}}\}$。

表5.5列出了10次迭代过程中硬币A和硬币B投掷为正面的概率。在第10次迭代时，算法收敛，得到Θ_{A}和Θ_{B}的值分别为0.797和0.520。

表5.5 两个硬币投掷结果为正面的概率迭代计算结果

迭代次数	硬币A为正面次数	硬币A为反面次数	硬币B为正面次数	硬币B为反面次数	硬币A投掷正面概率θ_{A}	硬币B投掷正面概率θ_{B}
1	21.30	8.57	11.70	8.43	$\hat{\theta}_{\mathrm{A}}^{(1)}=0.713$	$\hat{\theta}_{\mathrm{B}}^{(1)}=0.581$
2	19.21	6.56	13.79	10.44	$\hat{\theta}_{\mathrm{A}}^{(2)}=0.745$	$\hat{\theta}_{\mathrm{B}}^{(2)}=0.569$
3	19.41	5.86	13.59	11.14	$\hat{\theta}_{\mathrm{A}}^{(3)}=0.768$	$\hat{\theta}_{\mathrm{B}}^{(3)}=0.550$
4	19.75	5.47	13.25	11.53	$\hat{\theta}_{\mathrm{A}}^{(4)}=0.783$	$\hat{\theta}_{\mathrm{B}}^{(4)}=0.535$
5	19.98	5.28	13.02	11.72	$\hat{\theta}_{\mathrm{A}}^{(5)}=0.791$	$\hat{\theta}_{\mathrm{B}}^{(5)}=0.526$
6	20.09	5.19	12.91	11.81	$\hat{\theta}_{\mathrm{A}}^{(6)}=0.795$	$\hat{\theta}_{\mathrm{B}}^{(6)}=0.522$
7	20.14	5.16	12.86	11.84	$\hat{\theta}_{\mathrm{A}}^{(7)}=0.796$	$\hat{\theta}_{\mathrm{B}}^{(7)}=0.521$
8	20.16	5.15	12.84	11.85	$\hat{\theta}_{\mathrm{A}}^{(8)}=0.796$	$\hat{\theta}_{\mathrm{B}}^{(8)}=0.520$
9	20.17	5.15	12.83	11.85	$\hat{\theta}_{\mathrm{A}}^{(9)}=0.797$	$\hat{\theta}_{\mathrm{B}}^{(9)}=0.520$
10	20.18	5.15	12.82	11.85	$\hat{\theta}_{\mathrm{A}}^{(10)}=0.797$	$\hat{\theta}_{\mathrm{B}}^{(10)}=0.520$

在上面这个过程中，每一轮选择硬币A还是选择硬币B来完成10次投掷是一个隐变量，硬币A和硬币B投掷结果为正面的概率$\Theta=\{\Theta_A, \Theta_B\}$称为模型参数。

EM算法使用迭代方法来求解模型参数$\Theta=\{\Theta_A, \Theta_B\}$：先初始化模型参数，然后计算得到隐变量（EM算法的E步骤），接着基于观测投掷结果和当前隐变量值一起来最大化似然函数（即使得模型参数能够更好拟合观测结果），更新模型参数（EM算法的M步骤）。基于当前得到的模型参数，继续更新隐变量（EM算法的E步骤），然后继续最大化似然函数，更新模型参数（EM算法的M步骤）。以此类推，不断迭代下去，直到模型参数基本无变化，算法收敛。

5.5.2 三硬币投掷例子

假设有三枚质地材料不均匀的硬币（即每枚硬币投掷出现正反两面的概率不一定相等），这三枚硬币分别标记为0，1，2。约定出现正面记为H（head，头），出现反面记为T（tail，尾），一次试验的过程如下：首先掷硬币0，如果硬币0投掷结果为H，则选择硬币1投掷三次，如果硬币0投掷结果为T，则选择硬币2投掷三次。观测结果中仅记录硬币1和硬币2的投掷结果，不出现硬币0的投掷结果。因为硬币0的投掷结果没有被记录，所以是未观测到的数据（隐变量）。

未观测数据取值集合记为$C_Z=\{\mathrm{H}, \mathrm{T}\}$，观测数据取值集合记为$C_X=\{\mathrm{HHH}, \mathrm{TTT}, \mathrm{HTT}, \mathrm{THH}, \mathrm{HHT}, \mathrm{TTH}, \mathrm{HTH}, \mathrm{THT}\}$，模型参数集合（三枚硬币分别出现正面的概率）为$\Theta=\{\lambda, p_1, p_2\}$。在$n$次试验中，未观测到的数据序列记为$z$，观测到的数据序列记为$x$，观测到的数据序列中有$h$次为正面朝上，$t$次为反面朝上。易知有下面的式子成立：

$$P(x, z|\Theta)=P(z|\Theta)P(x|z,\Theta)$$

其中

$$P(z|\Theta)=\begin{cases}\lambda & \text{当 } z=\mathrm{H}\\ 1-\lambda & \text{当 } z=\mathrm{T}\end{cases}$$

$$P(x|z,\Theta)=\begin{cases}p_1^h(1-p_1)^t \text{当 } z=\mathrm{H}\\ p_2^h(1-p_2)^t \text{当 } z=\mathrm{T}\end{cases}$$

在硬币0掷出正面后，选择硬币1投掷三次所得“反正反”这一结果的概率如下计算：

$$P(x=\text{THT}, z=\text{H}|\Theta)=\lambda p_1(1-p_1)^2$$

在硬币0掷出反面后，选择硬币2投掷三次所得“反正反”这一结果的概率如下计算：

$$P(x=\text{THT}, z=\text{T}|\Theta)=(1-\lambda)p_2(1-p_2)^2$$

如果某次观测得到“反正反”这一投掷结果，则该投掷结果发生的概率如下计算：

$$\begin{aligned}P(x=\text{THT}|\Theta)&=P(x=\text{THT}, z=\text{H}|\Theta)+P(x=\text{THT}, z=\text{T}|\Theta)\\&=\lambda p_1(1-p_1)^2+(1-\lambda)p_2(1-p_2)^2\end{aligned}$$

如果某次观测得到“反正反”这一投掷结果，这一结果是由硬币0投掷为正面（未观测的数据）所促发的概率计算如下：

$$\begin{aligned}P(z=\text{H}|x=\text{THT}, \Theta)&=\frac{P(x=\text{THT}, z=\text{H}|\Theta)}{P(x=\text{THT}|\Theta)}\\&=\frac{\lambda p_1(1-p_1)^2}{\lambda p_1(1-p_1)^2+(1-\lambda)p_2(1-p_2)^2}\end{aligned}$$

这样，可以从观测数据来推测未观测数据的概率分布，即从硬币正面和反面观测结果来推测硬币0投掷为正面或反面这一隐变量。

情况1 为了了解未观测数据对模型参数求解带来的影响，这里先假设一个简单情况，在知道全部的数据（即包括上述的观测数据和未观测数据）情况下，来计算模型参数。

假设所观测的数据如下：(<HHH>, H), (<TTT>, T), (<HHH>, H), (<TTT>, T), (<HHH>, H)。

则可如下来估计参数Θ的取值，其中

$$\hat{\lambda}=\frac{count(\text{硬币0掷出正面})}{count(\text{试验次数})}=\frac{3}{5}$$

$$\widehat{p_1}=\frac{count(\text{硬币1掷出正面})}{count(\text{硬币1掷出次数})}=\frac{9}{3\times 3}=1$$

$$\widehat{p_2}=\frac{count(\text{硬币2掷出正面})}{count(\text{硬币2掷出次数})}=\frac{0}{2\times 3}=0$$

情况2 在硬币0所掷出结果无法观测情况下，假设得到了如下部分不完全观测数据：(<HHH>, _), (<TTT>, _), (<HHH>, _), (<TTT>, _), (<HHH>, _)。

与前面的推导类似，如下计算这一观测结果是由硬币0投掷为正面

或反面所引起的概率（求取期望步骤）。这一过程也就是计算隐变量取值（硬币0所投掷的结果）。

$$P(z=\mathrm{H}|x=\mathrm{HHH},\Theta)=\frac{P(x=\mathrm{HHH},z=\mathrm{H}|\Theta)}{P(x=\mathrm{HHH},z=\mathrm{H}|\Theta)+P(x=\mathrm{HHH},z=\mathrm{T}|\Theta)}$$

$$=\frac{\lambda p_1^3}{\lambda p_1^3+(1-\lambda)p_2^3}$$

$$P(z=\mathrm{T}|x=\mathrm{HHH},\Theta)=1-P(z=\mathrm{H}|x=\mathrm{HHH},\Theta)$$

$$P(z=\mathrm{H}|x=\mathrm{TTT},\Theta)=\frac{P(x=\mathrm{TTT},z=\mathrm{H}|\Theta)}{P(x=\mathrm{TTT},z=\mathrm{H}|\Theta)+P(x=\mathrm{TTT},z=\mathrm{T}|\Theta)}$$

$$=\frac{\lambda(1-p_1)^3}{\lambda(1-p_1)^3+(1-\lambda)(1-p_2)^3}$$

$$P(z=\mathrm{T}|x=\mathrm{TTT},\Theta)=1-P(z=\mathrm{H}|x=\mathrm{TTT},\Theta)$$

在上述对隐变量概率估计基础上，则可估计参数Θ的更新值（期望最大化步骤）：

$$\hat{\lambda}=\frac{3\times P(z=\mathrm{H}|x=\mathrm{HHH},\Theta)+2\times P(z=\mathrm{H}|x=\mathrm{TTT},\Theta)}{5}$$

$$\widehat{p_1}=\frac{3\times 3\times P(z=\mathrm{H}|x=\mathrm{HHH},\Theta)+0\times 2\times P(z=\mathrm{H}|x=\mathrm{TTT},\Theta)}{3\times 3\times P(z=\mathrm{H}|x=\mathrm{HHH},\Theta)+3\times 2\times P(z=\mathrm{H}|x=\mathrm{TTT},\Theta)}$$

$$\widehat{p_2}=\frac{3\times 3\times P(z=\mathrm{T}|x=\mathrm{HHH},\Theta)+0\times 2\times P(z=\mathrm{T}|x=\mathrm{TTT},\Theta)}{3\times 3\times P(z=\mathrm{T}|x=\mathrm{HHH},\Theta)+3\times 2\times P(z=\mathrm{T}|x=\mathrm{TTT},\Theta)}$$

表5.6给出了通过迭代手段不断更新Θ取值的过程。在这个过程中，首先参数Θ被初始化为$\lambda=0.5000$、$p_1=0.4000$和$p_2=0.8000$。

表5.6　三个硬币投掷中参数迭代更新过程

迭代次数	λ	p_1	p_2	$P(z=\mathrm{H}\|x=\mathrm{HHH},\Theta)$	$P(z=\mathrm{H}\|x=\mathrm{TTT},\Theta)$
1	0.5000	0.4000	0.8000	0.1111	0.9643
2	0.4524	0.1474	0.9739	0.0029	1.0000
3	0.4017	0.0043	1.0000	0.0000	1.0000
4	0.4000	0.0000	1.0000	0.0000	1.0000

可以看到，EM算法收敛到了在情况1中全部数据已知下的参数估计值。在最终结果收敛时，$\lambda=0.4000$表示投掷硬币0时有40%概率会正

面朝上。

表5.7给出了改变参数Θ初始化取值时的迭代更新过程。

表5.7　三个硬币投掷中参数初始值被更改后的迭代更新过程

迭代次数	λ	p_1	p_2	$P(z=\mathrm{H}\|x=\mathrm{HHH},\Theta)$	$P(z=\mathrm{H}\|x=\mathrm{TTT},\Theta)$
1	0.5000	0.5100	0.5000	0.5148	0.4849
2	0.5028	0.6143	0.5855	0.5388	0.449
3	0.5029	0.6428	0.5567	0.609	0.346
4	0.5038	0.7253	0.4728	0.7857	0.1255
5	0.5217	0.9037	0.2688	0.9765	0.0025
6	0.5869	0.9983	0.0342	1.0000	0.0000
7	0.6000	1.0000	0.0000	1.0000	0.0000

可见在EM算法中使用不同初始化参数时，EM算法收敛时得到了参数不同的取值结果，这是因为EM算法的收敛结果受到初始值的影响。在具体实践中，初始值的选择相当程度上依赖于对具体问题理解的先验知识（如对三枚硬币重量或外观的一个直观的考量）。

5.5.3　EM算法一般形式

在上面两个例子的基础上，本节介绍EM算法的一般形式。

对于n个相互独立的样本$X=\{x_1, x_2, \cdots, x_n\}$及其对应的隐变量$Z=\{z_1, z_2, \cdots, z_n\}$，在假设样本的模型参数为$\Theta$前提下，观测数据$x_i$的概率为$P(x_i|\Theta)$，完全数据$(x_i, z_i)$的似然函数为$P(x_i, z_i|\Theta)$。

在上面的表示基础上，优化目标为求解合适的Θ和Z使得对数似然函数最大：

$$(\Theta, Z)=\underset{\Theta, Z}{\operatorname{argmax}} L(\Theta, Z)=\underset{\Theta, Z}{\operatorname{argmax}} \sum_{i=1}^{n} \log \sum_{z_i} P(x_i, z_i|\Theta)$$

但是，优化求解含有未观测数据Z的对数似然函数$L(\Theta, Z)$十分困难，EM算法不断构造对数似然函数$L(\Theta, Z)$的一个下界（E步骤），然后最大化这个下界（M步骤），以迭代方式逼近模型参数所能取得极大似然值。

$$\sum_{i=1}^{n} \log \sum_{z_i} P(x_i, z_i|\Theta)=\sum_{i=1}^{n} \log \sum_{z_i} Q_i(z_i) \frac{P(x_i, z_i|\Theta)}{Q_i(z_i)}$$

$$\geqslant \underbrace{\sum_{i=1}^{n} \sum_{z_i} Q_i(z_i) \log \frac{P(x_i, z_i|\Theta)}{Q_i(z_i)}}_{\text{对数似然函数下界}}$$

在上式中，$Q_i(z_i)$是隐变量分布，满足：$\sum_{z_i} Q_i(z_i)=1(0\leqslant Q_i(z_i))$。

上述不等式中使用了Jensen不等式（Jensen's inequality）。对于凹函数f，Jensen不等式使得下面不等式成立：

$$\log\left(\mathrm{E}(f)\right)\geqslant \mathrm{E}\left(\log(f)\right)$$

其中$\mathrm{E}(f)=\sum_{i=1}^{n}\lambda_i f_i, \lambda_i\geqslant 0, \sum_{i=1}^{n}\lambda_i=1$

由于对数函数log的二阶导数$f(x)=-\frac{1}{x^2}<0, x\in\mathbb{R}^+$，因此对数函数是一个凹函数以及可使用Jensen不等式。

令$f_i=\frac{P(x_i,z_i|\Theta)}{Q_i(z_i)}$和$\lambda_i=Q_i(z_i)$，则根据Jensen不等式的定义，可将$\frac{P(x_i,z_i|\Theta)}{Q_i(z_i)}$视为第$i$个样本，$Q_i(z_i)$为第$i$个样本的权重。按照这样的约定，可得到如下式子：

$$E\left(\log\frac{P(x_i,z_i|\Theta)}{Q_i(z_i)}\right)=\sum_{z_i}\underbrace{Q_i(z_i)}_{\text{权重}}\underbrace{\log\frac{P(x_i,z_i|\Theta)}{Q_i(z_i)}}_{\text{样本值}}$$

于是，为了最大化$\sum_{i=1}^{n}\log\sum_{z_i}P(x_i,z_i|\Theta)$这一对数似然函数，只需最大化其下界$\sum_{i=1}^{n}\sum_{z_i}Q_i(z_i)\log\frac{P(x_i,z_i|\Theta)}{Q_i(z_i)}$。这个下界实际上是$\log\frac{P(x_i,z_i|\Theta)}{Q_i(z_i)}$的加权求和。由于权重$Q_i(z_i)$累加之和为1，因此$\sum_{z_i}Q_i(z_i)\log\frac{P(x_i,z_i|\Theta)}{Q_i(z_i)}$就是$\log\frac{P(x_i,z_i|\Theta)}{Q_i(z_i)}$的加权平均，也就是所谓的期望，这就是EM算法中expectation这一单词的来源。于是，EM算法就是不断最大化这一下界（M步骤），从而通过迭代的方式逼近模型参数的极大似然估计值。

显然，当Θ取值给定后，对数似然函数的下界只与$P(x_i, z_i)$和$Q_i(z_i)$相关。于是，通过调整$P(x_i, z_i)$和$Q_i(z_i)$取值，使得似然函数下界不断逼近似然函数真实值。那么，当不等式取等式时，调整后的似然函数下界等于似然函数真实值。当每个样本取值均相等时（也就是每个样本取值为同一个常数），Jensen不等式中的等式成立。于是令$\frac{P(x_i,z_i|\Theta)}{Q_i(z_i)}=c$（$c$为常数），得到$P(x_i,z_i|\Theta)=cQ_i(z_i)$。由于$\sum_{z_i}Q_i(z_i)=1$，可知$\sum_{z_i}P(x_i,z_i|\Theta)=c$。

于是，$Q_i(z_i)=\dfrac{P(x_i,z_i|\Theta)}{c}=\dfrac{P(x_i,z_i|\Theta)}{\sum\limits_{z_i}P(x_i,z_i|\Theta)}=\dfrac{P(x_i,z_i|\Theta)}{P(x_i|\Theta)}=P(z_i|x_i,\Theta)$。

也就是说，只要$Q_i(z_i)=P(z_i|x_i,\Theta)$，就能够保证对数似然函数最大值与其下界相等。

从上面的阐述可知，固定参数Θ后，只要从观测数据x_i和参数Θ出发，令$Q_i(z_i)=P(z_i|x_i,\Theta)$，则可以得到对数似然函数最大值的下界，这就是EM算法中的E步骤。然后，固定$Q_i(z_i)$，调整Θ，再去极大化对数似然函数最大值的下界，这就是EM算法的M步骤。

算法5.3　EM算法

输入：观测所得样本数据X及其对应的隐变量Z（即无法观测数据）、联合分布$P(X, Z|\Theta)$。
输出：模型参数Θ。

算法步骤：

1. 初始化参数取值Θ^0。
2. 求取期望步骤（E步骤）：计算$Q(\Theta|\Theta^t)=\sum\limits_{i=1}^{n}\sum\limits_{z_i}P(z_i|x_i,\Theta)\log P(x_i,z_i|\Theta)$。

   其含义是对数似然函数$\log P(x_i, z_i|\Theta)$在已观测数据X和当前参数Θ^t下去估计隐变量Z的条件概率分布$P(z_i|x_i, \Theta)$。

3. 期望最大化步骤（M步骤）：$\Theta^{t+1}=\underset{\Theta}{\operatorname{argmax}}\,Q(\Theta|\Theta^t)$。
4. 重复第2步和第3步，直到收敛。

在算法5.3中，固定上一步所得参数Θ^t情况下，通过计算Z的后验概率来得到能够逼近真实最大似然估计的下界，与算法中的求取期望步骤一致。在期望最大化步骤中，通过最大化对数似然函数，可以得到新的参数Θ^{t+1}，即：

$$\Theta^{t+1}=\underset{\Theta}{\operatorname{argmax}}\sum_{i=1}^{n}\sum_{z_i}Q_i(z_i)\log\frac{(x_i,z_i|\Theta)}{Q_i(z_i)}$$

因为$Q_i(z_i)=P(z_i|x_i,\Theta)$是由上一步的$\Theta^t$估计出的，与下一步要优化的$\Theta^{t+1}$无关，所以上式等价于：

$$\begin{aligned}\Theta^{t+1}&=\underset{\Theta}{\operatorname{argmax}}\sum_{i=1}^{n}\sum_{z_i}Q_i(z_i)\log p(x_i,z_i|\Theta)-Q_i(z_i)\log Q_i(z_i)\\&=\underset{\Theta}{\operatorname{argmax}}\sum_{i=1}^{n}\sum_{z_i}Q_i(z_i)\log p(x_i,z_i|\Theta)\end{aligned}$$

可见这一结果与算法5.3中的期望最大化步骤一致。广义EM算法中，E步骤是固定参数来优化隐变量分布，M步骤是固定隐变量分布来优化参数，两者不同交替迭代。

至此，证明了EM算法能够通过不断最大化下界来逼近最大似然估计值。由于EM算法是一个迭代算法，那么EM算法是否能够稳定收敛？

假设Θ^{t+1}和Θ^{t}是两个相邻的迭代更新得到的参数值，如果能证明$L(\Theta^{t}) \leqslant L(\Theta^{t+1})$，即可认为EM算法能够让似然单调增长。下面给出简略的证明。

$$L\left(\Theta^{t+1}\right) \geqslant \sum_{i=1}^{n} \sum_{z_i} Q_i\left(z_i\right) \log \frac{p\left(x_i, z_i \middle| \Theta^{t+1}\right)}{Q_i\left(z_i\right)}$$

$$\Rightarrow l\left(\Theta^{t+1}\right) \geqslant \sum_{i=1}^{n} \sum_{z_i} Q_i^{t}\left(z_i\right) \log \frac{p\left(x_i, z_i \middle| \Theta^{t+1}\right)}{Q_i^{t}\left(z_i\right)} \text{(特殊化)}$$

因为$\Theta^{t+1} = \underset{\Theta}{\operatorname{argmax}} \sum_{i=1}^{n} \sum_{z_i} Q_i\left(z_i\right) \log \frac{p\left(x_i, z_i \middle| \Theta\right)}{Q_i\left(z_i\right)}$，即对于$\Theta^{t}$（事实上可以是任意的$\Theta$）都有下面的式子成立：

$$\begin{aligned} \sum_{i=1}^{n} \sum_{z_i} Q_i^{t}\left(z_i\right) \log \frac{p\left(x_i, z_i \middle| \Theta^{t+1}\right)}{Q_i^{t}\left(z_i\right)} &\geqslant \sum_{i=1}^{n} \sum_{z_i} Q_i^{t}\left(z_i\right) \log \frac{p\left(x_i, z_i \middle| \Theta^{t}\right)}{Q_i^{t}\left(z_i\right)} \\ &= L\left(\Theta^{t}\right) \end{aligned}$$

至此，证明了$L\left(\Theta^{t+1}\right) \geqslant L\left(\Theta^{t}\right)$，即EM算法能够保证似然度取值单调增长。

5.6 小结

无监督学习从非标注样本出发来学习数据的分布，这是一个异常困难的工作。由于无法利用标注信息，因此无监督学习只能利用假设数据具有某些结构来进行学习。正如拉普拉斯所言“概率论只不过是把常识用数学公式表达了出来”，无监督学习就是把预设数据具有某种结构作为一种“知识”来指导模型的学习。

习题5

1. 可以从最小化每个类簇的方差这一视角来解释K-均值聚类的结果，下面对这一视角描述不正确的是（　　）。

   A. 最终聚类结果中每个聚类集合中所包含数据呈现出来的差异性最小

   B. 每个样本数据分别归属于与其距离最近的聚类质心所在聚类集合

   C. 每个簇类的方差累加起来最小

   D. 每个簇类的质心累加起来最小

2. 下面对相关性（correlation）和独立性（independence）描述不正确的是（　　）。

   A. 如果两维变量线性不相关，则皮尔逊相关系数等于0

   B. 如果两维变量彼此独立，则皮尔逊相关系数等于0

   C. “不相关”是一个比“独立”要强的概念，即不相关一定相互独立

   D. 独立指两个变量彼此之间不相互影响

3. 下面对主成分分析的描述不正确的是（　　）。

   A. 主成分分析是一种特征降维方法

   B. 主成分分析可保证原始高维样本数据被投影映射后，其方差保持最大

   C. 在主成分分析中，将数据向方差最大方向进行投影，可使得数据蕴含信息没有丢失，以便在后续处理过程中“彰显个性”

   D. 在主成分分析中，所得低维数据中每一维度之间具有极大相关度

4. 下面对特征人脸算法描述不正确的是（　　）。

   A. 特征人脸方法是一种应用主成分分析来实现人脸图像降维的方法

   B. 特征人脸方法是用一种称为“特征人脸（eigenface）”的特征向量按照线性组合形式来表达每一张原始人脸图像

   C. 每一个特征人脸的维数与原始人脸图像的维数一样大

   D. 特征人脸之间的相关度要尽可能大

5. 在潜在语义分析中，给定M个单词和N个文档所构成的单词-文档矩阵（term-document）矩阵，对其进行分解，将单词或文档映射到一个R维的隐性空间。下面描述不正确的是（　　）。

   A. 单词和文档映射到隐性空间后具有相同的维度

   B. 通过矩阵分解可重建原始单词-文档矩阵，所得到的重建矩阵结果比原始单词-文档矩阵更好捕获了单词-单词、单词-文档、文档-文档之间的隐性关系

   C. 这一映射过程中需要利用文档的类别信息

   D. 隐性空间维度的大小由分解过程中所得对角矩阵中对角线上不为零的系数个数所决定

6. 图5.9给出了K-均值聚类算法的初始状态和后续迭代结果。在图5.9左图给出的初始状态中，圆形代表被聚类数据点、

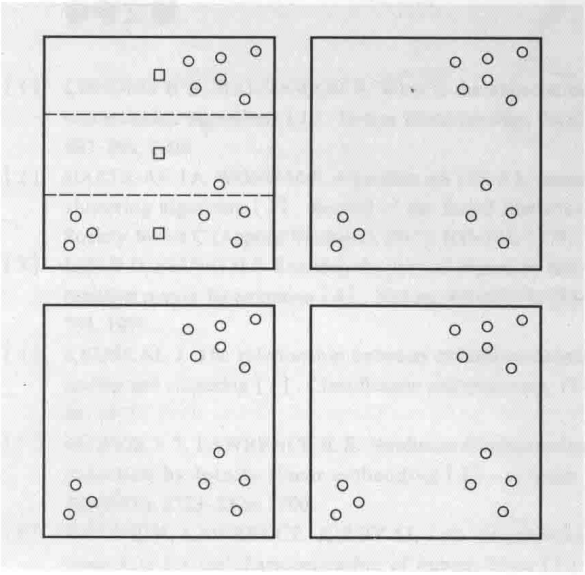


图5.9 K-均值聚类初始状态及后续迭代聚类结果示意图

方形代表聚类质心。给出后续迭代中聚类质心在对应图中的结果。

7. 说明在K-均值聚类算法执行的过程中，其目标函数$\sum_{i=1}^{K}\sum_{x\in G_i}\|x-c_i\|^2$是严格递减的，并解释为什么$K$-均值聚类算法可以确保在有限步内收敛。

8. 结合本章5.5.1小节的例子，根据如下5轮（每轮投掷10次硬币）观测的结果，使用EM算法分别估计硬币A和硬币B被投掷为正面的概率。

1	H	H	T	T	T	T	T	H	T	H
2	T	T	H	T	H	H	H	H	T	H
3	T	T	T	T	T	T	T	T	T	T
4	H	T	H	H	H	H	H	H	H	H
5	H	T	T	T	H	H	T	H	H	T

9. EM算法和K-均值聚类算法有着非常相似的迭代结构，可以说K-均值聚类算法是EM算法的一种特殊实现。如果将K-均值聚类中的聚类质心作为隐变量，试从EM算法角度来解释K-均值聚类算法，即描述K-均值聚类算法的E步骤和M步骤。

10. 在机器学习中，总体是特定研究任务中所定义的全部个体集合，样本是从总体中选择出来的个体子集。一般使用从包含有限个体数目的样本中所得到的信息来作出描述总体的一般结论。样本方差（sample variance）和总体方差是两个不一样的概念。使用样本均值来估计总体均值是无偏的（unbiased），但使用样本方差s^2来估计总体方差σ^2却是有偏的（biased）。总体方差$\sigma^2=\frac{1}{N}\sum_{i=1}^{N}(x_i-\mu)^2$，其中$N$和$\mu$分别是总体样本数量和总体均值。假设样本$x$包含了$n$个数据$\{x_1, x_2, x_3, \cdots, x_n\}$，在了解下面样本均值是总体均值的无偏估

计以及$E\left[\frac{1}{n}\sum_{i=1}^{n}(\bar{x}-x_i)^2\right]=\frac{n-1}{n}\sigma^2$的基础上，证明如果样本方差定义为$s^2=\frac{1}{n-1}\sum_{i=1}^{n}(x_i-\mu)^2$，则$E(s^2)=\sigma^2$。

a）样本均值是对总体均值无偏估计的证明过程

证明：

$$\begin{aligned}E[\bar{x}]&=E\left[\frac{x_1+x_2+\cdots+x_n}{n}\right]\\&=\frac{1}{n}E[x_1+x_2+\cdots+x_n]\\&=\frac{1}{n}(E[x_1]+E[x_2]+\cdots+E[x_n])\\&=\frac{1}{n}(\mu+\mu+\cdots+\mu)\\&=\frac{n\mu}{n}\\&=\mu\end{aligned}$$

可见，样本均值是对总体均值的无偏估计。

b）$E\left[\frac{1}{n}\sum_{i=1}^{n}(\bar{x}-x_i)^2\right]=\frac{n-1}{n}\sigma^2$

证明：

$$\begin{aligned}E\left[\frac{1}{n}\sum_{i=1}^{n}(\bar{x}-x_i)^2\right]&=E\left[\frac{1}{n}\sum_{i=1}^{n}\left((\bar{x}-\mu)-(x_i-\mu)\right)^2\right]\\&=E\left[\frac{1}{n}\sum_{i=1}^{n}\left((\bar{x}-\mu)^2-2(\bar{x}-\mu)(x_i-\mu)+(x_i-\mu)^2\right)\right]\\&=E\left[(\bar{x}-\mu)^2-\frac{2(\bar{x}-\mu)}{n}\sum_{i=1}^{n}(x_i-\mu)+\frac{1}{n}\sum_{i=1}^{n}(x_i-\mu)^2\right]\\&=E\left[(\bar{x}-\mu)^2-2(\bar{x}-\mu)^2+\frac{1}{n}\sum_{i=1}^{n}(x_i-\mu)^2\right]\\&=E\left[\frac{1}{n}\sum_{i=1}^{n}(x_i-\mu)^2\right]-E\left[(\bar{x}-\mu)^2\right]\\&=\sigma^2-\frac{\sigma^2}{n}\\&=\frac{n-1}{n}\sigma^2\end{aligned}$$

在上面的证明过程中，利用了如下的结果：

由于$E(X^2)=[E(X)]^2+Var(X)$、$Var(a+X)=Var(X)$、$Var(aX)=a^2Var(X)$

因此

$$E\left[(\bar{x}-\mu)^2\right]=\left[E(\bar{x}-\mu)\right]^2+Var(\bar{x}-\mu)=Var(\bar{x})=Var\left(\frac{x_1+x_2+\cdots+x_n}{n}\right)$$
$$=\frac{n\sigma^2}{n^2}=\frac{\sigma^2}{n}$$

从$E\left[\frac{1}{n}\sum_{i=1}^{n}(\bar{x}-x_i)^2\right]=\frac{n-1}{n}\sigma^2$可知，$\frac{1}{n}\sum_{i=1}^{n}(\bar{x}-x_i)^2$是对总体方差的一个低估，因此在用样本方差来代替总体方差时，需要将样本方差定义为$\frac{1}{n-1}\sum_{i=1}^{n}(x_i-\mu)^2$，这才能从样本方差出发，得到总体方差$\sigma^2$，这是一个无偏估计。

参考文献

[1] CHUONG B D, BATZOGLOU S. What is the expectation maximization algorithm? [J]. Nature Biotechnology, 26(8): 897–899, 2008.

[2] HARTIGAN J A, WONG M A. Algorithm AS 136: A k–means clustering algorithm [J]. Journal of the Royal Statistical Society. Series C (Applied Statistics), 28(1): 100–108, 1979.

[3] LEE D D, SEUNG H S. Learning the parts of objects by non–negative matrix factorization [J]. Nature, 401(6755): 788–791, 1999.

[4] KRUSKAL J. The relationship between multidimensional scaling and clustering [J]. Classification and clustering, 17–44, 1977.

[5] ROWEIS S T, LAWRENCE K S. Nonlinear dimensionality reduction by locally linear embedding [J]. science , 290(5500): 2323–2326, 2000.

[6] SIROVICH, LAWRENCE, KIRBY M. Low–dimensional procedure for the characterization of human faces [J]. Journal of the Optical Society of America A–Optics Image Science and Vision, 4(3): 519–524, 1987.

[7] DEMPSTER A P, LAIRD N M, RUBIN D B. Maximum likelihood from incomplete data via the em algorithm [J]. Journal of the Royal Statistical Society: Series B, 39(1): 1–38, 1977.

[8] HAMERL Y, GREG, ELKAN C. Alternatives to the k–means algorithm that find better clusterings [C] //Proceedings of the eleventh international conference on Information and knowledge management. New York: ACM press, 2002.

[9] VATTANI, ANDREA. K–means requires exponentially many iterations even in the plane [J]. Discrete & Computational Geometry, 45(4): 596–616, 2011.

[10] INABA, MARY, KATOH N, et al. Applications of weighted Voronoi diagrams and randomization to variance–based k–clustering [C] //Proceedings of the tenth annual symposium on Computational geometry. [S. l.] : [s. n.] , 1994.

[11] PEARSON K LIII. On lines and planes of closest fit to systems of points in space [J]. Philosophical Magazine and Journal of Science, 2(11): 559–572, 1901.

[12] HOTELLING H. Analysis of a complex of statistical variables into principal components [J]. Journal of educational psychology, 24(6): 417–441, 1933.

[13] PASINI G. Principal Component Analysis for Stock Portfolio Management [J]. International Journal of Pure and Applied Mathematics, 115(1): 153–167, 2017.

[14] HARGREAVES C A, CHANDRIKA K M. The Selection of winning stocks using principal component analysis [J]. American Journal of Marketing Research, 1(3): 183–188, 2015.

[15] JIRSA V K, et al. A theoretical model of phase transitions in the human brain [J]. Biological Cybernetics, 71(1): 27–35, 1994.

[16] MARKOVSKY I. Low Rank Approximation: Algorithms, Implementation, Applications [M]. Berlin: Springer, 2012.

[17] FOLTZ P W, DUMAIS S T. Personalized information delivery: An analysis of information filtering methods [J]. Communications of the ACM , 35(12): 51–61, 1992.

[18] GONG Y, LIU X. Creating generic text summarie [C] //

Proceedings of Sixth International Conference on Document Analysis and Recognition, [S.l.] : [s.n.] , 2001.

[19] BRADFORD R B. Efficient discovery of new information in large text databases [J] . International Conference on Intelligence and Security Informatics, 2005.

[20] WU C F J. On the convergence properties of the EM algorithm [J] . The Annals of statistics, 11(1): 95–103, 1983.

[21] LINDSTROM M J, DOUGLAS M B. Newton–Raphson and EM algorithms for linear mixed–effects models for repeated–measures data [J] . Journal of the American Statistical Association, 83(404): 1014–1022, 1988.

[22] DYK V, DAVID A. Fitting mixed–effects models using efficient EM–type algorithms [J] . Journal of Computational and Graphical Statistics, 9(1): 78–98, 2000.

[23] DIFFEY S M, et al. A new REML (parameter expanded) EM algorithm for linear mixed models [J] . Australian & New Zealand Journal of Statistics, 59(4): 433–448, 2017.

[24] MENG J, SHANG H, BIAN L. The Application On Intrusion Detection Based On K–Means Cluster Algorithm [J] . IEEE International Conference On Information Technology And Applications, 2009.

[25] MAHENDIRAN A, SARAVANAN N, SUBRAMANIAN N V, et al. Implementation Of K–Means Clustering In Cloud Computing Environment [J] . Research Journal Of Applied Sciences, Engineering and Technology , 4(10): 1391–1394, 2012.

[26] EZENKWU C P, OZUOMBA S, KALU C. Application Of K–Means Algorithm For Efficient Customer Segmentation: A Strategy For Targeted Customer Services [J] . International Journal Of Advanced Research in Artificial Intelligence, 4(10): 40–44, 2015.

[27] RUMELHART D E , HINTON G E , WILLIAMS R J . Learning internal representations by error propagation [M] // Neurocomputing: foundations of research. Cambridge, MA: MIT Press, 1988.

[28] GOODFELLOW I J, POUGET A J , MIRZA M , et al. Generative Adversarial Nets [C] // International Conference on Neural Information Processing Systems. Cambridge, MA: MIT Press, 2014.

[29] MACQUEEN J B. Methods for classification and Analysis of Multivariate Observations [C] //Proceedings of 5th Berkeley Symposium on Mathematical Statistics and Probability. [S. l.]: [s. n.] , 1967.

第6章 深度学习

6

Neurons that fire together, wire together
—— Donald Hebb

深度学习模型受到了神经系统层次化结构特性的启发，以“层层递进、逐层抽象”机制来完成特定任务。1949年心理学家唐纳德·赫布提出了有名的赫布理论（Hebbian theory）：神经元之间持续重复经验刺激可导致突触传递效能增加（neurons that fire together, wire together），即学习和记忆的本质是大脑结构在后期经验刺激下神经元之间强弱联结被改变的结果。

受到这一思想启发，1957年Frank Rosenblatt提出了一种用于解决二分类问题的线性分类器，其本质是仅包含输入层和输出层的简单神经网络，其中连接权重和阈值均为可学习优化的参数，这一分类器被称为“感知机（perceptron）”。但是感知机处理能力有限，这一方法在当时并未引起广泛关注。

在感知机的输入层和输出层之间加入多个隐藏层就构成了深度神经网络（deep neural network，DNN），也被称为多层感知机（multilayer perceptron，MLP）或前馈神经网络（feedforward neural network）。在早期的深度神经网络中，相邻隐藏层之间的神经元相互全连接（但是同一隐藏层内的神经元之间没有任何连接）。为了实现输入端和输出端之间的映射，深度神经网络在每个神经元中引入了具有非线性映射能力的激活函数（activation function）。非线性映射函数的引入，使得深度神经网络比传统感知机具有更好的表达能力。

为了能够“正确”地将输入数据映射到其给定的语义空间，在标注大数据驱动下，可通过链式求导（chain rule derivatives）和误差反向传播（error backpropagation）对神经网络参数进行优化学习。

综上所述，深度学习是一种基于深度神经网络，通过“端到端（end-to-end）”机制，来学习更好表达数据中所蕴含语义的良好特征方法。

本章6.1节介绍深度学习的发展历史，6.2、6.3和6.4节分别介绍前馈神经网络、卷积神经网络和循环神经网络，6.5节介绍深度学习在自然语言和计算机视觉领域的应用。

6.1 深度学习的历史发展

MOOC微视频：深度学习基本概念

长久以来，研究者不断探索人类认知的神经基础，这需要精确刻画不同层面的认知现象（例如，从所观测行为来推测神经元、神经系统和神经回路的工作机理），进而对在不同层面所观测现象之间的因果联系在现有计算载体上进行机械实现，以启发人工智能模型来提高性能和扩展认知能力。

1943年，神经科学家Warren McCulloch和逻辑学家Walter Pitts合作提出了“McCulloch-Pitts（MCP）neuron”的思想。MCP可对输入信号进行线性加权组合，再用符号函数来输出组合结果，以模拟大脑复杂活动模式。MCP是最早的神经网络雏形[19]。MCP模型虽简单，却开启了建立神经网络大厦地基的征途。但是，在MCP中，输入数据只能是0或1这样的二值化数据，网络中连接权重和符号函数阈值等参数无法从数据中学习，需要预先人为手工设置。

1949年，心理学家唐纳德·赫布（Donald Hebb）提出了著名的赫布理论（Hebbian theory），指出“神经元之间持续重复经验刺激可导致突触传递效能增加（Neurons that fire together，wire together）”，“神经元之间突触的强弱变化是学习与记忆的生理学基础”这一理论为联结主义人工智能研究提供了认知神经心理学基础[7]。

1958 年，David Hubel和Torsten Wiesel在实验中发现小猫后脑皮层中不同视觉神经元与瞳孔所受刺激之间存在某种对应关系，由此发现了一种被称为“方向选择性细胞（orientation selective cell）”的神经元细胞，从而揭示了“视觉系统信息分层处理”这一机制[11，12]。由于这一贡献，他们共同获得了1981年的诺贝尔生理学或医学奖。

神经网络研究的突破来自于Frank Rosenblatt在20世纪50年代所提出的“感知机（perceptron）”模型[22]。感知机模型是一个仅包含输入层和输出层的两层神经网络。对于一个线性可分二分类问题，从理论上可证明感知机能且一定能将线性可分的两类数据区分开来。在具体计算中，感知机先将输入数据按照一定权值进行线性累加，再利用符号函数对累加结果进行映射，以获得0或1形式的输出结果，来解决二分类问题。与MCP不同，感知机中的连接权重和符号函数阈值可以从数据中进行学习，且不再要求输入数据是二值化结果。1969年，Marvin Minsky和Seymour A. Papert出版了《感知机：计算几何学》[20]，书中证明了由输入层和输出层构成的感知机能力极其有限，甚至无法完成异或（XOR）这一基本逻辑计算问题。并且指出，如果在感知机中增加隐藏层，则会因计算量过大而无法学习得到良好的模型参数。这一书籍的出版使得神经网络早期发展陷入了低谷。

由于感知机中没有包含非线性变换操作的隐藏层，因此感知机表达能力较弱（如无法解决异或问题）。如果在感知机中加入多个隐藏层，那么所形成的多层感知机（multilayer perceptron，MLP）则具备更强表达能力，只是一旦增加了隐藏层，需要有效的计算方法来解决深度神经网络模型中参数的优化训练问题。

最早由Werbos提出[24，25]，并且由Rumelhar和Hinton等人[23]完善的误差反向传播（error backpropagation）算法解决了多层感知机中参数优化这一难题。在多层感知机中，由于隐藏层中每个神经元均包含具备非线性映射能力的激活函数（如sigmoid），因此其可用来构造复杂的

非线性分类函数。

2006年，Hinton在*Science*等期刊上发表了论文，首次提出了“深度信念网络（deep belief network）”模型[8]，在相关分类任务上取得的性能超过了传统浅层学习模型（如支持向量机），使得深度架构引起了大家的关注。与传统的训练方式不同，“深度信念网络”有一个“预训练”（pre-training）的过程，可为神经网络中连接权重找到一个接近最优解的值，之后再使用“微调”（fine-tuning）技术来对整个网络的参数进行优化训练。这两项技术大幅度减少了训练多层神经网络的时间开销。Hinton给多层神经网络相关的学习方法赋予了一个新名词——“深度学习”。

深度学习的兴起推动了不同类型深度学习网络模型的研究：为了解决前馈神经网络无法处理反馈信息的不足，循环神经网络（recurrent neural network，RNN）思想被提出，其中包括John Hopfield提出的霍普菲尔德反馈神经网络（Hopfield neural network）[10]、Elman-RNN[4]、Jordan-RNN[13]以及长短时记忆网络（long short-term memory network，LSTM）[9]、门控循环单元（gated recurrent unit，GRU）[2]；从早期进行手写体识别的卷积神经网络LeNet[17]到2012年推动深度学习成为热点、用于图像分类任务的卷积神经网络AlexNet[16]；从神经语言模型（neural language model）[1]到词向量模型（word2vec）[21]以及BERT（bidirectional encoder representations from transformers）模型[3]等。

深度学习的基本动机在于通过“端到端学习（end-to-end learning）”这一机制来构建多层神经网络，以学习隐含在数据内部的关系，从而使学习所得特征具有更强的表达能力。在大规模数据上所进行的实验已经表明：通过深度学习所得到的特征表示在自然语言理解、图像分类、语音处理和视频识别等领域表现了良好性能。

6.2 前馈神经网络

前馈神经网络（feedforward neural network）作为最基础的神经网络，一般包括输入层、隐藏层和输出层。每层神经元只和相邻层神经元相连，即每层神经元只接收相邻前序神经层中神经元所传来的信息，只给相邻后续神经层中神经元传递信息。在前馈神经网络中，同一层的神经

元之间没有任何连接，后续神经层不向其前序相邻神经层传递任何信息。前馈神经网络是目前应用最为广泛的神经网络之一。

6.2.1 若干概念

在正式介绍前馈神经网络之前，先介绍神经元、激活函数、输出单元以及损失函数等概念。

1. 神经元

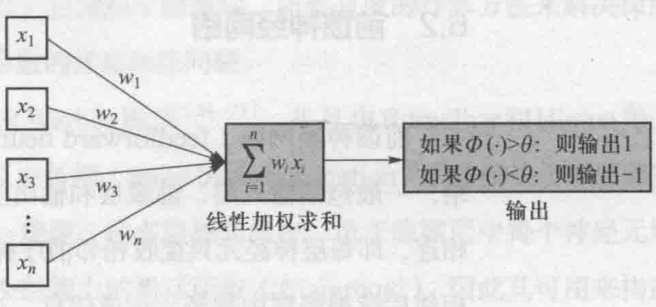
在生物学中，神经元细胞有兴奋与抑制两种状态。大多数神经元细胞在正常情况下处于抑制状态，一旦某个神经元受到刺激并且电位超过一定的阈值后，这个神经元细胞就被激活，处于兴奋状态，并向其他神经元传递信息。基于神经元细胞的结构特性与传递信息方式，神经科学家Warren McCulloch和逻辑学家Walter Pitts合作提出了“McCulloch-Pitts（MCP）neuron”模型[19]。在人工神经网络中，MCP模型成为人工神经网络中的最基本结构。MCP模型结构如图6.1所示。

从图6.1可见，给定n个二值化（0或1）的输入数据x_i（$1 \leqslant i \leqslant n$）与连接参数$w_i$（$1 \leqslant i \leqslant n$），MCP神经元模型对输入数据线性加权求和，然后使用函数$\Phi(\ )$将加权累加结果映射为0或1，以完成二分类的任务：

$$y = \Phi\left(\sum_{i=1}^{n} w_i x_i\right)$$

其中w_i为预先设定的连接权重值（一般在0和1中取一个值或者-1和1中取一个值），用来表示其所对应输入数据对输出结果的影响（即权重）。$\Phi(\cdot)$将输入端数据与连接权重所得线性加权累加结果与预先设定阈值θ进行比较，根据比较结果输出-1或1。具体而言，如果线性加权累加结果（即$\sum_{i=1}^{m} w_i x_i$）大于阈值θ，则函数$\Phi(\cdot)$的输出为1，否则为-1。也就是说，如果线性加权累加结果大于阈值θ，则神经元细胞处

图6.1 MCP模型结构



于兴奋状态，向后传递1的信息，否则该神经元细胞处于抑制状态而不向后传递信息。

从另外一个角度来看，对于任何输入数据x_i（$1 \leqslant i \leqslant n$），MCP模型可得到-1或1这样的输出结果，也就是实现了将输入数据分类到-1或1两个类别中，解决了二分类问题。

2. 激活函数

在现实世界中，非线性是世界万事万物运转的魂魄，恩利克·费米（Enrico Fermi）曾经说过，“圣经中并没有说过一切大自然的定律都可以用线性方程来表示”，线性系统实际上只是对少数简单非线性系统的理论近似。将神经网络用于分类任务，需要将输入数据（如图像、文本和听觉等信息）映射到其语义空间（如人脸、情感和旋律等），这一过程就是一个复杂的非线性变换。

神经网络使用非线性函数作为激活函数（activation function），通过对多个非线性函数进行组合，来实现对输入信息的非线性变换。为了能够使用梯度下降（梯度下降算法会在6.2.2节中详细说明）方法来训练神经网络参数，激活函数必须是连续可导的。

表6.1 激活函数基本性质

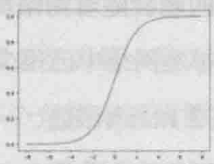
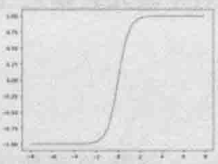
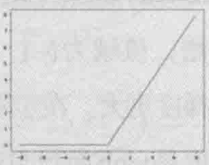
名称	函数	图像	导数	值域
sigmoid	$f(x)=\frac{1}{1+e^{-x}}$		$f'(x)=f(x)*(1-f(x))$	(0, 1)
tanh	$f(x)=\frac{1-e^{-2x}}{1+e^{-2x}}$		$f'(x)=1-f(x)^2$	(-1, 1)
ReLU	$f(x)=\begin{cases}0, \text{for } x\leqslant 0\\ x, \text{for } x>0\end{cases}$		$f(x)=\begin{cases}0, \text{for } x\leqslant 0\\ 1, \text{for } x>0\end{cases}$	$[0,+\infty)$

表6.1给出了常用的激活函数的基本信息。

（1）sigmoid函数

对于输入数据x，sigmoid函数的数学表达式如下：

$$f(x)=\frac{1}{1+e^{-x}}$$

选取sigmoid函数作为激活函数，因为其具有如下优点：① 概率形式输出，sigmoid函数值域为(0, 1)，因此使sigmoid函数输出可视为概率值；② 单调递增，sigmoid函数对输入x取值范围没有限制，但当x大于一定值后，函数输出无限趋近于1，而小于一定数值后，函数输出无限趋近于0，特别地，当$x=0$时，函数输出为0.5；③ 非线性变化，x取值在0附近时，函数输出值的变化幅度比较大（函数值变化陡峭），意味着函数在0附近容易被激活且是非线性变化，当x取值很大或很小时，函数输出值几乎不变，这是基于概率的一种认识与需要。

sigmoid函数的导数可如下推导得到：

$$f'(x)=\left(\frac{1}{1+e^{-x}}\right)'=\frac{e^{-x}}{\left(1+e^{-x}\right)^2}=\frac{1+e^{-x}-1}{\left(1+e^{-x}\right)^2}=\frac{1}{\left(1+e^{-x}\right)}\left(1-\frac{1}{\left(1+e^{-x}\right)}\right)$$

$$=f(x)\left(1-f(x)\right)$$

由于$0<f(x)<1$和$0<1-f(x)<1$，因此sigmoid函数的导数$f(x)(1-f(x))$小于1。这样，当把sigmoid函数作为激活函数时，在使用反向传播算法（反向传播算法会在6.2.2节中详细说明）更新参数过程中易出现导数过于接近0的情况，即梯度消失（vanishing gradient）的问题，并且这个问题会随着网络深度的增加而变得愈发严重。梯度消失这一问题使得在神经网络中已很少使用sigmoid函数作为激活函数。

（2）tanh函数

对于输入数据x，tanh函数的数学表达式如下：

$$f(x)=\frac{1-e^{-2x}}{1+e^{-2x}}$$

tanh函数的函数图和sigmoid函数较为相似，它在定义域上也为单调递增，值域为(−1, 1)。相较于sigmoid函数，tanh函数在坐标原点附近的梯度更大。在实际的使用中，使用tanh函数的神经网络更容易收敛。

但是tanh函数与sigmoid函数都面临梯度消失问题，即梯度会随着网络深度的增加而逐渐消失。

（3）ReLU函数

ReLU（rectified linear unit，修正线性单元）函数是目前使用较为普遍的激活函数，其数学表达式如下：

$$f(x)=\begin{cases}0, & x<0\\ x, & x\geqslant 0\end{cases}\qquad 或者 f(x)=\max(0,x)$$

ReLU函数的导数为：$f'(x)=\begin{cases}0, & x<0\\ 1, & x\geqslant 0\end{cases}$

当输入$x\geqslant 0$时，ReLU的导数为常数，这样可有效克服梯度消失这一问题。当$x<0$时，ReLU的梯度总是0，使得神经网络中若干参数取值为0，即参与分类等任务的神经元稀疏，这在一定程度上可克服机器学习中经常出现的过拟合问题。

（4）softmax激活函数

softmax函数一般用在多分类问题中，它是对逻辑斯蒂（logistic）回归的一种推广，也被称为多项逻辑斯蒂回归模型（multi-nominal logistic mode）。假设要实现k个类别的分类任务，softmax函数将输入数据x_i映射到第i个类别的概率y_i如下计算：

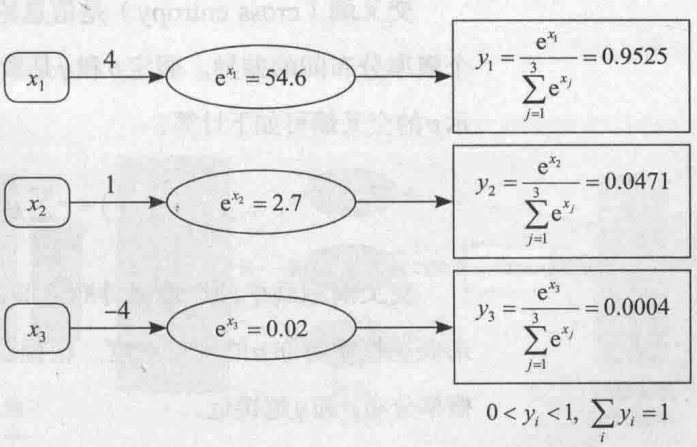
$$y_i=\text{softmax}(x_i)=\frac{e^{x_i}}{\sum_{j=1}^{k}e^{x_j}}$$

显然，$0<y_i<1$。图6.2给出了三分类问题的softmax输出示意。在图中，对于取值为4、1和–4的x_1、x_2和x_3，通过softmax变换后，将其映射到(0, 1)之间的概率值。

由于softmax输出结果的值累加起来为1，因此可将输出概率最大的作为分类目标（图6.2中被分类为第一类）。

也可以从如下另外一个角度来理解图6.2中的内容：给定某个输入数据，可得到其分类为三个类别的初始结果，分别用x_1、x_2和x_3来表

图6.2 三分类问题的softmax输出示意



示。这三个初始分类结果分别是4、1和-4。通过Softmax函数，得到了三个类别分类任务中以概率表示的更好的分类结果，即分别以95.25%、4.71%和0.04%归属于类别1、类别2和类别3。显然，基于这样的概率值，可判断输入数据属于第一类。可见，通过使用softmax函数，可求取输入数据在所有类别上的概率分布。

3. 损失函数

损失函数（loss function）又称为代价函数（cost function），用来计算模型预测值与真实值之间的误差。损失函数是神经网络设计中的一个重要组成部分。通过定义与任务相关的良好损失函数，在训练过程中可根据损失函数来计算神经网络的误差大小，进而优化神经网络参数。

下面介绍两种最常用的损失函数：均方误差损失函数和交叉熵损失函数。

（1）均方误差损失函数

均方误差损失函数通过计算预测值和实际值之间距离（即误差）的平方来衡量模型优劣。假设有n个训练数据x_i，每个训练数据x_i的真实输出为y_i，模型对x_i的预测值为$\hat{y}_i$。该模型在n个训练数据下所产生的均方误差损失可定义如下：

$$\mathrm{MSE}=\frac{1}{n}\sum_{i=1}^{n}\left(y_i-\hat{y}_i\right)^2$$

（2）交叉熵损失函数

在物理学中，“熵”被用来表示热力学系统所呈现的无序程度。香农将这一概念引入信息论领域，提出了“信息熵”概念，通过对数函数来测量信息的不确定性。

交叉熵（cross entropy）是信息论中的重要概念，主要用来度量两个概率分布间的差异。假定p和q是数据x的两个概率分布，通过q来表示p的交叉熵可如下计算：

$$H(p,q)=-\sum_{x}p(x)\cdot\log q(x)$$

交叉熵刻画了两个概率分布之间的距离，旨在描绘通过概率分布q来表达概率分布p的困难程度。根据公式不难理解，交叉熵越小，两个概率分布p和q越接近。

这里仍然以三分类问题为例，假设数据x属于类别1。记数据x的类

别分布概率为y，显然$y=(1, 0, 0)$代表数据x的实际类别分布概率。记$\hat{y}$代表模型预测所得类别分布概率。

那么对于数据x而言，其实际类别分布概率y和模型预测类别分布概率$\hat{y}$的交叉熵损失函数定义为：

$$\text{cross entropy} = -y \times \log\left(\hat{y}^{\mathrm{T}}\right)$$

很显然，一个良好的神经网络要尽量保证对于每一个输入数据，神经网络所预测类别分布概率与实际类别分布概率之间的差距越小越好，即交叉熵越小越好。于是，可将交叉熵作为损失函数来训练神经网络。

图6.3给出了一个三个类别分类的例子。由于输入数据x属于类别1，因此其实际类别概率分布值为$y=(y_1, y_2, y_3)=(1, 0, 0)$。经过神经网络的变换，得到了输入数据$x$相对于三个类别的预测中间值$(z_1, z_2, z_3)$。然后，经过softmax函数映射，得到神经网络所预测的输入数据x的类别分布概率$\hat{y}=\left(\widehat{y_1}, \widehat{y_2}, \widehat{y_3}\right)$。根据前面的介绍，$\widehat{y_1}$、$\widehat{y_2}$和$\widehat{y_3}$为(0, 1)范围之间的一个概率值。由于样本$x$属于第一个类别，因此希望神经网络所预测得到的$\widehat{y_1}$取值要远远大于$\widehat{y_2}$和$\widehat{y_3}$的取值。为了得到这样的神经网络，在训练中可利用如下交叉熵损失函数来对模型参数进行优化：

$$\text{cross entropy} = -\left(y_1 \times \log\left(\widehat{y_1}\right) + y_2 \times \log\left(\widehat{y_2}\right) + y_3 \times \log\left(\widehat{y_3}\right)\right)$$

在上式中，y_2和y_3均为0，y_1为1，因此交叉熵损失函数简化为：$-y_1 \times \log\left(\widehat{y_1}\right) = -\log\left(\widehat{y_1}\right)$。

在神经网络训练中，要将输入数据实际的类别概率分布与模型预测的类别概率分布之间的误差（即损失）从输出端向输入端传递，以便来优化模型参数。下面简单介绍根据交叉熵计算得到的误差从$\widehat{y_1}$传递给z_1和z_2（z_3的推导与z_2相同）的情况。

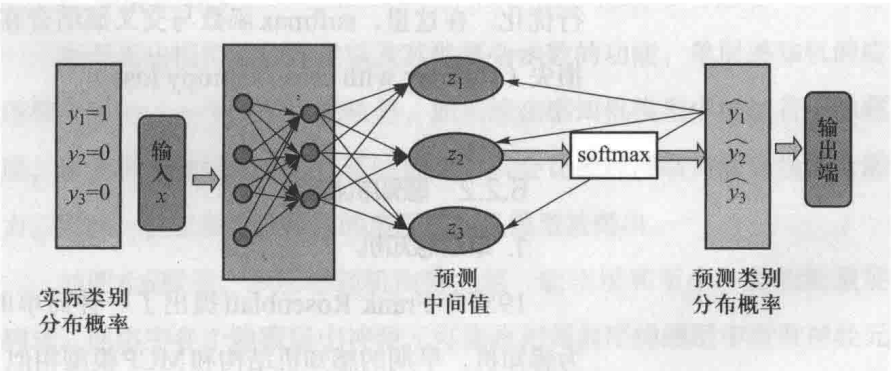


图6.3 三分类问题中输入x的交叉熵损失示意（x属于第一类）

$$\frac{\partial \hat{y}_1}{\partial z_1}=\frac{\partial}{\partial z_1}\left(\frac{e^{z_1}}{\sum_k e^{z_k}}\right)=\frac{\left(e^{z_1}\right)'\times\sum_k e^{z_k}-e^{z_1}\times e^{z_1}}{\left(\sum_k e^{z_k}\right)^2}=\frac{e^{z_1}}{\sum_k e^{z_k}}-\frac{e^{z_1}}{\sum_k e^{z_k}}\times\frac{e^{z_1}}{\sum_k e^{z_k}}$$

$$=\hat{y}_1\left(1-\hat{y}_1\right)$$

（注意：在推导中应用了$\left(e^x\right)'=e^x$，$\left(\frac{v}{u}\right)'=\frac{v'u-vu'}{u^2}$等公式）

由于交叉熵损失函数$-\log\left(\hat{y}_1\right)$对$\hat{y}_1$求导的结果为$-\frac{1}{\hat{y}_1}$，$\hat{y}_1\left(1-\hat{y}_1\right)$与$-\frac{1}{\hat{y}_1}$相乘为$\hat{y}_1-1$。这说明一旦得到模型预测输出$\hat{y}_1$，将该输出减去1就是交叉熵损失函数相对于$z_1$的偏导结果。

$$\frac{\partial \hat{y}_1}{\partial z_2}=\frac{\partial}{\partial z_2}\left(\frac{e^{z_1}}{\sum_k e^{z_k}}\right)=\frac{0\times\sum_k e^{z_k}-e^{z_1}\times e^{z_2}}{\left(\sum_k e^{z_k}\right)^2}=-\frac{e^{z_1}}{\sum_k e^{z_k}}\times\frac{e^{z_2}}{\sum_k e^{z_k}}=-\hat{y}_1\hat{y}_2$$

同理，交叉熵损失函数导数为$-\frac{1}{\hat{y}_1}$，$-\hat{y}_1\hat{y}_2$与$-\frac{1}{\hat{y}_1}$相乘结果为$\hat{y}_2$。这意味对除第一个输出结点以外的结点求偏导，得到其他结点的模型预测输出就是交叉熵损失函数相对于其他结点的偏导结果。在z_1、z_2和z_3得到偏导结果后，再通过链式法则（后续介绍）将损失误差继续往输入端传递即可。

在上面的例子中，假设所预测中间值(z_1, z_2, z_3)经过softmax映射后所得结果为(0.34, 0.46, 0.20)。由于已知输入数据x属于第一类，显然这个输出不理想而需要对模型参数进行优化。如果选择交叉熵损失函数来优化模型，则(z_1, z_2, z_3)这一层的偏导值为(0.34–1, 0.46, 0.20) = (–0.66, 0.46, 0.20)。

可以看出，softmax函数和交叉熵相互结合，为偏导计算带来了极大便利。偏导计算使得损失误差从输出端向输入端传递，来对模型参数进行优化。在这里，softmax函数与交叉熵结合在一起，因此也叫softmax损失（softmax with cross-entropy loss）。

6.2.2 感知机模型

1. 单层感知机

1957年Frank Rosenblatt提出了一种简单的人工神经网络，被称之为感知机。早期的感知机结构和MCP模型相似，由一个输入层和一个输

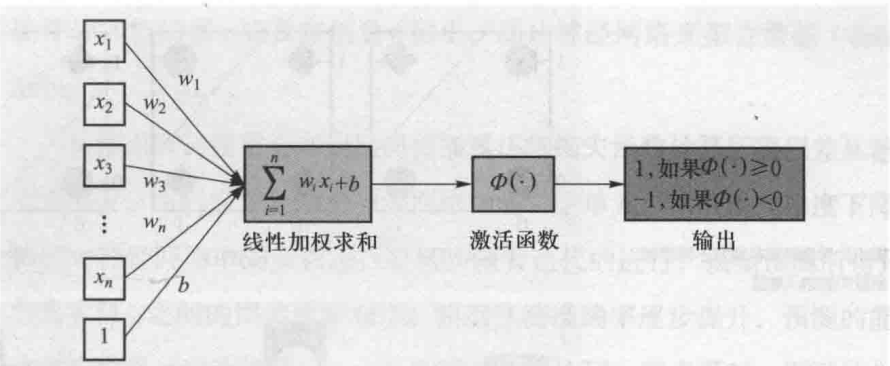


图6.4 感知机模型

出层构成，因此也被称为“单层感知机”。感知机的输入层负责接收实数值的输入向量，输出层则能输出−1或1两个值。单层感知机可作为一种两类线性分类模型，结构如图6.4所示。

对于具有n个输入以及对应连接权重系数为w_i的感知机，首先通过线性加权得到输入数据的累加结果z：$z = w_1x_1 + w_2x_2 + \cdots + b$。这里$x_1, x_2, \cdots, x_n$为感知机的输入，$b$为偏置项（bias）。然后将$z$作为激活函数$\Phi(\cdot)$的输入，激活函数会将$z$与某一阈值（此例中，阈值为0）进行比较，如果大于该阈值则感知器输出为1，否则输出为−1。通过这样的操作，输入数据被分类为1或−1这两个不同类别。

单层感知机与MCP模型在连接权重设置上是不同的，即感知机中连接权重参数并不是预先设定好的，而是通过多次迭代训练得到的。具体而言，单层感知机构建损失函数，来计算模型预测值与数据真实值之间的“误差”，通过最小化损失函数取值，来优化模型参数。

单层感知机可被用来区分线性可分数据。在图6.5中，逻辑与（AND）、逻辑与非（NAND）和逻辑或（OR）为线性可分函数，所以可利用单层感知机来模拟这些逻辑函数。但是，由于逻辑异或（XOR）是非线性可分的逻辑函数，因此单层感知机无法模拟逻辑异或函数的功能。

2. 多层感知机

由于无法模拟诸如异或以及其他复杂函数的功能，单层感知机的应用较为单一。一个简单的想法是，如果能在感知机模型中增加若干隐藏层，增强神经网络的非线性表达能力，就会让神经网络具有更强拟合能力。因此，包含多个隐藏层的多层感知机模型被提出。

如图6.6所示，多层感知机由输入层、输出层和至少一层的隐藏层构成。网络中各个隐藏层中神经元可接收相邻前序隐藏层中所有神经元

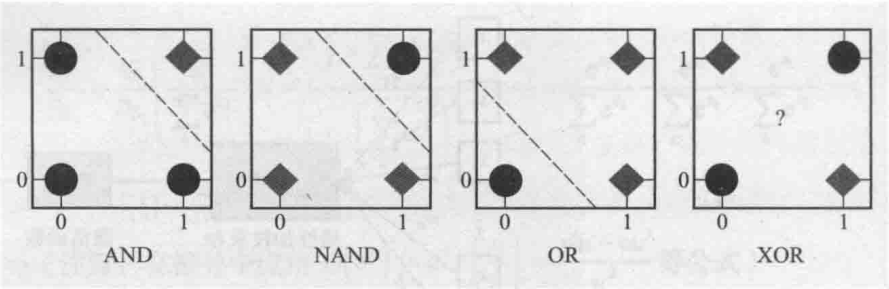


图6.5 单层感知机模拟不同逻辑函数功能的示意图

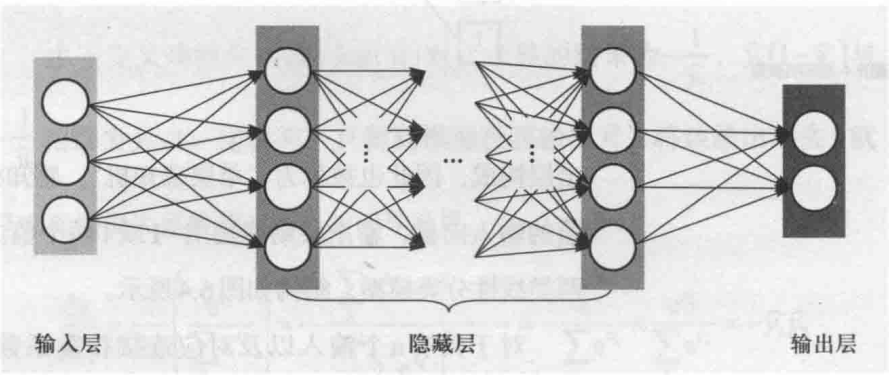


图6.6 多层感知机模型

传递而来的信息，经过加工处理后将信息输出给相邻后续隐藏层中所有神经元。

在多层感知机中，相邻层所包含的神经元之间通常使用“全连接”方式进行连接。所谓“全连接”是指两个相邻层之间的神经元相互成对连接，但同一层内神经元之间没有连接。多层感知机可以模拟复杂非线性函数功能，所模拟函数的复杂性取决于网络隐藏层数目和各层中神经元数目。

6.2.3 参数优化与学习

在设计完成一个多层神经网络的结构后（如多少个隐藏层，每个隐藏层中包含的神经元数目以及神经元采用的非线性激活函数等），需要对神经网络中参数（即相邻层中神经元和神经元之间的连接权重系数等）进行优化学习，以使得神经网络能够将给定输入数据映射到所期望的输出语义空间（如将一幅像素点构成的图像数据映射为人脸），完成分类识别等任务。

神经网络参数优化是一个监督学习的过程。给定n个标注样本数据(x_i, y_i)，这里x_i是输入数据、y_i是x_i的类别等标注信息。假设输入数据x_i经过神经网络处理后所得输出预测值为$\hat{y}_i$，则可通过损失函数来计算模型预测值$\hat{y}_i$与真实值y_i之间误差或损失，利用其来优化模型参数，以便

让神经网络按照x_i的真实信息y_i输出，即让神经网络来拟合数据（data fitting）。

具体而言，模型会利用反向传播算法将损失函数计算所得误差从输出端出发，由后向前传递给神经网络中的每个单元，然后通过梯度下降算法对神经网络中的参数进行更新。随着迭代的进行，模型预测所得$\hat{y}_i$与真实值y_i之间的误差逐步缩小，模型预测准确率逐步提升，预测的能力逐步增强，当迭代经过一定轮次或准确率达到一定水平时，则可认为模型参数已被优化完毕。

1. 梯度下降

梯度下降算法（gradient descent）是机器学习中对模型参数进行优化时所采用的常用方法。为了让模型预测结果与真实结果之间的损失误差最小，利用梯度下降法来逐步迭代优化模型参数。当损失误差最小后，此时所学习得到的模型参数即为模型的最优参数。

下面来说明损失函数梯度的反方向是损失误差下降最快的方向。假设损失函数$f(\boldsymbol{x})$是连续可微的多元变量函数，其泰勒展开如下：

$$f(\boldsymbol{x}+\Delta\boldsymbol{x})=f(\boldsymbol{x})+f'(\boldsymbol{x})\Delta\boldsymbol{x}+\frac{1}{2}f''(\boldsymbol{x})(\Delta\boldsymbol{x})^2+\cdots+\frac{1}{n!}f^{(n)}(\boldsymbol{x})(\Delta\boldsymbol{x})^n$$

$$f(\boldsymbol{x}+\Delta\boldsymbol{x})-f(\boldsymbol{x})\approx\left(\nabla f(\boldsymbol{x})\right)^{\mathrm{T}}\Delta\boldsymbol{x}$$

假设参数变量当前取值为$\boldsymbol{x}$，其对应的损失误差数值为$f(\boldsymbol{x})$。在不断促使损失误差减少的下一步迭代时，$\boldsymbol{x}$取值变化为$\boldsymbol{x}+\Delta\boldsymbol{x}$，则损失误差数值相应地变为$f(\boldsymbol{x}+\Delta\boldsymbol{x})$。

由于$f(\boldsymbol{x})$是损失函数，因此在迭代中希望$f(\boldsymbol{x}+\Delta\boldsymbol{x})$取值小于$f(\boldsymbol{x})$，即损失误差随着模型参数被优化而逐渐减少，使得$f(\boldsymbol{x}+\Delta\boldsymbol{x})<f(\boldsymbol{x})$，于是$\left(\nabla f(\boldsymbol{x})\right)^{\mathrm{T}}\Delta\boldsymbol{x}<0$。

在$\left(\nabla f(\boldsymbol{x})\right)^{\mathrm{T}}\Delta\boldsymbol{x}=\left\|\nabla f(\boldsymbol{x})\right\|\left\|\Delta\boldsymbol{x}\right\|\cos\theta$中，$\left\|\nabla f(\boldsymbol{x})\right\|$和$\left\|\Delta\boldsymbol{x}\right\|$分别为损失函数梯度的模和下一轮迭代中$\boldsymbol{x}$取值增量的模，两者均为正数。为了保证损失误差减少，只要保证$\cos\theta<0$。当$\theta=180°$时，$\cos\theta=-1$，这时损失函数减少的幅度值$\left(\nabla f(\boldsymbol{x})\right)^{\mathrm{T}}\Delta\boldsymbol{x}$取到最小。可见，当损失函数梯度$\nabla f(\boldsymbol{x})$和$\boldsymbol{x}$取值增量$\Delta\boldsymbol{x}$之间夹角为180度时，损失误差减少最多。于是，在迭代过程中为了最小化损失误差，每次迭代时可沿着损失函数梯度的反方向（即$\theta=180°$）以$\Delta\boldsymbol{x}$幅度来改变$\boldsymbol{x}$取值，这样会保证$f(\boldsymbol{x}+\Delta\boldsymbol{x})$和$f(\boldsymbol{x})$之间的差值最大（即损失误差下降最快）。

于是，当$\theta=180°$时，$f(\boldsymbol{x}+\Delta\boldsymbol{x})$和$f(\boldsymbol{x})$之间的差值为：$f(\boldsymbol{x}+\Delta\boldsymbol{x})-f(\boldsymbol{x})=\|\nabla f(\boldsymbol{x})\|\|\Delta\boldsymbol{x}\|\cos\theta=-\|\Delta\boldsymbol{x}\|\|\nabla f(\boldsymbol{x})\|$。这说明只要沿着损失函数梯度的反方向选取$\boldsymbol{x}$的增量$\Delta\boldsymbol{x}$，就会保证损失误差下降最多、下降最快，犹如从山峰处沿着最陡峭路径可快速走到山谷。在前面的推导中忽略了损失函数的二阶导数以及其高阶导数的取值，因此在实际中引入步长η，用$\boldsymbol{x}-\eta\nabla f(\boldsymbol{x})$来更新$\boldsymbol{x}$（在具体实现时$\eta$可取一个定值）。

从下面的公式可以计算得到变量的增量为$-\eta\nabla f(\boldsymbol{x})$

$$\underbrace{\boldsymbol{x}-\eta\nabla f(\boldsymbol{x})}_{\text{变量更新后的取值}}-\underbrace{\boldsymbol{x}}_{\text{变量当前取值}}=\underbrace{-\eta\nabla f(\boldsymbol{x})}_{\text{变量的增量}}$$

由于$f(\boldsymbol{x}+\Delta\boldsymbol{x})-f(\boldsymbol{x})=(\nabla f(\boldsymbol{x}))^{\mathrm{T}}\Delta\boldsymbol{x}=-\eta(\nabla f(\boldsymbol{x}))^2<0$，因此损失误差会不断减少。由于又是选择了损失函数的梯度反方向来改变变量权重，因此损失函数所计算误差会减少最快。

在上面的推导中，如果$\theta=0°$，即沿着函数梯度的方向以$\Delta\boldsymbol{x}$来改变$\boldsymbol{x}$取值，则$f(\boldsymbol{x}+\Delta\boldsymbol{x})-f(\boldsymbol{x})$之差为最大值。这意味着函数的梯度方向是函数值增加最快的方向，而梯度反方向则是函数值下降最快的方向。

这里要注意的是多元函数在某一点的梯度是由多元函数对每个变量的偏导数组成（即求函数梯度时需要对各个变量求偏导），其方向为函数在该点增加最快的方向，梯度大小为多元函数在该点的最大变化率。多元函数相对于它的某个自变量的偏导数，是假定其他自变量不变时，对该自变量的导数。

对于优化问题来说，凸函数具有一个良好性质：凸函数的局部最优解即是全局最优解。因此对于凸函数而言，梯度下降法总是能够找到函数的全局最小值。为了达到这一目的，损失函数一般采用凸函数。

2. 反向传播

反向传播算法（back propagation）主要利用损失函数来计算模型预测结果与真实结果之间的误差以优化调整模型参数，这一优化调整机制是从输出端向输入端，由后向前递进进行。

前面介绍过，损失函数梯度反方向是损失误差下降最快方向。在求取损失函数梯度时需要对各个变量求偏导，这一求取偏导过程要用到链式法则（Chain Rule）。

在图6.7中，*out*是一个神经单元，其在神经网络中完成两个任务：① 将相邻前序神经元传递而来的信息线性加权累加，得到结果$\mathcal{X}$，即

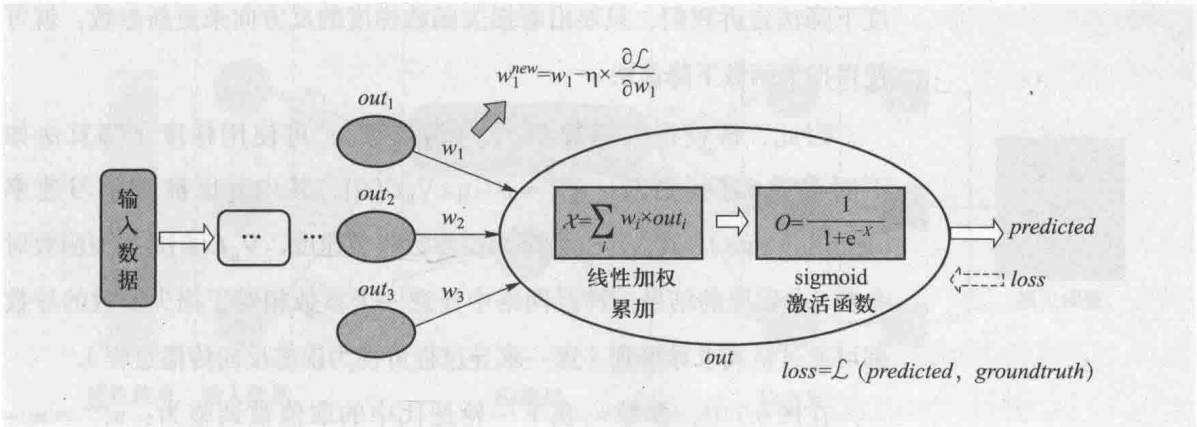


图6.7 链式求导与模型参数更新示意图

$\mathcal{X}=\sum_i w_i\times out_i\,(1\leqslant i\leqslant 3)$。这里$out_i$和$w_i$分别是与$out$前序相连的第$i$个神经元传递信息及第$i$个神经元与$out$之间的连接权重系数；② 使用激活函数对加权累加结果进行非线性变换。图6.7中使用的激活函数为sigmoid，即$\mathcal{O}=\frac{1}{1+e^{-\mathcal{X}}}$。可见，$\mathcal{O}$的大小就是神经网络对输入样本数据的预测输出。

图6.7中，模型预测结果$\mathcal{O}$与真实结果之间的损失误差$loss$由所定义的损失函数$\mathcal{L}$计算得到。现在的问题是如何根据这个误差来优化调整连接权重系数w_i（以及按照同样的模式来调整图中未出现的神经网络其他参数），使得训练得到的模型尽可能输出预期结果。

先看看如何来调整w_1。按照前面的讨论，损失函数$\mathcal{L}$取值下降最快的方向是其梯度方向的反方向。为了使损失函数$\mathcal{L}$取值减少（从而保证模型预测结果与实际结果之间的差距越来越小），需要求取损失函数$\mathcal{L}$相对于w_1的偏导，然后按照损失函数梯度的反方向选取一个微小的增量，来调整w_1的取值，就能够保证损失函数取值减少。即将w_1变为$w_1-\eta\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}w_1}$后，能使得损失误差减少。这里$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}w_1}$为损失函数$\mathcal{L}$对$w_1$的偏导。于是，需要求取损失函数$\mathcal{L}$对变量参数$w_1$的偏导。

由于w_1与加权累加函数$\mathcal{X}$和sigmoid函数均有关，因此$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}w_1}=\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\mathcal{O}}\frac{\mathrm{d}\mathcal{O}}{\mathrm{d}\mathcal{X}}\frac{\mathrm{d}\mathcal{X}}{\mathrm{d}w_1}$。在这个链式求导中：$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\mathcal{O}}$与损失函数的定义有关；$\frac{\mathrm{d}\mathcal{O}}{\mathrm{d}\mathcal{X}}$是对sigmoid函数求导，结果为$\frac{1}{1+e^{-\mathcal{X}}}\times\left(1-\frac{1}{1+e^{-\mathcal{X}}}\right)$；$\frac{\mathrm{d}\mathcal{X}}{\mathrm{d}w_1}$是加权累加函数$w_1\times out_1+w_2\times out_2+w_3\times out_3$对$w_1$求导，结果为$out_1$。

链式求导实现了损失函数对某个自变量求偏导，好比将损失误差从输出端向输入端逐层传播，通过这个传播过程来更新该自变量取值。梯

度下降法告诉我们，只要沿着损失函数梯度的反方向来更新参数，就可使得损失函数下降最快。

因此，假设损失函数$\mathcal{L}(\theta)$（θ为参数），可使用梯度下降算法如下对参数θ进行更新：$\theta^{new}=\theta-\eta\times\nabla_{\theta}\mathcal{L}(\theta)$。其中$\eta$也被称学习速率（learning rate），其大小会影响模型参数收敛速度，$\nabla_{\theta}\mathcal{L}(\theta)$是损失函数对参数$\theta$求偏导的结果。神经网络中任意一个参数相对于损失函数的导数都可通过链式求导得到（这一求导过程可视为误差反向传播过程）。

在图6.7中，参数w_1在下一轮迭代中的取值被调整为：$w_1^{new}=w_1-\eta\times\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}w_1}=w_1-\eta\times\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\mathcal{O}}\frac{\mathrm{d}\mathcal{O}}{\mathrm{d}\mathcal{X}}\frac{\mathrm{d}\mathcal{X}}{\mathrm{d}w_1}=w_1-\eta\times\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\mathcal{O}}\times\frac{1}{1+\mathrm{e}^{-\mathcal{X}}}\times\left(1-\frac{1}{1+\mathrm{e}^{-\mathcal{X}}}\right)\times out_1$。按照同样的方法，可调整$w_2$、$w_3$以及其他图中未显示参数的取值。经过这样的调整，在模型参数新的取值作用下，损失函数$\mathcal{L}(\theta)$会以最快下降方式逐渐减少，直至减少到最小值（即全局最小值）。

梯度下降算法可以分为批量梯度下降算法（batch gradient descent）、随机梯度下降算法（stochastic gradient descent）和小批量梯度下降算法（mini-batch gradient descent）等类型。批量梯度下降算法是在整个训练集上计算损失误差$\mathcal{L}(\theta)$。如果数据集较大，则会因内存容量不足而无法完成，同时这一方法收敛速度较慢。随机梯度下降算法是使用训练集中每个训练样本计算所得$\mathcal{L}(\theta)$来分别更新参数。虽然，随机梯度下降收敛速度会快一些，但可能出现所优化目标函数震荡不稳定现象。小批量梯度下降算法是选取训练集中小批量样本计算$\mathcal{L}(\theta)$，根据每一批量样本所得到的累加误差来更新参数，这样可保证训练过程更稳定，而且采用小批量训练方法也可利用矩阵计算优势。小批量梯度下降算法是目前最常用的梯度下降算法。

3. 误差反向传播与参数更新例子

在了解误差反向传播算法与梯度下降算法后，下面通过一个三分类的具体例子来介绍神经网络中参数更新过程。给定一个包含输入层、一层隐藏层和输出层的多层神经网络，如图6.8所示，其中隐藏层由两个神经元构成。网络使用Sigmoid函数作为神经元的激活函数，使用均方损失函数来计算网络输出值与实际值之间的误差。

在图6.8中，每个神经元完成如下两项任务：① 对相邻前序层所传递信息进行线性加权累加；② 对加权累加结果进行非线性变换。假设样

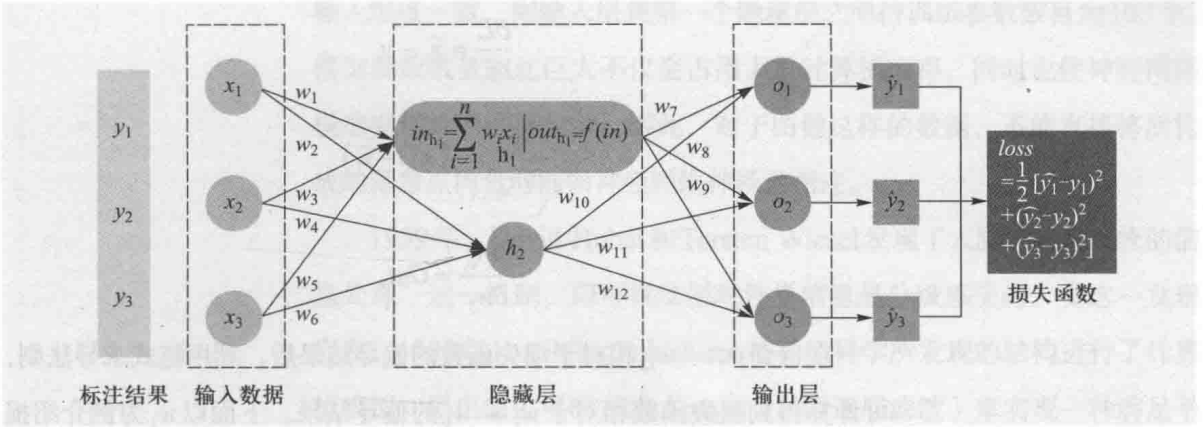


图6.8 包含一个隐藏层的多层感知机

本数据输入为（x_1, x_2, x_3），其标注信息为（y_1, y_2, y_3）。在三分类问题中，对于输入数据（x_1, x_2, x_3），y_1、y_2和y_3中只有一个取值为1，其余两个取值为0。

表6.2给出了h_1、h_2、o_1，o_2和o_3所承担两项任务的内容。

表6.2　在图6.8中神经网络各单元计算结果

单元	相邻前序神经元传递信息线性累加	非线性变换
h_1	$In_{h_1}=w_1*x_1+w_3*x_2+w_5*x_3$	$Out_{h_1}=sigmoid\left(In_{h_1}\right)$
h_2	$In_{h_1}=w_2*x_1+w_4*x_2+w_6*x_3$	$Out_{h_2}=sigmoid\left(In_{h_2}\right)$
o_1	$In_{o_1}=w_7*Out_{h_1}+w_{10}*Out_{h_2}$	$\widehat{y_1}=Out_{o_1}=sigmoid\left(In_{o_1}\right)$
o_2	$In_{o_2}=w_8*Out_{h_1}+w_{11}*Out_{h_2}$	$\widehat{y_2}=Out_{o_2}=sigmoid\left(In_{o_2}\right)$
o_3	$In_{o_3}=w_9*Out_{h_1}+w_{12}*Out_{h_2}$	$\widehat{y_3}=Out_{o_3}=sigmoid\left(In_{o_3}\right)$

一旦神经网络在当前参数下给出了预测结果$(\widehat{y_1}, \widehat{y_2}, \widehat{y_3})$后，通过均方误差损失函数来计算模型预测值与真实值$(y_1, y_2, y_3)$之间误差，记为$loss=\frac{1}{3}\sum_{i=1}^{3}\left(\hat{y}_i-y_i\right)^2$。

接下来通过梯度下降和误差反向传播方法，沿着损失函数梯度的反方向来更改参数变量取值，使得损失函数快速下降到其最小值。由于损失函数对w_7 ~ w_{12}的偏导计算相似，在此以w_7为例来介绍如何更新w_7这一参数取值。记损失函数为$\mathcal{L}(w)$，$\mathcal{L}(w)=\frac{1}{3}\sum_{i=1}^{3}\left(\hat{y}_i-y_i\right)^2$。损失函数$\mathcal{L}$对参数$w_7$的偏导可如下计算：

$$\delta_7=\frac{\partial\mathcal{L}}{\partial w_7}=\frac{\partial\mathcal{L}}{\partial\widehat{y_1}}\frac{\partial\widehat{y_1}}{\partial In_{o_1}}\frac{\partial In_{o_1}}{\partial w_7}$$

上述公式中每一项结果如下：

$$\frac{\partial \mathcal{L}}{\partial \widehat{y_1}} = \widehat{y_1} - y_1$$

$$\frac{\partial \widehat{y_1}}{\partial In_{o_1}} = \widehat{y_1} * \left(1 - \widehat{y_1}\right)$$

$$\frac{\partial In_{o_1}}{\partial w_7} = Out_{h_1}$$

在得到$w_7 \sim w_{12}$相对于损失函数的偏导结果后，利用链式求导法则，可计算得到损失函数相对于$w_1 \sim w_6$的偏导结果。下面以w_1为例介绍损失函数$\mathcal{L}$对w_1的偏导δ_1。

$$\begin{aligned}
\delta_1 &= \frac{\partial \mathcal{L}}{\partial w_1} \\
&= \frac{\partial \mathcal{L}}{\partial \widehat{y_1}} \frac{\partial \widehat{y_1}}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial \widehat{y_2}} \frac{\partial \widehat{y_2}}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial \widehat{y_3}} \frac{\partial \widehat{y_3}}{\partial w_1} \\
&= \frac{\partial \mathcal{L}}{\partial \widehat{y_1}} \frac{\partial \widehat{y_1}}{\partial In_{o_1}} \frac{\partial Out_{h_1}}{\partial In_{h_1}} \frac{\partial In_{h_1}}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial \widehat{y_2}} \frac{\partial \widehat{y_2}}{\partial In_{o_2}} \frac{\partial In_{o_2}}{\partial Out_{h_1}} \frac{\partial Out_{h_1}}{\partial In_{h_1}} \frac{\partial In_{h_1}}{\partial w_1} + \\
&\quad \frac{\partial \mathcal{L}}{\partial \widehat{y_3}} \frac{\partial \widehat{y_3}}{\partial In_{o_3}} \frac{\partial In_{o_3}}{\partial Out_{h_1}} \frac{\partial Out_{h_1}}{\partial In_{h_1}} \frac{\partial In_{h_1}}{\partial w_1} \\
&= \left(\frac{\partial \mathcal{L}}{\partial \widehat{y_1}} \frac{\partial \widehat{y_1}}{\partial In_{o_1}} \frac{\partial In_{o_1}}{\partial Out_{h_1}} + \frac{\partial \mathcal{L}}{\partial \widehat{y_2}} \frac{\partial \widehat{y_2}}{\partial In_{o_2}} \frac{\partial In_{o_2}}{\partial Out_{h_1}} + \frac{\partial \mathcal{L}}{\partial \widehat{y_3}} \frac{\partial \widehat{y_3}}{\partial In_{o_3}} \frac{\partial In_{o_3}}{\partial Out_{h_1}} \right) \frac{\partial Out_{h_1}}{\partial In_{h_1}} \frac{\partial In_{h_1}}{\partial w_1} \\
&= \left(\delta_7 + \delta_8 + \delta_9\right) \frac{\partial Out_{h_1}}{\partial In_{h_1}} \frac{\partial In_{h_1}}{\partial w_1}
\end{aligned}$$

在计算得到所有参数相对于损失函数$\mathcal{L}$的偏导结果后，利用梯度下降算法，通过$w_i^{new} = w_i - \eta \times \delta_i$来更新参数取值。然后不断迭代，直至模型参数收敛，此时损失函数减少到其最小值。

6.3 卷积神经网络

在前馈神经网络中，输入层的输入数据直接与第一个隐藏层中所有神经元相互连接。如果输入数据是一幅图像，需要把灰度图像（二维矩阵）或彩色图像（三维矩阵）转换为向量形式。则给定一幅分辨率$1\,000 \times 1\,000$的灰度图像，输入数据为一个1 000 000维的向量，如果输入数据与第一个隐藏层中所有神经元均相连，且第一个隐藏层维度和

输入维度一致，则输入层到第一个隐藏层之间待训练参数数目为10^{12}个，模型参数数量如此巨大不仅会占用大量计算机内存，同时也使神经网络模型变得难以训练收敛。因此，对于图像这样的数据，不能直接将所构成的像素点向量与前馈神经网络神经元相连。

1959年，David Hubel和Torsten Wiesel发现了人脑“视觉系统的信息处理”这一机制，即可视皮层对外界信息是分级感受的。受这一发现启发，1980年Kunihiko Fukishima将神经科学所发现的结构进行了计算机模拟，提出通过级联方式（cascade，即逐层滤波）来实现一种满足平移不变性的网络Neocognitron，这就是卷积神经网络的前身[14, 15]。20世纪90年代，LeCun等人，设计了一种被称为LeNet-5的卷积神经网络用于手写体识别，初步确立了卷积神经网络的基本结构。

如今，虽然针对不同应用场景的卷积神经网络结构已变得愈发复杂，性能也变得更加强健，但是究其本质，这些复杂网络结构仍是以卷积操作与池化操作为核心构建而成。本节对卷积神经网络中的卷积操作、池化操作和网络优化等内容做详细介绍。

6.3.1 卷积计算

图像中像素点具有很强的空间依赖性，卷积（convolution）就是针对像素点的空间依赖性来对图像进行处理的一种技术。

在图像卷积计算中，需要定义一个卷积核（kernel）。卷积核是一个二维矩阵，矩阵中数值为对图像中与卷积核同样大小的子块像素点进行卷积计算时所采用的权重。

如图6.9所示，给定一个权重分别为w_i（$1 \leqslant i \leqslant 9$）、大小为$3 \times 3$的卷积核以及一个$5 \times 5$大小灰度图像。该卷积核对图像的卷积操作就是分别以图像中g、h、i、l、m、n、q、r和s所在像素点位置为中心，形成一个3×3大小的图像子块区域，然后用卷积核所定义权重w_i对图像子块区域内每个像素点进行加权累加，所得结果即为图像子块区域中心像素点被卷积后（即滤波后）的结果。滤波也可视为在给定卷积核权重前提下，记住了邻域像素点之间的若干特定空间模式，而忽略了某些空间模式。

这里以像素点g为中心像素点进行滤波处理为例，以其为中心所形成的3×3大小子块中包含了其邻域的a、b、c、f、h、k、l、m 8个像素点，用卷积核中的权重w_i对这些像素点（含像素点g）进行加权累加，所得到

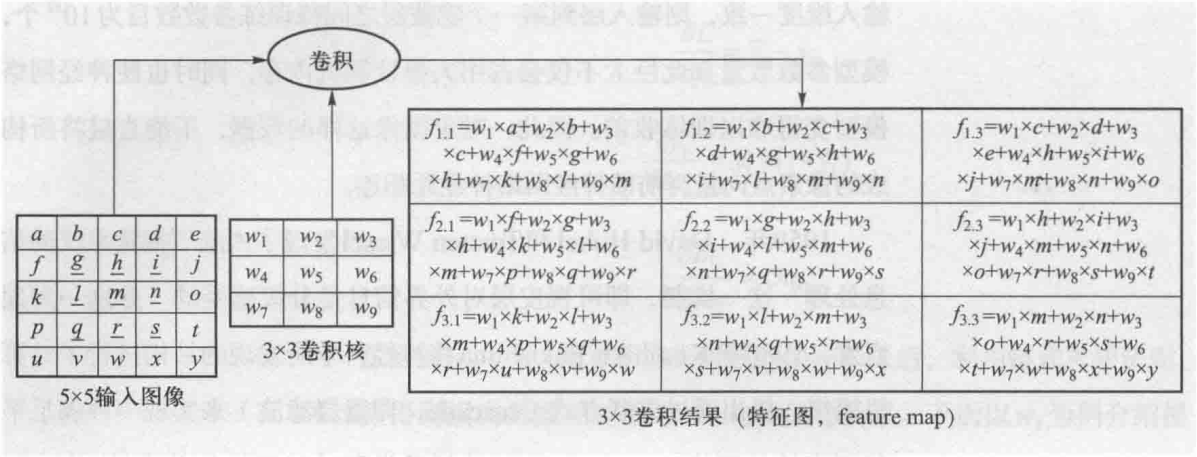


图6.9 图像卷积例子

的结果即为像素点g的卷积滤波结果，即$f_{1.1}=w_1\times a+w_2\times b+w_3\times c+w_4\times f+w_5\times g+w_6\times h+w_7\times k+w_8\times l+w_9\times m$。

同理$f_{1.2}$、$f_{1.3}$、$f_{2.1}$、$f_{2.2}$、$f_{2.3}$、$f_{3.1}$、$f_{3.2}$和$f_{3.3}$作为像素点h、i、l、m、n、q、r和s的卷积滤波结果，如图6.9所示，卷积滤波结果在卷积神经网络中被称为特征图（feature map）。

在图像卷积计算中，给定某个中心像素点，通过卷积核给其周围像素点赋予不同权重，来对该像素点值进行调整。卷积这一思想利用了图像中像素点存在“空间依赖度”的特点，即相邻像素点之间具有很强的相关性，而相隔较远的像素点之间的相关性很弱。卷积过程中需要留意如下三点：① 由于无法以被卷积图像边界像素点为中心，形成卷积核大小的图像子块区域，因此边界像素点无法参与卷积计算，在图6.9中，左图中第一行、第五行、第一列和第五列所包含像素点无法参与卷积计算；② 图像卷积计算对图像进行了下采样（down sampling）操作。如图6.9中被卷积图像分辨率为5×5，卷积后图像的分辨率为3×3，这样被卷积图像就被“约减”了，对卷积得到的图像结果不断卷积滤波，则原始图像就会被“层层抽象、层层约减”，从而使得蕴涵在图像中的重要信息“显山露水”；③ 卷积核中的权重系数w_i是通过数据驱动机制学习得到，其用来捕获图像中某像素点及其邻域像素点所构成的特有空间模式，一旦从数据中学习得到权重系数，这些权重系数就刻画了图像中像素点构成的空间分布的不同模式。

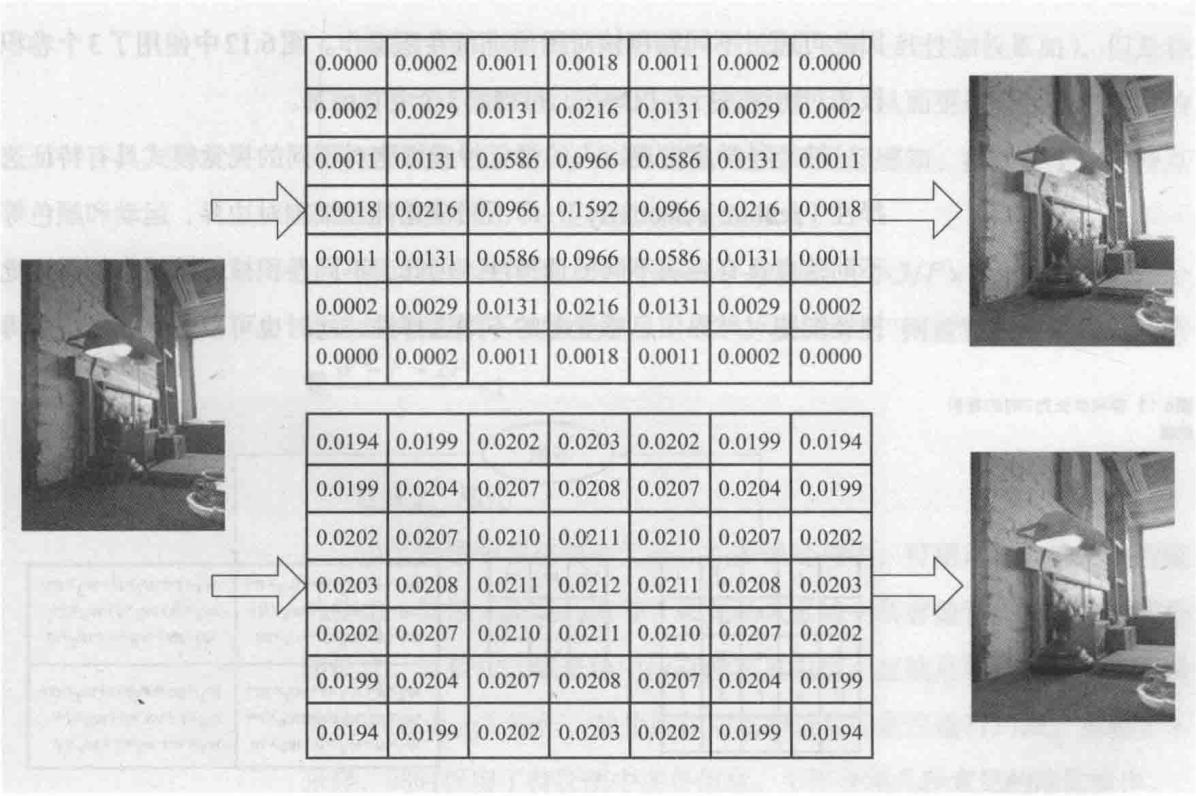
图像卷积操作符合David Hubel和Torsten Wiesel所揭示的“视觉系统信息分层处理”这一机制，即视觉感知是由低层细胞到高层细胞对原始输入信息不断抽象完成，更高层细胞拥有更高级的感受野，并且对一

些偏移、旋转等变换保持不变性。例如，无论如何旋转手机，给出不同旋转状态的手机图像，人脑总是可以从这些不同旋转角度的图像中辨认出手机这一语义对象。

图6.10给出了同一幅图像经过两个不同的7×7高斯卷积核进行卷积滤波的结果。高斯卷积核中的权重系数具有如下特点：卷积核中心位置权重系数的取值最大，从中心位置向四周边缘扩散过程中，权重系数的取值逐渐减少。在高斯卷积核中，所有权重系数累加之值为1。从图6.10可见，如果卷积核中心位置的权重系数越小且与其他卷积权重系数差别越小，则卷积所得到图像滤波结果越模糊，这称为图像平滑操作。在图6.10中，右下图与右上图相比，显得模糊，其原因在于右下图所对应的卷积核权重系数相差不大，使得像素点所对应卷积滤波结果之间彼此相差不大。可以想象，在卷积过程中，被卷积图像子块中心位置像素点所对应的卷积权重系数小且与其他卷积权重系数差别不大，则意味着该像素点色度值与其近邻像素点色度值之间差距在卷积后将减少，该像素点与其近邻像素点之间的色度值也将趋同，从而使得图像变“模糊”，这很好利用了图像相邻像素点之间存在“空间依赖”的特点。

图6.10 同一幅图像被两个不同卷积核计算结果

在卷积操作时，经常会采用Padding（填充）和Striding（步长）这



两种方法。

Padding：在前面介绍中可知，位于被卷积图像边缘位置的像素点无法形成一个以其为中心、与卷积核大小一致的图像子块区域，因此无法对边缘位置的像素点进行卷积滤波。为了使边缘位置的图像像素点也参与卷积滤波，Padding技术被提出。Padding在边缘像素点周围填充“0”（即0填充），使得可以以边缘像素点为中心而形成与卷积核同样大小的图像子块区域。注意，在这种Padding机制下，卷积后的图像分辨率将与卷积前图像分辨率一致，不存在下采样。

Striding：在卷积操作时，通常希望被卷积所得图像分辨率与卷积前图像分辨率相比会逐渐减少，即图像被约减。Striding通过改变卷积核在被卷积图像中移动步长大小来跳过一些像素，进行卷积滤波。当Stride = 1时，卷积核滑动跳过1个像素，这是最基本的单步滑动，也是标准的卷积模式，如图6.10中的例子所示。Stride = k表示卷积核移动跳过的步长是k。对于图6.10中的卷积操作，如果将步长改变为2，则可得到一个2 × 2的卷积滤波图像结果，如图6.11所示。

在实际操作中，给定一幅图像，希望通过不同权重系数的不同卷积核对原始图像进行卷积滤波，以捕获图像中相邻像素点所构成的不同空间模式，或者说用不同卷积核来感应原始图像中不同侧面的重要信息，因此可通过不同卷积核对图像进行卷积操作。图6.12中使用了3个卷积核来对图像进行卷积操作，可得到3个卷积结果。

神经科学家发现，人的视觉神经细胞对不同的视觉模式具有特征选择性（feature selectivity），即不同视觉神经细胞对边界、运动和颜色等不同信息具有强弱不同的选择性。因此，不同卷积核可被用来刻画视觉神经细胞对外界信息感受时的不同选择性。同时也可以看到，卷积所得

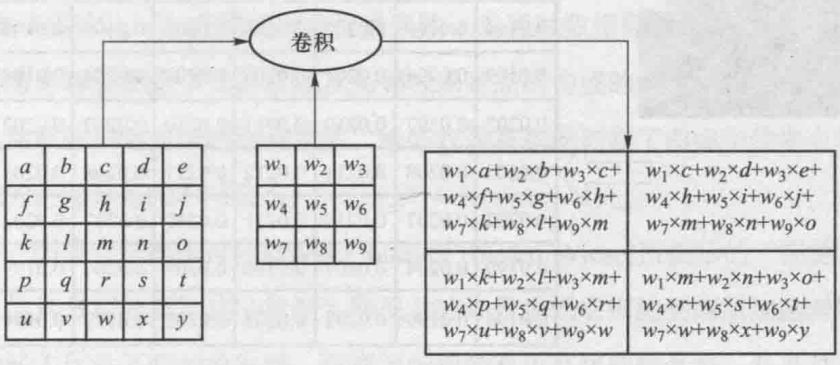


图6.11 卷积步长为2时的卷积结果

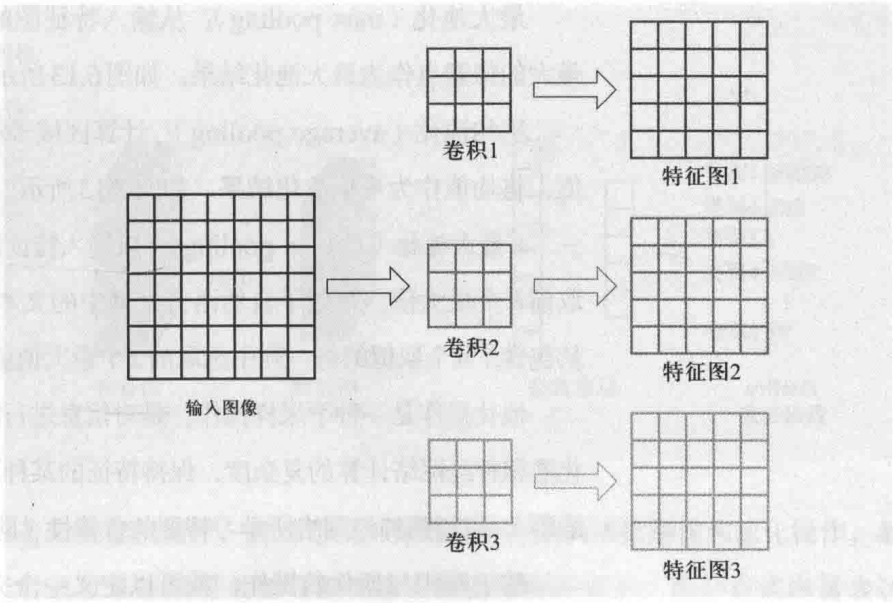


图6.12 3个卷积核对图像进行卷积示意图

结果中，每个输出点的取值仅依赖于其在输入图像中该点及其邻域区域点的取值，与这个区域之外的其他点取值均无关，该区域被称为感受野（receptive field），正所谓“管中窥豹、见微知著”。在卷积神经网络中，感受野是卷积神经网络每一层输出的特征图（feature map）上的像素点在输入图像上映射的区域大小。也就是说，感受野是特征图上一个点对应输入图像上的区域。

值得注意的是，卷积操作是一个线性变换（线性加权累加），但是卷积操作中的权重参数可学习，可被重复使用，从而更适合数据中蕴含的实际空间模式且减少了参数总数，这种“局部感知、参数共享”的特点减少了网络参数，一定程度地防止过拟合。

假设被卷积图像大小为$w \times w$，卷积核大小为$F \times F$，上下左右四个边缘填充像素行/列数为$P = [F/2]$，步长为S，则被卷积结果的分辨率是$\frac{W-F+2P}{S}+1$。

6.3.2 池化

由于图像中存在较多冗余，在图像处理中，可用某一区域子块的统计信息（如最大值或均值等）来刻画该区域中所有像素点呈现的空间分布模式，以替代区域子块中所有像素点取值，这就是卷积神经网络中的池化（pooling）操作。池化操作对卷积结果特征图进行约减，实现了下采样，同时保留了特征图中主要信息。下面介绍几种常见的池化操作。

最大池化（max pooling）：从输入特征图的某个区域子块中选择值最大的像素点作为最大池化结果。如图6.13所示。

平均池化（average pooling）：计算区域子块包含的所有像素点的均值，将均值作为平均池化结果。如图6.13所示。

K最大池化（K-max pooling）：对输入特征图的区域子块中的像素点取前K个最大值，常用于自然语言处理中的文本特征提取。如图6.14中，从包含了4个取值的每一列中选取前2个最大值就得到了K最大池化结果。

池化操作是一种下采样操作，是对信息进行约减和抽象的过程，可简化卷积神经网络计算的复杂度，保持特征的某种不变性（选择、平移和伸缩等），以增强神经网络所学习特征的鲁棒性（防止过拟合与过学习）。

基于卷积与池化的操作，就可以定义一个完整的卷积神经网络。在图6.15中，对于$32\times32\times3$的彩色图像，先用6个$5\times5\times3$的卷积核对其进行卷积操作（步长为1、无填充），得到6个28×28的卷积结果（特征图），然后以2×2大小进行池化，得到了6个14×14大小的特征图。最后通过若干个全连接层得到原始图像的向量表达，再对这个向量表达通过softmax函数进行分类识别，得到输入图像隶属于n个类别的概率。

对于输入的海量标注数据，通过多次迭代训练，卷积神经网络经过

图6.13 最大池化与平均池化操作示意图

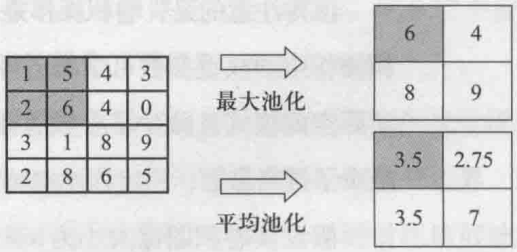
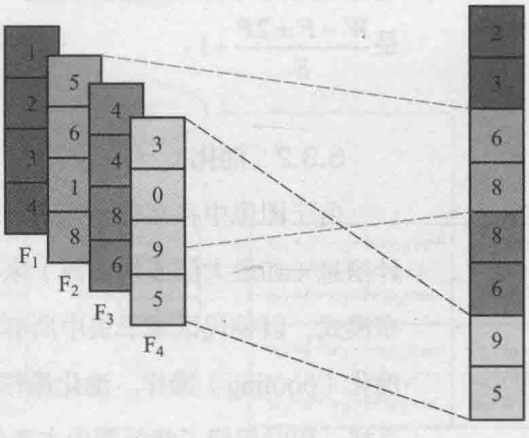


图6.14 K最大池化操作示意图



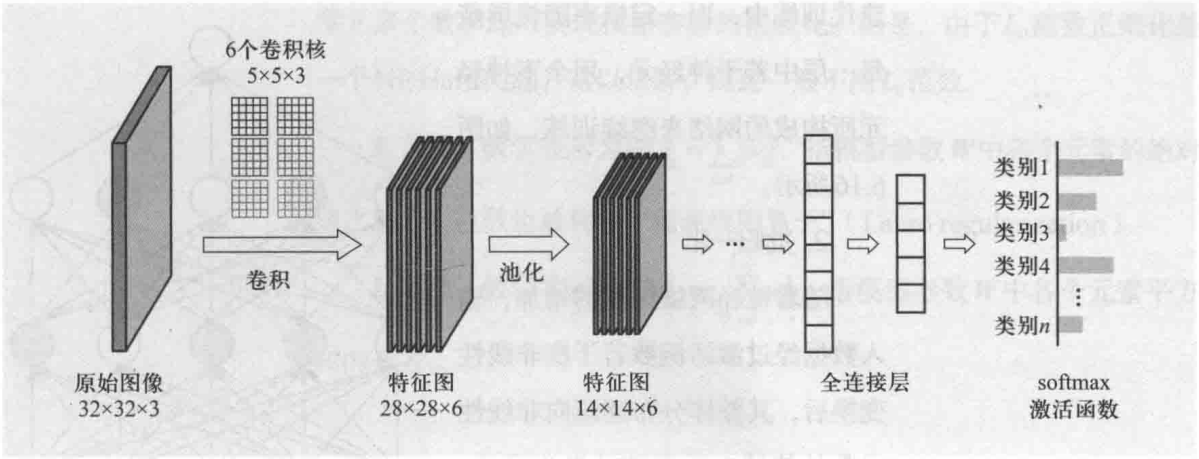


图6.15 基于卷积神经网络的图像分类示意图

若干次卷积操作，再对卷积所得结果进行激活函数操作和池化操作，最后通过全连接层来学习得到输入数据的特征表达，即分布式向量表达（distributed vector representation）。基于该向量表达，就可以完成诸如图像分类（如图6.15所示）、视觉对象定位等任务。

卷积操作提取了图像中的局部模式，所得不同特征图是对图像中像素点局部空间模式的不同角度描述，这些所提取的不同局部空间模式所对应的激活系数在全连接层被组合起来，用来进行识别和分类，全连接层起到了“部分组成整体”的作用。

如果一个神经网络中的激活函数采用sigmoid或ReLU等函数，则神经网络的参数就是卷积核权重系数和全连接系数等参数，这些参数都可以通过数据驱动方式学习得到。一般地，全连接层参数较多，可占整个网络参数的80% ~ 90%，优化这些参数会耗费过多的资源，甚至产生惊人的碳排放[6]。为了进一步减少参数学习，近期一些性能优异的网络模型如ResNet和GoogLeNet等均用全局平均池化（global average pooling，GAP）取代全连接层来融合学到的深度特征。

6.3.3 神经网络正则化

深度神经网络结构复杂、参数众多，很容易造成过拟合（over-fitting）。为了缓解神经网络在训练过程中出现的过拟合问题，需要采取一些正则化技术来提升神经网络的泛化能力（generalization）。

1. Dropout

Dropout指在训练神经网络过程中随机丢掉一部分神经元来减少神经网络复杂度，从而防止过拟合。Dropout的实现方法很简单：在每次

迭代训练中，以一定概率随机屏蔽每一层中若干神经元，用余下神经元所构成的网络来继续训练，如图6.16所示。

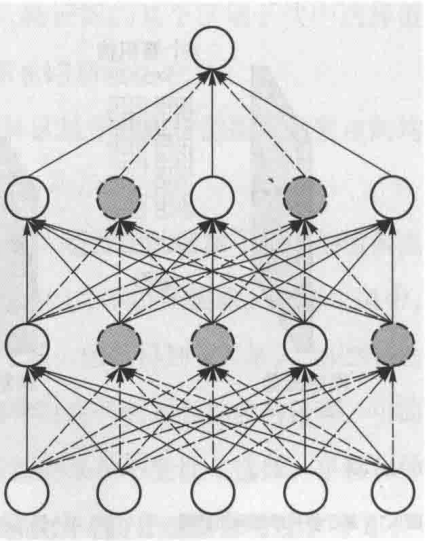


图6.16 使用Dropout的神经网络模型

2. 批归一化

随着神经网络深度的增加，输入数据经过激活函数若干次非线性变换后，其整体分布逐渐向非线性函数的值域上下限两端发生偏移。例如，对于sigmoid函数，它所接收的输入往往是数值较大的正数或者负数，经过sigmoid非线性变换后，结果大多偏向于1或者-1。在1或-1附近，sigmoid函数的梯度较小，导致在反向传播时，靠近输入端容易出现梯度消失问题，致使网络收敛很缓慢。

批归一化（batch normalization）就是通过规范化手段，把神经网络每层中任意神经元的输入值分布改变成均值为0、方差为1的标准正态分布，把偏移较大的分布强制映射为标准正态分布。经过批归一化的处理，激活函数的输入值被映射到非线性函数梯度较大的区域，使得梯度变大从而克服梯度消失问题，进而提高收敛速度。

3. L_1和L_2正则化

对于具有n个训练数据$\{(x_1, y_1), \cdots, (x_n, y_n)\}$的神经网络，其中$(x_i, y_i)$ $(1 \leqslant i \leqslant n)$分别为输入样本及其对应的标签。加入正则化项后，神经网络的损失函数一般可如下表示：

$$\min \frac{1}{n} \sum_{i=1}^{n} \underbrace{Loss\left(y_i, f\left(\boldsymbol{W}, x_i\right)\right)}_{\text{损失函数}} + \underbrace{\lambda}_{\text{正则化权重}} \times \underbrace{\boldsymbol{\Phi}(\boldsymbol{W})}_{\text{正则化项}}$$

其中，$f(\boldsymbol{W}, x_i)$表示参数为$\boldsymbol{W}$的神经网络对输入x_i的预测、$\boldsymbol{\Phi}(\boldsymbol{W})$为正则化项（又称惩罚项）、$\lambda$为正则项权重。注意，对卷积神经网络而言，$\boldsymbol{W}$表示神经元连接权重和卷积核权重等需要学习的参数。

正则化项$\boldsymbol{\Phi}(\boldsymbol{W})$一般用模型参数$\boldsymbol{W}$的范数形式来表示。假设神经网络模型中参数数目为$N$，则参数的范数形式主要有以下几种。

L_0范数：数学表示为$\|\boldsymbol{W}\|_0 = \sum_{i=1}^{N} \mathbb{I}[w_i \neq 0]$，其中$\mathbb{I}$是指示函数，若括号内表达式为真，函数值为1，反之为0。所以L_0范数指模型参数$\boldsymbol{W}$中非

零元素个数。L_0可实现模型参数的稀疏化。但是，由于L_0范数正则化是一个NP Hard问题，难以求解，因此一般不用L_0范数。

L_1范数：数学表示为$\|\boldsymbol{W}\|_1=\sum_{i=1}^{N}|w_i|$，指模型参数$\boldsymbol{W}$中各个元素的绝对值之和。$L_1$范数也被称为“稀疏规则算子”（Lasso regularization）。

L_2范数：数学表示为$\|\boldsymbol{W}\|_2=\sqrt{\sum_{i=1}^{N}w_i^2}$，指模型参数$\boldsymbol{W}$中各个元素平方和的开方。

6.4 循环神经网络

循环神经网络（recurrent neural network，RNN）是一类处理序列数据（如文本句子、视频帧等）时所采用的网络结构。先前介绍的前馈神经网络或卷积神经网络所需要处理的输入数据一次性给定，难以处理存在前后依赖关系的数据。

1990年，Jeffrey L. Elman提出循环神经网络的最初概念框架[4]，循环神经网络的本质是希望模拟人所具有的记忆能力，在学习过程中记住部分已经出现的信息，并利用所记住的信息影响后续结点输出。循环神经网络在自然语言处理，如语音识别、情感分析、机器翻译等领域有重要应用。

6.4.1 循环神经网络模型

图6.17展示了循环神经网络的结构图。由“循环”两字可知，循环神经网络在处理数据过程中构成了一个循环体。对于一个序列数据，在每一时刻t，循环神经网络单元会读取当前输入数据$\boldsymbol{x}_t$和前一时刻输入数据$\boldsymbol{x}_{t-1}$所对应的隐式编码结果$\boldsymbol{h}_{t-1}$，一起生成t时刻的隐式编码结果$\boldsymbol{h}_t$。接着将$\boldsymbol{h}_t$后传，去参与生成$t+1$时刻输入数据$\boldsymbol{x}_{t+1}$的隐式编码$\boldsymbol{h}_{t+1}$。如此循环处理，直至该序列数据被处理完毕。

可见，时刻t所得到的隐式编码$\boldsymbol{h}_t$是由上一时刻隐式编码$\boldsymbol{h}_{t-1}$和当前输入$\boldsymbol{x}_t$共同参与生成的，这可认为隐式编码$\boldsymbol{h}_{t-1}$已经“记忆”了t时刻之前的时序信息，或者说前序时刻信息影响了后续时刻信息的处理。与前馈神经网络和卷积神经网络在处理时需要将所有数据一次性输入不同，

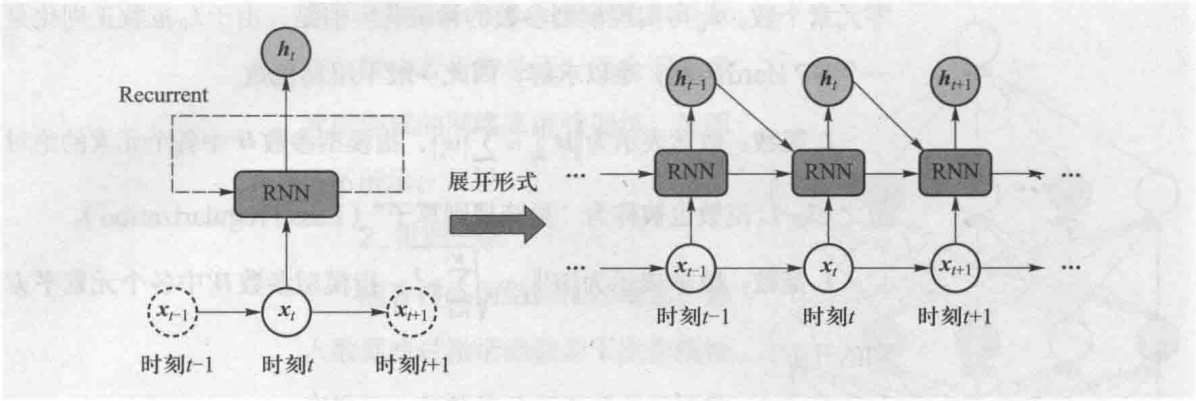


图6.17 循环神经网络结构及其展开形式示意图

循环神经网络在每一时刻都有数据输入且结合前一时刻所得结果进行计算，这体现了循环神经网络可刻画序列数据时序依赖这一重要特点。

在时刻t，一旦得到当前输入数据$\boldsymbol{x}_t$，循环神经网络会结合前一时刻$t-1$得到的隐式编码$\boldsymbol{h}_{t-1}$，如下产生当前时刻隐式编码$\boldsymbol{h}_t$：

$$\boldsymbol{h}_t = \Phi\left(U \times \boldsymbol{x}_t + W \times \boldsymbol{h}_{t-1}\right)$$

这里$\Phi(\cdot)$是激活函数，一般可为sigmoid或者tanh激活函数，使模型能够忘掉无关的信息，同时更新记忆内容。U与W为模型参数。从这里可看出，当前时刻的隐式编码输出$\boldsymbol{h}_t$不仅仅与当前输入数据$\boldsymbol{x}_t$相关，与网络已有的“记忆”$\boldsymbol{h}_{t-1}$也有着密不可分的联系。

为了更加直观展示在循环神经网络中前序时刻信息被“记住”来影响当前时刻信息编码，这里对上述公式进行如下变化：

$$\begin{aligned}\boldsymbol{h}_t &= \Phi\left(U \times \boldsymbol{x}_t + W \times \boldsymbol{h}_{t-1}\right) = \Phi\left(U \times \boldsymbol{x}_t + W \times \Phi\left(U \times \boldsymbol{x}_{t-1} + W \times \boldsymbol{h}_{t-2}\right)\right) \\ &= \Phi\left(U \times \underbrace{\boldsymbol{x}_t}_{t\text{时刻输入}} + W \times \Phi\left(U \times \underbrace{\boldsymbol{x}_{t-1}}_{t-1\text{时刻输入}} + W \times \Phi\left(U \times \underbrace{\boldsymbol{x}_{t-2}}_{t-2\text{时刻输入}} + \cdots\right)\right)\right)\end{aligned}$$

从上式可见，当前时刻编码中，的确均包含了历史信息，这也说明了循环神经网络能够记忆的原因，使得一些需要记忆历史过往信息的任务能够被循环神经网络有效处理。

实际上，按照时间将循环神经网络展开后，如图6.17所示，可以得到一个和前馈神经网络相似的网络结构。这个网络结构可利用反向传播算法和梯度下降算法来训练模型参数，这种训练方法称为“沿时间反向传播算法（backpropagation through time，BPTT）”。由于循环神经网络每个时刻都有一个输出，所以在计算循环神经网络的损失时，通常需要将所有时刻（或者部分时刻）上的损失进行累加。

下面以自然语言处理为例解释循环神经网络是如何处理时序数据的。假设希望将自然语言句子进行分类，分为饮食、体育、文化等不同类别。首先，对文本语句进行分词操作，如图6.18所示，循环神经网络在读入每个单词后，产生对应单词的隐式编码，并利用其来参与后续单词的隐式编码。一旦得到了每个单词的隐式编码后，通常可如下得到一个句子的向量编码：① 考虑到句子中所有单词的信息均被后向传递，因此将最后一个单词的输出作为整个句子向量编码表示；② 将每个单词的隐式编码进行加权平均，将加权平均结果作为句子的向量编码表示。这样，一旦得到了句子的向量编码表示，就可以利用某个分类器，对句子的类别进行判断。

在图6.18的示意中，“南北”“存在差异”“南甜北咸”“南方人”和“甜豆花”等单词之间存在前后依赖，这种依赖会被有效利用起来，得到各自单词的隐式编码以及句子的向量表达。

在循环神经网络中，根据输入序列数据与输出序列数据中所包含“单元”的多寡，循环神经网络可以实现“多对多”（即输入和输出序列数据中包含多个单元，常用于机器翻译）、“多对一”（即输入序列数据包含多个单元，输出序列数据只包含一个单元，常用于文本的情感分类）和“一对多”（即输入序列数据包含一个单元，输出序列数据包含多个单元，常用于图像描述生成）三种模式。如图6.19所示。

循环神经网络虽然具有一定的记忆能力，但当彼此相关信息在序列中相距较远时，循环神经网络无法捕捉到数据之中的时序依赖关系。除此之外，当输入序列过长时，循环神经网络也容易出现梯度消失（gradient vanishing）或者梯度爆炸（gradient exploding）的问题。

图6.20以词性标注为例给出了一个输入序列数据(x_1, …, x_{t-1}, x_t, x_{t+1}, …, x_T)被循环神经网络处理的示意图。在图6.20中，时刻t输入数

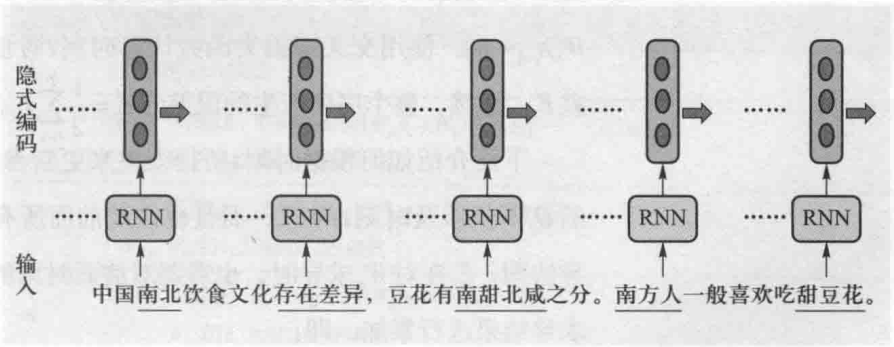


图6.18 循环神经网络中的应用示例

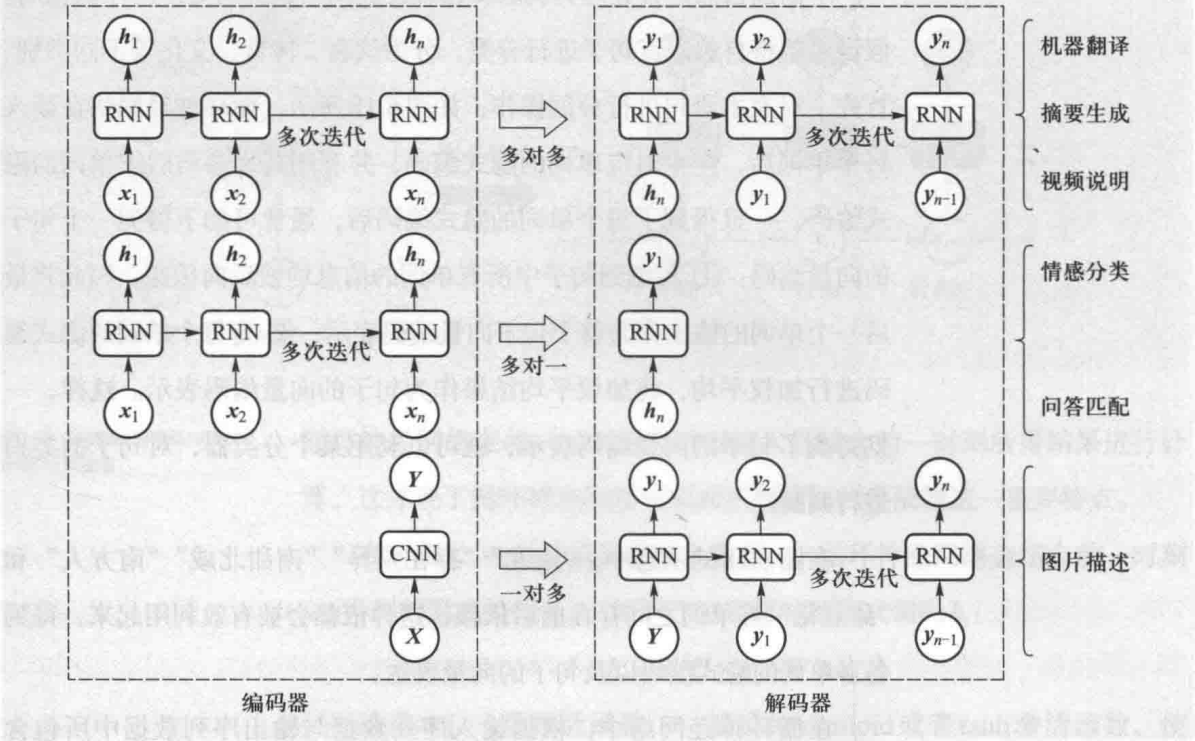


图6.19 输入和输出不同情况下循环神经网络结构示例

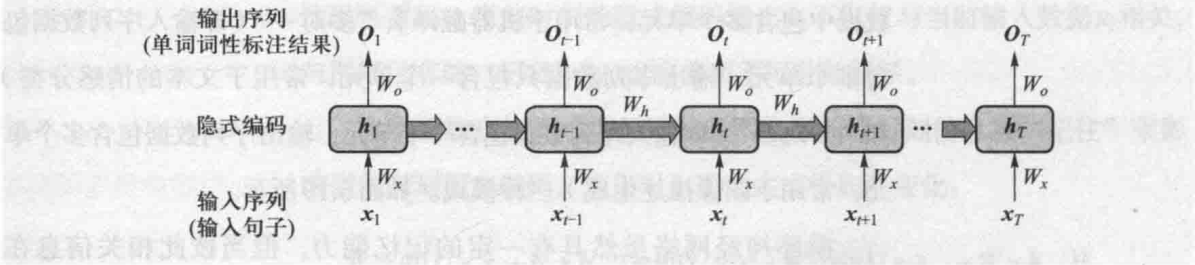


图6.20 循环神经网络中梯度传递示意图

据$\boldsymbol{x}_t$（文本中的单词）的隐式编码为$\boldsymbol{h}_t$，其真实输出为$\boldsymbol{y}_t$（单词词性），模型预测$\boldsymbol{x}_t$的词性是$\boldsymbol{O}_t$。参数W_x将$\boldsymbol{x}_t$映射为隐式编码$\boldsymbol{h}_t$，参数W_o将$\boldsymbol{h}_t$映射为预测输出$\boldsymbol{O}_t$、$\boldsymbol{h}_{t-1}$通过参数W_h参与$\boldsymbol{h}_t$的生成。图中W_x、W_o和W_h是复用参数（即同名参数在各时刻取值相同，也就是其值在各时刻被重复使用）。

假设时刻t的隐式编码$\boldsymbol{h}_t$通过如下计算得到：$\boldsymbol{h}_t=\tanh\left(W_x\boldsymbol{x}_t+W_h\boldsymbol{h}_{t-1}+b\right)$。使用交叉熵损失函数计算时刻$t$时预测输出与实际输出的误差$E_t$。显然，整个序列产生的误差为$E=\frac{1}{2}\sum_{t=1}^{T}E_t$。

下面介绍如何根据时刻t所得误差来更新参数W_x。在时刻t计算所得的$\boldsymbol{O}_t$不仅涉及时刻t的W_x，而且也涉及前面所有时刻的W_x，按照链式求导法则，E_t在对W_x求导时，也需要对前面时刻的W_x依次求导，然后再将求导结果进行累加，即：

$$\frac{\partial E_t}{\partial W_x}=\sum_{i=1}^{t}\frac{\partial E_t}{\partial O_t}\frac{\partial O_t}{\partial h_t}\left(\prod_{j=i+1}^{t}\frac{\partial h_j}{\partial h_{j-1}}\right)\frac{\partial h_i}{\partial W_x}$$

其中

$$\prod_{j=i+1}^{t}\frac{\partial h_j}{\partial h_{j-1}}=\prod_{j=i+1}^{t}\tanh'\times W_h$$

令$t=3$，则有：

$$\frac{\partial E_3}{\partial W_x}=\frac{\partial E_3}{\partial O_3}\frac{\partial O_3}{\partial h_3}\frac{\partial h_3}{\partial W_x}+\frac{\partial E_3}{\partial O_3}\frac{\partial O_3}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial W_x}+\frac{\partial E_3}{\partial O_3}\frac{\partial O_3}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial W_x}$$

由于tanh函数的导数取值位于0到1区间，对于长序列而言，若干多个0到1区间的小数相乘，会使得参数求导结果很小，引发梯度消失问题。E_t对W_h的求导类似，这里就不列出了。为了解决梯度消失问题，长短时记忆模型（long short-term memory，LSTM）被提出。

6.4.2 长短时记忆网络

与简单的循环神经网络结构不同，长短时记忆网络中引入了内部记忆单元（internal memory cell）和门（gate）结构来对当前时刻输入信息以及前序时刻所生成的信息进行整合和传递。在这里，内部记忆单元中的信息可视为对“历史信息”的累积。

常见的LSTM模型中有输入门（input gate）、遗忘门（forget gate）和输出门（output gate）三种门结构。对于给定的当前时刻输入数据$\boldsymbol{x}_t$和前一时刻隐式编码$\boldsymbol{h}_{t-1}$，输入门、遗忘门和输出门通过各自参数对其编码，分别得到三种门结构的输出$\boldsymbol{i}_t$、$\boldsymbol{f}_t$和$\boldsymbol{o}_t$。

在此基础上，再进一步结合前一时刻内部记忆单元信息$\boldsymbol{c}_{t-1}$来更新当前时刻内部记忆单元信息$\boldsymbol{c}_t$，最终得到当前时刻的隐式编码$\boldsymbol{h}_t$。表6.3描述了长短时记忆网络中所需符号及其含义。

表6.3 长短时记忆网络中所用符号及其含义

符号	内容描述或操作
$\boldsymbol{x}_t$	时刻t的输入数据
$\boldsymbol{i}_t$	输入门的输出：$\boldsymbol{i}_t=\text{sigmoid}\left(W_{xi}\boldsymbol{x}_t+W_{hi}\boldsymbol{h}_{t-1}+b_i\right)$ （W_{xi}、W_{hi}和b_i为输入门的参数）
$\boldsymbol{f}_t$	遗忘门的输出：$\boldsymbol{f}_t=\text{sigmoid}\left(W_{xf}\boldsymbol{x}_t+W_{hf}\boldsymbol{h}_{t-1}+b_f\right)$ （W_{xf}、W_{hf}和b_f为遗忘门的参数）
$\boldsymbol{o}_t$	输出门的输出：$\boldsymbol{o}_t=\text{sigmoid}\left(W_{xo}\boldsymbol{x}_t+W_{ho}\boldsymbol{h}_{t-1}+b_o\right)$ （W_{xo}、W_{ho}和b_o为输出门的参数）

续表

符号	内容描述或操作
$\boldsymbol{c}_t$	内部记忆单元的输出：$\boldsymbol{c}_t = \boldsymbol{f}_t \otimes \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \otimes \tanh\left(W_{xc}\boldsymbol{x}_t + W_{hc}\boldsymbol{h}_{t-1} + b_c\right)$ （W_{xc}、W_{hc}和b_c为记忆单元的参数。输入门$\boldsymbol{i}_t$控制有多少信息流入当前时刻内部记忆单元$\boldsymbol{c}_t$，遗忘门控制上一时刻内部记忆单元$\boldsymbol{c}_{t-1}$中有多少信息可累积到当前时刻内部记忆单元$\boldsymbol{c}_t$）
$\boldsymbol{h}_t$	时刻t输入数据的隐式编码：$\boldsymbol{h}_t = \boldsymbol{o}_t \otimes \tanh\left(\boldsymbol{c}_t\right)$ $= \boldsymbol{o}_t \otimes \tanh\left(\boldsymbol{f}_t \otimes \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \otimes \tanh\left(W_{xc}\boldsymbol{x}_t + W_{hc}\boldsymbol{h}_{t-1} + b_c\right)\right)$ （注：输入门、遗忘门和输出门的信息$\boldsymbol{i}_t$、$\boldsymbol{f}_t$、$\boldsymbol{o}_t$一起参与得到$\boldsymbol{h}_t$）
$\otimes$	两个向量中对应元素按位相乘（element-wise product）。 如：[10 6 3 7] ⊗ [0.2 0.1 0.8 0.5] = [2 0.6 2.4 3.5]

当给定序列数据中当前时刻t的输入数据$\boldsymbol{x}_t$和前一时刻隐式编码$\boldsymbol{h}_{t-1}$时，根据表6.3和图6.21，可按照如下公式来计算内部记忆单元信息和隐式编码。

输入门信息输出：$\boldsymbol{i}_t = \text{sigmoid}\left(W_{xi}\boldsymbol{x}_t + W_{hi}\boldsymbol{h}_{t-1} + b_i\right)$

遗忘门信息输出：$\boldsymbol{f}_t = \text{sigmoid}\left(W_{xf}\boldsymbol{x}_t + W_{hf}\boldsymbol{h}_{t-1} + b_f\right)$

输出门信息输出：$\boldsymbol{o}_t = \text{sigmoid}\left(W_{xo}\boldsymbol{x}_t + W_{ho}\boldsymbol{h}_{t-1} + b_o\right)$

内部记忆单元信息输出：$\boldsymbol{c}_t = \boldsymbol{f}_t \otimes \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \otimes \tanh\left(W_{xc}\boldsymbol{x}_t + W_{hc}\boldsymbol{h}_{t-1} + b_c\right)$

隐式编码输出：$\boldsymbol{h}_t = \boldsymbol{o}_t \otimes \tanh\left(\boldsymbol{c}_t\right)$或者$\boldsymbol{h}_t = \boldsymbol{o}_t \otimes \tanh\left(\boldsymbol{f}_t \otimes \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \otimes \tanh\left(W_{xc}\boldsymbol{x}_t + W_{hc}\boldsymbol{h}_{t-1} + b_c\right)\right)$

输入门、遗忘门和输出门通过各自参数对当前时刻输入数据$\boldsymbol{x}_t$和前一时刻隐式编码$\boldsymbol{h}_{t-1}$处理后，利用sigmoid对处理结果进行非线性映射，因此三种门结构的输出$\boldsymbol{i}_t$、$\boldsymbol{f}_t$和$\boldsymbol{o}_t$值域为(0, 1)。正是由于三个门结构的输出值为位于0到1之间的向量，因此其在信息处理中起到了“调控开关”的“门”作用。三个门结构所输出向量的维数、内部记忆单元的维数和隐式编码的维数均相等。

以输入门$\boldsymbol{i}_t = \text{sigmoid}\left(W_{xi}\boldsymbol{x}_t + W_{hi}\boldsymbol{h}_{t-1} + b_i\right)$为例，在sigmoid函数作用下，假设输入门$\boldsymbol{i}_t$的取值为(0.05, 0.1, 0.8)，则通过$\boldsymbol{i}_t \otimes \tanh\left(W_{xc}\boldsymbol{x}_t + W_{hc}\boldsymbol{h}_{t-1} + b_c\right)$中按位相乘⊗操作，输入门会“阻断”一些信息进行后续操作（如0.05和0.1所对应按位相乘的信息），也会“放入”一些信息进行后续操作（如0.8所对应按位相乘的信息）。当然，如果门的某一位取值为0，表示这一位所对应信息为全闭，禁止信息通过；如果门的某一位取值为1，表示这一位所对应的信息为全开，容许信息全部通过。遗忘门不容许一些信息向后传递，从另一个方面来说也是为了关注需要关注信息。

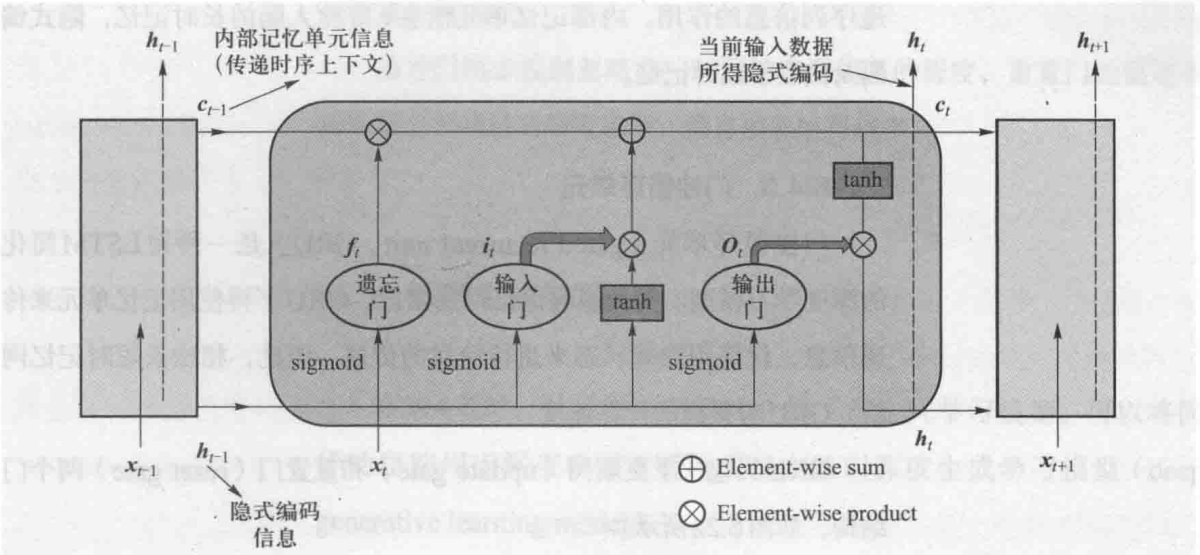


图6.21 长短时记忆网络模型

从$\boldsymbol{c}_t = \boldsymbol{f}_t \otimes \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \otimes \tanh\left(W_{xc}\boldsymbol{x}_t + W_{hc}\boldsymbol{h}_{t-1} + b_c\right)$可以看出，对当前时刻$t$所对应内部记忆单元中信息的调整不仅依赖于当前时刻$t$的输入信息，还涉及遗忘门信息、$t-1$时刻所对应内部记忆单元中信息、输入门信息和$t-1$时刻的隐式编码信息。

由于sigmoid函数的值域在(0, 1)之间，输入门、遗忘门和输出门三种门结构采用sigmoid这一非线性映射函数而起到了信息“控制门”的作用。在内部记忆单元和隐式编码中，使用了tanh函数而没有继续使用sigmoid函数，其原因在于tanh函数的值域为(–1, 1)，使得其在进行信息整合时可起到信息“增（为正）”或“减（为负）”的效果。

在介绍循环神经网络时，不可避免要涉及误差反向传播存在的梯度消失问题，下面来看LSTM是如何避免梯度消失问题的。

对于$\boldsymbol{c}_t = \boldsymbol{f}_t \otimes \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \otimes \tanh\left(W_{xc}\boldsymbol{x}_t + W_{hc}\boldsymbol{h}_{t-1} + b_c\right)$，有如下求导结果的存在：

$$\frac{\partial \boldsymbol{c}_t}{\partial \boldsymbol{c}_{t-1}} = \boldsymbol{f}_t + \frac{\partial \boldsymbol{f}_t}{\partial \boldsymbol{c}_{t-1}} \times \boldsymbol{c}_{t-1} + \cdots$$

可见，$\frac{\partial \boldsymbol{c}_t}{\partial \boldsymbol{c}_{t-1}}$求导的结果至少大于等于$\boldsymbol{f}_t$，即遗忘门的输出结果。如果遗忘门选择保留旧状态，则这一求导结果就接近1（或者接近向量1），使得梯度是存在的，从而避免了梯度消失问题。也就是说，LSTM通过引入门结构，在从t到$t+1$过程中引入加法来进行信息更新，避免了梯度消失问题。

整体来看，内部记忆单元信息$\boldsymbol{c}_t$和隐式编码$\boldsymbol{h}_t$这两种信息起到了传

递序列信息的作用。内部记忆单元信息c_t好比人脑的长时记忆，隐式编码h_t代表了短时记忆。

6.4.3 门控循环单元

门控循环单元（gated recurrent unit，GRU）是一种对LSTM简化的深度学习模型。与长短时记忆网络相比，GRU不再使用记忆单元来传递信息，仅使用隐藏状态来进行信息的传递。因此，相比长短时记忆网络，GRU有更高的计算速度。

GRU只包含了更新门（update gate）和重置门（reset gate）两个门结构，如图6.22所示。

给定当前时刻输入数据$\boldsymbol{x}_t$和前一时刻隐式编码$\boldsymbol{h}_{t-1}$，GRU利用更新门和重置门来输出当前时刻隐式编码，并且通过隐式编码将信息向后传递。记更新门和重置门信息为$\boldsymbol{z}_t$和$\boldsymbol{r}_t$，GRU的信息更新如下。

更新门信息输出：$\boldsymbol{z}_t=\text{sigmoid}\left(W_z\boldsymbol{x}_t+U_z\boldsymbol{h}_{t-1}+b_z\right)$

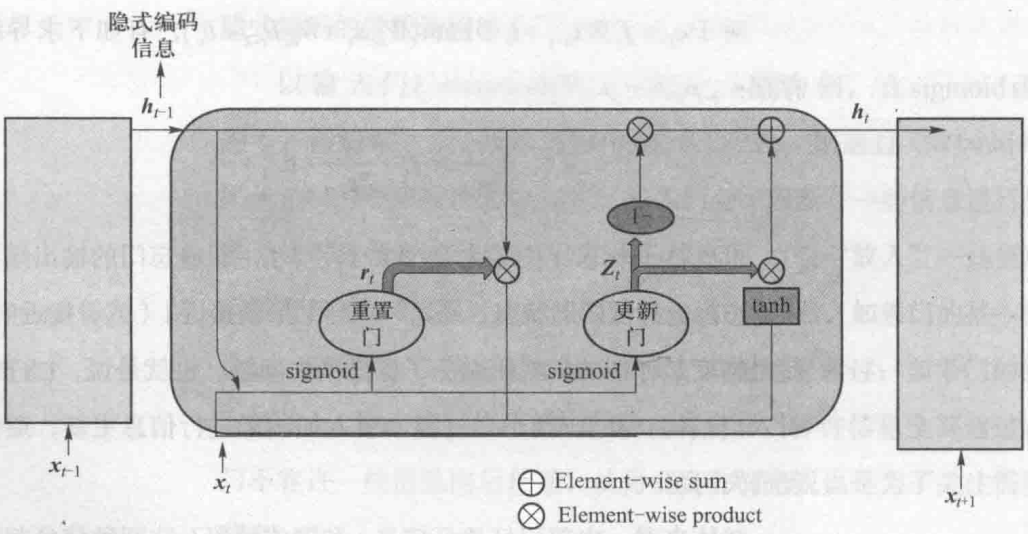
重置门信息输出：$\boldsymbol{r}_t=\text{sigmoid}\left(W_r\boldsymbol{x}_t+U_r\boldsymbol{h}_{t-1}+b_r\right)$

隐式编码输出：$\boldsymbol{h}_t=\left(1-\boldsymbol{z}_t\right)\otimes\boldsymbol{h}_{t-1}+\boldsymbol{z}_t\otimes\tanh\left(W_h\boldsymbol{x}_t+U_h\left(\boldsymbol{r}_t\otimes\boldsymbol{h}_{t-1}\right)+b_h\right)$

与循环神经网络一样，这里的W_z、U_z、b_z、W_r、U_r、b_r、W_h、U_h和b_h都是模型参数，需要通过训练优化。

更新门的作用类似于长短时记忆模型中的遗忘门和输入门。它决定了要忘记哪些信息以及哪些新信息需要被添加，用于控制前一时刻的状态信息被保留到当前状态中的程度，更新门的值越大说明前一时刻的状

图6.22 GRU网络结构图



态信息保留越多。

重置门用于控制忽略前一时刻的状态信息的程度，重置门的值越小说明信息可通过的程度越低，信息被忽略得越多。

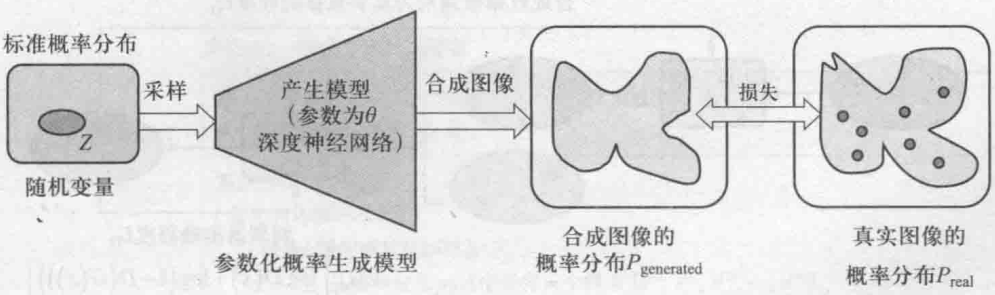
*6.5 深度生成学习

在第4章时，从浅层学习角度介绍了产生式学习模型。可以将同样的思路用于深度学习，所形成的方法叫深度生成学习模型（deep generative learning model）。

在本章之前的介绍中，神经网络模型从数据中提取出高层语义在数据中所蕴含的“模式”，并利用这些模式实现对数据的分类和检测等，这种模型通常称为判别模型，判别模型不关心数据如何生成，它只关心数据蕴含哪些模式以及如何将数据进行分类。与之相对的模型类型被称为生成模型（generative model），生成模型需要学习目标数据的分布规律，以合成属于目标数据空间的新数据。1965年诺贝尔物理学奖得主、加州理工学院物理学教授理查德·费曼（Richard Feynman）所言“不可造也，未能知也（What I cannot create, I do not understand）。如果人工智能算法能够捕捉所处理数据的细节、学习创造逼真的图像和逼真的声音，就能够帮助人工智能更好地理解数据本身。

图6.23给出了生成模型的示意图。以面向图像处理的生成模型为例，生成模型对满足标准概率分布（unit probability distribution）的随机变量z（如高斯分布或均匀分布）进行采样，通过参数化概率生成模型（如参数为θ的深度神经网络）来生成合成图像，接着优化所得合成图像的概率分布$P_{\text{generated}}$与真实图像的概率分布P_{real}之间误差，使得两者分布

图6.23 产生模型的示意图



不断接近，最终提高模型所合成图像的质量。

变分自编码器（variational auto-encoder，VAE）、自回归模型（autoregressive models）与生成对抗网络（generative adversarial network，GAN）等都是深度生成模型的典型代表，本节重点介绍生成对抗网络。

6.5.1 生成对抗网络

生成对抗网络是由Ian Goodfellow等人于2014年提出的一种生成模型[5]，该模型可视为两个神经网络相互竞争的零和游戏（zero-sum game），“以子之矛，陷子之盾”，最终达到纳什均衡。

生成对抗网络由一个生成器（generator，简称G）和一个判别器（discriminator，简称D）组成。GAN的核心是通过生成器和判别器两个神经网络之间的竞争对抗，不断提升彼此水平以使得生成器所生成数据（人工伪造数据）与真实数据相似，如图6.24所示，使判别器无法区分真实数据和生成数据。

生成器接收一个简单的噪声并输出一份合成数据，判别器对所接收数据的真实性进行评价，输出一个0 ~ 1之间数值。在模型初始阶段，生成器所产生的合成数据与真实数据之间差异相当明显，因此判别器可以很容易地分辨出真实数据和合成数据。在随后对抗中，生成器会产生更高质量的合成数据来欺骗判别器，判别器也将会进一步提升自己能力以区分合成数据和真实数据。如此往复数轮之后，生成器将能够习得真实数据的分布规律，并能够生成与真实数据差别细微的数据。

在生成对抗网络中，生成网络G的输入z来自预定义的噪声分布$p_z(z)$，输出$G(z)$属于样本空间。判别网络D被用来判断哪些数据来自真

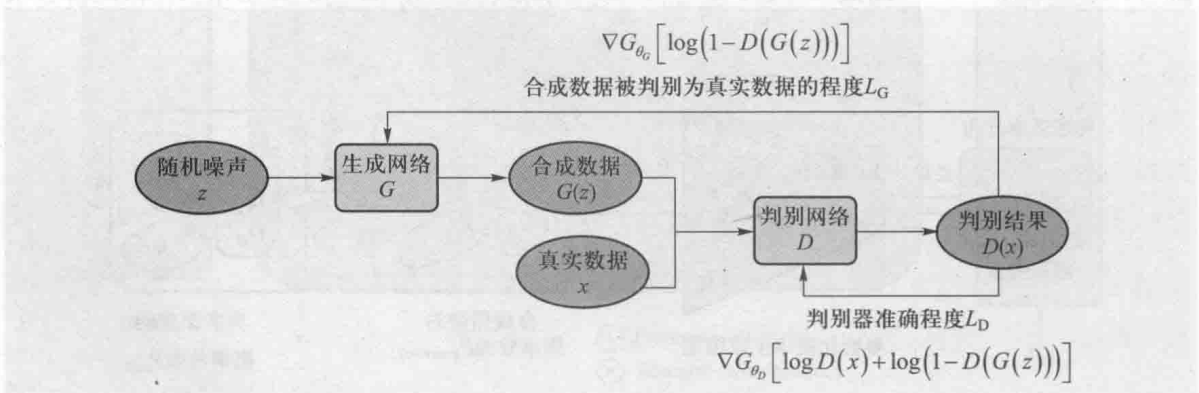


图6.24 生成对抗网络模型示意图

实数据x以及哪些数据是合成数据，输出$D(x)$是一个表示x来自真实数据分布$p_{data}(x)$的概率。GAN训练的过程就是判别网络D和生成网络G分别尝试最大化和最小化如下价值函数$V(D,G)$：

$$\min_G \max_D V(D,G)=\mathbb{E}_{x\sim p_{data}(x)}\left[\log D(x)\right]+\mathbb{E}_{z\sim p_z(z)}\left[\log\left(1-D(G(z))\right)\right]$$

注意到，$V(D, G)$的相反数，即$-V(D, G)$，实际上就等于判别器D的交叉熵损失函数。在训练D时，最大化$V(D, G)$的过程就是最小化判别器损失函数的过程。通过最大化$V(D, G)$，可以使得判别器对于真实数据的输出$D(x)$接近1，对于合成数据的输出$D(G(z))$接近0。生成器G的训练目的则是使得判别器无法正确分辨合成数据，即最大化判别器的损失。因此生成器G要让价值函数$V(D,G)$取值尽可能小。

由于价值函数$V(D, G)$中第一项与G无关，所以对生成器G的训练实际上就是在最小化$\mathbb{E}_{z\sim p_z(z)}\left[\log\left(1-D(G(z))\right)\right]$，即尽量使得所合成数据$G(z)$被判别器$D$判定为真实数据，使得$D(G(z))$的输出接近于1。

在理想的情况下，最终生成网络G所合成的数据分布$G(z)$与真实数据分布$p_{data}(x)$一致，判别器D将完全无法区分合成数据和真实数据，而对真实数据和合成的数据均输出0.5。Goodfellow等人证明了在有充足资源的情况下，GAN模型最终能够收敛于这一理想情况。

实际训练过程中，算法不可能准确计算出$\mathbb{E}_{x\sim p_{data}(x)}\left[\log D(x)\right]$的值，因此算法从真实数据和合成数据中同时抽取$m$个样本来近似计算期望值。

6.5.2 生成对抗网络算法

虽然在理论上判别器D始终能够学到最准确的判别方式，但是如果在每轮对抗中都将D训练到最优情况，将会耗费大量的时间，而且将不可避免地导致D对训练数据产生过拟合现象。因此，在实际实现过程中，每轮对抗中先训练数轮D，再训练一轮G，如此交替地进行。

算法6.1 训练生成对抗网络

输入：神经网络D、G，噪声分布$p_z(z)$，真实数据分布$p_{data}(x)$ 输出：神经网络参数θ_d、θ_g
算法步骤： （每轮训练循环执行。） 训练k轮生成对抗网络如下。 1. 从噪声分布$p_z(z)$中采样m个样本$\{z^{(1)}, z^{(2)}, z^{(3)}, \cdots, z^{(m)}\}$。 2. 从真实数据分布$p_{data}(x)$中采样$m$个样本$\{x^{(1)}, x^{(2)}, x^{(3)}, \cdots, x^{(m)}\}$。

3. 沿梯度上升方向更新判别器参数：

$$\nabla_{\theta_d}\frac{1}{m}\sum_{i=1}^{m}\left[\log D\left(x^{(i)}\right)+\log\left(1-D\left(G\left(z^{(i)}\right)\right)\right)\right]$$

4. 从噪声分布$p_z(z)$中采样m个样本$\{z^{(1)}, z^{(2)}, z^{(3)}, \cdots, z^{(m)}\}$。

5. 沿梯度下降方向更新生成器参数：

$$\nabla_{\theta_g}\frac{1}{m}\sum_{i=1}^{m}\log\left(1-D\left(G\left(z^{(i)}\right)\right)\right)$$

在训练初期，因为生成数据质量较低，判别器D可以轻易将其区分，此时$\log\left(1-D\left(G(z)\right)\right)$无法为生成器$G$提供显著的梯度，因此可以使用$-\log D\left(G(z)\right)$来代替损失函数。这么做并不会影响其能够收敛至最优解的性质，且可以在训练初期提供足够的梯度。

在后续的应用中，发现GAN具有收敛困难，且非常依赖于选择合适的交替训练轮次等不足。为此，Martin Arjovsky等人提出了Wasserstein GAN[32]，对模型结构进行了一些修改。在Wasserstein GAN中，模型输出不使用以概率值形式输出的sigmoid激活函数，而是直接根据输出值大小评价其效果。相应地，目标函数中也不需要再取对数：

$$\min_G\max_D V(D,G)=\mathbb{E}_{x\sim p_{data}(x)}\left[D(x)\right]-\mathbb{E}_{z\sim p_z(z)}\left[D\left(G(z)\right)\right]$$

此外，在每一轮参数更新后，需要将参数的值截断到绝对值不超过一个设定的常数ε以确保网络满足李普希兹（Lipschitz）连续的条件。经过这些修改后的生成对抗网络仍然能在理论上被证明收敛于最优解，同时网络的训练效率得到了极大的提升。

GAN在某种意义上避免了马尔可夫链式的学习机制，这使得它能够区别于传统的概率生成模型。当真实数据的概率密度不可计算时，传统依赖于数据内在自然解释机制的生成模型（如主题模型等）难以直接应用来计算数据概率分布，但是GAN引入了非常聪明的内部对抗博弈机制，可以来计算和逼近不是很容易计算的目标函数。GAN为无监督学习提供了一个有力的框架，而无监督学习被认为是人工智能的“黑洞（dark matter）”。

6.5.3 条件生成对抗网络

生成网络可应用于许多任务，如不同类别图像生成、文字到图像的转换、图像风格迁移、图像超分辨率重建以及图像编辑等。

早期生成对抗网络通过随机变量来学习真实数据分布，这一数据生成方式太过自由，无法适应不同任务，于是生成对抗网络也出现了

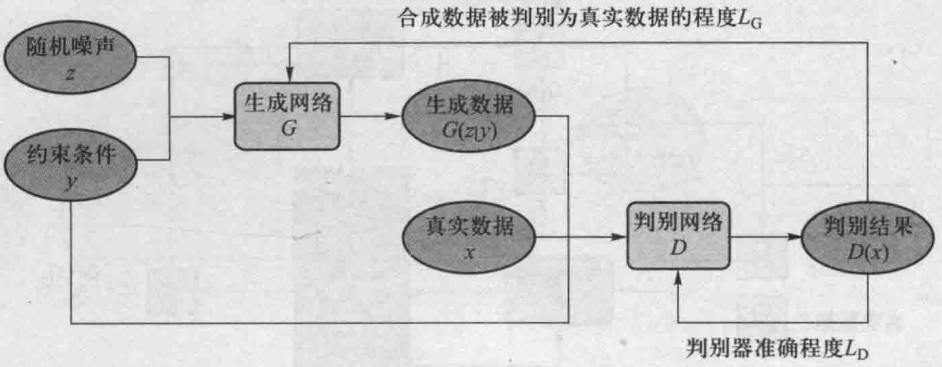
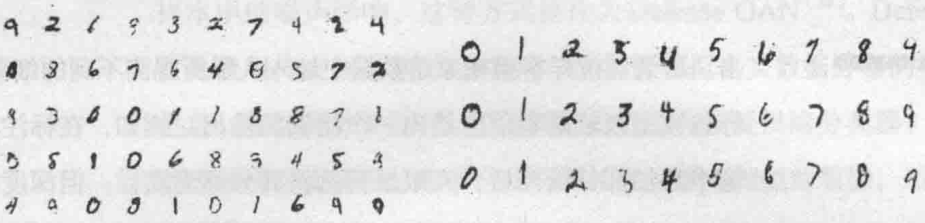


图6.25 条件生成对抗网络结构模型示意图



(a) 生成对抗网络产生的随机手写数字图像　(b) 根据条件生成的不同数字的手写图像

图6.26 不同生成对抗网络模型产生的手写数字图像

一些变体，其中很重要的一个变体就是条件生成对抗网络（conditional generative adversarial networks，CGAN）[33]。

在生成数据中，往往不仅要求所生成数据逼真，还会要求其符合一定的条件。在图6.25所示条件生成对抗网络结构中，条件生成对抗网络将条件约束y分别输入给生成器和判别器，生成器会根据条件约束y和随机噪声z一同生成数据，判别器需要判断数据是否是满足该条件的真实样本。

这样，生成的数据不仅能够接近真实数据，还能够通过调节约束，获得符合特定要求的生成数据。例如，在手写数字的生成中，原始的生成对抗网络只能随机产生某个数的手写图像，而条件生成对抗网络则可以根据要求，产生某个特定数字的手写图像。

在图6.26中，图（a）是传统生成对抗网络得到的手写体图像数据，图（b）是在模型中施加指定需要生成某个固定手写体图像所得到的结果。

6.5.4 用生成对抗网络抵御对抗样本攻击

虽然生成对抗网络的初衷是用于学习真实数据的概率分布，但其同样能够应用于提升判别器性能以抵御对抗样本（adversarial samples）攻

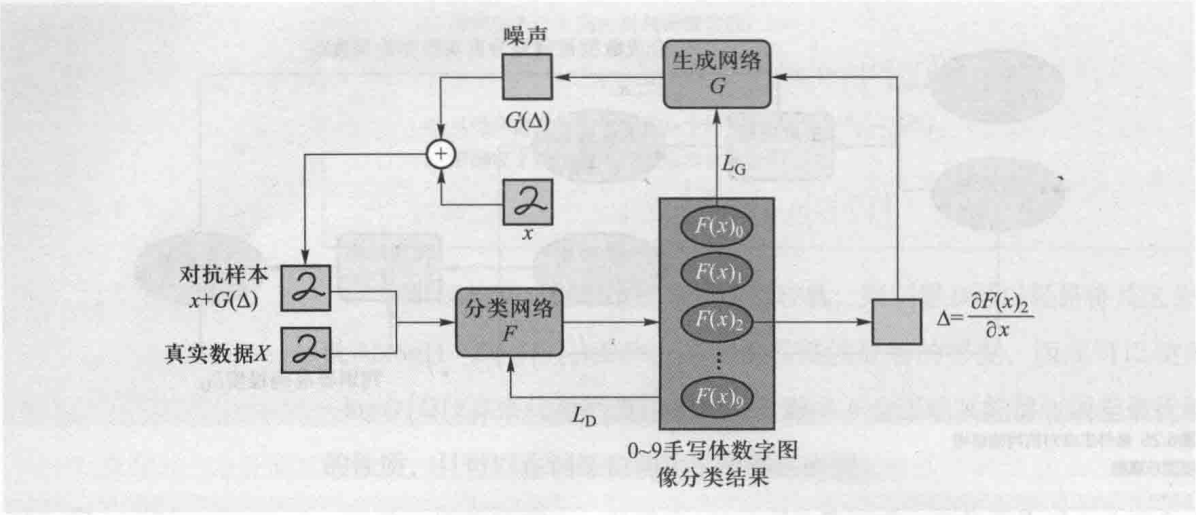


图6.27 对抗生成训练器模型

击。所谓对抗样本指在原始数据中加入人眼所观测不到的细微噪声扰动，导致模型以较高置信度给出一个错误的输出。例如，在标注为熊猫的图像中添加部分噪声后，人眼虽仍然将其分类为熊猫，但深度模型却以接近100%的置信度将其错误分类为长臂猿[27]。

一种非常直观的做法是让生成器扮演产生对抗样本的攻击者，而判别器扮演分类器。分类器的目标是将原始数据和添加了噪声的对抗样本都分到正确的类别中去，而生成器的目标则是生成能够尽可能降低判别准确率的含噪数据[34]。

图6.27给出了对抗生成训练器模型的示意图。在训练中，生成器使用判别器对原始图像的梯度作为输入，产生一个噪声$G(\Delta)$，$G(\Delta)$与原始图像x叠加组成对抗样本，判别器将原始图像和所生成对抗样本一同作为训练数据再继续训练模型。

在图6.27中，数字图像2的分类误差会被算法反馈，由生成网络根据误差梯度来生成噪声数据，进而与真实数据叠加得到对抗样本。

图6.28 生成对抗训练产生的攻击样本。左为原始真实图像数据，右为训练中产生的对抗样本图像

图6.28给出了训练所生成的对抗样本图像。

当然，如果任意地生成噪声，那么生成器自然可以产生一份面目全非的数据使得分类器无法判别。因此在目标函数中需要增加一个约束项对噪声强度进行限制。通过这样的对抗训练，分类器能够将添加了噪声的样本也进行正确的分类，而生成器则必须对原始数据做出较大的改动才能够骗过判别器，这样的改动往往确实改变了原始数据的内容，即使人类也无法正确

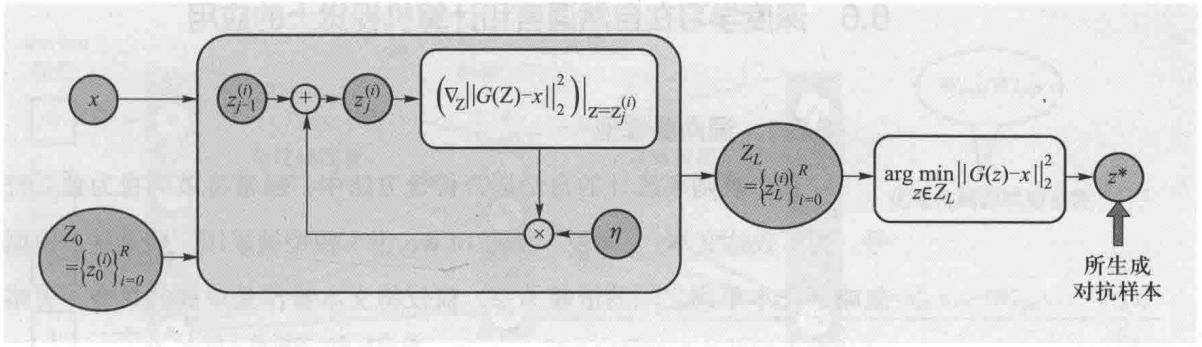


图6.29 Defense GAN模型示意图

判别其属于哪类数据，这样的改动也就使得对抗样本失去了意义。

另一种做法是使用GAN学习出真实数据的分布，以此来消除对抗样本中的噪声影响，这种方式被称为Defense GAN[35]。Defense GAN首先使用传统GAN的方法得到一个能够模拟真实数据分布的生成模型。之后，在使用分类器时，不是将原始数据直接提供给分类器，而是从生成模型所对应概率分布中合成与原始数据最为相似的数据，用它来代替原始数据，交给分类器进行识别。

如此一来，对抗样本将被一个与其相近的、来自生成器输出空间的数据所替代。训练数据中并不存在这样精心设计的样本，因此通过这样的替代，就能在一定程度上滤去噪声带来的影响，从而确保分类器的准确率。

如图6.29所示，在Defense GAN中，当需要识别一个样本x时，首先生成器会产生R个不同的随机噪声z。对于每个随机噪声，生成样本$G(z)$，并计算$G(z)$与x的差异，用所得梯度来更新z的取值。经过L次循环之后，就能够得到与x相似度较高的样本$G(z)$。最后选择出经过L次循环处理的R个不同的随机噪声中与x最接近的样本$G(z^*)$，将$G(z^*)$交给判别器正常进行识别判断。通过如上过程，可生成带有扰动的替代样本，以提高判别器的识别分类性能，防止恶意样本攻击，如图6.30所示。

图6.30 Defense GAN生成的训练用样本，上方为输入的对抗样本，下方为生成器产生的用提高判别器性能的替代样本

这种方法的特点在于不需要对判别模型的结构和训练过程进行任何改动，因此可以作为额外组件直接加在已训练好的模型上，来抵御对抗样本的攻击。

6.6 深度学习在自然语言和计算机视觉上的应用

6.6.1 词向量模型

在基于规则和统计的自然语言传统方法中，通常将单词视为独立符号。为了表达文本，“词袋”（bag of words）模型被采用，但是这一模型忽略了文本单词之间的依赖关系，仅仅将文本看作是单词的集合（忽略了单词之间的先后次序）。

在词袋模型中，一个单词按照词典序被表示为一个词典维数大小的向量（被称为one-hot vector），向量中该单词所对应位置按照其在文档中出现与否取值为0或1。1表示该单词在文档中出现，0表示没有出现，如图6.31所示。每个单词被表达为单词向量后，一些与自然语言相关的工作（如聚类、同义词计算、主题挖掘等）可以依次开展。

然而，随着文本数目增加，单词词典大小也会增加，使得词袋模型中每个单词的维数不断增加。因此，使用词袋模型来表示单词时，往往会遭遇维度灾难的问题。除此之外，这种表达方法无法有效计算单词与单词之间的语义相似度。例如，“高兴”与“愉快”两个单词的向量表达会很不相同，虽然其具有很强的语义相似性。

为了刻画不同单词之间的语义相关性，研究人员希望使用一种分布式向量表达（distributed vector representation）来对不同单词进行表达。利用深度学习模型，可将每个单词表征为N维实数值的分布式向量。这样一来可以把对文本内容的分析简化为N维向量空间中的向量运算，如两个单词在向量空间上所计算结果可用来衡量这两个单词之间的相似度。用深度学习算法生成每个单词的向量表达，词向量（Word2Vec）是较为

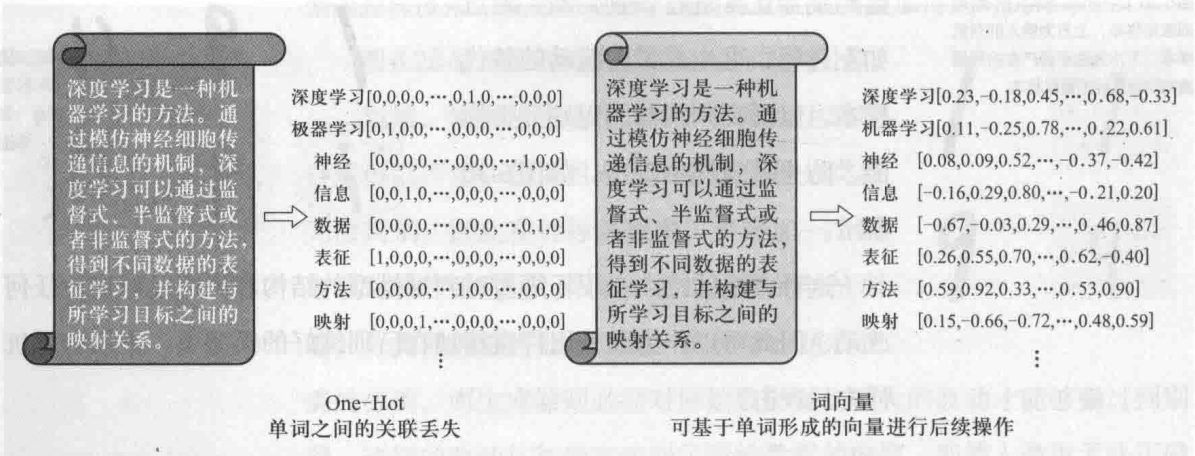


图6.31 词典模型与词向量模型表示文本

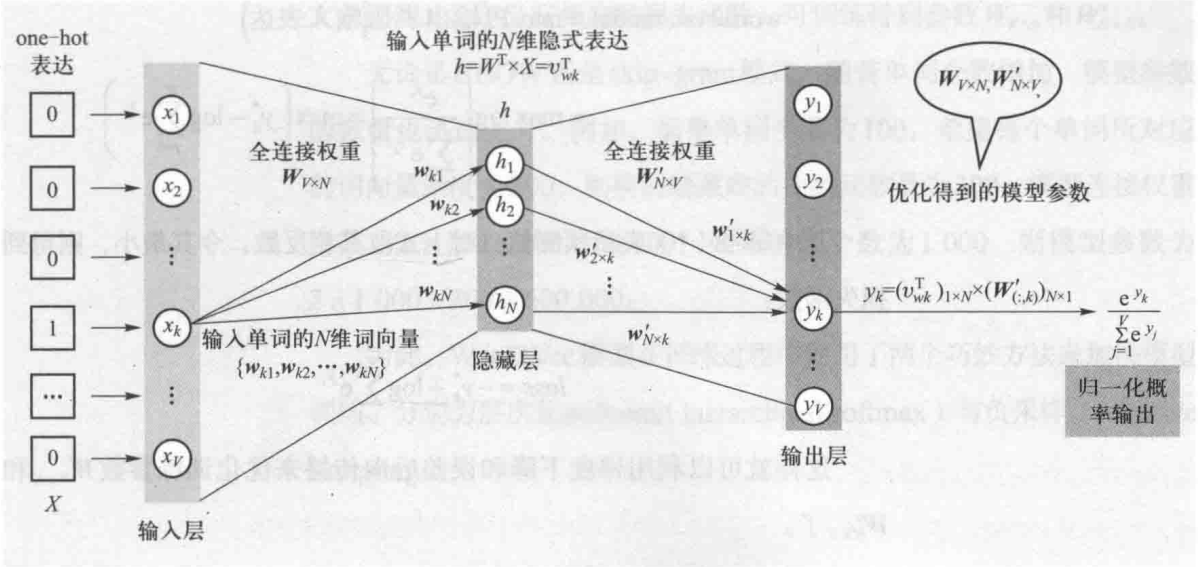


图6.32 单个单词的词向量（Word2Vec）生成示意图

经典的模型。

假设词典中有V个不同的单词，现在考虑如何生成第k个单词的N维词向量，如图6.32所示。首先，将该单词表示成V维one-hot向量$\boldsymbol{X}$，向量$\boldsymbol{X}$中第k个位置取值为1，其余位置取值均为0。隐藏层神经元个数为N，每个神经元记为h_i（$1\leqslant i\leqslant N$）。向量$\boldsymbol{X}$中每个x_i（$1\leqslant i\leqslant V$）与隐藏层神经元是全连接，连接权重矩阵为$\boldsymbol{W}_{V\times N}$。输出层是$V$维归一化的概率值，其中$y_k$对应第$k$个单词归一化的概率值，显然其取值应该远大于其他输出所对应的归一化概率值。

在图6.32中，输入层中x_k与隐藏层连接权重为$\{\boldsymbol{w}_{k1}, \boldsymbol{w}_{k2}, \cdots, \boldsymbol{w}_{kN}\}$，输入单词的$N$维隐式表达$\boldsymbol{h}=\boldsymbol{W}^{\mathrm{T}}\times\boldsymbol{X}=v_{wk}^{\mathrm{T}}$。所得到的隐藏层结果与输出层是全连接，全连接权重矩阵为$\boldsymbol{W}'_{N\times V}$。从图中可知，第$k$个单词对应的输出为$y_k=\left(v_{wk}^{\mathrm{T}}\right)_{1\times N}\times\left(\boldsymbol{W}'_{(:,k)}\right)_{N\times 1}$。这里$\boldsymbol{W}'_{(:,k)}$指的从连接权重矩阵$\boldsymbol{W}'_{N\times V}$中取出第$k$列的向量。接着对$y_k$通过Softmax函数进行归一化，得到第$k$个单词对应的归一化概率输出。显然，第$k$个单词归一化的概率值应该远远大于输出层中其他位置所对应的归一化概率值。

这里的$\boldsymbol{W}_{V\times N}$和$\boldsymbol{W}'_{N\times V}$为模型参数。一旦训练得到了如图6.32的神经网络，就可以将输入层中x_k与隐藏层的连接权重$\{w_{k1}, w_{k2}, \cdots, w_{kN}\}$作为第$k$个单词的$N$维词向量（word vector）。

为了训练词向量模型，可将训练目标如下进行形式化描述：

$$\text{word2vec model} = \max \mathrm{P}(\text{输出表达}|\text{输入表达})$$

$$= \max \log\left(\frac{\mathrm{e}^{y_k^*}}{\sum_{j=1}^{V}\mathrm{e}^{y_j}}\right) = \max\left(y_k^* - \log\sum_{j=1}^{V}\mathrm{e}^{y_j}\right)$$

其中星号（*）表示预测值。对上式取其相反数，令其最小，则得到损失函数：

$$loss = -y_k^* + \log\sum_{j=1}^{V}\mathrm{e}^{y_j}$$

这样就可以利用梯度下降和误差后向传播来优化训练参数$\boldsymbol{W}_{V\times N}$和$\boldsymbol{W}'_{N\times V}$了。

通常Word2Vec的模型参数有两种训练模式。一种是continuous bag-of-words（CBOW），即根据某个单词所处的上下文单词来预测该单词。另一种是skip-gram，即利用某个单词来分别预测该单词的上下文单词。

图6.33给出了基于CBOW来得到词向量的示意图。给出包含c个单词的句子$\{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_{k-1}, \boldsymbol{x}_k, \boldsymbol{x}_{k+1}, \cdots, \boldsymbol{x}_c\}$。在CBOW中，可用$\boldsymbol{x}_k$的前序$k-1$个单词和后续$c-k$个单词一起来预测$\boldsymbol{x}_k$。每个输入单词仍然是$V$维向量（可以是one-hot的表达），隐式编码为每个输入单词所对应编码的均值$\boldsymbol{h} = \frac{1}{c-1}\boldsymbol{W}^{\mathrm{T}} \times (\boldsymbol{x}_1 + \cdots + \boldsymbol{x}_{k-1} + \boldsymbol{x}_{k+1} + \cdots + \boldsymbol{x}_c)$。对于计算得到的隐式编码，要求其输出归一化概率中，单词$\boldsymbol{x}_k$所对应的y_k取值最大，即用$\boldsymbol{x}_k$的上下

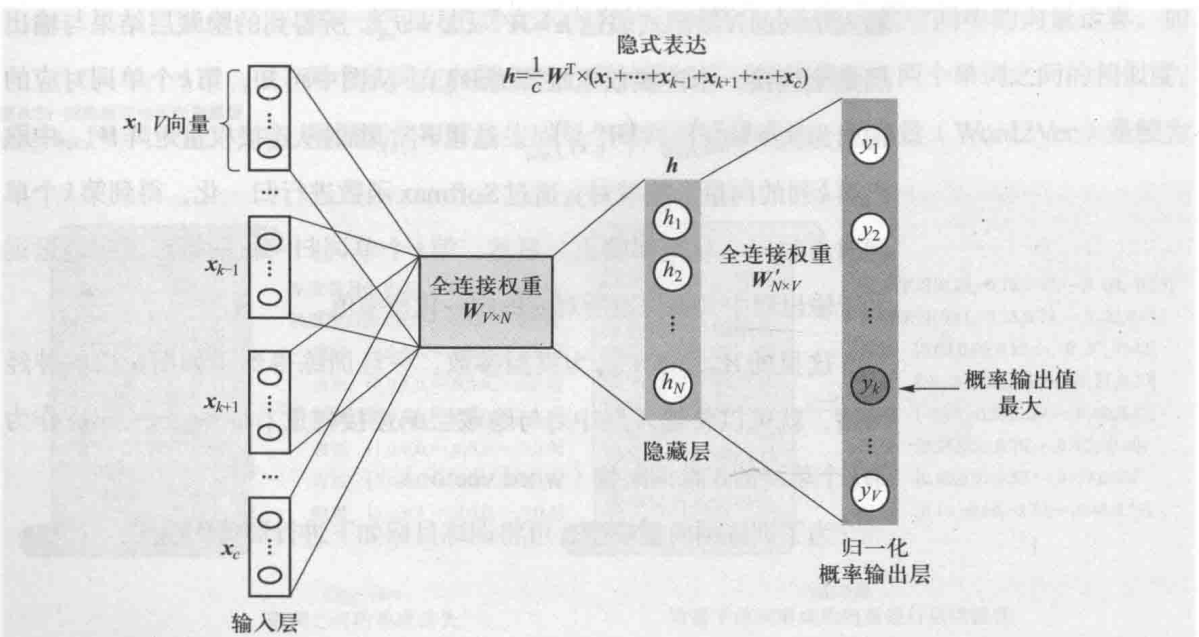


图6.33 基于CBOW模型（continuous bag-of-words model）的单词词向量（Word2Vec）生成示意图

文来预测x_k。定义好相应的损失函数，可训练得到参数$W_{V\times N}$和$W'_{N\times V}$。

无论是CBOW还是skip-gram模式，随着单词个数增加，模型参数的数量也迅速上升。例如，如果单词个数为100，希望每个单词所对应的词向量维度为300，则模型隐藏层的神经元数量为300，模型连接权重参数为$2\times 100\times 300=60\ 000$；如果单词个数为1 000，则模型参数为$2\times 1\ 000\times 300=600\ 000$。

为此，Word2Vec模型在训练过程中使用了两个巧妙方法来加快模型训练，分别为层次化softmax（hierarchical softmax）与负采样（negative sampling）。

6.6.2 图像分类与目标定位

图像分类和目标定位（image classification and object localization）是机器视觉领域的核心问题之一，其不仅将图像分为不同类别，而且对图像中的视觉对象进行定位。

如图6.34所示，给定一张图像，图像分类和目标定位模型需要在图像中找到视觉对象目标的位置。视觉对象目标的位置可用包围盒（bounding box）来表示，即一个矩形方框左上角的横坐标和纵坐标以及该矩形框的高与宽。

目前，已有的图像分类和目标定位算法可以分为两类：① 基于候选区域的目标检测方法，这类方法大体上分为两个步骤，第一步为产生候选区域，第二步为对候选区域分类，基于候选区域的目标检测方法的典型代表有R-CNN[26]，Fast R-CNN[28]，Faster R-CNN[29]等；② 单次目标检测方法，即不经过候选区域检测这个过程，直接对输入图像进行处理而得到视觉对象区域预测结果，也被称为单阶段模型，单次目标检测方法的典型代表有SSD27[30]，YOLO[31]等。下面基于单次目标检测方法思想来介绍深度学习的应用。

在图像分类和目标定位过程中，单次目标检测模型不仅要检测物体位置，同时也要判断所检测物体的类别标签。因此，图像分类和目标定位需要同时完成回归问题（位置判断）与分类问题（物体识别）两个任务，这种同时解决两个及两个以上问题的任务为多任务学习（multi-task learning）。

输入一幅图像，首先利用卷积神经网络来提取视觉特征。在图6.34

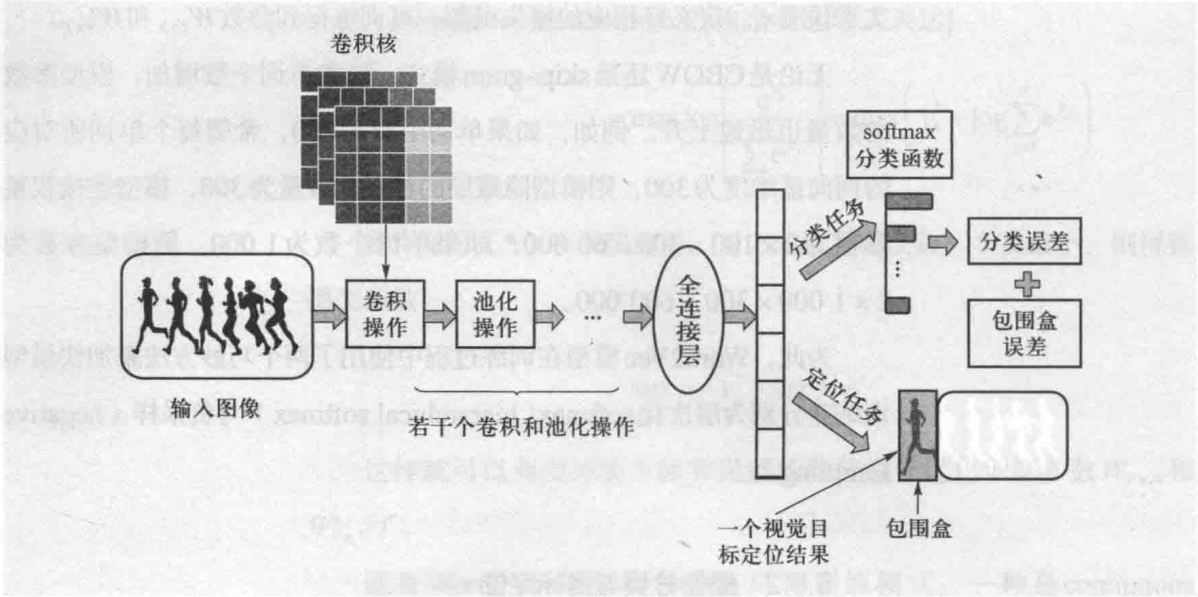


图6.34 利用卷积神经网络进行图像分类与目标定位示意图

中，给定一幅图像，卷积神经网络经过若干次卷积与池化的操作，可提取一个特征向量来表示输入图像。这个向量作为输入特征分别传送给两个任务，即分类任务和定位任务。分类任务中，softmax分类函数将全连接层所得特征向量映射为归一化概率值，用来表示每个概念在输入图像中出现的概率。在softmax分类函数的输出概率向量中，向量中每一维代表输入图像中某个类别出现的概率。在训练中，可用交叉熵等损失函数来计算模型学习的误差。目标定位中，将全连接层输出向量转换为一个四维向量，四维向量分别代表一个包围盒的左上角横坐标和纵坐标以及该矩形框的宽与高取值，并使用均方差损失函数计算模型的误差。最终，将两个任务所得误差相加，作为整个模型的误差，用于计算模型梯度，更新模型参数。这样，训练好的模型可以同时对图像分类以及对图像中视觉目标对象进行定位。

6.7 小结

合适的特征表达是识别与分类等任务的基础。早期的机器学习任务中，一般根据各种物理成像原理、编码有效性等，通过手工来构造（handcrafted）相关特征，如直方图、纹理和形状描述等，提升了一些识别任务的性能。

与手工构造特征不同，深度学习通过“端到端（end to end）”学习方式来形成对原始数据的有效特征表达，其基本动机在于通过构建多层网络来学习隐含在数据内部的关系，从而使学习得到的特征具有更强的表达能力和泛化能力。

将深度学习应用于语义概念识别和理解时，一般思路是将深度学习得到的特征（如卷积神经网络中最后一个全连接层的输出以及词向量模型中经由隐藏层计算得到隐式编码）输入给分类模型（如softmax函数），然后根据损失函数计算所得“误差”来对模型进行优化。这一犹如“黑盒子”的学习机制使得深度学习结果缺乏解释性，从而难以可信（如易受到对抗样本攻击等），如何将知识引入学习过程，得到可信安全、解释性更强、较少依赖标注大数据、自适应更好的深度学习方法是目前的研究热点。

习题6

1. 下面对误差反向传播（error back propagation，BP）描述不正确的是（　　）。

A. BP算法是一种将输出层误差反向传播给隐藏层进行参数更新的方法

B. BP算法将误差从后向前传递，获得各层单元所产生误差，进而依据这个误差来让各层单元修正各单元参数

C. 对前馈神经网络而言，BP算法可调整相邻层神经元之间的连接权重大小

D. 在BP算法中，每个神经元单元可包含不可偏导的映射函数

2. 以下关于梯度下降和随机梯度下降的说明，哪一项是正确的？（　　）

i. 在梯度下降和随机梯度下降中，为了最小化损失函数，通常使用循环迭代的方式不断更新模型参数；

ii. 在每次迭代中，随机梯度下降需要计算训练集所有样本的误差和，用于更新模型参数；

iii. 在每次迭代中，梯度下降使用所有数据或者部分训练数据，用于更新模型参数。

A. i　　B. ii

C. iii　　D. i, ii

E. ii, iii　　F. i, ii, iii

3. 关于sigmoid激活函数，下列描述正确的是（　　）。

A. 它是凸函数，凸函数无法解决非凸问题

B. 它可以有负值

C. 它无法配合交叉熵损失函数使用

D. 当输入值过大或者过小时，梯度趋近于0，容易造成梯度消失问题

4. 下面对前馈神经网络这种深度学习方法描述不正确的是（　　）。

A. 是一种端到端学习的方法

B. 是一种监督学习的方法

C. 实现了非线性映射

D. 隐藏层数目大小对学习性能影响不大

5. 图6.21展示了长短时记忆网络的详细结构。如下描述正确的是（　　）。

A. 如果输入x_t为0向量，则$h_t = h_{t-1}$

B. 如果f_t非常小或者为0，则误差不会被反向传播到较早的时间节点

C. f_t、i_t和o_t的输出是非负数

D. f_t、i_t和o_t的输出可以被看作是概率分布，其输出为非负数且和为1

6. 关于生成对抗网络的训练，以下说法正确的是（　　）。

A. 为了加速生成对抗网络的训练，通常采用交替训练一轮判别器和多轮生成器的方法

B. 当生成器能够产生足以以假乱真的数据，即$D(G(z))$的输出稳定于接近1的值时，训练就完成了

C. 目标函数需要在训练初期提供足够的梯度，使生成器更快开始产生能够混淆判别器的数据

D. 生成对抗网络在经过足够长时间的训练后可以确保收敛到全局最优

7. 在训练生成对抗网络时，发现损失函数的值很小，但生成器产生的数据质量并不高这可能是因为（　　）。

A. 判别器欠拟合　　B. 判别器过拟合

C. 生成器欠拟合　　D. 生成器过拟合

8. 请问深度学习和传统的机器学习有什么区别？

9. （1）请说明单层感知机为何无法模拟逻辑异或操作？

（2）请设计一个前馈神经网络，用于拟合异或函数。

10. 在前馈神经网络中，所有的参数能否被初始化为0？如果不能，能否全部被初始化为其他的相同值？为什么？

11. “编码器-解码器”框架被广泛用于机器翻译中，其中编码器用于对源语言进行编码，解码器用于产生目标语言。当编码器编码完毕后，会将所得信息传给解码器，同时给解码器“BOS”标签，告知解码器可以开始解码。解码器依次产生目标语言词汇，当产生“EOS”标签时解码结束。图6.35给出了通用机器翻译框架，如果目标语言的词汇量大小为V，y_t'为模型预测的V维概率向量、w_t为所预测词语、y_t为真实值。

（1）写出解码器的损失函数。

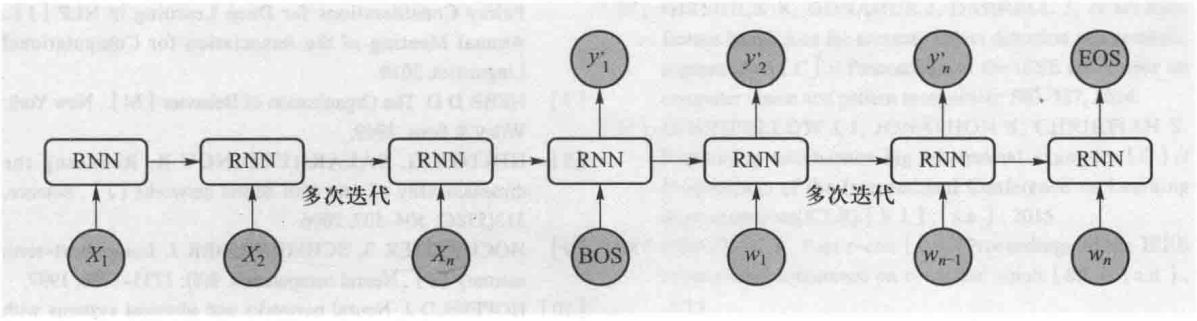


图6.35 机器翻译中“编码-解码”框架

（2）在模型过程中，通常会使用“微调”的方式更新预训练词向量。请写出w_{n-1}梯度。

*12. 请证明为什么LSTM会解决循环神经网络（RNN）中所遇到的梯度消失问题。

RNN的计算公式可以写为：

$$c_t = W \cdot [x_t, c_{t-1}] + b$$

假设时序长度为T，则在梯度反向传播过程中，T时刻的误差l_T对W的梯度为：

$$\frac{\partial l_T}{\partial W} = \sum_{i=1}^{T} \frac{\partial l_T}{\partial c_T} * \left(\prod_{j=i+1}^{T} \frac{\partial c_j}{\partial c_{j-1}} \right) * \frac{\partial c_T}{\partial W}$$

$\prod_{j=i+1}^{T} \frac{\partial c_j}{\partial c_{j-1}}$中每一项都是介于0和1之间的小数，当$T$很大时，$\prod_{j=i+1}^{T} \frac{\partial c_j}{\partial c_{j-1}}$值趋近于0，从而产生梯度消失问题。

在LSTM中，$c_t = f_t \cdot c_{t-1} + i_t \cdot \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$，因此$\frac{\partial c_t}{\partial c_{t-1}} = f_t$，由于$f_t$通常为1或者为0，当为1时，所有梯度能够在LSTM中传递，当为0时，说明上一时刻的信息对当前时刻没有影响，因此没有必要传回梯度更新参数。这样，LSTM解决了RNN的梯度消失问题。

参考文献

[1] BENGIO Y, DUCHARME R, VINCENT P, et al. A neural probabilistic language model [J]. Journal of Machine Learning Research, 3: 1137–1155, 2003.

[2] CHO K, MERRIENBOER B, GULCEHRE C, et al. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation [J]. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), 1724–1734, 2014.

[3] DEVLIN J, CHANG M W, LEE K, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [J]. arXiv: 1810. 04805, 2018.

[4] ELMAN J L. Finding structure in time [J]. Cognitive Science, 14 (2), 1990.

[5] GOODFELLOW I J, POUGET A J, MIRZA M, et al. Generative Adversarial Nets [C] // International Conference on Neural Information Processing Systems. Cambridge, MA: MIT Press, 2014.

[6] STRUBELL E, GANESH A, MCCALLUM A. Energy and

Policy Considerations for Deep Learning in NLP [J]. Annual Meeting of the Association for Computational Linguistics, 2019.

[7] HEBB D O. The Organization of Behavior [M]. New York: Wiley & Sons, 1949.

[8] HINTON G, SALAKHUTDINOV R. Reducing the dimensionality of data with neural networks [J]. Science, 313(5786): 504–507, 2006.

[9] HOCHREITER S, SCHMIDHUBER J. Long short–term memory [J]. Neural computation, 9(8): 1735–1780, 1997.

[10] HOPFIELD J. Neural networks and physical systems with emergent collective computational abilities [J]. Proceedings of the national academy of sciences, 79(8): 2554–2558, 1982.

[11] HUBEL D, WIESEL T. Receptive fields of single neurones in the cat's striate cortex [J]. The Journal of Physiology, 148 (3): 574–591, 1959.

[12] HUBEL D, WIESEL T. Receptive fields, binocular interaction and functional architecture in the cat's visual cortex [J]. The Journal of physiology, 160(1): 106–154, 1962.

[13] JORDAN M I. Serial order: A parallel distributed processing approach [J]. Advances in Psychology, 121: 471–495, 1997.

[14] FUKUSHIMA K. Neocognitron: A Self–organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position [J]. Biological Cybernetics, 36, 193–202, 1980.

[15] FUKUSHIMA K. Neocognitron: A hierarchical neural network capable of visual pattern recognition [J]. Neural networks, 1(2): 119–130, 1988.

[16] KRIZHEVSKY A, SUTSKEVER I, HINTON G. ImageNet classification with deep convolutional neural networks [C] // Proceedings of the 25th International Conference on Neural Information Processing Systems. New York: Curran Associates Inc. 1091–1105, 2012.

[17] LECUN Y, BOTTOU L, BENGIO Y. Patrick Haffner Gradient–Based Learning Applied to Document Recognition [J]. Proceedings of the IEEE, 86(11): 2278–2324, 1998.

[18] LECUN Y, BENGIO Y, HINTON G. Deep learning [J]. Nature, 521(7553): 436–444, 2015.

[19] MCCULLOCH W S, PITTS W. A logical calculus of the ideas immanent in nervous activity [J]. The Bulletin of Mathematical Biophysics, 5(4): 115–133, 1943.

[20] MINSKY M, PAPERT S A. Perceptrons: An Introduction to Computational Geometry [M]. Cambridge, MA: The MIT Press, 1969.

[21] MIKOLOV T, CHEN K, CORRADO G, et al. Efficient estimation of word representations in vector space [C] // Proceedings of the International Conference on Learning Representations(ICLR). [S.l.]: [s.n.], 2013.

[22] ROSENBLATT F. Principles of Neuro dynamics: Perceptrons and the Theory of Brain Mechanisms [M]. Washington, DC: Spartan Books, 1962.

[23] RUMELHART D E, HINTON G, WILLIAMS R J. Learning representations by back–propagating errors [J]. Nature, 323 (6088): 533–536, 1986.

[24] WERBOS P J. Backpropagation Through Time: What It Does and How to Do It [J]. Proceedings of the IEEE, 78(10): 1550–1560, 1990.

[25] WERBOS P J. Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences [D]. Cambridge, MA: Harvard University, 1974.

[26] GIRSHICK R, DONAHUE J, DARRELL T, et al. Rich feature hierarchies for accurate object detection and semantic segmentation [C] // Proceedings of the IEEE conference on computer vision and pattern recognition: 580–587, 2014.
[27] GOODFELLOW I J, JONATHON S, CHRISTIAN S. Explaining and harnessing adversarial examples [C] // Proceedings of the International Conference on Learning Representations(ICLR). [S. l.] : [s.n.], 2015.
[28] GIRSHICK R. Fast r-cnn [C] //Proceedings of the IEEE international conference on computer vision. [S.l.] : [s.n.], 2015.
[29] REN S, HE K, GIRSHICK R, et al. Faster r-cnn: Towards real-time object detection with region proposal networks [J]. Advances in neural information processing systems, 2015.
[30] LIU W, ANGUELOV D, ERHAN D, et al. Ssd: Single shot multibox detector [J]. European conference on computer vision, 2016.
[31] REDMON J, DIVVALA S, GIRSHICK R, et al. You only look once: Unified, real-time object detection [C] // Proceedings of the IEEE conference on computer vision and pattern recognition. [S.l.] : [s.n.], 2016.
[32] ARJOVSKY M, SOUMITH C, BOTTOU L. Wasserstein generative adversarial networks [J]. International conference on machine learning, 2017.
[33] MIRZA M, OSINDERO S. Conditional generative adversarial nets [J]. arXiv preprint arXiv: 1411. 1784, 2014.
[34] LEE H, HAN S, LEE J. Generative adversarial trainer: Defense to adversarial perturbations with gan [J]. arXiv preprint arXiv: 1705. 03387, 2017.
[35] SAMANGOUEI P, KABKAB M, CHELLAPPA R. Defense-gan: Protecting classifiers against adversarial attacks using generative models [J]. arXiv preprint arXiv: 1805. 06605, 2018.

第7章 强化学习

7

谋定而后动，知止而有得

强化学习是智能体（agent）在不断与其所处环境交互中进行学习的一种方法。在这种方法中，智能体通过“尝试与试错”和“探索与利用”等机制在所处状态采取行动，不断与环境交互，直至进入终止状态，根据在终止状态所获得的奖惩来改进行动策略，序贯完成决策任务。

在强化学习中，学习信号以奖励形式出现，智能体在与环境交互中去取得最大化收益，这种学习方式既不是从已有数据出发，也不是依赖于已有知识的学习方式，犹如“tabula rasa（拉丁语）”所蕴含“一张白纸绘蓝图”之义，从“授之以鱼”迈向“授之以渔”。

近年来，随着深度学习的发展，对强化学习的研究取得了一定进展，强化学习方法也成为智能系统设计的核心技术之一，被广泛应用于人工智能和机器人学等领域。

本章主要介绍强化学习形式化定义、基本求解方法、深度强化学习以及若干例子分析。

7.1 强化学习问题定义

7.1.1 强化学习基本概念

MOOC微视频：强化学习定义

人类在日常生活中通过对环境进行探索和与环境进行交互而不断学习。例如，婴幼儿在学习走路时，在其踉踉跄跄向前走一步的过程中撞到了障碍物时会因感受到疼痛而哇哇大哭。通过这一次与环境交互的经验，婴幼儿以后走路时就会避免向有障碍物的方向前进。如果将这个过程进行抽象化，与环境交互而不断学习的过程可如下描述：婴幼儿通过动作（向前走一步）对环境产生影响；环境向其反馈状态的变化（撞到障碍物上）；婴幼儿评估当前动作得到收益（感受到疼痛）；婴幼儿会更新在前方有障碍物这个状态下所做出动作的过往策略（避免向有障碍物的方向前进）。

强化学习模仿了上述学习过程。作为了解强化学习的第一步，先介绍学习过程中要使用的若干概念。

- **智能体**（agent）：智能体是强化学习算法的主体，它能够根据经验做出主观判断并执行动作，是整个智能系统的核心。

- **环境**（environment）：智能体以外的一切统称为环境，环境在与智能体的交互中，能被智能体所采取的动作影响，同时环境也能向智能体反馈状态和奖励。虽说智能体以外的一切都可视为环境，但在设计算法时常常会排除不相关的因素建立一个理想的环境模型来对算法功能进行模拟。

- **状态**（state）：状态可以理解为智能体对环境的一种理解和编码，

通常包含了对智能体所采取决策产生影响的信息。

- **动作**（action）：动作是智能体对环境产生影响的方式，这里说的动作常常指概念上的动作，如果是在设计机器人时还需考虑动作的执行机构。

- **策略**（policy）：策略是智能体在所处状态下去执行某个动作的依据，即给定一个状态，智能体可根据一个策略来选择应该采取的动作。

- **奖励**（reward）：奖励是智能体序贯式采取一系列动作后从环境获得的收益。注意奖励概念是现实中奖励和惩罚的统合，一般用正值来代表实际奖励，用负值来代表实际惩罚。

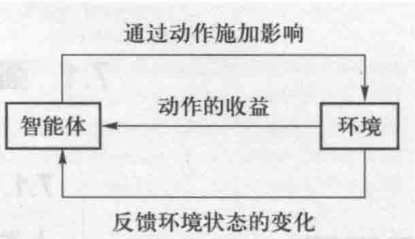


图7.1 强化学习的过程

如图7.1所示，强化学习实际上是智能体在与环境交互中去学习能帮助其获得最大化奖励这一策略的过程。在每一次迭代中，智能体根据当前策略选择一个动作，该动作影响了环境，导致环境发生了改变，智能体此时从环境得到状态变化和奖励反馈等信息，并根据这些反馈来更新其内部的策略。这一过程与人类从与环境交互中学习这一方式具有相似之处。

强化学习求解的是探索性和交互性的问题，从这个角度来看，强化学习类似于本书第3章介绍的搜索算法，尤其是蒙特卡洛树搜索算法（即在探索中寻找问题的答案）。同时强化学习中包含了学习过程，因此也与本书第4和第5章介绍的监督学习和无监督学习存在相通之处。

表7.1总结了监督学习、无监督学习和强化学习各自的特点，从四个角度来比较这三种方法。

从学习的依据上来看，监督学习基于有标注的监督信息来更新模型，无监督学习则对数据中存在的模式或结构进行假设来学习（如假设数据分布为高斯分布而去求解参数），而强化学习则是基于智能体对奖励的评估来进行学习。

在介绍监督学习和无监督学习算法时，通常假设数据是事先给定的，然而强化学习的数据来自于智能体与环境交互产生的经验，通常是在学习过程中产生的；有监督学习和强化学习均可以认为涉及了决策过程，监督学习根据特征来决定样本的标签，而强化学习则是序贯决策问题，稍后决策往往会受到先前决策的影响。

监督学习的目标是得到一个从样本到语义标签的映射，无监督学习的目标是学习数据的分布模式，而强化学习的目标则是学习一个使奖励最大化的策略。

表7.1 三种学习方式特点对比

	有监督学习	无监督学习	强化学习
学习依据	基于监督信息	基于对数据结构的假设	基于评估
数据来源	一次给定	一次给定	在时序交互中产生
决策过程	单步决策 （如分类和识别等）	无	序贯决策 （如棋类博弈）
学习目标	样本到语义标签的映射	数据的分布模式	选择能够获取最大收益的状态到动作的映射

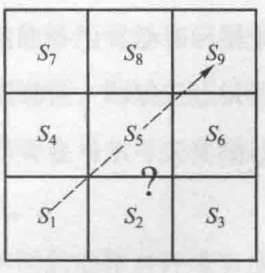
这里以围棋为例来解释上表所列强化学习对应特点。在围棋博弈的强化学习中，算法如果要知悉某一落子走法是否能以极大概率带来胜局，必须先尝试该走法后再观察智能体是否后续能赢得比赛，这就是基于评估的学习；算法的训练数据主要来自智能体不断模拟对局而产生，因此可以说训练算法的数据是在交互中产生；一局围棋可能涉及上百次落子决策，而这些决策之间是相互关联的，因为不同的布局方式可能导致智能算法在后期采取不同策略。

在围棋对抗中，学习算法还有一些是表7.1中没有提到，但也十分常见的特点。例如，奖励有滞后性，智能体每次落子后并不能立刻得到该落子所能够带来的奖励信息，而是要在棋局回合终结时才能得到一个明确的奖励；同时，由于围棋问题所面临的搜索空间巨大，算法在每次落子前不可能枚举出所有潜在落子优劣后再做出决定，而是通过采样方法来求解寻找一个较佳落子。

7.1.2 马尔可夫决策过程

本节将对7.1.1节中介绍的交互过程进行形式化定义。为了方便说明，本节将基于一个简单的例子来介绍相关定义。

图7.2 机器人寻路问题



如图7.2所示的3×3的网格，假设有一个机器人位于s_1，试图从s_1这一初始位置向s_9这一目标位置移动。假设机器人每一步只能向上或者向右移动一个方格，到达目标位置s_9则会获得奖励且游戏终止，机器人在移动过程中如果

越出方格则会被惩罚且被损坏，并且游戏终止。那么，如何学习一种策略能够帮助机器人从s_1走到s_9？这是一个典型的搜索问题，由此可见强化学习问题与搜索问题存在共通之处。

对照7.1.1节中介绍的概念，在这个问题中，智能主体为迷宫机器人，环境是3×3的方格，状态指的是机器人在当前时刻所处方格，状态取值范围为$\{s_1, s_2, \cdots, s_9, s_d\}$，其中$s_d$表示机器人被损坏时所处状态。机器人每次可采取的动作为向上或向右移动一个方格，机器人所获得的奖励包括它到达s_9时得到的正奖励，以及越界时得到的负奖励（惩罚）。要说明的是，机器人在到达s_d状态会被惩罚（即越界），在到达s_9时被奖赏，到达其他状态时不被奖赏，也不被惩罚。

明确了相应的概念之后，为了准确定义这个问题，需要形式化描述机器人与环境的交互过程。这里采用随机过程来描述机器人与环境之间的交互。

一个随机过程实际上是一列随时间变化的随机变量。当时间是离散量时，一个随机过程可以表示为$\{X_t\}_{t=0,1,2,\cdots}$，这里每个X_t都是一个随机变量，这被称为离散随机过程。为了方便分析和求解，通常要求通过合理的问题定义使得一个随机过程满足马尔可夫性（Markov property），即满足如下性质：

$$P\left(X_{t+1}=x_{t+1}\middle|X_0=x_0, X_1=x_1, \cdots, X_t=x_t\right)=P\left(X_{t+1}=x_{t+1}\middle|X_t=x_t\right)\quad（式7.1）$$

这个公式的直观解释为：下一刻的状态X_{t+1}只由当前状态X_t决定（而与更早的所有状态均无关）。满足马尔可夫性的离散随机过程被称为马尔可夫链（Markov chain）。

定义离散马尔可夫过程$\{S_t\}_{t=0,1,2,\cdots}$，其中S_t表示机器人第t步的状态。在这个问题中，S_t就是机器人第t步时所处的方格位置，每个随机变量S_t的取值范围称为状态集合S，在机器人寻路的问题中$S=\{s_1, s_2, \cdots, s_9, s_d\}$。由于$\{S_t\}_{t=0,1,2,\cdots}$满足马尔可夫性，因此可以定义状态转移概率$P(S_{t+1}|S_t)$。

为了在问题定义中反映机器人在完成目标过程中所获得的奖惩，需要在问题定义中加入奖励概念。因此，定义奖励函数$R: S\times S\mapsto\mathbb{R}$，其中$R(S_t, S_{t+1})$描述了从第$t$步状态转移到第$t+1$步状态所获得奖励。为了简化符号，记$R_{t+1}:=R(S_t, S_{t+1})$。在机器人寻路问题中，奖励值可定义如下：当$S_{t+1}=s_9$时奖励值为1，当$S_{t+1}=s_d$时惩罚值为$-1$，其他情况下奖励值为0。

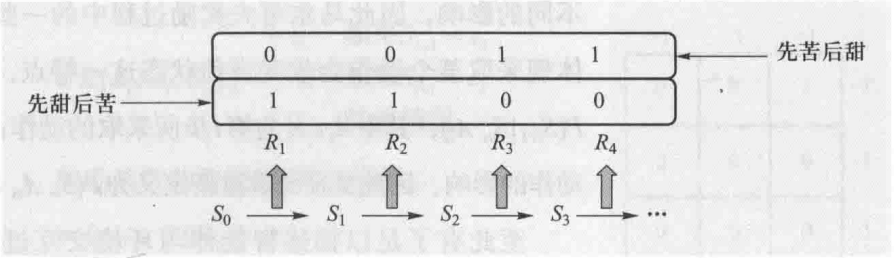


图7.3 两个奖励序列的比较

不难注意到机器人寻路问题中涉及多次状态转移，因此这个过程会产生一系列的奖励值$(R_1, R_2, \cdots,)$。这样，就需要比较不同状态转移所产生不同奖励序列的优劣。图7.3中给出了相同状态序列所对应两个不同奖励序列(0, 0, 1, 1)和(1, 1, 0, 0)。这两个序列中所有奖励之和均为2，区别在于在前一个奖励机制中对后期状态序列进行奖励，可谓先苦后甜；另一个奖励机制中对前期状态序列进行奖励，可谓先甜后苦。

为了比较不同奖励机制的优劣，在每个时刻定义回报（return）来反映该时刻可得到的累加奖励：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \qquad (式7.2)$$

其中$\gamma \in [0, 1]$是折扣因子（discount factor），$R_{t+k}(t>0)$表示$t+k$时刻获得的奖励。t时刻所获得的回报G_t反映了该时刻之后的累加奖励。当折扣因子γ小于1时，距离当前时刻越远的奖励对该时刻反馈贡献越少。假设$\gamma = 0.99$，则图7.3中两个奖励序列在0时刻的回报值分别为

$$(0,0,1,1): G_0 = 0 + 0.99 \times 0 + 0.99^2 \times 1 + 0.99^3 \times 1 = 1.950\,4$$

$$(1,1,0,0): G_0 = 1 + 0.99 \times 1 + 0.99^2 \times 0 + 0.99^3 \times 0 = 1.99$$

根据回报值可以认为(1, 1, 0, 0)是一个更好的奖励序列。这一结果也与现实生活中的认知活动相符，即距离某个时刻越近，所给予的奖励会对该时刻回报产生更大影响。因此，强化学习中的奖励机制更加注重短期收益，而非长期收益。

在马尔可夫链模型$MP = (S, P)$中加入奖励函数和折扣因子后，可得到的模型被称为马尔可夫奖励过程（Markov reward process，MRP），其形式化定义为$MRP = (S, P, R, \gamma)$。这个模型虽然能够用奖励和回报来刻画智能体的目标，但是仍然不能体现机器人的能动性，即缺乏让机器人与环境进行交互的手段。为了解决这个问题，需要在马尔可夫奖励过程中引入动作，让动作成为机器人影响环境的手段。

定义智能体能够采取的动作集合为A。由于不同的动作对环境造成

不同的影响，因此马尔可夫奖励过程中的一些定义要随之修改。为了体现采取某个动作会改变当前状态这一特点，将转移概率重新定义为$P(S_{t+1}|S_t, A_t)$，其中$A_t \in A$为第t步所采取的动作；同时，奖励也可能受到动作的影响，因此奖励函数重新定义为$P(S_t, A_t, S_{t+1})$。

至此有了足以描述智能体与环境交互过程的所有要素，可如下定义马尔可夫决策过程（Markov decision process，MDP）[4, 36]为$MDP=(S, A, P, R, \gamma)$。其中的参数解释如下。

- 状态集合S：所求解问题中所有可能出现的状态所构成的集合，这个集合可能是一个有限的集合，也可能是一个无限的集合。
- 动作集合A：所求解问题中智能体能够采取的所有动作所构成集合，这个集合同样可以是有限的，也可以是无限的，本章所述内容中均假设该集合是一个有限集合。
- 状态转移概率$P(S_{t+1}|S_t, A_t)$：表示在当前状态S_t下采取了动作A_t后进入下一时刻状态S_{t+1}的概率。显然，状态转移概率满足马尔可夫性。状态转移可以是概率性的（stochastic），也可以是确定的（deterministic）。确定的状态转移指在给定状态S_t下采取了动作A_t后，转移到某一状态的概率为1。
- 奖励函数$R(S_t, A_t, S_{t+1})$：在状态S_t执行了动作A_t后到达状态S_{t+1}时，智能体能够得到的奖励。
- 折扣因子γ：后续时刻奖励对当前动作的价值系数，$\gamma \in [0, 1]$。

若用马尔可夫决策过程来刻画图7.2中的机器人寻路问题，则该马尔可夫决策过程中所有的信息如下。

- $S=\{s_1, s_2, \cdots, s_9, s_d\}$
- $A=\{$上，右$\}$

状态转移函数如图7.4所示。

图7.4 机器人寻路问题的状态转移函数

$P(S_{t+1}|S_t, a_t=$右$)$

S_t \ S_{t+1}	S_1	S_2	S_3	…	S_9	S_d
S_1	0	1	0	…	0	0
S_2	0	0	1	…	0	0
S_3	0	0	0	…	0	1
⋮				⋮		⋮
S_8	0	0	0	…	1	0
S_9	0	0	0	…	0	1

$P(S_{t+1}|S_t, a_t=$上$)$

S_t \ S_{t+1}	S_1	S_4	S_7	…	S_9	S_d
S_1	0	1	0	…	0	0
S_4	0	0	1	…	0	0
S_7	0	0	0	…	0	1
⋮				⋮		⋮
S_6	0	0	0	…	1	0
S_9	0	0	0	…	0	1

- $R(S_t, A_t, S_{t+1}) = \begin{cases} 1, & \text{如果} S_{t+1} = s_9 \\ -1, & \text{如果} S_{t+1} = s_d \\ 0, & \text{其他情况} \end{cases}$

奖励函数的取值如图7.5所示。

−1	−1	−1	
0	0	1	−1
0	0	0	−1
0	0	0	−1

图7.5 机器人寻路问题中的奖惩函数

- 令 $\gamma = 0.99$。
- 初始状态 $S_0 = s_1$，终止状态集合 $S_T = \{s_9, s_d\}$。

值得注意的是，一般情况下，初始状态和终止状态并不包括在马尔可夫决策过程的定义中，因为在标准的马尔可夫决策过程模型中，初始状态和终止状态可以认为是独立于问题的固定状态。如果指定了与问题相关的初始状态和终止状态，那么只需添加一个虚拟的初始状态和一个虚拟的终止状态，令虚拟的初始状态能够以一定的概率转移到真正的初始状态，令真正的终止状态能够以概率1转移到虚拟的终止状态，这样一来问题便化为了标准的马尔可夫决策过程。为了简化表达，本章在定义具体问题时会引入初始状态和终止状态这两个概念。

马尔可夫决策过程刻画了智能体与环境的交互特点，这一交互可用图7.6简洁概括。图中，智能体从初始状态 S_0 开始，根据策略执行动作 A_0，得到奖励值 R_1，同时状态转移到 S_1，以此类推。于是，可得到一个状态序列 $(S_0, S_1, \cdots)$，该序列称为**轨迹**（trajectory），轨迹长度可以是无限的，也可以有终止状态 S_T。状态序列中包含终止状态的问题叫**分段**（episodic）问题，不包含终止状态的问题叫**持续**（continuing）问题。在分段问题中，一个从初始状态到终止状态的完整轨迹称为一个**片段**（episode）。

本章讨论的主要是分段问题，有时也将动作和奖励记入状态序列中，用 $(S_0, A_0, R_1, S_1, A_1, R_2, S_2, \cdots, S_T)$ 来描述一个片段。

7.1.3 强化学习问题定义

马尔可夫决策过程被广泛用于描述强化学习问题中智能体与环境的交互过程[1, 35]，但其并不能对智能体选择动作的方法建模。

为了形式化定义强化学习问题，有必要建立智能体选择动作的模型，

图7.6 智能体与环境的交互过程

$$S_0 \xrightarrow[A_0]{R_1} S_1 \xrightarrow[A_1]{R_2} S_2 \xrightarrow[A_2]{R_3} S_3 \xrightarrow[A_3]{R_4} \cdots$$

这就是策略函数 $\pi: S \times A \mapsto [0,1]$，$\pi(s, a)$ 表示智能体在状态 s 下采取动作 a 的概率。策略函数可以是概率性的，也可以是确定的。确定的策略函数指在给定状态 s 情况下，只有一个动作 a 使得概率 $\pi(s, a)$ 取值为 1。对于确定的策略函数，为了简化符号，记 $a = \pi(s)$。

一个好的策略函数应该能够使得智能体在采取了一系列行动后可得到最佳奖励，即最大化每一时刻的回报值 $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$。从 G_t 的定义可知，G_t 要根据一次包含了终止状态的轨迹序列来计算，因此定义如下函数。

- **价值函数**（value function）：$V: S \mapsto \mathbb{R}$，其中 $V_\pi(s) = \mathbb{E}_\pi\left[G_t \middle| S_t = s\right]$ 表示智能体在时刻 t 处于状态 s 时，按照策略 π 采取行动时所获得回报的期望。价值函数衡量了某个状态的好坏程度，反映了智能体从当前状态出发后在将来还能获得多少好处。

- **动作-价值函数**(action-value function)：$q: S \times A \mapsto \mathbb{R}$，其中 $q_\pi(s, a) = \mathbb{E}_\pi\left[G_t \middle| S_t = s, A_t = a\right]$，表示智能体在时刻 t 处于状态 s 时，选择了动作 a 后，在 t 时刻后根据策略 π 采取行动所获得回报的期望。

价值函数和动作-价值函数反映了智能体在某一策略下所对应状态序列获得回报的期望，它比回报本身更加准确地刻画了智能体的目标。注意，价值函数和动作-价值函数的定义之所以能够成立，离不开决策过程所具有的马尔可夫性，即当位于当前状态 s 时，无论当前时刻 t 的取值是多少，一个策略回报值的期望是一定的（当前状态只与前一状态有关，与时间无关）。

至此，强化学习可以转化为一个策略学习问题，其定义为：给定一个马尔可夫决策过程 $MDP = (S, A, P, R, \gamma)$，学习一个最优策略 π^*，对任意 $s \in S$ 使得 $V_{\pi^*}(s)$ 值最大。

7.1.4 贝尔曼方程（Bellman equation）

贝尔曼方程（Bellman equation）也被称为动态规划方程（dynamic programming equation），由理查德·贝尔曼（Richard Bellman）提出[5]。

已知价值函数和动作价值函数的定义如下：

$$V_\pi(s) = \mathbb{E}_\pi\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \middle| S_t = s\right]$$

$$q_\pi(s, a) = \mathbb{E}_\pi\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \middle| S_t = s, A_t = a\right]$$

如果对价值函数的公式稍加变形，可得：

$$
\begin{aligned}
V_\pi(s) &= \mathbb{E}_\pi\left[R_{t+1}+\gamma R_{t+2}+\gamma^2 R_{t+3}+\cdots \middle| S_t=s\right] \\
&= \mathbb{E}_{a\sim\pi(s,\cdot)}\left[\mathbb{E}_\pi\left[R_{t+1}+\gamma R_{t+2}+\gamma^2 R_{t+3}+\cdots \middle| S_t=s, A_t=a\right]\right] \\
&= \sum_{a\in A} \underbrace{\pi(s,a)}_{\text{采取动作}a\text{的概率}} \times \underbrace{q_\pi(s,a)}_{\text{采取动作}a\text{后带来的回报期望}} \\
&= \sum_{a\in A}\pi(s,a)q_\pi(s,a)
\end{aligned}
\qquad (式7.3)
$$

可见，可以用动作-价值函数来表达价值函数，即从状态s出发，采用策略π完成任务所得回报的期望，可如下计算：在状态s下可采取的每个动作的概率值与采取这一动作而获得价值的乘积之和。这里的$\pi(s, a)$表示在状态s下采取动作a的概率，$q_\pi(s, a)$为在状态s下采取动作a后的回报期望。也就是说，状态s的价值可用该状态下可采取所有动作而取得的期望价值来表述。

同理，也可根据动作-价值函数的定义给出以下推导：

$$
\begin{aligned}
q_\pi(s,a) &= \mathbb{E}_\pi\left[R_{t+1}+\gamma R_{t+2}+\gamma^2 R_{t+3}+\cdots \middle| S_t=s, A_t=a\right] \\
&= \mathbb{E}_{s'\sim P(\cdot|s,a)}\left[R(s,a,s')+\gamma\mathbb{E}_\pi\left[R_{t+2}+\gamma R_{t+3}+\cdots \middle| S_{t+1}=s'\right]\right] \\
&= \sum_{s'\in S} \underbrace{P(s'|s,a)}_{\text{在状态}s\text{采取行动}a\text{进入状态}s\text{'的概率}} \times\Bigg[\underbrace{R(s,a,s')}_{\text{在}s\text{采取}a\text{进入}s\text{'得到的回报}} +\gamma\times \underbrace{V_\pi(s')}_{\text{在}s\text{'获得的回报期望}}\Bigg] \\
&= \sum_{s'\in S} P(s'|s,a)\left[R(s,a,s')+\gamma V_\pi(s')\right]
\end{aligned}
\qquad (式7.4)
$$

于是，也可用价值函数来表示动作-价值函数，即在状态s下采取动作a所取得的价值，可如下计算：采取某个具体动作a进入状态s'的概率，乘以进入后续状态s'所得回报$R(s, a, s')$与后续状态s'价值函数的折扣值之和，然后对在状态s采取动作a后所进入全部状态的如上取值进行累加。这里$P(s'|s, a)$表示在状态s下采取动作a转移到状态s'的概率。也就是说，在某个状态下执行某一个动作所取得价值可以通过执行该动作之后进入的所有状态获得的瞬时奖励和后续状态可取得价值的期望来表示。

如果将（式7.3）中$q_\pi(s, a)$替换为（式7.4）中形式，可以得到如下的价值函数贝尔曼方程：

$$
\begin{aligned}
V_\pi(s) &= \sum_{a\in A}\pi(s,a)\sum_{s'\in S}P(s'|s,a)\left[R(s,a,s')+\gamma V_\pi(s')\right] \\
&= \mathbb{E}_{a\sim\pi(s,\cdot)}\mathbb{E}_{s'\sim P(\cdot|s,a)}\left[R(s,a,s')+\gamma V_\pi(s')\right]
\end{aligned}
\qquad (式7.5)
$$

从这个公式可知，价值函数取值与时间没有关系，只与策略π、在策略π下从某个状态转移到其后续状态所取得的回报以及在后续所得回

报有关。

进一步分析价值函数的贝尔曼方程，可见状态s下可获得“好处”$V_\pi(s)$由两个部分构成：一个是在状态s执行当前动作所得到的瞬时奖励，另外一个是在后续状态所能得回报的期望（价值）的折扣值。该式中出现了期望，是因为在状态s下可以一定概率进入到多个后续状态，且从状态s进入某个后续状态均会有不同的价值。

由于在每个状态都有多种动作可供智能体选择，在每个状态施以某个具体动作后可按照一定概率转移到一个新的后续状态，因此相比于关心每个状态的价值，显然关心在当前状态下施以某个动作带来的价值更加直观实用。如果智能体已经知道在某个状态下所有可供选择动作带来的不同价值，那么智能体在当前状态就应该去选择能带来最大价值的对应动作去执行，这就是设计动作-价值函数的初衷。

动作-价值函数中的回报与价值函数中的回报值不一样，动作-价值函数中的回报是在某个状态下执行完某个具体动作之后取得回报的期望值，而价值函数中的回报值是在某个状态下选择所有动作执行后所得回报的期望值。

对于动作-价值函数，只需将（式7.4）中的$V_\pi(s')$替换成（式7.3）中的形式，就可以得到如下的动作-价值函数贝尔曼方程：

$$\begin{aligned} q_\pi(s,a) &= \sum_{s'\in S} P(s'|s,a)\left[R(s,a,s')+\gamma\sum_{a'\in A}\pi(s',a')q_\pi(s',a')\right] \\ &= \mathbb{E}_{s'\sim P(\cdot|s,a)}\left[R(s,a,s')+\gamma\mathbb{E}_{a'\sim\pi(s',\cdot)}\left[q_\pi(s',a')\right]\right] \end{aligned} \quad \text{（式7.6）}$$

（式7.6）说明动作-价值函数取值同样与时间没有关系，而是与瞬间奖励和下一步的状态和动作有关。动作-价值函数表示在状态s下采取了动作a后获得的“好处”，这也可以分为两部分：一是从状态s采取动作a后带来的瞬时奖励，二是进入后续状态后根据当前策略选择动作所得期望回报的折扣值。

贝尔曼方程描述了价值函数或动作-价值函数的递推关系，是研究强化学习问题的重要手段。

其中价值函数的贝尔曼方程描述了当前状态价值函数和其后续状态价值函数之间的关系，即当前状态价值函数等于瞬时奖励的期望加上后续状态的（折扣）价值函数的期望。

而动作-价值函数的贝尔曼方程描述了当前动作-价值函数和其后续动作-价值函数之间的关系，即当前状态下的动作-价值函数等于瞬时奖励的期望加上后续状态的（折扣）动作-价值函数的期望。

在实际中，需要计算得到最优策略以指导智能体在当前状态如何选择一个可获得最大回报的动作。求解最优策略的一种方法就是去求解最优的价值函数或最优的动作-价值函数（即基于价值方法，value-based approach）。一旦找到了最优的价值函数或动作-价值函数，自然而然也就是找到最优策略。当然，在强化学习中还有基于策略（policy-based）和基于模型（model-based）等不同方法。

7.2 基于价值的强化学习

7.2.1 策略迭代的基本模式

回顾强化学习问题的定义：给定一个马尔可夫决策过程 $MDP=(S, A, P, R, \gamma)$，强化学习会寻找一个最优策略 π^*，在策略 π^* 作用下使得任意状态 $s \in S$ 对应的价值函数 $V_{\pi^*}(s)$ 取值最大。

为了求解最优策略 π^*，图7.7中展示了一种思路：从一个任意的策略开始，首先计算该策略下价值函数（或动作-价值函数），然后根据价值函数调整改进策略使其更优，不断迭代这个过程直到策略收敛。通过策略计算价值函数的过程叫做策略评估（policy evaluation），通过价值函数优化策略的过程叫做策略优化（policy improvement），策略评估和策略优化交替进行的强化学习求解方法叫做通用策略迭代（generalized policy iteration，GPI）。

几乎所有的强化学习方法都可以用通用策略迭代来解释，本节将从通用策略迭代的角度来介绍基于价值的强化学习方法，方便起见，本节

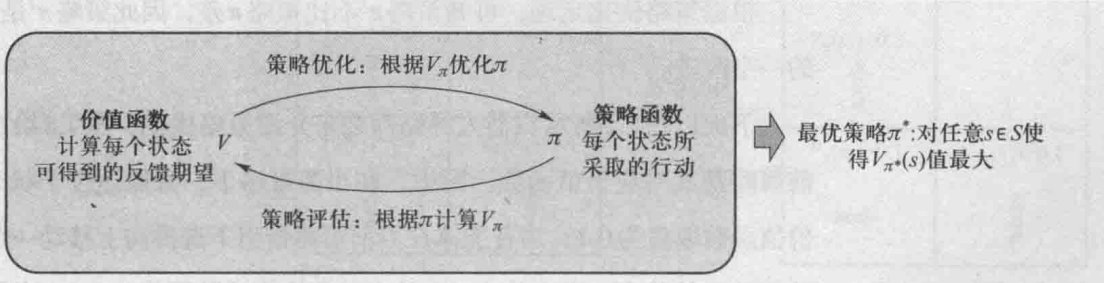


图7.7 基于通用策略迭代来学习最优策略的模式

讨论的是一种简单的情况，即假设策略是确定的（deterministic）。

7.2.2 策略优化定理

本小节介绍通用策略迭代中的策略优化步骤。在讨论如何优化策略之前，首先需要明确什么是“更好”的策略。分别给出π和π'两个策略，如果对于任意状态$s \in S$，有$V_\pi(s) \leqslant V_{\pi'}(s)$，那么可以认为策略$\pi'$不比策略$\pi$差，可见“更优”策略是一个偏序关系。

可以证明，给定任意给定状态$s \in S$，如果两个策略π和π'满足如下条件：

$$q_\pi\left(s, \pi'(s)\right) \geqslant q_\pi\left(s, \pi(s)\right)$$

那么对于该任意给定状态$s \in S$，有

$$V_{\pi'}(s) \geqslant V_\pi(s)$$

即策略π'不比策略π差。这个结论称为策略优化定理（policy improvement theorem）[5]。注意$q_\pi\left(s, \pi'(s)\right)$的含义并不是在当前状态$s$下按照策略$\pi'$行动的回报期望，而是在当前状态$s$下按照策略$\pi'$去选择动作$\pi'(s)$，但是从状态$s$进入后续状态后，后续状态以后（直至终止状态）的所有动作仍然按照原有策略π来选择，所得到的回报期望。注意，按照前面所给出的定义，如果策略π在状态s时只有一个确定的动作可供选择，那么这个动作可以记为$\pi(s)$（$\pi'(s)$也具有同样的含义）。

假设当前策略为π，其对应的价值函数和动作-价值函数为V_π和q_π。构造策略π'使得对于任意$s \in S$，有如下条件成立：

$$\pi'(s) = \operatorname{argmax}_a q_\pi(s, a) \qquad \text{（式7.7）}$$

于是对于任意$s \in S$，有

$$\begin{aligned} q_\pi\left(s, \pi'(s)\right) &= q_\pi\left(s, \operatorname{argmax}_a q_\pi(s, a)\right) \\ &= \max_a q_\pi(s, a) \\ &\geqslant q_\pi\left(s, \pi(s)\right) \end{aligned}$$

根据策略优化定理，可知策略π'不比策略π差，因此策略π'是策略π的一个改进。

下面以图7.2所示机器人寻路问题来介绍策略优化。图7.8给出了当前策略及其对应价值函数。图中，在当前策略下，智能体位于状态s_1的价值函数取值为0.1。当智能体在当前策略指引下选择向上移动一个方格的行动，从状态s_1进入状态s_4，状态s_4的价值函数取值为0.3。当前策略

所给出的其他信息，也可从图7.8周知。

下面来看智能体如何通过策略优化来改变在状态s_1所采取的行动。由于智能体在状态s_1能够采取“向上移动一个方格”或“向右移动一个方格”两个行动中的一个，首先计算状态s_1选择这两个不同动作后分别所得动作-价值函数取值：

$$\begin{aligned} q_\pi(s_1, \text{上}) &= \sum_{s' \in S} P(s'|s_1, \text{上}) \left[R(s_1, \text{上}, s') + \gamma V_\pi(s') \right] \\ &= 1 \times (0 + 0.99 \times 0.3) + 0 \times \cdots = 0.297 \end{aligned}$$

$$\begin{aligned} q_\pi(s_1, \text{右}) &= \sum_{s' \in S} P(s'|s_1, \text{右}) \left[R(s_1, \text{右}, s') + \gamma V_\pi(s') \right] \\ &= 1 \times (0 + 0.99 \times 0.4) + 0 \times \cdots = 0.396 \end{aligned}$$

可见，智能体在状态s_1选择“向上移动一个方格”行动所得回报$q_\pi(s_1, \text{上})$值为0.297，选择“向右移动一个方格”行动所得回报$q_\pi(s_1, \text{右})$值为0.396。显然，智能体在状态s_1应该选择“向右移动一个方格”行动，这样能够获得更大的回报。

于是，经过策略优化后，状态s_1处的新策略为$\pi'(s_1) = \text{argmax}_a q_\pi(s_1, a) = \text{右}$，则将$s_1$处的策略从“上”更新为“右”。其他状态的情况可用类似的方法计算得到。

总结来说，给定策略π、价值函数V_π和动作-价值函数q_π，那么算法可以根据（式7.7）计算得到更好或同等的策略。

7.2.3 策略评估方法

假定当前策略为π，策略评估指的是根据策略π来计算相应的价值函数V_π或动作-价值函数q_π。本小节将介绍在状态集合有限前提下三种常见的策略评估方法，它们分别是基于动态规划的方法、基于蒙特卡洛采

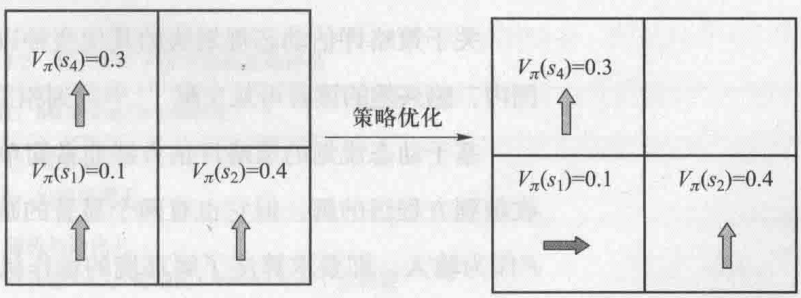


图7.8 对状态s_1所采取的行动进行策略优化的示意图

样的方法和时序差分（temporal difference）法。状态集合非常大甚至无限的情况将在7.2.5小节中讨论。

（式7.5）给出了关于价值函数的贝尔曼方程，这个方程描述了不同状态下价值函数的递推关系。如果对所有状态均列出贝尔曼方程，即让所有状态在（式7.5）左侧分别出现一次，会得到$|S|$个一次方程，如果将每个状态的价值函数看做未知数，那么这就是一个有$|S|$个方程所组成的$|S|$元一次方程组。这个方程组的解对应了当前策略下每个状态的价值。

当然，求解线性系统的方法都可以用来求解价值函数，如高斯消元法等。在这些方法中，迭代法有其独特的优势，如适合处理稀疏矩阵、便于随时停止等，在下一小节中还将看到，迭代法还易与策略优化过程相结合。

使用迭代法求解贝尔曼方程组思路与动态规划思想不谋而合。实际上，在递推关系基础上来利用动态规划思想求解价值函数被统称为策略评估的动态规划法。算法7.1给出了其中一种典型的方法，即使用高斯-赛德尔迭代法（Gauss-Seidel method）来求解价值函数的贝尔曼方程组。这个算法不断利用递推关系来更新每个状态的价值函数，直到价值函数收敛。

仍然以图7.8所示情况为例，如果要更新$V_\pi(s_1)$，则可如下操作：

$$\begin{aligned} q_\pi(s_1, \text{上}) &= \sum_{s' \in S} P(s'|s_1, \text{上})\left[R(s_1, \text{上}, s') + \gamma V_\pi(s')\right] \\ &= 1 \times (0 + 0.99 \times 0.3) + 0 \times \cdots = 0.297 \\ V_\pi(s_1) &= \sum_{a \in A} \pi(s_1, a) q_\pi(s_1, a) \\ &= 1 \times 0.297 + 0 \times \cdots = 0.297 \end{aligned}$$

因此$V_\pi(s_1)$的值被更新为0.297，对其他状态进行相同操作，并不断迭代直到V_π收敛。

关于策略评估动态规划法的其他变种以及收敛分析并不在本书的范围内，感兴趣的读者可从文献[7]中找到相关内容。

基于动态规划的策略评估方法非常简单，实际应用中V_π也通常能够收敛到方程组的解，但它也有两个显著的缺点：要求状态转移概率函数P作为输入，即要求算法了解环境的运作机理（也就是说要对环境构建明确的模型），这常常是不现实的；若用动态规划法精确计算每个状态的

价值函数，在状态集合很大时将导致较低效率。这两个原因导致了算法7.1在很多情况下并不实用。

算法7.1 基于动态规划的策略评估

函数：DPPolicyEvaluation（ ）

输入：策略π，状态转移函数P，奖励函数R
输出：价值函数V_π

1 随机初始化V_π
2 **repeat**
3 　**foreach** $s \in S$ **do**
4 　　$V_\pi(s) \leftarrow \sum_{a \in A} \pi(s,a) \sum_{s' \in S} P(s'|s,a)\left[R(s,a,s') + \gamma V_\pi(s')\right]$
5 　**end**
6 **until** V_π收敛

在现实中，智能体通常无法知道在当前状态s采取行动a后转移到其后续状态s'的概率（即算法7.1第4行中$P(s'|s, a)$取值），然后基于这个概率来计算状态s的价值函数。

但是根据数理统计的知识，期望可以通过样本均值来估计的，这正是蒙特卡洛方法（Monte-Carlo method）[3, 12]的核心思想。之所以可以通过样本均值来估计期望，是因为大数定理指出：对于独立同分布（independent and identically distributed，IID）的样本数据，当样本足够大的时候，样本平均值向期望值收敛。要注意的是均值是一个统计量（对所观测样本的统计而言），期望是一种概率论概念（对随机变量的出现概率而言）。

按照这样的思路可使用蒙特卡洛方法来进行策略评估：给定状态s，从该状态出发不断采样后续状态，得到不同的采样序列。通过这些采样序列来分别计算状态s的回报值，对这些回报值取均值，作为对状态s价值函数的估计，从而避免对状态转移概率的依赖。算法7.2给出了基于蒙特卡洛方法的策略评估过程。

算法7.2 基于蒙特卡洛方法的策略评估

函数：MCPolicyEvaluation（ ）

输入：策略π
输出：价值函数V_π

1 随机初始化V_π
2 对任意$s \in S$，初始化$s.returns$为一个空列表
3 **for** $i \leftarrow 1$ **to** MaxIter **do**

```
4   根据π产生片段D
5   foreach s ∈ S do
6       G←s在D中初次出现时的反馈
7       append (s.returns,G)
8       V_π(s)←average(s.returns)
9   end
10 end
```

例如，在图7.2的机器人寻路问题中，假设通过采样得到了以下两条从状态s_1出发的轨迹：(s_1, s_4, s_7, s_d)和$(s_1, s_4, s_7, s_8, s_9)$，那么在这两条轨迹中$s_1$对应的回报值分别为$0+\gamma\times 0+\gamma^2\times(-1)=-0.980$和$0+\gamma\times 0+\cdots+\gamma^3\times 1=0.970$。由于在解决比较复杂的问题时，从一个状态出发可以得到接近“无限”多个后续状态序列（特别是状态为一个连续量时），因此可将s_1的价值函数$V(s_1)$视为随机变量。又根据大数定理所言，当随机变量采样所得样本足够大时，随机变量平均值向期望值收敛。因此，s_1的价值函数$V_\pi(s_1)$的估计值可为$V(s_1)=\frac{1}{2}(0.970-0.980)=-0.005$。当然，这里为了简化讲解，只是采样了两条从状态$s_1$出发的轨迹序列。在实际计算中，可从状态$s_1$出发采样更多的轨迹序列，计算$s_1$相对于这些轨迹系列价值函数的均值，作为$s_1$的价值函数取值。

蒙特卡洛采样法避免了动态规划法所面临的难点问题：一方面蒙特卡洛采样法不依赖状态之间的转移概率，另一方面蒙特卡洛法能够将计算资源集中在感兴趣的（被采样的）状态上，以此克服状态过多时计算效率低下的问题。

但是，蒙特卡洛采样法也存在许多不足之处：一方面是对价值函数估计有时候难以准确，因为从某个状态出发，可采样得到的轨迹序列数量通常很多，导致根据不同轨迹序列计算所得价值函数取值之间方差很大，用这些有限的轨迹序列来估计价值函数可能不准确；另一方面，在很多实际问题中，只有转移到终止状态才能获知回报值，这导致采样轨迹序列较长，严重影响算法效率。

为了解决上述不足，时序差分（temporal-difference）这一策略评估方法被提出[28]。时序差分法可以看作蒙特卡罗方法和动态规划方法的有机结合。时序差分算法与蒙特卡洛方法相似之处在于，时序差分方法从实际经验中获取信息，无需提前获知环境模型的全部信息。时序差分算法与动态规划方法的相似之处在于，时序差分方法能够利用前序已知

信息来进行在线实时学习，无需等到整个片段结束（终止状态抵达）再进行价值函数的更新。

时序差分流程如算法7.3所示。在第1行中，价值函数被随机初始化，同时终止状态的价值函数初始化为0。第2行进入外层循环，这个循环不断通过采样片段轨迹来更新轨迹中的状态所对应的价值函数，直到价值函数收敛。第3行选择一个状态为初始状态（记为s）。第4行进入内层循环，这个循环从s出发，不断采样后续状态，更新后续状态的价值函数，直至抵达终止状态。令当前状态为s，当前策略为π，第5行根据当前策略选择一个当前状态可采取的动作。第6行执行该动作，并从环境获取奖励和进入后续状态。第7行更新当前状态的价值函数。第8行更新当前状态。

算法7.3　基于时序差分法的策略评估

函数：TemporalDifference（ ）

输入：策略π
输出：价值函数V_π

```
1  随机初始化 V_π
2  repeat
3  |  s ← 初始状态
4  |  repeat
5  |  |  a~π(s, ·)
6  |  |  执行动作a，观察奖励R和下一个状态s'
7  |  |  V_π(s) ← V_π(s) + α[R + γV_π(s') − V_π(s)]
8  |  |  s ← s'
9  |  until s 是终止状态
10 until V_π收敛
```

动态规划法根据贝尔曼方程$V_\pi(s)=\mathbb{E}_{a\sim\pi(s,\cdot),s'\sim P(\cdot|s,a)}\left[R(s,a,s')+\gamma V_\pi(s')\right]$迭代更新价值函数，问题在于计算期望$\mathbb{E}_{a\sim\pi(s,\cdot),s'\sim P(\cdot|s,a)}$时要求算法事先知道状态之间的转移概率，这往往是不现实的。为了解决这个问题，时序差分法借鉴蒙特卡洛法思想，通过采样a和s'来估计$V_\pi(s)$的取值$R(s,a,s')+\gamma V_\pi(s')$。

由于通过采样进行计算，所得结果可能不准确，因此时序差分法并没有将这个估计值照单全收，而是以α作为权重来接受新的估计值，即把价值函数更新为$(1-\alpha)V_\pi(s)+\alpha\left[R+\gamma V_\pi(s')\right]$，对这个式子稍加整理就能得到算法7.3中第7行形式：$V_\pi(s)\leftarrow V_\pi(s)+\alpha\left[R+\gamma V_\pi(s')-V_\pi(s)\right]$。这里的$R+\gamma V_\pi(s')$为时序差分目标，$R+\gamma V_\pi(s')-V_\pi(s)$为时序差分偏差。

时序差分法和蒙特卡洛法都是通过采样若干个片段来进行价值函数

更新的，但是时序差分法并非使用一个片段中的终止状态所提供的实际回报值来估计价值函数，而是根据下一个状态的价值函数来估计，这样就克服了采样轨迹的稀疏性可能带来样本方差较大的问题，同时也缩短了反馈周期。和蒙特卡洛法一样，时序差分法也会将计算资源倾向于部分状态，关于这个特点的利弊和应对方式，在后面的章节会有更加详细的讨论。

关于时序差分法的收敛性问题本书不作详细讨论，相关内容可参考[8, 28]。

以图7.8中左子图的情况为例，这里给出当参数$\alpha=0.5$时，用时序差分法来计算$V_\pi(s_1)$的过程。首先根据$\pi(s_1)$得到动作“$a=$上”，根据$P(\cdot|s_1,\text{上})$采样得到下一步所抵达的状态s_4，因此将$V_\pi(s_1)$的值如下更新：

$$\begin{aligned}V_\pi(s_1)&\leftarrow V_\pi(s_1)+\alpha\left[R(s_1,\text{上},s_4)+\gamma V_\pi(s_4)-V_\pi(s_1)\right]\\&=0.1+0.5\times\left[0+0.99\times0.3-0.1\right]=0.199\end{aligned}$$

在对片段进行采样的同时，不断以上述方法更新当前状态的价值函数，不断迭代直到价值函数收敛为止。

本小节介绍了三种根据策略求解价值函数的方法，这些方法通过不同的方式估计当前状态的价值函数取值，然后通过迭代法进行更新。一个好的估计必须同时降低估计的偏差（bias）和样本方差（variance），这两个目标常常是相互矛盾的，但也有一些改进方案尝试在两者间取得平衡[19, 35]，本书对此不展开进一步的讨论。

7.2.4 基于价值的强化学习算法

7.2.2小节和7.2.3小节分别介绍了策略优化和策略评估的方法，根据通用策略迭代的思路，只需迭代执行这两个步骤，就能得到求解强化学习问题的算法。按照最直观的想法，由策略优化和策略评估交替迭代得到的算法如算法7.4所示。其中第3行的PolicyEvaluation函数可以使用任何7.2.3小节中介绍的方法，当然为了配合策略优化定理，策略评估需要将计算价值函数V_π调整为计算动作-价值函数q_π。

算法7.4 直观构造的强化学习算法

函数：PolicyIteration（ ）
输入：马尔可夫决策过程$MDP=(S, A, P, R, \gamma)$ 输出：策略π

```
1 随机初始化π
2 repeat
3 |  q_π ← PolicyEvaluation(π)
4 |  foreach s ∈ S do
5 |  |  π(s) ← argmax_a q_π(s, a)
6 |  end
7 until π收敛
```

算法7.4存在的主要问题是，策略评估函数本身也是迭代求解的，如果每次都要等待策略评估函数收敛，必然会影响算法执行的效率。另一方面，随着策略评估迭代次数的增多，动作-价值函数的变化也会越来越小，有时候甚至小到不足以影响新策略的取值，这种情况下继续对价值函数进行迭代计算的意义就不大了。

实际上，以更细的粒度进行策略评估和策略优化的迭代通常并不会影响算法的收敛性，而且往往有着更高的效率。例如，在策略评估动态规划法的基础上，每次迭代中只对一个状态进行策略评估和策略优化，就可以得到算法7.5，这个算法被称为价值迭代（value iteration）算法[20]。

算法7.5　价值迭代算法

函数：ValueIteration（ ）

输入：马尔可夫决策过程$MDP=(S, A, P, R, \gamma)$
输出：策略π

```
1 随机初始化 V_π
2 repeat
3 |  foreach s ∈ S do
4 |  |  V_π(s) ← max_a Σ_s' P(s'|s,a)[R(s,a,s') + γV_π(s')]
5 |  end
6 until V_π收敛
7 π(s) := arg max_a Σ_s' P(s'|s,a)[R(s,a,s') + γV_π(s')]
```

算法7.5中的第4行可以拆分成如下两步，即：

$$\begin{aligned}\pi(s) &\leftarrow \operatorname{argmax}_a q_\pi(s,a)\\ V_\pi(s) &\leftarrow q_\pi(s,\pi(s))\end{aligned} \qquad (式7.8)$$

其中$q_\pi(s,a)=\sum_{s'}P(s'|s,a)\left[R(s,a,s')+\gamma V_\pi(s')\right]$。这两步分别对应了策略优化和策略评估两个阶段。也就是说，在算法7.5的内层循环中，就已经交替进行了一次策略优化和一次策略评估。

以图7.2所示的机器人寻路问题为例，假设价值函数的初值如图7.9（a）所示，则完成算法7.5中一次外层循环后，价值函数的值如图7.9（b）所示。

以状态s_8为例：

$$q_\pi(s_8, 右) = \sum_{s'} P(s'|s_8, 右)\left[R(s_8, 右, s') + \gamma V_\pi(s')\right]$$
$$= 1\times(1+0.99\times 0)+0\times\cdots = 1$$
$$q_\pi(s_8, 上) = \sum_{s'} P(s'|s_8, 上)\left[R(s_8, 上, s') + \gamma V_\pi(s')\right]$$
$$= 1\times(-1+0.99\times 0)+0\times\cdots = -1$$
$$V(s_8) = \max_a q_\pi(s_8, a) = \max\{1, -1\} = 1$$

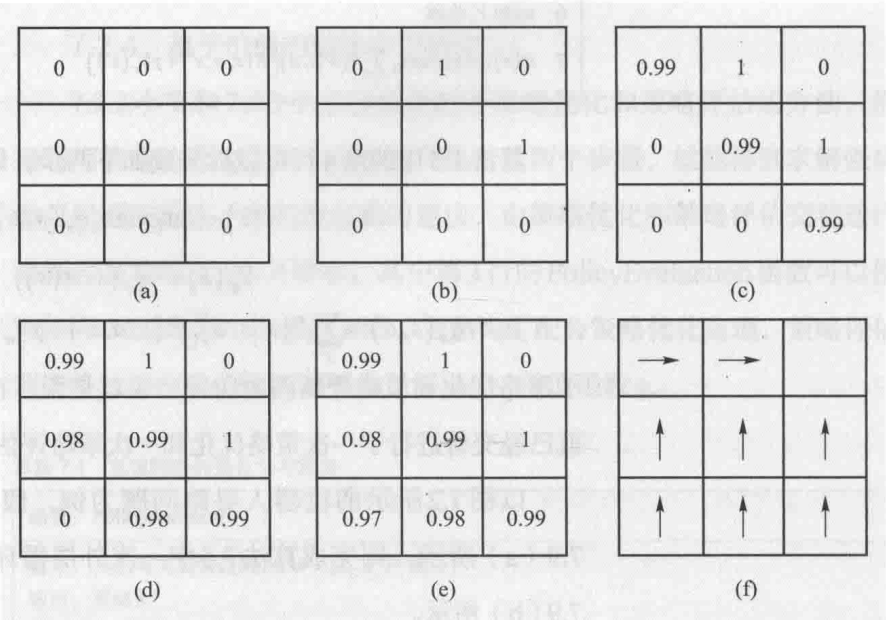
因此状态s_8的价值函数更新为1，同理可更新其他状态的价值函数。图7.9（c）和图7.9（d）分别为完成了2次和3次外层循环后的价值函数，而图7.9（e）则为完成5次外层循环后的结果，这个结果已经收敛。

从这些图中可以看出，由于s_8向右移动一个方格进入终止状态会获得1这样的回报，在价值迭代过程中，可以看到这会使得产生这一回报的状态由于选择了相应行动而获得相应价值，如状态s_8的价值函数从初始时取值0更新为1（见图7.9（a）和图7.9（b））。

图7.9（f）给出了根据图7.9（e）计算出的策略（动作-价值相同的情况优先取动作“上”）。可见智能体从s_1出发，根据图7.9（f）中的策略，可以沿着轨迹$(s_1, s_4, s_7, s_8, s_9)$到达目标。

价值迭代是一种经典的强化学习算法，但是它与用于策略评估的动态规划法具有同样的不足，即需要依赖状态转移函数P和计算所有状态所带来的效率低下问题。

图7.9 价值迭代法的求解过程



为了解决这两个不足，可以用相同的思路构造基于时序差分法的强化学习算法。如算法7.6所示，这个算法被称为Q学习（Q-Learning）算法[35]。

对比算法7.3和算法7.6，可以发现策略评估的时序差分法和Q学习算法主要区别在于算法7.6中的第5行和第7行。算法7.6的第5行通过寻找最大的动作-价值函数来确定当前的动作，这对应了策略优化的过程，可以认为其是先将当前策略更新为$\pi(s)\leftarrow\text{argmax}_a q_\pi(s,a)$，然后根据更新以后的策略来选择动作。

注意算法7.6中第7行，不难发现Q学习中直接记录和更新动作-价值函数q_π而不是价值函数V_π，这是因为策略优化要求已知动作-价值函数q_π，如果算法仍然记录价值函数V_π，在不知道状态转移概率的情况下将无法求出q_π。

根据（式7.4）中动作-价值函数与价值函数的关系，当动作和下一时刻状态都已经（通过采样）确定时，当前动作-价值函数的估计值为：

$$q_\pi(s,a)=R(s,a,s')+\gamma V_\pi(s')$$

根据策略优化的思想，在状态s'的动作应为$\pi(s')=\text{argmax}_{a'}q_\pi(s',a')$，因此：

$$V_\pi(s')=q_\pi(s',\pi(s'))=\max_{a'}q_\pi(s',a')$$

综合以上两式可得：

$$q_\pi(s,a)=R(s,a,s')+\gamma\max_{a'}q_\pi(s',a')$$

这就是算法7.6第7行更新公式中使用的动作-价值函数的估计值。

算法7.6　Q学习算法

函数：QLearning（ ）

输入：马尔可夫决策过程$MDP=(S,A,P,R,\gamma)$
输出：策略π

```
1  随机初始化 q_π
2  repeat
3  |  s ← 初始状态
4  |  repeat
5  |  |  a ← argmax_a' q_π(s,a')
6  |  |  执行动作 a，观察奖励 R 和下一个状态 s'
7  |  |  q_π(s,a) ← q_π(s,a) + α[R + γ max_a' q_π(s',a') − q_π(s,a)]
8  |  |  s ← s'
9  |  until s 是终止状态
10 until q_π 收敛
11 π(s) := argmax_a q(s,a)
```

结合图7.2所示的机器人寻路问题来说明Q学习的过程。图7.10（a）表示算法的初始状态，其中a/b表示对应状态的动作-价值函数的取值，斜线上方的a表示$q_\pi(s, 上)$，斜线右侧的b表示$q_\pi(s, 右)$。因此图7.10（a）中显示除终止状态外的所有的向上的动作-价值函数值为0.2，所有向右的动作-价值函数值为0。当然，令这些动作-价值函数的初始值全部为0也是可以的，图7.10（a）中所设置的初始值只是为了让这个例子更容易说明策略学习。图7.10（a）可视为初始的策略。

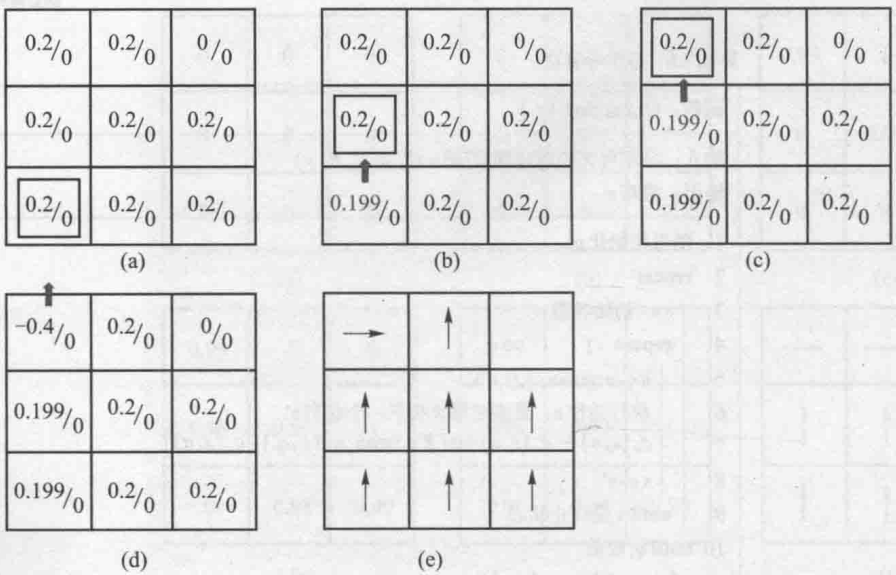
图7.10（a）中的黑色方框代表了智能体当前所处的状态位置，即算法7.6中变量s的值。从图7.10（a）为出发，在算法第5行，根据当前策略求出智能体应该采取的动作$a=\text{argmax}_a q_\pi(s_1, a)=$ 上，执行这个动作，得到奖励$R=0$和进入下一状态$s'=s_4$，因此可如下更新对应的动作-价值函数：

$$q_\pi(s_1, 上) \leftarrow q_\pi(s_1, 上)+\alpha\left[R+\gamma \max_{a'} q_\pi(s', a')-q_\pi(s, a)\right]$$
$$=0.2+0.5\times\left[0+0.99\times\max\{0, 0.2\}-0.2\right]=0.199$$

同时令当前状态为s_4，由此得到了图7.10（b）。同理算法再执行一次内层循环得到图7.10（c），接着得到图7.10（d）。这个过程中每一步都通过后续状态的动作-价值函数更新当前状态的动作-价值函数，图7.10（b）和图7.10（c）都没有对策略产生影响。

图7.10 Q学习的一个片段的执行过程

当智能体在图7.10（d）中到达了损坏状态s_d，一个负的奖励（即奖



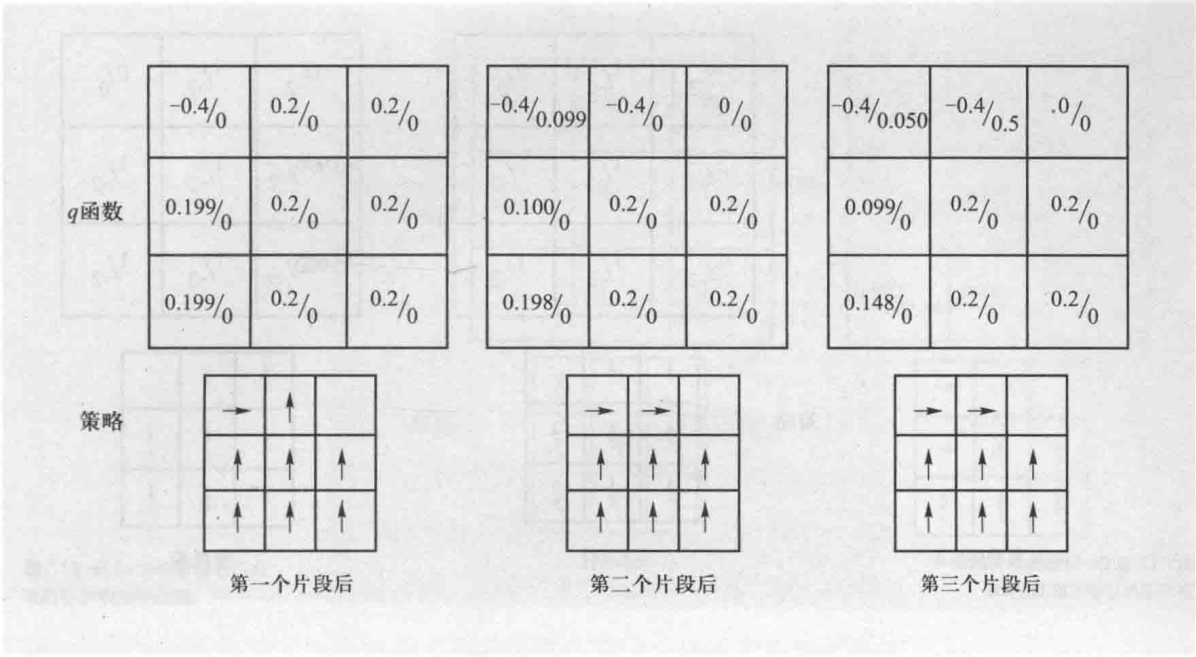


图7.11 Q学习执行三个片段的过程

励为-1）使$q_\pi(s_7, 上)$变为负值，此时策略$\pi(s_7)$从“向上移动一个方格”变成了“向右移动一个方格”，至此智能体完成了一个片段（一次外层循环）的更新。

图7.11展示了Q学习算法执行三个片段的过程，从第二个片段结束时起，算法已经学习得到了沿着轨迹$(s_1, s_4, s_7, s_8, s_9)$到达目标状态$s_9$的策略。实际上，虽然此时动作-价值函数尚未收敛，但此后策略已经不会再发生变化了。

7.2.5 探索与利用

本书3.4.1节中探讨了序贯决策问题中探索（exploration）与利用（exploitation）的关系，基于蒙特卡洛采样思想的时序差分法和Q学习也面临着这个问题。7.2.3节中介绍蒙特卡洛法和时序差分法时曾经提到过它们的一个特性，其会重点关注当前可知部分状态，但也因为未能准确估值而无法利用其余状态。这个特性之所以会产生是因为算法对当前策略进行了“充分”利用，按照已知策略行动会使得有些状态比其他状态更容易被访问到，同时也就无法访问到某些状态。显然，算法过于侧重部分状态，过于强调利用，这是有问题的，例如，算法可能始终不去尝试那些估计价值较低但实际上更好的策略。

在实际中这个问题是很容易出现的，例如，使用Q学习求解图7.2中的机器人寻路问题时，如果将动作-价值函数初始化为1/-2，即除了

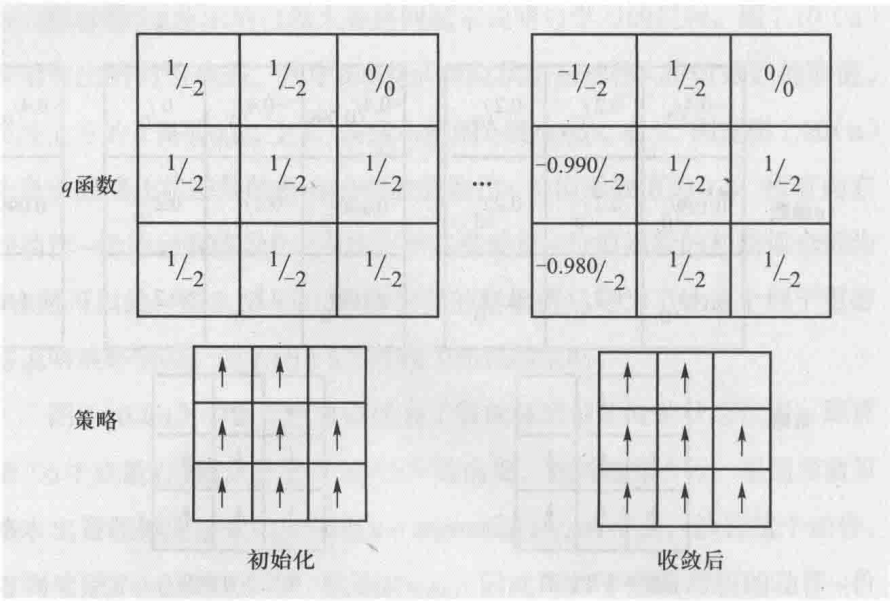


图7.12 动作-价值函数初始化为1/-2时Q学习算法的流程

终止状态，对于每个状态，智能体更倾向于向上移动一个方格而非向右一个方格。在这样初始策略下，Q学习执行过程如图7.12所示，可见即使在Q学习算法收敛以后，所得到的策略仍然不能使得机器人抵达目标状态s_9，而是沿着轨迹(s_1, s_4, s_7, s_d)导致机器人被损坏。

图7.12中问题出现的原因概括来说就是$q_\pi(\cdot, 右)$的初始值太小，导致机器人被损坏产生的-1奖励不足以推动智能体来改变策略。既然外部刺激不足以使机器人尝试新的策略，那么不妨从内部入手为智能体改变固有策略来添加一个探索的动力。

一种最简单的方法是直接在算法7.6中第5行决定行动时添加一个鼓励探索的机制，例如，以一个小概率随机选择一个动作（从而避开固有的策略），这个策略称为ϵ贪心（ϵ-greedy）探索策略，其定义如下：

$$\epsilon-\text{greedy}(s)=\begin{cases}\underset{a}{\text{argmax}}\, q(s,a), & \text{以}1-\epsilon\text{的概率}\\ \text{随机的}a\in \text{A}, & \text{以}\epsilon\text{的概率}\end{cases} \qquad (式7.9)$$

使用ϵ贪心探索策略的Q学习算法流程如算法7.7所示。值得注意的是，算法7.6中的第5行和第7行各有一个max的操作，均为策略优化的体现。而采用探索的算法7.7的Q-Learning函数中，只有第5行的max操作改成使用ϵ贪心策略，而第7行的max操作仍被保留，像这样采样时策略和更新时策略彼此不同的算法称为离策略（off-policy）算法。反之，采样时策略和更新时策略相同的算法称为在策略（on-policy）算法。Q学习是一个离策略算法，它对应的在策略版本称为Sarsa算法[39]。

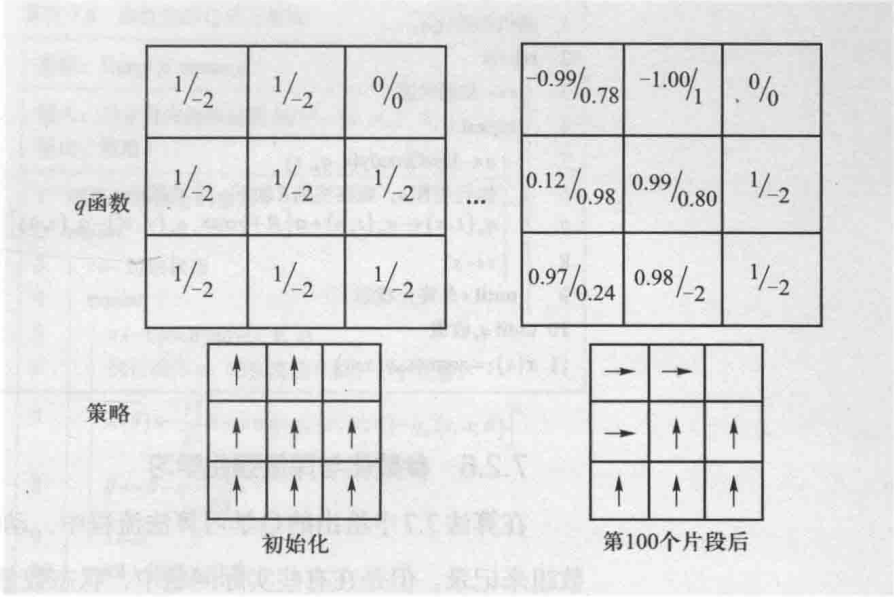


图7.13 使用ϵ贪心策略进行探索的Q学习的执行过程

如果使用算法7.7中的Q学习算法，那么图7.12中的问题便会迎刃而解。在将动作-价值函数初始化为与图7.12中相同取值后，使用算法7.7来求解，如图7.13所示为用计算机模拟的一次求解过程。

从图7.13中可知，算法在执行了100个片段后，得到了经过轨迹$(s_1, s_4, s_5, s_8, s_9)$到达目标状态的策略。由于$\epsilon$贪心策略具有概率性，因此图7.13中的结果并不是确定的，有可能经过100个片段仍不能得到一个合适的策略，且在图7.13中也能发现存在还没有被访问过的状态s_3。但随着迭代次数的增加，算法探索到最优策略的可能性也会显著增加，通过适当增大参数ϵ的值也能够促进算法乐于探索。

算法7.7 使用ϵ贪心算法进行探索的Q学习

函数：EpsGreedy（ ）

输入：状态s，动作-价值函数q_π，参数ϵ
输出：动作a

```
n~Uniform(0, 1)
if n<ε then
    a ← 从A中随机选择
else
    a ← argmax_{a'} q_π(s, a')
end
```

函数：QLearning（ ）

输入：马尔可夫决策过程$MDP=(S, A, P, R, \gamma)$
输出：策略π

```
1  随机初始化 q_π
2  repeat
3  |  s ← 初始状态
4  |  repeat
5  |  |  a ← EpsGreedy(s, q_π, ϵ)
6  |  |  执行动作 a，观察奖励 R 和下一个状态 s'
7  |  |  q_π(s,a) ← q_π(s,a) + α[R + γ max_a' q_π(s',a') − q_π(s,a)]
8  |  |  s ← s'
9  |  until s 是终止状态
10 until q_π 收敛
11 π(s) := argmax_a q(s,a)
```

7.2.6 参数化与深度强化学习

在算法7.7中给出的Q学习算法流程中，动作-价值函数需要用一个数组来记录。但是在有些实际问题中，状态数量会非常多甚至是无限的，这样会造成如下两个问题：首先是算法很难用一个数组来存储动作-价值函数的值；其次是有些状态的访问次数可能很少甚至根本没有被访问过，这些状态的价值估计是不可靠的。

为了解决这两个问题，一种解决方案是将动作-价值函数参数化（parametrize），即用一个回归模型来拟合q_π函数。如果回归模型是一个深度神经网络，那么这样的算法就被称为深度强化学习（deep reinforcement learning）算法。

参数化能够解决无限状态下动作-价值函数的存储问题，因为算法只需要记住一组参数，动作-价值函数的具体取值可以通过这一组参数算出。参数化也有助于缓解对很多状态的价值估计不准确的问题，在一个连续的状态空间上，如果动作-价值函数是连续的，那么至少对那些访问次数较多的状态所对应小邻域内的状态进行价值估计，其估计结果也是有一定精度来保障的。

算法7.8给出了参数化的Q学习算法[35]，如果使用深度神经网络来拟合动作-价值函数，则该算法被称为深度Q学习。和非参数化的算法7.7进行对比，不难发现，区别主要在算法7.8中第7行和第8行的动作-价值函数更新。类似7.2.3节对时序差分法的分析，$R+\max_{a'} q_\pi(s',a';\boldsymbol{\theta})$是对$q_\pi(s,a)$的估计值，因此算法7.8中第7行计算的是$q_\pi(s,a)$的估计值和当前值之间误差的平方，第8行则是将该值作为损失函数，利用梯度下降法来更新参数$\boldsymbol{\theta}$。关于梯度下降法和深度学习的相关内容，请参见本书的第6章。

算法7.8　参数化的Q学习算法

函数：DeepQLearning（ ）

输入：马尔可夫决策过程$MDP=(S, A, P, R, \gamma)$
输出：策略π

1　随机初始化q_π的参数$\boldsymbol{\theta}$
2　**repeat**
3　　$s\leftarrow$初始状态
4　　**repeat**
5　　$a\leftarrow\text{EpsGreedy}(s, \boldsymbol{\theta}, \epsilon)$
6　　执行动作a，观察奖励R和下一个状态s'
7　　$L(\boldsymbol{\theta})\leftarrow\frac{1}{2}\left[R+\gamma\max_{a'}q_\pi(s', a';\boldsymbol{\theta})-q_\pi(s, a;\boldsymbol{\theta})\right]^2$
8　　$\boldsymbol{\theta}\leftarrow\boldsymbol{\theta}-\eta\frac{\partial L(\boldsymbol{\theta})}{\partial\boldsymbol{\theta}}$
9　　$s\leftarrow s'$
10　　**until** s是终止状态
11　**until** q_π收敛
12　$\pi(s):=\text{argmax}_a q(s, a)$

然而，动作-价值函数的参数化也带来了如下一些新的问题。

第一个问题是由于相邻的样本来自同一条轨迹，因此样本之间相关性太强，使得集中优化相关性强的样本会导致神经网络处理其他样本无法取得较好效果。Q学习的一个特性是倾向于反复采样更新一部分状态，这部分状态来源于Q学习算法采样的轨迹，其并不是均匀分布在整个状态空间中。Q学习这个特性在使用数组记录动作-价值函数时只会导致对部分状态的价值估计越来越准确。但是在使用回归模型拟合动作-价值函数时，连续对一组相关性强的状态进行更新虽然会使模型对这些状态的价值估计越来越准确，但也会导致对那些没有被更新状态的价值估计产生更大偏差。

以简单的线性回归问题为例，若要用一个线性回归模型拟合若干二维数据点，则回归直线如图7.14中实线所示会取得较好效果。但是如果回归模型多次连续对A和B两点误差进行优化，最后可能会得到图7.14中虚线所示回归直线。基于同样的原理，动作-价值函数的参数化可能导致算法不稳定或产生较大偏差。

另一个问题是，在算法7.8中第7行计算损失函数时，参数$\boldsymbol{\theta}$被同时用来计算动作-价值函数的目标值（估计值）$R+\max_{a'}q_\pi(s', a';\boldsymbol{\theta})$和当前值$q_\pi(s, a;\boldsymbol{\theta})$，因此对$\boldsymbol{\theta}$的更新会同时影响这两个值。对于当前样本的损失函数计算公式而言，令损失函数值为0的动作-价值函数取值不是唯一

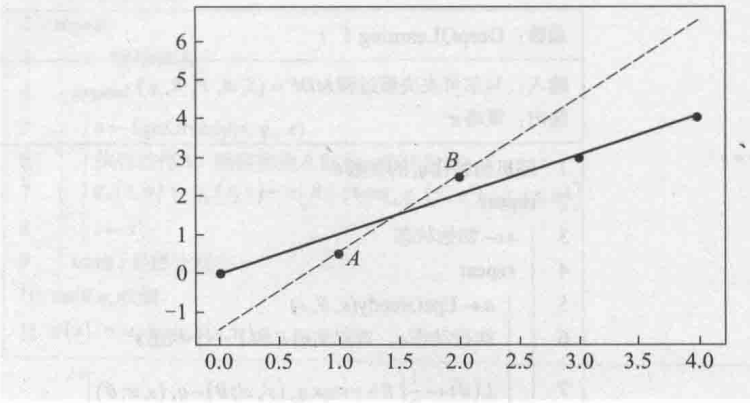


图7.14 回归模型集中优化相关性强样本所产生问题示例

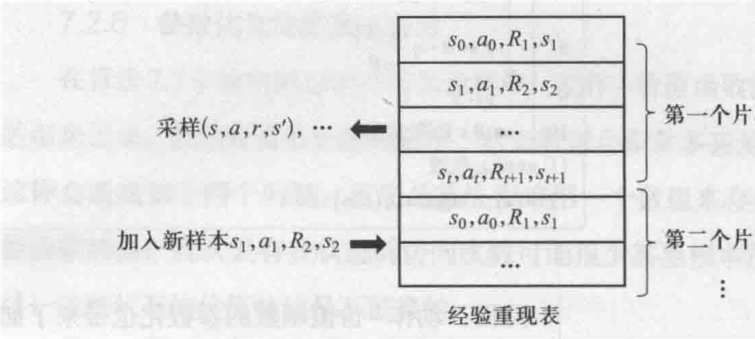


图7.15 经验重现

的，算法既可以调整当前值$q_\pi(s, a; \boldsymbol{\theta})$使其等于目标值，也可以调整目标值使其等于当前值。在单个样本优化目标不明确的情况下，由于采样具有概率性，这样会使算法的收敛变得不稳定。

在Mnih等人提出的深度Q网络（deep Q-network，DQN）[14]中，这两个问题得到了针对性的解决。

对于样本相关性太强的问题，DQN采用经验重现（experience replay）的方法来应对，如图7.15所示。经验重现将算法探索的片段以(s, a, R, s')的四元组形式存在一张表中，每当算法探索得到一个新的四元组，便将该四元组加入经验重现表；而在算法对参数$\boldsymbol{\theta}$进行更新时，则从经验重现表中随机选取一批样本用于计算损失函数。使用经验重现主要有两个好处，一方面是样本之间的相关性显著减弱，因为随机采样得到的样本状态会更均匀地分布在已探索的样本空间中。另一方面过去的经验可以被重复利用，因此提高了信息利用的效率。

对于使用同样的参数来计算动作-价值函数的当前值和目标值的问题，DQN使用目标网络的方法来应对。所谓目标网络，即用另一组参数$\boldsymbol{\theta}^-$来计算动作-价值函数的预测值。具体来说，对损失函数进行了如下修改：

$$L(\boldsymbol{\theta})=\frac{1}{2}\left[R+\gamma\max_{a'}q_{\pi}\left(s',a';\boldsymbol{\theta}^{-}\right)-q_{\pi}\left(s,a;\boldsymbol{\theta}\right)\right]^{2}$$

参数$\boldsymbol{\theta}^{-}$保持相对稳定，以较低的频率更新$\boldsymbol{\theta}^{-}\leftarrow\boldsymbol{\theta}$。因此在算法更新$\boldsymbol{\theta}$时，目标值可以看作是一个常数，求解目标变成了一个普通的回归问题，这样使得算法更加稳定。

DQN能够广泛地应用于许多问题中，Mnih等人将DQN应用于训练玩主机游戏的智能体。在这个问题中，智能体读取当前游戏画面，为了在游戏中获取高得分，根据策略模拟控制器的输入。此时状态即为游戏画面，动作为控制器的不同输入，奖励为游戏得分。智能体并不知道状态转移概率，但是能够通过环境（即游戏）来模拟执行动作的结果。

DQN使用参数化的Q学习来求解游戏的策略，其中拟合动作-价值函数的是一个深度卷积神经网络，如图7.16所示。每当输入一个状态（一个游戏画面），神经网络为每个动作输出对应的动作-价值函数。配合算法7.8以及经验重现和目标网络的改进，智能体在不少游戏上都取得了不错的分数。关于具体的神经网络结构和一些实现细节，可参考文献[14]，其中涉及的一些深度神经网络的知识可参见本书第6章的内容。

DQN是近年来深度强化学习（deep reinforcement learning，DRL）研究中最有代表性的工作之一。随着深度学习与传统强化学习的结合，人们逐渐认识到强化学习能够广泛应用于解决实际中常见的决策问题。由于强化学习和人类学习过程的相似性，有不少研究者甚至认为深度强化学习指明了一条迈向通用人工智能的道路。

在与深度学习结合之前，强化学习并未得到如当前这般广泛的关注，这是因为未参数化的或用浅层模型参数化的强化学习并没有足够的能力解决复杂的现实问题。深度学习让强化学习算法能够在更复杂的问题中拟合价值函数或策略函数，从而使其解决问题的能力得到了提升。但是

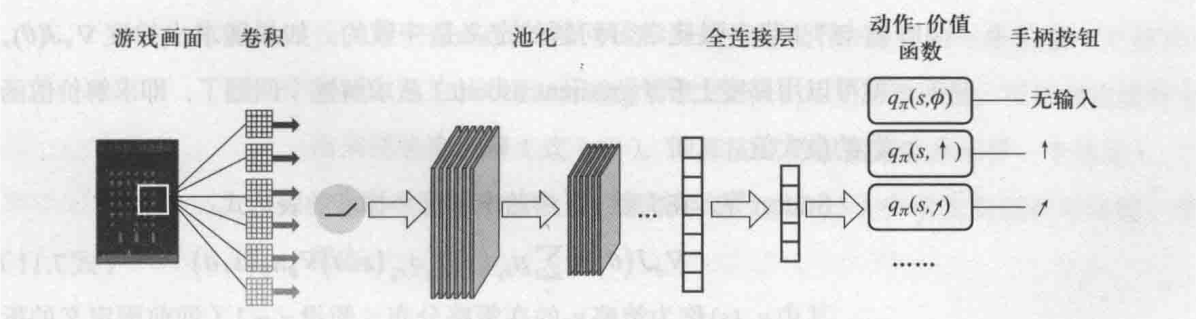


图7.16 用于游戏的DQN动作-价值函数模型

目前的强化学习算法还存在诸多的问题和局限，导致它们在大多数实际问题上仍然无能为力。尽管如此，随着研究的进展，越来越多的深度强化学习算法被应用于实际问题中，产生了不可估量的价值。

7.3 基于策略的强化学习

7.2节中讨论了若干强化学习的方法，其中有些方法用表格记录动作-价值函数，有些方法将动作-价值函数参数化后使用梯度下降求解。这些方法有一个共同的特点，就是以对价值函数或动作-价值函数的建模为核心。

实际中也可以换一个思路，通过直接参数化策略函数的方法求解强化学习问题，由于算法需要求参数化的策略函数的梯度，因此这些方法称为策略梯度法（policy gradient methods）。本节中将介绍几种基于策略梯度的强化学习算法以及背后原理。

7.3.1 策略梯度定理

策略函数的参数化可以表示为$\pi_\theta(s, a)$，其中$\boldsymbol{\theta}$为一组参数，函数取值表示在状态s下选择动作a的概率。和Q学习的ϵ贪心策略相比，这种参数化的一个显著好处是：选择一个动作的概率是随着参数的改变而光滑变化的，实际上这种光滑性对算法收敛有更好的保证。

为了优化策略函数，首先需要有一个对策略函数优劣进行衡量的标准。不妨假设强化学习问题的初始状态为s_0（7.1.2小节中解释过为什么可以这样假设），不难定义算法希望达到的最大化目标为

$$J(\boldsymbol{\theta}) := V_{\pi_\theta}(s_0) \qquad (式7.10)$$

这是一个关于参数$\boldsymbol{\theta}$的函数，其值为从初始态出发的价值函数，这与7.1节中强化学习问题的定义是一致的。如果能求出梯度$\nabla_\theta J(\boldsymbol{\theta})$，就可以用梯度上升（gradient ascent）法求解这个问题了，即求解价值函数的最大值。

Sutton等人在文献[29]中给出了这个梯度的表达式：

$$\nabla_\theta J(\boldsymbol{\theta}) \propto \sum_s \mu_{\pi_\theta}(s) \sum_a q_{\pi_\theta}(s, a) \nabla_\theta \pi_\theta(s, a) \qquad (式7.11)$$

其中$\mu_{\pi_\theta}(s)$称为策略π_θ的在策略分布，假设$\gamma = 1$（即前面定义的折

扣系数）。在持续问题中，$\mu_{\pi_\theta}(s)$为算法从s_0出发经过无限多步后位于状态s的概率；在分段问题中，$\mu_{\pi_\theta}(s)$为归一化后的算法从s_0出发访问s的次数的期望。当$\gamma \in (0, 1)$时，则需要给每个状态的$\mu_{\pi_\theta}(s)$值加上一个权重。在保证算法通用性的前提下，为了简化说明，下文在进行公式推导时始终假设$\gamma = 1$，但是在算法流程中会体现折扣系数γ的影响。

（式7.11）称为**策略梯度定理**（policy gradient theorem），它提供了策略梯度法的求解思路：如果能够计算或估计出（式7.11）中的梯度，智能体就能直接对策略函数进行优化。

7.3.2 基于蒙特卡洛采样的策略梯度法

根据（式7.11）计算策略梯度并非一个简单的问题，其中对μ_{π_θ}和q_{π_θ}的准确估计本来就是难题，更不用说进一步计算$\nabla_\theta J(\boldsymbol{\theta})$了。好在蒙特卡洛法能被用来估计这类问题的取值，为此首先要对（式7.11）进行如下的适当变形：

$$\begin{aligned}\nabla_\theta J(\boldsymbol{\theta}) &\propto \sum_s \mu_{\pi_\theta}(s) \sum_a q_{\pi_\theta}(s,a) \nabla_\theta \pi_\theta(s,a) \\ &= \sum_s \mu_{\pi_\theta}(s) \sum_a \pi_\theta(s,a) \left[q_{\pi_\theta}(s,a) \frac{\nabla_\theta \pi_\theta(s,a)}{\pi_\theta(s,a)} \right] \\ &= \mathbb{E}_{s,a\sim\pi} \left[q_{\pi_\theta}(s,a) \frac{\nabla_\theta \pi_\theta(s,a)}{\pi_\theta(s,a)} \right] \\ &= \mathbb{E}_{s,a\sim\pi} \left[q_{\pi_\theta}(s,a) \nabla_\theta \ln \pi_\theta(s,a) \right] \end{aligned} \quad \text{（式7.12）}$$

（式7.12）是梯度策略定理的一个常见变形，但由于（式7.12）中存在q_{π_θ}，算法无法直接使用蒙特卡洛法来求取其中的期望。为此，进一步将（式7.12）变形为：

$$\begin{aligned}\nabla_\theta J(\boldsymbol{\theta}) &\propto \mathbb{E}_{s,a\sim\pi} \left[\mathbb{E}_{\mathcal{T}(s,a)\sim\pi} \left[G_t | s, a \right] \nabla_\theta \ln \pi_\theta(s,a) \right] \\ &= \mathbb{E}_{s,a,\mathcal{T}(s,a)\sim\pi} \left[G_t \nabla_\theta \ln \pi_\theta(s,a) \right] \end{aligned} \quad \text{（式7.13）}$$

其中$\mathcal{T}(s,a)$表示从状态s开始执行动作a得到的一条轨迹（不包括s和a），G_t为从状态s开始沿着轨迹$\mathcal{T}(s,a)$运动所得回报。可以使用蒙特卡洛采样法来求解（式7.13），即算法只需根据策略来采样一个状态s、一个动作a和将来的轨迹，就能构造（式7.13）中求取期望所对应的一个样本。

算法7.9 REINFORCE算法

函数：REINFORCE（ ）

输入：马尔可夫决策过程$MDP=(S, A, P, R, \gamma)$
输出：策略π

1 随机初始化$\boldsymbol{\theta}$
2 **repeat**
3 根据策略π_θ采样一个片段$s_0, a_0, R_1, s_1, \cdots, s_{T-1}, a_{T-1}, R_T$
4 **for** $t \leftarrow 0$ **to** $T-1$ **do**
5 $G \leftarrow \sum_{k=1}^{T-t} \gamma^{k-1} R_{t+k}$
6 $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \gamma^t G \nabla_{\boldsymbol{\theta}} \ln \pi_{\boldsymbol{\theta}}(s_t, a_t)$
7 **end**
8 **until** $\boldsymbol{\theta}$收敛

使用（式7.13）估计策略梯度的算法称为REINFORCE[38]，其流程如算法7.9所示，这个算法利用采样得到的片段来估计（式7.13）中的梯度，并使用梯度上升法来优化策略。注意算法7.9中第6行的更新公式并不是完全根据（式7.13）得到的，相比之下多了一个γ^t，这个差别正是由前文所说的公式推导时假设$\gamma=1$导致的。在实现中，这个γ^t常常会被省略，实际上，7.3.2和7.3.3小节中所有的算法策略参数更新中的这个γ^t都常常在实现时被省略。另一个需要注意的地方是算法7.9的第6行中参数与梯度相加而非减去梯度，这是由于算法的目标是最大化$J(\boldsymbol{\theta})$而不是如常见的问题定义一样最小化该函数。

算法7.9通过执行完一个完整片段所得实际反馈值来估计策略梯度，这一点和策略评估的蒙特卡洛法很相似，两个算法都要求问题是分段的，且只有在一个片段结束后算法才能进行更新操作。

在REINFORCE算法的推导过程中，可以得到（式7.12）中的策略梯度定理。观察（式7.12），很容易联想到参数估计中的极大似然估计法，其中$\pi_\theta(s, a)$可以类比似然概率。（式7.12）在每个样本的对数似然概率梯度前添加了权重$q_{\pi_\theta}(s, a)$，当$q_{\pi_\theta}(s, a)$越大时，算法越倾向于增加动作a被选择的概率，这与最大化价值的目标是一致的。

从这个角度来理解REINFORCE算法，不难想到一个问题：假设某个状态s_0下有两种可选动作a_0和a_1，满足$q_{\pi_\theta}(s_0, a_0)>q_{\pi_\theta}(s_0, a_1)>0$，那么理想状态下学习得到的策略应满足$\pi_\theta(s_0, a_0)>\pi_\theta(s_0, a_1)$。但如果样本片段中只出现了$a_1$而没有出现$a_0$，由于$q_{\pi_\theta}(s_0, a_1)>0$，所以改变$\boldsymbol{\theta}$的值使得$\pi_\theta(s_0, a_1)$增大（相应地$\pi_\theta(s_0, a_0)$减小）有助于提高样本的加权对数似然

概率，但这显然与期望的结果不相符。在实际中，这个问题确实会影响REINFORCE方法的稳定性和收敛速度。

为了解决这个问题，一种直观的想法是让那些“不好”动作所对应的动作-价值函数取值变为负数，这样算法就会倾向于减小这些动作的概率。为了实现这个想法，需要解决两个问题，第一是如何调整动作-价值函数的值才能保证优化目标不变，第二是怎样定义“不好”的动作。

首先观察（式7.12），可以发现如下性质：

$$\begin{aligned}\nabla_\theta J(\boldsymbol{\theta}) &\propto \mathbb{E}_{s,a\sim\pi}\left[q_{\pi_\theta}(s,a)\nabla_\theta \ln \pi_\theta(s,a)\right] \\ &= \mathbb{E}_{s,a\sim\pi}\left[\left(q_{\pi_\theta}(s,a)-b(s)+b(s)\right)\nabla_\theta \ln \pi_\theta(s,a)\right] \\ &= \mathbb{E}_{s,a\sim\pi}\left[\left(q_{\pi_\theta}(s,a)-b(s)\right)\nabla_\theta \ln \pi_\theta(s,a)\right]+\mathbb{E}_{s,a\sim\pi}\left[b(s)\nabla_\theta \ln \pi_\theta(s,a)\right]\end{aligned}$$

其中

$$\begin{aligned}\mathbb{E}_{s,a\sim\pi}\left[b(s)\nabla_\theta \ln \pi_\theta(s,a)\right] &= \mathbb{E}_{s\sim\pi}\left[b(s)\sum_a \pi_\theta(s,a)\frac{\nabla_\theta \pi_\theta(s,a)}{\pi_\theta(s,a)}\right] \\ &= \mathbb{E}_{s\sim\pi}\left[b(s)\nabla_\theta \sum_a \pi_\theta(s,a)\right] \\ &= \mathbb{E}_{s\sim\pi}\left[b(s)\nabla_\theta 1\right] \\ &= 0\end{aligned}$$

因此

$$\nabla_\theta J(\boldsymbol{\theta}) \propto \mathbb{E}_{s,a\sim\pi}\left[\left(q_{\pi_\theta}(s,a)-b(s)\right)\nabla_\theta \ln \pi_\theta(s,a)\right] \quad （式7.14）$$

其中$b(s)$是一个关于状态的函数。（式7.14）的含义是，在动作-价值函数上任意减去一个关于状态的函数不会影响策略梯度。这回答了第一个问题，即通过设计一个任意函数$b(s)$就可以达到调整动作-价值函数取值而不影响优化目标的目的。

至于解决什么是“不好”的动作这一问题，不妨如下定义“不好”的动作：动作-价值小于期望的动作是“不好”的。从（式7.3）可知动作-价值函数关于动作分布的期望正是价值函数。因此在对算法优化过程中，对于任意的动作a，当$q_{\pi_\theta}(s,a)<V_{\pi_\theta}(s)$时，调整$q_{\pi_\theta}(s,a)$的值使其小于0。显然，价值函数$V_{\pi_\theta}(s)$就是一个合适的$b(s)$函数[38]。

至此，要实现以上想法还差最后一步，即如何估计价值函数V_{π_θ}，这是一个熟悉的策略评估问题。为了配合REINFORCE方法，可采用价值函数参数化的蒙特卡洛法进行策略评估，其流程如算法7.10所示。

算法7.10　使用基准函数的REINFORCE方法

函数：BaselineREINFORCE（ ）

输入：马尔可夫决策过程$MDP=(S, A, P, R, \gamma)$
输出：策略π

1　随机初始化$\boldsymbol{\theta}, \boldsymbol{w}$
2　**repeat**
3　　根据策略π_θ采样一个片段$s_0, a_0, R_1, s_1, \cdots, s_{T-1}, a_{T-1}, R_T$
4　　**for** $t \leftarrow 0$ **to** $T-1$ **do**
5　　$G \leftarrow \sum_{k=1}^{T-t} \gamma^{k-1} R_{t+k}$
6　　$\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta_w \gamma^t \frac{1}{2} \nabla_w \left[G - V_w(s_t)\right]^2$
7　　$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta_\theta \gamma^t \left[G - V_w(s_t)\right] \nabla_\theta \ln \pi_\theta(s_t, a_t)$
8　　**end**
9　**until** $\boldsymbol{\theta}$收敛

在算法7.10中，价值函数用一个参数为$\boldsymbol{w}$的函数V_w来估计。在迭代过程中，算法将对价值函数的估计值和实际反馈值之间误差的平方作为损失函数，用梯度下降法来更新参数$\boldsymbol{w}$。在第7行更新参数$\boldsymbol{\theta}$时，用$G-V_w(s_t)$而非G作为样本的权重。这可从理论角度来解释，以价值函数作为基准函数可以将不同状态的动作-价值函数平移到0周围，从而减小策略梯度的样本方差，有利于促进算法收敛。

7.3.3　Actor-Critic算法

既然REINFORCE算法采用蒙特卡洛采样的策略梯度法，那么是否能够从时序差分的角度设计一个策略梯度求解方法呢？时序差分与蒙特卡洛采样的核心差别在于使用下一时刻状态的价值函数来估计当前状态的价值函数，而不是使用整个片段的反馈值，体现在算法中即使用$R+\gamma V_{\pi_\theta}(s')$来代替算法7.9中的变量$G$。算法7.11给出了基于时序差分的策略梯度法的流程，其名称为Actor-Critic算法[2, 40]。

算法7.11　Actor-Critic算法

函数：ActorCritic（ ）

输入：马尔可夫决策过程$MDP=(S, A, P, R, \gamma)$
输出：策略π

```
1  随机初始化θ, w
2  repeat
3  |  s←初始状态
4  |  t←0
5  |  repeat
6  |  |  a~π_θ(s, ·)
7  |  |  执行动作a，观察奖励R和下一时刻状态s'
8  |  |  w←w+η_w γ^t [R+γV_w(s')−V_w(s)]∇_w V_w(s)
9  |  |  θ←θ+η_θ γ^t [R+γV_w(s')]∇_θ ln π_θ(s, a)
10 |  |  t←t+1
11 |  until s是终止状态
12 until θ收敛
```

在算法7.11中，第9行用$R+\gamma V_w(s')$作为$q_{\pi_\theta}(s, a)$的估计值，而非当前片段的实际回报。第8行中仍然用价值函数估计值和当前值之间误差的平方作为损失函数，但值得注意的一点是，算法假设$\nabla_w V_w(s')=0$，即假设目标值中的$V_w(s')$是一个常数，这与Q学习中目标网络的做法不谋而合。

Actor-Critic算法最初的研究出现时，人们对策略梯度法还没有形成体系化认识。研究者们认为参数化的策略函数就像一个执行者（actor），负责动作的选择，而参数化的价值函数就像一个评论家（critic），评价执行者选出的动作的好坏，并成为执行者更新策略的依据，Actor-Critic算法由此得名。算法7.11将策略函数作为执行者并将价值函数作为评论家，实际上评论家也可以是动作-价值函数，关于此时算法应该如何设计，本书留给读者思考。

算法7.10引入了一种基准函数的思想，这在Actor-Critic方法中也是一种有效的优化手段。算法7.12展示了以价值函数为基准函数的Actor-Critic方法，这个算法和算法7.11相比，其区别只在第9行中使用了基准函数$V_w(s)$。实际上，动作-价值函数与价值函数之差$A_{\pi_\theta}(s, a)=q_{\pi_\theta}(s, a)-V_{\pi_\theta}(s)$有一个专门的名称，叫“优势（advantage）函数”。算法7.12中$q_{\pi_\theta}(s, a)$的估计值与基准函数$V_w(s)$之差可以看作是对优势函数的无偏估计，因此算法7.12被称为优势Actor-Critic（advantage actor-critic，A2C）算法[①]。A2C算法及其异步版本A3C（asynchronous advantage actor-critic）[15]算法是实际应用中最常见的强化学习算法之一。

① 许多文献中的Actor-Critic算法实际上指的是本书中的A2C算法。

算法7.12　优势Actor-Critic算法

函数：A2C（ ）

输入：马尔可夫决策过程$MDP=(S, A, P, R, \gamma)$
输出：策略π

随机初始化$\boldsymbol{\theta}, w$
repeat
　$s\leftarrow$初始状态
　$t\leftarrow 0$
　repeat
　　$a\sim\pi_{\boldsymbol{\theta}}(s, \cdot)$
　　执行动作a，观察奖励R和下一时刻状态s'
　　$\boldsymbol{w}\leftarrow \boldsymbol{w}+\eta_w\gamma^t\left[R+\gamma V_w(s')-V_w(s)\right]\nabla_w V_w(s)$
　　$\boldsymbol{\theta}\leftarrow \boldsymbol{\theta}+\eta_\theta\gamma^t\left[R+\gamma V_w(s')-V_w(s)\right]\nabla_{\boldsymbol{\theta}}\ln\pi_{\boldsymbol{\theta}}(s, a)$
　　$t\leftarrow t+1$
　until s是终止状态
until $\boldsymbol{\theta}$收敛

7.4　深度强化学习的应用

7.4.1　深度强化学习在围棋游戏中的应用

谈到近年发生的人工智能领域的重要事件，AlphaGo的出现及其与职业围棋选手的对弈无疑是大家最为熟悉的例子之一，这不仅让人工智能成为公众视野中的热点话题，也让研究人员认识到强化学习和深度学习的应用潜力。本小节将以AlphaGo为例，介绍深度强化学习的应用。

围棋是一种两人博弈游戏，一方执黑子，另一方执白子，对弈双方在19×19方格组成的棋盘上轮流下子，最后以围起的空地多者为胜利方。由于围棋的巨大搜索空间，它长期以来被认为是人工智能最难以驾驭的经典游戏。

经典的围棋对弈算法大多基于蒙特卡洛树搜索，大致能达到业余高段的水平。AlphaGo[22]将最近得到快速发展的深度学习、强化学习与蒙特卡洛树搜索进行有机结合，最终成功地击败了高段位职业选手。

如果用马尔可夫决策过程的五个要素来描述围棋问题，状态可以认为是围棋的盘面状况（19×19方格组成的棋盘）、动作为在棋盘上落子、奖励为对弈的胜负（胜为+1，负为-1），围棋中状态转移是确定的，即从一个状态出发，每个动作只会导致一个固定的后续状态。围棋游戏一个片段（episode）产生的轨迹（trajectory）如图7.17所示。

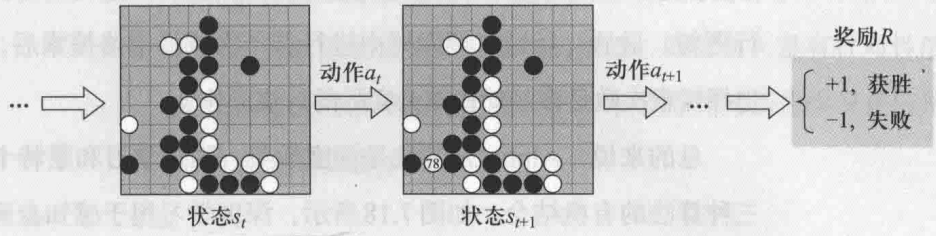


图7.17 围棋游戏一个片段的轨迹

深度学习在AlphaGo中被用于拟合策略函数和价值函数。回顾策略函数和价值函数的定义，可知此时策略函数的输入为一个盘面状态，输出为在每个位置落子的概率；价值函数的输入也为一个盘面状态，而输出则为一个标量，表明在当前局势下胜利的概率。

AlphaGo主要基于蒙特卡洛树搜索框架（详见本书第3章），其中扩展（expansion）和模拟（simulation/rollout）步骤依赖监督学习得到的策略网络。算法从人类棋手16万局对弈的棋谱中采样得到一个数据集，并通过监督学习的方式拟合一个深度策略网络和一个浅层策略网络。其中深度策略网络用于在扩展时赋予每个动作先验概率，而浅层策略网络用于快速地模拟一局游戏。

蒙特卡洛树搜索的选择（selection）和反向传播（back propagation）过程则与强化学习算法相结合，以下详细介绍这部分算法。

AlphaGo在监督学习得到的深度策略网络的基础上，使用策略梯度法（即7.3节介绍的使用基准函数的REINFORCE算法）微调这个深度策略网络。为了减少强化学习算法对敌方策略过拟合，AlphaGo在产生片段轨迹时从旧版策略（即若干次迭代之前的策略）中随机选择一个作为对手。

根据强化学习得到的深度策略网络，AlphaGo通过策略评估的方法拟合了当前策略下的深度价值网络。这里使用的方法类似策略评估的蒙特卡洛法，通过策略网络自我对弈产生大量的轨迹数据。为了避免过拟合，AlphaGo采取了类似经验重现的思路，从这些轨迹数据中采样了相关性较小的部分用于训练价值网络。

通过强化学习得到的深度价值网络应用于蒙特卡洛树搜索的选择步骤。具体来说，在蒙特卡洛树搜索中需要选择一个动作时，AlphaGo根据当前动作的先验概率（通过监督学习的策略网络得到）、该动作被访问的次数、该动作的累积价值综合判断应选择哪个动作。在反向传播步骤

中，搜索树中每个动作的访问次数和累积价值则根据当前的价值函数进行更新。最后，经过在搜索树中进行若干次蒙特卡洛搜索后，AlphaGo选择探索次数最多的动作作为实际的行棋动作。

总的来说，AlphaGo算法是深度学习、强化学习和蒙特卡洛树搜索三种算法的有机结合。如图7.18所示，深度学习用于感知盘面信息，并拟合价值函数和策略函数；强化学习用于微调和优化策略函数以及价值函数；蒙特卡洛树搜索则根据强化学习的结果进行搜索并最终给出行棋方案。

AlphaGo Zero[27]是对AlphaGo算法的一个改进，它保留了AlphaGo的基本框架，但AlphaGo Zero不依赖人类的对弈数据进行预训练，而是完全通过随机的初始值和自我对弈来实现策略的提升。虽然根据人类的对弈数据进行预训练是一种能令强化学习快速收敛的有效手段，但这也可能使训练结果局限于人类并不完全正确的经验。实验证明这种从零开始自我博弈的训练方法学到了不同于人类已知策略但效果更好的策略。另一个重要的改变是，AlphaGo Zero不再依赖浅层神经网络执行蒙特卡洛树搜索的模拟过程，而是直接根据价值函数来得到模拟结果。此外，AlphaGo Zero还在特征提取和深度神经网络的设计上做出了一些改进，具体内容可参见文献[27]。

值得注意的是，无论是AlphaGo还是AlphaGo Zero，它们的成功都依赖大量的算法设计和训练技巧，例如，某组参数应该如何初始化，训

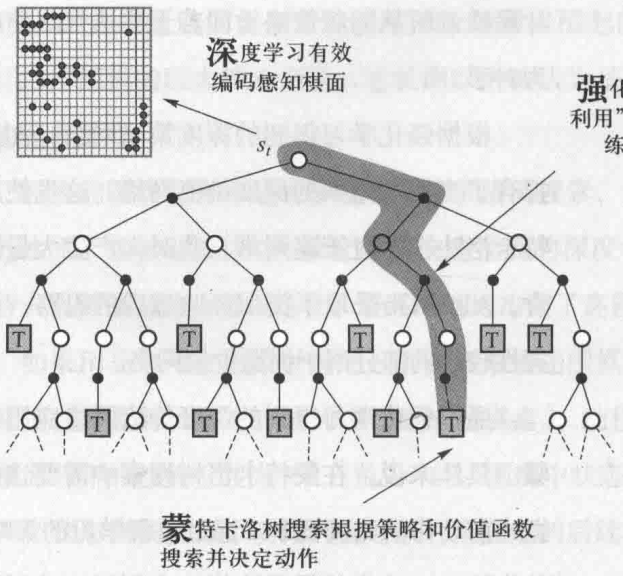


图7.18 AlphaGo算法的三个重要组成部分

练样本应该如何采集或某个超参数应该如何设置等（详见相关文献）。这反映了目前深度强化学习面临的困境，即需要大量有针对性的具体措施来保证算法的效率和稳定性。下一小节将针对强化学习在应用过程中遇到的这些困难进行探讨。

7.4.2 深度强化学习在实际应用中的问题

虽然有不少成功将深度强化学习算法应用于实际问题的例子，但若想通过简单地套用经典的强化学习算法来解决实际问题，多半是不能成功的。本小节将从以下几个方面介绍深度强化学习应用到实际问题中所遇到的困难。

奖励的设置。在很多实际问题中，智能体往往要经历一个漫长的决策过程，才能达成目标而获得奖励。虽然从理论上来说强化学习框架能够应用于奖励滞后的问题，但要根据奖励带来的少量信息决定大量的决策，显然是十分困难的，这常常导致深度强化学习的训练过程难以收敛。

这个问题实际上已经得到了广泛的研究，其中奖励塑造（reward shaping）[17]是一种经典的解决方案。奖励塑造通过调整每一步（包括中间步骤）的奖励值，在保持策略不变的情况下对训练过程加以指导，从而加速了算法的收敛。另一种常见的方法是设置短期目标，当这些短期目标达成时智能体将会得到一个中间奖励（intermediate reward）。这种想法进一步演变为层次强化学习（hierarchical reinforcement learning）[13]算法，即先制定一个宏观目标，再针对这个目标制定具体的实施计划，在宏观和微观两个层次同时进行强化学习。

制定短期目标虽然能缓解奖励稀疏的问题，但无法保证这些预先设定的短期目标不会使目标偏离真正的目标，从而导致智能体无法学习得到最优策略。更有甚者，有些问题的（短期）目标本身定义就不明确。例如，要训练一个游泳智能体，应该怎样根据常识来制定短期目标？又如，要用强化学习来优化城市的道路规划，那么“好”的道路规划是否有明确的量化定义？因此，在很多强化学习的应用场景下，奖励的设置成了最关键，却也是最困扰的问题。

样本的采集。根据文献[11]中提供的实验数据，在训练雅达利（Atari）游戏智能体时（7.2.5小节的DQN便是一个例子），常见的深度强化学习算法通常需要上亿帧的游戏画面作为训练数据才能达到人类的

平均水平，这意味着强化学习的训练常常要消耗大量的计算资源。如果计算资源足够，那么通过异步（asynchrounous）算法并行训练深度强化学习模型[15]尚能令人满意。

但更严重的问题是，实际情况下有时并不能提供一个良好的交互环境：例如，使用强化学习算法训练一个智能对话系统，如果要在真实的环境中进行训练，则需动用大量的人力来与智能体模拟对话；又如，要将强化学习应用于医疗、交通、安全等领域，显然不可能允许智能体在真实的环境中试错。因此，针对这类问题，如何采集强化学习训练用的样本便是首要的难题。

一个直观的解决方案是创建一个虚拟的环境，模拟环境的运行，然后在这个虚拟环境中训练强化学习算法。但是这个方法要求对实际环境的深刻理解，否则虚拟的环境将没有参考价值，这通常不是一件容易的事情。

另外一种思路是，如果采集样本的代价很大，那么算法必须更加善于利用历史数据。以经验重现（experience replay）为代表的一些离策略（off-policy）方法[16, 33]能够在与环境交互的同时提高历史数据（经验）的利用效率，而一些基于模仿学习（imitation learning）的算法[34]采用先模仿历史数据再离线优化的方法实现了不依赖环境交互的强化学习。

局部最优解与探索。基于蒙特卡洛法的强化学习算法（包括绝大多数目前常用的深度强化学习算法）容易收敛于一个局部最优解，而这个局部最优解带来的可能并不是人们能接受的策略，例如，在图7.12中，强化学习收敛得到的策略最终会导致机器人损坏。

正如在7.2.4小节中所讨论的，探索是解决这个问题的一个常见手段，其中ϵ-贪心算法是一种简单实用的探索方法。然而ϵ-贪心针对一些陷入局部最优解的情况也无能为力，如图7.19中，假设智能体位于A点，想要学习一个到达B点的策略，一种直观的思路是采用当前位置到目标的直线距离的倒数来初始化价值函数（图中虚线表示到B点的等距线）。如果此时只利用而不探索，则智能体会陷入移动到C点的局部最优解。如果此时采用ϵ-贪心的探索策略，为了探索找到箭头所示的最优解，智能体必须在D点之前的每一次行动都违反当前价值函数的指导，而这种概率极小。

为了解决图7.19的问题，设置短期目标是一种可行的思路，另外也

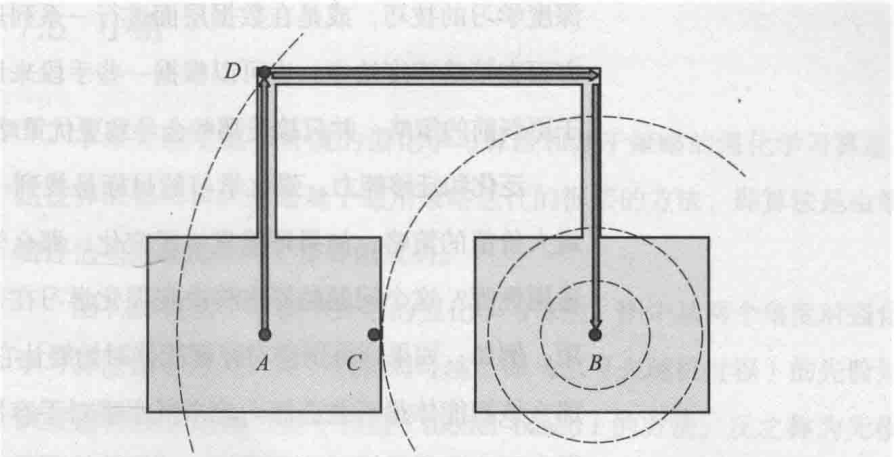


图7.19 一个难以探索的例子

可以采取一些有针对性的探索策略，例如，优先探索访问次数较少的或长时间未访问的状态[9, 31]。总的来说，探索方法对（深度）强化学习算法的收敛速度和学习结果的好坏有着不容忽视的影响，采用不同的探索方法，甚至在ε-贪心算法中选择不同的ϵ值，得到的结果可能有着巨大的差别。

训练时的不稳定性与方差。强化学习算法的不稳定性在讨论探索时可见一斑，因为不同的探索强度（不同的ϵ值）可能会导致差异很大的结果。更有甚者，对于一个完全相同的环境和算法，不同的随机种子也可能导致相差很大的结果[10]，这意味着，很多情况下强化学习算法并不是每一次都能成功的。这个问题在强化学习的实际应用中是致命的，因为大多情况下人们不会接受一个可能会失败的不稳定算法。

强化学习不稳定的原因很大程度上可以归结为蒙特卡洛法中样本方差过大。7.3节中介绍的利用优势函数替代价值函数的做法正是出于减小方差的目的，这个思路也可以应用到其他强化学习算法中（如Q学习[32]）。基于同样的理由，方差较小的时序差分策略评估法比蒙特卡洛策略评估法更受青睐。时序差分法的问题是：用回归模型（如深度神经网络）来估计下一个状态的价值函数并不是无偏的，即这样可能会引入偏差（bias），人们在这个问题上通常会采取平滑或者均值的手段来取得折中[23, 28]。

一些训练技巧也能够有效地提高训练过程的稳定性，例如，在7.4.1小节AlphaGo的例子中介绍的预训练便是一种常见且实用的技巧，此外Bengio等提出的课程学习（curriculum learning）[6]方法也能有效地辅助算法收敛。另一种提高算法稳定性的思路是更加谨慎地更新参数，一方面这可以通过一些独立于强化学习算法的技术来实现，如梯度截断等

深度学习的技巧，或是在数据层面进行一系列过滤和平滑的操作；另一方面在策略梯度法中，也可以根据一些手段来评估更新后的策略是否优于更新前的策略，并只接受那些会导致更优策略的更新[24, 25]。

泛化和迁移能力。强化学习的目标是找到一个在固定环境下能得到最大价值的策略，如果环境发生了变化，那么学习得到的策略还有多少适用性呢？这个问题的答案将决定强化学习在一系列实际场景中是否实用。例如，如果训练围棋对弈智能体时始终让它与水平很低的对手对弈，那么该智能体很可能会倾向攻击低水平对手容易犯的一些低级错误，但遇上不会犯低级错误的高手时，该智能体可能会一筹莫展；又如在训练自动驾驶算法时，智能体始终以城郊为环境进行样本采集和训练，那么有理由怀疑训练得到的策略是否能够在城区做出合适的反应。实际上，研究[37, 42]指出，深度强化学习常常以各种形式对环境过拟合（overfit），甚至有可能存在两个在训练环境上表现相似的智能体，在测试环境上却有着完全不同的表现。

最常见的提高泛化能力的方案是使用多个环境来训练，例如，可用一组（回报较低的）环境来计算当前的奖励[21]，或者可以通过对抗学习的方式选择使当前智能体不稳定的环境来训练[18]，还可以对环境进行编码并将其作为状态的一部分[41]。这些方法虽然能够提高算法的泛化能力，却需要与多个环境进行交互，增加了训练难度。

一种可能更加实用的做法是假设模型具有可迁移性，即在训练环境上训练好强化学习智能体后，在测试环境（即实际场景）中对该智能体的策略进行微调（fine-tuning）[22]，理想情况下策略的微调应该能够高效地令原策略适应新的环境。

总的来说，就目前的强化学习研究进展而言，如果想要将深度强化学习应用于实际问题，尤其是非常复杂的实际问题，那么必须在每个环节上都精心设计和调整算法，才有可能发挥强化学习算法的强大能力。本小节讨论了几个强化学习算法在应用中常见的问题，这些问题的存在使强化学习算法难以得到大规模的实际应用。乐观来看，随着对强化学习研究的不断深入，这些问题终将得到解决。

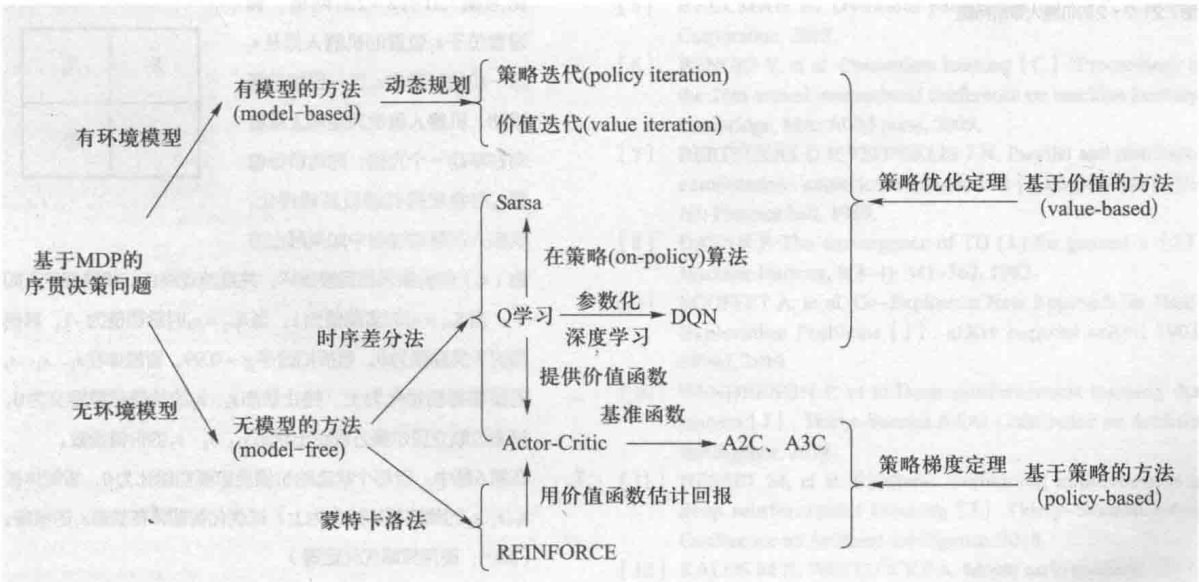
7.5 小结

本章介绍了基于价值的强化学习算法和基于策略的强化学习算法，这些算法都可以认为是基于通用策略迭代的框架的方法，即算法是由策略评估和策略优化两个步骤组成的。

图7.20概括了本章中介绍的强化学习算法。图中从两个角度对强化学习算法做了分类，其中依靠对环境（即马尔可夫随机过程）的先验知识或建模的算法称为基于模型（model-based）的方法，反之称为无模型的方法（model-free）；只对价值函数建模并利用策略优化定理求解的方法称为基于价值（value-based）的方法，对策略函数建模并利用策略梯度定理求解的方法称为基于策略（policy-based）的方法。

“欲粟者务时，欲治者因势”。强化学习作为一种通用的策略学习框架，向人们展示了其强大的能力和应用前景。虽然强化学习尚不成熟，导致它在很多问题上并不容易应用，但是对强化学习的研究，必将推动人工智能的发展并带来更加广泛的应用。

图7.20 本章所介绍方法之间相互关系示意图



习题7

1. 下面对强化学习、监督学习和深度卷积神经网络学习的描述正确的是（　　）。

A. 评估学习方式、有标注信息学习方式、端到端学习方式

B. 有标注信息学习方式、端到端学习方式、端到端学习方式

C. 评估学习方式、端到端学习方式、端到端学习方式

D. 无标注学习、有标注信息学习方式、端到端学习方式

2. 在强化学习中，通过哪两个步骤的迭代，来学习得到最佳策略？（　　）

A. 价值函数计算与动作-价值函数计算

B. 动态规划与Q-Learning

C. 贪心策略优化与Q-learning

D. 策略优化与策略评估

3. 在强化学习中，哪个机制的引入使得强化学习具备了在利用与探索中寻求平衡的能力？（　　）

A. ϵ贪心策略　　B. 蒙特卡洛采样

C. 动态规划　　D. 贝尔曼方程

4. 与马尔可夫奖励过程相比，马尔可夫决策过程引入了哪一个新的元素？（　　）

A. 反馈　　B. 动作

C. 终止状态　　D. 概率转移矩阵

5. 在本章内容范围内，“在状态s，按照某个策略行动后在未来所获得回报值的期望”，这句话描述了状态s的（　　）；“在状态s，按照某个策略采取动作a后在未来所获得回报值的期望”，这句话描述了状态s的（　　）。

A. 策略优化　　B. 价值函数

C. 动作-价值函数　　D. 采样函数

6. 将图7.2中的机器人寻路问题简化为图7.21的2×2的网格，假设有位于s_1位置的机器人拟从s_1这一初始位置向s_4这一目标位置移动。机器人每次只能向上或者向右移动一个方格，到达目标位置s_4则会获得奖励且游戏终止，机器人在移动过程中如果越出方格（s_d）则会被惩罚且被损坏，并且游戏终止。奖励值定义如下：当$S_{t+1}=s_4$时奖励值为1，当$S_{t+1}=s_d$时惩罚值为-1，其他情况下奖励值为0。若折扣因子$\gamma=0.99$，智能体在s_1、s_2、s_3的策略都初始化为上，终止状态s_4、s_d的价值函数定义为0，试通过联立贝尔曼方程给出状态s_1、s_2、s_3的价值函数。

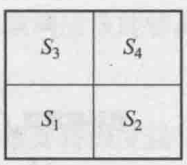


图7.21 2×2的机器人寻路问题

7. 在第6题中，若每个状态的价值函数都初始化为0，智能体在s_1、s_2、s_3的策略都初始化为上，试优化智能体在状态s_3的策略。（提示：使用策略优化定理）

8. 在第6题中，若每个状态的价值函数都初始化为0，智能体在

s_1、s_2、s_3的策略都初始化为上，试使用算法7.5中的价值迭代算法优化智能体在每个状态下的价值函数和策略，并给出迭代过程。

图7.22 Q学习算法的初始状态

0.1/0	0/0
0.1/0	0.1/0

9. 在第6题中，若图7.22表示算法的初始状态，其中a/b表示对应状态的动作-价值函数的取值，斜线左侧的a表示$q_\pi(s, 上)$，斜线右侧的b表示$q_\pi(s, 右)$。若$\alpha = 0.5$，试给出算法7.6中的Q学习算法的一个片段的执行过程，并给出执行完该片段后每个状态的策略。

10. 采样时策略和更新时策略彼此不同的算法称为离策略算法。反之，采样时策略和更新时策略相同的算法称为在策略算法。算法7.7给出的Q学习是一个离策略算法，请给出其对应的在策略版本——Sarsa算法。

*11. 回顾将价值函数作为Actor-Critic算法的基准函数的推导过程，若将基准函数设置为$h(s, a)$，指出推导过程中的哪一步不再适用。

参考文献

[1] ANDREAE J H. Learning Machines: A Unified View [J]. Standard Telecommunications Laboratories, 1966.

[2] BARTOA G, SUTTONR S, ANDERSONC W. Neuronlike adaptive elements that can solve difficult learning control problems [J]. IEEE transactions on systems, man, and cybernetics, 5: 834-846, 1983.

[3] BARTO A G, DUFF M. Monte Carlo matrix inversion and reinforcement learning [J]. Advances in Neural Information Processing Systems, 1994.

[4] BELLMAN R. A Markovian decision process [J]. Journal of Mathematics and Mechanics, 6(5): 679-684, 1957.

[5] BELLMAN R. Dynamic programming [J]. Courier Corporation, 2013.

[6] BENGIO Y, et al. Curriculum learning [C] //Proceedings of the 26th annual international conference on machine learning. Cambridge, MA: ACM press, 2009.

[7] BERTSEKAS D P, TSITSIKLIS J N. Parallel and distributed computation: numerical methods [M]. Englewood Cliffs, NJ: Prentice hall, 1989.

[8] DAYAN P. The convergence of TD (λ) for general λ [J]. Machine learning, 8(3-4): 341-362, 1992.

[9] ECOFFET A, et al. Go-Explore: a New Approach for Hard-Exploration Problems [J]. arXiv preprint arXiv: 1901.10995, 2019.

[10] HENDERSON P, et al.Deep reinforcement learning that matters [J]. Thirty-Second AAAI Conference on Artificial Intelligence, 2018.

[11] HESSEL M, et al. Rainbow: Combining improvements in deep reinforcement learning [J] .Thirty-Second AAAI Conference on Artificial Intelligence, 2018.

[12] KALOS M H, WHITLOCKP A. Monte carlo methods [M]. New Jersey: John Wiley & Sons, 2009.

[13] KULKARNI T D, et al. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation [J]. Advances in neural information processing systems, 2016.

[14] MNIH V, et al. Human-level control through deep reinforcement learning [J]. Nature, 7540: 518-533, 2015.

[15] MNIH V, et al. Asynchronous methods for deep reinforcement learning [J]. International conference on machine learning. 2016.

[16] MUNOS R, et al. Safe and efficient off-policy reinforcement learning [J]. Advances in Neural Information Processing Systems, 2016.

[17] NG A Y, HARADA D, RUSSELL S. Policy invariance under reward transformations: Theory and application to reward shaping [C] //Proceedings of the International Conference on Machine Learning(ICML). [S.l.]: [s.n.], 1999.

[18] PINTO L, et al.Robust adversarial reinforcement learning [C] //Proceedings of the International Conference on Machine Learning(ICML). [S.l.]: [s.n.], 2017.

[19] PRECUP D, et al. Eligibility traces for off-policy policy evaluation [J]. Computer Science Department Faculty Publication Series, 2000.

[20] PUTERMAN M L, MOON C S. Modified policy iteration algorithms for discounted Markov decision problems [J]. Management Science, 24(11): 1127-1137, 1978.

[21] RAJESWARAN A, et al. Epopt: Learning robust neural network policies using model ensembles [J]. arXiv preprint arXiv: 1610.01283, 2016.

[22] RUSUA A, et al. Progressive neural networks [J]. arXiv preprint arXiv: 1606.04671, 2016.

[23] SCHULMAN J, et al. High-dimensional continuous control using generalized advantage estimation [J]. arXiv preprint arXiv: 1506.02438, 2015.

[24] SCHULMAN J, et al. Trust region policy optimization [C] //Proceedings of the International Conference on Machine Learning(ICML). [S.l.]: [s.n.], 2015.

[25] SCHULMAN J, et al. Proximal policy optimization algorithms [J]. arXiv preprint arXiv: 1707.06347, 2017.

[26] SILVER D, et al. Mastering the game of Go with deep neural networks and tree search [J]. Nature, 529(7587): 484-489, 2016.

[27] SILVER D, et al. Mastering the game of go without human knowledge [J]. Nature, 550(7676): 354-360, 2017.

[28] SUTTON R S. Learning to predict by the methods of temporal differences [J]. Machine learning, 3(1): 9-44, 1988.

[29] SUTTON R S, et al. Policy gradient methods for reinforcement learning with function approximation [J]. Advances in neural information processing systems, 2000.

[30] SUTTON R S, BARTOAG.Reinforcement learning: An introduction [M]. Boston: MIT press, 2018.

[31] THRUN S B. Efficient exploration in reinforcement learning [M].Boston: Springer, 1992.

[32] WANG Z, et al.Dueling network architectures for deep reinforcement learning [J].arXiv preprint arXiv: 1511.06581, 2015.

[33] WANG Z, et al. Sample efficient actor-critic with experience replay [J]. arXiv preprint arXiv: 1611.01224, 2016.

[34] WANG Q, et al. Exponentially weighted imitation learning for batched historical data [J].Advances in Neural Information Processing Systems, 2018.

[35] WATKINS C. Learning from delayed rewards [D]. Cambridge: King's College, 1989.
[36] WHITE D J. Dynamic programming [M]. Edinburgh: Oliver & Boyd, 1969.
[37] WHITESON S, et al. Protecting against evaluation overfitting in empirical reinforcement learning [J]. 2011 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL), 2011.
[38] WILLIAMS R J. Simple statistical gradient-following algorithms for connectionist reinforcement learning [J]. Machine learning, 8(3–4): 229–256, 1992.
[39] WILSON S W. ZCS: A zeroth level classifier system [J]. Evolutionary computation, 2(1): 1–18, 1994.
[40] WITTEN I H. An adaptive optimal controller for discrete-time Markov environments [J]. Information and control, 34(4): 286–295, 1977.
[41] YU W, et al. Preparing for the unknown: Learning a universal policy with online system identification [J]. arXiv preprint arXiv: 1702.02453, 2017.
[42] ZHANG C, et al. A study on overfitting in deep reinforcement learning [J]. arXiv preprint arXiv: 1804.06893, 2018.

第8章 人工智能博弈

8

两害相权取其轻，
两利相权取其重

《论语·阳货》曾记载，“子曰：饱食终日，无所用心，难矣哉！不有博弈者乎？为之，犹贤乎已。”宋朝理学家朱熹对这一段的注解中写道：“博，局戏；弈，围棋也。”东汉马融在《围棋赋》说：“三尺之局兮，为战斗场。”可见，古人很早就开始对博弈之道进行了研究。

与其他游戏需要诉诸运气不同，围棋类竞智活动中每一步行动均需由棋手本人意志来决定，每一颗棋子摆放都会衍生出无穷尽可能，蕴含无穷道理，这也是博弈论被重视的原因。

博弈论（game theory）以往是经济学的一个分支，但是随着计算机科学的发展，人工智能技术越来越多地应用在博弈论的研究当中，传统博弈论与人工智能的深度融合闪烁着决策智能的光芒。现代博弈论的思想起源于1944年冯·诺依曼与奥斯卡·摩根斯特恩合著《博弈论与经济行为》。

竞争博弈中的博弈行为是多个带有相互竞争性质的主体，为了达到各自目标和利益，采取的带有对抗性质的行为，即“两害相权取其轻，两利相权取其重”。

现代博弈论主要研究博弈行为中最优的对抗策略及其稳定局势，协助对弈者在一定规则范围内寻求最合理的行为方式，推动机器学习从“数据拟合”过程中以“求取最优解”为核心向博弈对抗过程中“求取均衡解”为核心转变。

8.1 博弈论的相关概念

8.1.1 博弈论的诞生

MOOC微视频：
博弈相关概念

“博弈”这一概念自古有之，古汉语中的“博弈”二字最初指的是六博和围棋两种游戏。其中，围棋之道蕴含着古人谋划策略的智慧，甚至常常被比作行军打仗，而诸如《孙子兵法》等讲述兵书战法的古代典籍更是凸显了古人对策略的重视。《史记·孙子吴起列传》中记载的田忌赛马故事就是中国古代博弈思想的体现。如果将齐威王所拥有的上等、中等、下等三种类别的马匹分别记作“A+”“B+”和“C+”，田忌所拥有的上等、中等、下等三种类别的马匹分别记作“A-”“B-”和“C-”，表8.1给出了两人所拥有的马匹进行对局的胜负情况。

表8.1 齐威王与田忌进行赛马中所采取的不同对局

对局	齐王马	田忌马	结果	对局	齐王马	田忌马	结果
1	A+	A-	齐王胜	1	A+	C-	齐王胜
2	B+	B-	齐王胜	2	B+	A-	田忌胜
3	C+	C-	齐王胜	3	C+	B-	田忌胜

如果田忌分别用自己的上等马、中等马和下等马与齐王的上等马、中等马和下等马比赛，都会因为自己的马略逊一筹而以0∶3比分输掉比

赛。但是，如果田忌先用自己的下等马与齐王的上等马对决，再用自己的上等马和中等马分别与齐王的中等马和下等马对决，则会以2∶1比分赢得比赛，这也就是所谓的“以己之长，攻彼之短”的博弈思想。

现代博弈论的思想起源于1944年冯·诺依曼（John von Neumann）与奥斯卡·摩根斯特恩（Oskar Morgenstern）合著的《博弈论与经济行为》[4]。这本书以数学形式来阐述了博弈论及其应用。将博弈论发扬光大的是约翰·纳什（John Forbes Nash Jr.）。1950年，纳什在其博士论文中提出“非合作博弈（non-cooperative games）”，并给出博弈过程中存在均衡解这一纳什均衡（Nash equilibrium）思想[2, 3]。纳什本人因为这一学术贡献于1994年获得了诺贝尔经济学奖。

博弈论主要研究的一个内容是博弈行为中最优的对抗策略及其稳定局势，并协助人们在一定规则范围内寻求最合理的行为方式。所谓博弈行为指的是带有相互竞争性质的主体，为了达到各自目标和利益，采取的带有对抗性质的行为。博弈论研究的目的是为博弈的参与者谋取最大的利益，也就是“两害相权取其轻，两利相权取其重”，《墨子·大取》中也有“利之中取大、害之中取小”的相似思想。

8.1.2 博弈论术语与囚徒困境

博弈中，参与博弈的决策主体被称为**玩家**或**参与者**（player），这些参与者通常被认为是完全理性的。在博弈中，参与者总能或多或少的获得一些与博弈相关的知识，如博弈的进展以及对手采取的策略等，这些知识被称为**信息**（information）。此外，博弈需要遵循一定的**规则**（rule），这些规则包括参与者行动的先后顺序、参与者在采取特定行动后导致的后果等。

参与者可以采取的行动方案被称为**策略**（strategy），策略必须是一整套在采取行动之前就已经准备好的完整方案，其中一个参与者所能采纳策略的全体组合形成了其所拥有的**策略集**（strategy set）。当然参与者可以按照一定的概率来随机选择若干不同的行动，这种策略被称为**混合策略**（mixed strategy）。相应地，如果参与者能够确定的选择行为，这种策略被称为**纯策略**（pure strategy）。

参与者采取了各自的行动之后形成的状态被称为**局势**（outcome），而在不同的局势下，各个参与者所得到的利益或回报被称为博弈的**收**

益（payoff），混合策略下对应的玩家收益应当为期望收益（expected payoff）。

现在以囚徒困境（prisoner's dilemma）为例，介绍博弈的过程和结果。

1950年，兰德公司的梅里尔·弗勒德（Merrill Flood）和梅尔文·德雷希尔（Melvin Dresher）拟定了相关困境理论，后来美国普林斯顿大学数学家阿尔伯特·塔克（Albert W. Tucker）以“囚徒方式”对相关困境理论进行如下阐述：警方逮捕了共同犯罪的甲、乙两人，由于警方没有掌握充分的证据，所以将两人分开审讯。若一人认罪并指证对方，而另一方保持沉默，则此人会被当即释放，沉默者会被判监禁10年；若两人都保持沉默，则根据已有的犯罪事实（无充分证据）两人各判半年；若两人都认罪并相互指证，则两人各判5年。

表8.2展示囚徒困境这个例子中甲和乙以非合作方式进行博弈的结果。

表8.2　囚徒困境中甲和乙通过非合作方式采取行动获得的收益

	乙沉默（合作）	乙认罪（背叛）
甲沉默（合作）	二人各服刑半年	乙被释放，甲服刑10年
甲认罪（背叛）	甲被释放，乙服刑10年	二人各服刑5年

在这一博弈过程中，参与者为甲和乙两个嫌疑犯，两人只能独立决策而不能得知对方所做出的决定，即以非合作方式来进行博弈。两人可采取的策略集$s=\{$认罪，沉默$\}$。表8.3给出了甲和乙分别采取不同行动后所得到局势及其收益情况。

表8.3　囚徒困境中甲乙博弈局势及收益

局势	甲收益	乙收益
甲认罪，乙沉默	0	-10
甲认罪，乙认罪（均衡解）	-5	-5
甲沉默，乙认罪	-10	0
甲沉默，乙沉默（最优解）	-0.5	-0.5

从表8.3可知，甲、乙二人同时保持沉默对两者而言是最优解。但是实际上，甲和乙两人均倾向于选择同时认罪而达到一种均衡解，这也是这一博弈被称为困境的原因。

在博弈过程中，对甲而言，如果乙选择沉默，那么甲选择认罪会被

释放，而选择沉默会被判刑半年，所以甲选择认罪，如果乙选择认罪，那么甲选择认罪会被判刑五年，而选择沉默会被判刑十年，所以甲仍然应该选择认罪，所以不论乙的选择是什么，甲都应该选择认罪，对乙也是同理，不论甲的选择是什么，乙都应该选择认罪。最终在囚徒困境的博弈中，甲和乙都从理性角度来考虑了各自选择，选择了同时认罪这一行为。在甲和乙选择认罪这个结果中，如果甲或乙单独改变自己的选择（其他人的选择保持不变），则对改变选择的人而言不会获得更好的收益。如果甲将“认罪”选择更改为“沉默”，在乙保持“认罪”这一选择前提下，甲将从获得“5年处罚”变为获得“10年处罚”的结果，可见这一结果与甲不改变选择相比，所获得收益更加糟糕。

因此，甲和乙两人同时认罪是一种稳定局势，而其他情况都不是稳定的局势，同时囚徒困境也说明了总体最优局势（即甲和乙同时保持沉默）并不是均衡局势。另外，囚徒困境也说明了个人最佳选择不等于团体最佳选择。

8.1.3 博弈的分类

现代博弈论根据博弈规则的不同将博弈分为不同的种类。

按照博弈过程中是否允许参与者之间合作可以划分为**合作博弈**（cooperative game）和**非合作博弈**（non-cooperative game）。在合作博弈中，部分参与者可以组成联盟以获得更大的收益，与此相反，非合作博弈中参与者在决策时都彼此独立，不事先达成合作意向。

按照博弈过程中参与者决策的时间可以划分为**静态博弈**（static game）和**动态博弈**（dynamic game）。在静态博弈中，所有参与者同时决策，或参与者互相不知道对方的决策；而动态博弈中参与者所采取行动的先后顺序由规则决定，且后行动者知道先行动者所采取的行动。

按照博弈过程中参与者对信息的了解程度可以划分为**完全信息博弈**（complete information game）和**不完全信息博弈**（incomplete information game）。所谓完全信息是指所有参与者均了解其他参与者的策略集、收益等信息，而不完全信息指并非所有参与者均掌握了所有信息。若博弈的参与者知道所有参与者之前采取的行动，这种博弈被称为**完美信息博弈**（perfect information game）。

按照博弈局势中所有参与者的总收益可以划分为**零和博弈**（zero-sum

game）和**非零和博弈**（non-zero-sum game）。所谓零和博弈是指在终结局势下，所有参与者的收益总和为零，而非零和博弈是指在终结局势下，所有参与者的收益总和不为零。

前文提到的囚徒困境属于非合作、完全信息的不完美信息静态博弈。

8.1.4 纳什均衡

博弈的稳定局势即为**纳什均衡**（Nash equilibrium），其指参与者做出了这样一种策略组合，在该策略组合上，任何参与者单独改变策略都不会得到好处。换句话说，如果在一个策略组合上，当所有博弈参与者都不改变策略时，就不会有任何博弈参与者会有动机来改变其自身做出的策略，则该策略组合就是一个纳什均衡策略。囚徒困境中两人同时认罪就是一种纳什均衡。

在博弈论知识结构上，纳什均衡概念具有基础性地位，这个概念的基础性如同波尔兹曼分布概念之于统计物理、DNA双螺旋结构发现之于生物学。纳什均衡概念体现了一种静态（也被称为稳态）平衡。在博弈中能够达到纳什均衡解的基础是基于博弈参与者完全理性这样的假设。

下面以雇主对雇员监督这一例子，来介绍混合策略下完全信息静态博弈中求取纳什均衡解的过程。

这一例子阐释了雇主检查工作与雇员偷懒之间进行的博弈。假设雇员在不偷懒情况下工作的付出为H，对公司的贡献为V；雇主给雇员所发工资为W，雇主去检查工作的成本为C；若雇主发现雇员偷懒则会对其收取罚金F。根据实际情况可知$H<W<V$, $W>C$，表8.4给出了雇主与雇员每次采取某一行动后的收益。

表8.4　雇主－雇员每次采取对应行动后的收益

		雇员	
		偷懒	不偷懒
雇主	检查	$-C+F$, $-F$	$V-W-C$, $W-H$
	不检查	$-W$, W	$V-W$, $W-H$

在这场博弈过程中，博弈参与者是雇主与雇员两人，由于是静态博弈过程，所以两人无法得知对方采取行动。对雇员来说，可以采取的策略集为$s_{employee}=\{$偷懒，不偷懒$\}$；对雇主来说，可以采取的策略集为

$s_{employer}=\{$检查，不检查$\}$。两人均按照混合策略采取行动。记雇主检查工作的概率为α、雇员偷懒的概率为β，雇主和雇员对应的收益情况表8.5所示。

表8.5　雇主－雇员之间博弈的期望收益

参与者	采取策略	期望收益
雇主	检查	$T_1=\beta(-C+F)+(1-\beta)(V-W-C)$
	不检查	$T_2=-\beta W+(1-\beta)(V-W)$
雇员	偷懒	$T_3=-\alpha F+(1-\alpha)W$
	不偷懒	$T_4=\alpha(W-H)+(1-\alpha)(W-H)=W-H$

现在寻找雇主－雇员两人博弈中所存在的稳定局势：无论雇主以何种概率进行检查，雇员偷懒或不偷懒的收益都一样，即$T_3=T_4$。在稳定局势中，无论雇员如何改变自己的策略，都不会使得自己的收益增加。同理，无论雇员以何种概率偷懒，雇主检查或不检查的收益都一样，即$T_1=T_2$，此时无论雇主如何改变自己的策略也不会使其收益增加。也就是说，在稳定局势下，如果其他参与者不改变行动，则任何一个参与者单独改变自己的行动，都不会使自己的收益增加。

在纳什均衡下，由于$T_3=T_4$，可知雇主检查工作的概率为：

$$\alpha=\frac{H}{W+F}$$

同理，由于$T_1=T_2$，可知雇员偷懒的概率为：

$$\beta=\frac{C}{W+F}$$

求解得到纳什均衡结果具有一定的指导性意义。将雇员偷懒的概率$\beta=\frac{C}{W+F}$代入雇主的期望收益，可得在均衡局势下，雇主的预期收益如下：

$$T_1=T_2=V-W-\frac{CV}{W+F}$$

对上式中W求导，可知当$W=\sqrt{CV}-F$时，雇主的收益最大，其值为$T_{max}=V-2\sqrt{CV}+F$。根据这一结论，雇主给雇员支付工资为$\sqrt{CV}-F$时，可使得自身的收益最大，这里可以看出博弈论对帮助人们科学决策具有重要作用。

除了提出了纳什均衡，纳什本人还提出了**纳什定理**：任何有限策略下的博弈都有一个混合策略纳什均衡。即**若参与者有限，每位参与者采取策略的集合有限，收益函数为实值函数，则博弈对抗必存在混合策略意义下的纳什均衡**[2]。当然，纳什定理仅仅是一个存在性定理，求解纳什均衡或近似纳什均衡的算法设计与复杂度证明仍然是当前算法博弈论（algorithmic game theory）的研究热点。

本书对于纳什定理的证明不作详细说明，这里使用数学归纳法对于双人的零和动态博弈进行均衡点存在性的证明，即证明**策梅洛定理**（Zermelo's theorem）：**对于任意一个有限步的双人完全信息零和动态博弈，一定存在先手必胜策略或后手必胜策略或双方保平策略**[5]。

假设双人博弈过程结果只有胜、负、平三种结果，博弈过程最长持续N步，下面通过数学归纳法来证明策梅洛定理。

（1）当$N=1$时，先手玩家必选择自己收益最大的策略，若存在获胜的行动，则存在先手必胜策略。若不存在使先手玩家获胜的行动，则先手玩家会选择平局的行动，于是存在双方保平策略。若仅存在后手玩家获胜的行动，则存在后手必胜策略。

（2）假设当$N=1, 2, \cdots, k$时，均存在先手必胜策略、后手必胜策略或双方保平策略。

（3）当$N=k+1$时，将先手玩家行动一次后的局势视为开启了一个新的博弈。在这开启的新博弈中，先手玩家为原博弈的后手玩家，新博弈中的后手玩家为原博弈的先手，应用（2）中的假设，新开启的博弈必存在先手必胜策略或后手必胜策略或双方保平策略，则原博弈亦必存在后手必胜策略或先手必胜策略或双方保平策略。

（4）综上所述，对于任意一个有限步的双人完全信息零和动态博弈，一定存在先手必胜策略或后手必胜策略或双方保平策略。

提请注意的是，策梅洛定理仅对两人博弈有效，如果博弈竞技者超过了2人，如对于3人博弈，策梅洛定理无法保证3方中一定有一方获胜、其他两方必败或者三方和局的策略。

策梅洛定理为本书第3章所介绍的最小最大（minimax）搜索算法的合理性提供了理论基础，即在两人对抗博弈的有限搜索空间内，一定存在先手必胜策略或后手必胜策略或双方保平策略。当然，对于围棋这种搜索空间非常大的博弈，应用策梅洛定理也就没有意义了。策梅洛定

理的提出比纳什均衡早了二十多年，纳什定理其实可以看作策梅洛定理对非零和博弈和多人博弈的推广。

8.1.5 人工智能与博弈论

计算机科学与博弈论的交叉领域非常多，现代博弈论的奠基人冯·诺依曼同时也是现代计算机之父，目前所使用的计算机架构即为“冯·诺依曼体系结构”。

随着互联网的兴起，博弈论在互联网经济、共享经济等诸多领域都与计算机科学产生交叉。人工智能解决的许多问题也越来越多地体现出博弈论的思想，如多智能体系统、AI游戏玩家、广告推荐等。

总体而言，人工智能与博弈论的交叉领域主要分为**博弈策略求解**和**博弈规则设计**两个方面。

8.2 博弈策略求解

8.2.1 研究问题

博弈论为许多问题求解提供了数学模型，同时纳什定理说明了博弈论解的存在性。这样，可以借鉴人工智能算法，对博弈论的均衡局势或最优策略进行求解。当然，博弈策略求解过程中主要面临的挑战是如何高效地搜索最优策略。

利用人工智能算法可以对很多数学上无法求解的问题进行近似模拟，如围棋走子策略等对大规模解空间搜索，又如信息安全、商业竞争这种动态博弈的均衡局势求解问题，还有扑克这种非完全信息的博弈问题。下面详细介绍一种非完全信息博弈的策略求解算法。

8.2.2 虚拟遗憾最小化算法

对于一个有N个玩家参加的博弈，玩家i在博弈中采取的策略记为σ_i。对于所有玩家来说，他们的所有策略构成了一个策略组合，记作$\sigma=\{\sigma_1,\sigma_2,\cdots,\sigma_N\}$。策略组中，除玩家$i$外，其他玩家的策略组合记作$\sigma_{-i}=\{\sigma_1,\sigma_2,\cdots,\sigma_{i-1},\sigma_{i+1},\cdots,\sigma_N\}$。

给定策略组合σ，玩家i在终结局势下的收益记作$u_i(\sigma)$。在给定其

他玩家的策略组合 σ_{-i} 的情况下，对玩家 i 而言的**最优反应策略** σ_i^* 满足如下条件：

$$u_i\left(\sigma_i^*, \sigma_{-i}\right) \geqslant \max_{\sigma_i' \in \Sigma i} u_i\left(\sigma_i', \sigma_{-i}\right)$$

$\sum i$ 是玩家 i 可以选择的所有策略，如上条件表示当玩家 i 采用最优反应策略时，玩家 i 能够获得最大收益。

在策略组合 σ^* 中，如果每个玩家的策略相对于其他玩家的策略而言都是最佳反应策略，那么策略组合 σ^* 就是一个**纳什均衡**（Nash equilibrium）策略。即策略组 $\sigma^* = \left\{\sigma_1^*, \sigma_2^*, \cdots, \sigma_N^*\right\}$ 对任意玩家 $i = 1, \cdots, N$，满足如下条件：

$$u_i\left(\sigma^*\right) \geqslant \max_{\sigma_i' \in \Sigma i} u_i\left(\sigma_1^*, \sigma_2^*, \cdots, \sigma_i', \cdots \sigma_N^*\right)$$

在博弈策略求解的过程中，希望求解得到每个玩家最优反应策略，若所有玩家都是理性的，则算法求解最优反应策略就是一个纳什均衡。考虑到计算资源有限这一前提，难以通过遍历博弈中所有策略组合来找到一个最优反应策略，因此需要找到一种能快速发现近似纳什均衡的方法。

接下来给大家介绍通过遗憾最小化（regret minimization）算法来近似求解纳什均衡。遗憾最小化算法是一种根据以往博弈过程中所得遗憾程度来选择未来行为的方法。

玩家 i 在过去 T 轮中采取策略 σ_i 的累加遗憾值定义如下：

$$Regret_i^T\left(\sigma_i\right) = \sum_{t=1}^{T}\left(u_i\left(\sigma_i, \sigma_{-i}^t\right) - u_i\left(\sigma^t\right)\right) \qquad (式8.1)$$

其中 σ^t 和 σ_{-i}^t 分别表示第 t 轮中所有玩家的策略组合和除了玩家 i 以外的策略组合。简单地说，累加遗憾值代表着在过去 T 轮中，玩家 i 在每一轮中选择策略 σ_i 所得收益与采取其他策略所得收益之差的累加。

在得到玩家 i 的所有可选策略的遗憾值后，可以根据遗憾值的大小来选择后续第 $T+1$ 轮博弈的策略，这种选择方式被称为**遗憾匹配**[6]。当然，通常遗憾值为负数的策略被认为不能提升下一时刻收益，所以如下定义有效遗憾值：

$$Regret_i^{T,+}\left(\sigma_i\right) = \max\left(Regret_i^T\left(\sigma_i\right), 0\right) \qquad (式8.2)$$

利用有效遗憾值的遗憾匹配可得到玩家 i 在 T 轮后第 $T+1$ 轮选择策

略σ_i的概率$P\left(\sigma_i^{T+1}\right)$为：

$$P(\sigma_i^{T+1})=\begin{cases}\dfrac{Regret_i^{T,+}(\sigma_i)}{\sum\limits_{\sigma_i'\in\Sigma_i}Regret_i^{T,+}(\sigma_i')} & 当\sum\limits_{\sigma_i'\in\Sigma_i}Regret_i^{T,+}(\sigma_i')>0\\ \dfrac{1}{|\Sigma_i|} & 其他\end{cases} \quad （式8.3）$$

$|\Sigma_i|$表示玩家i所有策略的总数。显然，如果在过往T轮中策略σ_i所带来的遗憾值大、其他策略σ_i'所带来的遗憾值小，则在第$T+1$轮选择策略σ_i的概率值$P\left(\sigma_i^{T+1}\right)$就大。也就是说，带来越大遗憾值的策略具有更高的价值，因此其在后续被选择的概率就应该越大。如果没有一个能够提升前T轮收益的策略，则在后续轮次中随机选择一种策略。依照一定的概率选择行动是为了防止对手发现自己所采取的策略（如采取遗憾值最大的策略）。

这种遗憾匹配的方式既能启发式地提升接下来博弈中的预期收益，又能利用概率采样的随机性避免被对手猜到自己的策略。

接下来通过石头-布-剪刀（rock-paper-scissors，RPS）的游戏来具体介绍遗憾值最小化算法过程。石头-剪刀-布游戏有两名玩家参加，在博弈时两人同时决策，表8.6给出了这个游戏可能出现的不同局势及收益。

表8.6　石头-剪刀-布游戏的局势及收益（u_A, u_B）

		玩家B		
		石头(R)	剪刀(S)	布(P)
玩家A	石头(R)	(0, 0)	(1, -1)	(-1, 1)
	剪刀(S)	(-1, 1)	(0, 0)	(1, -1)
	布(P)	(1, -1)	(-1, 1)	(0, 0)

假设第一局时，玩家A出“石头（R）”、玩家B出“布（P）”。从表8.6可见，第一局中玩家A的收益$u_A(R,P)=-1$，玩家B的收益为$u_B(P,R)=1$。从表8.6所给信息可计算得到玩家A在第一局中没有选择“布”这一策略的遗憾值$Regret_A^1(P)=u_A(P,P)-u_A(R,P)=0-(-1)=1$，玩家A在第一局中没有选择“剪刀”这一策略的遗憾值$Regret_A^1(S)=u_A(S,P)-u_A(R,P)=1-(-1)=2$。注意因为玩家A实际选择出石头（R），所以没有选择出“石头”的遗憾值$Regret_A^1(R)=0$。根据遗憾匹配算法，在第一局结束后，玩家A在第二局选择“石头”“剪刀”和“布”

这三个策略的概率分别为0、$\frac{2}{3}$、$\frac{1}{3}$。

同时，从表8.6所给信息可计算得到玩家B在第一局中没有选择“石头”这一策略的遗憾值$Regret_B^1(R)=u_B(R,R)-u_B(R,P)=0-1=-1$，玩家B在第一局中没有选择“剪刀”这一策略的遗憾值$Regret_B^1(S)=u_B(R,S)-u_A(R,P)=-1-1=-2$。对玩家B而言，$\sum_{\sigma_i'\in\Sigma_i}Regret_i^{T,+}(\sigma_i')=-3<0$，因此下一轮中以$\frac{1}{|\Sigma_i|}=\frac{1}{3}$的概率从“石头”“剪刀”和“布”中选择一个策略。

假设玩家A在第二局按照遗憾值最大的策略选择“剪刀”，玩家B随机选择“石头”，根据公式$Regret_A^2(\sigma_i)=\sum_{t=1}^{2}\left(u_A\left(\sigma_i,\sigma_{-i}^t\right)-u_A\left(\sigma^t\right)\right)$可计算得到玩家A的累加遗憾值（如表8.7所示）。

表8.7　玩家A在两轮后所得到的遗憾值

遗憾值/策略	石头	剪刀	布
第一轮	0	2	1
第二轮	1	0	2
$Regret_A^2$	1	2	3

从上表可知，前二局结束后，在第三局时，玩家A选择“石头”“剪刀”和“布”的概率分别为$\frac{1}{6}$、$\frac{2}{6}\left(\frac{1}{3}\right)$、$\frac{3}{6}\left(\frac{1}{2}\right)$。需要注意的是，玩家A既可以按照遗憾值最小在第三轮来选择“布”这一策略，又可以随机选择“石头”或“剪刀”以防止自己的策略被对手察觉。

遗憾最小化算法在实际使用中需要反复模拟博弈多次，才能拟合到近似的最优反应策略。当然，并不是所有的博弈都像石头-剪刀-布游戏这样一次决策就可以得出胜负，对于围棋和扑克等需要多次决策才能判断胜负的博弈问题，那么应该怎么办呢？

为了解决这个问题，需要计算状态转移序列中每个状态的“虚拟遗憾值”，这就是**虚拟遗憾值最小化算法**（counterfactual regret minimization）[7]。

对于任何序贯决策的博弈对抗，可将博弈过程表示成一棵博弈树，博弈树中的每一个中间结点都是一个信息集I，信息集中包含了博弈中当前的状态。给定博弈树的每一个结点，玩家都可以从一系列的动作中选择一个，然后状态发生转换，如此周而复始，直到终局（博弈树的叶子

结点）。玩家在当前状态下可采取的策略就是当前状态下所有可能动作的一个概率分布。

具体而言，在信息集I下，玩家可以采取的行动集合记作$A(I)$。玩家i所采取的行动$a_i \in A(I)$可认为是其采取的策略σ_i的一部分。在信息集I下采取行动a所代表的策略记为$\sigma_{I\to a}$。这样，要计算虚拟遗憾值的对象就是博弈树中每个中间结点在信息集下所采取的行动，并根据遗憾值匹配得到该结点在信息集下应该采取的策略$\sigma_{I\to a}$。

在一次博弈中，所有玩家交替采取的行动序列记为h（从根结点到当前结点的路径），对于所有玩家的策略组合σ，行动序列h出现的概率记为$\pi^{\sigma}(h)$，不同的行动序列可以从根结点到达当前结点的信息集I（即不同决策路径可到达博弈树中同一个中间结点）。在策略组合σ下，所有能够到达该信息集的行动序列的概率累加就是该信息集的出现概率，即$\pi^{\sigma}(I)=\sum_{h\in I}\pi^{\sigma}(h)$。

博弈的终结局势集合也就是博弈树中叶子结点的集合，记为Z。对于任意一个终结局势$z\in Z$，玩家i在此终点局势下的收益记作$u_i(z)$。给定行动序列h，依照策略组合σ最终到达终结局势z的概率记作$\pi^{\sigma}(h,z)$。

在策略组合σ下，对玩家i而言，如下计算从根结点到当前结点的行动序列路径h的虚拟价值：

$$v_i(\sigma,h)=\sum_{z\in Z}\underbrace{\pi_{-i}^{\sigma}(h)}_{\text{不考虑玩家}i\text{的策略到达当前结点概率}}\times\underbrace{\pi^{\sigma}(h,z)}_{\text{从当前结点到叶子结点概率}}\times\underbrace{u_i(z)}_{\text{叶子结点}z\text{收益}} \quad \text{（式8.4）}$$

在（式8.4）中，$\pi_{-i}^{\sigma}(h)$表示从根结点出发，不考虑玩家i的策略，仅考虑其他玩家策略而经过路径h到达当前结点的概率。也就是说，即使玩家i有其他策略，总是要求玩家i在每次选择时都选择路径h中对应的动作，以保证从根结点出发能够到达当前结点。可见，行动序列路径h的虚拟价值等于如下三项结果的乘积：不考虑玩家i的策略（仅考虑其他玩家策略）经过路径h到达当前结点的概率、从当前结点走到叶子结点（博弈结束）的概率、所到达叶子结点的收益。

在定义了行动序列路径h的虚拟价值之后，就可如下计算玩家i在基于路径h到达当前结点采取行动a的遗憾值：

$$r_i(h,a)=v_i(\sigma_{I\to a},h)-v_i(\sigma,h) \quad \text{（式8.5）}$$

可见，该遗憾值是玩家i通过行动序列h到达当前结点采取行动a得到的虚拟价值减去采用策略σ所得到路径h的虚拟价值。对能够到达同

一个信息集I（即博弈树中同一个中间结点）的所有行动序列的遗憾值进行累加，即可得到信息集I的遗憾值：

$$r_i(I,a)=\sum_{h\in I} r_i(h,a) \tag{式8.6}$$

类似于遗憾最小化算法，虚拟遗憾最小化的遗憾值是T轮重复博弈后的累加值：

$$Regret_i^T(I,a)=\sum_{t=1}^{T} r_i^t(I,a) \tag{式8.7}$$

$r_i^t(I,a)$表示玩家i在第t轮中于当前结点选择行动a的遗憾值，进一步可以定义有效虚拟遗憾值：

$$Regret_i^{T,+}(I,a)=\max\left(R_i^T(I,a),0\right) \tag{式8.8}$$

根据有效虚拟遗憾值进行遗憾匹配以计算经过T轮博弈后，玩家i在信息集I情况下于后续$T+1$轮选择行动a的概率：

$$\sigma_i^{T+1}(I,a)=\begin{cases}\dfrac{Regret_i^{T,+}(I,a)}{\sum\limits_{a\in A(I)} Regret_i^{T,+}(I,a)} & \text{当} \sum\limits_{a\in A(I)} Regret_i^{T,+}(I,a)>0 \\ \dfrac{1}{|A(I)|} & \text{其他}\end{cases} \tag{式8.9}$$

$|A(I)|$表示当前信息集下能够采取的行动总数，在没有行动能够提升当前信息集的虚拟价值时，可通过随机的方式决定接下来采取的行动。当然在实际中为了防止对手知晓选手自己的策略，可在适当时候不按照虚拟遗憾值最小来选择策略，而是随机选择一个策略。

在虚拟最小化算法的求解过程中，同样需要反复模拟多轮博弈来拟合最佳反应策略，算法步骤如下：

（1）初始化遗憾值和累加策略表为0；

（2）采用随机选择的方法来决定策略；

（3）利用当前策略与对手进行博弈；

（4）计算每个玩家采取每次行为后的遗憾值；

（5）根据博弈结果计算每个行动的累加遗憾值大小来更新策略；

（6）重复（3）到（5）步若干次，不断的优化策略；

（7）根据重复博弈最终的策略，完成最终的动作选择。

为了更好地说明虚拟遗憾值最小化算法，这里以库恩扑克（Kunh's pocker）为例来详细展示虚拟遗憾最小化算法的流程。

库恩扑克[8]是一种简单的有限注扑克游戏，由两名玩家进行游戏博弈，游戏中仅提供牌值为1、2和3的三张纸牌。每轮中每位玩家各持一张纸牌，每位玩家根据各自判断来决定是否追加定额赌注。摊牌阶段时来比较未弃牌玩家的底牌大小，底牌值最大的玩家即为胜者。

双人库恩扑克这一游戏的规则以及游戏中定义的收益如表8.8所示。

表8.8　双人库恩扑克游戏规则

玩家A	玩家B	玩家A	结果
过牌	过牌	游戏结束	牌值大的玩家 +1
加注	加注	游戏结束	牌值大的玩家 +2
过牌	加注	过牌	玩家B +1
过牌	加注	加注	牌值大的玩家 +2
加注	过牌	游戏结束	玩家A +1

如表8.8第五行所示，玩家A过牌后，玩家B选择加注，然后玩家A选择加注，这时根据玩家A和玩家B的手中纸牌大小来判断博弈输赢，在这一局势下，纸牌牌面值大的玩家收益为2。其他博弈路径的输赢规则可参见表8.8中其他行所示结果。

根据库恩扑克的游戏规则，可以画出库恩扑克的先手玩家（A）的博弈树，如图8.1所示。

在初始根结点下，三条边分别代表玩家A拿到三张不同手牌的情况。库恩扑克的信息集（即博弈树的中间结点）有12个：$\{1, 1P, 1B, 1BP, 2, 2P, 2B, 2PB, 3, 3P, 3B, 3PB\}$。在信息集中，$P$表示过牌、$B$表示下注。

要注意的是从根结点到达每个信息集的路径只有一条，该路径所对应的行动序列也只有一种，例如，信息集$1PB$（博弈树中的一个中间结点）可通过如下路径到达：

1(玩家A拿到大小为1纸牌)$\xrightarrow{P}$ $1P$(玩家A选择过牌)$\xrightarrow{P}$ $1PB$(玩家B选择加注)

假设初始情况下，两个玩家都以随机选择的策略进行决策，即在任一结点，两者都以50%概率分别选择过牌或加注两个动作。按照这种策略，若第一轮中玩家A采取的博弈过程为$1\xrightarrow{P}1P\xrightarrow{B}1PB\xrightarrow{B}Z_2$，则通过这条路径所到达的叶子结点收益为$u_A(Z_2)=-2$。如图8.2所示。

这里以信息集$\{1PB\}$为例，来计算玩家A通过路径$1\xrightarrow{P}1P\xrightarrow{B}1PB$到达当前结点$\{1PB\}$后选择“过牌”行动的遗憾值。

在当前策略下，行动序列路径$h=\{P\to B\}$产生的概率为：

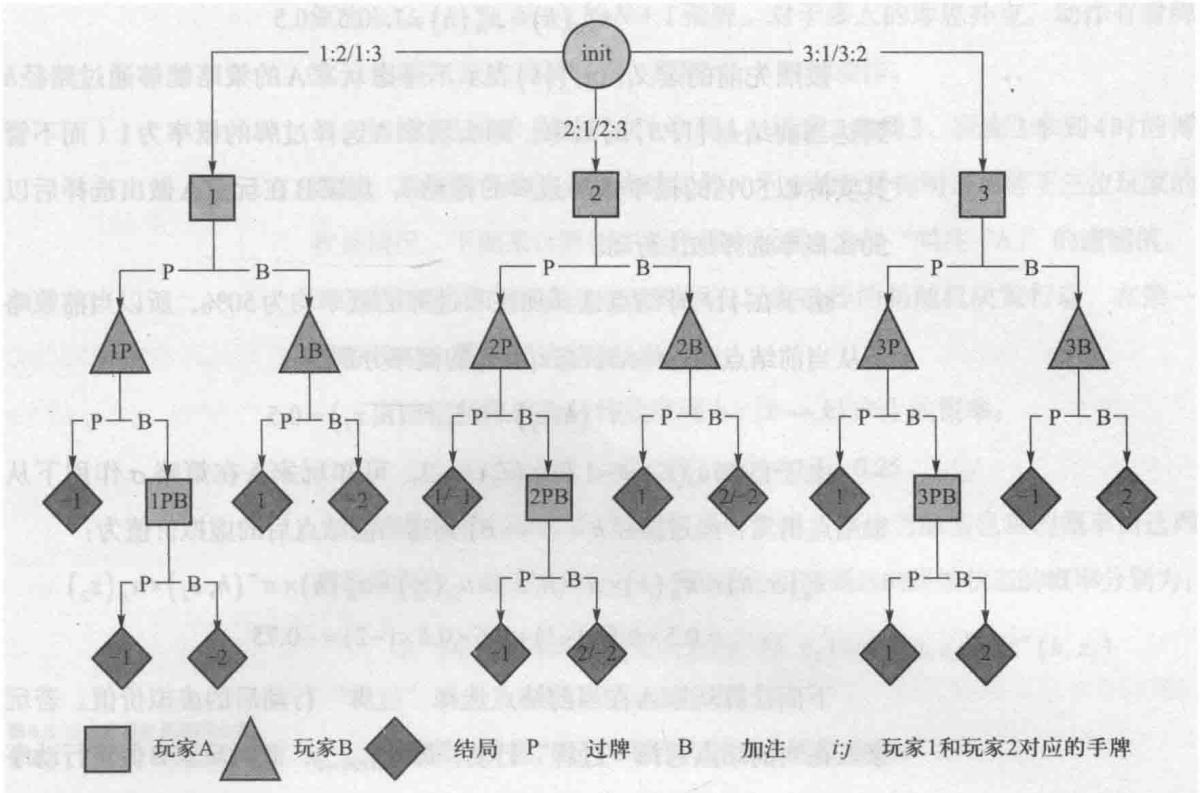


图8.1 先手玩家A对应的库恩扑克博弈树

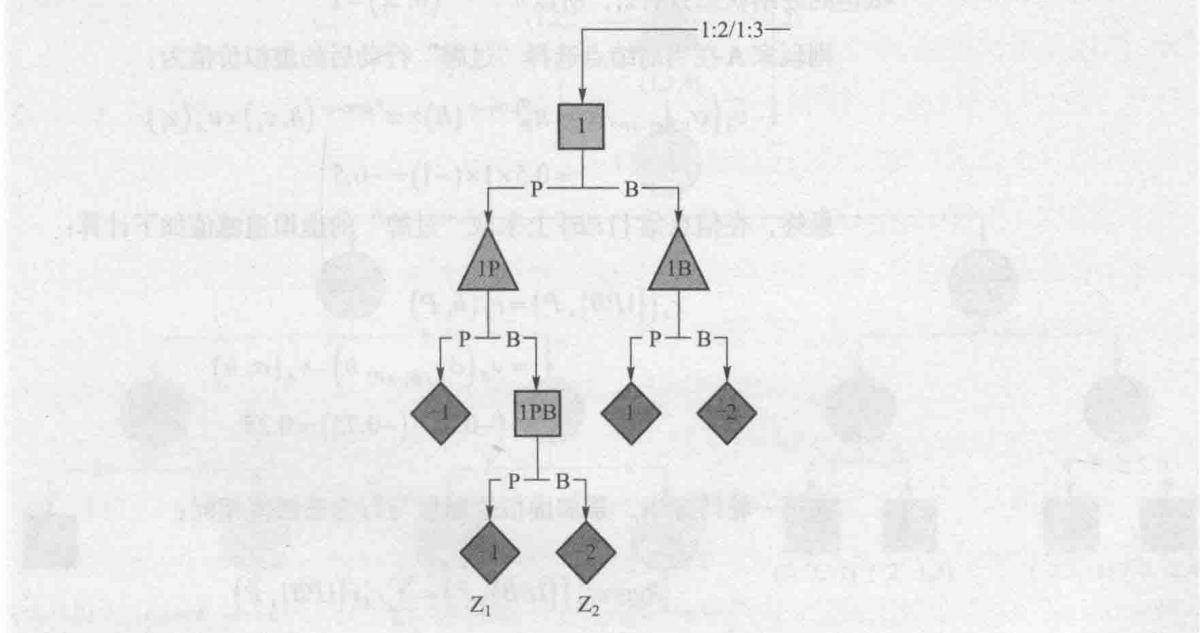


图8.2 先手玩家手牌为1的库恩扑克博弈树

$$\pi_{-A}^{\sigma}(h)=\pi_{B}^{\sigma}(h)=1\times 0.5=0.5$$

按照先前的定义，$\pi_{-A}^{\sigma}(h)$表示不考虑玩家A的策略能够通过路径h到达当前结点$\{1PB\}$的概率，那么玩家A选择过牌的概率为1（而不管其实际以50%的概率选择过牌的行动），玩家B在玩家A做出选择后以50%概率选择加注行动。

由于在$\{1PB\}$结点选择加注和过牌的概率均为50%，所以当前策略下从当前结点到达终结状态z_1和z_2的概率分别为：

$$\pi^{\sigma}(h,z_1)=0.5,\pi^{\sigma}(h,z_2)=0.5$$

由于已知$u_A(Z_1)=-1$和$u_A(Z_2)=-2$，可知玩家A在策略σ作用下从根结点出发、经过路径$h=\{P\to B\}$到达当前结点后的虚拟价值为：

$$\begin{aligned}v_A(\sigma,h)&=\pi_B^{\sigma}(h)\times\pi^{\sigma}(h,z_1)\times u_A(z_1)+\pi_B^{\sigma}(h)\times\pi^{\sigma}(h,z_2)\times u_A(z_2)\\&=0.5\times 0.5\times(-1)+0.5\times 0.5\times(-2)=-0.75\end{aligned}$$

下面计算玩家A在当前结点选择“过牌”行动后的虚拟价值。若玩家A在当前结点选择“过牌”行动，即$\sigma_{\{1PB\}\to P}$，此时玩家B促使行动序列$h=\{P\to B\}$发生的概率仍然为$\pi_B^{\sigma_{\{1PB\}\to P}}(h)=0.5$。由于从当前结点出发抵达的终结状态只有$z_1$，所以$\pi^{\sigma_{\{1PB\}\to P}}(h,z_1)=1$

则玩家A在当前结点选择“过牌”行动后的虚拟价值为：

$$\begin{aligned}v_A\left(\sigma_{\{1PB\}\to P},h\right)&=\pi_B^{\sigma_{\{1PB\}\to P}}(h)\times\pi^{\sigma_{\{1PB\}\to P}}(h,z_1)\times u_A(z_1)\\&=0.5\times 1\times(-1)=-0.5\end{aligned}$$

最终，在信息集$\{1PB\}$上采取“过牌”的虚拟遗憾值如下计算：

$$\begin{aligned}r_A(\{1PB\},P)&=r_A(h,P)\\&=v_A\left(\sigma_{\{1PB\}\to P},h\right)-v_A(\sigma,h)\\&=(-0.5)-(-0.75)=0.25\end{aligned}$$

在第一轮博弈中，累加虚拟遗憾值与行为遗憾值相同：

$$Regret_A^1(\{1PB\},P)=\sum_{t=1}^{1}r_A^t(\{1PB\},P)$$

类似的，可以计算信息集$\{1PB\}$上采取“加注”策略的遗憾值：

$$Regret_A^1(\{1PB\},B)=r_A(\{1PB\},B)=r_A(h,B)=-0.25$$

于是，根据遗憾值匹配算法计算结果，在下一轮的策略中会选择“过牌”策略。

库恩扑克的规则可以扩展到多人的情况下，对于N个玩家，使用牌

值为1, 2, …, N, $N+1$的$N+1$张牌。对于多人的库恩扑克，动作有看牌（k），叫注（b），弃牌（f），跟牌（c）四种动作。

如图8.3所示是当玩家1拿到1、玩家2拿到3、玩家3拿到4时的博弈树，其中黑色的结点为终结局势，下方的括号标明该局势下三位玩家的收益情况。下面来计算图中蓝色框中玩家3选择“叫注（b）”的遗憾值。

仍然假设在游戏开始时刻所有玩家选择的是随机决策行动，在第一轮游戏中，最终到达状态z_1。

当前信息集包含的行动序列$h=\{k \to k\}$产生的概率：

$$\pi^{\sigma}_{\{1,2\}}(h)=0.5\times 0.5=0.25$$

此时玩家1和玩家2均随机采取策略，到达当前信息集的概率由这两名玩家决定。在当前状态下，到达5个可能到达的终结状态的概率分别为：

$$\pi^{\sigma}(h,z_1)=0.5,\pi^{\sigma}(h,z_2)=\pi^{\sigma}(h,z_3)=\pi^{\sigma}(h,z_4)=\pi^{\sigma}(h,z_5)$$
$$=0.5\times 0.5\times 0.5=0.125$$

对玩家3而言，五个终结状态对应的收益为：

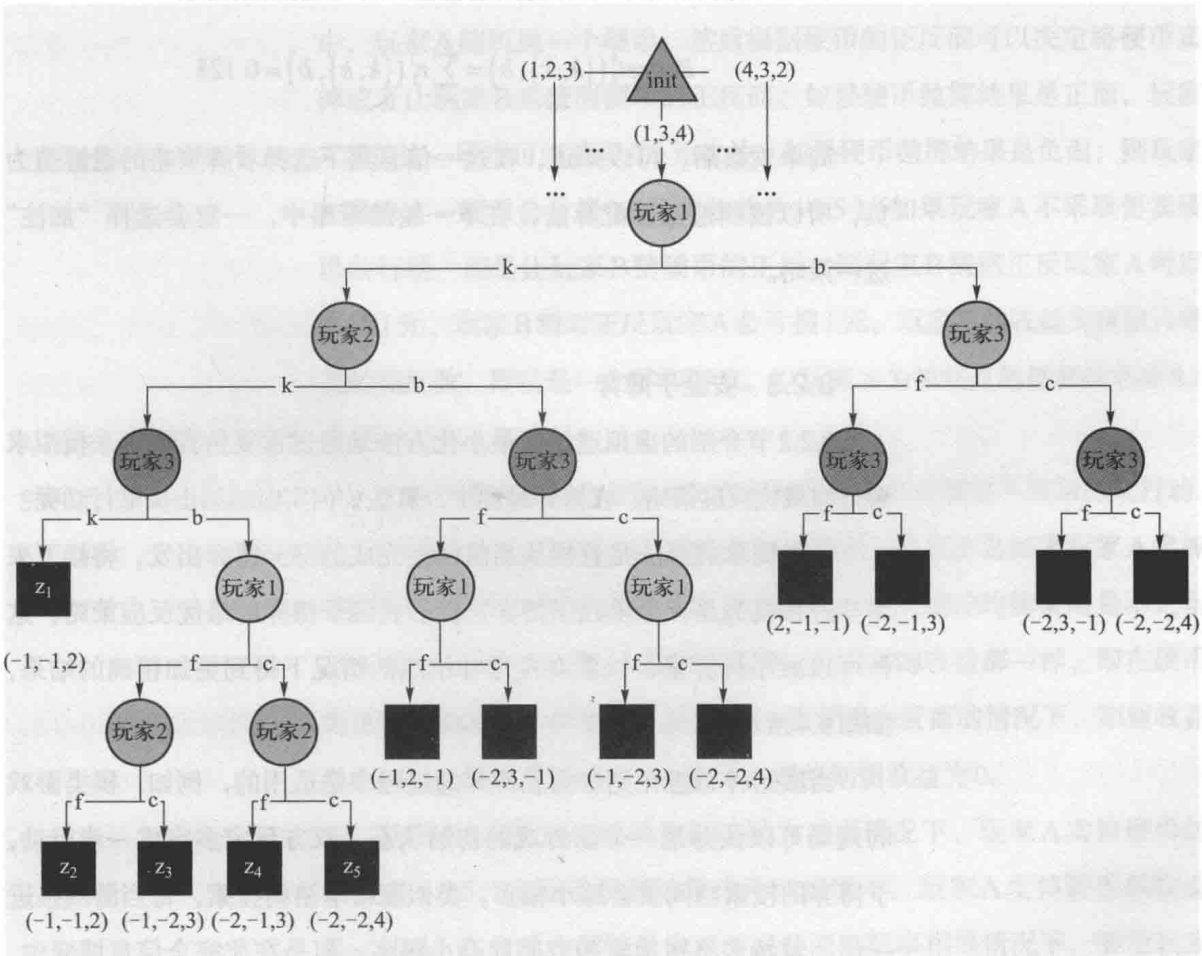


图8.3 三人库恩扑克的部分博弈树

$$u_3(z_1)=2,\ u_3(z_2)=2,\ u_3(z_3)=3,\ u_3(z_4)=3,\ u_3(z_5)=4$$

则可以计算当前策略下行动序列h的虚拟价值：

$$\begin{aligned}v_3(\sigma,h)&=\sum_{z\in\{z_1,z_2,z_3,z_4,z_5\}}\pi_{-3}^{\sigma}(h)\pi^{\sigma}(h,z)u_3(z)\\&=0.25+0.062\,5+0.093\,75+0.093\,75+0.125=0.625\end{aligned}$$

若在这一信息集下一定选择“叫注”的策略，即$\sigma_{\{k,k\}\to b}$，此时最终到达的状态只可能有4个，即$Z=\{z_2,z_3,z_4,z_5\}$，且到达这些终结结点的概率$\pi^{\sigma_{\{k,k\}\to b}}(h,z_2)=\pi^{\sigma_{\{k,k\}\to b}}(h,z_3)=\pi^{\sigma_{\{k,k\}\to b}}(h,z_4)=\pi^{\sigma_{\{k,k\}\to b}}(h,z_5)=1\times0.5\times0.5=0.25$。则在这一策略下，行动序列$h$的虚拟价值为：

$$\begin{aligned}v_3\left(\sigma_{\{k\to k\}\to b},h\right)&=\sum_{z\in\{z_2,z_3,z_4,z_5\}}\pi_{-3}^{\sigma_{\{k\to k\}\to b}}(h)\pi^{\sigma_{\{k\to k\}\to b}}(h,z)u_3(z)\\&=0.125+0.187\,5+0.187\,5+0.25=0.75\end{aligned}$$

于是，此时选择叫注策略的遗憾值计算如下：

$$r_3(\{k,k\},b)=v_3\left(\sigma_{\{k,k\}\to b},h\right)-v_3(\sigma,h)=0.75-0.625=0.125$$

在第一轮博弈中，累加虚拟遗憾值与行为遗憾值相同：

$$Regret_3^1(\{k,k\},b)=\sum_{t=1}^{1}r_3^t(\{k,k\},b)=0.125$$

简单观察后，可以知道，在这一信息集下选择看牌策略的遗憾值为负，所以按照遗憾匹配算法，在下一轮的策略中，一定会选择“加注”这一策略。

8.2.3 安全子博弈

8.2.2节介绍的虚拟遗憾值最小化方法是通过重复仿真博弈来模拟求解近似最优反应策略。在博弈过程中，算法如何实时地给出决策行动呢?

一个简单的想法是直接从当前已经完成的部分博弈出发，将接下来博弈过程视为是一个单独子博弈，然后找到子博弈的最优反应策略，这样可以减小计算量，以便在接近叶结点的情况下得到更加精确的结果，如图8.4所示。

当然这种思想在完全信息博弈的过程中是适用的，例如，棋类游戏的残局可以认为是一个新游戏的初始状态，双方玩家多完成一步行动，子博弈的搜索空间就会缩小很多，类似蒙特卡洛树搜索，每当游戏推进一步，蒙特卡洛树搜索的空间就会小很多。但是在非完全信息博弈中，

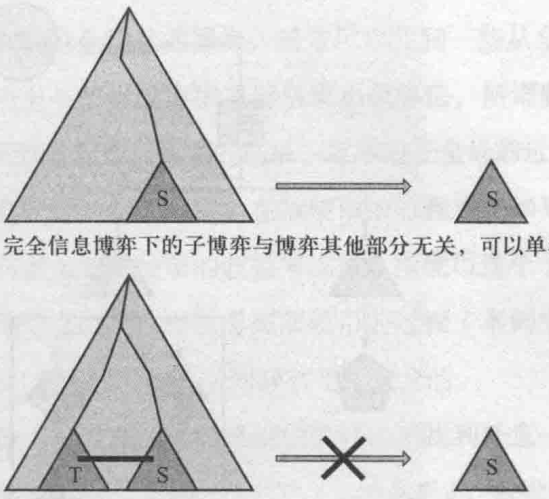


图8.4 完全与非完全信息博弈的子博弈

这种直接搜索子博弈空间的方法并不能生效，即使是博弈树中不能到达的分支，对当前子博弈也是有影响的。

下面以一个简单的抛硬币游戏来给大家说明这一观点。在这个游戏中，玩家A随机抛一个硬币，然后根据硬币的正反面可以决定将硬币卖掉或者让玩家B来猜测硬币的正反面。如果硬币抛掷结果是正面，玩家A卖掉这一硬币可以获得0.5元收益，如果硬币抛掷结果是负面，则玩家A卖掉这一硬币会亏损0.5元（收益为-0.5）。如果玩家A不采取售卖硬币的行动，而是让玩家B猜硬币的正反，则玩家B猜错正反玩家A可以获得1元，玩家B猜对正反玩家A会亏损1元。玩家B的收益为玩家A收益的相反数，即这是一个零和游戏。以玩家A为根结点的博弈树如图8.5所示。

在这个游戏中，由于玩家B在不知道硬币正反情况下采取决策行动，所以对玩家B而言这是一个不完全信息博弈。在玩家B知道玩家A卖掉硬币所获得收益的情况下，对玩家B来说，他的纳什均衡策略是不论玩家A选择卖掉硬币还是让其进行猜测，玩家A的收益都一样，即在硬币为正面的情况下，期望收益总为0.5，在硬币为反面的情况下，期望收益总为-0.5，在这个游戏中，玩家A的最终期望收益为0。

从图8.5可见，在硬币投掷结果为正面情况下，玩家A卖掉硬币的收益$T_1=0.5$，在硬币投掷结果为反面情况下，玩家A卖掉硬币的收益$T_2=-0.5$。在硬币投掷结果为正面或反面的概率相等情况下，要使玩家

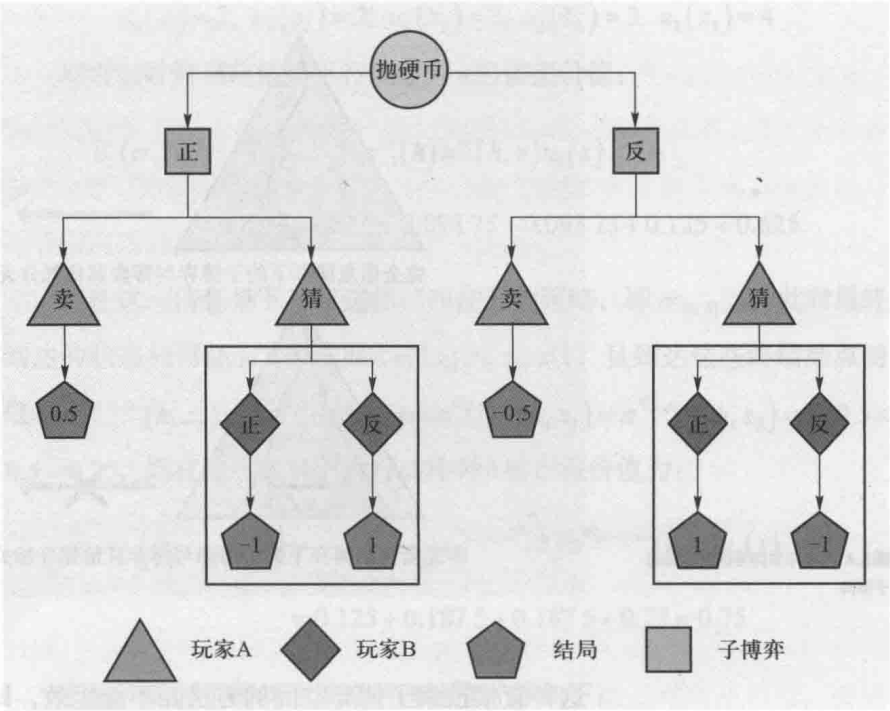


图8.5 抛硬币游戏博弈树

A选择卖硬币和猜硬币的收益一样，则玩家B的策略要使得玩家A在硬币投掷结果为正面情况下选择猜硬币这一行动的期望收益 $T_3=0.5$，在硬币投掷结果为反面情况下选择猜硬币这一行动的期望收益 $T_4=-0.5$。

假设玩家B猜硬币投掷结果为正面的概率为 a，对应的其猜硬币投掷结果为反面的概率为 $(1-a)$，则有：

$$T_3=-a+(1-a)=0.5,\quad T_4=a-(1-a)=-0.5$$

从而可求得玩家B猜测硬币投掷结果为正面的概率为0.25，猜测硬币投掷结果为反面的概率为0.75。在玩家B按照这样的概率来猜测硬币投掷结果为正面或反面的情况下，将使得玩家A的最终期望收益为0。

在这场博弈中，即使玩家A选择了让玩家B来猜测硬币投掷结果的正反，玩家B也必须根据玩家A选择卖掉硬币的期望收益来计算自己纳什均衡的策略。也就是说，玩家B的“子博弈”还与图8.5所示博弈树中“卖”的分支相关。

假设玩家A在硬币投掷结果为正面情况下卖掉硬币的收益 $T_1=-0.5$，在硬币投掷结果为反面情况下卖掉硬币的收益 $T_2=0.5$，那么类似地可以求出玩家B的纳什均衡反应策略应该是以0.25概率来猜测硬币投掷结果是反面，以0.75概率来猜测硬币投掷结果是正面。如果单独考虑猜硬币的情况，而不考虑卖硬币的情况，在正面和反面出现概率相同的情况下，

玩家B应该以0.5的概率猜测正面，以0.5的概率猜测反面。

对于一个非完全信息的博弈，通常可以找到一些从全局出发的近似解法，如使用上一节中的虚拟遗憾值最小化算法，所谓**安全子博弈**是指在子博弈的求解过程中，得到的结果一定不差于全局的近似解法。

在上面抛硬币的例子当中，在猜硬币的过程中，如果玩家B的行动是通过预测玩家A卖掉硬币的收益来决定，猜硬币这个子博弈就是安全的。反之如果玩家B的行动仅考虑猜硬币的过程（本例中为不考虑A买硬币的收益），则猜硬币这个子博弈就是不安全的。

更一般地，可以通过虚拟价值的估计，来找到任意一个博弈的安全子博弈。给定双人博弈中博弈树中的一个信息集I_1，假设玩家1在信息集I_1'采取行动a'可到达信息集I_1。如果在信息集I_1'采取行动a^*进入另外一个信息集可得到更大的价值，则意味着玩家1所采取的行动只获得了局部最优解（在信息集I_1）。可将采取行动a^*和行动a'所分别获得价值之间的差距作为信息集I_1的奖励，将这一奖励增加到信息集I_1的价值评估中。已经证明，只要信息集I_1的奖励足够小，那么信息集I_1所对应的子博弈就是安全的[17]。行动a^*和行动a'价值的差距如何评估呢？可以通过信息集I_1和信息集I_1'虚拟价值的下限来决定。

8.3 博弈规则设计

8.3.1 研究问题

在现实生活中，如果所有博弈者都追求自己利益最大化，很可能会导致两败俱伤的下场，那么应该如何设计博弈的规则使得博弈的最终局势能尽可能达到整体利益的最大化呢？

例如，许多城市为了避免机动车过快增长造成城市拥堵，因此限制了车牌的发放量。这样，车牌的发放方式就是需要决策者经过周密的考虑后决定的。通常而言，车牌的发放既要能满足有紧迫需求的人，又要满足普通家庭的日常需求，所以很多城市都采取了车牌竞价和随机摇号两种方式发放车牌。

利用人工智能的算法和思想可以为许多博弈规则设计问题提供帮助，如拍卖竞价、供需匹配、名额分配等。这些规则的设计过程中往往面临

着计算量大，复杂度高等问题，为复杂的问题寻找有效的算法就是博弈规则设计的主要研究问题。

8.3.2 双边匹配算法

在生活中，人们常常会碰到与资源匹配相关的决策问题（如求职就业、报考录取等），这些需要双向选择的情况被称为是**双边匹配问题**。在双边匹配问题中，需要双方互相满足对方的需求才会达成匹配。

稳定的匹配是指没有任何人能从偏离稳定状态中获益。如果将匹配问题看作是一种合作博弈的话，那么稳定状态就是这个博弈问题的纳什均衡解。

稳定婚姻问题（stable marriage problem）就是一个典型的双边匹配问题，这个问题是指在给定成员偏好顺序的情况下，为两组成员寻找稳定的匹配。假设有n个单身男性构成的集合$M=\{m_1,m_2,\cdots,m_n\}$，以及n个单身女性构成的集合$F=\{f_1,f_2,\cdots,f_n\}$。对于任意一名单身男性m_i，都有自己爱慕的单身女性的顺序$s_{m_i}:=f_{m_{i,1}}\succ f_{m_{i,2}}\succ\cdots\succ f_{m_{i,n}}$，这里$f_{m_{i,j}}$表示第$i$名男性所喜欢单身女性中排在第$j$位的单身女性，同理对于任意一名单身女性$f_i$也有其爱慕的单身男性顺序$s_{f_i}:=m_{f_{i,1}}\succ m_{f_{i,2}}\succ\cdots\succ m_{f_{i,n}}$，$m_{f_{i,j}}$表示第$i$名女性所喜欢单身男性中排在第$j$位的单身男性。算法的最终目标是为这$2n$个男士和女士匹配得到$n$对伴侣，每一对伴侣可以表示为$(m_i,f_j)$。

在这种匹配的过程中并不是简单的价高者得，而是要充分考虑双方的意愿。对于两对潜在的匹配伴侣(m_i,f_j)和(m_l,f_k)，如果s_{m_i}中有$f_k\succ f_j$，且s_{f_k}中有$m_i\succ m_l$，即对第i个男性来说，相比于第j个女性，他更喜欢第k个女性。同时，对第k个女性来说，她更喜欢第i个男性胜过第l个男性。这样(m_i,f_j)和(m_l,f_k)就都不是稳定的匹配，因为m_i和f_k都会去选择其更希望的匹配。

稳定婚姻问题的稳定解是指不存在未达成匹配的两个人都更倾向于选择对方而不是自己当前的匹配对象。反过来讲，不稳定婚姻匹配是指存在不是伴侣的一男和一女，彼此可接受对方，而且其要么单身，要么对目前自己的伴侣的好感度没有对方高。

对于这个问题可能大家会立刻想到一种“修补”策略的匹配方式，即先对所有人按照一男一女随机匹配，如果出现不稳定的匹配，则马上

重新匹配，没有匹配在一起的两人重新组成伴侣，最后直到不存在不稳定的匹配。表面上看，这种“修补”策略一定能得到稳定的匹配结果，但是在“修补”的过程中很容易陷入死循环，假设有4名单身男性{1, 2, 3, 4}和4名单身女性{*A*, *B*, *C*, *D*}，他（她）们的爱慕序列如表8.9所示。

表8.9　稳定婚姻匹配偏好序列

男性	偏好	女性	偏好
1	$A \succ D \succ C \succ B$	***A***	$3 \succ 4 \succ 2 \succ 1$
2	$A \succ B \succ C \succ D$	***B***	$3 \succ 2 \succ 4 \succ 1$
3	$A \succ C \succ D \succ B$	***C***	$1 \succ 3 \succ 4 \succ 2$
4	$B \succ A \succ D \succ C$	***D***	$2 \succ 4 \succ 3 \succ 1$

按照“修补”策略，匹配和修补过程如表8.10所示。

表8.10　修补策略下的稳定婚姻匹配过程

匹配1	修补	匹配2	修补	匹配3	修补	匹配4	修补	匹配5
(1, *A*)	(2, *A*)	(1, *B*)	(4, *A*)	(1, *B*)	(4, *B*)	(1, *A*)	(2, *B*)	(1, *A*)
(2, *B*)		(2, *A*)		(2, *D*)		(2, *D*)		(2, *B*)
(3, *C*)		(3, *C*)		(3, *C*)		(3, *C*)		(3, *C*)
(4, *D*)		(4, *D*)		(4, *A*)		(4, *B*)		(4, *D*)

可以看出，最终的匹配结果与初始结果相同，算法陷入死循环。

1962年，美国数学家大卫·盖尔和博弈论学家沙普利提出了针对双边稳定匹配问题的解决算法（也被称为Gale-Shapely算法或G-S算法），并将其应用于稳定婚姻问题的求解[10]，算法过程如下。

（1）单身男性向最喜欢的女性表白。

（2）所有收到表白的女性从向其表白男性中选择最喜欢的男性，暂时匹配。

（3）未匹配的男性继续向没有拒绝过他的女性表白。收到表白的女性如果没有完成匹配，则从这一批表白者中选择最喜欢男性。即使收到表白的女性已经完成匹配，但是如果她认为有她更喜欢的男性，则可以拒绝之前的匹配者，重新匹配。

（4）如此循环迭代，直到所有人都成功匹配为止。

这一过程中，男生使用贪心策略表白，而女生具有选择权，一旦出现不稳定的匹配，即替换当前匹配。

对于表8.9中的例子，匹配过程如表8.11所示。

表8.11 G-S算法下的稳定婚姻匹配过程

第一轮		第二轮		第三轮		第四轮		第五轮	
表白	选择	表白	选择	表白	选择	表白	选择	表白	选择
(1, *A*)	\	(1, *D*)	(1, *D*)	\	(1, *D*)	\	\	(1, *C*)	(1, *C*)
(2, *A*)	\	(2, *B*)	(2, *B*)	\	(2, *B*)	\	(2, *B*)	\	(2, *B*)
(3, *A*)	(3, *A*)	\	(3, *A*)	\	(3, *A*)	\	(3, *A*)	\	(3, *A*)
(4, *B*)	(4, *B*)	\	\	(4, *A*)	\	(4, *D*)	(4, *D*)	\	(4, *D*)

在第一轮中，4名男性分别向自己最喜欢的女性表白，而收到3人表白的女性*A*选择了自己最喜欢的男性3，另一个收到表白的女性*B*选择了男性4；在第二轮中，尚未匹配的男性1和男性2继续向自己第二喜欢的对象表白，收到表白的女性*B*选择了自己更喜欢的男性2而放弃了男性4，同理继续三轮表白和选择，所有人都找到了自己的伴侣，且所有匹配都是稳定的。可以看出，使用G-S算法得到了稳定匹配的结果。

8.3.3 单边匹配算法

在匹配问题中，除了需要双向选择的双边匹配问题，还有一种类似于以物易物方式的交换匹配问题，被称为**单边匹配问题**，如室友的匹配或者座位的分配。这些问题中分配的对象都是不可分的标的物，他们只能属于一个所有者，且可以属于任何一个所有者。

对于这种单边匹配问题，1974年，沙普利和斯卡夫提出了针对单边匹配问题的稳定匹配算法“最大交易圈算法（Top-trading cycle，TTC）”[11]，算法过程如下。

（1）首先记录每个标的物的初始占有者，或者对物品进行随机分配。

（2）每个交易者连接一条指向他最喜欢的标的物的边，并从每一个标的物连接到其占有者或是具有高优先权的交易者。

（3）此时形成一张有向图，且必存在环，这种环被称为“交易圈”，对于交易圈中的交易者，将每人指向结点所代表的标的物赋予交易者，同时交易者放弃原先占有的标的物，占有者和匹配成功的标的物离开匹配市场。

（4）接着从剩余的交易者和标的物之间重复进行交易圈匹配，直到无法形成交易圈，算法停止。

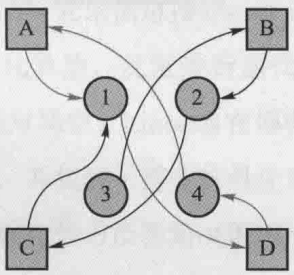
稳定室友匹配问题就是一个典型的单边匹配问题。假设某寝室有A、B、C、D四位同学和1、2、3、4四个床位，当前给A、B、C、D四位同学随机分配4、3、2、1四个床位。已知四个同学对床位的偏好顺序如表8.12所示。

表8.12　床位偏好顺序

同学	偏好
A	$1 \succ 2 \succ 3 \succ 4$
B	$2 \succ 1 \succ 4 \succ 3$
C	$1 \succ 2 \succ 4 \succ 3$
D	$4 \succ 3 \succ 1 \succ 2$

图8.6 第一轮单边匹配图

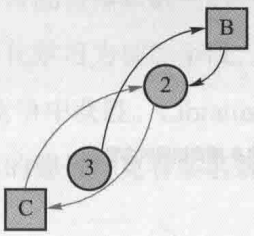
依照算法所述，在第一轮匹配中可以得到一个标的物和分配者的有向图，如图8.6所示。



从图8.6可以看出，A和D之间构成一个交易圈，可达成交易，所以A得到床位1，D得到床位4，之后将A和D以及1和4从匹配图中移除。

图8.7 第二轮单边匹配图

在第二轮的匹配中，一样可以得到一个如图8.7的有向图。



从图8.7可以看出，B和C都希望得到床位2，无法再构成交易圈，但是由于C是床位的本身拥有者，所以C仍然得到床位2，B只能选择床位3。

最后交易结果A→1，B→3，C→2，D→4。

8.4　非完全信息博弈的实际应用

前面介绍了博弈论以及人工智能博弈的基本内容，如图8.8所示，其中策略求解中的虚拟遗憾值最小化算法和安全子博弈模型主要针对的是非完全信息的博弈，而完全信息的博弈可以使用在前面章节中学过的搜索策略来寻找纳什均衡解。相对于完全信息的博弈，不完全信息下的博弈更加符合实际的应用场景，能够从棋牌游戏走向人们的生活，指导

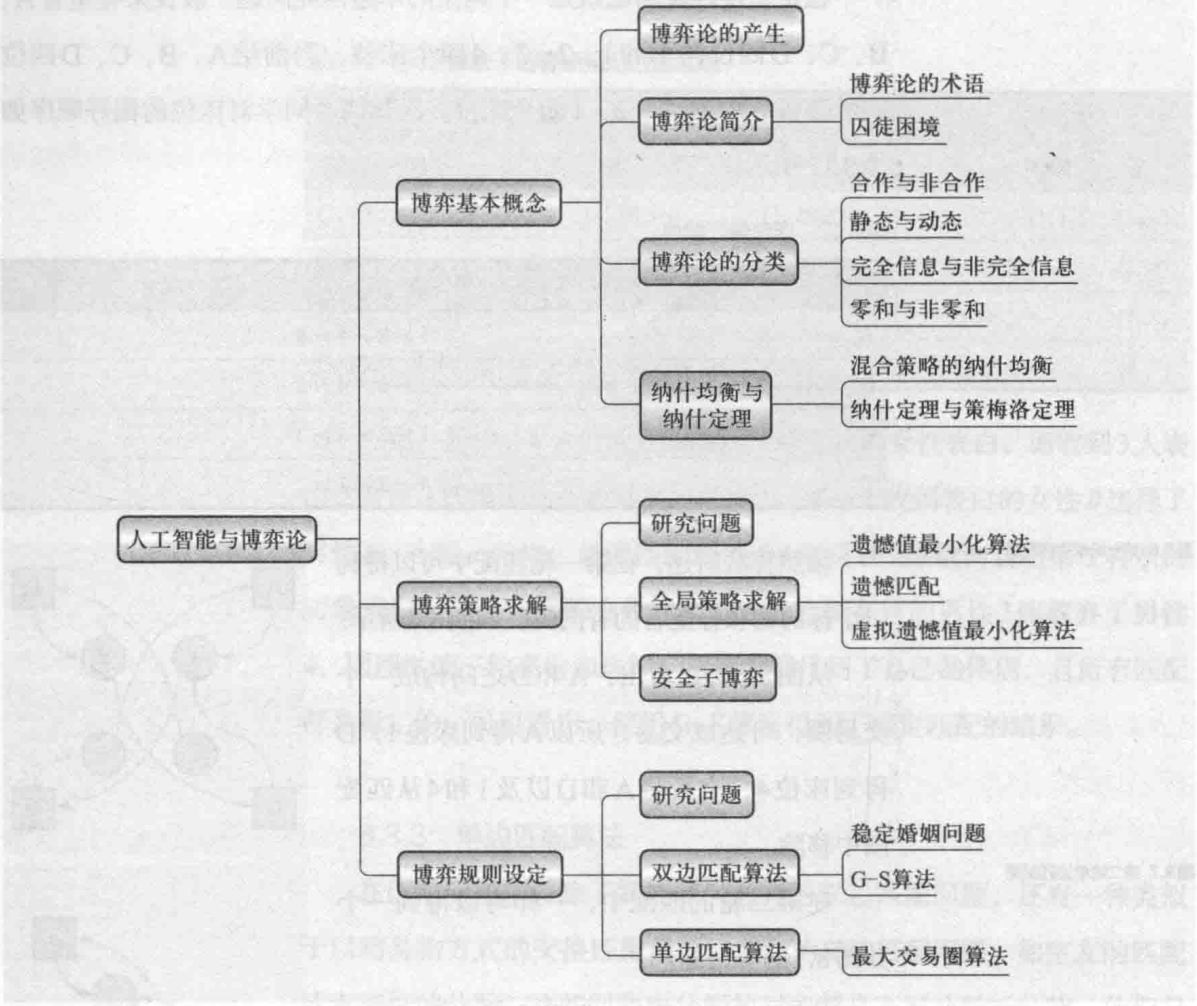


图8.8 博弈论研究内容

人们的科学决策。

在非完全信息的纳什均衡求解中，博弈树通常规模非常大，为了削减搜索空间，人们需要使用一些抽象算法对原有博弈问题进行压缩，即合并相关博弈树状态、压缩搜索层数以及剪枝等，最终得到一个相对简单的抽象问题，然后求解这个抽象问题的纳什均衡，最后将所求解结果映射回原问题当中，于是得到原问题的纳什均衡解[12]。图8.9给出了这一过程的示意图。

此外，深度强化学习越来越多地被应用于博弈的策略求解。强化学习的主要思想是通过试错和搜索来优化自身的行动[13]，这一过程可以直接被应用于博弈的决策中。在连续动态的环境中，通过强化学习的训练，可以让参与博弈的AI智能体在不同情况下能够找到可获得最大收益的最佳策略[14]，在许多游戏AI中，都运用了这种方法来进行对战，并最终战胜人类玩家[15, 16]。

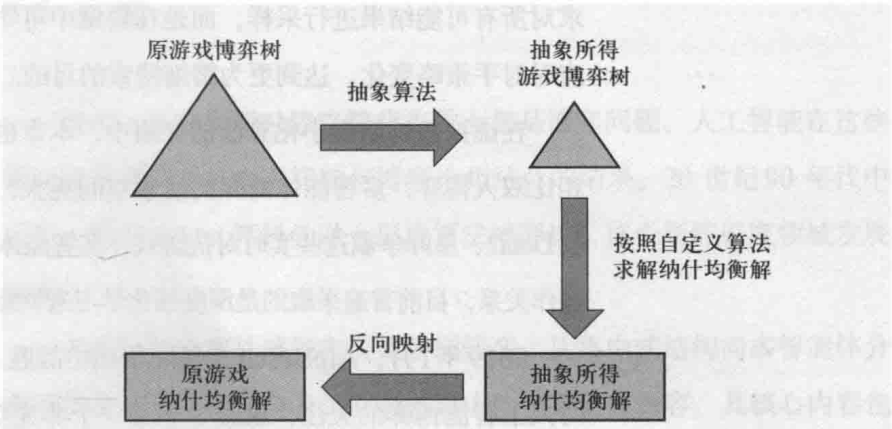


图8.9 求解非完全信息博弈纳什均衡的一般方法

近年来，非完全信息下的德州扑克智能系统Libratus在与人类选手对弈比赛中屡战屡捷，德州扑克就是一种典型的非完全信息博弈。在进行游戏前，Libratus首先通过虚拟遗憾值最小化算法，反复进行迭代，进行自我博弈，锻炼进行游戏的能力。在实际对局中，Libratus在博弈中不断寻找安全子博弈，进一步缩小搜索空间，并找到子博弈的最佳策略。在每局比赛结束后，将游戏最终结果通过强化学习反馈给决策网络以进一步优化参数[17]。

除了双人博弈，多智能体博弈也是目前人工智能与博弈研究的一个热点，多智能体博弈通常使用的是多智能体的强化学习为训练手段，能够同时操控多个玩家完成合作或者在多玩家的竞争中获胜。Libratus最初的应用就是双人无限注的德州扑克，但是实际的德州扑克有多名玩家参加的。

最近，Libratus被拓展成德州扑克多人博弈系统Pluribus，Pluribus在六人无限注德州扑克的较为简单场景下击败人类专业选手[1]。在训练中，Pluribus 采用了蒙特卡洛虚拟遗憾最小化算法（Monte Carlo counterfactual regret minimization，MCCFR）。MCCFR随机考虑一部分行动，来选择应该采取的决定。在每一次迭代中，Pluribus根据在场玩家已经展示出来的策略来模拟一盘游戏，然后在模拟游戏中寻找最适合自己的最优策略。每一回合，Pluribus都会加入一个虚拟遗憾值，使它会后悔上次没有用其他更好的策略，以保证Pluribus在下一轮倾向于选择上次后悔没选的策略。Pluribus就这样在“吾日三省吾身”中不断提高自己的水平。在实际对决了，为了应对对手可能会改变策略这一情况，Pluribus采用了有限前瞻搜索（Depth-limited search）方法，不苛

求对所有可能结果进行采样，而是在搜索中可“截断搜索”，以自适应来应对对手策略变化，达到更为精细搜索的目的。

在虚拟遗憾值最小化算法的介绍中，本章也举出了多人博弈的示例。相比双人博弈，多智能体博弈的搜索空间更大，在自由性更高的游戏中，如Dota2、星际争霸这些实时对抗游戏，多智能体之间同时存在竞争关系和合作关系，目前普遍采取的是深度强化学习来训练多智能体进行策略求解。

2019年1月，AlphaStar在星际争霸中战胜人类玩家，也引起了人们对人工智能博弈的关注，星际争霸是一个非完全信息的博弈，在游戏中玩家不能直接看到所有的信息，而是需要通过侦查获取，同时每局游戏持续的时间很长，较早做出的决策可能会影响到后期的游戏局势，当然，作为一款玩家实时对抗游戏，所有的行为都没有轮次限制而是各自实时决策，最后星际争霸的行动空间非常非常大，需要操控的智能体也非常多。AlphaStar的决策是直接将所有需要执行操作的智能体及其属性输入深度神经网络，输出一系列可执行的指令。在训练时，AlphaStar首先通过监督学习模仿人类选手的操作，这一过程中AlphaStar学到了一些宏观的游戏策略以及基础的微观操作，接着同时使用多个模型进行演化学习，即在多个模型相互博弈的过程中优胜劣汰，这一过程训练使用强化学习的方法，不断探索广阔的策略空间，得到一些更强更有效的策略，并最终达到近似的纳什均衡解[16]。

在非完全信息的机制设计中，一个典型的问题就是买卖双方在信息不对称的情况下，卖方应该如何进行销售，这同时也是人工智能与经济学交叉的一个研究热点。基于强化学习的优化机制设计就是一种利用人工智能的设计方法，例如，对于稀缺资源的分配问题，就可以使用强化学习来解决[19]。此外，使用人工智能的方法，如决策树、线性模型等，也可以对经济行为进行建模，并通过求解优化得到较好的博弈机制。例如，美国医疗中使用的肾交换系统，除了使用最大交易圈算法，还结合了许多分类模型，自动化地完成最优匹配[20]。

由于现实的决策过程通常是非完全信息的，所以利用人工智能的技术对非完全信息博弈进行深入研究是具有重要意义的，博弈论的思想也不断对人工智能的发展产生积极的影响。

8.5 小结

围棋、扑克和即时战略游戏本质上都是博弈问题。人工智能在这些领域取得进展正是人工智能与博弈论相结合的结果。20 世纪90 年代中后期，博弈论和计算科学融合形成算法博弈论，这个新的研究领域发展迅速。

人工智能正在从感知向决策智能转变，从集中式结构向多智能体分布式结构演进，这使得人工智能博弈成为重要研究内容，其核心内容包括算法博弈论及计算经济学、算法与机器学习相结合的机器学习理论、博弈论与机器学习相结合的多智能体学习等。

习题8

1. 标志着现代博弈理论初步形成的事件是（　　）。
   A. 1944年冯·诺依曼与奥斯卡·摩根斯特恩合著《博弈论与经济行为》的出版
   B. 纳什均衡思想的提出
   C. 囚徒困境思想的提出
   D. 冯·诺依曼计算机的实现
2. 下面对博弈研究分类不正确的是（　　）。
   A. 合作博弈与非合作博弈
   B. 静态博弈与动态博弈
   C. 完全信息博弈与不完全信息博弈
   D. 囚徒困境与纳什均衡
3. 下面对纳什均衡描述正确的是（　　）。
   A. 参与者所作出的这样一种策略组合，在该策略组合上，任何参与者单独改变策略都不会得到好处
   B. 在一个策略组合上，当所有其他人都改变策略时，也无法破坏先前的博弈平衡，则该策略组合就是一个纳什均衡
   C. 参与者所作出的这样一种策略组合，在该策略组合上，若干参与者改变策略后，大家都不会得到更多好处
   D. 参与者所作出的这样一种策略组合，在该策略组合上，有且只有1个参与者改变策略后，其收益会增加
4. 下面对混合策略纳什均衡描述正确的是（　　）。
   A. 博弈过程中，博弈方通过概率形式随机从可选策略中选择一个策略而达到的纳什均衡被称为混合策略纳什均衡
   B. 博弈过程中，博弈方通过非概率形式随机从可选策略中选择一个策略而达到的纳什均衡被称为混合策略纳什均衡
   C. 博弈过程中，博弈方以概率形式随机从可选收益中选择一

个收益，而达到的纳什均衡被称为混合策略纳什均衡

D. 博弈过程中，博弈方以非概率形式随机从可选收益中选择一个收益，而达到的纳什均衡被称为混合策略纳什均衡

5. 在遗憾最小化算法中，玩家i按照如下方法来计算其在每一轮产生的悔恨值：（　　）。

A. 其他玩家策略不变，只改变玩家i的策略后，所产生的收益之差

B. 所有玩家策略均改变，所产生的收益之差

C. 至少改变1个以上玩家的策略，所产生的收益之差

D. 每个玩家策略不变，只改变收益函数，所产生的收益之差

6. 下面关于子博弈说法正确的是（　　）。

A. 在求解博弈的纳什均衡过程中，安全子博弈的均衡解一定是全局的均衡解

B. 在求解博弈的纳什均衡过程中，安全子博弈的均衡解一定不差于全局均衡解

C. 完全信息博弈中的安全子博弈只包含当前节点的后继节点

D. 非完全信息博弈中的安全子博弈不一定包含当前节点的后继节点

7. 对于一个博弈问题，可以同时存在多个纳什均衡解。假设有甲和乙两名猎手一起去打猎，打猎目标可以选择猎兔、也可以选择猎鹿。表8.13中给出了在狩猎博弈中，甲和乙两名猎手获得收益的情况。由于每位猎人可以单独猎兔，而猎鹿需要两位猎手合作才能完成，因此猎兔获得收益较小，猎鹿获得收益较大。

表8.13　猎鹿博弈中采取行动及获得的收益

	甲猎鹿	甲猎兔
乙猎鹿	两人收益均为5	甲收益为3，乙收益为0
乙猎兔	甲收益为0，乙收益为3	两人收益均为3

根据表8.13找出不同的纳什均衡解，分析不同纳什均衡中所有参与者收益总和是否相等。将表8.13例子与囚徒困境例子相结合，说明纳什均衡解不一定是最优解。

8. 在本章8.2.2小节中，用石头-布-剪刀的游戏介绍了遗憾最小化算法和遗憾匹配，尝试计算在二轮中，假设玩家A在第二局按照遗憾值最大的策略选择“剪刀”，玩家B随机选择“布”，游戏结束后，玩家B不同行动的遗憾值以及在第三轮中可能采取的每种行动的概率。

9. 在本章8.2.2小节中，用双人库恩扑克的游戏介绍了虚拟遗憾值最小化算法和遗憾匹配，尝试计算在所有玩家都随机选择行动的状态下，玩家A的手牌为2时，通过路径 $2 \xrightarrow{P} 2P \xrightarrow{B} 2PB$ 到达节点 $\{2PB\}$ 后选择“加注”行动的虚拟遗憾值。

*10. 非完全信息博弈是生活中更常见的一种博弈方式，能否在生活中找到应用非完全信息的安全子博弈思想的案例呢？

11. 使用G-S匹配算法对表8.14所示偏好的男女进行匹配。稳定匹配结果是否唯一存在？使用G-S匹配得到的结果有什么特点？

表8.14 稳定婚姻匹配偏好序列

男性	偏好	女性	偏好
1	$B \succ A \succ C$	*A*	$2 \succ 1 \succ 3$
2	$C \succ B \succ A$	*B*	$3 \succ 2 \succ 1$
3	$A \succ C \succ B$	*C*	$1 \succ 3 \succ 2$

12. 假设某医院有七名医生负责一周的夜班值守，每人值一天夜班，但是不同医生希望值夜班的偏好有所不同（见表8.15）。

表8.15 值班日偏好顺序

医生	偏好
1	$1 \succ 2 \succ 3 \succ 4 \succ 5 \succ 6 \succ 7$
2	$5 \succ 4 \succ 3 \succ 2 \succ 1 \succ 7 \succ 6$
3	$6 \succ 1 \succ 2 \succ 4 \succ 3 \succ 7 \succ 5$
4	$2 \succ 3 \succ 1 \succ 4 \succ 5 \succ 6 \succ 7$
5	$3 \succ 4 \succ 5 \succ 2 \succ 1 \succ 7 \succ 6$
6	$6 \succ 2 \succ 4 \succ 3 \succ 5 \succ 1 \succ 7$
7	$4 \succ 2 \succ 3 \succ 1 \succ 5 \succ 7 \succ 6$

请根据值班日偏好合理安排值班表。

参考文献

[1] BROWN N, Sandholm T. Superhuman AI for multiplayer poker [J]. Science, 2019.

[2] NASH J F. Equilibrium points in n-person games [J]. Proceedings of the National Academy of Sciences of the United States of America, 36(1): 48-49, 1950.

[3] NASH J F. Non-Cooperative Games [D]. Princeton: Princeton University, 1950.

[4] NEUMANN J, MORGENSTERN O. Theory of Games and Economic Behavior [M]. Princeton: Princeton University Press, 1944.

[5] SCHWALBE U, WALKER P. Zermelo and the early history of game theory [J].Games and economic behavior, 34(1): 123-137, 2001.

[6] GREENWALD A, LI Z, MARKS C. Boundsfor Regret-Matching Algorithms [C] //International Symposium on Artificial Intelligence & Mathematics. [S.l.]: [s.n.], 2003.

[7] ZINKEVICH M, et al.Regret minimization in games with incomplete information [J].Advances in neural information processing systems. 2008.

[8] KUHNH W. A simplified two-person poker [J].

Contributions to the Theory of Games 1: 97–103, 1950.
[9] BROWN N, SANDHOLM T. Safe and nested subgame solving for imperfect-information games [C] //Proceedings of Advances in Neural Information Processing Systems 30 (NIPS 2017). [S.l.] :[s.n.], 2017.
[10] GALE D, SHAPLEY L.College admissions and the stability of marriage [J] .The American Mathematical Monthly, 69.1: 9–15, 1962.
[11] SHAPLEY L, HERBERT S.On cores and indivisibility [J] . Journal of mathematical economics, 1(1): 23–37, 1974.
[12] SANDHOLM T. Solving imperfect-information games [J] . Science, 347(6218): 122–123, 2015.
[13] SUTTONR S, BARTOA G. Reinforcement learning: An introduction [M] . Boston: MIT press, 2018.
[14] RACANIERE S, et al. Imagination-augmented agents for deep reinforcement learning [J] .Advances in neural information processing systems, 2017.
[15] JADERBERG M, et al.Human-level performance in first-person multiplayer games with population-based deep reinforcement learning [J] . arXiv preprint arXiv: 1807.01281, 2018.
[16] ARULKUMARAN K, ANTOINE C, JULIAN T. AlphaStar: An Evolutionary Computation Perspective [J] . arXiv preprint arXiv: 1902.01724, 2019.
[17] BROWN N, TUOMAS S. Superhuman AI for heads-up no-limit poker: Libratus beats top professionals [J] . Science, 359(6374): 418–424, 2018.
[18] TANG P. Reinforcement mechanism design [C] // Twenty-Sixth International Joint Conference on Artificial Intelligence. [S.l.] :[s.n.], 2017.
[19] PEROLAT J, et al.A multi-agent reinforcement learning model of common-pool resource appropriation [J] . Advances in Neural Information Processing Systems, 2017.
[20] GENTRY S E, SEGEV D L, MONTGOMERY R A.A comparison of populations served by kidney paired donation and list paired donation [J] . American Journal of Transplantation 5(8): 1914–1921, 2005.

第9章 人工智能未来发展和趋势

9

凡是过往，皆为序章
——莎士比亚

人工智能经过60多年演进，正呈现深度学习。跨界融合、人机协同、群智开放、自主操控等新特征。人工智能具有辐射效应、放大效应和溢出效应，正在引发链式突破，加速新一轮科技革命和产业变革进程，成为新一轮产业变革的核心驱动力。

当前，人工智能有如下五个重要特点，推动其迈向新的发展阶段[8][9]：从人工知识表达技术到大数据驱动知识学习；从处理类型单一的数据到跨媒体认知、学习和推理；从追求“机器智能”到迈向人机混合的增强智能；从聚焦研究“个体智能”到基于互联网络的群体智能；从机器人到自主无人系统。人工智能作为一种使能技术，正在特定领域取得了重要进展，目前正处于感知智能适应性差、认知机理不明、通用智能发展乏力的关键阶段。

本书前面章节主要从逻辑推理和数据智能角度对人工智能模型和算法进行了介绍。“白发如新、倾盖如故”，对已有知识的学习以及对未来知识的学习都会激发学习热情，本章主要简略讨论若干人工智能发展的热点，这些热点从不同角度推动从弱人工智能（领域相关人工智能）向强人工智能（通用人工智能）跨越、推动从“赋能”到“用能”的转变，保障人工智能的健康发展[10]。

9.1 类脑计算

理解人类认知并建立可计算认知模型，需要厘清从行为到神经系统和回路再到细胞核分子不同层面的因果关联，因此理解人类认知的神经基础是促进人工智能取得长足发展一个很有吸引力但遥远的目标[1]。从对大脑观测理解中，抽取对人工智能有启发性的内容，为脑启发计算或生物计算带来启示，是目前脑科学与人工智能交叉一个活跃方向。

人类大脑具有感知、识别、学习、联想、记忆和推理等功能，并非全部用符号计算形式来实现。这些功能与大脑结构存在着对应关系，并且大脑的神经网络系统具有多层反馈机制，如来自于高级“控制”脑区到初级视觉脑区的反馈信号，形成了基于内容和语义的视觉“选择性注意”机制。类脑计算就是受上述脑功能和脑神经网络连接机制启发的一种计算架构，它以神经形态计算的模式来部分模拟大脑功能与其结构的对应关系和反馈连接，增强人工智能及其计算效率，不完全依赖现有冯·诺依曼计算结构，也不是复制人类大脑或简单地建造一种模拟神经元功能的芯片，更不是去完全替代冯·诺依曼计算结构。然而，至今对人类认知功能如何从复杂动态（时空演变）的大脑神经结构中产生，依然没有形成较为完整的认识。

与计算机执行常规程序中遇到错误会崩溃不同，人脑能够应对不确定性、对错误具有一定的容错机制（fault tolerance），具备对未知领域进

行建模和学习的能力。在认知学领域有一个具有争议的理论，认为感知、运动控制、记忆和推理优化等大脑功能，都通过大脑将“现有经验”和“未来期望”进行比较而进行。也就是说，大脑在所接受外界现有信息基础上主动构建模型，计算所构建模型与所期望“假设空间（hypothesis space）”之间的概率，并加以调整优化，即比较“现有经验”和“未来期望”来进行决策。

人类大脑可自我学习和进化，具有适应性、有常识性知识和直觉能力等特征，机器智能是完成人类事先设计的任务，具有按部就班执行、无法拥有常识等特征。这些区别的存在使得人脑呈现“小数据、大任务”的特点，即人脑仅通过小部分数据就可习得某种能力，机器智能则呈现“大数据、小任务”的特点，即需要海量数据来强化任务学习。例如，AlphaGo Zero在“集合封闭、规则完备、约束有限”的场景下，经过40天训练后，总计运行约2900万次自我对弈，才以89∶11击败了所有的AlphaGo程序，一个职业围棋选手一辈子下棋的局数仅为千余。从这里可看出，机器智能需要海量数据才能达到人脑依靠小数据达到的水平。

“见一叶落，而知岁之将暮；审堂下之阴，而知日月之行，阴阳之变；见瓶水之冰，而知天下之寒，鱼鳖之藏也”，人类大脑能在“落叶”和“岁暮”、“冰”和“寒”与“藏”之间游刃有余进行推理和联想，展现人类思维的无穷魅力，与机器智能具有较大差别。

从现有理论和模型来看，机器智能很难在归纳、演绎、联想等智能活动方面展示出“类人”智能。

9.2 自动化机器学习

深度学习目前在诸多领域取得了显著进展，但是设计一个特定任务的深度模型困难重重，因此在实践中只能依赖“炼金术”来经验式调参，因此神经网络结构的自动化学习和参数优化成为研究热点，自动化机器学习（auto machine learning，AutoML）研究应运而生。

自动化机器学习包括神经网络结构的搜索（neural network architecture search，NAS）、超参数优化（hyperparameter optimization）以及元学习（meta-learning）三个主要方面。其中超参数优化主要是选

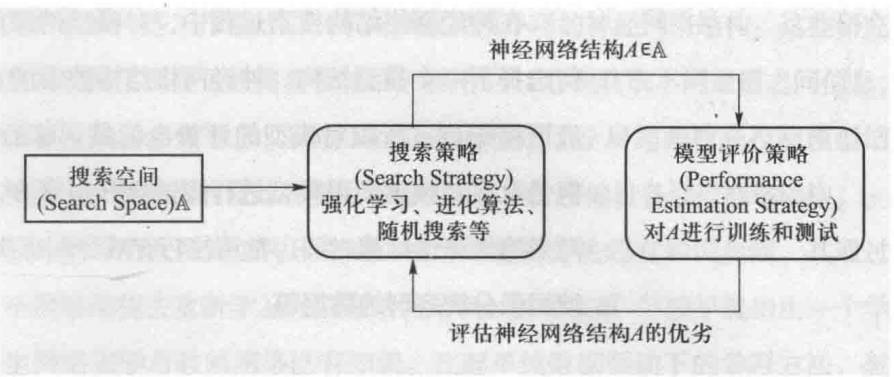


图9.1 神经网络结构搜索过程

择机器学习模型中效果最好的超参数，元学习主要是找到针对特定问题最合适的机器学习模型或算法，神经网络结构搜索主要是针对特定的任务找到最合适的深度学习模型结构。在深度学习领域的神经网络结构搜索其实包含了超参数优化以及元学习的任务。

神经网络结构搜索的一般搜索过程如图9.1所示[11]。在图9.1中，算法利用搜索策略从搜索空间中选取一种神经网络结构，通过模型评价策略来评估这个网络结构的优劣，搜索策略根据评估结果，完成模型设计或继续搜索其他结构。如此迭代，得到效果较好的神经网络结构。

对神经网络结构搜索的研究也主要集中在搜索空间、搜索策略和模型评价策略这三个领域[2]。

神经网络结构搜索的搜索空间指的是在搜索过程中原则上可能搜索得到的所有神经网络结构，具体来讲就是对搜索中的神经网络结构进行了一些约束，当然神经网络结构搜索的目的之一是发现超出人类设计思想的网络结构，所以对于结构的限制过多会导致搜索的结果变差，同时不加限制的结构搜索可能导致搜索空间过大而耗费过多的时间和计算资源。较为简单的神经网络搜索的搜索空间使用的是链式模型，即网络的每一层级依次链接，另一种较为复杂的搜索空间使用的是分支模型，即允许网络中存在跨层的跳跃链接，其效果通常优于链式模型但其搜索空间远大于链式模型。

搜索策略会根据模型评估反馈结果来不断遍历搜索空间。在遍历过程中面临如何对探索和利用进行平衡（exploration-exploitation trade-off）的问题，也就是说既要根据现有已经被探索的模型反馈来寻找较好的神经网络结构，又要积极遍历更多尚未被探索的模型结构以避免陷入局部最优解。为此，在探索中常采用随机搜索、进化算法、强化学习等方法。

在神经网络结构搜索过程中，对模型的评价可用来指导搜索策略如何选择下一个模型结构。神经网络结构搜索的目标是找到效果较好的神经网络模型，所以对模型的评价也就是训练后模型在测试集上的表现。但是反复对搜索所得模型进行测试会开销繁多，所以需要使用一定的技巧来缩短评价计算时间。常用技巧有减少训练次数、使用较小的训练集、使用低分辨率训练数据等。

9.3 神经网络模型压缩

深度神经网络所体现的性能需要“数据、模型、算力、场景”之间相互配合，离不开深度神经网络中海量参数下所进行的巨大计算。随着5G和物联网发展，各种移动设备和嵌入式设备等边缘计算结点需要通过深度学习模型来进行感知和认知，因此边缘计算与推理成为一个热点。

边缘计算和推理受到结点存储量和耗电量等限制，需要更加精简的深度模型和加速网络模型推理速度。当前神经网络模型压缩和加速方法有网络剪枝（pruning）、量化（quantization）、低秩矩阵分解（low-rank matrix factorization），知识蒸馏（knowledge distillation）以及设计小巧的网络结构等。

低秩矩阵分解可以将高维的矩阵用多个低维度的矩阵逼近求和，从而减少计算量。神经网络的剪枝工作在20世纪就已经被关注，如Yann LeCun在1990年提出了剪枝方法OBD（optimal brain damage）[7]。神经网络剪枝方法认为在复杂的网络连接之中，存在一些不重要连接，可以剪去这些不重要连接，使网络变得稀疏而减少计算量，并且也会保证网络性能不下降。

神经网络量化也是一个值得研究的方向。一种方案是通过使用低精度权值来替代原来浮点数权值以加速运算，或者通过只使用1个比特来表示权值。另外一种方案是对权值进行聚类，将接近的权值用其聚类中心表示，这样也减少了参数存储空间。参考文献[3]结合了剪枝，聚类量化技术，并且对得到的权值进行霍夫曼编码，在不影响模型精度的情况下取得了惊人的压缩率（AlexNet压缩到$\frac{1}{35}$，VGG压缩到$\frac{1}{49}$）。

为了在手机和其他嵌入式设备上使用深度学习技术，以MobileNets[5]

为代表的一些工作设计了一系列参数规模小巧的神经网络结构。这些研究主要利用了深度可分离卷积技术，利用1×1卷积获取不同通道之间信息，大幅度减少参数总量，同时MobileNets使用了全局超参数来控制模型结构大小，使模型的设计者能更好地针对不同的存储设备设计网络结构。

把一个大网络结构压缩成小网络结构的方法还有知识蒸馏，其通过小网络结构去重新学习大网络结构知识。参考文献[4]较早提出由一个学生网络去学习教师网络已有知识。比起单纯数据驱动下的学习方法，教师网络输出结果的信息熵含量更高，隐藏了一些有益的抽象知识。知识蒸馏的研究重点有如下两个方面：① 定义知识，可以把教师网络的最终输出结果作为知识，也可以将中间层结果作为知识，还可以把不同样本之间的关系作为知识；② 知识迁移，除了计算学生网络输出结果和教师网络输出结果之间的交叉熵作为损失函数，还可以利用对抗生成模型中的判别器来指导学生网络的学习等。

9.4 人工智能芯片

所谓人工智能芯片（neural processing unit）是指“实现智能算法的芯片”（而不是“具有智能的芯片”）。在实现某个智能算法时，一般用低延迟（low latency）和高吞吐量（high throughput）来评价人工智能芯片的性能。

人工智能芯片之所以在完成智能算法中具备低延迟和高吞吐量特点，是因为同一个算法用接近硬件的编程语言来实现。从技术架构来看，人工智能芯片主要有如下4类：① 通用性芯片，如GPU（graphics processing unit）；② 以可编程门阵列FPGA（field programmable gate array）为代表的半定制化芯片；③ 专用集成电路ASIC（application specific integrated circuit）全定制化芯片，如谷歌的TPU（tensor processing unit）；④ 类脑芯片（即神经形态芯片，neuromorphic processing unit）。

人工智能芯片发展受制于冯·诺依曼体系结构瓶颈（也被称为存储墙）和CMOS（complementary metal oxide semiconductor，互补金属氧化物半导体）工艺及器件现有不足。

在传统的“冯·诺依曼架构”计算机中，计算模块和存储单元彼此分离，处理单元从存储器提取数据，加工处理完毕后再写回存储器，处理单元耗费了大量时间在数据提取和回送，导致了不必要的时延，这被称为存储墙（storage wall）。为了克服这个困难，当前人工智能芯片重点关注如何有效减少访问存储器次数，从而降低访问存储器的时延代价。

基于CMOS工艺的计算机芯片是构建当前计算系统的基础，摩尔定律所昭示的计算机系统性能提升速度来自于CMOS集成尺寸不断缩小。当前，由于器件尺寸缩小接近物理极限，摩尔定律逐渐失效，使得持续提高集成密度变得越来越困难。

为了克服上述困难，目前解决方案是提供大量存储空间的片上存储器技术，并探索利用片上存储器去构建未来的智能芯片架构，存算一体（计算存储一体）形式的非冯·诺依曼架构将成为高性能智能计算的突破。

神经形态芯片是一种基于类脑计算的非布尔逻辑的新型计算芯片。与关注于智能算法加速不同，神经形态芯片更加强调器件、架构和算法等原理上的仿生。现有神经形态计算芯片的技术路线主要包括基于CMOS的神经形态芯片（如TrueNorth，BrainScaleS，NeuroGrid和Loihi等）和基于新型器件的神经形态芯片（如忆阻器、相变单元、自旋器件和flash等），但该领域研究仍面临多方面的挑战。

9.5 量子机器学习

在目前计算机中，比特（bit）是所有信息的最小表达单位，每一个比特值只存在0或1两种可能取值。由于0与1可以理解为开（On）或关（OFF）两种状态，这一表示可以来完成各种逻辑运算，因此计算机就是通过一连串的0与1的组合来执行运算、存储等任务。

与此不同，量子计算机基本信息单位是量子比特（qubit），与传统比特最大的差异是，量子比特可同时具有0、1及其线性叠加态（superposition）。研究者认为，如果能研发具有更多量子比特的量子计算机，就可利用量子纠缠、相干叠加和非局域性（两个粒子在距离很大情况下能瞬时相互影响）等量子力学相关特性，加速经典机器学习的执行速度，这一“量子 + 人工智能”交叉研究为人工智能带来了新的发展

希望。如科学家认为，当量子比特超过49或50个左右，该量子计算机的计算能力将大幅超越传统计算机、甚至是超级计算机，实现量子霸权（quantum supremacy）。

2017年，《自然》杂志以*Quantum Machine Learning*（量子机器学习）[6]为题发表长篇综述，总结了目前该交叉领域的研究进展和未来展望。目前学术界对量子机器学习领域包括两方面研究：一方面是借助量子力学的相干叠加或者纠缠等特性，实现比经典算法性能更佳的量子机器学习算法，如量子主成分分析、量子支持向量机（含核方法）、深度量子学习（deep quantum learning，如用量子训练方法代替对比散度（contrastive divergence）这一经典算法来训练Boltzmann机）等加速算法以及能有效解决线性方程组求解的HHL（Harrow, Hassidim and Lloyd algorithm8）算法等；另一方面，将难以解决的量子物理问题用经典人工智能算法来解决，形成有效的信息提取和分类等模型，也是量子机器学习研究范畴。

现有“量子 + 人工智能”研究成果还比较零散，量子算法完成任务时间仅在特定领域超越了传统算法，如Shor算法（其解决大质数分解的问题，即把一个大数分解成两个质数相乘）和Grover算法（其解决从任意多元素中找到目标元素）等。虽然量子算法的上限和潜力远高于经典算法，但是量子算法的适用范围仍不清晰，为了在实际场景中充分发挥量子算法的加速优势，未来需要研究和设计可以提高机器学习效率的量子算法，以及量子机器学习的算法理论，包括如何对量子可学习的概念类进行数学刻画、量子神经网络的计算能力和局限等。

9.6 人工智能伦理与治理

当前人工智能所呈现的人机协同和自主智能等特点，使得算法、机器和系统成为人类社会不可或缺的一个组成部分，隐私泄露、大数据杀熟、机器杀手、机器换人等现象出现，给社会治理、法律规范等带来了严峻挑战。

人工智能发展不仅面临技术上瓶颈，也出现了人工智能与人类之间关系如何和谐共存问题。对人工智能参与下所创造的社会进行治理，带

来的伦理学讨论不再是人与人之间的关系，也不是与自然界的既定事实（如动物、环境和生态）之间的关系，而是人类与自己所发明的一种产品构成的关联，于是人工智能伦理则关注人-机、机-机以及人-机共融所形成社会形态应该需要遵守的道德准则。

《新一代人工智能发展规划》明确提出“把握人工智能技术属性和社会属性高度融合的趋势”。在人工智能推进过程中，既要加强人工智能研发和应用力度，赋能实体经济；又要预判人工智能与实体经济拥抱可能对社会各个方面带来的一些新挑战和冲击。2019年5月，经济合作与发展组织各成员国签署了人工智能原则，即“负责任地管理可信AI的原则（principles for responsible stewardship of trustworthy AI）”，成为人工智能治理方面的首个政府间国际共识，确立了以人为本的发展理念和敏捷灵活的治理方式。2019年6月，国家新一代人工智能治理专业委员会发布《新一代人工智能治理原则——发展负责任的人工智能》，提出了人工智能治理的框架和行动指南，给出了和谐友好、公平公正、包容共享、尊重隐私、安全可控、共担责任、开放协作、敏捷治理等人工智能发展原则。

人工智能是赋能经济、造福社会的牵引技术，人工智能技术需要与人类合作，增强和提高人类的生活和生产力，让每个人的生活更美好。人工智能社会的治理离不开人工智能技术的保障，人工智能技术的广泛应用又会推动伦理和治理的演化。在这个过程中，需要牢牢把握人工智能技术属性和社会属性相互耦合的特点，以技术发展推动可信、可靠和安全的人工智能融入社会和大众。

9.7 人工智能算法开发框架

人工智能不单纯是一个模型、一手技术、一项产品或一个应用，而是理论博大深厚、技术生机勃勃、产品落地牵引、应用赋能社会的综合协同体，它将从各个角度与层次，宏观、中观和微观地改变人们的学习、工作与生活。为了推动人工智能赋能场景，需要搭建人工智能算法开发框架或平台。

作为当前人工智能重要的技术手段，一些公司或团体推出了若干深

度学习算法开发框架来推动其所设计的生态，如TensorFlow、MXNet、Caffe2、飞浆（PaddlePaddle）、PyTorch等。这些深度学习开发框架向下会与芯片和计算机系统对接，向上会承各种业务模型和行业应用，成为智能时代的“操作系统”。

从感知智能向认知智能迈进以赋予机器推理之道、由机器单独完成单一任务转成机器相互协作来完成城市级复杂任务使命、对隐藏在数据这一人工智能燃料引擎中的隐私予以重视以推动数据共享、有机协调存算能力来破解冯诺伊曼架构中“内存墙”桎梏，都需要人工智能开发框架或平台来协调模型算法或软硬件来完成。

“上士闻道，勤而行之；中士闻道，若存若亡；下士闻道，大笑之”，人工智能算法框架或平台研制是建设人工智能生态所需的重中之重。

9.8 小结

《荀子·正名》用如下话语描述了人类的“智能”：知之在人者谓之知，知有所合谓之智、所以能之在人者为之能，能有所合谓之能。这句话从“知”（知觉，即人所固有感知外界客观世界的能力，如视觉、听觉和触觉等能力）、“智”（智慧：基于感知结果形成知识和概念等）、“本能”（基于所得到知识，本能的做出判断和行动）、“智能”（感知、认知、决策和行动等能力的统一）描述了人类智能从感知，到认知，到决策与行动的全过程。模拟人类智能是一个漫长的历史进程，在不断夯实与壮大人类文明大厦的绚丽征途中，对人类智能这一最为复杂的系统，“我们必须知道，我们必将知道”！

参考文献

[1] THE BRAIN WORKING GROUP OF THE ACD. Brain 2025: A Scientific Vision, 2014.

[2] ELSKEN T, METZEN J H, HUTTER F. Neural architecture search: A survey [J]. arXiv preprint arXiv:1808.05377, 2018.

[3] HANS, MAO H, DALLY W J. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding [J]. arXiv preprint arXiv:1510.00149, 2015.

[4] HINTON G, ORIOL V, JEFF D. Distilling the knowledge in a neural network [J] .arXiv preprint arXiv:1503.02531, 2015.

[5] HOWARD A G, et al.Mobilenets: Efficient convolutional neural networks for mobile vision applications [J]. arXiv preprint arXiv:1704.04861, 2017.

[6] JACOB B, WITTEK P, PANCOTTI N, et al. Quantum Machine Learning [J]. Nature, 549: 195–202, 2017.

[7] LECUN Y, DENKER J S, SOLLA S A. Optimal brain damage [J]. Advances in neural information processing systems, 1990.

[8] PAN Y. Heading toward artificial intelligence 2.0 [J]. Engineering, 2(4): 409–413, 2016.

[9] PAN Y. special issue on artificial intelligence 2.0: theories and applications [J]. Frontiers of Information Technology & Electronic Engineering, 19(1): 1–2, 2018.

[10] 吴飞，阳春华，兰旭光，等. 人工智能的回顾与展望 [J]. 中国科学基金，32（3）: 262–265, 2018.

[11] ZOPH B, LE Q V. Neural architecture search with reinforcement learning [J]. arXiv preprint arXiv: 1611.01578, 2016.

附录 设计实验

实验一　黑白棋AI算法

1. 实验内容

黑白棋（reversi），也叫苹果棋，翻转棋，是一个经典的策略性游戏。一般棋子双面为黑白两色，故称“黑白棋”。因为行棋之时将对方棋子翻转，变为己方棋子，故又称“翻转棋”。棋子双面为红、绿色的称为“苹果棋”。它使用8×8的棋盘，由两人执黑子和白子轮流下棋，最后子多方为胜方。随着网络的普及，黑白棋作为一种最适合在网上玩的棋类游戏正在逐渐流行起来。中国主要的黑白棋游戏站点有Yahoo游戏、中国游戏网、联众游戏等。

游戏规则：

棋局开始时黑棋位于e4和d5，白棋位于d4和e5，如图1所示。

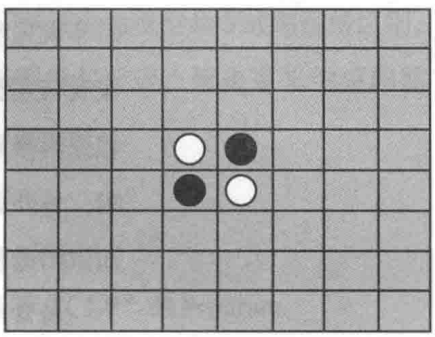


图1 黑白棋初始图

（1）黑方先行，双方交替下棋。

（2）一步合法的棋步包括：在一个空格新落下一个棋子，并且翻转对手一个或多个棋子。

（3）新落下的棋子与棋盘上已有的同色棋子间，对方被夹住的所有棋子都要翻转过来。可以是横着夹，竖着夹，或是斜着夹。夹住的位置上必须全部是对手的棋子，不能有空格。

（4）一步棋可以在数个（横向、纵向、对角线）方向上翻棋，任何被夹住的棋子都必须被翻转过来，棋手无权选择不去翻某个棋子。

（5）除非至少翻转了对手的一个棋子，否则就不能落子。如果一方没有合法棋步，也就是说不管他下到哪里，都不能至少翻转对手的一个棋子，那他这一轮只能弃权，而由他的对手继续落子直到他有合法棋步可下。

（6）如果一方至少有一步合法棋步可下，他就必须落子，不得弃权。

（7）棋局持续下去，直到棋盘填满或者双方都无合法棋步可下。

（8）如果某一方落子时间超过1分钟，则判该方失败。

2. 实验要求

（a）使用MCTS算法实现miniAlphaGo for Reversi；

（b）MCTS算法部分需要自己实现，尽量不使用现成的包、工具或者接口；

（c）在博弈过程中，miniAlphaGo每一步所花费的时间以及总时间需要显示出来；

（d）需要有简单的图形界面用于人机博弈交互；

（e）使用语言不限。

实验二　逻辑推理实验

1. 实验内容

逻辑编程是一种编程典范，它设置答案须匹配的规则来解决问题，而非设置步骤来解决问题。过程是“事实 + 规则 = 结果”。人工智能的发展与逻辑编程的发展是一个相辅相成的过程，早期的人工智能以规则和逻辑推理作为主要研究方向，这在逻辑编程的发展中发挥了重要的影响，另外更好更快地逻辑编程也推动了人工智能的发展，例如专家系统、知识图谱和自动定理证明。

最常用的逻辑编程语言是Prolog，这种语言最初被运用于自然语言等研究领域。现在它已广泛应用在人工智能的研究中，它可以用来建造专家系统、自然语言理解、智能知识库等。本实验将学习Prolog编程语言，并尝试使用Prolog语言解决斑马问题以及八皇后问题。

斑马问题：5个不同国家且工作各不相同的人分别住在一条街上的5所房子里，每所房子的颜色不同，每个人都有自己养的不同宠物，喜欢喝不同的饮料。根据以下提示，你能告诉我哪所房子里的人养斑马，哪所房子里的人喜欢喝矿泉水吗？

a）英国人住在红色的房子里。

b）西班牙人养了一条狗。

c）日本人是一个油漆工。

d）意大利人喜欢喝茶。

e）挪威人住在左边的第一个房子里。

f）绿房子在白房子的右边。

g）摄影师养了一只蜗牛。

h）外交官住在黄房子里。

i）中间那个房子的人喜欢喝牛奶。

j）喜欢喝咖啡的人住在绿房子里。

k）挪威人住在蓝色的房子旁边。

l）小提琴家喜欢喝橘子汁。

m）养狐狸的人所住的房子与医生的房子相邻。

n）养马的人所住的房子与外交官的房子相邻。

八皇后问题：如何能够在8×8的国际象棋棋盘上放置八个皇后，使得任何一个皇后都无法直接吃掉其他的皇后？为了达到此目的，任两个皇后都不能处于同一条横行、纵行或斜线上。

2. 实验要求

（1）基本掌握逻辑编程的思想，了解逻辑编程与命令式编程的区别；

（2）能够依据给定的事实以及规则编写代码，解决逻辑约束问题（CLP）。

3. 实验环境

通过SWI-Prolog在线平台创建基于“CLP”的Program。

实验三　线性回归模型

1. 实验内容

图像是一种非常常见的信息载体，但是在图像的获取、传输、存储过程中可能由于各种原因使得图像受到噪声的影响。如何去除噪声的影响，恢复图像原本的信息是计算机视觉中的重要研究问题。

常见的图像恢复算法有基于空间域的中值滤波、基于小波域的小波去噪、基于偏微分方程的非线性扩散滤波等。本次实验要对图像添加高斯噪声，并对添加噪声的图像进行基于线性回归模型的去噪。

2. 实验要求

（1）生成受损图像。

a）受损图像（X）是由原始图像$\left(\mathrm{I}\in\mathcal{R}^{H*W*C}\right)$添加了不同噪声遮罩（noise masks）$\left(\mathrm{M}\in\mathcal{R}^{H*W*C}\right)$得到的（$\mathrm{X}=\mathrm{I}\odot\mathrm{M}$），其中⊙是逐元素相乘。

b）噪声遮罩仅包含{0, 1}值。对原图的噪声遮罩可以每行分别用0.8/0.4/0.6的噪声比率产生，即噪声遮罩每个通道每行80%/40%/60%的像素值为0，其他为1。

（2）使用区域二元线性回归模型，进行图像恢复。

（3）评估误差为所有恢复图像（R）与原始图像（I）的2-范数之和，此误差越小越好。$\text{error}=\sum_{i=1}^{3}\text{norm}\left(\mathrm{R}_i(:)-\mathrm{I}_i(:),2\right)$，其中(:)是向量化操作。

3. 实验环境

可以使用基于Python的OpenCV库进行图像相关处理，使用Numpy库进行相关数值运算。

实验四　人脸识别

1. 实验内容

特征脸（eigenface）是第一种有效的人脸识别方法，通过在一大组描述不同人脸的图像上进行主成分分析（PCA）获得。

本次实验要求构建一个自己的人脸库，可以通过现有的人脸库添加自己的人脸图像形成，人脸库中的每一张图像都只包含一张人脸且眼睛的中心位置已知。

在实验前首先根据眼睛的位置将图片对齐处理，在训练过程中，首先要求出平均脸，然后将前K个特征脸保存下来，利用这K个特征脸对测试人脸进行识别，此外对于任意给定的一张人脸图像，可以使用这K个特征脸对原图进行重建。

2. 实验要求

（1）不使用OpenCV的Eigenface接口，通过求解人脸图像的特征值与特征向量构建特征脸模型。

（2）利用特征脸模型进行人脸识别和重建，比较使用不同数量特征脸的识别与重建效果。

3. 实验环境

可以使用基于Python的OpenCV库进行图像相关处理，使用Numpy库进行相关数值运算。

实验五　深度学习

1. 实验内容

自己选择一个机器学习相关的问题，并利用深度学习尝试解决这个问题。问题的选择可以是计算机视觉（人脸检测、人脸识别、图像分类等）、自然语言处理（语音识别、机器翻译、中文分词等）、推荐系统（兴趣地点推荐、用户评分预测等）、数据预测分析（红酒品质分类、金融数据预测等）。

目前多数深度学习模型都是依赖数据驱动的，所以在选择研究问题时，首先要考虑的是数据的获取，目前网络上有很多公开数据集可以供大家使用，例如UCI数据集，提供了大量机器学习的数据集。当然也可以自己收集数据，形成自己的数据集。

模型的训练过程往往需要花费大量计算资源，推荐大家使用华为云ModelArts或者Google Cloud Platform等云端训练平台进行训练。

2. 实验要求

（1）基本掌握一种深度学习开发库的使用。

（2）能够使用深度学习库搭建自己的深度学习模型。

（3）通过训练使模型在特定问题上得到较好的效果。

3. 实验环境

使用Python的TensorFlow、PyTorch等开源库。

实验六　强化学习

1. 实验内容

使用强化学习Q-Learning算法设计一个可以在4×4大小的迷宫中找到宝藏的agent。迷宫可以使用一个系列字符表示，字符“o”表示agent的位置，字符“T”表示宝藏的位置，该位置的收益为1，且为游戏的终结状态，字符“P”表示陷阱的位置，该位置的收益为-1，且为游戏的终结状态，字符“_”表示收益为0的中间状态。

初始情况下，迷宫如下表示：

```
o _ _ _ _
_ P _ _ P
_ _ P _ _
_ _ _ _ T
```

每次行动agent可以选择向不超过迷宫边界的相邻位置移动一格，通过Q-Learning算法希望agent可以最终找到一条通向宝藏的路径。

通过改变决策ϵ的大小，考察agent选择路径方式的变化。

2. 实验要求

（1）实现一个用字符表示的迷宫环境，能够对agent的行动以及收益返回相应的结果，并可视化地表示出来。

（2）实现Q-Learning算法，包括Q-Table的更新以及根据Q-Table进行决策的函数。

（3）利用Q-Learning算法训练一个能够在迷宫中找到宝藏的agent。

3. 实验环境

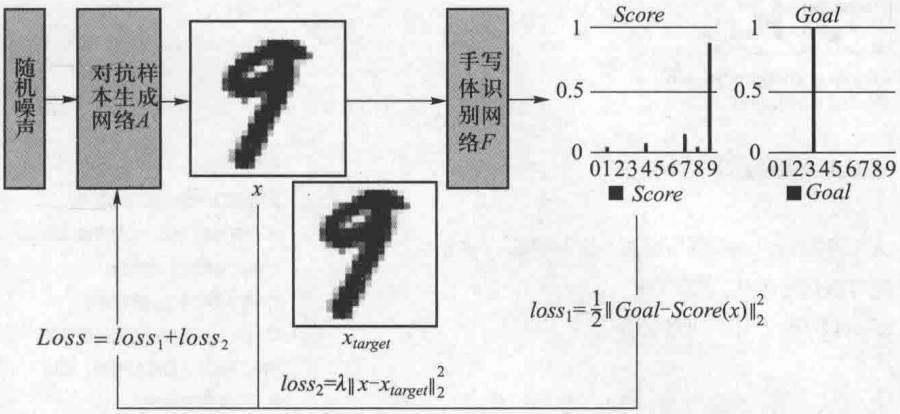
不限制编程语言。

实验七　对抗样本

1. 实验内容

首先训练一个能够识别手写体数字的分类网络F，然后尝试针对这

个分类网络训练一个对抗样本生成网络A，这个对抗生成网络可以生成类似手写体数字9的对抗样本，这个对抗样本会被手写体识别网络识别为数字3。



在生成的过程中，可以考虑直接对手写体数字9的图像增加较小的噪声，也可以直接从随机噪声中生成类似于9的图像。

2. 实验要求

（1）训练一个手写体数字识别网络，能够对已有的数据集实现手写体分类。

（2）固定手写体识别网络的参数，训练一个对抗样本生成网络，生成人眼判定为9而被识别网络判定为3的对抗样本。

（3）使用对抗样本生成网络生成若干对抗样本，测试生成对抗样本对手写体识别网络的攻击成功率。

3. 实验环境

可以使用TensorFlow或PyTorch等深度学习框架实现各个模型。

新一代人工智能系列教材

□ 权威一流的编委会

编委会由高等教育出版社联合国家新一代人工智能战略咨询委员会成立，潘云鹤院士担任编委会主任，郑南宁院士、高文院士、吴澄院士、陈纯院士和林金安副总编辑担任编委会副主任。编委会成员大多来自教育部人工智能科技创新专家组工作组和教育部高等学校计算机类专业教学指导委员会。

□ 来自科研教学一线、朝气勃勃的作者团队

所有作者长期从事人工智能方向及人工智能交叉学科的前沿基础科学研究，强调教学和科研相互促进、实践培养与知识创新彼此结合、技术属性与社会属性相得益彰的人工智能教学先进理念。

□ 紧扣前沿、富有时代特色的系列教材内容

紧扣新一代人工智能发展特点，系列教材包含人工智能基础理论、算法模型、技术系统、硬件芯片和伦理安全以及“智能 +”学科等内容，建设在线开放课程，促进优秀教育资源高质量共享。

ISBN 978-7-04-053466-5

9 787040 534665 >

定价 37.00元